

Linear Algebra: Theory, Intuition, Code

Mike X Cohen

sincXpress

Linear Algebra: Intuition, Math, Code

Mike X Cohen

2 February 2021

Contents

1 Introduction
- 1.1 What is linear algebra and why learn it?
- 1.2 About this book
- 1.3 Prerequisites
- 1.4 Exercises and code challenges
- 1.5 Online and other resources

2 Vectors
- 2.1 Scalars
- 2.2 Vectors: geometry and algebra
- 2.3 Transpose operation
- 2.4 Vector addition and subtraction
- 2.5 Vector-scalar multiplication
- 2.6 Exercises
- 2.7 Answers
- 2.8 Code challenges
- 2.9 Code solutions

3 Vector multiplication
- 3.1 Vector dot product: Algebra
- 3.2 Dot product properties
- 3.3 Vector dot product: Geometry
- 3.4 Algebra and geometry
- 3.5 Linear weighted combination
- 3.6 The outer product
- 3.7 Hadamard multiplication
- 3.8 Cross product
- 3.9 Unit vectors
- 3.10 Exercises
- 3.11 Answers
- 3.12 Code challenges
- 3.13 Code solutions

4 Vector spaces
- 4.1 Dimensions and fields
- 4.2 Vector spaces
- 4.3 Subspaces and ambient spaces

4.4 Subsets
4.5 Span
4.6 Linear independence
4.7 Basis
4.8 Exercises
4.9 Answers
5 Matrices
5.1 Interpretations and uses of matrices
5.2 Matrix terms and notation
5.3 Matrix dimensionalities
5.4 The transpose operation
5.5 Matrix zoology
5.6 Matrix addition and subtraction
5.7 Scalar-matrix mult.
5.8 "Shifting" a matrix
5.9 Diagonal and trace
5.10 Exercises
5.11 Answers
5.12 Code challenges
5.13 Code solutions
6 Matrix multiplication
6.1 "Standard" multiplication
6.2 Multiplication and eqns.
6.3 Multiplication with diagonals
6.4 LIVE EVIL
6.5 Matrix-vector multiplication
6.6 Creating symmetric matrices
6.7 Multiply symmetric matrices
6.8 Hadamard multiplication
6.9 Frobenius dot product
6.10 Matrix norms
6.11 What about matrix division?
6.12 Exercises
6.13 Answers
6.14 Code challenges
6.15 Code solutions
7 Rank

7.1 Six things about matrix rank
7.2 Interpretations of matrix rank
7.3 Computing matrix rank
7.4 Rank and scalar multiplication
7.5 Rank of added matrices
7.6 Rank of multiplied matrices
7.7 Rank of A, AT, ATA, and AAT
7.8 Rank of random matrices
7.9 Boosting rank by "shifting"
7.10 Rank difficulties
7.11 Rank and span
7.12 Exercises
7.13 Answers
7.14 Code challenges
7.15 Code solutions
8 Matrix spaces
8.1 Column space of a matrix
8.2 Column space: A and AAT
8.3 Determining whether v â^^ C(A)
8.4 Row space of a matrix
8.5 Row spaces of A and ATA
8.6 Null space of a matrix
8.7 Geometry of the null space
8.8 Orthogonal subspaces
8.9 Matrix space orthogonalities
8.10 Dimensionalities of matrix spaces
8.11 More on Ax = b and Ay = 0
8.12 Exercises
8.13 Answers
8.14 Code challenges
8.15 Code solutions
9 Complex numbers
9.1 Complex numbers and â„‚
9.2 What are complex numbers?
9.3 The complex conjugate
9.4 Complex arithmetic
9.5 Complex dot product

9.6 Special complex matrices
9.7 Exercises
9.8 Answers
9.9 Code challenges
9.10 Code solutions
10 Systems of equations
10.1 Algebra and geometry of eqns.
10.2 From systems to matrices
10.3 Row reduction
10.4 Gaussian elimination
10.5 Row-reduced echelon form
10.6 Gauss-Jordan elimination
10.7 Possibilities for solutions
10.8 Matrix spaces, row reduction
10.9 Exercises
10.10 Answers
10.11 Coding challenges
10.12 Code solutions
11 Determinant
11.1 Features of determinants
11.2 Determinant of a 2Ã—2 matrix
11.3 The characteristic polynomial
11.4 3Ã—3 matrix determinant
11.5 The full procedure
11.6 Î” of triangles
11.7 Determinant and row reduction
11.8 Î” and scalar multiplication
11.9 Theory vs practice
11.10 Exercises
11.11 Answers
11.12 Code challenges
11.13 Code solutions
12 Matrix inverse
12.1 Concepts and applications
12.2 Inverse of a diagonal matrix
12.3 Inverse of a 2Ã—2 matrix
12.4 The MCA algorithm

12.5 Inverse via row reduction
12.6 Left inverse
12.7 Right inverse
12.8 The pseudoinverse, part 1
12.9 Exercises
12.10 Answers
12.11 Code challenges
12.12 Code solutions
13 Projections
13.1 Projections in â„ 2
13.2 Projections in â„ N
13.3 Orth and par vect comps
13.4 Orthogonal matrices
13.5 Orthogonalization via GS
13.6 QR decomposition
13.7 Inverse via QR
13.8 Exercises
13.9 Answers
13.10 Code challenges
13.11 Code solutions
14 Least-squares
14.1 Introduction
14.2 5 steps of model-fitting
14.3 Terminology
14.4 Least-squares via left inverse
14.5 Least-squares via projection
14.6 Least-squares via row-reduction
14.7 Predictions and residuals
14.8 Least-squares example
14.9 Code challenges
14.10 Code solutions
15 Eigendecomposition
15.1 Eigenwhatnow?
15.2 Finding eigenvalues
15.3 Finding eigenvectors
15.4 Diagonalization

15.5 Conditions for diagonalization
15.6 Distinct, repeated eigenvalues
15.7 Complex solutions
15.8 Symmetric matrices
15.9 Eigenvalues singular matrices
15.10 Eigenlayers of a matrix
15.11 Matrix powers and inverse
15.12 Generalized eigendecomposition
15.13 Exercises
15.14 Answers
15.15 Code challenges
15.16 Code solutions
16 The SVD
16.1 Singular value decomposition
16.2 Computing the SVD
16.3 Singular values and eigenvalues
16.4 SVD of a symmetric matrix
16.5 SVD and the four subspaces
16.6 SVD and matrix rank
16.7 SVD spectral theory
16.8 Low-rank approximations
16.9 Normalizing singular values
16.10 Condition number of a matrix
16.11 SVD and the matrix inverse
16.12 MP Pseudoinverse, part 2
16.13 Code challenges
16.14 Code solutions
17 Quadratic form
17.1 Algebraic perspective
17.2 Geometric perspective
17.3 The normalized quadratic form
17.4 Evecs and the qf surface
17.5 Matrix definiteness
17.6 The definiteness of ATA
17.7 Î» and definiteness
17.8 Code challenges
17.9 Code solutions

18 Covariance matrices
- 18.1 Correlation
- 18.2 Variance and standard deviation
- 18.3 Covariance
- 18.4 Correlation coefficient
- 18.5 Covariance matrices
- 18.6 Correlation to covariance
- 18.7 Code challenges
- 18.8 Code solutions

19 PCA
- 19.1 PCA: interps and apps
- 19.2 How to perform a PCA
- 19.3 The algebra of PCA
- 19.4 Regularization
- 19.5 Is PCA always the best?
- 19.6 Code challenges
- 19.7 Code solutions

20 The end.
- 20.1 The end... of the beginning!
- 20.2 Thanks!

ChapterÂ 1
Introduction to this book

1.1 What is linear algebra and why learn it?

Linear algebra is the branch of mathematics concerned with vectors and matrices, their linear combinations, and operations acting upon them. [1] Linear algebra has a long history in pure mathematics, in part because it provides a compact notation that is powerful and general enough to be used in geometry, calculus, differential equations, physics, economics, and many other areas.

[1] **FACT** :
Linear algebra is
SUUUUPER
IMPORTANT!!

But the importance and application of linear algebra is quickly increasing in modern applications. Many areas of science, technology, finance, and medicine are moving towards large-scale data collection and analysis. Data are often stored in matrices, and operations on those dataâ€”ranging from statistics to filtering to machine learning to computer graphics to compressionâ€”are typically implemented via linear algebra operations. Indeed, linear algebra has arguably exceeded statistics and time series analysis as the most important branch of mathematics in which to gain proficiency for data-focused areas of science and industry.

Human civilization is moving towards increasing digitization, quantitative methods, and data. Therefore, knowledge of foundational topics such as linear algebra are increasingly important. One may (indeed: *should*) question the appropriateness and utility of the trend towards "big data" and the over-reliance on algorithms to make decisions for us, but it is inarguable that familiarity with matrix analysis, statistics, and multivariate methods have become crucial skills for any data-related job in academia and in industry.

1.2 About this book

The purpose of this book is to teach you how to think about and work with matrices, with an eye towards applications in machine learning, multivariate statistics, time series, and image processing. If you are interested in data science, quantitative biology, statistics, or machine learning and artificial intelligence, then this book is for you. If you donâ€™t have a strong background in mathematics, then donâ€™t be concerned: You need only high-school math and a bit of dedication to learn linear algebra from this book.

This book is written with the self-studying reader in mind. Many people do not realize how important linear algebra is until after university, or they do not meet the requirements of university-level linear algebra courses (typically, calculus). Linear algebra textbooks are often used as a compendium to a lecture-based course embedded in a traditional university math program, and therefore can be a challenge to use as an independent resource. I hope that this book is a self-contained resource that works well inside or outside of a formal course.

Many extant textbooks are theory-oriented, with a strong focus on abstract concepts as opposed to practical implementations. You might have encountered such books: They avoid showing numerical examples in the interest of generalizations; important proofs are left "as an exercise for the reader"; mathematical statements are simply plopped onto the page without discussion of relevance, importance, or application; and there is no mention of whether or how operations can be implemented in computers.

I do not write these as criticismsâ€”abstract linear algebra is a beautiful topic, and infinite-dimensional vector spaces are great. But for those interested in using linear algebra (and mathematics more generally) as a tool for understanding data, statistics, deep learning, etc., then abstract treatments of linear algebra may seem like a frustrating waste of time. My goal here is to present applied linear algebra in an approachable and comprehensible way, with little focus on abstract concepts that lack a clear link to applications.

Ebook version The ebook version is identical to the physical version of this book, in terms of the text, formulas, visualizations, and code. However, the formatting is necesarily quite different. The book was designed to be a

physical book; and thus, margins, fonts, text and figure placements, and code blocks are optimized for pages, not for ereaders.

Therefore, I recommend getting the physical copy of the book if you have the choice. If you get the ebook version, then please accept my apologies for any ugly or annoying formatting issues. If you have difficulties reading the code, please download it from github.com/mikexcohen/LinAlgBook.

Equations This is a math book, so you wonâ€™t be surprised to see equations. But math is more than just equations: In my view, the purpose of math is to understand concepts; equations are one way to present those concepts, but words, pictures, and code are also important. Let me outline the balance:

1. Equations provide rigor and formalism, but they rarely provide intuition.
2. Descriptions, analogies, visualizations, and code provide intuition but often lack sufficient rigor.

This balance guides my writing: Equations are pointless if they lack descriptions and visualizations, but words and pictures without equations can be incomplete or misinterpreted.

So yes, there is a respectable number of equations here. There are three levels of *hierarchy* in the equations throughout this book. Some equations are simple or reminders of previously discussed equations; these are lowest on the totem pole are are presented in-line with text like this: $x\,(\,yz\,)\,=\,(\,xy\,)\,z$.

More important equations are given on their own lines. The number in parentheses to the right will allow me to refer back to that equation later in the text (the number left of the decimal point is the chapter, and the number to the right is the equation number).

$$\sigma = x(yz) = (xy)z \tag{1.1}$$

And the most important equationsâ€”the ones you should really make sure to understand and be comfortable using and reproducingâ€”are presented in their own box with a title:

Eqn. box title=**Something important!**

$$\ddot{I}f = x\,(yz) = (xy)\,z \qquad (1.2)$$

Algebraic and geometric perspectives on matrices Many concepts in linear algebra can be formulated using both geometric and algebraic (analytic) methods. This "dualism" promotes comprehension and I try to utilize it often. The geometric perspective provides visual intuitions, although it is usually limited to 2D or 3D. The algebraic perspective facilitates rigorous proofs and computational methods, and is easily extended to N-D. When working on problems in â„ ^{2} or â„ ^{3}, I recommend sketching the problem on paper or using a computer graphing program.

Just keep in mind that *not every* concept in linear algebra has both a geometric and an algebraic concept. The dualism is useful in many cases, but itâ€™s not a fundamental fact that necessarily applies to all linear algebra concepts.

1.3 Prerequisites

The obvious . Dare I write it? You need to be motivated to learn linear algebra. Linear algebra isnâ€™t so difficult, but itâ€™s also not so easy. An intention to learn applied linear algebraâ€”and a willingness to expend mental energy towards that goalâ€”is the single most important prerequisite. Everything below is minor in comparison.

High-school math . You need to be comfortable with arithmetic and basic algebra. Can you solve for x in $4x^{2} = 9$? Then you have enough algebra knowledge to continue. Other concepts in geometry, trigonometry, and complex numbers ($a\ ib, e^{i\delta\ \alpha f}$) will be introduced as the need arises.

Calculus . Simply put: none. I strongly object to calculus being taught before linear algebra. No offense to calculus, of course; itâ€™s a rich, beautiful, and incredibly important subject. But linear algebra can be learned without any calculus, whereas many topics in calculus involve some linear algebra. Furthermore, many modern applications of linear algebra invoke no calculus concepts. Hence, linear algebra should be taught assuming no calculus background.

Vectors, matrices and <insert fancy-sounding linear algebra term here> . If this book is any good, then you donâ€™t need to know anything about linear algebra before reading it. That said, some familiarity with matrices and matrix operations will be beneficial.

Programming . Before computers, advanced concepts in mathematics could be understood only by brilliant mathematicians with great artistic skills and a talent for being able to visualize equations. Computers changed that. Now, a reasonably good mathematics student with some perseverance and moderate computer skills can implement and visualize equations and other mathematical concepts. The computer deals with the arithmetic and low-level graphics, so you can worry about the concepts and intuition.

This doesnâ€™t mean you should forgo solving problems by hand; it is only through laboriously solving lots and lots of problems on paper that you will internalize a deep and flexible understanding of linear algebra. However, only simple (often, integer) matrices are feasible to work through by hand; computer simulations and plotting will allow you to understand an equation visually, when staring at a bunch of letters and Greek characters might give you nothing but a sense of dread. So, if you really want to learn modern, applied linear algebra, itâ€™s helpful to have some coding proficiency in a language that interacts with a visualization engine.

I provide code for all concepts and problems in this book in both MATLAB and Python. I find MATLAB to be more comfortable for implementing linear algebra concepts. If you donâ€™t have access to MATLAB, you can use Octave, which is a free cross-platform software that emulates nearly all MATLAB functionality. But the popularity of Python is undeniable, and you should use whichever program you (1) feel more comfortable using or (2) anticipate working with in the future. Feel free to use any other coding

language you like, but it is your responsibility to translate the code into your preferred language.

I have tried to keep the code as simple as possible, so you need only minimal coding experience to understand it. On the other hand, this is not an intro-coding book, and I assume some basic coding familiarity. If you understand variables, for-loops, functions, and basic plotting, then you know enough to work with the book code.

To be clear: You do not *need* any coding to work through the book. The code provides additional material that I believe will help solidify the concepts as well as adapt to specific applications. But you can successfully and completely learn from this book without looking at a single line of code.

1.4 Practice, exercises and code challenges

Math is not a spectator sport. If you simply read this book without solving any problems, then sure, youâ€™ll learn something and I hope you enjoy it. But to really understand linear algebra, you need to solve problems.

Some math textbooks have a seemingly uncountable number of exercises. My strategy here is to have a manageable number of exercises, with the expectation that you can solve *all* of them.

There is a hierarchy of problems to solve in this book:

Practice problems are a few problems at the end of chapter subsections. They are designed to be easy, the answers are given immediately below the problems, and are simply a way for you to confirm that you get the basic idea of that section. If you canâ€™t solve the practice problems, go back and re-read that subsection.

Exercises are found at the end of each chapter and focus on drilling and practicing the important concepts. The answers (yes, *all* of them; not just the odd-numbered) follow the exercises, and in many cases you can also check your own answer by solving the problem on a computer (using MATLAB or Python). Keep in mind that these exercises are designed to be

solved by hand, and you will learn more by solving them by hand than by computer.

Code challenges are more involved, require some effort and creativity, and can only be solved on a computer. These are opportunities for you to explore concepts, visualizations, and parameter spaces in ways that are difficult or impossible to do by hand. I provide my solutions to all code challenges, but keep in mind that there are many correct solutions; the point is for you to explore and understand linear algebra using code, not to reproduce my code.

If you are more interested in concepts than in computer implementation, feel free to skip the coding challenges.

1.5 Online and other resources

Although I have tried to write this book to be a self-contained one-stop-shop for all of your linear algebra needs, it is naive to think that everyone will find it to be the perfect resource that I intend it to be. Everyone learns differently and everyone has a different way of understanding and visualizing mathematical concepts.

If you struggle to understand something, donâ€™t jump to the conclusion that you arenâ€™t smart enough; a simpler possibility is that the explanation I find intuitive is not the explanation that you find intuitive. I try to give several explanations of the same concept, in hopes that youâ€™ll find traction with at least one of them.

Therefore, you shouldnâ€™t hesitate to search the Internet or other textbooks if you need different or alternative explanations, or if you want additional exercises to work through.

This book is based on an online course that I created. The book and the course are similar but not entirely redundant. You donâ€™t need to enroll in the online course to follow along with this book (or the other way around). I appreciate that some people prefer to learn from online video lectures while others prefer to learn from textbooks. I am trying to cater to both kinds of learners.

You can find a list of all my online courses at sincxpress.com.

ChapterÂ 2
Vectors

Welcome to the second step of conquering linear algebra! (The first step was investing in this book.) This and the next few chapters lay the foundation upon which all subsequent chapters (and, really, all of linear algebra) are built.

2.1 Scalars

We begin not with vectors but with scalars. You already know everything you need to know about scalars, even if you donâ€™t yet recognize the term.

A "scalar" is just a single number, like 4 or -17.3 or *Ï€* . In other areas of mathematics, these could be called *constants* . Donâ€™t let their simplicity fool youâ€”scalars play multiple important roles in linear algebra, and are central to concepts ranging from subspaces to linear combinations to eigendecomposition.

Why are single numbers called "scalars"? Itâ€™s because single numbers "scale," or stretch, vectors and matrices without changing their direction. This will become clear and intuitive later in this chapter when you learn about the geometric interpretation of vectors.

Scalars can be represented as a point on a number line that has zero in the middle, negative numbers to the left, and positive numbers to the right (Figure 2.1). But donâ€™t confuse a point on this number line with an arrow from zero to that numberâ€”thatâ€™s a vector, not a scalar.



FigureÂ 2.1: Scalars as points on a line. The scalar 1.5 is demarcated.

Notation In this book (as in most linear algebra texts), scalars will be indicated using Greek lowercase letters such as *Î»* , *Î±* , or *Î³* . This helps disambiguate scalars from vectors and matrices.

Code Variables representing scalars in Python and MATLAB are trivially easy to create. In fact, Iâ€™m only showing the code here to introduce you to the

formatting of code blocks.

Code blockÂ 2.1: Python

```
aScalarÂ =Â 4
```

Code blockÂ 2.2: MATLAB

```
aScalarÂ =Â 4;
```

Computers have different numeric variable types, and distinguish between, for example, 4 and 4.0 (those are, respectively, an int and a float). However, variable typing is not relevant now, so you donâ€™t need to worry about it. I mention it here only for the experienced coders.

Practice problems Solve for $\hat{I}»$.

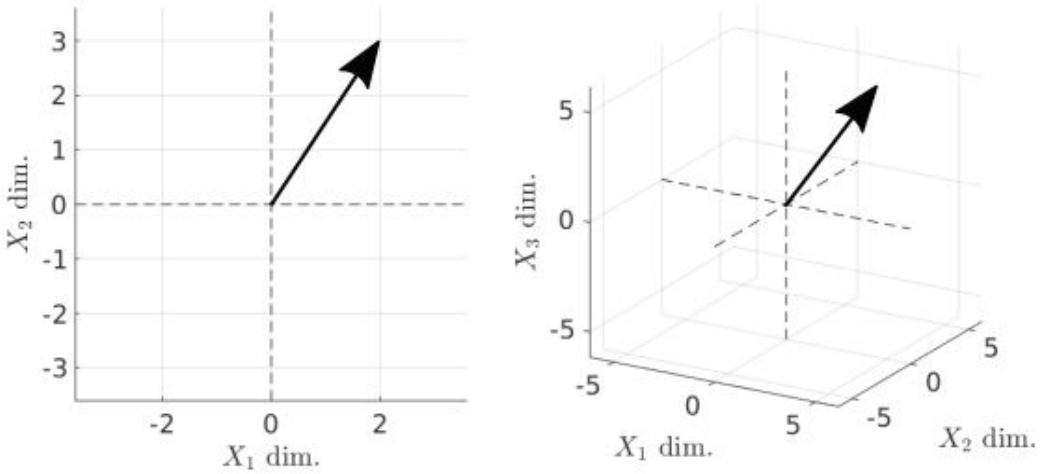
- **a)** $2\ \hat{I}» = 9$
- **b)** $5\ â^{\wedge}•\hat{I}» = 7$
- **c)** $Ï€^{3} = e^{\hat{I}»}$
- **d)** $4(\ \hat{I}» + 3) = â^{\wedge}’\ 6$
- **e)** $.\ 5\ \hat{I}» = .\ 25$
- **f)** $8\ \hat{I}» = 0$
- **g)** $\hat{I}»^{2} = 36$
- **h)** $\hat{I}» = 5\ \hat{I}»^{2}$

Answers

- **a)** $\hat{I}» = 9\ â^{\wedge}•\ 2$
- **b)** $\hat{I}» = 5\ â^{\wedge}•\ 7$
- **c)** $\hat{I}» = 3\ln Ï€$
- **d)** $\hat{I}» = â^{\wedge}’\ 9\ â^{\wedge}•\ 2$
- **e)** $\hat{I}» = .\ 5$
- **f)** $\hat{I}» = 0$
- **g)** $\hat{I}» = Â±\ 6$
- **h)** $\hat{I}» = \frac{1}{5}$ (for $\hat{I}»â‰ \ 0$)

2.2 Vectors: geometry and algebra

Geometry A vector is a line , which is determined by a magnitude (length) and a direction. Lines can exist on a coordinate system with any number of dimensions greater than zero (we deal only with integer-dimensional spaces here; apologies to the fractal-enthusiasts). The dimensionality of the vector is the dimensionality of the coordinate system. Figure 2.2 illustrates vectors in two-dimensions (2D) and in 3D.



FigureÂ 2.2: Vectors as lines with a direction and length. Left plot depicts vector [2 3] and right plot depicts vector [2 3 5]. The arrow is the head of the vector.

It is important to know that the definition of a vector does not include its starting or ending locations. That is, the vector [1 -2] simply means a line that goes one unit in the positive direction in the first dimension, and two units in the negative direction in the second dimension. This is the key difference between a *vector* and a *coordinate* : For any coordinate system (think of the standard Cartesian coordinate system), a given coordinate is a unique point in space. A vector, however, is any lineâ€”anywhereâ€”for which the end point (also called the *head* of the vector) is a certain number of units along each dimension away from the starting point (the *tail* of the vector).

On the other hand, coordinates and vectors are coincident when the vector starts at the origin (the [0,0] location on the graph). A vector with its tail at the origin is said to be in its *standard position* . This is illustrated in Figure 2.3 : The *coordinate* [1 -2] and the *vector* [1 -2] are the same when the vector is in its standard position. But the three thick lines shown in the figure are all the same vector [1 -2].

For a variety of reasons, itâ€™s convenient to show vectors in their standard positions. Therefore, you may assume that vectors are always drawn in standard position unless otherwise stated.

Code Vectors are easy to create and visualize in MATLAB and in Python. The code draws the vector in its standard position.

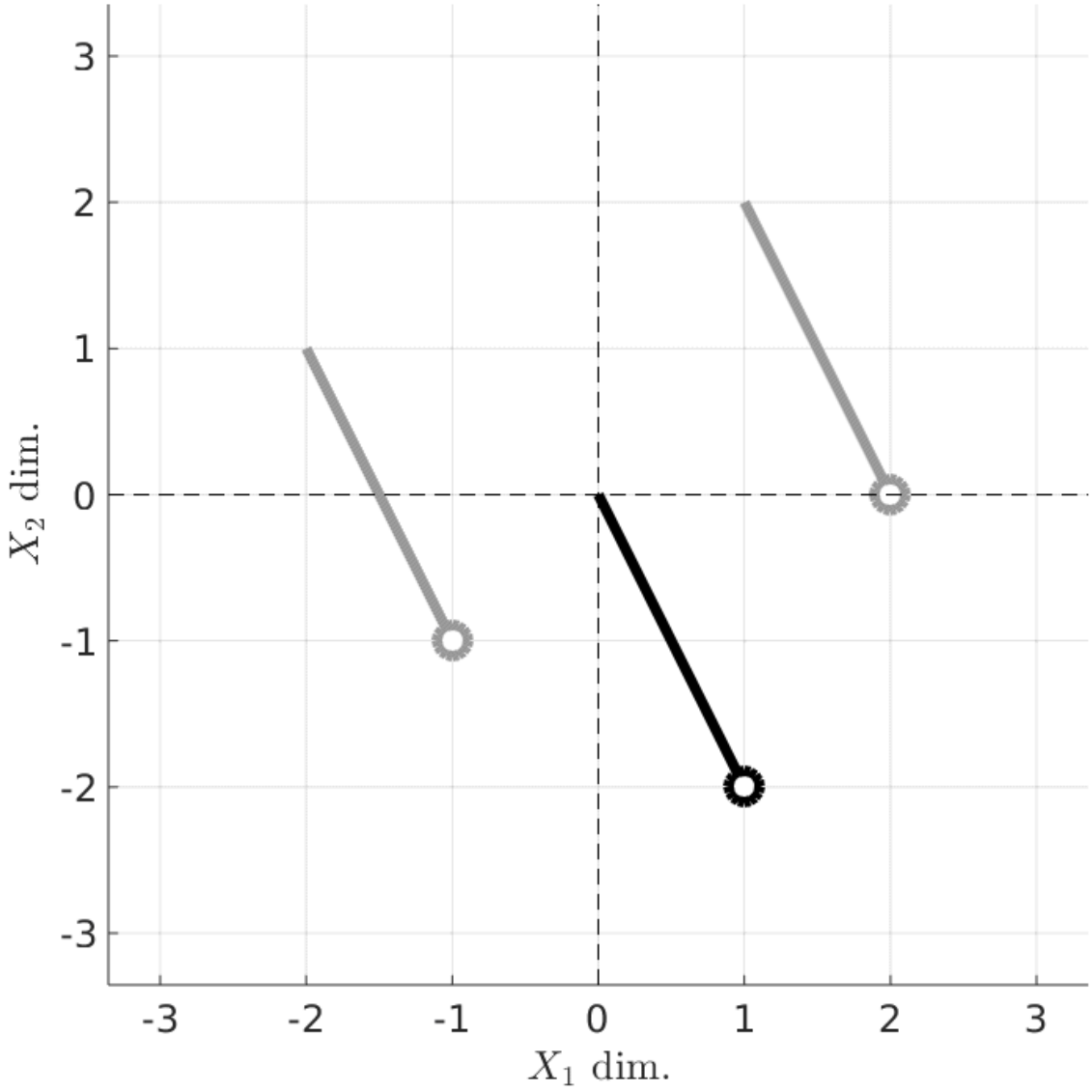
Code blockÂ 2.3: Python

```
importÂ numpyÂ asÂ npÂ
importÂ matplotlib.pyplotÂ asÂ pltÂ
vÂ =Â np.array([2,-1])Â
plt.plot([0,v[0]],[0,v[1]])Â
plt.axis([-3,3,-3,3]);
```

Code blockÂ 2.4: MATLAB

```
vÂ =Â [2Â -1];Â
plot([0,v(1)],[0,v(2)])Â
axis([-3,3,-3,3])
```

Note that Python requires you to load in libraries (here, numpy and matplotlib.pyplot) each time you run a new session. If youâ€™ve already imported the library in your current session, you wonâ€™t need to re-import it.



FigureÂ 2.3: The three coordinates (circles) are distinct, but the three vectors (lines) are the same, because they have the same magnitude and direction ([1 -2]). When the vector is in its standard position (the black vector), the head of the vector [1 -2] overlaps with the coordinate [1 -2].

Algebra A vector is an ordered list of numbers. The number of numbers in the vector is called the *dimensionality* of the vector. Here are a few examples of 2D and 3D vectors.

$$\begin{bmatrix} 1 & -2 \end{bmatrix}, \begin{bmatrix} 4 & 1 \end{bmatrix}, \begin{bmatrix} 10000 & 0 \end{bmatrix}$$

$$\begin{bmatrix} \pi^3 & \sqrt{e^2} & 0 \end{bmatrix}, \begin{bmatrix} 3 & 1 & 4 \end{bmatrix}, \begin{bmatrix} 2 & -7 & 8 \end{bmatrix}$$

The ordering of the numbers is important. For example, the following two vectors are *not* the same, even though they have the same dimensionality and the same elements.

$$\begin{bmatrix}3\\1\end{bmatrix}, \begin{bmatrix}1\\3\end{bmatrix}$$

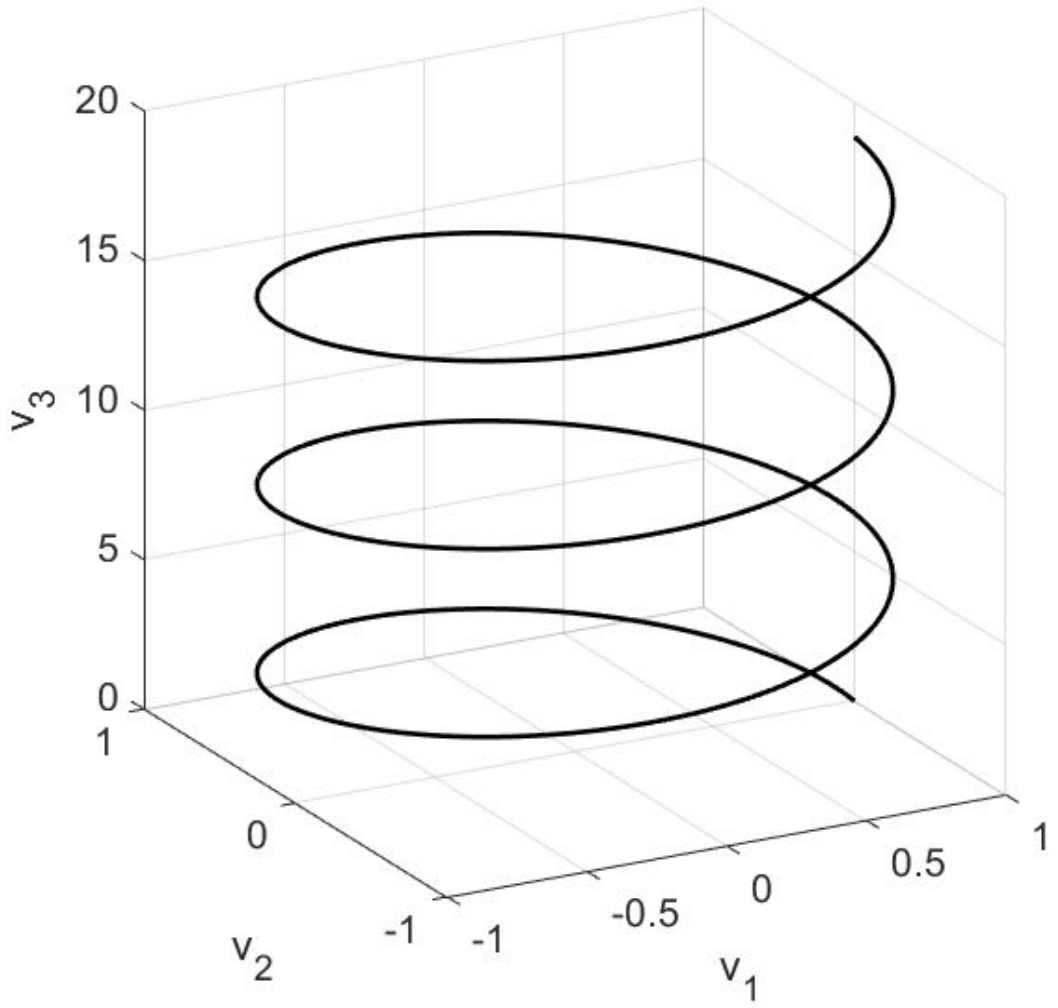
Brackets Vectors are indicated using either square brackets or parentheses . I think brackets are more elegant and less open to misinterpretation, so I use them consistently. But you will occasionally see parentheses used for vectors in other linear algebra sources. For example the following two objects are the same:

$$\begin{bmatrix}2 & 5 & 5\end{bmatrix}, \begin{pmatrix}2 & 5 & 5\end{pmatrix}$$

Be careful, however: Not all enclosing brackets simply signify vectors. The following two objects are different from the above. In fact, they are not even vectorsâ€”more on this in a few pages!

$$\begin{vmatrix}2 & 5 & 5\end{vmatrix}, \begin{Vmatrix}2 & 5 & 5\end{Vmatrix}$$

The geometric perspective on vectors is really useful in 2D and tolerable in 3D, but the algebraic perspective allows us to extend vectors into any dimensionality. Want to see a 6D vector algebraically? No problem: [3 4 6 1 -4 5]. Want to visualize that 6D vector as a line in a 6D coordinate space? Yeah, good luck with that.



FigureÂ 2.4: The geometric perspective of the vector in equation 2.1 .

Vectors are not limited to numbers; the elements can also be functions. Consider the following vector function.

$$\mathbf{v} = \begin{bmatrix} \cos(t) & \sin(t) & t \end{bmatrix} \tag{2.1}$$

(*t* is itself a vector of time points.) Vector functions are used in multivariate calculus, physics, and differential geometry. However, in this book, vectors will comprise single numbers in each element. If you want to work with the above vector for N discrete time points, then you would use a 3 Ã— *N* matrix instead of a vector-valued function.

Vector orientation Vectors can be "standing up" or "lying down." An upright vector is called a *column vector* and a lying down vector is called a *row vector* .

The dimensionality of a vector is simply the number of elements, regardless of its orientation. Column vs. row vectors are easy to distinguish visually:

$$\text{Column vectors: } \begin{bmatrix} 7 \\ 3 \\ 5 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ -9 \\ -9 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\text{Row vectors: } \begin{bmatrix} 1 & 4 & -4 & \sqrt{7} \end{bmatrix}, \begin{bmatrix} 0 & 1 \end{bmatrix}, \begin{bmatrix} 42 & 42 \end{bmatrix}$$

IMPORTANT: By convention, always assume that vectors are in column orientation, unless stated otherwise. Assuming that all vectors are column-oriented reduces ambiguity about other operations involving vectors. This is an arbitrary choice, but itâ€™s one that most people follows (not just in this book).

Sometimes, the orientation of a vector doesnâ€™t matter (this is more often the case in code), while other times it is hugely important and completely changes the outcome of an equation.

Code The semicolon in MATLAB inside square brackets is used for vertical concatenation (that is, to create a column vector). In Python, lists and numpy arrays have no intrinsic orientation (meaning they are neither row nor column vectors). In some cases that doesnâ€™t matter, while in other cases (for example, in more advanced applications) it becomes as hassle. Additional square brackets can be used to impose orientations.

Code blockÂ 2.5: Python

```
v1Â =Â [2,5,4,7]Â #Â listÂ
v2Â =Â np.array([2,5,4,7])#Â array,Â noÂ orientationÂ
v3Â =Â np.array([Â [2],[5],[4],[7]Â ])#Â col.Â vectorÂ
v4Â =Â np.array([Â [2,5,4,7]Â ])Â #Â rowÂ vector
```

Code blockÂ 2.6: MATLAB

```
v1Â =Â [2Â 5Â 4Â 7];Â %Â rowÂ vectorÂ
v2Â =Â [2;Â 5;Â 4;Â 7];Â %Â columnÂ vector
```

Notation In written texts, vectors are indicated using lower-case boldface letters, for example: vector **v** . When taking notes on paper (strongly encouraged to maximize learning!), you should draw a little arrow on top of the letter to indicate vectors, like this: v

To indicate a particular element inside a vector, a subscript is used with a non-bold-faced letter of the vector. For example, the second element in the vector **v** = $\begin{bmatrix}4 & 0 & 2\end{bmatrix}$ is indicated $v_2 = 0$. More generally, the ith element is indicated as v_i .

Itâ€™s important to see that the letter is not bold-faced when referring to a particular element. This is because subscripts can also be used to indicate different vectors in a set of vectors. Thus, v_i is the ith element in vector **v** , but $\mathbf{v}_i$ is the ith vector is a series of related vectors ($\mathbf{v}_1$, $\mathbf{v}_2$, ..., $\mathbf{v}_i$). I know, itâ€™s a bit confusing, but unfortunately thatâ€™s common notation and youâ€™ll have to get used to it. I try to make it clear from context whether Iâ€™m referring to vector element v_i or vector $\mathbf{v}_i$.

Zeros vector There is an infinite number of possible vectors, because there is an infinite number of ways of combining an infinity of numbers in a vector.

That said, there are some special vectors that you should know about. The vector that contains zeros in all of its elements is called the zeros vector . A vector that contains some zeros but other non-zero elements is not the zeros vector; itâ€™s just a regular vector with some zeros in it. To earn the distinction of being a "zeros vector," all elements must be equal to zero.

The zeros vector can also be indicated using a boldfaced zero: **0** . That can be confusing, though, because **0** also indicates the zeros matrix. Hopefully, the correct interpretation will be clear from the context.

The zeros vector has some interesting and sometimes weird properties. One weird property is that it doesnâ€™t have a direction. I donâ€™t mean its direction is zero, I mean that its direction is undefined. Thatâ€™s because the zeros vector is simply a point at the origin of a graph. Without any magnitude, it doesnâ€™t make sense to ask which direction it points in.

Practice problems State the type and dimensionality of the following vectors (e.g., "four-dimensional column vector"). For 2D vectors, additionally draw the vector starting from the origin.

a) $\begin{bmatrix}1\\2\\3\\1\end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 & 3 & 1 \end{bmatrix}$
c) $\begin{bmatrix} -1 \\ \pi \end{bmatrix}$
d) $\begin{bmatrix} 7 & 1/3 \end{bmatrix}$

Answers

a) 4D column
b) 4D row
c) 2D column
d) 2D row

_
-|*|- ***Reflection*** *This gentle introduction to scalars and vectors seems simple, but you may be surprised to learn that nearly all of linear algebra is built up from scalars and vectors. From humble beginnings, amazing things emerge. Just think of everything you can build with wood planks and nails. (But donâ€™t think of what I could build â€” Iâ€™m a terrible carpenter.)* **-|*|-**

2.3 Transpose operation

Now you know some of the basics of vectors. Letâ€™s start learning what you can do with them. The mathy term for doing stuff with vectors is *operations that act upon vectors* .

You can transform a column vector to a row vector by transposing it. The transpose operation simply means to convert columns to rows, and rows to columns. The values and ordering of the elements stay the same; only the orientation changes. The transpose operation is indicated by a super-scripted ^{T} (some authors use an italics ^{T} but I think it looks nicer in regular font). For example:

$$\begin{bmatrix} 4 & 3 & 0 \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} 4 & 3 & 0 \end{bmatrix}$$

$$\begin{bmatrix}4 & 3 & 0\end{bmatrix}^{TT} = \begin{bmatrix}4 & 3 & 0\end{bmatrix}$$

Double-transposing a vector leaves its orientation unchanged. It may seem a bit silly to double-transpose a vector, but it turns out to be a key property in several proofs that you will encounter in later chapters.

As mentioned in the previous section, [1] we assume that vectors are columns. Row vectors are therefore indicated as a transposed column vector. Thus, **v** is a column vector while $\mathbf{v}^T$ is a row vector. On the other hand, column vectors written inside text are often indicated as transposed row vectors, for example $\mathbf{w} = \begin{bmatrix}1 & 2 & 3\end{bmatrix}^T$.

[1] Reminder: Vectors are columns unless otherwise specified.

Code Transposing is easy both in MATLAB and in Python.

Code blockÂ 2.7: Python

```
v1Â =Â np.array([Â [2,5,4,7]Â ])Â #Â rowÂ vectorÂ
v2Â =Â v1.TÂ #Â columnÂ vector
```

Code blockÂ 2.8: MATLAB

```
v1Â =Â [2Â 5Â 4Â 7];Â %Â rowÂ vectorÂ
v2Â =Â v1â€™;Â %Â columnÂ vector
```

2.4 Vector addition and subtraction

Geometry To add two vectors **a** and **b** , put the start ("tail") of vector **b** at the end ("head") of vector **a** ; [2] the new vector that goes from the tail of **a** to the head of **b** is vector **a** +**b** (Figure 2.5).

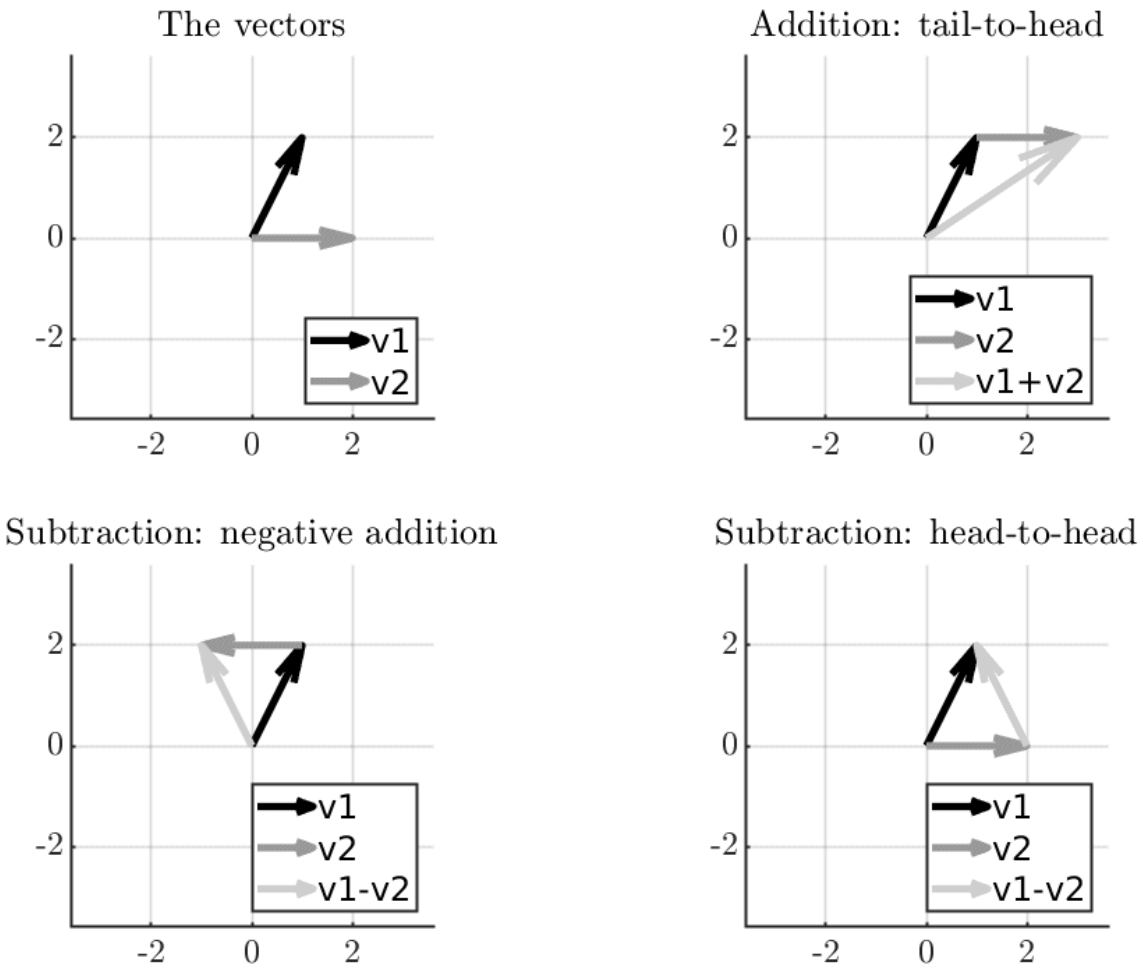
[2] Remember that a vector is defined by length and direction; the vector can start anywhere.

Vector addition is commutative , which means that **a** + **b** = **b** + **a** . This is easy to demonstrate: Get a pen and piece of paper, come up with two 2D vectors, and follow the procedure above. Of course, thatâ€™s just a demonstration, not a proof. The proof will come with the algebraic interpretation.

There are two ways to think about subtracting two vectors. One way is to multiply one of the vectors by -1 and then add them as above (Figure 2.5 , lower left). Multiplying a vector by -1 means to multiply each vector element by -1 (vector [1 1] becomes vector [-1 -1]). Geometrically, that flips the vector by 180 â˜˜ .

The second way to think about vector subtraction is to keep both vectors in their standard position, and draw the line that goes from the head of the subtracted vector (the one with the minus sign) to the head of the other vector (the one without the minus sign) (Figure 2.5 , lower right). That resulting vector is the difference. Itâ€™s not in standard position, but that doesnâ€™t matter.

You can see that the two subtraction methods give the same difference vector. In fact, they are not really *different* methods; just different ways of thinking about the same method. That will become clear in the algebraic perspective below.



FigureÂ 2.5: Two vectors (top left) can be added (top right) and subtracted (lower two plots).

Do you think that **a** âˆ’ **b** = **b** âˆ’ **a** ? Letâ€™s think about this in terms of scalars. For example: 2 âˆ’ 5 = âˆ’ 3 but 5 âˆ’ 2 = 3 . In fact, the magnitudes of the results are the same, but the signs are different. Thatâ€™s because 2 âˆ’ 5 = âˆ’ (5 âˆ’ 2) . Same story for vectors: **a** âˆ’ **b** = âˆ’ (**b** âˆ’ **a**) . The resulting difference vectors

are not the same, but they are related to each other by having the same magnitude but flipped directions. This should also be intuitive from inspecting Figure 2.5 : $\mathbf{v}_2$ â^' $\mathbf{v}_1$ would be the same line as $\mathbf{v}_1$ â^' $\mathbf{v}_2$ but with the arrow on the other side; essentially, you just swap the tail with the head.

Algebra The algebraic interpretation of vector addition and subtraction is what you probably intuitively think it should be: element-wise addition or subtraction of the two vectors. Some examples:

$$\begin{bmatrix}1 & 2\end{bmatrix} + \begin{bmatrix}3 & 4\end{bmatrix} = \begin{bmatrix}4 & 6\end{bmatrix}$$

$$\begin{bmatrix}0\\12\end{bmatrix} + \begin{bmatrix}-2\\-4\end{bmatrix} = \begin{bmatrix}-2\\8\end{bmatrix}$$

$$\begin{bmatrix}1\\2\\3\end{bmatrix} \text{â^'} \begin{bmatrix}9\\-8\\7\end{bmatrix} = \begin{bmatrix}-8\\10\\-4\end{bmatrix}$$

More formally:

Eqn. box title=**Vector addition and subtraction**

$$\mathbf{c} = \mathbf{a} + \mathbf{b} = \begin{bmatrix}a_1+b_1 & a_2+b_2 & \dots & a_n+b_n\end{bmatrix}^T \qquad (2.2)$$

Important: Vector addition and subtraction are valid only when the vectors have the same dimensionality.

Practice problems Solve the following operations. For 2D vectors, draw both vectors starting from the origin, and the vector sum (also starting from the origin).

a) $\begin{bmatrix}4 & 5 & 1 & 0\end{bmatrix} + \begin{bmatrix}-4 & -3 & 3 & 10\end{bmatrix}$

b) $\begin{bmatrix}4\\2\\0\end{bmatrix} \text{â^'} \begin{bmatrix}6\\-4\\60\end{bmatrix} + \begin{bmatrix}2\\-5\\40\end{bmatrix}$

c) $\begin{bmatrix}1\\0\end{bmatrix} + \begin{bmatrix}1\\2\end{bmatrix}$

d)

$\begin{bmatrix}2\\2\end{bmatrix}$ â^’ $\begin{bmatrix}3\\4\end{bmatrix}$

e) $\begin{bmatrix}-3\\1\end{bmatrix}+\begin{bmatrix}3\\-1\end{bmatrix}$

f) $\begin{bmatrix}1\\4\end{bmatrix}+\begin{bmatrix}2\\8\end{bmatrix}$

Answers

a) $\begin{bmatrix}0 & 2 & 4 & 10\end{bmatrix}$

b) $\begin{bmatrix}0\\1\\100\end{bmatrix}$

c) $\begin{bmatrix}2\\2\end{bmatrix}$

d) $\begin{bmatrix}-1\\-2\end{bmatrix}$

e) $\begin{bmatrix}0\\0\end{bmatrix}$

f) $\begin{bmatrix}3\\12\end{bmatrix}$

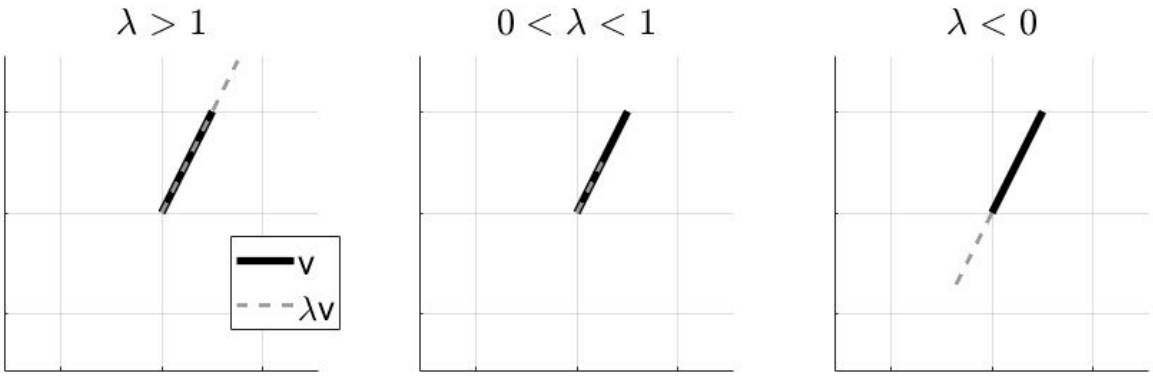
–

2.5 Vector-scalar multiplication

Geometry *Scaling* a vector means making it shorter or longer without changing its angle (that is, without rotating it) (Figure 2.6).

The main source of confusion here is that scaling a vector by a negative number means having it point "backwards." You might think that this is a "different" direction, and arguably it isâ€”the vector is rotated by 180 â^˜ . For now, letâ€™s say that the result of vector-scalar multiplication (the scaled vector) must form either a 0 â^˜ or a 180 â^˜ angle with the original vector. There is a deeper and more important explanation for this, which has to do with an infinitely long line that the vector defines (a "1D subspace"); youâ€™ll learn about this in Chapter 4.

Still, the important thing is that the scalar does not rotate the vector off of its original orientation. In other words, vector direction is invariant to scalar multiplication.



FigureÂ 2.6: Multiplying a scalar (*Î»*) by a vector (**v**) means to stretch or shrink the vector without changing its angle. *Î»* > 1 means the resulting vector will be longer than the original, and 0 < *Î»* < 1 means the resulting vector will be shorter. Note the effect of a negative scalar (*Î»* < 0): The resulting scaled vector points "the other way" but it still lies on the same imaginary infinitely long line as the original vector.

Strange things can happen in mathematics when you multiply by 0. Vector-scalar multiplication is no different: Scaling a vector by 0 reduces it to a point at the origin. That point cannot be said to have any angle, so itâ€™s not really a sensible question whether scaling a vector by 0 preserves or changes the angle.

Algebra Scalar-vector multiplication is achieved by multiplying each element of the vector by the scalar. For scalar *Î»* and vector **v** , a formal definition is:

_____
_

Eqn. box title=**Scalar-vector multiplication**

$$\lambda \mathbf{v} = \begin{bmatrix} \lambda v_1 & \lambda v_2 & \dots & \lambda v_n \end{bmatrix}^T \qquad (2.3)$$

_____
_____

This definition holds for any number of dimensions and for any scalar. Here is one example:

$$3\begin{bmatrix}-1 & 3 & 0 & 2\end{bmatrix} = \begin{bmatrix}-3 & 9 & 0 & 6\end{bmatrix}$$

Because the scalar-vector multiplication is implemented as element-wise multiplication, it obeys the commutative property. That is, a scalar times a vector is the same thing as that vector times that scalar: $\hat{I}»\ \mathbf{v} = \mathbf{v}\ \hat{I}»$. This fact becomes key to several proofs later in the book.

Code Basic vector arithmetic (adding, subtracting, and scalar-multiplying) is straightforward.

Code blockÂ 2.9: Python

```
importÂ numpyÂ asÂ npÂ
v1Â =Â np.array([2,5,4,7])Â
v2Â =Â np.array([4,1,0,2])Â
v3Â =Â 4*v1Â -Â 2*v2
```

Code blockÂ 2.10: MATLAB

```
v1Â =Â [2Â 5Â 4Â 7];Â
v2Â =Â [4Â 1Â 0Â 2];Â
v3Â =Â 4*v1Â -Â 2*v2
```

–

Practice problems Compute scalar-vector multiplication for the following pairs:

a) â^’ 2$\begin{bmatrix}4 & 3 & 0\end{bmatrix}$

b) (â^’ 9 + 2 Ã— 5)$\begin{bmatrix}0\\4\\3\end{bmatrix}$

c) $0\begin{bmatrix}3\\3.14\times\pi^{3.14}\\9\\-234987234\end{bmatrix}$

d) $\hat{I}»\begin{bmatrix}0\\3\\1\\11\end{bmatrix}$

Answers

a) $\begin{bmatrix} -8 & -6 & 0 \end{bmatrix}$

b) $\begin{bmatrix} 0 \\ 4 \\ 3 \end{bmatrix}$

c) $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

d) $\begin{bmatrix} 0 \\ \lambda 3 \\ \lambda \\ \lambda 11 \end{bmatrix}$

_

-|*|- ***Reflection*** *Vector-scalar multiplication is conceptually and computationally simple, but do not underestimate its importance: Stretching a vector without rotating it is fundamental to many applications in linear algebra, including eigendecomposition. Sometimes, the simple things (in mathematics and in life) are the most powerful.* -|*|-

2.6 Exercises

1.Â

Simplify the following vectors by factoring out common scalars. For example, [2 4] can be simplified to 2[1 2].

a) $\begin{bmatrix} 6 \\ 3 \\ 33 \\ 9 \end{bmatrix}$

b) $\begin{bmatrix} 48 \\ 12 \\ 24 \\ 60 \end{bmatrix}$

c) $\begin{bmatrix} 12 \\ 18 \\ 24 \end{bmatrix}$

d)

$$\begin{bmatrix} 45 \\ 36 \\ 99 \\ 72 \\ 27 \end{bmatrix}$$

2.Â
Draw the following vectors [ ] using the listed starting point ().

a) $\begin{bmatrix} 2 & 2 \end{bmatrix}$ (0,0)
b) $\begin{bmatrix} 6 & 12 \end{bmatrix}$ (1,-2)
c) $\begin{bmatrix} -1 & 0 \end{bmatrix}$ (4,1)
d) $\begin{bmatrix} \pi & e \end{bmatrix}$ (0,0)
e) $\begin{bmatrix} 1 & 2 \end{bmatrix}$ (0,0)
f) $\begin{bmatrix} 1 & 2 \end{bmatrix}$ (-3,0)
g) $\begin{bmatrix} 1 & 2 \end{bmatrix}$ (2,4)
h) $\begin{bmatrix} -1 & -2 \end{bmatrix}$ (0,0)
i) $\begin{bmatrix} -3 & 0 \end{bmatrix}$ (1,0)
j) $\begin{bmatrix} -4 & -2 \end{bmatrix}$ (0,3/2)
k) $\begin{bmatrix} 8 & 4 \end{bmatrix}$ (1,1)
l) $\begin{bmatrix} -8 & -4 \end{bmatrix}$ (8,4)

3.Â
Label the following as column or row vectors, and state their dimensionality. Your answer should be in the form, e.g., "three-dimensional column vector."

a) $\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \end{bmatrix}$
b) $\begin{bmatrix} 5 \\ 0 \\ 3 \end{bmatrix}$
c) $\begin{bmatrix} 0 & 0 \end{bmatrix}$
d) $\begin{bmatrix} \sin(2) \\ e^{\pi} \\ 1/3 \end{bmatrix}$
e) $\begin{bmatrix} 20 & 4000 & 80000 & .1 & 0 & 0 \end{bmatrix}$

4.Â

Perform vector-scalar multiplication on the following. For 2D vectors, additionally draw the original and scalar-multiplied vectors.

a) $3\begin{bmatrix}1\\2\end{bmatrix}$

b) $\frac{1}{3}\begin{bmatrix}12 & 6\end{bmatrix}$

c) $0\begin{bmatrix}e^{10000}\\1\\0\\\sqrt{\pi}\end{bmatrix}$

d) $4\begin{bmatrix}-3\\3\end{bmatrix}$

e) $\hat{I}\text{»}\begin{bmatrix}a & b & c & d & e\end{bmatrix}$

f) $\hat{I}^3$ [0 0 0 0 0]

5.Â

Add or subtract the following pairs of vectors. Draw the individual vectors and their sum (all starting from the origin), and confirm that the algebraic and geometric interpretations match.

a) $\begin{bmatrix}0\\1\end{bmatrix}+\begin{bmatrix}1\\0\end{bmatrix}$

b) $\begin{bmatrix}6\\-2\end{bmatrix}$ â^’ $\begin{bmatrix}2\\-6\end{bmatrix}$

c) $\begin{bmatrix}2\\1\end{bmatrix}+\begin{bmatrix}1\\2\end{bmatrix}$

d) $\begin{bmatrix}2\\3\end{bmatrix}+\begin{bmatrix}2\\3\end{bmatrix}$

e) $\begin{bmatrix}2\\3\end{bmatrix}$ â^’ $\begin{bmatrix}2\\3\end{bmatrix}$

f) $\begin{bmatrix}1\\1\end{bmatrix}+\begin{bmatrix}1\\-1\end{bmatrix}$

g) $\begin{bmatrix}4\\2\end{bmatrix}+\begin{bmatrix}1\\0\end{bmatrix}$

h) $\begin{bmatrix}-3\\-5\end{bmatrix}+\begin{bmatrix}7\\3\end{bmatrix}$

i) $\begin{bmatrix}-5\\-2\end{bmatrix}+\begin{bmatrix}7\\7\end{bmatrix}$

j) $\begin{bmatrix}1\\2\end{bmatrix}+\begin{bmatrix}1\\3\end{bmatrix}$ â^’ $\begin{bmatrix}3\\-2\end{bmatrix}$

k) $\begin{bmatrix}-2\\-5\end{bmatrix}$ â^’ $\begin{bmatrix}-7\\-6\end{bmatrix}$ â^’ $\begin{bmatrix}0\\4\end{bmatrix}$

l) $\begin{bmatrix}0\\1\end{bmatrix}$ â^’ $\begin{bmatrix}3\\3\end{bmatrix}+\begin{bmatrix}1\\2\end{bmatrix}$

2.7 Answers

1.Â

a) $3\begin{bmatrix}2\\1\\11\\3\end{bmatrix}$

b) $12\begin{bmatrix}4\\1\\2\\5\end{bmatrix}$

c) $6\begin{bmatrix}2\\3\\4\end{bmatrix}$

d) $9\begin{bmatrix}5\\4\\11\\8\\3\end{bmatrix}$

2.Â

This one you should be able to do on your own. You just need to plot the lines from their starting positions. The key here is to appreciate the distinction between vectors and coordinates (they overlap when vectors are in the standard position of starting at the origin).

3.Â

a) 5D row vector

b) 3D column vector

c) 2D row vector

d) 3D column vector

e) 6D row vector

4.Â
Iâ€™ll let you handle the drawing; below are the algebraic solutions.

a) $\begin{bmatrix} 3 \\ 6 \end{bmatrix}$

b) $\begin{bmatrix} 4 & 2 \end{bmatrix}$

c) $\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

d) $\begin{bmatrix} -12 \\ 12 \end{bmatrix}$

e) $\begin{bmatrix} \lambda a \\ \lambda b \\ \lambda c \\ \lambda d \\ \lambda e \end{bmatrix}^{\mathrm{T}}$

f) $\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \end{bmatrix}$

5.Â

These should be easy to solve if you passed elementary school arithmetic. There is, however, a high probability of careless mistakes (indeed, the further along in math you go, the more likely you are to make arithmetic errors).

a) $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

b) $\begin{bmatrix} 4 \\ 4 \end{bmatrix}$

c) $\begin{bmatrix} 3 \\ 3 \end{bmatrix}$

d) $\begin{bmatrix} 4 \\ 6 \end{bmatrix}$

e) $\begin{bmatrix} 0 \\ 0 \end{bmatrix}$

f) $\begin{bmatrix} 2 \\ 0 \end{bmatrix}$

g) $\begin{bmatrix} 5 \\ 2 \end{bmatrix}$

h) $\begin{bmatrix} 4 \\ -2 \end{bmatrix}$

i) $\begin{bmatrix} 2 \\ 5 \end{bmatrix}$

j) $\begin{bmatrix} -1 \\ 7 \end{bmatrix}$

k) $\begin{bmatrix} 5 \\ -3 \end{bmatrix}$

l) $\begin{bmatrix} -2 \\ 0 \end{bmatrix}$

2.8 Code challenges

1.Â

Create a 2D vector **v** , 10 scalars that are drawn at random from a normal (Gaussian) distribution, and plot all 10 scalar-vector multiplications on top of each other. What do you notice?

2.9 Code solutions

1.Â

Youâ€™ll notice that all scaled versions of the vector form a line. Note that the Python implementation requires specifying the vector as a numpy array, not as a list.

Code blockÂ 2.11: Python

```
importÂ numpyÂ asÂ npÂ
importÂ matplotlib.pyplotÂ asÂ pltÂ
vÂ =Â np.array([1,2])Â
plt.plot([0,v[0]],[0,v[1]])Â
forÂ iÂ inÂ range(10):Â
Â Â Â Â sÂ =Â np.random.randn()Â
Â Â Â Â svÂ =Â s*vÂ
Â Â Â Â plt.plot([0,sv[0]],[0,sv[1]])Â
plt.grid(â€™onâ€™)Â
plt.axis([-4,4,-4,4]);
```

Code blockÂ 2.12: MATLAB

```
vÂ =Â [Â 1Â 2Â ];Â
plot([0Â v(1)],[0Â v(2)])Â
```

```
holdÂ onÂ
forÂ i=1:10Â
Â Â Â Â sÂ =Â randn;Â
Â Â Â Â svÂ =Â s*v;Â
Â Â Â Â plot([0Â sv(1)],[0Â sv(2)])Â
endÂ
gridÂ on,Â axis([-1Â 1Â -1Â 1]*4)
```

ChapterÂ 3
Vector multiplications

In this chapter, we continue our adventures through the land of vectors. Warning: There are a few sections here that might be challenging. Please donâ€™t get discouragedâ€”once you make it through this chapter, you can be confident that you can make it through the rest of the book.

There are four ways to multiply a pair of vectors. They are: dot product, outer product, element-wise multiplication, and cross product. The dot product is the most important and owns most of the real estate in this chapter.

3.1 Vector dot product: Algebra

The dot product , also called the inner product or the scalar product (not to be confused with the scalar-vector product), is **one of the most important operations in all of linear algebra** . It is the basic computational building-block from which many operations and algorithms are built, including convolution, correlation, the Fourier transform, matrix multiplication, signal filtering, and so on.

The dot product is a single number that provides information about the relationship between two vectors. This fact (two vectors produce a scalar) is why itâ€™s sometimes called the "scalar product." The term "inner product" is used when the the two vectors are continuous functions. I will use only the term *dot product* for consistency.

Algebraically , to compute the dot product, you multiply the corresponding elements of the two vectors, and then sum over all the individual products. In the box on the next page, the middle three terms show the various notations to indicate the dot product, while the final term shows the algebraic definition.

Eqn. box title=**The dot product**

$$\hat{I}\pm = \mathbf{a}\ \hat{a}\check{z}\ldots\ \mathbf{b} = \hat{a}\ddot{Y}\ddot{}\ \mathbf{a}\ ,\ \mathbf{b}\ \hat{a}\ddot{Y}\copyright = \mathbf{a}^{T}\mathbf{b} = \hat{a}\hat{}\check{z}_{i=1}^{n} a_i b_i \qquad (3.1)$$

An example:

$$\begin{bmatrix}1 & 2 & 3 & 4\end{bmatrix}\ \hat{a}\check{z}\ldots\begin{bmatrix}5 & 6 & 7 & 8\end{bmatrix} = 1\ \tilde{A}\text{—}\ 5 + 2\ \tilde{A}\text{—}\ 6 + 3\ \tilde{A}\text{—}\ 7 + 4\ \tilde{A}\text{—}\ 8$$
$$= 5 + 12 + 21 + 32$$
$$= \mathbf{70}$$

I will mostly use the notation $\mathbf{a}^{T}\mathbf{b}$ for reasons that will become clear after learning about matrix multiplication.

Why does the dot product require two vectors of equal dimensionality? Try to compute the dot product between the following two vectors:

$$\begin{bmatrix}3\\4\\5\\0\\2\end{bmatrix} \cdot \begin{bmatrix}-2\\0\\1\\8\end{bmatrix} = 3\times-2+4\times0+5\times1+0\times8+2\times???$$

You cannot complete the operation because there is nothing to multiply the final element in the left vector. Thus, the dot product is defined only between two vectors that have the same dimensionality.

You can compute the dot product between a vector and itself. Equation 3.2 shows that this works out to be the sum of squared elements, and is denoted â˜¥ **a** â˜¥ [2] . The term â˜¥ **a** â˜¥ is called the [1] *magnitude* , the *length* , or the *norm* of vector **a** . Thus, the dot product of a vector with itself is called the magnitude-squared, the length-squared, or the squared-norm, of the vector.

[1] If the vector is mean-centeredâ€”the average of all vector elements is subtracted from each elementâ€”then the dot product of a vector with itself is call *variance* in statistics lingo.

$$\mathbf{a}^{\mathrm{T}}\mathbf{a} = \|\mathbf{a}\|^2 = \sum_{i=1}^{n} \mathrm{a}_i \mathrm{a}_i = \sum_{i=1}^{n} \mathrm{a}_i^2 \tag{3.2}$$

I guess thereâ€™s a little voice in your head wondering whether itâ€™s called the length because it corresponds to the geometric length of the vector. Your intuition is correct, as you will soon learn. But first, we need to discuss some of the properties of the dot product that can be derived algebraically.

Code There are several ways to compute the dot product in Python and in MATLAB; here I will show one method.

Code blockÂ 3.1: Python

```
v1 Â  = Â  np . array ([2,5,4,7]) Â
v2 Â  = Â  np . array ([4,1,0,2]) Â
dp Â  = Â  np . dot ( v1 , v2 )
```

Code blockÂ 3.2: MATLAB

```
v1 Â  = Â  [2 Â  5 Â  4 Â  7]; Â
v2 Â  = Â  [4 Â  1 Â  0 Â  2]; Â
dp Â  = Â  dot ( v1 , v2 )
```

3.2 Dot product properties

Associative property The associative property of the dot product can be interpreted in two ways. First is the associative property of the vector dot product with a scalar. [2] In fact, this is simply the associative property of scalar-vector multiplication embedded inside the dot product, so itâ€™s really nothing new. I mention it here because it is important for proofs later in the book

[2] The *associative law* is that parentheses can be moved around, e.g., (*x* + *y*)+ *z* = *x* +(*y* + *z*) and *x* (*yz*) = (*xy*) *z*

$$\gamma(\mathbf{u}^{\mathrm{T}}\mathbf{v}) = (\gamma\mathbf{u}^{\mathrm{T}})\mathbf{v} = \mathbf{u}^{\mathrm{T}}(\gamma\mathbf{v}) = (\mathbf{u}^{\mathrm{T}}\mathbf{v})\gamma \tag{3.3}$$

But the second interpretation is what most people refer to when discussing dot products. Let us consider three vectors and investigate what happens when we move the parentheses around. In other words, is the following statement true?

$$\mathbf{u}^{\mathrm{T}}(\mathbf{v}^{\mathrm{T}}\mathbf{w}) = (\mathbf{u}^{\mathrm{T}}\mathbf{v})^{\mathrm{T}}\mathbf{w} \tag{3.4}$$

The answer is No. To understand why, letâ€™s start by assuming that all three vectors have the same dimensionality. In that case, each side of the equation is individually a valid mathematical operation, but the two sides of the equation will differ from each other. **In fact, neither side is a dot product** . The left-hand side of Equation 3.4 becomes the vector-scalar product between row vector $\mathbf{u}^{\mathrm{T}}$ and the scalar resulting from the dot product $\mathbf{v}^{\mathrm{T}}\mathbf{w}$. Thus, the left-hand side of the equation is a row vector. Similar story for the right-hand side: It is the scalar-vector multiplication between the scalar $\mathbf{u}^{\mathrm{T}}\mathbf{v}$ and the column vector **w** (donâ€™t be confused by transposing a scalar: $4^{\mathrm{T}} = 4$).

Therefore, the two sides of the equation are not equal; they wouldnâ€™t even satisfy a "soft equality" of having the same elements but in a different orientation.

Letâ€™s see a quick example to make sure this is clear.

$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

$$\mathbf{u}^{\mathrm{T}}(\mathbf{v}^{\mathrm{T}}\mathbf{w}) = \begin{bmatrix} 1 & 2 \end{bmatrix} \left(\begin{bmatrix} 1 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} \right) = \begin{bmatrix} 11 & 22 \end{bmatrix} \tag{3.5}$$

$$(\mathbf{u}^{\mathrm{T}}\mathbf{v})^{\mathrm{T}}\mathbf{w} = \left(\begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \end{bmatrix} \right)^{\mathrm{T}} \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 14 \\ 21 \end{bmatrix} \tag{3.6}$$

And it gets even worse, because if the three vectors have different dimensionalities, then one or both sides of Equation 3.4 might even be invalid. Iâ€™ll let you do the work to figure this one out, but imagine what would happen if the dimensionalities of **u** , **v** , and **w** , were, respectively, 3, 3, and 4.

The conclusion here is that the *vector dot product does not obey the associative property* . (Just to make life confusing, matrix multiplication does obey the associative property, but at least you donâ€™t need to worry about that for several more chapters.)

Commutative property The commutative property holds for the vector dot product. This means that you can swap the order of the vectors that are being "dot producted" together (Iâ€™m not sure if dot product can be used as a verb like that, but you know what I mean), and the result is the same.

Eqn. box title=**The dot product is commutative**

$$\mathbf{a}^{\mathrm{T}}\mathbf{b} = \mathbf{b}^{\mathrm{T}}\mathbf{a} \tag{3.7}$$

Commutivity holds because the dot product is implemented element-wise, and each element-wise multiplication is simply the product of two scalars. Scalar multiplication is commutative, and therefore the dot product is commutative.

$$\sum_{i=1}^{n} a_i b_i = \sum_{i=1}^{n} b_i a_i \tag{3.8}$$

Distributive property This one also holds for the dot product, and it turns out to be really important [3] for showing the link between the algebraic definition of the dot product with the geometric definition of the dot product, which you will learn below.

[3] The *distributive law* is that scalars distribute inside parentheses, e.g., $a\,(\,b + c\,) = ab + ac$

When looking at the equation below, keep in mind that the sum of two vectors is simply another vector. (Needless to say, Equation 3.9 is valid only when all three vectors have the same dimensionality.)

Eqn. box title=**The dot product is distributive**

$$\mathbf{w}^T(\mathbf{u} + \mathbf{v}) = \mathbf{w}^T\mathbf{u} + \mathbf{w}^T\mathbf{v} \qquad (3.9)$$

The distributive property says that we can break up a dot product into the sum of two dot products, by breaking up one of the vectors into the sum of two vectors. Conversely, you can turn this around: We can combine two dot products into one by summing two vectors into one vector, as long as the two dot products share a common vector (in this example, $\mathbf{w}$).

Why is Equation 3.9 true? This has to do with how the dot product is defined as the sum of element-wise multiplications. Common terms can be combined across sums, which brings us to the following:

$$\sum_{i=1}^{n} w_i(u_i + v_i) = \sum_{i=1}^{n} w_i u_i + \sum_{i=1}^{n} w_i v_i \qquad (3.10)$$

Numerical examples always help build intuition. Letâ€™s have a look:

$$\mathbf{u} = \begin{bmatrix}1\\2\end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix}1\\3\end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix}2\\3\end{bmatrix}$$

$$\mathbf{w}^T(\mathbf{u} + \mathbf{v}) = \begin{bmatrix}2 & 3\end{bmatrix}\left(\begin{bmatrix}1\\2\end{bmatrix} + \begin{bmatrix}1\\3\end{bmatrix}\right) = \begin{bmatrix}2 & 3\end{bmatrix}\begin{bmatrix}2\\5\end{bmatrix} = 19 \qquad (3.11)$$

$$\mathbf{w}^T\mathbf{u} + \mathbf{w}^T\mathbf{v} = \begin{bmatrix}2 & 3\end{bmatrix}\begin{bmatrix}1\\2\end{bmatrix} + \begin{bmatrix}2 & 3\end{bmatrix}\begin{bmatrix}1\\3\end{bmatrix} = 8 + 11 = 19 \qquad (3.12)$$

Equation 3.13 below shows an example of applying the distributive property to the dot product between a vector and itself, where the vector is expressed as the sum of two vectors. Youâ€™ve already learned that the dot product of a vector with itself is the magnitude squared; the equation below expands on this idea.

$$\begin{aligned}(\mathbf{u} + \mathbf{v})^T(\mathbf{u} + \mathbf{v}) = \|\mathbf{u} + \mathbf{v}\|^2 &= \mathbf{u}^T\mathbf{u} + 2\,\mathbf{u}^T\mathbf{v} + \mathbf{v}^T\mathbf{v} \\ &= \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 + 2\,\mathbf{u}^T\mathbf{v} \qquad (3.13)\end{aligned}$$

Notice that combining $\mathbf{u}^T\mathbf{v}$ and $\mathbf{v}^T\mathbf{u}$ into $2\,\mathbf{u}^T\mathbf{v}$ is valid because of the commutivity property.

This result may seem like an uninteresting academic exercise, but itâ€™s not: Equation 3.13 will allow us to link the algebraic and geometric interpretations of the dot product.

Cauchy-Schwarz inequality The Cauchy-Schwarz inequality provides an upper bound for the dot product between two vectors. In particular, the inequality is:

$$|\mathbf{v}^{\mathrm{T}}\mathbf{w}| \leq \|\mathbf{v}\|\|\mathbf{w}\| \tag{3.14}$$

In English, this inequality says that the magnitude (absolute value) of the dot product between two vectors is no larger than the product of the norms of the individual vectors. The expressions will equal each other when one vector is a scaled version of the other vector, that is, when $\mathbf{v} = \hat{I}» \mathbf{w}$.

The Cauchy-Schwarz inequality comes up often in linear algebra. In this book, it is relevant for normalizing the quadratic form of a matrix (Chapter 17), which in turn is part of the basis of the principal components analysis (Chapter 19).

There are many proofs of this inequality; Iâ€™m going to show one that relies on the geometric perspective of the dot product. So put a mental pin in this inequality and weâ€™ll come back to it in a few pages. _____

Practice problems Compute the dot product between the following pairs of vectors.

a) $\begin{bmatrix}-4\\-2\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}1\\3\end{bmatrix}$

b) $\begin{bmatrix}2\\3\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}2\\-3\end{bmatrix}$

c) $\begin{bmatrix}7\\-2\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}-7\\-24\end{bmatrix}$

d) $\begin{bmatrix}3/2\\4/5\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}2/3\\-5/4\end{bmatrix}$

e) $\begin{bmatrix}0\\1\\2\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}-2\\-1\\0\end{bmatrix}$

f) $\begin{bmatrix}4\\1\\3\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}4\\1\\3\end{bmatrix}$

g) $\begin{bmatrix}7/2\\-3\\6\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}10\\3.5\\-4\end{bmatrix}$

h) $\begin{bmatrix}81\\3\\9\end{bmatrix}^{\mathrm{T}}\frac{1}{3}\begin{bmatrix}1\\1\\1\end{bmatrix}$

Answers

a) -10
b) -5
c) -1
d) 0
e) -1
f) 26
g) 1/2
h) 31 = (81+3+9) *â^•* 3

_____

3.3 Vector dot product: Geometry

Geometrically, the dot product is the cosine of the angle between the two vectors, times the lengths of the two vectors. That seems very different from the algebraic definition, and itâ€™s also not intuitive that those are the same operation. In this section, we will discover some properties and implications of the geometric formula for the dot product; then, in the following section, you will see that the geometric and algebraic formulae are simply different ways of expressing the same concept.

Eqn. box title=**Geometric definition of the dot product**

$$\mathbf{a}^{\mathrm{T}}\mathbf{b} = \hat{a}\text{¥}\,\mathbf{a}\,\hat{a}\text{¥}\hat{a}\text{¥}\,\mathbf{b}\,\hat{a}\text{¥}\cos(\eth \quad \text{œ}f_{ab}) \qquad (3.15)$$

Note that if both vectors have unit length ($|\mathbf{a}| = |\mathbf{b}| = 1$), then Equation 3.15 reduces to the cosine of the angle between the two vectors.

Equation 3.15 can be rewritten to give an expression for the angle between two vectors.

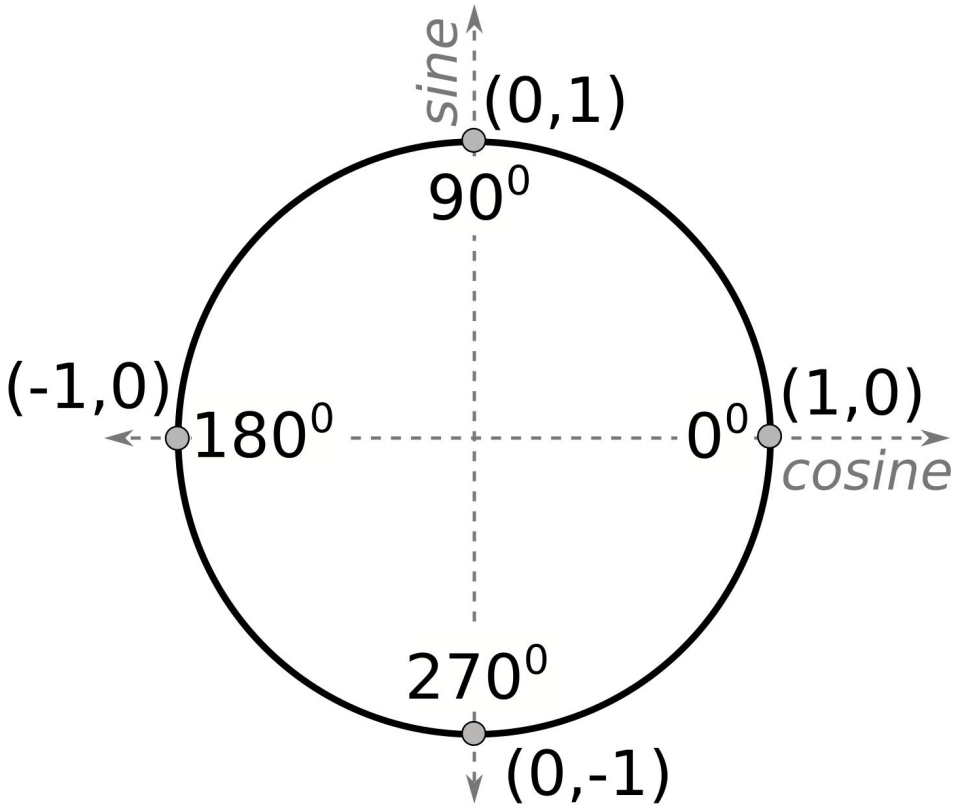
[4] In statistics, Equation 3.16 with suitable normalization is called the *Pearson correlation coefficient*. More on this in Chapter 18!

$$\cos(\theta_{ab}) = \frac{\mathbf{a}^{\mathrm{T}}\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} \qquad (3.16)$$

$$\theta_{ab} = \cos^{-1}\left(\frac{\mathbf{a}^{\mathrm{T}}\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}\right) \qquad (3.17)$$

One of the interesting features of Equation 3.17 is that it generalizes the concept of an angle between two lines in 2D, to an angle between two vectors in any higher-dimensional space.
The sign of the dot product between two vectors is determined entirely by the angle between the two vectors. This is a powerful relationship and it provides a geometric intuition for basically every application of the dot product (and there are many, as you will learn throughout this book and in your adventures in linear algebra).

First, letâ€™s understand *why* the sign of the dot product is determined exclusively by the angle between the two vectors. Equation 3.15 says that the dot product is the product of three quantities: two magnitudes and a cosine. Magnitudes are lengths, and therefore cannot be negative (magnitudes can be zero for the zeros vector, but letâ€™s assume for now that we are working with non-zero-magnitude vectors). The cosine of an angle can range between -1 and +1. Thus, the first two terms (â^¥ *a* â^¥â^¥ *b* â^¥) are necessarily non-negative, meaning the cosine of the angle between the two vectors alone determines whether the dot product is positive or negative.

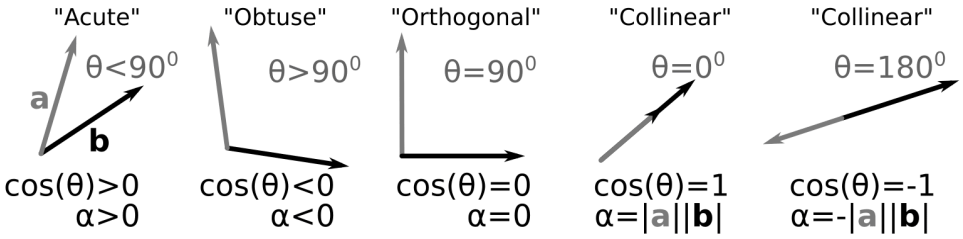


FigureÂ 3.1: Unit circle. The x-axis coordinate corresponds to the cosine of the angle from the origin to each point.

With that in mind, we can group dot products into five categories according to the angle between the vectors (Figure 3.2) (in the list below, *ð œƒ* is the angle between the two vectors and *Î±* is the dot product):

1. *ð œƒ* < 90 â€˜ *Î±* > 0 . The cosine of an acute angle is always positive, so the dot product will be positive.
2. *ð œƒ* > 90 â€˜ *Î±* < 0 . The cosine of an obtuse angle is always negative, so the dot product will be negative.
3. *ð œƒ* = 90 â€˜ *Î±* = 0 . The cosine of a right angle is zero, so the dot product will be zero, regardless of the magnitudes of the vectors. This is such an important case that it has its own name: **orthogonal** . [5] Commit to memory that if two vectors meet at a right angle, their dot product is exactly zero, and they are said to be orthogonal to each other. This important concept is central to statistics, machine-learning, eigendecomposition, SVD, the Fourier transform, and so on. The symbol for perpendicularity is an upside-down "T." So you can write that two vectors are orthogonal as **w** âŠ¥ **v** .
4. *ð œƒ* = 0 â€˜ *Î±* = â¥ *a* â¥â¥ *b* â¥ . The cosine of 0 is 1, so the dot product reduces to the product of the magnitudes of the two vectors. The term for this situation is **collinear** (meaning on the same line). This also means that the dot product of a vector with itself is simply â¥ **a** â¥ ², which is the same result obtained algebraically in Equation 3.2 .
5.

ð œf = 180 â˚ â†’ Î± = â^’â^¥ *a* â^¥â^¥ *b* â^¥ . Basically the same situation as above, but with a negative sign because cos (180 â˚) = â^’ 1 . Still referred to as collinear.



[5] Important: Vectors are *orthogonal* when they meet at a 90 â˚ angle, and orthogonal vectors have a dot product of 0.

FigureÂ 3.2: Inferences that can be made about the sign of the dot product (Î±), based on the angle (ð œf) between the two vectors. Visualization is in 2D, but the terms and conclusions extend to any number of dimensions.

Keep in mind that the *magnitude* of the dot product depends on all three terms (the magnitudes of the two vectors and the cosine of the angle); the discussion above pertains only to the *sign* of the dot product as negative, zero, or positive.

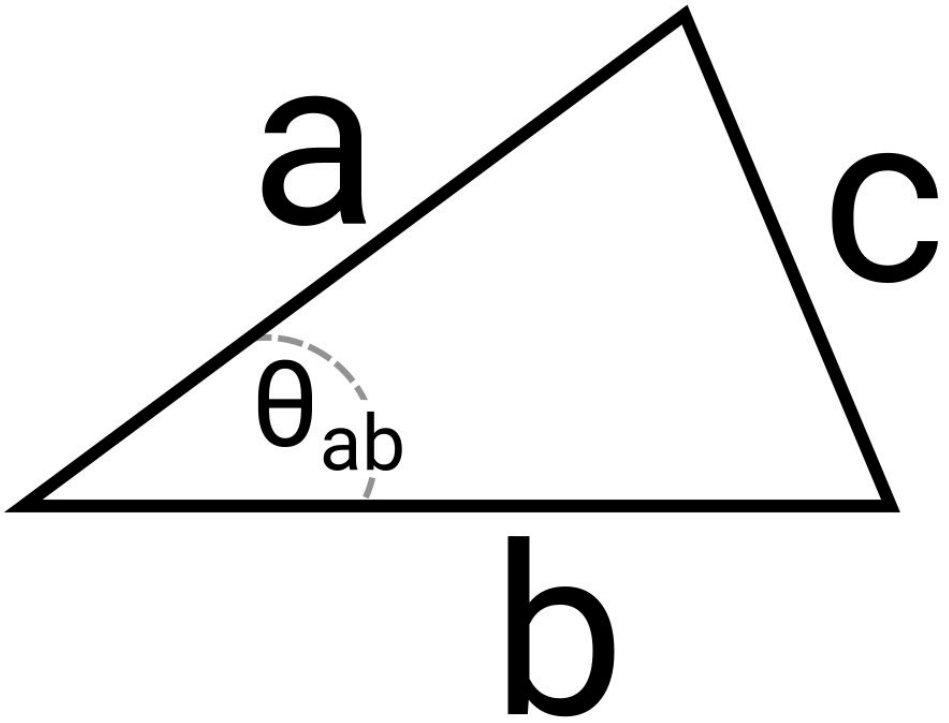
3.4 Algebraic and geometric equivalence

Were you surprised that the algebraic and geometric equations for the dot product looked so different? Itâ€™s hard to see that theyâ€™re the same, but they really are, and thatâ€™s what weâ€™re going to discover now. The two expressions are printed again below for convenience.

$$\mathbf{a}^{\mathrm{T}}\mathbf{b} = \sum_{i=1}^{n} \mathrm{a}_i \mathrm{b}_i = \|\mathbf{a}\|\|\mathbf{b}\| \cos(\theta_{ab}) \tag{3.18}$$

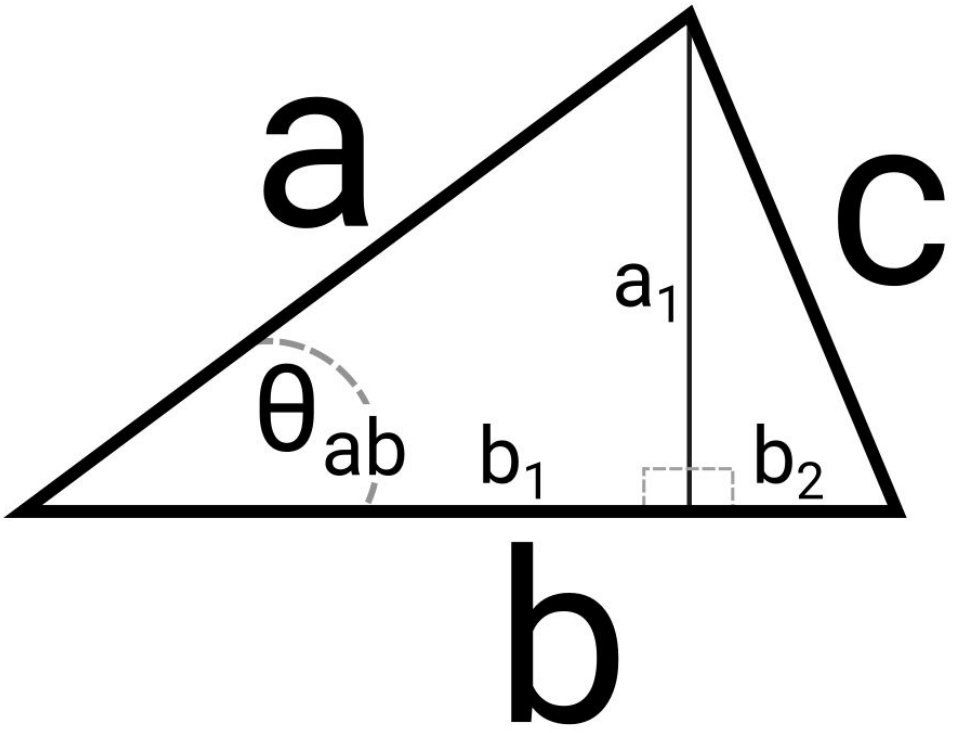
Proving the equivalence between the algebraic and geometric formulations of the dot product requires knowing that the dot product is distributive and commutative (equations 3.7 and 3.9) and knowing a formula from geometry called the Law of Cosines. Iâ€™m going to start by proving the Law of Cosines, and then youâ€™ll see how that is used to prove Equation 3.18 .

The Law of Cosines is an extension of the Pythagorean theorem ($a^2 + b^2 = c^2$) (you can also think of the Pythagorean theorem as a special case of the Law of Cosines). The Law of Cosines gives a formula for the length of side *c* , given the lengths of sides *a* and *b* and given the angle between *a* and *b* (Figure 3.3).



FigureÂ 3.3: The Law of Cosines. The goal is to find the length of *c* given lengths *a* and *b* , and the angle between them.

Now, if ð œf $_{bc}$ = 90 ãˆ , then we have a right-triangle and the Pythagorean theorem applies. But how do we solve for *c* without any right angles? The trick here is to create some right angles. In particular, we cut the triangle into two pieces, each of which is a right triangle (Figure 3.4).



FigureÂ 3.4: The Law of Cosines, part 2.

Now *c* is the hypotenuse of right-triangle a_1-b_2-*c* . So now the goal is to find a_1 and b_2. Weâ€™ll need to reach back to some high-school memories, in particular the Soh-Cah-Toa rule that relates the ratio of triangle sides to their angles. Letâ€™s start by defining a_1 in terms of *a* and the angle between *a* and *b* :

$$\sin(\eth \quad \text{æ} f_{ab}) = \frac{a_1}{a} \tag{3.19}$$

$$a_1 = a \sin(\eth \quad \text{æ} f_{ab}) \tag{3.20}$$

We can solve for b_1 in a similar way:

$$\cos(\eth \quad \text{æ} f_{ab}) = \frac{b_1}{a} \tag{3.21}$$

$$b_1 = a \cos(\eth \quad \text{æ} f_{ab}) \tag{3.22}$$

Next we need to find an expression for b_2. The trick here is to express b_2 in terms of quantities we already know.

$$b = b_1 + b_2 \tag{3.23}$$

$$b_2 = b \hat{a}\check{}\text{'} b_1 \tag{3.24}$$

$$b_2 = b \hat{a}\check{}\text{'} a \cos(\eth \quad \text{æ} f_{ab}) \tag{3.25}$$

At this point, we have created a right-triangle with side-lengths defined in terms of the original values *a* and *b* , and that means that we can apply the Pythagorean theorem on those quantities. From there, we simply work through some algebra to arrive at the Law of Cosines in Equation 3.30 :

$$c^2 = a_1^2 + b_2^2 \tag{3.26}$$

$$= (a \sin(\theta_{ab}))^2 + (b - a\cos(\theta_{ab}))^2 \quad (3.27)$$

$$= a^2 \sin^2(\theta_{ab}) + b^2 + a^2 \cos^2(\theta_{ab}) - 2ab\cos(\theta_{ab}) \quad (3.28)$$

$$= a^2(\sin^2(\theta_{ab}) + \cos^2(\theta_{ab})) + b^2 - 2ab\cos(\theta_{ab}) \quad (3.29)$$

$$= a^2 + b^2 - 2ab\cos(\theta_{ab}) \quad (3.30)$$

Recall the trig identity that $\cos^2(\theta) + \sin^2(\theta) = 1$. Notice that when $\theta_{ab} = 90^\circ$, the third term in Equation 3.30 drops out and we get the familiar Pythagorean theorem.

I realize this was a bit of a long tangent, but we need the Law of Cosines to prove the equivalence between the algebraic and geometric equations for the dot product.

Letâ€™s get back on track. Instead of thinking about the lengths of the triangles as a, b, and c, we can think about the edges of the triangles as *vectors* **a**, **b**, and **c**, and thus their lengths are $\|\mathbf{a}\|$, $\|\mathbf{b}\|$, and $\|\mathbf{c}\|$.

Therefore, we can write Equation 3.30 as

$$a^2 + b^2 - 2ab\cos(\theta_{ab}) = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\|\mathbf{a}\|\|\mathbf{b}\|\cos(\theta_{ab}) \quad (3.31)$$

Now that weâ€™re thinking of these triangle sides as vectors, we can re-write vector **c** as the subtraction of vectors **a** and **b** (see Figure 2.5): $\mathbf{c} = \mathbf{a} - \mathbf{b}$. And therefore,

$$\|\mathbf{c}\| = \|\mathbf{a} - \mathbf{b}\|$$

With that in mind, we can expand the definition of vector **c** using the distributive property:

$$\|\mathbf{a}\|^2 = (\mathbf{a} - \mathbf{b})^T(\mathbf{a} - \mathbf{b}) = \mathbf{a}^T\mathbf{a} - 2\mathbf{a}^T\mathbf{b} + \mathbf{b}^T\mathbf{b}$$
$$= \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\mathbf{a}^T\mathbf{b} \quad (3.32)$$

Weâ€™re almost done with the proof; we just need to do some simplifications. Notice that we have discovered two different ways of writing out the magnitude-squared of **c**. Letâ€™s set those equal to each other and do a bit of simplification.

$$\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\mathbf{a}^T\mathbf{b} = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\|\mathbf{a}\|\|\mathbf{b}\|\cos\theta$$

Notice that $\|\mathbf{a}\|^2$ and $\|\mathbf{b}\|^2$ appear on both sides of the equation, so these simply cancel. Same goes for the factor of -2. That leaves us with the remarkable conclusion of the equation we started with in this section, re-printed here for your viewing pleasure.

$$\mathbf{a}^T\mathbf{b} = \|\mathbf{a}\|\|\mathbf{b}\|\cos\theta \quad (3.15)$$

Whew! That was a really long proof! Iâ€™m pretty sure itâ€™s the longest one in the entire book. But it was important, because we discovered that the algebraic and geometric definitions of the dot product are merely different interpretations of the same operation.

Proof of Cauchy-Schwarz inequality I know youâ€™ve been patiently waiting for this proof. As a brief reminder of what I wrote on page Â§: The magnitude of the dot product between two vectors is less than or equal to the product of the norms of the two vectors. The proof comes from taking the absolute value of Equation 3.18:

$$|\mathbf{a}^T\mathbf{b}| = \|\mathbf{a}\|\|\mathbf{b}\||\cos(\theta_{ab})| \quad (3.33)$$

$$|\mathbf{a}^T\mathbf{b}| \leq \|\mathbf{a}\|\|\mathbf{b}\| \quad (3.34)$$

The absolute value of the cosine of an angle is between 0 and 1, and therefore, when dropping that term, it is trivial that the left-hand side cannot be larger than the right-hand side.

I also wrote that the equality holds when the two vectors form a linearly dependent set. Two co-linear vectors meet at an angle of 0 â˚ or 180 â˚ , and the absolute value of the cosines of those angles is 1.

-|*|- ***Reflection*** *There is a lot to say about the dot product. Thatâ€™s no surpriseâ€”the dot product is one of the most fundamental computational building blocks in linear algebra, statistics, and signal processing, out of which myriad algorithms in math, signal processing, graphics, and other calculations are built. A few examples to whet your appetite for later chapters and real-world applications: In statistics, the cosine of the angle between a pair of normalized vectors is called the Pearson correlation coefficient; In the Fourier transform, the magnitude of the dot product between a signal and a sine wave is the power of the signal at the frequency of the sine wave; In pattern recognition and feature-identification, when the dot product is computed between multiple templates and an image, the template with the largest-magnitude dot product identifies the pattern or feature most present in the image.* -|*|-

3.5 Linear weighted combination

The term "linear weighted combination" comes up so often in linear algebra that itâ€™s worth honoring it with its own section. You might hear other related names for this same operation, such as "linear mixture" or "weighted combination" (the "linear" part is assumed). Sometimes, the term "coefficient" is used instead of "weight."

Linear weighted combination simply means scalar-vector multiplication and addition: Take some set of vectors, multiply each vector by any scalar, and add them to produce a single vector.

Eqn. box title=**Linear weighted combination**

$$\mathbf{w} = \hat{I}»_1 \mathbf{v}_1 + \hat{I}»_2 \mathbf{v}_2 + ... + \hat{I}»_n \mathbf{v}_n \qquad (3.35)$$

It is assumed that all vectors $\mathbf{v}_i$ have the same dimensionality, otherwise the addition is invalid. The $\hat{I}»$ â€™s can be any real number, including zero.

Technically, you could rewrite Equation 3.35 for subtracting vectors, but because subtraction can be handled by setting the $\hat{I}»_i$ to be negative, itâ€™s easier to discuss linear weighted combinations in terms of addition.

An example:

$$\lambda_1 = 1,\ \lambda_2 = 2,\ \lambda_3 = -3, \quad \mathbf{v_1} = \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix},\ \mathbf{v_2} = \begin{bmatrix} -4 \\ 0 \\ -4 \end{bmatrix},\ \mathbf{v_3} = \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix}$$

$$\mathbf{w} = \lambda_1\mathbf{v_1} + \lambda_2\mathbf{v_2} + \lambda_3\mathbf{v_3} = \begin{bmatrix} -7 \\ -4 \\ -13 \end{bmatrix}$$

Code Linear weighted combinations are easy to implement, as the code below demonstrates. In Python, the data type is important; test what happens when the vectors are lists instead of numpy arrays. In Chapter 6, you will learn how to make these computations more compact using matrix-vector multiplication.

Code blockÂ 3.3: Python

```
import Â  numpy Â  as Â  np Â
l1 Â  = Â  1 Â
l2 Â  = Â  2 Â
l3 Â  = Â  -3 Â
v1 Â  = Â  np . array ([4,5,1]) Â
v2 Â  = Â  np . array ([-4,0,-4]) Â
v3 Â  = Â  np . array ([1,3,2]) Â
l1 * v1 Â  + Â  l2 * v2 Â  + Â  l3 * v3
```

Code blockÂ 3.4: MATLAB

```
l1 Â  = Â  1; Â
l2 Â  = Â  2; Â
l3 Â  = Â  -3; Â
v1 Â  = Â  [4 Â  5 Â  1]â€™; Â
v2 Â  = Â  [-4 Â  0 Â  -4]â€™; Â
v3 Â  = Â  [1 Â  3 Â  2]â€™; Â
l1 * v1 Â  + Â  l2 * v2 Â  + Â  l3 * v3
```

3.6 The outer product

The outer product is a way of combining two vectors to produce a matrix. The outer product matrix has many applications in advanced linear algebra topics, including eigendecomposition and the singular value decomposition, image compression, and multivariate signal processing. In later chapters you will learn more about the properties of the outer product, for example that it is a rank-1 matrix. But for now, just focus on the mechanics of creating an outer product matrix from two vectors.

But letâ€™s start with a bit of notation. The outer product is indicated using a notation is that initially confusingly similar to that of the dot product. In the definitions below, **v** is an M -element column vector and **w** is an N -element column vector.

$$\text{Dot product} : \mathbf{v}^T\mathbf{w} = 1 \text{ Ã— } 1$$

$$\text{Outer product} : \mathbf{v}\mathbf{w}^T = M \text{ Ã— } N$$

This notation indicates that the dot product ($\mathbf{v}^T\mathbf{w}$) is a 1 Ã— 1 array (just a single number; a scalar) whereas the outer product ($\mathbf{v}\mathbf{w}^T$) is a matrix whose sizes are defined by the number of elements in the vectors.

The vectors do not need to be the same size for the outer product, unlike with the dot product. Indeed, the dot product expression above is valid only when $M = N$, but the outer product is valid

even if M â‰ N .

I realize that this notation may seem strange. I promise it will make perfect sense when you learn about matrix multiplication. Essentially, the dot product and outer product are special cases of matrix multiplication.

Now letâ€™s talk about how to create an outer product. There are three perspectives for creating an outer product: The element perspective, the column perspective, and the row perspective. The result is the same; they are just different ways of thinking about the same operation.

Element perspective Each element i,j in the outer product matrix is the scalar multiplication between the ith element in the first vector and the jth element in the second vector. This also leads to the formula for computing the outer product.

Eqn. box title=**The outer product**

$$(\mathbf{v}\mathbf{w}^T)_{i,j} = v_i w_j \qquad (3.36)$$

Here is an example using letters instead of numbers, which helps make the formula clear.

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} \begin{bmatrix} d & e & f \end{bmatrix} = \begin{bmatrix} ad & ae & af \\ bd & be & bf \\ cd & ce & cf \end{bmatrix}$$

The element perspective lends itself well to a formula, but itâ€™s not always the best way to conceptualize the outer product.

Column perspective Each column in the outer product matrix comes from repeating the left vector ($\mathbf{v}$ using the notation above) but scaled by each element in the row vector on the right ($\mathbf{w}^T$). In other words, each column in the outer product matrix is the result of scalar-vector multiplication, where the vector is the column vector (repeated) and the scalar comes from each element of the row vector. Thus, the number of columns in the matrix equals the number of elements in the row vector. Notice that in the example below, each column of the outer product matrix is a scaled version of the left column vector.

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \begin{bmatrix} 4 & 1 & -1 \end{bmatrix} = \begin{bmatrix} 4 & 1 & -1 \\ 8 & 2 & -2 \\ 12 & 3 & -3 \\ 16 & 4 & -4 \end{bmatrix}$$

Row perspective Iâ€™m sure you can already guess how this is going to work: Form the outer product matrix one row at a time, by repeating the row vector M times (for M elements in vector $\mathbf{w}$), each time scaling the vector by each element in the column vector. In the example below, notice that each row in the outer product matrix is a scaled version of the row vector.

$$\begin{bmatrix} 4 \\ 1 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 8 & 12 & 16 \\ 1 & 2 & 3 & 4 \\ -1 & -2 & -3 & -4 \end{bmatrix}$$

If you look closely at the two examples above, youâ€™ll notice that when we swapped the order of the two vectors, the two outer product matrices look the same but with the columns and rows swapped. In fact, thatâ€™s not just a coincidence of this particular example; thatâ€™s a general property of the outer product. Itâ€™s fairly straightforward to prove that this is generally the case, but you need to learn more about matrix multiplications before getting to the proof. Iâ€™m trying to build excitement for you to stay motivated to continue with this book. I hope itâ€™s working!

Code The outer product, like the dot product, can be implemented in several ways. Here are two of them. (Notice that MATLAB row vector elements can be separated using a space or a comma, but separating elements with a semicolon would produce a column vector, in which case you wouldnâ€™t need the transpose.)

Code blockÂ 3.5: Python

```
v1 Â  = Â  np . array ([2,5,4,7]) Â
v2 Â  = Â  np . array ([4,1,0,2]) Â
op Â  = Â  np . outer ( v1 , v2 )
```

Code blockÂ 3.6: MATLAB

```
v1 Â  = Â  [2,5,4,7]â€™; Â
v2 Â  = Â  [4 Â  1 Â  0 Â  2]â€™; Â
op Â  = Â  v1 * v2 â€™;
```

Practice problems Compute the outer product of the following pairs of vectors.

a) $\begin{bmatrix} -1 \\ 1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix}^{\mathrm{T}}$

b) $\begin{bmatrix} 4 \\ 6 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix}^{\mathrm{T}}$

c) $\begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}^{\mathrm{T}}$

d) $\begin{bmatrix} 1 \\ 3 \\ 5 \\ 7 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}^{\mathrm{T}}$

Answers

a) $\begin{bmatrix} -2 & -3 \\ 2 & 3 \end{bmatrix}$

b) $\begin{bmatrix} 8 & 12 \\ 12 & 18 \end{bmatrix}$

c)

$$\begin{bmatrix} -1 & -2 & -3 \\ 0 & 0 & 0 \\ 1 & 2 & 3 \end{bmatrix}$$

d) $$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 3 & 3 & 0 \\ 0 & 5 & 5 & 0 \\ 0 & 7 & 7 & 0 \end{bmatrix}$$

3.7 Element-wise (Hadamard) vector product

This is actually the way of multiplying two vectors that you might have intuitively guessed before reading the past few sections.

Also called the Hadamard product, this operation involves multiplying each corresponding element in the two vectors. The resulting product vector therefore is the same size as the two multiplying vectors. And thus, Hadamard multiplication is defined only for two vectors that have the same number of elements.

There is no widely agreed-upon notation for Hadamard multiplication, but a circle with a dot is fairly common:

$$\mathbf{c} = \mathbf{a} \odot \mathbf{b} = \begin{bmatrix} a_1 b_1 & a_2 b_2 & \dots & a_n b_n \end{bmatrix} \tag{3.37}$$

An example:

$$\begin{bmatrix} 3 \\ 6 \\ 9 \\ 0 \end{bmatrix} \odot \begin{bmatrix} 2 \\ -2 \\ 4 \\ -4 \end{bmatrix} = \begin{bmatrix} 6 \\ -12 \\ 36 \\ 0 \end{bmatrix}$$

Element-wise vector multiplication doesnâ€™t have too many applications in abstract linear algebra. In fact, itâ€™s arguably not really a *linear algebra operation* â€”itâ€™s more like a convenient and compact way to organize a set of regular scalar multiplications. In practical applications, thatâ€™s how itâ€™s most often used: You might have two sets of variables that need to be individually multiplied, so you store them as vectors in MATLAB or Python, and then implement Hadamard multiplication.

Code There are many parts of Python and MATLAB syntax that are nearly identical. Matrix multiplication, however, is annoyinglyâ€”and confusinglyâ€”different between the languages. Notice that Python uses an asterisk (*) for element-wise multiplication whereas MATLAB uses a dot-asterisk (.*). Itâ€™s a subtle but important distinction.

Code blockÂ 3.7: Python

```
v1 Â  = Â  np . array ([2,5,4,7]) Â
v2 Â  = Â  np . array ([4,1,0,2]) Â
v3 Â  = Â  v1 Â  * Â  v2
```

Code blockÂ 3.8: MATLAB

```
v1 Â  = Â  [2,5,4,7]; Â
v2 Â  = Â  [4,1,0,2]; Â
v3 Â  = Â  v1 Â  .* Â  v2 ;
```

Practice problems Compute the element-wise product between the following pairs of vectors.

a) $\begin{bmatrix}1\\3\\2\end{bmatrix}$ âŠ™ $\begin{bmatrix}-1\\1\\-2\end{bmatrix}$

b) $\begin{bmatrix}0\\0\\0\end{bmatrix}$ âŠ™ $\begin{bmatrix}\pi\\e^5\\2\end{bmatrix}$

c) $\begin{bmatrix}1.3\\1.4\\3.2\end{bmatrix}$ âŠ™ $\begin{bmatrix}3\\5\end{bmatrix}$

Answers

a) $\begin{bmatrix}-1\\3\\-4\end{bmatrix}$

b) $\begin{bmatrix}0\\0\\0\end{bmatrix}$

c) undefined!

3.8 Cross product

The cross product is defined only for two 3-element vectors, and the result is another 3-element vector. It is commonly indicated using a multiplication symbol (Ã—).

The formula for computing the cross-product seems a bit bizarre:

$$\mathbf{a}\times\mathbf{b} = \begin{bmatrix}a_2b_3 - a_3b_2\\a_3b_1 - a_1b_3\\a_1b_2 - a_2b_1\end{bmatrix} \tag{3.38}$$

The magnitude of the cross product has a geometry-based formula, which is interesting to compare to the geometric formula for the dot product (Equation 3.15).

$$\|\mathbf{a}\times\mathbf{b}\| = \|\mathbf{a}\|\|\mathbf{b}\|\sin(\theta_{ab}) \tag{3.39}$$

The cross product is used often in geometry, for example to create a vector **c** that is orthogonal to the plane spanned by vectors **a** and **b** . It is also used in vector and multivariate calculus to compute surface integrals. However, the cross product is not really used in data analysis, statistics, machine-

learning, or signal-processing. Iâ€™m not going to discuss the cross-product again in this book; it is included here in the interest of completeness.

Practice problems Compute the cross-product between the following pairs of vectors.

a) $\begin{bmatrix}5\\3\\4\end{bmatrix}, \begin{bmatrix}-2\\1\\-1\end{bmatrix}$

b) $\begin{bmatrix}1\\1\\1\end{bmatrix}, \begin{bmatrix}2\\2\\2\end{bmatrix}$

c) $\begin{bmatrix}1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\end{bmatrix}$

d) $\begin{bmatrix}1302\\1403\end{bmatrix}, \begin{bmatrix}3\\5\end{bmatrix}$

Answers

a) $\begin{bmatrix}-7\\-3\\11\end{bmatrix}$

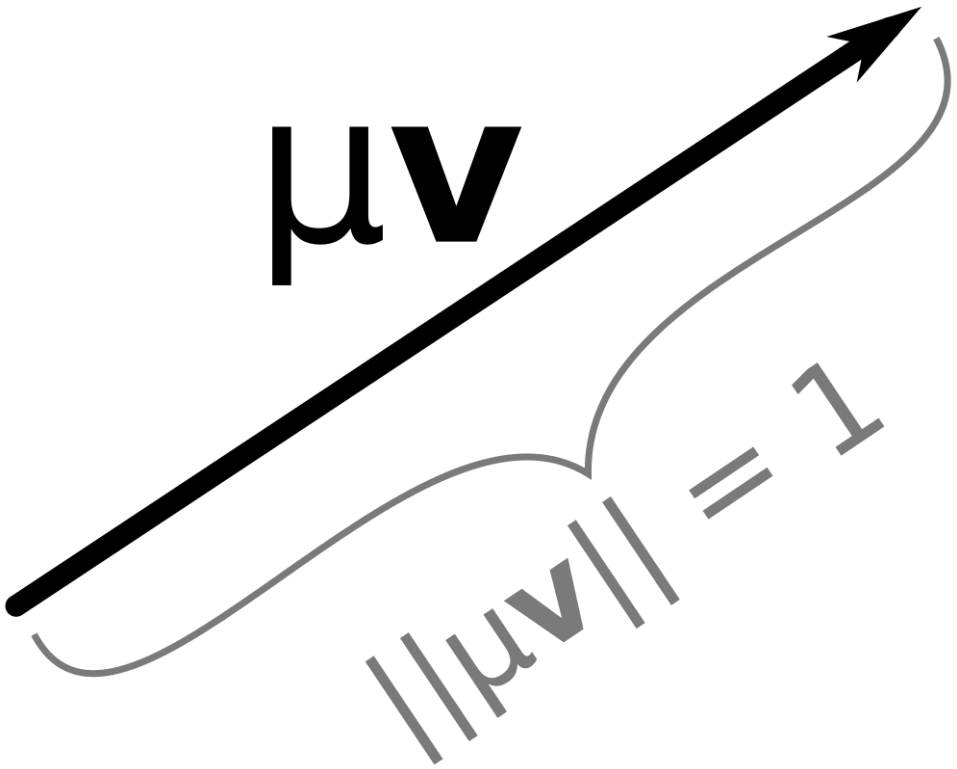
b) $\begin{bmatrix}0\\0\\0\end{bmatrix}$

c) $\begin{bmatrix}0\\0\\1\end{bmatrix}$

d) undefined.

3.9 Unit vectors

It is often useful to have a vector with a length of one: âˆ¥ **v** âˆ¥ = 1 (Figure 3.5). Unit vectors are convenient in many linear algebra applications, particularly on computers (e.g., to avoid numerical precision inaccuracies for vectors with very small magnitudes). Unit vectors also allow for creating a special kind of matrix called an orthogonal matrix.



FigureÂ 3.5: A unit vector.

Therefore, the goal of this section is to derive a formula for computing a unit vector in the same direction as some other vector **v** that is not necessarily a unit vector. The way to do this is to find some scalar *Î¼* that satisfies our criteria: [6]

[6] *s.t.* means "such that" or "subject to"

$$\mu\mathbf{v} \;\; s.t. \;\; \|\mu\mathbf{v}\| = 1 \tag{3.40}$$

How to choose *Î¼* ? Letâ€™s start by thinking about how you would create a "unit-norm number" (that is, find *Î¼* such that *Î¼* times a number has an absolute value of 1). Letâ€™s figure this out using the number 3.

$$|\, \hat{I}¼\, 3\,| \;=\; 1$$
$$\hat{I}¼ \;=\; 1\, \hat{a}\hat{}\bullet\, 3$$

Deriving the solution is simple: Divide both sides by the magnitude (that is, the absolute value) of the number (the reason for the absolute value here is that we want *Î¼* = 1 *â*ˆ• 3 even if we start with -3 instead of +3).

Now extend this concept to vectors: set *Î¼* to be the reciprocal (the inverse) of the magnitude of the vector. In terms of notation, a unit vector is commonly given a hat to indicate that it has a magnitude of one (**v** â†’ **v**). _____

Eqn. box title=**Creating a unit-length vector**

$$\mathbf{v} = \frac{1}{\|\mathbf{v}\|}\mathbf{v} = \frac{1}{\sqrt{\sum_{i=1}^{n} \mathrm{v}_i^2}}\mathbf{v} \tag{3.41}$$

_____

_____

The norm of the vector, â‹¥ **v** â‹¥ , is a scalar, which means (1) division is allowed (division by a full vector is not defined) and (2) importantly, *the direction of the vector does not change* . Here is a simple example:

$$\mathbf{v} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \qquad \hat{\mathbf{v}} = \frac{1}{\sqrt{0^2 + 2^2}} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

This example also shows why the divisor is the magnitude (the square root of sum of squared vector elements), and not the squared magnitude $\mathbf{v}^T \mathbf{v}$. It is also clear that the unit vector **v** points in the same direction as **v** .

It is worth mentioning explicitly what you might have already deduced: The unit vector is defined only for non-zero vectors. This makes sense both algebraically (Equation 3.41 would involve division by zero) and geometrically (a vector with no length has no direction, hence, another vector in the same direction is not a sensible construct).

Taking *Î¼* = 1 *â‹•* â‹¥ **v** â‹¥ allows for a quick proof that the unit vector really does have unit length:

$$\|\mu\mathbf{v}\| = \frac{1}{\|\mathbf{v}\|}\|\mathbf{v}\| = 1 \tag{3.42}$$

Code Fortunately, both Python and MATLAB have built-in functions for computing the norm of a vector.

Code blockÂ 3.9: Python

```
v Â  = Â  np . array ([2,5,4,7]) Â
vMag Â  = Â  np . linalg . norm ( v ) Â
v_unit Â  = Â  v Â  / Â  vMag
```

Code blockÂ 3.10: MATLAB

```
v Â  = Â  [2,5,4,7]; Â
vMag Â  = Â  norm ( v ); Â
v_unit Â  = Â  v Â  / Â  vMag ;
```

Practice problems Compute a unit vector in the same direction as the following vectors. Then confirm that the magnitude of the resulting vector is 1.

a) $\begin{bmatrix} 1 \\ 3 \end{bmatrix}$

b) $\begin{bmatrix} 3 \\ 4 \end{bmatrix}$

c) $\begin{bmatrix} 6 \\ -8 \end{bmatrix}$

d) $\begin{bmatrix} .1 \\ .2 \\ .4 \\ .2 \end{bmatrix}$

Answers

a) $\frac{1}{\sqrt{10}}\begin{bmatrix}1\\3\end{bmatrix}$

b) $\begin{bmatrix}3/5\\4/5\end{bmatrix}$

c) $\begin{bmatrix}3/5\\-4/5\end{bmatrix}$

d) $\frac{1}{\sqrt{.25}}\begin{bmatrix}.1\\.2\\.4\\.2\end{bmatrix} = \begin{bmatrix}.2\\.4\\.8\\.4\end{bmatrix}$

3.10 Exercises

1.Â

Compute the dot product between the following pairs of vectors.

a) $\begin{bmatrix}0\\a\end{bmatrix}, \begin{bmatrix}b\\0\end{bmatrix}$

b) $\begin{bmatrix}a\\b\end{bmatrix}, \begin{bmatrix}c\\d\end{bmatrix}$

c) $\begin{bmatrix}a\\c\end{bmatrix}, \begin{bmatrix}b\\d\end{bmatrix}$

d) $\begin{bmatrix}1\\4\end{bmatrix}, \begin{bmatrix}3\\-3\end{bmatrix}$

e) $\begin{bmatrix}1\\4\\0\end{bmatrix}, \begin{bmatrix}3\\-3\\9\end{bmatrix}$

f) $\begin{bmatrix}10\\14\\-3\end{bmatrix}, \begin{bmatrix}-1\\1\\-6\end{bmatrix}$

g) $\begin{bmatrix}1\\a\end{bmatrix}, \begin{bmatrix}b\\1\end{bmatrix}$

h) $\begin{bmatrix}2i\\4-i\\3\end{bmatrix}, \begin{bmatrix}3\\3\\1\end{bmatrix}$

i) $\begin{bmatrix}1\\2\end{bmatrix}, \begin{bmatrix}2\\-1\end{bmatrix}$

j) $\begin{bmatrix}1\\-2\\-3\\4\\5\\-6\end{bmatrix}, \begin{bmatrix}-4\\3\\4\\-5\\6\\-2\end{bmatrix}$

k)

$\begin{bmatrix} 2 \\ -5 \\ 0 \\ 0 \\ 2 \\ -8 \end{bmatrix}, \begin{bmatrix} -1 \\ 5 \\ 0 \\ -1 \\ 0 \\ -4 \end{bmatrix}$

l) $\begin{bmatrix} 2.47 \\ -2.47 \end{bmatrix}, \begin{bmatrix} \pi^3 \\ \pi^3 \end{bmatrix}$

m) $\begin{bmatrix} 2 \\ 3 \\ -3 \\ 2 \end{bmatrix}, \begin{bmatrix} 0 \\ 12 \\ -8 \\ 4 \end{bmatrix}$

n) $\begin{bmatrix} 2 \\ -1 \\ -2 \end{bmatrix}, \begin{bmatrix} -2 \\ 1 \\ -\frac{5}{2} \end{bmatrix}$

o) $\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} a \\ b \end{bmatrix}$

2.Â

Assume that â^¥ **x** â^¥ = â^¥ **y** â^¥ = 1 . Determine whether each of the following equations is necessarily true, necessarily false, or could be true depending on the elements in **x** and **y** .

a) (**x** â^' **y**) ^{T} (**x** â^' **y**) = 2(1 â^' **x** ^{T} **y**)

b) â^¥ (**x** â^' $\frac{1}{2}$ **y**) â^¥ ^{2} = $\frac{5}{4}$ â^' **x** ^{T} **y**

c) $\frac{\mathbf{x}^{T}\mathbf{y}}{\mathbf{x}^{T}\mathbf{x}}$ â^' $\frac{\mathbf{y}^{T}\mathbf{y}}{\mathbf{x}^{T}\mathbf{y}\mathbf{y}^{T}\mathbf{y}}$ = 0

d) â^¥ **y** + $\frac{2}{3}$ **x** â^¥ ^{2} = $\frac{4}{9}$ (1 + 4 **x** ^{T} **y**)

e) **x** ^{T} **x** + **x** ^{T} **x** â^' 2 **y** ^{T} **y** = **x** ^{T} **y**

f) $\frac{\frac{1}{2}\mathbf{x}^{T}\mathbf{x}\frac{1}{2}\mathbf{y}^{T}\mathbf{y}}{\mathbf{y}^{T}(\frac{1}{4}\mathbf{y})} + \frac{\|\mathbf{x}\|^{2}}{\|\mathbf{x}\|^{2}\|\mathbf{y}\|^{3}}$ â^' y^Tyy^Ty$\frac{}{\mathbf{x}^{T}\mathbf{x}}$ =1

3.Â

Compute the angle $\delta \quad \text{œ} f$ between the following pairs of vectors. See if you can do with without a calculator!

a) $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix}$

b) $\begin{bmatrix} 10 \\ 12 \\ 4 \end{bmatrix}, \begin{bmatrix} 2.5 \\ 3 \\ 1 \end{bmatrix}$

c) $\begin{bmatrix} -1 \\ -2 \\ 3 \end{bmatrix}, \begin{bmatrix} 3 \\ 6 \\ -9 \end{bmatrix}$

4.Â

Compute the outer product between the following pairs of vectors. Implement problems a-c element-wise (the *i,j* th element in the product matrix is the product of the *i* th element in the left

vector and the jth element in the right vector); problems d-f row-wise (each row in the product matrix is the right-vector scaled by each element in the left vector); and problems g-i column-wise (each column in the product matrix is the left-vector scaled by each element in the right-vector).

a) $\begin{bmatrix}1\\2\end{bmatrix}\begin{bmatrix}1\\2\end{bmatrix}^{\mathrm{T}}$

b) $\begin{bmatrix}-1\\3\\0\end{bmatrix}\begin{bmatrix}0\\1\\3\end{bmatrix}^{\mathrm{T}}$

c) $\begin{bmatrix}1\\0\\0\end{bmatrix}\begin{bmatrix}0\\1\\0\end{bmatrix}^{\mathrm{T}}$

d) $\begin{bmatrix}1\\2\\3\\4\end{bmatrix}\begin{bmatrix}5\\6\\7\\8\end{bmatrix}^{\mathrm{T}}$

e) $\begin{bmatrix}1\\2\\3\\4\end{bmatrix}\begin{bmatrix}10\\20\\30\\40\end{bmatrix}^{\mathrm{T}}$

f) $\begin{bmatrix}1\\1\\2\\2\end{bmatrix}\begin{bmatrix}a\\b\\c\\d\end{bmatrix}^{\mathrm{T}}$

g) $\begin{bmatrix}4\\2\\3\\1\end{bmatrix}\begin{bmatrix}6\\7\\5\end{bmatrix}^{\mathrm{T}}$

h) $\begin{bmatrix}3\\40\end{bmatrix}\begin{bmatrix}10\\2\\30\\4\end{bmatrix}^{\mathrm{T}}$

i) $\begin{bmatrix}a\\b\\c\\d\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}1\\1\\2\\2\end{bmatrix}$

5.Â

Determine whether the following vectors are unit vectors.

a) $\frac{1}{\sqrt{3}}\begin{bmatrix}1\\1\\1\end{bmatrix}$

b) $\begin{bmatrix}.2\\.64\\.1\\.4\end{bmatrix}$

c) $\begin{bmatrix} 3/5 \\ 0 \\ 0 \\ .8 \end{bmatrix}$

d) $\frac{1}{\sqrt{90}} \begin{bmatrix} 1 \\ 3 \\ 5 \\ 2 \\ 4 \\ 6 \end{bmatrix}$

6.Â
What is the magnitude of vector *Î¼* **v** for the following *Î¼* ?

- **a)** *Î¼* = 0
- **b)** *Î¼* = â^¥ **v** â^¥
- **c)** *Î¼* = 1 *â^•* â^¥ **v** â^¥
- **d)** *Î¼* = 1 *â^•* â^¥ **v** â^¥ 2

3.11 Answers

1.Â

- **a)** 0
- **b)** *ac* + *bd*
- **c)** *ab* + *cd*
- **d)** â^’ 9
- **e)** â^’ 9
- **f)** 22
- **g)** *a* + *b*
- **h)** 15 + 3 *i*
- **i)** 0
- **j)** 0
- **k)** 5
- **l)** 0
- **m)** 68
- **n)** 0
- **o)** 0

2.Â

- **a)** True
- **b)** True
- **c)** Depends
- **d)** False
- **e)** Depends
- **f)** True

3.Â

These you can solve by inspecting the elements of the vectors, without applying the dot product formula.

- **a)** ð œf = Ï€âˆ• 2 (90 â˚)
- **b)** ð œf = 0
- **c)** ð œf = Ï€ (180 â˚)

4.Â
Of course, the answers are the same regardless of the perspective you use; the goal is to become comfortable with different ways of thinking about the outer product.

- **a)** $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$
- **b)** $\begin{bmatrix} 0 & -1 & -3 \\ 0 & 3 & 9 \\ 0 & 0 & 0 \end{bmatrix}$
- **c)** $\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$
- **d)** $\begin{bmatrix} 5 & 6 & 7 & 8 \\ 10 & 12 & 14 & 16 \\ 15 & 18 & 21 & 24 \\ 20 & 24 & 28 & 32 \end{bmatrix}$
- **e)** $\begin{bmatrix} 10 & 20 & 30 & 40 \\ 20 & 40 & 60 & 80 \\ 30 & 60 & 90 & 120 \\ 40 & 80 & 120 & 160 \end{bmatrix}$
- **f)** $\begin{bmatrix} a & b & c & d \\ a & b & c & d \\ 2\,a & 2\,b & 2\,c & 2\,d \\ 2\,a & 2\,b & 2\,c & 2\,d \end{bmatrix}$
- **g)** $\begin{bmatrix} 24 & 28 & 20 \\ 12 & 14 & 10 \\ 18 & 21 & 15 \\ 6 & 7 & 5 \end{bmatrix}$
- **h)** $\begin{bmatrix} 30 & 400 \\ 6 & 80 \\ 90 & 1200 \\ 12 & 160 \end{bmatrix}^{\mathrm{T}}$
- **i)** $\begin{bmatrix} a & a & 2a & 2a \\ b & b & 2b & 2b \\ c & c & 2c & 2c \\ d & d & 2d & 2d \end{bmatrix}$

5.Â

- **a)** Yes

b) No
c) Yes
d) No

6.Â

a) 0
b) â^¥ **v** â^¥ ^{2}
c) 1
d) 1 $\hat{a}$^• â^¥ **v** â^¥

3.12 Code challenges

1.Â

Create a linear weighted combination of three vectors. Youâ€™ll learn in later chapters that linear combinations are most efficiently represented as matrix-vector multiplication, but weâ€™ll keep things simple for now. Create three 5-D vectors and a fourth vector that contains the weights for each vector. Then create the weighted sum of those vectors. Next, modify the code to compute the weighted mixture of four 5-D vectors. What is the relationship between the dimensionality of the to-be-summed vectors, the number of vectors to sum, and the dimensionality of the coefficients vector?

2.Â

Develop a method to use the dot product to compute the average of a set of numbers in a vector. (Hint: consider the vector of all ones, sometimes written **1** .)

3.Â

What if some numbers were more important than other numbers? Modify your answer to the previous question to devise a method to use the dot product to compute a weighted mean of a set of numbers.

3.13 Code solutions

1.Â

Only the basic code solutions are presented here. Keep in mind that these challenges are also about critical thinking, not just writing a few lines of code.

Code blockÂ 3.11: Python

```
Â  Â  Â  Â  v1 Â  = Â  np . array ([1,2,3,4,5]) Â
Â  Â  Â  Â  v2 Â  = Â  np . array ([2,3,4,5,6]) Â
Â  Â  Â  Â  v3 Â  = Â  np . array ([3,4,5,6,7]) Â
Â  Â  Â  Â  w Â  = Â  [-1,3,-2] Â
Â  Â  Â  Â  result Â  = Â  v1 * w [0] Â  + Â  v2 * w [1] Â  + Â  v3 * w [2]
```

Code blockÂ 3.12: MATLAB

```
Â  Â  Â  Â  v1 Â  = Â  [1,2,3,4,5]; Â
Â  Â  Â  Â  v2 Â  = Â  [2,3,4,5,6]; Â
Â  Â  Â  Â  v3 Â  = Â  [3,4,5,6,7]; Â
Â  Â  Â  Â  w Â  = Â  [-1 Â  3 Â  -2]; Â
Â  Â  Â  Â  result Â  = Â  v1 * w (1) Â  + Â  v2 * w (2) Â  + Â  v3 * w (3)
```

2.Â

The trick is to compute the average of N numbers by summing them together after multiplying by 1/N. Mathematically, you would write this as $\mathbf{v}^T \mathbf{1} \frac{1}{N}$.

Code blockÂ 3.13: Python

```
Â  Â  Â  Â  Â  Â  Â  Â  Â  v Â  = Â  [ Â  7, Â  4, Â  -5, Â  8, Â  3 Â  ] Â
Â  Â  Â  Â  Â  Â  Â  Â  Â  o Â  = Â  np . ones ( len ( v )) Â
Â  Â  Â  Â  Â  Â  Â  Â  Â  ave Â  = Â  np . dot ( v , o ) Â  / Â  len ( v )
```

Code blockÂ 3.14: MATLAB

```
Â  Â  Â  Â  Â  Â  Â  Â  Â  v Â  = Â  [ Â  7 Â  4 Â  -5 Â  8 Â  3 Â  ]â€™; Â
Â  Â  Â  Â  Â  Â  Â  Â  Â  o Â  = Â  ones ( size ( v )); Â
Â  Â  Â  Â  Â  Â  Â  Â  Â  ave Â  = Â  dot ( v , o ) Â  / Â  length ( v )
```

3.Â

We donâ€™t know which numbers are more important than others, so I will use randomized weights. In fact, the solution here is simply the the dot product between the data vector and any other vector that sums to one.

Code blockÂ 3.15: Python

```
Â  Â  Â  Â  Â  Â  Â  Â  Â  v Â  = Â  [ Â  7, Â  4, Â  -5, Â  8, Â  3 Â  ] Â
Â  Â  Â  Â  Â  Â  Â  Â  Â  w Â  = Â  np . random . rand ( len ( v )) Â
Â  Â  Â  Â  Â  Â  Â  Â  Â  wAve Â  = Â  np . dot ( v , Â  w / sum ( w ))
```

Code blockÂ 3.16: MATLAB

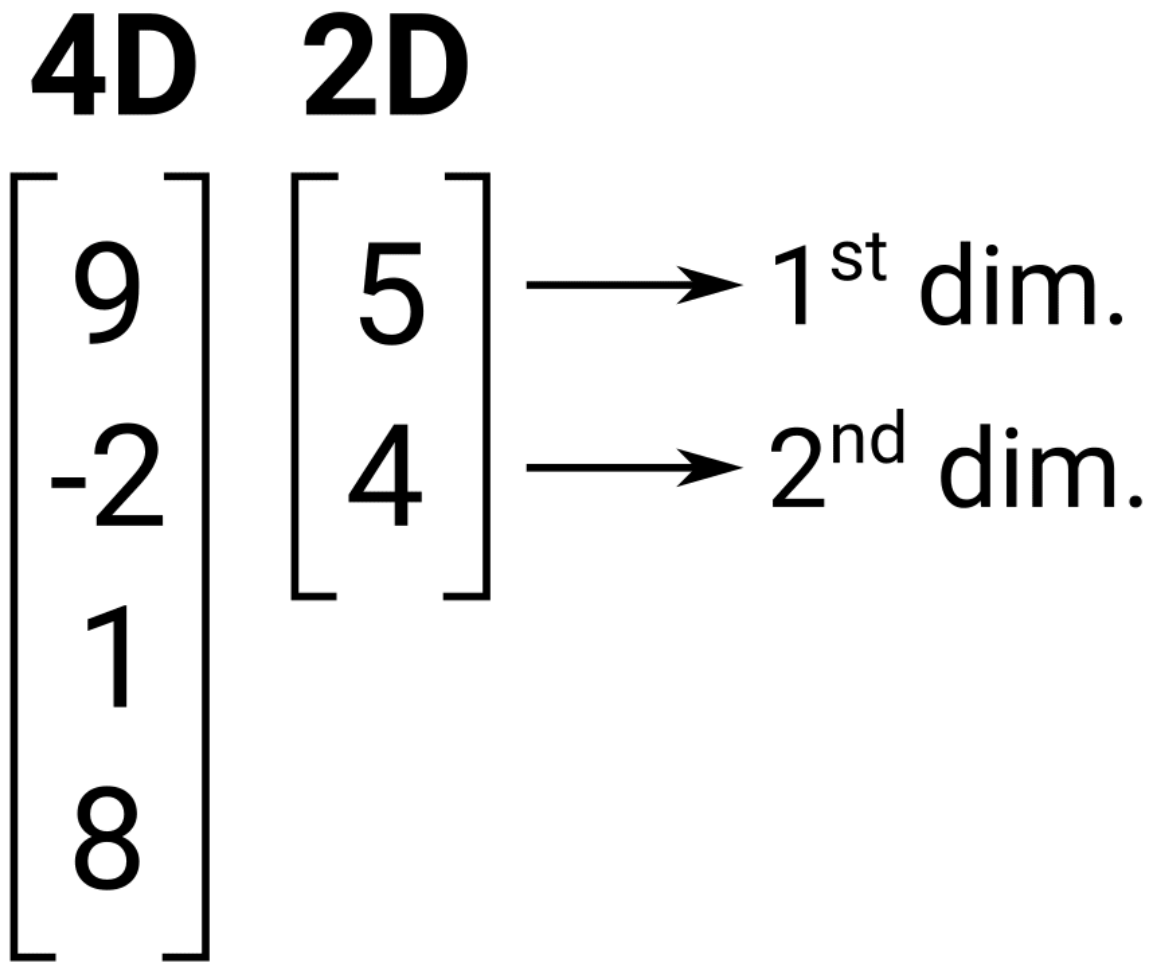
```
Â  Â  Â  Â  Â  Â  Â  Â  Â  v Â  = Â  [ Â  7, Â  4, Â  -5, Â  8, Â  3 Â  ]; Â
Â  Â  Â  Â  Â  Â  Â  Â  Â  w Â  = Â  rand ( size ( v )); Â
Â  Â  Â  Â  Â  Â  Â  Â  Â  wAve Â  = Â  dot ( v , Â  w / sum ( w ))
```

ChapterÂ 4
Vector spaces

4.1 Dimensions and fields in linear algebra

Dimension I provided a brief definition of the term "dimension" in the previous chapterâ€”the number of elements in a vectorâ€”but given the importance of this concept for understanding vector/matrix subspaces, systems, and decompositions, itâ€™s worth expanding on the concept of dimension in more detail.

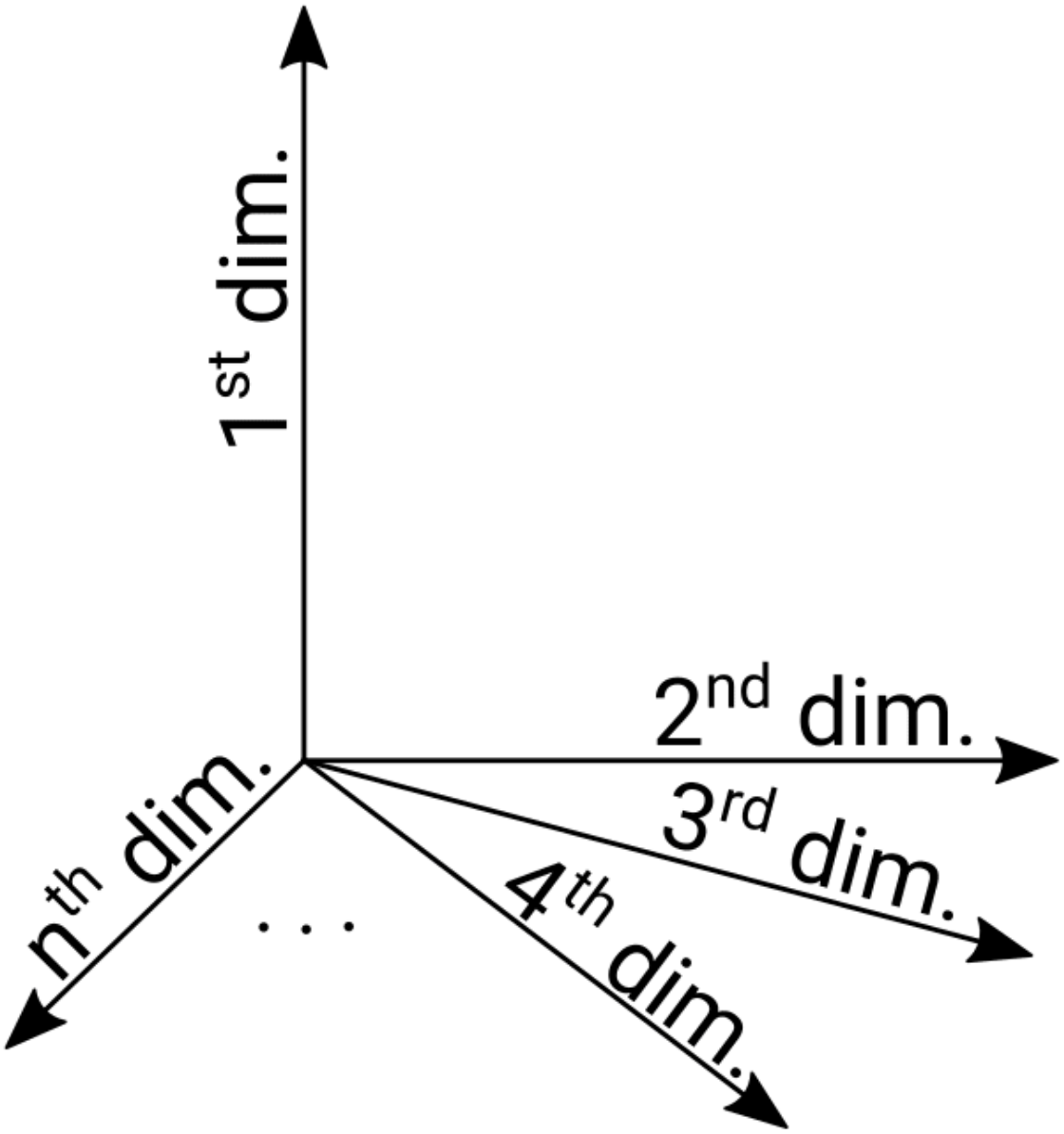
Algebraically , the dimensionality of a vector is simply the number of elements in the vector. The order is meaningful: In the vector [3 1 5], the element in the second dimension is 1, and the number 5 is in the third dimension (Figure 4.1). On the other hand, there is no hierarchical relationship amongst the dimensions â€” the first dimension isnâ€™t intrinsically more important than the second dimension.



FigureÂ 4.1: Each position in a vector is a dimension, and the element in that position is that value along that dimension.

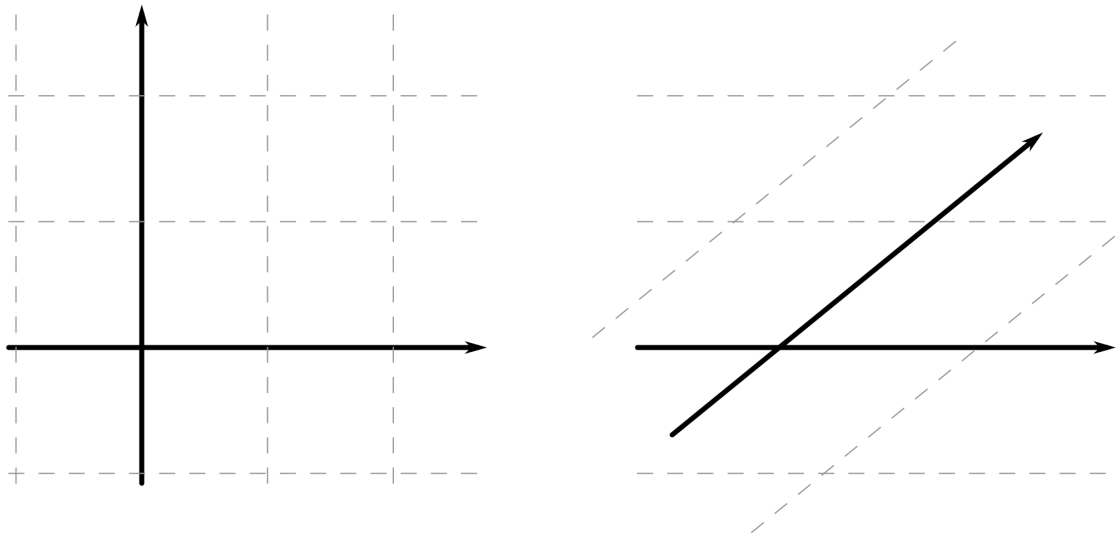
Geometrically , the dimensionality of a vector is the number of coordinate axes in which that vector exists. A 2D vector is a line in a 2D space (think of the typical Cartesian XY plane); a 3D vector is a line in a 3D space. Note that both a 2D vector and a 3D vector are both *lines* , but they exist in a different number of *dimensions* . Higher dimensional spaces are often indicated in the style of Figure 4.2 .

As you learned in the previous chapter, the value of each vector element tells you how far along that coordinate axis to travel. This is easiest to conceptualize for vectors in their standard position, because the endpoint of the vector is the same as the coordinate. For example, the vector [1 3] in standard position is a line that goes from the origin to the coordinate point (1,3). Hence: two elements, two coordinate axes.



FigureÂ 4.2: Depicting a high-dimensional space on 2D paper.

A major departure from the typical Cartesian axis youâ€™re familiar withâ€”and a beautiful feature of linear algebraâ€”is that the coordinate axes need not be orthogonal; that is, they need not meet at right angles (Figure 4.3). Orthogonal axes have several useful properties, but non-orthogonal axes also have many useful properties, and are key to many applications, including data compression. More on this in the section on Bases.



FigureÂ 4.3: The familiar Cartesian plane (left) has orthogonal coordinate axes. However, axes in linear algebra are not constrained to be orthogonal (right), and non-orthogonal axes can be advantageous.

Fields The term "field" in mathematics might be new to you. Perhaps youâ€™ve seen these fancy capital letters like â„ , â„‚ , â„š , or â„¤ . These letters (typeface *blackboard bold* and sometimes called "hollow letters") refer to fields. A field in mathematics is a set of numbers for which the basic arithmetic operations (addition, subtraction, multiplication, division) are defined.

The â„ stands for real, as in, real numbers. In Chapter 9, weâ€™ll use the field â„‚ for complex numbers. Thereâ€™s also the field â„š for rational numbers (a rational number is not a number that makes good decisions; itâ€™s one that is defined by a ratio $\frac{p}{q}$).

In this book, weâ€™ll use fields mainly to indicate dimensionality. For example, $\mathbb{R}^2$ is the set of all real two-dimensional vectors, and $\mathbb{R}^4$ means a 4-element vector of the form $\begin{bmatrix} a & b & c & d \end{bmatrix}$, where *a* , *b* , *c* , and *d* are any real numbers.

Sometimes, the exact dimensionality doesnâ€™t matter but the relative dimensionalities do. For example, if I told you that vector **w** is in $\mathbb{R}^M$ and **v** is in $\mathbb{R}^N$, then you would know that the dot product $\mathbf{w}^T\mathbf{v}$ is defined only if $M = N$.

When taking notes by hand, fields are often written as R^2, and when typing on a computer, fields are often indicated as R2 or R^6. _____

Practice problems Identify the *N* in â„ ^{N} for the following vectors.

a) $\begin{bmatrix} 1 & -3 & 4 & 2 \end{bmatrix}$

b) $\begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix}$

c) $\begin{bmatrix} \pi & e^e \end{bmatrix}$

d) 17

Answers

a) â„ ^{4}

b) â„ ^{3}

c) â„ ^{2}

d) â„ ^{1}

_____

_

4.2 Vector spaces

A *vector space* refers to any set of objects for which addition and scalar multiplication are defined.[1] Addition and scalar multiplication obey the following axioms; these should all be sensible requirements based on your knowledge of arithmetic:

[1] An *axiom* is a statement that is taken to be true without requiring a formal proof.

Additive inverse : $\mathbf{v} + (\hat{a}\hat{\ }' \mathbf{v}) = \mathbf{0}$

Associativity : $(\mathbf{v} + \mathbf{w}) + \mathbf{u} = \mathbf{v} + (\mathbf{w} + \mathbf{u})$

Commutativity : $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$

Additive identity : $\mathbf{v} + \mathbf{0} = \mathbf{v}$

Multiplicative identity : $\mathbf{v}\, 1 = \mathbf{v}$

Distributivity : $(\hat{I}\pm + \hat{I}^2)(\mathbf{v} + \mathbf{w}) = \hat{I}\pm\, \mathbf{v} + \hat{I}\pm\, \mathbf{w} + \hat{I}^2\, \mathbf{v} + \hat{I}^2\, \mathbf{w}$

Vector spaces make little appearance in this book outside this section. Instead, vector *sub* spaces are much more prominent. In fact, I bring up vector spaces here

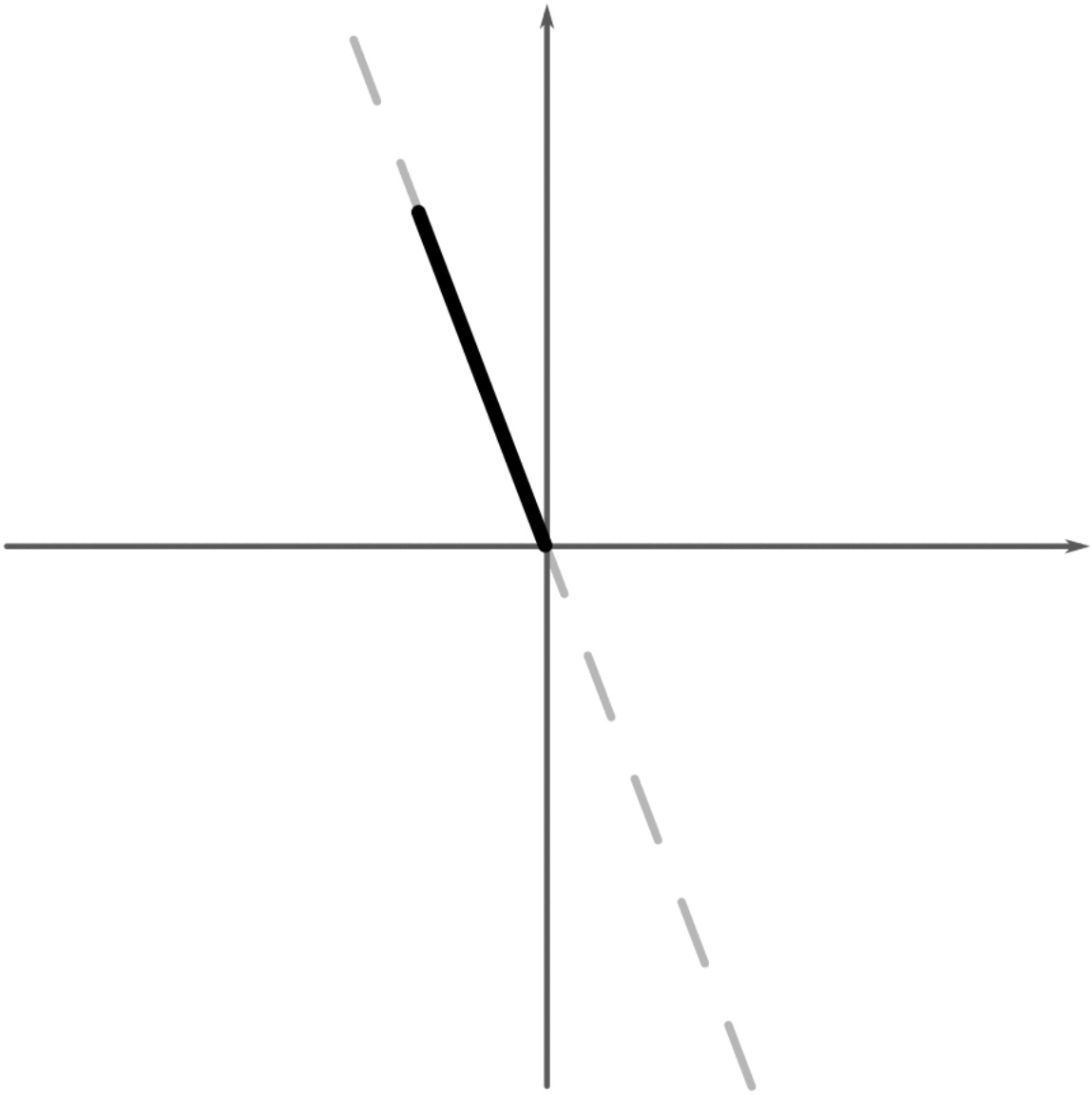
only so they are not confused with vector *sub* spaces. Three letters can make a big difference, as people struggling to finish their PhD can confirm.

4.3 Subspaces and ambient spaces

Vector subspaces are central to nearly all advanced concepts in, and applications of, linear algebra. Fortunately, the concept of a subspace is not so complicated, and it has geometric and algebraic explanations that I hope will become intuitive after some exposure and a few examples.

Geometry A subspace is the set of all points that you can reach by stretching and combining a collection of vectors (that is, addition and scalar multiplication).

Letâ€™s start with a simple example of the vector **v** =[-1 2]. In its standard position, this is a line from the origin to the coordinate (-1,2). This vector on its own is not a subspace. However, consider the set of all possible vectors that can be obtained by *Î»* **v** for the infinity of possible real-valued *Î»* â€™s, ranging from â^'â^ž to + â^ž : That set describes an infinitely long line *in the same direction* as **v** , and is depicted in Figure 4.4 (showing the entire subspace would require an infinitely long page).



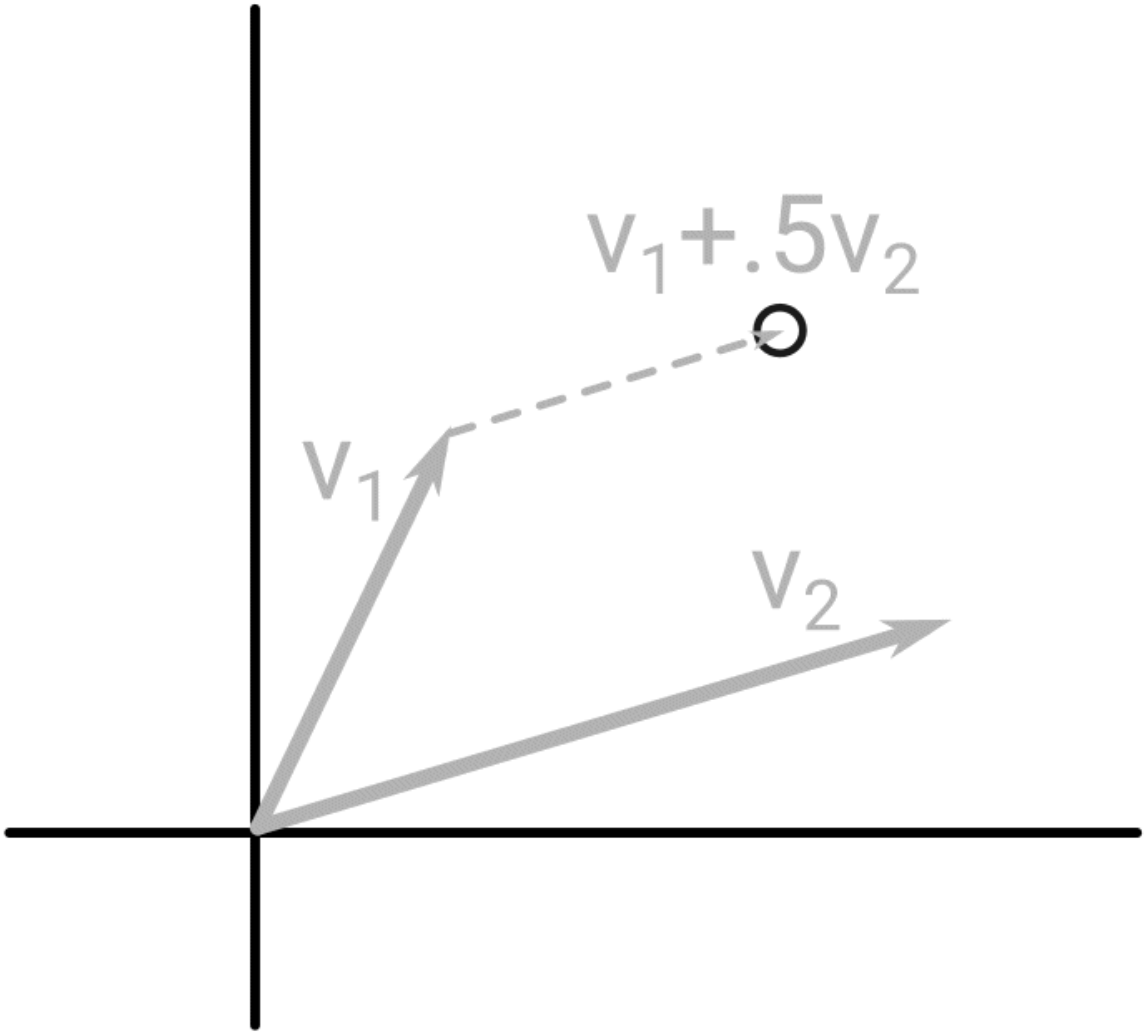
FigureÂ 4.4: A 1D subspace (gray dashed line) created from a vector (solid black line).

That gray dashed line is the set of all points that you can reach by scaling and combining all vectors in our collection (in this case, itâ€™s a collection of one vector). That gray line extends infinitely far in both directions, although the vector **v** is finite.

This is the important sense in which *Î»* **v** does not change the "direction" of **v** , which was mentioned in a previous chapter.

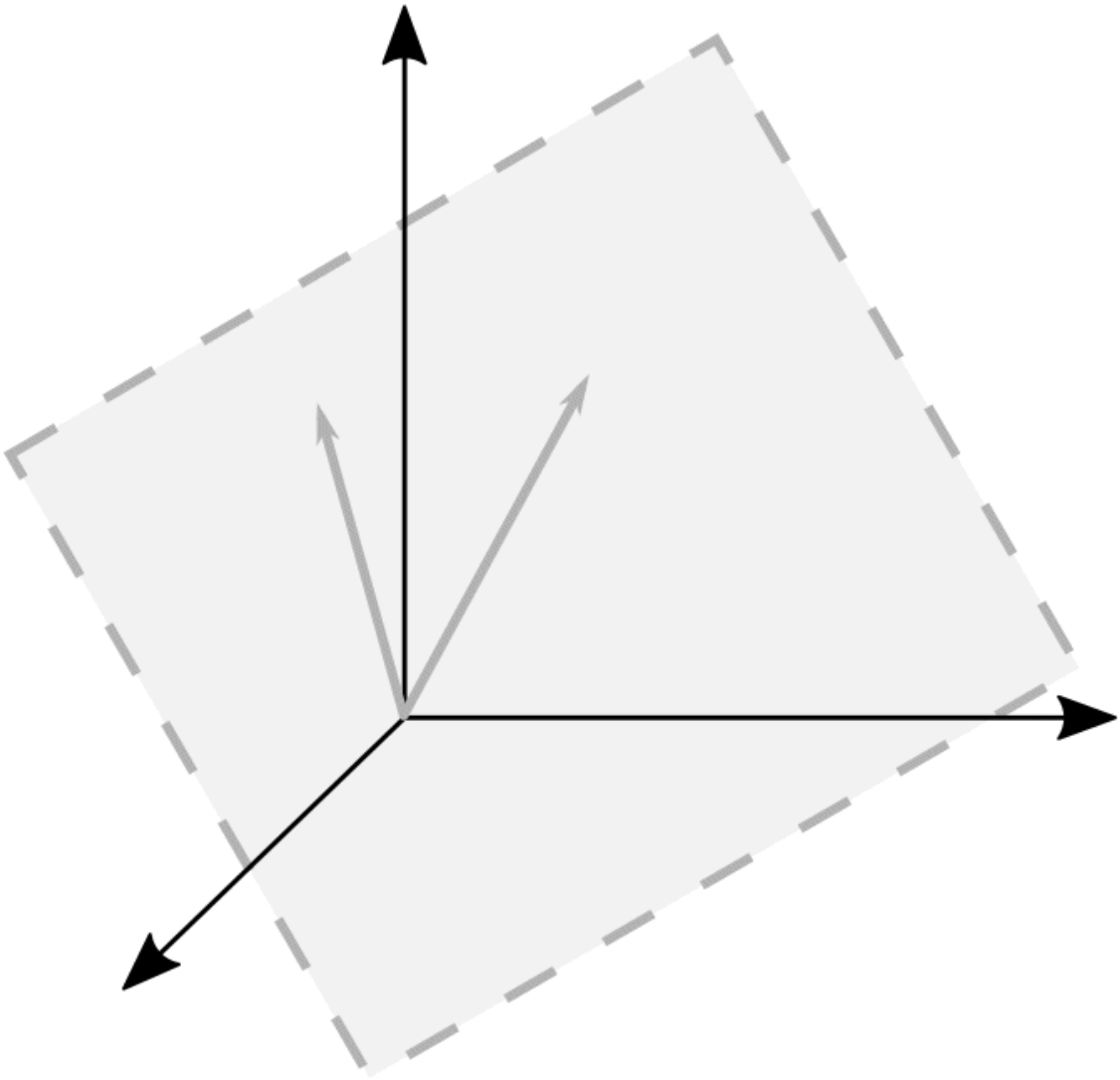
So the subspace obtained from one vector is an infinitely long line. What happens when you have two vectors? They each individually have an associated infinitely long 1D subspace. But the definition of a vector subspace allows us to combine these vectors. And you learned in the previous chapter that adding two vectors geometrically gives a third vector that can point in a different direction compared to the two adding vectors.

So by scaling and adding two vectors, we can reach many points that are not within the 1D subspaces defined by either vector alone. Figure 4.5 shows an example of combining two vectors to reach a point that could not be reached by either vectorâ€™s subspace alone.



FigureÂ 4.5: The linear combination of two vectors can reach a point that is not within the 1D subspaces defined by either vector alone.

In fact, the set of *all* points reachable by scaling and adding two vectors (that is, the linear weighted combination of those two vectors) creates a new 2D subspace, which is a *plane* that extends infinitely far in all directions of that plane. Figure 4.6 shows how this looks in 3D: The subspace created from two vectors is a plane. Any point in the plane can be reached by some linear combination of the two grey vectors.



FigureÂ 4.6: A 2D subspace in a 3D ambient space, created by all possible weighted linear combinations of two vectors (gray lines). The dashed border indicates that the plane extends to âˆž in all directions.

Ambient dimensionality Figure 4.6 also shows the distinction between a *subspace* and the *ambient space* in which that subspace is embedded.[2] In this

case, the subspace is 2-dimensional, and it is embedded in an ambient 3-dimensional space. The ambient dimensionality is the dimensionality of the vectorsâ€”the *N* in â„ ^{N} .

[2] Note that in *linear* algebra we deal only with *linear* dimensions.

So, how many subspaces are possible within an ambient space? The answer is infinity, for any ambient dimensionality more than 1.

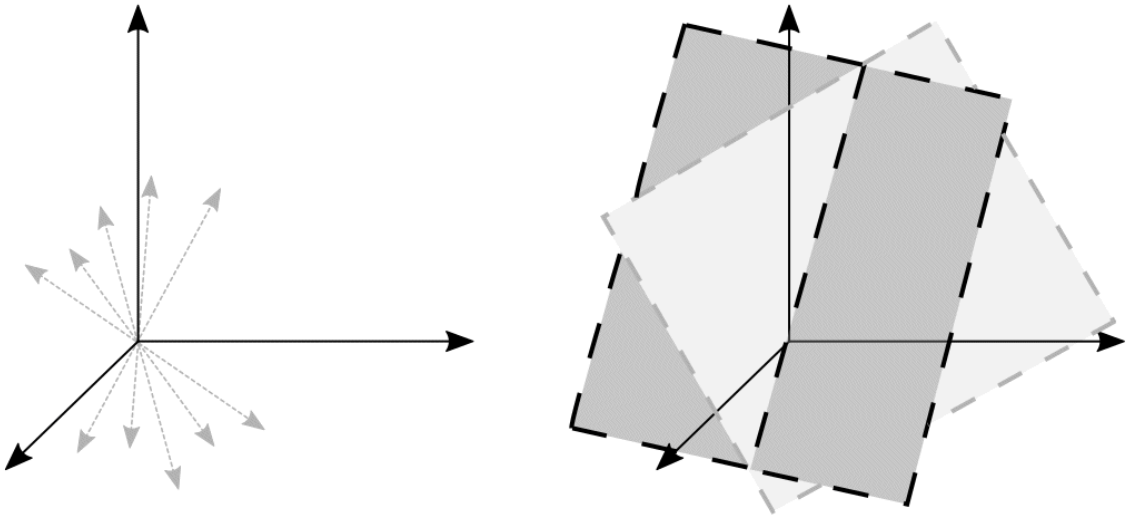
But let me unpack that answer by working through the answer to a slightly different question: How many *subspace dimensionalities* are possible with an N-dimensional ambient space? That answer is a finite number, and it turns out to be N+1. Let us count the dimensionalities, using â„ ^{3} as a visualizable example.

The smallest possible subspace is the one defined by the zeros vector [0 0 0]. This vector is at the origin of the ambient 3D space, and any scalar $\hat{I}»$ times this vector is still at the origin. This is the only subspace that is a single point, and it is thus a zero-dimensional subspace.

Next, we have 1-dimensional subspaces, which are defined as the scalar multiples of any non-zeros vector. Basically, any line in 3D can give rise to a 1D subspace that is embedded in the ambient 3 dimensions, as long as that line goes through the origin (any subspace must contain the origin for the case $\hat{I}» = 0$). How many such 1D subspaces are there? The answer is the number of unique lines in 3D that pass through the origin, which is infinity: there is an infinite number of lines in 3D that pass through the origin.

Now for the 2-dimensional subspaces. These are formed by taking all linear combinations of two vectorsâ€”two linesâ€”in 3D. These vectors themselves donâ€™t need to pass through the origin, but the plane that is formed by combining all scaled versions of these vectors must include the origin (Figure 4.7). It is also intuitive that there is an infinite number of 2D planes in 3D ambient space that pass through the origin.

But wait a minuteâ€”will *any* two vectors form a plane? No, the vectors must be distinct from each other. This should make sense intuitively: two vectors that lie on the same line cannot define a unique plane. In a later section, Iâ€™ll define this concept as *linear independence* and provide a formal explanation; for now, try to use your visual intuition and high-school geometry knowledge to understand that a unique plane can be defined only from two vectors that are different from each other.



FigureÂ 4.7: Left: five (grey lines) of the infinity of possible 1D subspaces embedded in an ambient 3D space. Right: two of the infinity of possible 2D subspaces in the same ambient space. Remember that all subspaces must pass through the origin.

Finally, we have the 3D subspace defined by all linear combinations of three vectors. As with the plane case, the three vectors that span this 3D subspace must point in different directions from each other, otherwise you could form only a plane or a line. How many distinct 3D subspaces are possible in a 3D ambient space? Exactly one: the entire ambient space. The subspace already is the entire ambient space, so there is no way to get an extra, unique, subspace. (Of course, there are different sets of vectors that you could use to create this subspace; more on this in the section on Bases!)

So there you have it: For an N-dimensional ambient space, there are N+1 possible dimensions for subspaces (0 through N), and an infinite number of possible subspaces, except for the one 0-dimensional subspace and the one N-dimensional subspace.

-|*|- ***Reflection*** *The visualization gets a bit hairy after three dimensions. In a 4D ambient space, there is an infinite number of unique 3D subspaces. Each of those 3D subspaces is like a cube that extends infinitely in all directions and yet is somehow still only an infinitely thin slice of the 4D space. I can make this work in my mind by thinking about time as the 4th dimension: There is a single instant in time in which an infinitely expansive space exists, but for all of time before and after, that space doesnâ€™t exist (and yes, I am aware that time had a beginning*

and might have an end; itâ€™s just a visualization trick, not a perfect analogy). Now take a moment to try to visualize what an 18D subspace embedded in an ambient â„ [96] *"looks like." You can understand why we need the algebraic perspective to prevent overcrowding at psychiatric institutions...* -|*|-

Algebra Except for a bit of additional notation (possibly funny-looking, if you are not used to proofs-based math texts), youâ€™ll see that the algebraic definition is the same as the geometric definition, but it might feel more rigorous because there are equations.

A subspace is the set of all points that can be created by all possible linear combinations of vector-scalar multiplication for a given set of vectors in â„ [N] and all scalars in â„ . Subspaces are often indicated using italicized upper-case letters, for example: the subspace V . That same notation is also used to indicate a set of vectors. Youâ€™ll need to infer the correct reference from context.

In words, a subspace is the set of all points that satisfies the following conditions:

- Closed under addition and scalar multiplication. [3]
- Contains the zeros vector **0** .

The first condition means that for some vector **v** â^ V (a vector **v** contained in vector subspace V), multiplying **v** by any scalar $\hat{I}»$ and/or adding any other scaled vector $\hat{I}±$ **w** that is also contained inside the vector subspace V produces a new vector that remains in that same subspace.

[3] "Closed" means that what happens in the subspace, stays in the subspace.

In math terms, that first definition for a subspace translates to:

_____

_

Eqn. box title=**Algebraic definition of a subspace**

$$\hat{a}€ \mathbf{v}, \mathbf{w} \hat{a}^ V, \hat{a}€ \hat{I}», \hat{I}± \hat{a}^ \hat{a}„ ; \hat{I}» \mathbf{v} + \hat{I}± \mathbf{w} \hat{a}^ V \quad (4.1)$$

_____

_____

Read outloud, this statement would be "for any vectors **v** and **w** contained in the vector subspace V , and for any real-valued scalars $\hat{I}»$ and $\hat{I}±$, any linearly weighted combination of **v** and **w** is still inside vector subspace V ." Below is an

example of a 1D subspace *V* defined by all linear combinations of a row vector in â„ [3] .[4]

[4] Read outloud: "The subspace *V* is defined as the set of all real-valued scalar multiples of the row vector [1 3 4]."

$$V = \{\lambda \begin{bmatrix} 1 & 3 & 4 \end{bmatrix}, \lambda \in \mathbb{R}\} \tag{4.2}$$

Note that this example is not a line in â„ [1] ; it is a 1D subspace embedded in a 3D ambient space â„ [3] . Now consider the following specific vectors.

$$\begin{bmatrix} 3 & 9 & 12 \end{bmatrix} \in V$$
$$\begin{bmatrix} -2 & -6 & -8 \end{bmatrix} \in V$$
$$\begin{bmatrix} 1 & 3 & 5 \end{bmatrix} \notin V$$

The first two vectors are contained in the subspace *V* . Algebraically thatâ€™s the case because you can find some *Î»* such that *Î»* [1 3 4] = [3 9 12]; same for the second vector. Geometrically, the first two vectors are collinear with the original vector [1 3 4]; theyâ€™re on the same infinitely long time, just scaled by some factor.

That third vector, however, is *not* contained in the subspace *V* because there is no possible *Î»* that can multiply the vector to produce [1 3 5]. Geometrically, that vector points in some direction that is different from the subspace *V* .

Letâ€™s consider another example: The set of all points in â„ [2] with non-negative y-values. Is this a vector subspace? It contains the point [0,0], which is a requirement. However, you can find some point in the set (e.g., **v** =[2,3]) and some scalar *Î»* (e.g., -1) such that *Î»* **v** is *outside the set* . Thus, this set is *not* closed under scalar multiplication, and it is therefore not a subspace. In fact, this an example of a *subset* .

4.4 Subsets

In linear algebra, *subspace* and *subset* are two entirely different concepts, but they are easily confused because of the similarity of the names. Subsets are actually not important for the linear algebra topics covered in this book. But it is important to be able to distinguish subsets from subspaces.

A *subset* , in contrast to a subspace, is a region in space that can have boundaries instead of extending to infinity, and it need not include the origin. Examples

include a quadrant of the 2D Cartesian plane, a collection of equally spaced points around some center coordinate, and sphere in 3D. A few additional examples:

- The set of all points on the XY plane such that $x > 0$ and $y > 0$.
- The set of all points such that $4 > x > 2$ and $y > x^2$.
- The set of all points such that $y = 4x$, for x ranging from âˆ’âˆž to + âˆž .

These are all valid sub*sets* . The third example is also a sub*space* , because the definition of that set is consistent with the definition of a subspace: an infinitely long line that passes through the origin.

Practice problems Identify whether the following subsets are also subspaces. If they are not subspaces, find a vector in the set and a scalar such that $\hat{I}$» **v** is outside the set.

a) All points in â„ ^{2}
b) All points on the line $y = 2x + .1$
c) Points satisfying $x^2 + y^2 + z^2 = 1$
d) All points on the line $y = 2x + 0$

Answers

a) subspace and subset
b) subset
c) subset (doesnâ€™t contain the origin!)
d) subspace and subset

4.5 Span

Geometry Span is a really similar concept as subspace, and they are easy to confuse. A subspace is the region of ambient space that can be reached by any linear combination of a set of vectors. And then those vectors *span* that subspace. You can think about the difference using grammar: a subspace is a noun and span is a verb. A set of vectors spans, and the result of their spanning is a subspace.

For example, the subspace defined as all of â„ ^{2} can be created by the span of the vectors [0 1] and [1 0]. That is to say, all of â„ ^{2} can be reached by some linear weighted combination of those two vectors.

Another example: The vector [0 1] spans a 1D subspace that is embedded inside â„ ^{2} (not â„ ^{1} ! Itâ€™s in â„ ^{2} because the vector has two elements). The vector [1 2] also spans a 1D subspace, but itâ€™s a different 1D subspace from that spanned by [0 1].

Algebra The span of a set of vectors is the set of all points that can be obtained by any linear weighted combination of those vectors (Equation 4.3).

Eqn. box title=**Algebraic definition of span**

$$span\left(\{\mathbf{v}_1, \hat{a}\epsilon|, \mathbf{v}_n\}\right) = \{\hat{I}\pm_1 \mathbf{v}_1 + \hat{a}\epsilon| + \hat{I}\pm_n \mathbf{v}_n, \hat{I}\pm \hat{a}\hat{} \hat{a}\} \quad (4.3)$$

You will recognize the right-hand-side of this equation as the definition of linear weighted combination, introduced in Equation 3.35 .

Span comes up often in discussions of matrix spaces, least-squares fitting, and eigendecomposition. In Chapter 8, for example, you will learn that something called the "column space" of a matrix is essentially the subspace spanned by the columns of that matrix.

In the span? A frequent question in linear algebra is whether one vector is "in the span" of another vector or set of vectors. This is just some fancy math-speak for asking whether you can create some vector **w** by scalar-multiplying and adding vectors from set *S* .

Thus, a vector **w** is in the span of the vector set *S* if **w** can be exactly created by some linear combination of vectors in set *S* . For example, consider the following two vectors **w** and **v** and set *S* . The question at hand is whether either of these vectors is in the span of *S* . The answers are given and justified below, but see if you can work through the problem on your own before reading further.

$$\mathbf{v} = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}, \ \mathbf{w} = \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix}. \quad S = \left\{ \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 7 \\ 0 \end{bmatrix} \right\}$$

Letâ€™s start with vector **v** . We have a positive answer here: **v** is in the span of *S* . Written formally:

$$\mathbf{v} \in span(S) \text{ because } \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} = \frac{5}{6} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} + \frac{1}{6} \begin{bmatrix} 1 \\ 7 \\ 0 \end{bmatrix}$$

Thus,

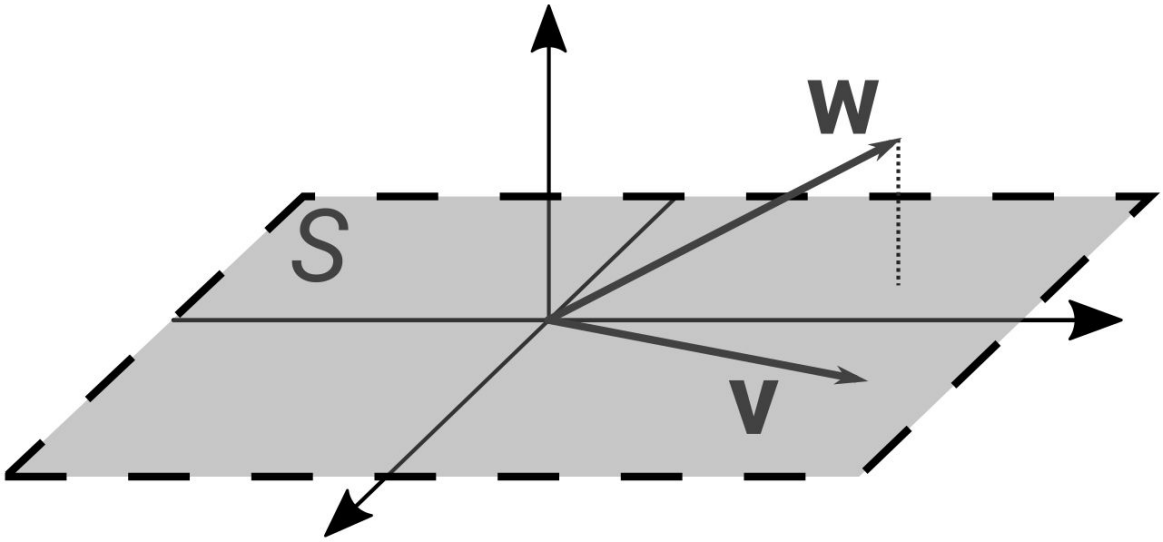
v
can be obtained by a weighted combination of vectors in set

S
. The weightings (in this case, 5/6 and 1/6) might not be immediately obvious just from looking at the vectors. In fact, it is generally not trivial to find the correct weights just by visual inspection, unless you have a really nice linear algebra teacher who gives you easy problems. There are algorithms for determining whether a vector is in the span of some vector set, and, if so, for finding the correct weights. Those algorithms are not too complicated, but they rely on concepts that you havenâ€™t yet learned about (primarily: determinant and Gaussian elimination). So weâ€™ll come back to those algorithms later.

For now, itâ€™s important to understand the concept that the span of a set of vectors is the entire subspace that can be reached by any linear combination of those vectors (often verbalized as "this vector set spans that subspace"), and that we often want to determine whether a given vector is contained in the span of that set.

In contrast to the difficulty of finding whether **v** was in the span of *S* , it should be obvious why **w** is *not* in the span of *S* : The third element in **w** is nonzero, whereas all vectors in set *S* have a third element of zero. There is no linear combination of zeros that will produce a nonzero value, therefore **w** cannot possibly be in the span of *S* .

There is a geometric interpretation of this example (Figure 4.8): The span of *S* is a 2D plane embedded in a 3D ambient space; vector **v** is a line in the plane, whereas vector **w** pops out of that plane. We could also describe **v** as being coplanar to set *S* .



FigureÂ 4.8: The span of set *S* is a 2D plane at z=0. Vector **v** is in the span of that set, and thus lies in that plane. Vector **w** is outside the span, which means it points out of the plane.

One thing to keep in mind about span: It doesnâ€™t matter if the vectors in the set are linear combinations of other vectors in that same set. For example, the following set spans a 2D subspace (a plane) embedded in ambient â„ ^{4}. There are five vectors, but notice that the first three and the last two are collinear. Hence, five vectors in total, but together they span a 2D subspace.

$$\left\{ \begin{bmatrix} 1 \\ 2 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} -3 \\ -6 \\ 0 \\ -3 \end{bmatrix}, \begin{bmatrix} 10 \\ 20 \\ 0 \\ 10 \end{bmatrix}, \begin{bmatrix} 0 \\ 4 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 2 \\ .5 \\ 0 \end{bmatrix} \right\}$$

How do I know that this set spans only a 2D subspace? Well, I know because Iâ€™ve already read the chapter on matrix rank and so I know the algorithm for determining the subspace dimensionality. As I wrote in the section on determining whether a vector is in the span of a set, it is, in most cases, impossible to look at a set of vectors and "see" the subspace dimensionality. You will learn several algorithms for computing this, but for now, focus on the concept that it is possible for a set to contain five 4D vectors that together span a 2D subspace.

-|*|- ***Reflection*** *Span makes me think of a robot holding a laser pointer in a dark room, and each new vector in the set is a degree of freedom of movement, like an extra joint in the robotâ€™s arm. With one vector the robot can shine the light*

only in one fixed direction. With two vectors the robot can swivel its arm and illuminate a plane. Three vectors means the robot can move its arm in all directions and can follow a mosquito around the room so I can finally kill it. The analogy kindof breaks down in higher dimensions. Maybe there are quantum curled-up mosquitos and we need better robots to hunt them down. **-|*|-**

_

Practice problems Determine whether each vector is in the span of the associated set.

a) $\mathbf{v} = \begin{bmatrix} \pi^3 \\ \ln(e^3) \end{bmatrix}$, $\mathbf{w} = \begin{bmatrix} -\sqrt{2} \\ 1 \end{bmatrix}$. $S = \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\}$

b) $\mathbf{p} = \begin{bmatrix} -5 \\ 5 \\ 25 \end{bmatrix}$, $\mathbf{q} = \begin{bmatrix} 5 \\ 5 \\ 25 \end{bmatrix}$. $T = \left\{ \begin{bmatrix} -1 \\ 0 \\ 4 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right\}$

c) $\mathbf{m} = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}$, $\mathbf{x} = \begin{bmatrix} 3 \\ 2 \\ 4 \\ 0 \end{bmatrix}$. $U = \left\{ \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right\}$

Answers

a) both
b) **p** yes, **q** no
c) Invalid dimensions

_____

4.6 Linear independence

[5] Linear independence (sometimes simplified to "independence"; formally, linear algebra is concerned only with *linear* independence), like many concepts introduced in these first three chapters, is central to many theoretical and applied topics in linear algebra, ranging from matrix rank to statistics to the singular value decomposition.

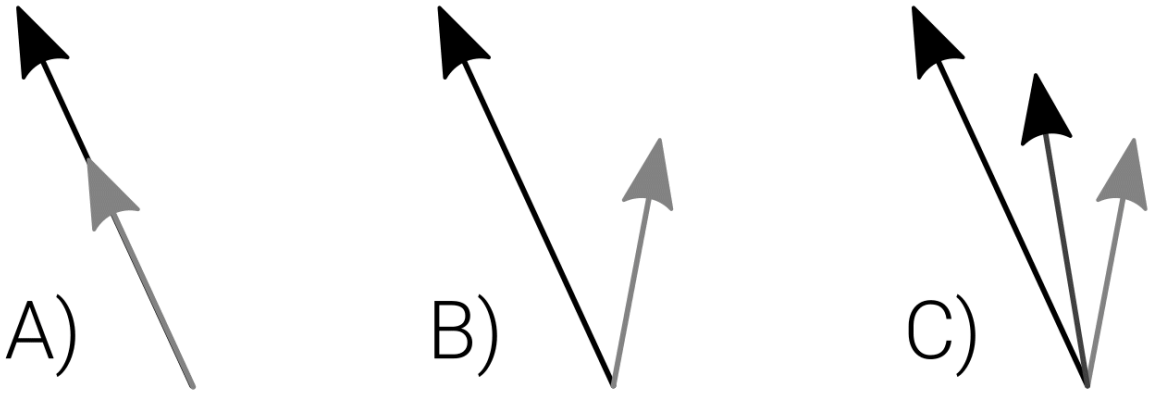
[5] Some authors abbreviate linear independence as L.I.; I will refrain from this indulgence to avoid overburdening you with abbreviations to memorize.

One important thing to know about linear independence before reading the rest of this section is that independence is a property of a *set* of vectors. That is, a set of vectors can be linearly independent or linearly dependent; it doesnâ€™t make sense to ask whether a single vector, or a vector within a set, is independent. You will learn why this is the case over the next few pages.

Geometry A set of vectors is independent [6] if the dimensionality of the subspace spanned by that set of vectors is equal to the number of vectors in that set. For example, a set with one vector spans a line (assuming it is not the zeros vector) and is always an independent set (1 vector, 1 dimension); a linearly independent set of two vectors spans a plane (2 vectors, 2 dimensions); an independent set with three vectors spans a 3D space (3 vectors, 3 dimensions).

[6] A set of vectors is independent if the subspace dimensionality equals the number of vectors.

But having two vectors in a set doesnâ€™t necessarily endow that set with linear independence. Consider the sets of vectors in Figure 4.9 ; assume these are all 2D vectors. The left-hand set (panel A) contains two vectors that are collinear; this set is linearly *de* pendent because you can create one vector as a scaled version of another vector. Now consider the middle set (panel B): Again, two vectors, but they point in different directions; it is not possible to create one vector as a scaled version of the other vector.



FigureÂ 4.9: Geometric sets of vectors in â„ [2] illustrating (in)dependence. The set in panel A is dependent because there are two vectors but they span a 1D subspace. The set in panel B is independent because the two vectors span a 2D subspace. The set in panel C is dependent because three vectors cannot span more than a 2D subspace in â„ [2] .

Finally, consider the right-hand set (panel C): This set of three vectors in â„ [2] is linearly dependent, because any one of the vectors can be obtained by a linear

combination of the other two vectors. In this example, the middle vector can be obtained by averaging the other two vectors (that is, summing them and scalar-multiplying by $\hat{I}\!\!\!

linear (in)dependence is a property of a set of vectors, not of individual vectors. You could just as easily isolate $\mathbf{v}_1$ or $\mathbf{v}_3$ on the left-hand side of Equation 4.5 .

The important point is that it is possible to create any one vector in the set as some linear weighted combination of the other vectors in the same set.

On the other hand, it is possible to create subsets of those sets that are linearly independent. For example, the sets $\{\mathbf{v}_2, \mathbf{v}_3\}$, $\{\mathbf{v}_1, \mathbf{v}_3\}$, and $\{\mathbf{v}_1, \mathbf{v}_2\}$ are all independent sets.

The next two examples show *independent* sets.

$$\{\mathbf{w}_1, \mathbf{w}_2\} = \left\{ \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 4 \\ 7 \end{bmatrix} \right\}$$

$$\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\} = \left\{ \begin{bmatrix} 0 \\ 2 \\ 5 \end{bmatrix}, \begin{bmatrix} -27 \\ 0 \\ -37 \end{bmatrix}, \begin{bmatrix} 3 \\ 1 \\ 9 \end{bmatrix} \right\}$$

Try as hard as you can and for as long as you like; you will never be able to define any one vector in each set using a linear weighted combination of the other vectors in the same set. Thatâ€™s easy to see in the first set: When considering only the first two rows, then $\mathbf{w}_1 = 2\,\mathbf{w}_2$. However, this weighted combination fails for the third row. Mapping this back onto the geometric perspective, these two vectors are two separate lines in $\hat{a}_{\!\!\!

_

Eqn. box title=**Linear dependence**

$$\mathbf{0} = \hat{I}\text{»}_1 \mathbf{v}_1 + \hat{I}\text{»}_2 \mathbf{v}_2 + \ldots + \hat{I}\text{»}_n \mathbf{v}_n, \hat{I}\text{»} \hat{a}\text{€}^{\text{TM}}\text{s} \quad (4.6)$$

---

0 (sometimes written 0) indicates the zeros vector. Of course, if all $\hat{I}$» â€™s are zero, then equation 4.6 is trivially true, so we require that at least one $\hat{I}\text{»}\hat{a}\text{‰}\ 0$. If equation 4.6 is *not* true for at least one $\hat{I}\text{»}\hat{a}\text{‰}\ 0$, then the set is linearly *in* dependent.

This may seem like a strange definition: Where does it come from and why is the zeros vector so important? Some rearranging, starting with subtracting $\hat{I}\text{»}_1 \mathbf{v}_1$ from both sides of the equation, will reveal why this equation indicates dependence:

$$\hat{I}\text{»}_1 \mathbf{v}_1 = \hat{I}\text{»}_2 \mathbf{v}_2 + \ldots + \hat{I}\text{»}_n \mathbf{v}_n$$

$$\mathbf{v}_1 = \frac{\lambda_2}{\lambda_1}\mathbf{v}_2 + \ldots + \frac{\lambda_n}{\lambda_1}\mathbf{v}_n, \hat{I}\text{»} \hat{a}\text{€}^{\text{TM}}\text{s} \quad , \hat{I}\text{»}_1 \hat{a}\text{‰}\ 0 \quad (4.7)$$

Because the $\hat{I}$» â€™s are scalars, then $\hat{I}\text{»}_n \hat{a}\text{•}\hat{I}\text{»}_1$ is also just some scalar. If you like, you could replace all the fractional constants with some other constant, e.g., $\hat{I}^2{}_n = \hat{I}\text{»}_n \hat{a}\text{•}\hat{I}\text{»}_1$.

The point is that with a bit of re-arranging, equation 4.6 states that one vector can be expressed as some linear weighted combination of the other vectors in the set. Now you can see the correspondence between the math equation and the English definition of linear dependence. You can also see another justification for the claim that at least one $\hat{I}\text{»}\hat{a}\text{‰}\ 0$.

Equation 4.6 also reveals an interesting property of linear dependence: Any set of vectors that contains the zeros vector is guaranteed to be linearly dependent. Why is that the case?

Imagine that $\mathbf{v}_1 = \mathbf{0}$, $\hat{I}\text{»}_1 = 1000$, and all other $\hat{I}\text{»}_n$ =0. Then the right-hand side of the equation equals the left-hand side of the equation for at least one $\hat{I}\text{»}\hat{a}\text{‰}\ 0$. Thus, any set of vectors that contains the zeros vector is a dependent set.

Determining whether a set is linearly dependent or independent Before learning about matrix-based algorithms for computing whether a set is linear independent, you can apply a 4-step procedure to determine the linear dependency of a set of vectors. Note that this is a strategy for solving exercise problems or exam problems by hand; this method does not scale up to larger matrices, and itâ€™s way too time-consuming to use in any real-world applications. Nonetheless, this procedure is useful to understand to help you internalize the concept of linear independence.

Step 1:Â

Count the number of vectors (call that number M) in the set and compare to N in â„ ^{N} . As mentioned earlier, if $M > N$, then the set is necessarily dependent. If M â‰¤ N then you have to move on to step 2.

Step 2:Â

Check for a vector of all zeros. Any set that contains the zeros vector is a dependent set.

Step 3:Â

If youâ€™ve gotten this far, it means you need to start doing some trial-and-error educated guesswork. Start by looking for zeros in the entries of some vectors, with the knowledge that zeros in some vectors in combination with non-zero entries in corresponding dimensions in other vectors is a tip towards independence (you cannot create something from nothing, with the possible exception of the big bang).

Step 4:Â

This is where the real educated guesswork comes in. Start by creating one element as a weighted combination of other vectors, and see whether that same weighted combination will work for the other dimensions. Consider the following set of vectors:

$$\left\{ \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}, \begin{bmatrix} 4 \\ 5 \\ 8 \end{bmatrix} \right\}$$

Look across the first dimension (top row of vector elements) and notice that 2 times the first vector plus 1 times the second vector produces the first element in the third vector. That same set of weights is also true for the second dimension. However, it doesnâ€™t work for the third dimension. That proves that the third vector cannot be expressed as this linear combination of the first two. You have to repeat this process (try to find weights of M-1 vector entries that equal the M th vector entry) for each of M

vectors. Eventually, you will determine that the set is linearly independent or dependent.

Here is another, slightly different, way to think about step 4, which brings back the formal definition of linear dependence, and also will help prepare you for matrix concepts like null space, inverse, and eigendecomposition: The goal is to come up with a set of coefficients for each vector such that the weighted sum of all vectors gives the zeros vector. For the first dimension, the coefficients (-2, -1, 1) will produce a zero (âˆ’ 2 Ã— 1 + âˆ’ 1 Ã— 2 + 1 Ã— 4 = 0). Those same coefficients will also work for the second dimension, but they donâ€™t work for the third dimension. This means either (1) a different set of coefficients could work for all three dimensions, or (2) the set is linearly independent.

Step 4 is really the limiting factor here. Again, by this point in the book, your main concern should be with the concept and definition of linear independence; scalable and rigorous methods will be introduced in later chapters.

-|*|- ***Reflection*** *Consistent terminology is important in any branch of knowledge, and confusing or redundant terminology can hinder progress and create confusion. Unfortunately, the term independence has different meanings in different areas of mathematics. In statistics, two variables are "independent" if the correlation coefficient between them is zero. In linear algebra, that case would be called "orthogonal." What is called "independent" in linear algebra would be called "correlated" in statistics. There is no easy solution to terminological confusions; you just have to be flexible and make sure to clarify your terms when talking to an audience from a different background.* -|*|-

---

Practice problems Determine whether each set is linearly dependent or independent.
If the set is dependent, come up with a set of coefficients that illustrates their dependence.

a) $\left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} \right\}$

b) $\left\{ \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} \right\}$

c)

d) $\left\{ \begin{bmatrix} 0 \\ 7 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 6 \end{bmatrix}, \begin{bmatrix} -2 \\ 17 \\ -6 \end{bmatrix} \right\}$

$\left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 2.000001 \end{bmatrix} \right\}$

Answers

a) Dependent (1,0,0)
b) Dependent (1,-3,1)
c) Dependent (-3, 2, 1)
d) Independent

4.7 Basis

A basis is the combination of span and independence: A set of vectors { $\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_n$ } forms a *basis* for some subspace of â„ ^{N} if it (1) spans that subspace and (2) is an independent set of vectors.

Geometrically, a basis is like a ruler for a space. The basis vectors tell you the fundamental units (length and direction) to measure the space they describe. For example, the most common basis set is the familiar Cartesian axis basis vectors, which contains only 0s and 1s:

$\mathbb{R}^2 : \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\}, \quad \mathbb{R}^3 : \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\}$

[7] These are called the "standard" basis vector sets.

This basis set is so widely used because of its simplicity: Each basis vector has unit length and all vectors in the set are mutually orthogonal (that is, the dot product of any vector with any other vector is zero). These sets fulfill the definition of basis because they (1) are linearly independent sets that (2) span â„ ^{2} or â„ ^{3}.

However, it is not necessary to span all of â„ ^{N} to be a basis. The two examples below are basis sets for subspaces embedded in â„ ^{2} and â„ ^{3}.

$$S_1 = \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\}, \qquad S_2 = \left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \right\}$$

Set S_1 is an independent set but does not cover all of â„ ^{2} â€”instead, it spans a 1D subspace of â„ ^{2}, which is the line at Y=0. Set S_2 is a basis for a plane on the XY axes where Z=0.

Now consider the following sets of vectors; which of these forms a basis for â„ ^{3}? How about â„ ^{2}?

$$M_1 = \left\{ \begin{bmatrix} -3 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 13 \\ 0 \\ 0 \end{bmatrix} \right\}$$

$$M_2 = \left\{ \begin{bmatrix} -4 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} \pi \\ 3 \\ 0 \end{bmatrix}, \begin{bmatrix} 3 \\ 2\pi \\ 3 \end{bmatrix} \right\}$$

$$M_3 = \left\{ \begin{bmatrix} -3 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 13 \\ 0 \\ 1 \end{bmatrix} \right\}$$

The answers are that set M_2 is a basis for â„ ^{3} because there are three vectors in â„ ^{3} that form a linearly independent set.

None of the sets is a basis of â„ ^{2} because all of these vectors live in â„ ^{3}.

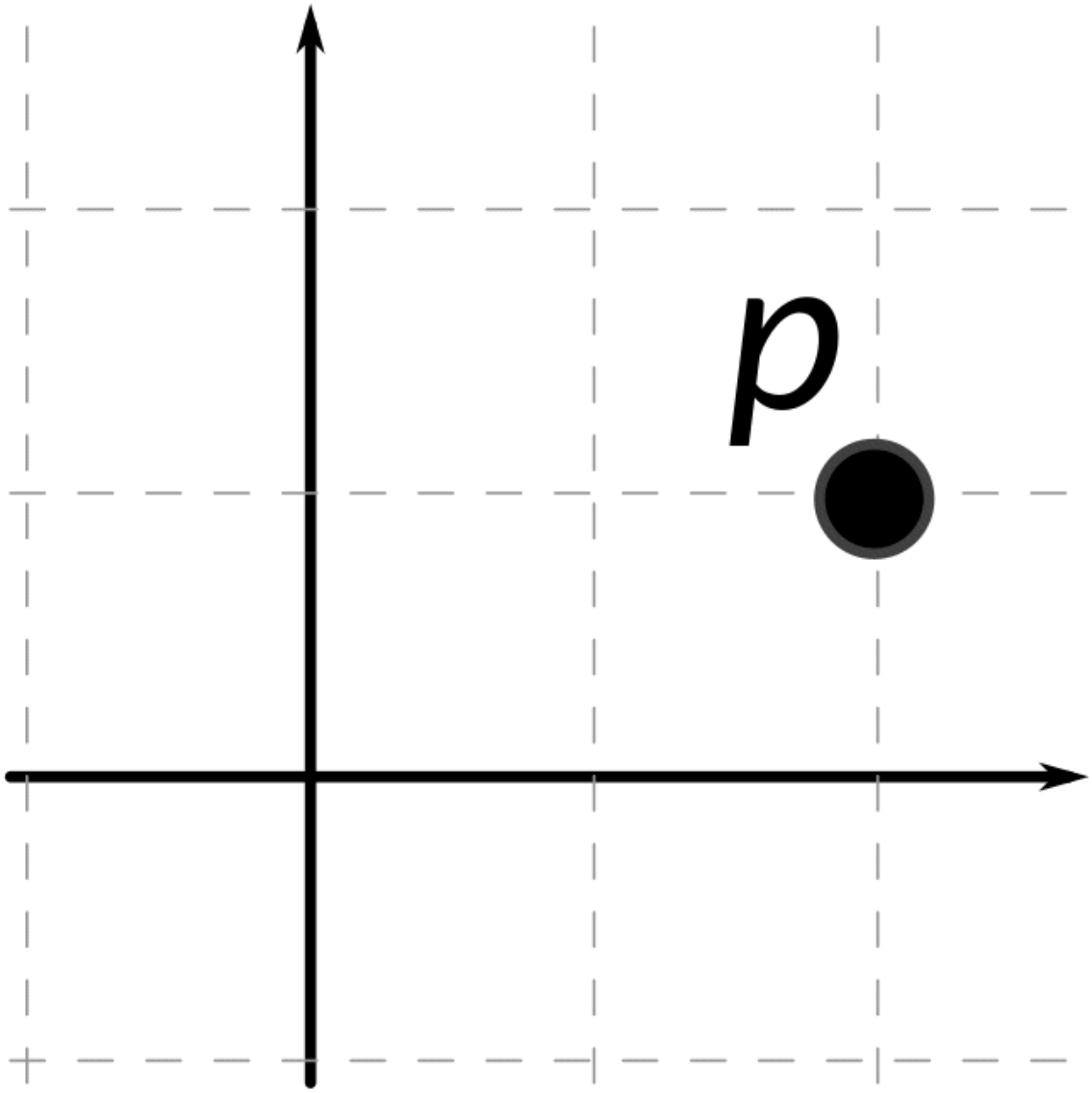
Set M_1 is not a basis set, because the set is linearly dependent. However, a set comprising any one of those vectors would be a basis for a 1D subspace (a line) embedded in ambient 3D space, which, for a Cartesian space, would be the line on the X-axis at Y=Z=0.

Set M_3 is also not a basis set because one can define the first or second vectors as a linear combination of the other vectors. However, the third vector and either the first or second vector would form a basis for a 2D subspace, which is a plane in â„ ^{3}.

Letâ€™s consider the geometric consequence of different bases for the same subspace. The graph in Figure 4.10 shows a point *p* ; how would you identify this point using the three basis sets below?

$$S = \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\}, \qquad T = \left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 2 \end{bmatrix} \right\}, \qquad U = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\}$$

Identifying point *p* using different bases means to determine the set of coefficients that multiply each basis vector to get from the origin to *p* . Letâ€™s start with basis set *S* . Actually, this is the standard Cartesian basis set. Thus, $p_{[S]}$ (this is the notation to indicate point *p* using basis set *S*) is simply [2,1].



FigureÂ 4.10: A point on a graph has different coordinates depending on the basis vectors.

For basis set T , we have $p_{[T]} = [2,-.5]$. Why is this the correct answer? Starting from the origin, draw a vector that is two times the first vector in set T , and then add -.5 times the second vector in set T . That will get you from the origin to point p .

Now for set U . Ah, this is a trick. In fact, set U is not a basis set, because it is not a linearly independent set. No set with the zeros vector can be independent. For our exercise here, it is impossible to reach point p using the vectors in set U , because $span(U)$ is a 1D subspace that does not touch point p .

Why is it important that the set be linearly independent? You might think that it should be sufficient for the set to span the subspace, and any additional vectors are just there, you know, for fun. But any given vector in the subspace spanned by a basis should have a *unique* coordinate using that basis. For example, consider the following set:

$$\left\{ \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \begin{bmatrix} 0 \\ 2 \end{bmatrix} \right\}$$

This set is **not** a basis for â„ ^{2} , but letâ€™s pretend for a moment that it is. The vector that, in standard coordinates, is given by [-2 6] can be obtained by scaling these "basis vectors" by (-6,4,0) or (0,-2,6) or (-2,0,4), or an infinite number of other possibilities.[8] Thatâ€™s confusing. Therefore, mathematicians decided that a vector must have *unique* coordinates within some basis set, which happens only when the set is linearly independent.

[8] Independent bases ensure uniqueness.

Infinite bases Although any given vector has unique coordinates within a basis (assuming that basis spans the subspace the vector lives in), the reverse is not the case: There is an infinite number of bases that describe any subspace. You might have already guessed that this is the case from the discussion around Figure 4.10 . Here is an example of a small and finite number of distinct bases for â„ ^{2} . (In fact, *any* linearly independent set of two 2D vectors is a basis for all of â„ ^{2} .)

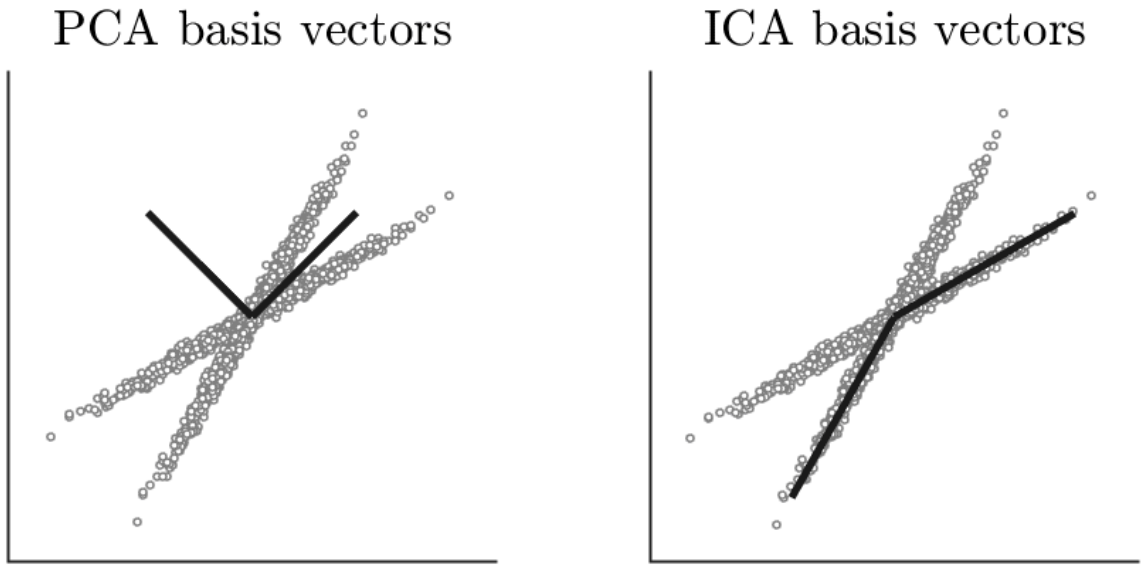
$$\left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ \frac{1}{2} \end{bmatrix} \right\}, \left\{ \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\}, \left\{ \begin{bmatrix} e \\ 2 \end{bmatrix}, \begin{bmatrix} .5 \\ 4^{74} \end{bmatrix} \right\}$$

For example, using the first two sets, the Cartesian-basis coordinate (3,4) would be obtained by scaling the basis vectors by [3,8] and [-3,4], respectively.

Why have so many possible bases? Shouldnâ€™t we be happy with a small number of bases, for example the familiar Cartesian basis sets?

The truth is that not all basis sets are created equal. Some bases are better than others, and some problems are easier to solve in certain bases and harder to solve in other bases. For example, that third basis set above is valid, but would be a huge pain to work with. In fact, finding optimal basis sets is *one of the most important problems* in multivariate data science, in particular data compression and components analyses.

Consider Figure 4.11 : The dots correspond to data points, and the black lines correspond to basis vectors. The basis set in the left graph was obtained via principal components analysis (PCA) whereas the basis set in the right graph was obtained via independent components analysis (ICA). (Youâ€™ll learn the math and implementation of PCA in Chapter 19.) Thus, these two analyses identified two different basis sets for â„ [2] ; both bases are valid, but which one describes the patterns in the data better?



FigureÂ 4.11: This example of 2D datasets (both graphs show the same data) with different basis vectors (black lines) is an example of the importance of choosing basis sets for characterizing and understanding patterns in datasets.

-|*|- ***Reflection*** *In this section we are discussing only basis vectors. You may have also heard about basis functions or basis images. The concept is the sameâ€”a basis is the set of the minimum number of metrics needed to describe something. In the Fourier transform, for example, sine waves are basis functions, because all signals can be represented using sine waves of different frequencies, phases, and amplitudes.* -|*|-

4.8 Exercises

1.Â

Determine whether each vector is in the span of sets *S* and *T* , and if so, what coefficients could produce the given vectors from the sets.

$$S = \left\{ \begin{bmatrix} 0 \\ 1 \\ 3 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 3 \end{bmatrix} \right\} \qquad T = \left\{ \begin{bmatrix} 1 \\ .5 \\ 0 \end{bmatrix}, \begin{bmatrix} 4 \\ 1 \\ 1 \end{bmatrix} \right\}$$

a) $\begin{bmatrix} 2 \\ 2 \\ 6 \end{bmatrix}$

b) $\begin{bmatrix} 0 \\ 4 \\ 12 \end{bmatrix}$

c) $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

d) $\begin{bmatrix} -10 \\ -2 \\ -3 \end{bmatrix}$

e) $\begin{bmatrix} -3 \\ 2 \\ -3 \end{bmatrix}$

2.Â

Determine whether the following vector is in the set spanned by the bracketed vectors, in other words, whether **u** âˆˆ *S* = $\{\mathbf{v_1}, \ldots, \mathbf{v_n}\}$

a) $\begin{bmatrix}5\\1\end{bmatrix}, \left\{ \begin{bmatrix}1\\0\\3\end{bmatrix}, \begin{bmatrix}1\\1\\3\end{bmatrix} \right\}$

b) $\begin{bmatrix}4\\1\\12\end{bmatrix}, \left\{ \begin{bmatrix}1\\0\\3\end{bmatrix}, \begin{bmatrix}1\\1\\3\end{bmatrix} \right\}$

c) $\begin{bmatrix}0\\1\\3\end{bmatrix}, \left\{ \begin{bmatrix}1\\0\\3\end{bmatrix}, \begin{bmatrix}-1\\1\\2\end{bmatrix} \right\}$

3.Â

Label the following sets as independent or dependent. For dependent sets, determine whether it is possible to modify only one element of one vector to change it to an independent set.

a) $\left\{ \begin{bmatrix}1\\2\end{bmatrix}, \begin{bmatrix}2\\1\end{bmatrix} \right\}$

b) $\left\{ \begin{bmatrix}1\\2\end{bmatrix}, \begin{bmatrix}1\\-2\end{bmatrix} \right\}$

c) $\left\{ \begin{bmatrix}1\\2\end{bmatrix}, \begin{bmatrix}-1\\-2\end{bmatrix} \right\}$

d) $\left\{ \begin{bmatrix}1\\2\end{bmatrix}, \begin{bmatrix}1\\-2\end{bmatrix}, \begin{bmatrix}-1\\-2\end{bmatrix} \right\}$

e) $\left\{ \begin{bmatrix}1\\0\end{bmatrix}, \begin{bmatrix}3\\0\end{bmatrix} \right\}$

f) $\left\{ \begin{bmatrix}5\\12\end{bmatrix}, \begin{bmatrix}-3\\4\end{bmatrix}, \begin{bmatrix}10\\23\end{bmatrix} \right\}$

g) $\left\{ \begin{bmatrix}1\\2\\3\end{bmatrix}, \begin{bmatrix}1\\2\\4\end{bmatrix} \right\}$

h) $\left\{ \begin{bmatrix}1\\2\\3\end{bmatrix}, \begin{bmatrix}8\\16\\24\end{bmatrix} \right\}$

i) $\left\{ \begin{bmatrix}1\\0\\1\end{bmatrix}, \begin{bmatrix}2\\1\\2\end{bmatrix}, \begin{bmatrix}3\\1\\3\end{bmatrix} \right\}$

4.Â

Determine the value of *Î»* that would make the following sets of vectors dependent.

a) $\left\{ \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 4.5 \\ \lambda \\ 13.5 \end{bmatrix} \right\}$

b) $\left\{ \begin{bmatrix} 0 \\ 0 \\ \lambda \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} a \\ b \\ 5 \\ c \\ d \end{bmatrix} \right\}$

c) $\left\{ \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 4 \\ \lambda \\ 5 \end{bmatrix}, \begin{bmatrix} 5 \\ 0 \\ 8 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right\}$

5.Â

The following sets of vectors are dependent sets. For each set, determine the number of vectors to remove to create an independent set with the most number of vectors.

a) $\left\{ \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 3 \\ 4 \\ 5 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 5 \\ 4 \\ 5 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 6 \\ 8 \\ 10 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 4 \\ 4 \\ 5 \\ 1 \end{bmatrix} \right\}$

b) $\left\{ \begin{bmatrix} 4 \\ 3 \\ 2 \end{bmatrix}, \begin{bmatrix} 4 \\ 4 \\ 3 \end{bmatrix}, \begin{bmatrix} 7 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 3 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} \right\}$

6.Â

Determine whether the following are descriptions of subspaces and subsets, or only subsets.

a) The set of points y such that $y = 2x$.

b) The set of points y such that $y = 2x + .01$.

c) The point at the origin in â„ ^{5} .

d) The set of all points in â„ ^{3} with positive magnitude.

7.Â

What is the dimensionality of the subspace spanned by the following vector subspaces?

a) $\left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 7 \\ 0 \\ 0 \\ 0 \end{bmatrix} \right\}$

b) $\left\{ \begin{bmatrix} 1 \\ 2 \\ 6 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ 4 \\ 12 \\ 6 \end{bmatrix} \right\}$

c) $\left\{ \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \right\}$

d) $\left\{ \begin{bmatrix} 6 \\ 9 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix}, \begin{bmatrix} 4 \\ 6 \\ 2 \end{bmatrix} \right\}$

8.Â
Remove one vector in the following sets to create a basis set for a 2D subspace.

a) $\left\{ \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} \right\}$

b) $\left\{ \begin{bmatrix} 1 \\ 4 \\ 3 \end{bmatrix}, \begin{bmatrix} 0 \\ -4 \\ 5 \end{bmatrix}, \begin{bmatrix} 3 \\ 12 \\ 9 \end{bmatrix} \right\}$

c) $\left\{ \begin{bmatrix} -3 \\ 2 \\ 13 \end{bmatrix}, \begin{bmatrix} 4.5 \\ -3 \\ -19.5 \end{bmatrix}, \begin{bmatrix} -1.5 \\ 1 \\ 6 \end{bmatrix} \right\}$

4.9 Answers

1.Â

- **a)** Neither
- **b)** *S* : [4,0]
- **c)** Both [0,0]
- **d)** *T* : [2,-3]
- **e)** *S* : [2,-3]

2.Â

- **a)** no
- **b)** yes
- **c)** no

3.Â
Most dependent sets can be made independent with one value-change if there are N â‰¤ M vectors in â„^{M}.

- **a)** Independent
- **b)** Independent
- **c)** Dependent, yes
- **d)** Dependent, no
- **e)** Dependent, yes
- **f)** Dependent, no
- **g)** Independent
- **h)** Dependent, yes
- **i)** Dependent, yes

4.Â

- **a)** $\hat{I}» = 9$
- **b)** $\hat{I}» = 0$ or anything if
  $a = b = c = d = 0$
- **c)** Any $\hat{I}»$

5.Â
The answers below indicate the number of vectors that can be removed. In these examples, there are different collections of vectors that could produce linearly independent sets.

- **a)** 2
- **b)** 2

6.Â
The strategy is to think about whether any point or vector **p** in the subset would still be in the subset for any scalar-multiple $\hat{I}¼$ **p** , including $\hat{I}¼ = 0$.

- **a)** Both.
- **b)** Subset only.

c) Both.
d) Subset only.

7.Â

a) 1D
b) 2D
c) 3D
d) 1D

8.Â

a) First or second
b) First or third
c) First or second

ChapterÂ 5
Matrices

5.1 Interpretations and uses of matrices

The goal of this chapter is to introduce you to matrices: what they look like, how to refer to them, several important special matrices, and basic matrix arithmetic.

In this book, we will work only with matrices that are rows Ã— columns, or matrices that you could print out and lay flat on a table. The terminology can get a bit confusing here, because you might think that these are 2D matrices. However, although they would be in two physical dimensions when printed on a piece of paper, the number of dimensions in which a matrix lives is more open to interpretation compared to vector dimensionality (more on this later). Matrices that would require a 3D printer (or, for higher dimensions, a hyper-printer) occupy cubes or hypercubes in physical space, and are called *tensors* . Tensors are useful for storing data and for representing, for example, various physical forces acting on an object. But they will not be further discussed in this book.

Below are two example matrices. Itâ€™s sometimes useful to think of a matrix as a set of column vectors standing next to each other, or as a set of row vectors stacked on top of each other.

$$\begin{bmatrix} 1 & 4 & 0 & 2 \\ -4 & 5 & 0 & 0 \end{bmatrix}, \quad \begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix}$$

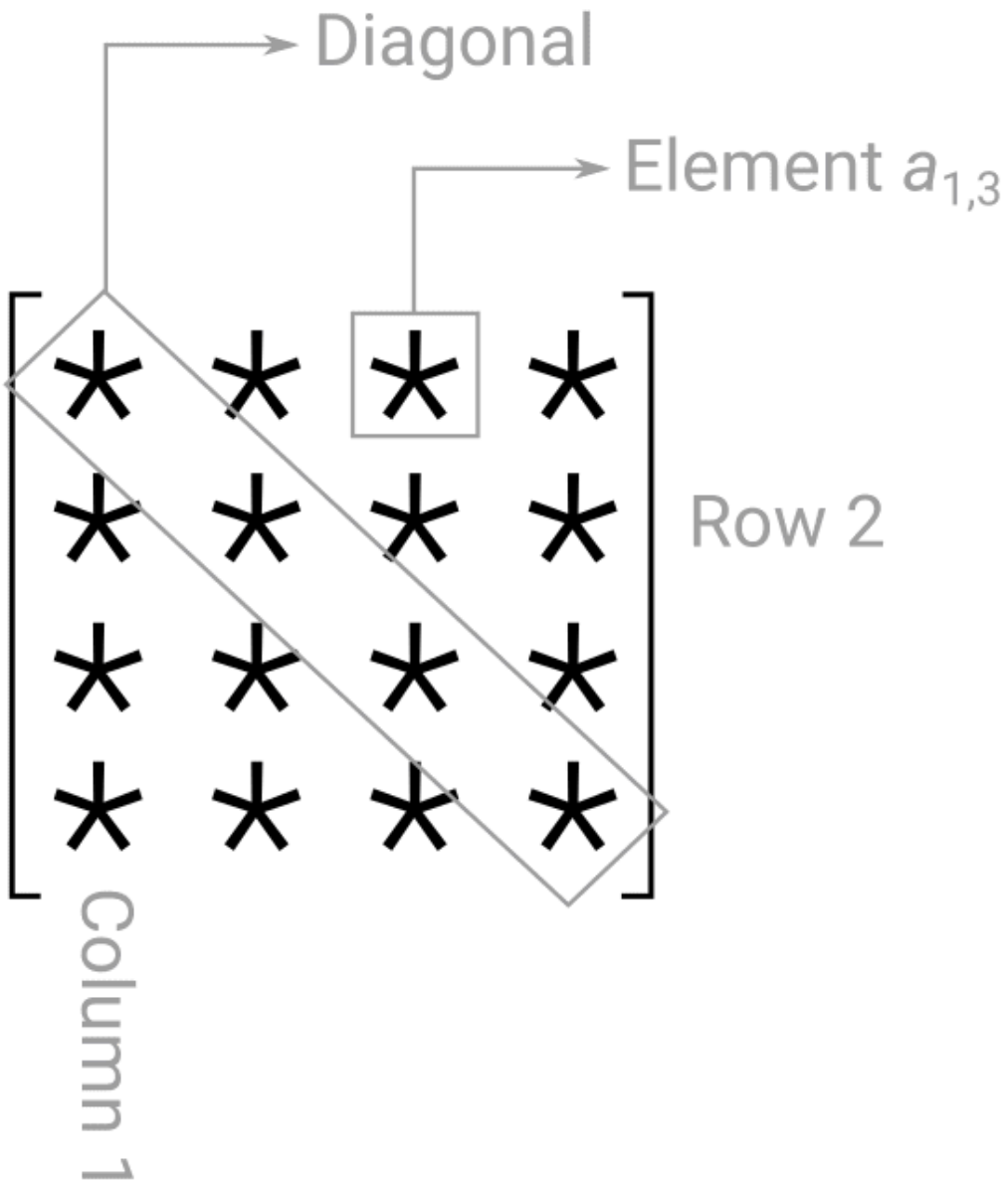
Matrices are ubiquitous in pure and applied mathematics because they have many different purposes. To give you an idea of their versatility, below is a non-exhaustive list of some uses of matrices.

- Representing a linear transformation or mapping
- Storing partial derivatives of a multivariable system
- Representing a system of equations
- Storing data (e.g., features Ã— observations)
- Representing regressors for statistical modeling
- Storing geometric transformations for computer graphics
- Storing kernels used in filtering or convolution

- Representing finance information from different sectors of an economy or business
- Housing parameters for a model that predicts changes in the spread of an infectious disease

5.2 Matrix terminology and notation

The first step of learning anything new is to become acquainted with the terminology. When referring to an entire matrix, a boldface capital letter is used (matrix **A**), and when referring to individual elements of a matrix, a lower-case letter with subscripts is used (matrix **A** contains elements $a_{i,j}$). The matrix-related terms that you need to know include row, column, element, block, diagonal, skew-diagonal, and off-diagonal. Several of these are illustrated in Figure 5.1 .



FigureÂ 5.1: Terminology for several key matrix components.

A matrix can comprise smaller matrices, which leads to a useful block-matrix notation (Figure 5.2). Block matrices are not featured in this book, but itâ€™s good to know about the notation.

$$\mathbf{A} = \begin{bmatrix} \mathbf{D} & \mathbf{0} \\ \mathbf{1} & \mathbf{D} \end{bmatrix} = \begin{bmatrix} 3 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 1 & 1 & 3 & 0 \\ 1 & 1 & 0 & 4 \end{bmatrix}$$

$$\mathbf{D} = \begin{bmatrix} 3 & 0 \\ 0 & 4 \end{bmatrix} \quad \mathbf{0} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \quad \mathbf{1} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

FigureÂ 5.2: A "block matrix" is a matrix that comprises smaller matrices. This can facilitate visual inspection and computations.

When referring to matrix sizes, and when referring to indices in a matrix, it is assumed that you refer first to *rows* then to *columns* . The number of rows is typically indicated as M while the number of columns is typically indicated as N . This is, of course, an arbitrary choice, but common convention curtails confusion.

A mnemonic for remembering this convention is **M** *R* **N** i*C* e guy, for **M** *R* ows and **N** *C* olumns (Figure 5.3). When multiple matrices are shown, the sizes might be indicated by other letters like K and L , or by subscripts like M_1 , M_2 and N_1 , N_2 .



FigureÂ 5.3: Notation for referring to matrix sizes: "Mr. Nice guy": M rows, N columns. I know, itâ€™s cheesey, but that makes it memorable.

5.3 Matrix dimensionalities

The dimensionality of matrices is more flexible and more versatileâ€”and therefore more confusingâ€”compared to the dimensionality of vectors. The dimensionality of a vector is simply the number of elements in that vector. Matrices can have different interpretations of dimensionality, depending on the application and the information stored in the matrix. The possible dimensionalities of an *M* Ã— *N* matrix containing real-valued numbers include:

- â„ MÃ— N
- â„ MN , if each matrix element is its own dimension (this is the closest interpretation to the vector dimensionality definition).

- â„ ^{M} , if the matrix is conceptualized as a series of column vectors (each column contains *M* elements and is thus in â„ ^{M}).
- â„ ^{N} , if the matrix is conceptualized as a stack of row vectors.

In practice, the dimensionality of a given matrix is made on a case-by-case basis, and is either explicitly stated or is inferred from context.

5.4 The transpose operation

You learned about the *transpose* operation on vectors. Transposing matrices is the same concept: swap rows for columns and vice-versa.

The notation is also the same as for vectors: the transpose of **A** is **A** ^{T} (also as mentioned previously, the T is sometimes italicized, so **A** ^{T} = **A** ^{T}). Setting **B** to be the transpose of **A** (that is, **B** = **A** ^{T} leads to the formal definition:

–

Eqn. box title=**Transposing a vector or matrix**

$$\mathbf{B}_{i,j} = \mathbf{A}_{j,i} \qquad (5.1)$$

–

Notice that element (*row* , *column*) in matrix **B** corresponds to element (*column* , *row*) in matrix **A** .

An important property of the transpose is that the transpose of a transposed matrix is the original matrix. This property will have a central role in many linear algebra proofs.

$$\mathbf{A}^{\mathrm{TT}} = \mathbf{A} \qquad (5.2)$$

Here is an example of a matrix transposed.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

One easy-to-make mistake when taking the transpose is to change the order of the rows and columns accidentally. Compare the following result (**not** the transpose!) to above:

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\end{bmatrix}^{\mathrm{T}} \neq \begin{bmatrix}4 & 1\\5 & 2\\6 & 3\end{bmatrix}$$

Code Transposing matrices is the same as transposing vectors. The code below shows additional ways to transpose a matrix.

Code blockÂ 5.1: Python

```
AÂ =Â np.random.randn(2,5)Â
At1Â =Â A.TÂ
At2Â =Â np.transpose(A)
```

Code blockÂ 5.2: MATLAB

```
AÂ =Â randn(2,5);Â
At1Â =Â Aâ€™;Â
At2Â =Â transpose(A);
```

–

Practice problems Apply the transpose operator as indicated.

a) $\begin{bmatrix}0 & 1 & 0\\2 & 0 & 3\end{bmatrix}^{\mathrm{T}}$

b) $\begin{bmatrix}16 & 6 & 0\\6 & 10 & -3\\0 & -3 & 22\end{bmatrix}^{\mathrm{T}}$

c) $\begin{bmatrix}-1 & -3 & 1\\-4 & 5 & 1\\-5 & -4 & 6\\0 & 5 & 5\end{bmatrix}^{\mathrm{TT}}$

Answers

a) $\begin{bmatrix} 0 & 2 \\ 1 & 0 \\ 0 & 3 \end{bmatrix}^{\mathrm{T}}$

b) $\begin{bmatrix} 16 & 6 & 0 \\ 6 & 10 & -3 \\ 0 & -3 & 22 \end{bmatrix}$

c) $\begin{bmatrix} -1 & -3 & 1 \\ -4 & 5 & 1 \\ -5 & -4 & 6 \\ 0 & 5 & 5 \end{bmatrix}$

5.5 Matrix zoology

There are many categories of special matrices that are given names according to their properties, or sometimes according to the person who discovered or characterized them.

Following is a non-exhaustive list and description of several special matrices. Itâ€™s useful to have some familiarity with these kinds of matrices now; throughout the book (and your adventures through the wonderland of linear algebra and its applications) youâ€™ll see these matrices many times. For now, try to familiarize yourself with these classes of matrices without worrying too much about their applications or myriad special properties.

Square or rectangular Very simply: A square matrix is a matrix that has the same number of rows as columns, thus an N Ã— N matrix or M Ã— N if $M = N$.

A non-square matrix is called a rectangular matrix . You can see a square and a rectangular matrix below.

$$\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 & 2 & 3 & -5 \\ -1 & 0 & 3 & 23 & 0 \\ -2 & -3 & 0 & 0 & 0 \end{bmatrix}$$

Let me guess what you are thinking: "But Mike, a square is also a rectangle!" Yes, dear reader, thatâ€™s technically true. However, for ease of comprehension, it is assumed that a "rectangular matrix" is one in which the number of rows does not equal the number of columns, thus *Mâ‰ N* . "Non-square" is also acceptable.

Rectangular matrices are also sometimes referred to as "fat" or "wide" if they have more columns than rows, or as "skinny" or "tall" if they have more rows than columns. Below you can see a wide matrix (left) and a tall matrix (right).

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \\ 9 & 10 \end{bmatrix}$$

Symmetric A matrix is symmetric if it is "mirrored" across the diagonal, meaning the upper-right of the matrix is a flipped version of the lower-left of the matrix. Formally, matrix **A** is symmetric if it equals its transpose:

_____

_

Eqn. box title=**Definitions of a symmetric matrix**

$$\mathbf{A} = \mathbf{A}^{T} \qquad (5.3)$$

$$a_{i,j} = a_{j,i} \qquad (5.4)$$

_____

_____

Only square matrices can be symmetric, because if a matrix is [1] size M Ã— N , its transpose is size N Ã— M , and Equation 5.3 says that the matrix must equal its transpose. Thus, $M = N$ is a necessary (though not sufficient) condition for symmetry. For this reason, vectors cannot be symmetric.

[1] People sometimes refer to "square symmetric matrices." Technically, this is redundant, because squareness is required for symmetry.

Here are a few examples of symmetric matrices.

$$\begin{bmatrix} 1 & 4 & \pi \\ 4 & 7 & 2 \\ \pi & 2 & 0 \end{bmatrix},$$

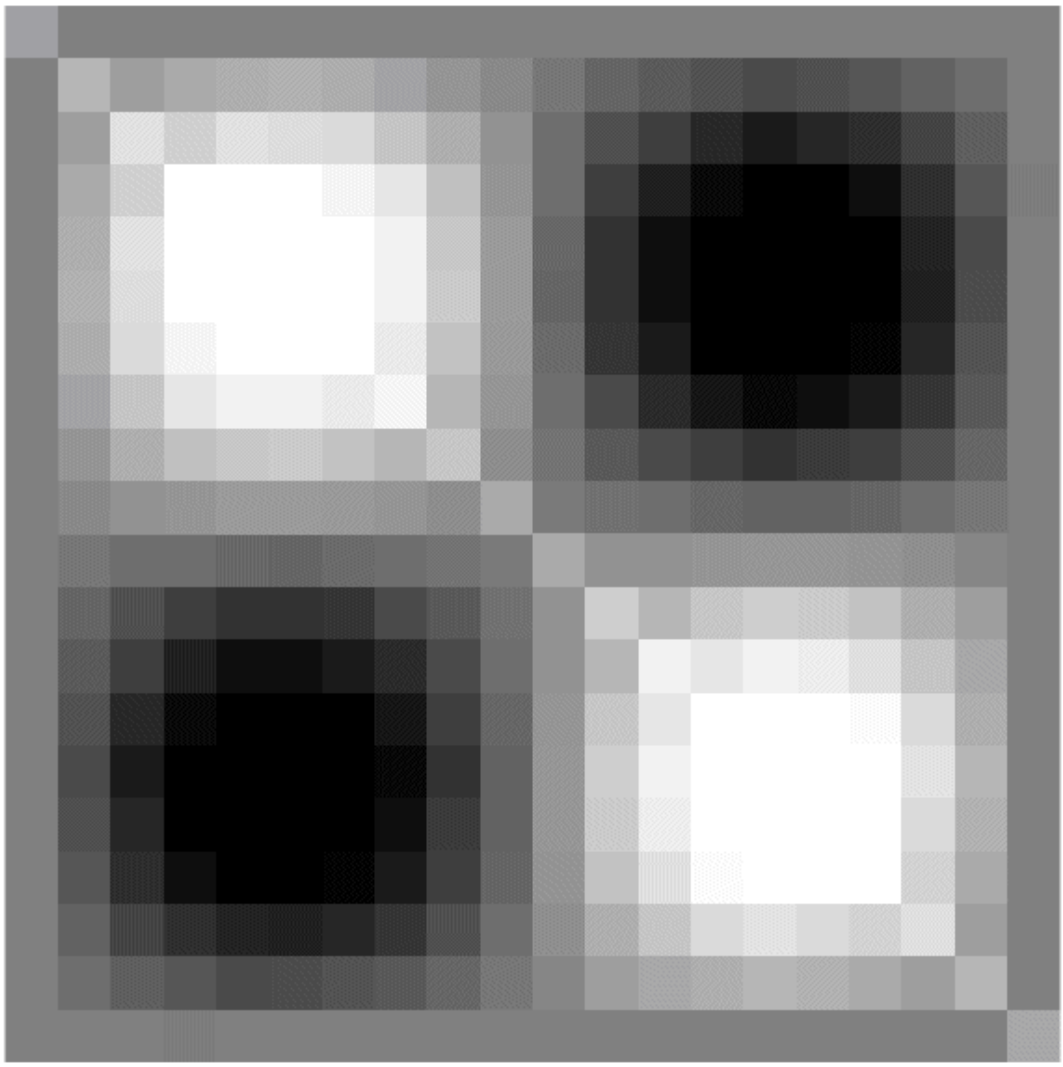
$$\begin{bmatrix} a & b \\ b & c \end{bmatrix},$$

$$\begin{bmatrix} a & e & f & g \\ e & b & h & i \\ f & h & c & j \\ g & i & j & d \end{bmatrix}$$



A matrix that is not symmetric is called non-symmetric or asymmetric. All rectangular matrices, for example, are non-symmetric.

Symmetric matrices have lots of special properties, as you will learn throughout this book. If matrices lived in a capitalist society, symmetric matrices would be the beautiful, cultured, educated, generous, happy, successful, and wealthy upper-class that get all the attention and special privileges.



FigureÂ 5.4: Symmetric matrices can have a nice visual representation. The grayscale of each box corresponds to the numerical value of the matrix.

Fortunately, hope is not lost for non-symmetric matrices. It is possible to create a symmetric matrix from a non-symmetric (even rectangular!) matrix. Indeed, creating symmetric from non-symmetric matrices is central to many statistics and machine learning applications, such as least-squares and eigendecomposition. For example, you might have heard of a "covariance matrix"; this is simply a symmetric matrix created from a rectangular data matrix. There are two ways to create a symmetric matrix from a non-symmetric matrix, both of which will be discussed in the next chapter.

Skew-symmetric A skew-symmetric matrix is a matrix where the lower-triangle is the sign-flipped version of the upper-triangle. Here are two examples of 3 Ã—

3 skew-symmetric matrices.

$$\begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 3 \\ -2 & -3 & 0 \end{bmatrix}, \begin{bmatrix} 0 & -4 & 8 \\ 4 & 0 & -5 \\ -8 & 5 & 0 \end{bmatrix}$$

It is no accident that the diagonal elements are all zeros. The diagonal must equal its negative, and the only number for which that is true is 0 (0 = âˆ’ 0). Thus, all skew-symmetric matrices have diagonals of all zeros.

More formally, a skew-symmetric matrix has the following matrix and element definitions.

$$\mathbf{A} = \text{â}\text{ˆ}\text{’ } \mathbf{A}^{\mathrm{T}} \tag{5.5}$$

$$a_{i,j} = \text{â}\text{ˆ}\text{’ } a_{j,i} \tag{5.6}$$

Identity The identity matrix is the matrix equivalent of the number 1, in that any number *x* times 1 is *x* . In the case of matrices, any vector **v** times the identity matrix (indicated by **I**) is that same vector **v** , and any matrix **A** times **I** is still **A** . In other words:

$$\mathbf{vI} = \mathbf{v}$$

$$\mathbf{AI} = \mathbf{A}$$

You might initially think that the identity matrix would [2] be a matrix of all 1â€™s, however, it actually has 1â€™s only on the diagonal and all 0â€™s on the off-diagonals. **I** is always a square matrix.

[2] The matrix indicated by **1** has all elements equal to 1; this is called a "ones matrix."

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

Subscripts are sometimes provided to indicate the size of the matrix, as in the following examples. If there is no subscript, you can assume that **I** is the appropriate size to make the equation valid.

$$\mathbf{I}_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The identity matrix has many important functions. It features in many proofs, in the matrix inverse, and in eigendecomposition and regularization.

Although it is simply called "the identity matrix," a more accurate term would be "the multiplicative identity matrix." That can be contrasted with the *additive* identity matrix, which is the zeros matrix.

Zeros The zeros matrix is a matrix of all zeros. That might sound like a pretty boring matrix, but... well, it is. As you might guess, any matrix times the zeros matrix is still the zeros matrix, just like any number times 0 is 0 (letâ€™s stick to finite numbers to avoid headaches from thinking about what happens when you multiply 0 by infinity).

The zeros matrix is indicated by **0** . As with the matrix **I** , a subscript is sometimes used when itâ€™s necessary to indicate the size of the matrix **0** . This can be confusing, because the zeros *vector* can also be indicated using **0** . Hopefully the context will clarify the correct interpretation.

$$\mathbf{0}_N = \begin{bmatrix} 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}$$

Zeros matrices can be square or rectangular, but in most cases you can assume that the zeros matrix is square; rectangular matrices are often used in code, for example when initializing data matrices.

Analogous to how **I** is actually the *multiplicative identity matrix* , **0** is more accurately called the *additive identity matrix* . This is because **AI** = **A** but **A** + **I** *â‰ * **A** , and **A** + **0** = **A** but **A0** *â‰ * **A**

Code Many special matrices can be created easily using dedicated functions. Notice that zeros matrices in Python require two numbers to specify the size (the number of rows and the number of columns).

Code blockÂ 5.3: Python

```
IÂ =Â np.eye(4)Â
OÂ =Â np.ones(4)Â
ZÂ =Â np.zeros((4,4))
```

Code blockÂ 5.4: MATLAB

```
IÂ =Â eye(4);Â
OÂ =Â ones(4);Â
ZÂ =Â zeros(4);
```

$\mathbf{A}^{\mathbf{T}}\mathbf{A}$ Pronounced "A transpose A" and sometimes written "AtA," this is one of the most important matrix forms in all of applied linear algebra. Creating $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ involves matrix multiplication, which is the topic of the next chapter. However, there are several key properties of $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ that are worth listing here. Their true value will become apparent as you progress through this book. Each of the following claims will be discussed and proven in later chapters; for now, simply marvel at the glorious existential goodness of $\mathbf{A}^{\mathrm{T}}\mathbf{A}$:

- It is a square matrix, even if **A** is rectangular.
- It is symmetric, even if **A** isnâ€™t.
- It is full-rank if **A** is full column-rank.
- It is invertible if **A** is full column-rank.
- It has the same row space as **A** .
- It has orthogonal eigenvectors.
- It is positive (semi)definite.
- It has non-negative, real-valued eigenvalues.
- It is called a "covariance matrix" if **A** is a data matrix.
- It often looks pretty (e.g., Figure 5.4).

All of these properties arise simply from multiplying a matrix by its transpose. These same properties hold for $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ as well, but $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ looks nicer in print.

Diagonal A diagonal matrix has all zeros on the off-diagonals, and only the diagonal elements (going from top-left to bottom-right) may contain non-zero elements. **I** is an example of a diagonal matrix, as are the following two matrices. [3]

[3] Itâ€™s OK if some or all diagonal elements are zero, as long as all off-diagonal elements are zero.

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}, \begin{bmatrix} \pi & 0 & 0 & 0 \\ 0 & e^{x^2} & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

More formally, a diagonal matrix is defined as follows.

(5.7)

$$a_{i,j} = \begin{cases} \alpha_n, & \text{if } i = j \\ 0, & \text{if } i \neq j \end{cases}$$

If all diagonal elements are the same, then the matrix can be written as a constant times the identity matrix, or *Î»* **I** . For example,

$$\begin{bmatrix} 7 & 0 \\ 0 & 7 \end{bmatrix} = 7 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = 7\mathbf{I}_2$$

Diagonal matrices can be rectangular, as in the following examples.

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 6 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 7 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Diagonal matrices are useful because they simplify operations including matrix multiplication and matrix powers ($\mathbf{A}^n$). Transforming a matrix into a diagonal matrix is called *diagonalization* and can be achieved via eigendecomposition or singular value decomposition, as you will learn in later chapters.

The bold-faced **D** is often used to indicate a diagonal matrix. However, there are also plenty of diagonal matrices that are not labeled **D** . For example, **Î›** and **Î£** are diagonal matrices used in eigendecomposition and singular value decomposition, respectively.

The opposite of a diagonal matrix is called a hollow matrix. A hollow matrix has all zeros on the diagonal, while the off-diagonal elements may be non-zero (they can also be zero, as long as the diagonal elements are zero). Skew-symmetric matrices are hollow. Hollow matrices appear in applications as distance matrices (every node is zero distance away from itself), but I wonâ€™t discuss them further in this book.

As shown in Figure 5.1 , the diagonal of a matrix goes from the top-left to the lower-right. The *anti-diagonal* goes from the top-right to the lower left.

Code In both Python and MATLAB, the same function extracts the diagonal elements of a matrix, and produces a diagonal matrix given an input vector.

Code blockÂ 5.5: Python

```
DÂ =Â np.diag([1,2,3,5])Â #Â diagonalÂ matrixÂ
RÂ =Â np.random.randn(3,4)Â
dÂ =Â np.diag(R)Â #Â diagonalÂ elements
```

Code blockÂ 5.6: MATLAB

```
DÂ =Â diag([1,2,3,5]);Â %Â diagonalÂ matrixÂ
RÂ =Â randn(3,4);Â
dÂ =Â diag(R);Â %Â diagonalÂ elements
```

Augmented An augmented matrix is the result of concatenating two or more matrices column-wise, as in this example:

$$\begin{bmatrix} 1 & 4 & 2 \\ 3 & 1 & 9 \\ 4 & 2 & 0 \end{bmatrix} \sqcup \begin{bmatrix} 7 & 2 \\ 7 & 2 \\ 7 & 1 \end{bmatrix} = \left[\begin{array}{ccc|cc} & 1 & & 4 & 2 & 7 & 2 \\ {}^{4}\text{The symbol } || \text{ is sometimes used to indicate concatenation, but it's also used for vector} & & & & & & \\ {}^{4}\text{magnitude and parallel lines.} & & 3 & 1 & 9 & 7 & 2 \\ & 4 & & 2 & 0 & 7 & 1 \end{array}\right]$$

Two matrices can be augmented only if they have the same number of rows; the number of columns need not match.

The vertical bar in the concatenated matrix indicates the break between the two original matrices, and is sometimes omitted. Augmented matrices are used in various applications including solving systems of linear equations and computing the matrix inverse.

Code "Augmenting" is called concatenation in Python and MATLAB. In Python, make sure to state the correct axis along which to concatenate.

Code blockÂ 5.7: Python

```
AÂ =Â np.random.randn(3,5)Â
BÂ =Â np.random.randn(3,4)Â
ABÂ =Â np.concatenate((A,B),axis=1)
```

Code blockÂ 5.8: MATLAB

```
AÂ =Â randn(3,5);Â
BÂ =Â randn(3,4);Â
ABÂ =Â [AÂ B];
```

Triangular Triangular matrices are half-way between a diagonal matrix and a full matrix. Triangular matrices come in two forms: upper-triangular (meaning only the diagonal and elements above it may contain non-zero elements) and lower-triangular (meaning only the diagonal and elements below it may contain non-zero elements). Here are two examples of triangular matrices.

$$\begin{bmatrix} 1 & 8 & 4 \\ 0 & 2 & 1 \\ 0 & 0 & 9 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 & 0 \\ 3 & 2 & 0 & 0 \\ 6 & 5 & 9 & 0 \\ 2 & 0 & 4 & 1 \end{bmatrix}$$

Triangular matrices can be rectangular. Sometimes, the zeros are omitted (either to save ink, because the author was too lazy to type in the 0â€™s, or for visual aesthetics). Here are two more examples of triangular matrices.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 3 & 2 & 0 & 0 & 0 \\ 6 & 0 & 9 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 1 & 3 & 2 & 1 \\ & 2 & 7 & 9 \\ & & 0 & 5 \\ & & & 2 \end{bmatrix}$$

Triangular matrices appear in matrix decompositions including QR decomposition, LU decomposition, and Cholesky decomposition.

Code MATLAB and Python have dedicated functions to extract upper and lower triangular matrices. Tip: You can provide an optional second input k to isolate the elements above or below the kth diagonal.

Code blockÂ 5.9: Python

```
AÂ =Â np.random.randn(5,5)Â
LÂ =Â np.tril(A)Â #Â extractÂ theÂ lowerÂ triangleÂ
UÂ =Â np.triu(A)Â #Â extractÂ theÂ upperÂ triangle
```

Code blockÂ 5.10: MATLAB

```
AÂ =Â randn(5);Â
LÂ =Â tril(A);Â %Â extractÂ theÂ lowerÂ triangleÂ
UÂ =Â triu(A);Â %Â extractÂ theÂ upperÂ triangle
```

Dense and sparse A matrix in which most or all matrix elements are non-zero is called a dense matrix (sometimes also called a "full matrix"). This term is used only when it is necessary in context, for example, when comparing a diagonal matrix with a dense matrix. You wouldnâ€™t normally call every matrix a "dense matrix."

A sparse matrix is one that contains mostly zeros and a relatively small number of non-zero elements. Sparse matrices are extremely computationally efficient, and therefore, a lot of modern algorithms in numerical linear algebra emphasize

sparse matrices. Notice that the 10 Ã— 10 sparse matrix below can be efficiently represented by listing the row and column indices that contain non-zero elements.

$$\begin{bmatrix} 0 & 0 & \mathbf{4} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \mathbf{8} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad \Rightarrow \quad \begin{matrix} (1,3) & 4 \\ (4,6) & 8 \end{matrix}$$

Orthogonal A matrix is called orthogonal if it satisfies the following two criteria.

1. All of its columns are pairwise orthogonal. That means that the dot product between any two columns is exactly 0.
2. Each column *i* has âˆ¥ **Q** $_i$ âˆ¥ = 1 , in other words, each column is unit magnitude. Remember that the magnitude of a vector (in this case, the column of a matrix) is computed as the dot product of that vector with itself.

The letter **Q** is often used to indicate an orthogonal matrix (though not exclusively). The formal definition of an orthogonal matrix is:

–

Eqn. box title=**Definitions of an orthogonal matrix**

$$\langle \mathbf{Q}_i, \mathbf{Q}_j \rangle = \begin{cases} 1, & \text{if } i = j \\ 0, & \text{if } i \neq j \end{cases} \tag{5.8}$$

–

Recall that $\langle\rangle$ is one of the dot-product notations. A more compact way of writing Equation 5.8 using matrix multiplication is

$$\mathbf{Q}^{\mathrm{T}}\mathbf{Q} = \mathbf{I} \tag{5.9}$$

Youâ€™ll learn in the next chapter why this notation is equivalent, but itâ€™s worth already being exposed to it here, because it will set you up to understand

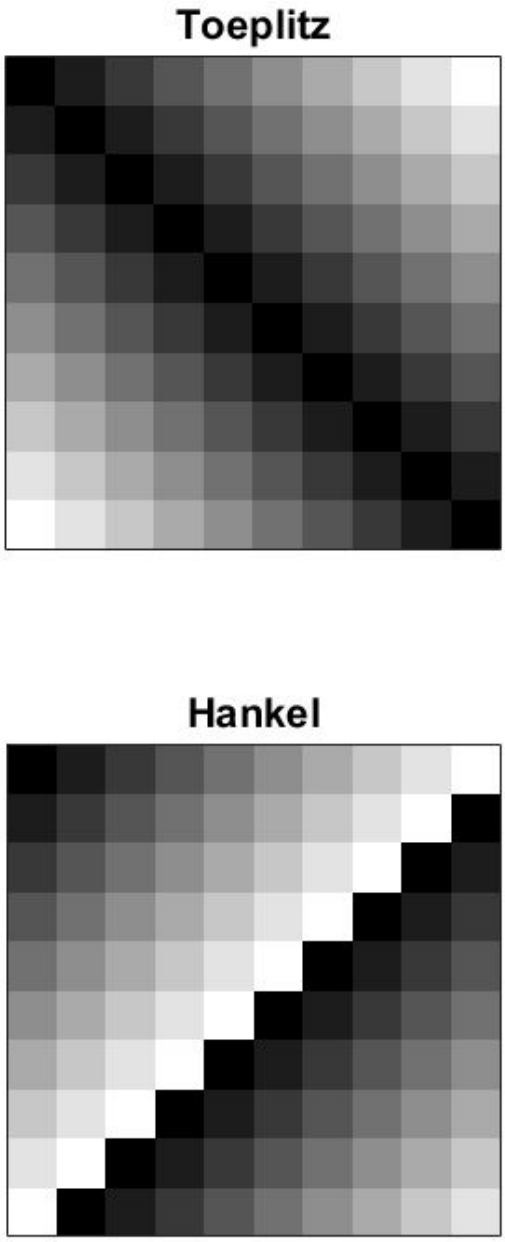
the relationship between the transpose and the inverse of an orthogonal matrix.

Toeplitz Toeplitz and Hankel matrices (below) are closely related to each other. Both involve creating new rows of a matrix as systematic rearangements of elements in previous rows. One of the remarkable properties of Toeplitz and Hankel matrices is that they can have rank $r > 1$ even if they are created from a rank $r = 1$ vector.

In a Toeplitz matrix, all diagonals contain the same element. The matrix below shows a Toeplitz matrix created from a vector. This Toeplitz matrix is also symmetric.

$$\begin{bmatrix} a & b & c & d \end{bmatrix} \Rightarrow \begin{bmatrix} a & b & c & d \\ b & a & b & c \\ c & b & a & b \\ d & c & b & a \end{bmatrix}$$

Notice that the main diagonal is the same as the first element of the vector (a), the next off-diagonal is the second element of the vector (b), and so on.



FigureÂ 5.5: Visualization of Toeplitz and Hankel matrices created from a vector with integers from 1 to 10.

Hankel A Hankel matrix is kindof like a rotated Toeplitz matrix. Compare the following matrix to the Toeplitz matrix above, created from the same vector.

$$\begin{bmatrix} a & b & c & d \end{bmatrix} \Rightarrow \begin{bmatrix} a & b & c & d \\ b & c & d & 0 \\ c & d & 0 & 0 \\ d & 0 & 0 & 0 \end{bmatrix}$$

A Hankel matrix can also "wrap around" to produce a full matrix, like this:

$$\begin{bmatrix} a & b & c & d \\ b & c & d & a \\ c & d & a & b \\ d & a & b & c \end{bmatrix}$$

Notice that the ith column (and the ith row) of the Hankel matrix comes from starting the vector at the ith element and wrapping around. You can also see how the anti-diagonals relate to the vector.

The algorithm to compute a Hankel matrix involves populating a matrix **Y** from elements of a vector **x** :

_____

_

Eqn. box title=**Creating a Hankel matrix from a vector**

$$\mathbf{Y}_{i,j} = \mathbf{x}_{i+j\hat{a}\tilde{}1} \qquad (5.10)$$

_____

_____

Hankel matrices are used in time series convolution and in advanced signal-processing methods such as time-delay embedding. Hankel matrices also look pretty (Figure 5.5) and have aesthetically pleasing properties in eigendecomposition.

Code Creating Toeplitz matrices is intuitive; creating dense Hankel matrices looks a bit strange at first, but you need to specify the final row of the matrix, or else it will have zeros like in the first example above.

Code blockÂ 5.11: Python

```
fromÂ scipy.linalgÂ importÂ hankel,toeplitzÂ
tÂ =Â [1,2,3,4]Â
TÂ =Â toeplitz(t)Â
HÂ =Â hankel(t,r=[2,3,4,1])
```

Code blockÂ 5.12: MATLAB

```
tÂ =Â 1:4;Â
TÂ =Â toeplitz(t);Â
```

```
HÃ =Ã hankel(t,t([endÃ 1:end-1]));
```

5.6 Matrix addition and subtraction

Now that you know some matrix terminology and some special matrices, it is time to begin learning how to work with matrices. The rest of this chapter focuses on the "easy" arithmetic operations on matrices, and the next few chapters will deal with more advanced topics. We begin with basic matrix arithmetic.

Matrix addition is simple, and it works how you would think it should work: Add or subtract each corresponding element in the two matrices. For addition to be a valid operation, both matrices must be the same sizeâ€”*M* Ã— *N* â€”and the resulting matrix will also be *M* Ã— *N* .

Writing out matrix addition looks like this: **C** = **A** + **B** . Each element in **C** is defined as

$$\mathrm{c}_{i,j} = \mathrm{a}_{i,j} + \mathrm{b}_{i,j} \tag{5.11}$$

Below is one example.

$$\begin{bmatrix} 1 & 2 & 5 \\ 9 & 8 & 7 \end{bmatrix} + \begin{bmatrix} -1 & 0 & -5 \\ 3 & a & \pi \end{bmatrix} = \begin{bmatrix} 0 & 2 & 0 \\ 12 & 8+a & 7+\pi \end{bmatrix}$$

Needless to say, matrix subtraction works exactly the same way, except with a minus sign instead of a plus sign.

Like vector addition, matrix addition is commutative, meaning that

$$\mathbf{C} = \mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$$

This may seem trivial based on your elementary-school arithmetic knowledge, but it is important to mention these properties explicitly, because, for example, matrix multiplication is not commutative. The commutivity of matrix addition is also a key step in a proof about creating symmetric matrices, which you will see in a few pages!

–

Practice problems Solve the following matrix arithmetic problems.

a) $\begin{bmatrix} 1 & 1 \\ 2 & 3 \end{bmatrix} + \begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 5 \\ -4 & 6 \\ -3 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}$

c) $\begin{bmatrix} 2 & 4 & 6 \\ 1 & 3 & 5 \\ 4 & 2 & 0 \\ 5 & 6 & 7 \end{bmatrix} + \begin{bmatrix} 9 & 8 \\ 7 & 6 \\ 5 & 4 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 5 & 4 \\ 4 & 4 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 6 \\ -3 & 7 \\ -2 & 0 \end{bmatrix}$

c) Invalid operation!

5.7 Scalar-matrix multiplication

Matrix multiplication is sufficiently varied and involved that it requires its own chapter. However, *scalar-matrix multiplication* is simple and intuitive.

Scalar-matrix multiplication means to multiply a matrix by a single number. It works by multiplying each matrix element by that scalarâ€”exactly the same as scalar-vector multiplication:

$$\delta \begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} \delta a & \delta b \\ \delta c & \delta d \end{bmatrix} = \begin{bmatrix} a\delta & b\delta \\ c\delta & d\delta \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \delta \tag{5.12}$$

Notice that because scalar-matrix multiplication is implemented element-wise, it is commutative. Each individual matrix element is simply a number, and scalar multiplication obeys the commutative law.

This means that you can move the scalar around, and the result is unchanged. That feature turns out to be crucial for several proofs and derivations. The following illustrates the idea.

$$\lambda\mathbf{AB} = \mathbf{A}\lambda\mathbf{B} = \mathbf{AB}\lambda$$

5.8 "Shifting" a matrix

We can combine matrix addition with scalar-matrix multiplication to apply a procedure called "shifting" a matrix. To "shift" a matrix means to add to the matrix a multiple of the identity matrix. Shifting is applied only to square matrices. The new, shifted, matrix is often denoted as **A** ; the tilde character on top indicates that the matrix is similar to, but not the same as, the original matrix **A** .

_____

_

Eqn. box title=**"Shifting" a matrix**

$$\mathbf{A} = \mathbf{A} + \hat{I}\!\!\raise.3ex\hbox{»}\,\mathbf{I}\;,\hat{A}\;\hat{A}\;\;\mathbf{A}\;\hat{a}\!\!\raise.3ex\hbox{˜}\;\hat{a}\!\!\raise.3ex\hbox{„}\qquad {}^{M}\,\tilde{A}\!\!\raise.3ex\hbox{—}\,{}^{M}\;,\hat{I}\!\!\raise.3ex\hbox{»}\;\hat{a}\!\!\raise.3ex\hbox{˜}\;\hat{a}\!\!\raise.3ex\hbox{„} \tag{5.13}$$

_____

_____

Here is an example with real numbers.

$$\begin{bmatrix} 1 & 3 & 0 \\ 1 & 3 & 0 \\ 2 & 2 & 7 \end{bmatrix} + .1 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1.1 & 3 & 0 \\ 1 & 3.1 & 0 \\ 2 & 2 & 7.1 \end{bmatrix}$$

There are three properties of matrix shifting that can be seen from this example:

1. Only the diagonal elements are affected; shifting does not change the off-diagonal elements. This is obvious from Equation 5.13 because the identity matrix has all zeros on the off-diagonal elements.
2. The first two rows of the example matrix are identical before shifting, and different after shifting. Thus, shifting a matrix can make matrices with redundant rows (or columns) distinct.
3. When $\hat{I}\!\!\raise.3ex\hbox{»}$ is close to zero, then **A** is really similar to **A** . Indeed, when $\hat{I}\!\!\raise.3ex\hbox{»} = 0$, then **A** = **A** . In practical applications, $\hat{I}\!\!\raise.3ex\hbox{»}$ is often selected to be as small as possible while large enough to satisfy other constraints.

Shifting a matrix has important applications in statistics, machine learning, deep learning, and other related fields. For example, a procedure called "regularization" involves shifting matrices. Regularization is a crucial operation

for solving systems of equations or fitting statistical models to low-rank data. When you learn about matrix rank (chapter 7), youâ€™ll see that shifting will transform any rank-deficient matrix into a full-rank matrix.

Code Shifting in code illustrates both scalar multiplication and addition. Also note the potential for confusion that the variable `l` (lower-case letter "L") can look like the number `1` and the upper-case letter "I."

Code blockÂ 5.13: Python

```
importÂ numpyÂ asÂ npÂ
lÂ =Â .01Â
IÂ =Â np.eye(4)Â
AÂ =Â np.random.randn(4,4)Â
AsÂ =Â AÂ +Â l*I
```

Code blockÂ 5.14: MATLAB

```
lÂ =Â .01;Â
IÂ =Â eye(4);Â
AÂ =Â randn(4);Â
AsÂ =Â AÂ +Â l*I;
```

–

Practice problems Shift the following matrices according to the specified *Î»* .

a) $\begin{bmatrix} .4 & 3 & 0 \\ 2 & 1 & 0 \\ .1 & .4 & 0 \end{bmatrix}$, $Î» = .1$

b) $\begin{bmatrix} 43 & 42 & 42 \\ 234 & 746 & 12 \\ 0 & 33 & 1001 \end{bmatrix}$, $Î» = â^’ 1$

Answers

a) $\begin{bmatrix} .5 & 3 & 0 \\ 2 & 1.1 & 0 \\ .1 & .4 & .1 \end{bmatrix}$

b)

$$\begin{bmatrix} 42 & 42 & 42 \\ 234 & 745 & 12 \\ 0 & 33 & 1000 \end{bmatrix}$$

–

5.9 Diagonal and trace

The diagonal elements of a matrix can be extracted and placed into a vector. This is used, for example, in statistics: the diagonal elements of a covariance matrix contain the variance of each variable. The following defines the procedure.

_____

Eqn. box title=**Extracting the diagonal elements into a vector**

$$\mathbf{v} = diag\ (\ \mathbf{A}\ )$$
$$v_{\ i} = a_{\ i,i}\ ,\ i = \{\ 1\ ,\ 2\ ,...,\ \min(\ M,N\ )\ \} \qquad (5.14)$$

Note that this formula does not require the matrix to be square. Observe the following examples (diagonal elements highlighted).

$$diag\left(\begin{bmatrix} 5 & 3 & 8 \\ 1 & 0 & 4 \\ 8 & 6 & 2 \end{bmatrix}\right) = \begin{bmatrix} 5 \\ 0 \\ 2 \end{bmatrix},\quad diag\left(\begin{bmatrix} 5 & 3 & 8 & 3 & 9 \\ 1 & 0 & 4 & 3 & 1 \\ 8 & 6 & 2 & 9 & 9 \end{bmatrix}\right) = \begin{bmatrix} 5 \\ 0 \\ 2 \end{bmatrix},\quad diag\left(\begin{bmatrix} 5 & 3 \\ 1 & 0 \\ 8 & 6 \\ 3 & 5 \\ 9 & 7 \\ 8 & 2 \end{bmatrix}\right) = \begin{bmatrix} 5 \\ 0 \end{bmatrix}$$

Trace The trace is an operation that produces a single number from a squre matrix. It is indicated as *tr* (**A**) and is defined as the sum of all diagonal elements of a matrix:

–

Eqn. box title=**Trace is the sum of diagonal elements**

$$tr\ (\ A\ ) = \hat{a}\hat{\ }\text{‘}_{\ i\ =1}^{\ M}\ a_{\ i,i} \qquad (5.15)$$

Notice that the off-diagonal elements do not contribute to the trace; thus, two very different matrices (different sizes, different off-diagonal elements) can have the same trace.

The trace operation has two applications in machine learning: It is used to compute the Frobenius norm of a matrix (a measure of the magnitude of a matrix) and it is used to measure the "distance" between two matrices.

IMPORTANT The trace is defined only for square matrices. This may seem strange, considering that the trace is the sum of the diagonal elements and the diagonal exists for rectangular matrices. The reason for this rule has to do with a property of eigendecomposition: The trace of a matrix equals the sum of its eigenvalues. Eigendecomposition is valid only on square matrices, and so ancient and wise linear algebraticians decreed that only square matrices can have a trace.

Code Extracting the diagonal from a matrix was shown on page Â§ .

Code blockÂ 5.15: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.random.randn(4,4)Â
trÂ =Â np.trace(A)
```

Code blockÂ 5.16: MATLAB

```
AÂ =Â randn(4);Â
trÂ =Â trace(A);
```

–

Practice problems Compute the trace of the following matrices.

a) $\begin{bmatrix} -3 & 8 & 0 \\ 0 & -3 & -6 \\ -2 & 3 & -3 \end{bmatrix}$

b) $\begin{bmatrix} -3 & -5 & 4 \\ 1 & -3 & 1 \\ 6 & -1 & 10 \end{bmatrix}$

c) $\begin{bmatrix} 8 & -8 & 9 \\ 7 & -3 & 5 \\ 0 & 5 & -5 \end{bmatrix}$

Answers

a) âˆ’ 9
b) 4
c) 0

5.10 Exercises

1.Â

For the following matrix and vectors, solve the given arithmetic problems, or state why they are not solvable.

$$\mathbf{u} = \begin{bmatrix} 2 \\ 4 \\ 1 \end{bmatrix}, \mathbf{v} = \begin{bmatrix} 3 \\ 5 \\ 0 \\ 2 \end{bmatrix}, \mathbf{w} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 5 \end{bmatrix}, \mathbf{A} = \begin{bmatrix} -1 & -2 & -6 & -6 \\ 1 & -1 & 0 & -2 \\ -1 & 0 & -1 & -4 \end{bmatrix}$$

a) $\mathbf{w}\mathbf{u}^T + \mathbf{A}$
b) $\mathbf{w}\mathbf{u}^T + \mathbf{A}^T$
c) $\mathbf{u}\mathbf{v}^T - \mathbf{A}$
d) $\mathbf{v}\mathbf{w}^T - \mathbf{A}$
e) $\mathbf{v}\mathbf{w}^T + \mathbf{A}^T$

2.Â

Perform the following matrix operations, when the operation is valid.

$$\mathbf{A} = \begin{bmatrix} 2 & 4 & 3 \\ 0 & 1 & 3 \end{bmatrix}, \mathbf{B} = \begin{bmatrix} -2 & -1 & 3 \\ 6 & -7 & 7 \end{bmatrix},$$

$$\mathbf{C} = \begin{bmatrix} 0 & -6 \\ -3 & -2 \\ -2 & 7 \end{bmatrix} \mathbf{D} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 2 & 4 \end{bmatrix}$$

a) $\mathbf{A} + 3\mathbf{B}$
b) $\mathbf{A} + \mathbf{C}$
c) $\mathbf{C} - \mathbf{D}$
d) $\mathbf{D} + \mathbf{C}$
e) $\mathbf{A}^T + \mathbf{D}$
f) $(\mathbf{A} + \mathbf{B})^T + 2\mathbf{C}$

g) $3\,\mathbf{A} + (\mathbf{B}^{T} + \mathbf{C})^{T}$

h) â^' $4(\mathbf{A}^{T} + \mathbf{C})^{T} + \mathbf{D}$

3.Â

An NÃ— N matrix has N^2 elements. For a symmetric matrix, however, not all elements are unique. Create 2Ã— 2 and 3Ã— 3 symmetric matrices and count the number of total elements and the number of possible unique elements. Then work out a formula for the number of possible unique elements in such a matrix.

4.Â

Identify the following types of matrices from the list provided in the section "A zoo of matrices." Note that some matrices can be given multiple labels.

a) $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 6 \end{bmatrix}$

d) $\begin{bmatrix} a & b & c \\ -b & d & e \\ -c & -e & f \end{bmatrix}$

e) $\begin{bmatrix} 0 & b & c \\ -b & 0 & f \\ -c & -e & 0 \end{bmatrix}$

f) $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 32 & 0 & 0 \\ 0 & 0 & 42 & 0 \end{bmatrix}$

5.Â

To "decompose" a matrix means to represent a matrix using the sum or product of other matrices. Letâ€™s consider an additive decomposition, thus starting from some matrix **A** and setting **A** = **B** + **C** . You can also use more matrices: **A** = **B** + ... + **N** . Decompose the following matrices **A** . Are your decompositions unique? (A decomposition is "unique" if it has exactly one solution.)

$$\begin{bmatrix} 1 & 3 \\ 2 & 5 \end{bmatrix}, \quad \begin{bmatrix} 1 & 3 & 3 \\ 2 & 5 & 7 \end{bmatrix}$$

6.Â

Create a Hankel matrix from the vector

$$\begin{bmatrix} 1 & 2 & 3 & 1 & 2 & 3 \end{bmatrix}$$

7.Â

Determine whether the following matrices are orthogonal.

a) $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{bmatrix}$

b) $\frac{1}{\sqrt{9}} \begin{bmatrix} 1 & 2 \\ -2 & 2 \\ -2 & 1 \end{bmatrix}$

c) $\mathbf{I}_{17}$

8.Â

Consider two *M* Ã— *N* matrices **A** and **B** . Is the sum of their traces equal to the trace of their sum? That is, *tr* (**A**) + *tr* (**B**) = *tr* (**A** + **B**) ? Try for a few examples, then see if you can work out a proof for this property.

a) $\begin{bmatrix} 3 & -4 \\ 3 & -9 \end{bmatrix}, \begin{bmatrix} 4 & 4 \\ 6 & 0 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 & 5 \\ -5 & 0 & 5 \\ -9 & 4 & 3 \end{bmatrix}, \begin{bmatrix} 5 & 5 & 5 \\ 4 & 3 & 4 \\ 1 & 1 & 9 \end{bmatrix}$

c) $\begin{bmatrix} a & d & e \\ f & b & g \\ h & i & c \end{bmatrix}, \begin{bmatrix} j & m & n \\ o & k & p \\ q & r & l \end{bmatrix}$

9.Â

Hereâ€™s another neat property of the trace: The trace of the outer product is equal to the dot product: *tr* ($\mathbf{vw}^{\mathrm{T}}$) = $\mathbf{v}^{\mathrm{T}}\mathbf{w}$. Demonstrate this property using the following sets of vectors.

a) $\begin{bmatrix} 3 \\ -5 \\ 1 \end{bmatrix} \begin{bmatrix} -2 \\ 1 \\ 5 \end{bmatrix}$

b)

$$\begin{bmatrix} 1 \\ -3 \\ 5 \\ 2 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \\ 1 \\ 4 \end{bmatrix}$$

c) $$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} \begin{bmatrix} e \\ f \\ g \\ h \end{bmatrix}$$

10.Â

Perform the indicated matrix operations using the following matrices and scalars. Determine the underlying principle regarding trace, matrix addition, and scalar multiplication.

$$\mathbf{A} = \begin{bmatrix} 5 & -3 \\ 2 & -3 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} -4 & -1 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} a & c \\ b & d \end{bmatrix}, \quad \lambda = 5, \quad \alpha = -3$$

a) *tr* (**A**)
b) *tr* (**B**)
c) *tr* (**A** + **B**)
d) *tr* (*Î»* **C**)
e) *Î»* *tr* (**C**)
f) *Î»* *tr* (*Î±* **C**)
g) *Î±* *tr* (*Î»* **C**)
h) *tr* (*Î±* **A** + *Î»* **B**)
i) (*Î»Î±*) *tr* (**A** + **B**)
j) *tr* (*Î»* **A** + *Î»* **B**)
k) *Î»* *tr* (**A** + **B**)
l) *tr* (**A** + **B** ^{T})

5.11 Answers

1.Â

a) Size mismatch.
b) $$\begin{bmatrix} 1 & 5 & 0 \\ -2 & -1 & 0 \\ -4 & 4 & 0 \\ 4 & 18 & 1 \end{bmatrix}$$
c)

$$\begin{bmatrix} 7 & 12 & 6 & 10 \\ 11 & 21 & 0 & 10 \\ 4 & 5 & 1 & 6 \end{bmatrix}$$

d) Size mismatch.

e) Size mismatch.

2.Â

a) $\begin{bmatrix} -4 & 1 & 12 \\ 18 & -20 & 24 \end{bmatrix}$

b) Invalid.

c) $\begin{bmatrix} -1 & -8 \\ -6 & -6 \\ -4 & 3 \end{bmatrix}$

d) $\begin{bmatrix} 1 & -4 \\ 0 & 2 \\ 0 & 11 \end{bmatrix}$

e) $\begin{bmatrix} 3 & 2 \\ 7 & 5 \\ 5 & 7 \end{bmatrix}$

f) $\begin{bmatrix} 0 & -6 \\ -3 & -10 \\ 2 & 24 \end{bmatrix}$

g) $\begin{bmatrix} 4 & 8 & 10 \\ 0 & -6 & 23 \end{bmatrix}$

h) Invalid.

3.Â

$n(n+1)/2$

4.Â

a) square, diagonal, symmetric

b) square, upper-triangular

c) symmetric

d) square, skew-symmetric if $a = d = f = 0$

e) square, skew-symmetric if $e = f$

f) rectangular, diagonal

5.Â
The additive decomposition is definitely not unique; there is an infinity of possible matrices that can sum to produce a given **A** . One interesting solution is to create matrices of all zeros and one non-zero element. For example:

$$\begin{bmatrix} 1 & 3 \\ 2 & 5 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 3 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 2 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 5 \end{bmatrix}$$

The reason why this is an interesting decomposition is that the four matrices are convenient *basis matrices* for the vector space â„ 2 Ã— 2
6.Â

$$\begin{bmatrix} 1 & 2 & 3 & 1 & 2 & 3 \\ 2 & 3 & 1 & 2 & 3 & 1 \\ 3 & 1 & 2 & 3 & 1 & 2 \\ 1 & 2 & 3 & 1 & 2 & 3 \\ 2 & 3 & 1 & 2 & 3 & 1 \\ 3 & 1 & 2 & 3 & 1 & 2 \end{bmatrix}$$

7.Â

- **a)** No, the third column does not have magnitude = 1.
- **b)** No, because columns are not orthogonal.
- **c)** Yes, all identities matrices are orthogonal.

8.Â
This property (sum of traces equals trace of sums) holds because of the element-wise definition of both trace and matrix addition:

$$\sum_{i=1}^{M} a_{i,i} + \sum_{i=1}^{M} b_{i,i} = \sum_{i=1}^{M} (a_{i,i} + b_{i,i})$$

- **a)** -2 in both cases
- **b)** 21 in both cases
- **c)** *a* + *b* + *c* + *j* + *k* + *l* in both cases

9.Â

- **a)** -6
- **b)** 0
- **c)** *ae* + *bf* + *cg* + *dh*

10.Â
The underlying principle here is the same as in question 8: The trace is a linear operator, so you can scalar-multiply and sum, and trace remains the same.

a) 2
b) -1
c) 1
d) 5 *a* + 5 *d*
e) 5(*a* + *d*)
f) 5(âˆ’ 3 *a* âˆ’ 3 *d*)
g) âˆ’ 3(5 *a* + 5 *d*)
h) -11
i) -15
j) 5
k) 5
l) 1

5.12 Code challenges

1.Â
The goal of this exercise is to create a matrix that contains the dot products between all pairs of columns in two other matrices. First, create two 4x2 matrices (that is, 4 rows, 2 columns) of random numbers. Then write a double-for loop in which you compute the dot products between each column of both matrices. The *i*,*j* element of the resulting matrix will be the dot product between column *i* of the first matrix and column *j* of the second matrix.

2.Â
Create a symmetric matrix by starting with a dense random numbers matrix and applying three matrix operations: convert to triangular, transpose, matrix addition.

3.Â
Create a diagonal matrix of size 4x8 without using the `diag()` function. The diagonal elements should be 1,2,3,4. How much of your code would you need to change to create an 8x4 diagonal matrix?

5.13 Code solutions

1.Â

The purpose of this challenge is to get you thinking about the mechanics of matrix multiplication.

Code blockÂ 5.17: Python

```
AÂ =Â np.random.randn(4,2)Â
BÂ =Â np.random.randn(4,2)Â
CÂ =Â np.zeros((2,2))Â
forÂ coliÂ inÂ range(2):Â
Â Â forÂ coljÂ inÂ range(2):Â
Â Â Â Â C[coli,colj]=np.dot(A[:,coli],B[:,colj])
```

Code blockÂ 5.18: MATLAB

```
AÂ =Â randn(4,2);Â
BÂ =Â randn(4,2);Â
CÂ =Â zeros(2);Â
forÂ coli=1:2Â
Â Â forÂ colj=1:2Â
Â Â Â Â C(coli,colj)Â =Â dot(A(:,coli),B(:,colj));Â
Â Â endÂ
end
```

2.Â

You just discovered one way to make a symmetric matrix!

Code blockÂ 5.19: Python

```
AÂ =Â np.random.randn(4,4)Â
AlÂ =Â np.tril(A)Â
SÂ =Â AlÂ +Â Al.T
```

Code blockÂ 5.20: MATLAB

```
AÂ =Â randn(4);Â
AlÂ =Â tril(A);Â
SÂ =Â AlÂ +Â Alâ€™;
```

3.Â

The point here is to appreciate that indexing the diagonal of a matrix involves the *i*,*i* indices.

Code blockÂ 5.21: Python

```
DÂ =Â np.zeros((4,8))Â
forÂ dÂ inÂ range(min(D.shape)):Â
Â Â Â Â D[d,d]Â =Â d+1
```

Code blockÂ 5.22: MATLAB

```
DÂ =Â zeros(4,8);Â
forÂ d=1:min(size(D))Â
Â Â Â Â D(d,d)Â =Â d;Â
end
```

ChapterÂ 6
Matrix multiplication

Multiplying matrices is considerably more complicated than multiplying regular numbers. The normal rules you know about multiplication (e.g., a Ã— b = b Ã— a) often donâ€™t apply to matrices, and there are extra rules you will need to learn. Furthermore, there are several ways of multiplying matrices. And to make matters more complicated, not all pairs of matrices can be multiplied. So, take a deep breath, because this chapter is your first deep-dive into linear algebra!

6.1 "Standard" matrix multiplication

For lack of a better term, this method will be called the "standard" method. Unless otherwise explicitly stated, you can assume (in this book and elsewhere) that two matrices next to each other (like this: **AB**) indicates "standard" matrix multiplication.

Terminology The first thing to know about matrix multiplication is that it is not commutative, so **AB** *â‰ * **BA** . There are exceptions where this is the case (for example, **AI** = **IA**), but those exceptions are rare. For this reason, even the terminology of matrix multiplication is complicated.

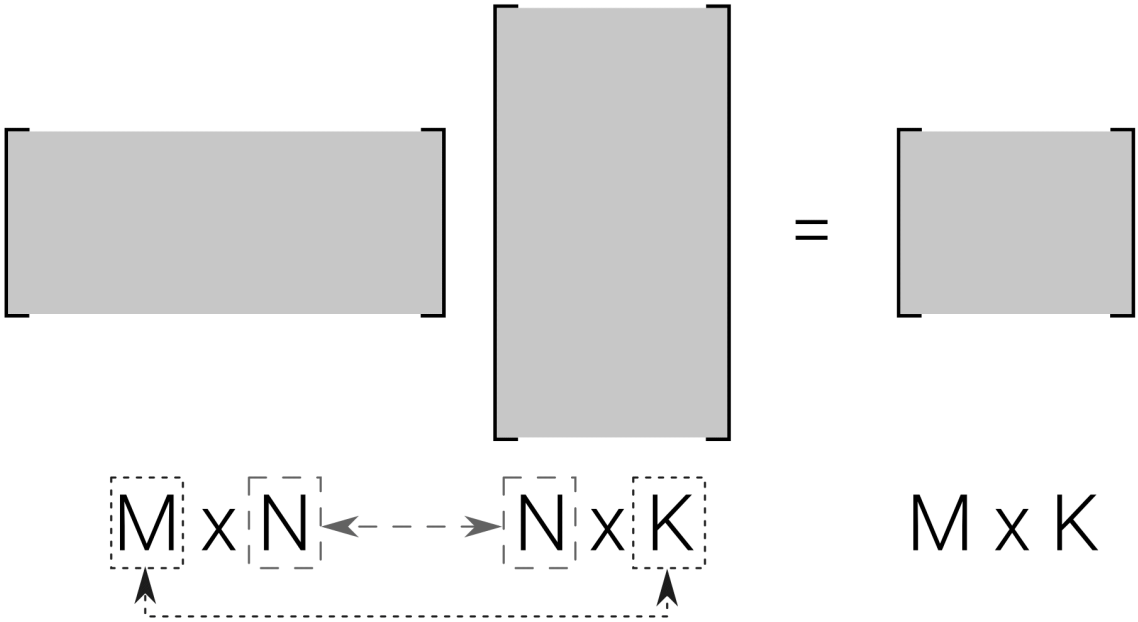
The following five statements are ways to say the operation **AB** out loud (e.g., when trying to show off to your math-inclined friends and family):

"**A** times **B** "
"**A** left-multiplies **B** "
"**A** pre-multiplies **B** "
"**B** right-multiplies **A** "
"**B** post-multiplies **A** "

Validity Before learning how standard matrix multiplication works, you need to learn when matrix multiplication *is valid* . The rule for multiplication validity is simple and visual, and you need to memorize this rule before learning the mechanics of multiplication.

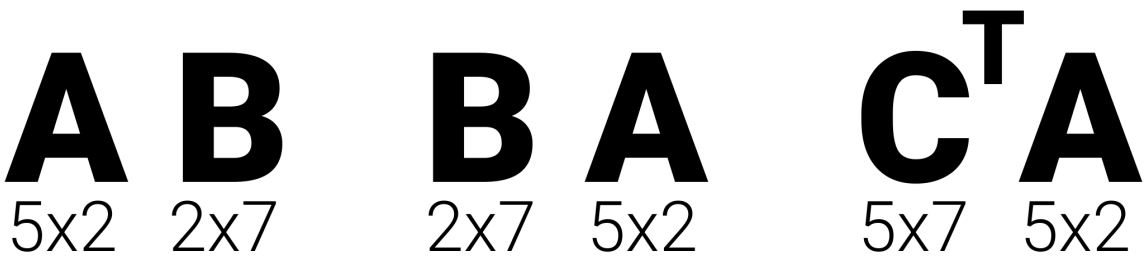
If you write the matrix sizes underneath the matrices, then matrix multiplication is valid only when the two "inner dimensions" match, and the size of the resulting

product matrix is given by the "outer dimensions." By "inner" and "outer" Iâ€™m referring to the spatial organization of the matrix sizes, as in Figure 6.1 .



FigureÂ 6.1: Visualization of the rule for matrix multiplication validity. Note the reference to "inner dimensions" (N) and "outer dimensions" (M and K).

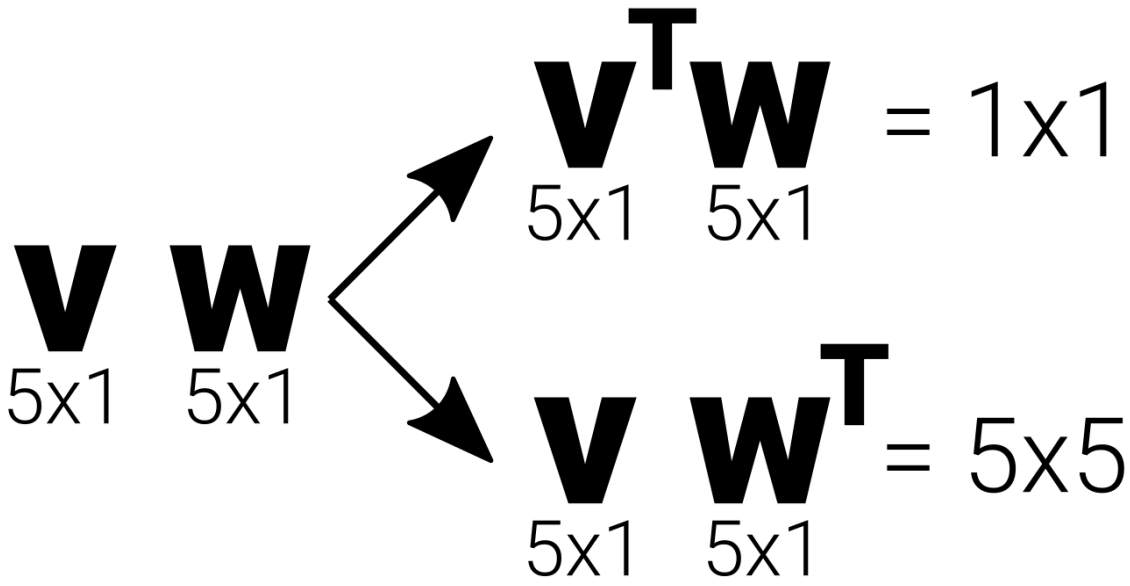
Consider the following matrices; are these multiplications valid?



The first pair (**AB**) is valid because the "inner" dimensions match (both 2). The resulting product matrix will be of size 5 Ã— 7 . The second pair shows the lack of commutativity in matrix multitplication: The "inner" dimensions (7 and 5) do not match, and thus the multiplication is not valid. The third pair is an interesting case. You might be tempted to call this an invalid operation; however, when transposing **C** , the rows and columns swap, and so the "inner" dimensions become consistent (both 5). So this multiplication is valid.

Hereâ€™s something exciting: you are now armed with the knowledge to understand the notation for the dot product and outer product. In particular, you

can now appreciate why the order of transposition ($\mathbf{v}^T\mathbf{w}$ or $\mathbf{v}\mathbf{w}^T$) determines whether the multiplication is the dot product or the outer product (Figure 6.2).



FigureÂ 6.2: For two adjacent column vectors, transposing the first vs. the second vector is the difference between the dot product vs. the outer product.

Practice problems For the following matrices, determine whether matrix multiplication is
valid, and, if so, the size of the product matrix.

$$\mathbf{A} \in \mathbb{R}^{2\times4}, \quad \mathbf{B} \in \mathbb{R}^{3\times4}, \quad \mathbf{C} \in \mathbb{R}^{4\times4}$$

a) **AB**
b) **AC**
c) $\mathbf{AB}^T$
d) $\mathbf{BCA}^T$
e) $\mathbf{BB}^T\mathbf{A}$

Answers

a) no
b) 2 Ã— 4
c) 2 Ã— 3
d) 3 Ã— 2
e) no

Code Unfortunately, matrix multiplication is confusingly different between MATLAB and Python. Pay close attention to the subtle but important differences (@ vs. *).

Code blockÂ 6.1: Python

```
M1Â =Â np.random.randn(4,3)Â
M2Â =Â np.random.randn(3,5)Â
CÂ =Â M1Â @Â M2
```

Code blockÂ 6.2: MATLAB

```
M1Â =Â randn(4,3);Â
M2Â =Â randn(3,5);Â
CÂ =Â M1Â *Â M2
```

It is now time to learn how to multiply matrices. There are four ways to think about and implement matrix multiplication. All four methods give the same result, but provide different perspectives on what matrix multiplication means. Itâ€™s useful to understand all of these perspectives, because they provide insights into matrix computations in different contexts and for different problems. It is unfortunate that many linear algebra textbooks teach only the dot-product method (what I call the "element perspective").

(1) The "element perspective" Each element $c_{i,j}$ in **AB** = **C** is the dot product between the ith row in **A** and the jth column in **B** . The equation below shows how you would create the top-left element in the product matrix.

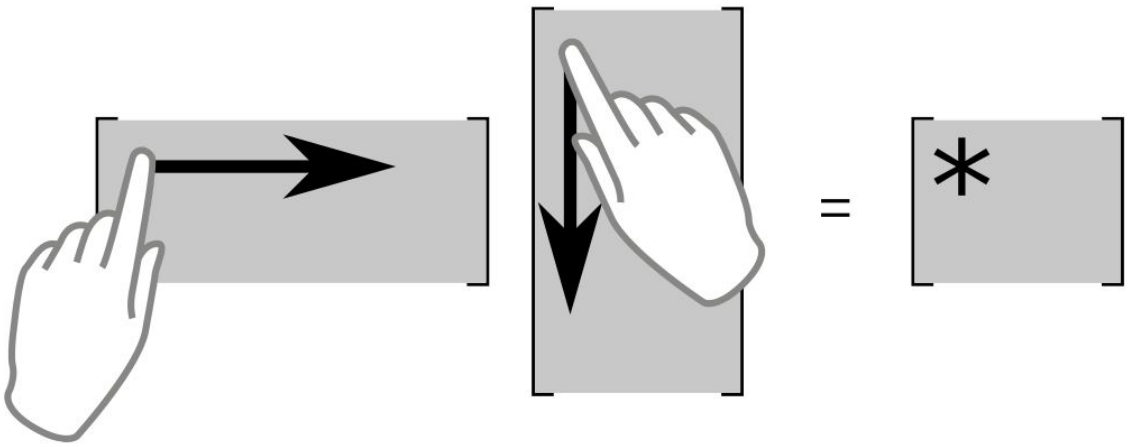
[1] [1]For convenience, I will label all matrices in the following examples as $\mathbf{AB} = \mathbf{C}$.

$$\begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}\begin{bmatrix}a & b\\ c & d\end{bmatrix} = \begin{bmatrix}1a+2c & \\ & \end{bmatrix} \quad (6.1)$$

This makes it clear that element $c_{i,j}$ comes from the dot product between row $\mathbf{a}_i$ and column $\mathbf{b}_j$.

Here is another example; make sure you see how each element in the product matrix is the dot product between the corresponding row or the left matrix and column in the right matrix.

$$\begin{bmatrix}3 & 4\\ -1 & 2\\ 0 & 4\end{bmatrix}\begin{bmatrix}5 & 1\\ 3 & 1\end{bmatrix} = \begin{bmatrix}3\cdot5+4\cdot3 & 3\cdot1+4\cdot1\\ -1\cdot5+2\cdot3 & -1\cdot1+2\cdot1\\ 0\cdot5+4\cdot3 & 0\cdot1+4\cdot1\end{bmatrix} = \begin{bmatrix}27 & 7\\ 1 & 1\\ 12 & 4\end{bmatrix}$$

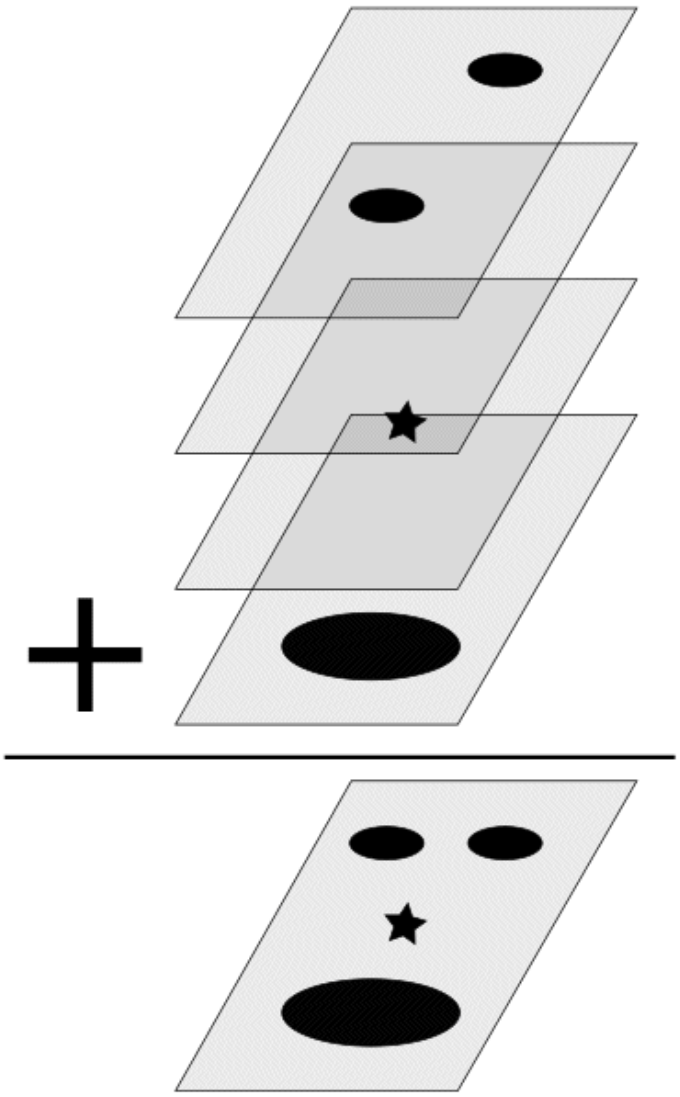


FigureÂ 6.3: Visual representation of the mechanism of computing each element of the matrix multiplication product as the vector dot product between each row of the left matrix (from left-to-right and each column of the right matrix (from top-to-bottom).

There is a hand gesture that you can apply to remember this rule: extend your pointer fingers of both hands; simultaneously move your left hand from left to right (across the row of the left matrix) while moving your right hand down towards you (down the column of the right matrix) (Figure 6.3).

Below you can see another visualization of matrix multiplication from the element perspective. This visualization facilitates three important features of matrix multiplication.

$$\begin{bmatrix} — & a_1 & — \\ — & a_2 & — \\ & \vdots & \\ — & a_n & — \end{bmatrix} \begin{bmatrix} | & | & & | \\ b_1 & b_2 & \cdots & b_n \\ | & | & & | \end{bmatrix} = \begin{bmatrix} a_1 \cdot b_1 & a_1 \cdot b_2 & \cdots & a_1 \cdot b_n \\ a_2 \cdot b_1 & a_2 \cdot b_2 & & a_2 \cdot b_n \\ \vdots & & \ddots & \vdots \\ a_n \cdot b_1 & a_n \cdot b_2 & \cdots & a_n \cdot b_n \end{bmatrix}$$



FigureÂ 6.4: A simulacrum of building up a matrix one layer at a time. Each layer is the same size as the product yet provides only partial information.

- The diagonal of the product matrix **C** contains dot products between rows and columns of the same ordinal position (row *i* in **A** and column *i* in **B**).
- The lower-triangle of **C** contains dot products between *later* rows in **A** and *earlier* columns in **B** (row *i* in **A** and column *j* in **B** , where $i > j$).
- The upper-triangle of **C** contains dot products between *earlier* rows in **A** and *later* columns in **B** (row *i* in **A** and column *j* in **B** , where $i < j$).

The first point above is relevant for understanding data covariance matrices. The second and third points are important for understanding several matrix

decompositions, most importantly QR decomposition and generalized eigendecomposition.

(2) The "layer perspective" In contrast to the element perspective, in which each element is computed independently of each other element, the layer perspective involves conceptualizing the product matrix as a series of layers, or "sheets," that are summed together. This is implemented by creating outer products from the columns of **A** and the rows of **B** , and then summing those outer products together.

Remember that the outer product is a matrix. Each outer product is the same size as **C** , and can be thought of as a layer. By analogy, imagine constructing an image by laying transparent sheets of paper on top of each other, with each sheet containing a different part of the image (Figure 6.4).

Below is an example using the same matrix as in the previous section. Make sure you understand how the two outer product matrices are formed from column $\mathbf{a}_i$ and row $\mathbf{b}_j$. You can also use nearly the same hand gesture as with the element perspective (Figure 6.3), but swap the motions of the left and right hands.

$$\begin{bmatrix} 3 & 4 \\ -1 & 2 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 5 & 1 \\ 3 & 1 \end{bmatrix} = \begin{bmatrix} 15 & 3 \\ -5 & -1 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 12 & 4 \\ 6 & 2 \\ 12 & 4 \end{bmatrix} = \begin{bmatrix} 27 & 7 \\ 1 & 1 \\ 12 & 4 \end{bmatrix}$$

[2] Notice that in each of the "layer matrices," the columns form a dependent set (the same can be said of the rows). However, the sum of these singular matricesâ€”the product matrixâ€”has columns that form a linearly *in* dependent set.

[2] Each of these layers is a rank-1 matrix. Rank will be discussed in more detail in a separate chapter, but for now, you can think of a rank-1 matrix as containing only a single columnâ€™s worth of information; all the other columns are scaled versions.

The layer perspective of matrix multiplication is closely related to the spectral theorem of matrices, which says that any matrix can be represented as a sum of rank-1 matrices. Itâ€™s like each rank-1 matrix is a single color and the matrix is the rainbow. This important and elegant idea is the basis for the singular value decomposition, which you will learn about in Chapter 16.

(3) The "column perspective" From the column perspective, all matrices (the multiplying matrices and the product matrix) are thought of as sets of column vectors. Then the product matrix is created one column at a time.

The first column in the product matrix is a linear weighted combination of all columns in the left matrix, where the weights are defined by the elements in the

first column of the right matrix. The second column in the product matrix is again a weighted combination of all columns in the left matrix, except that the weights now come from the second column in the right matrix. And so on for all *N* columns in the right matrix. Letâ€™s start with a simple example:

$$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}\begin{bmatrix}a & b\\c & d\end{bmatrix} = \begin{bmatrix} a\begin{bmatrix}1\\3\end{bmatrix} + c\begin{bmatrix}2\\4\end{bmatrix} & b\begin{bmatrix}1\\3\end{bmatrix} + d\begin{bmatrix}2\\4\end{bmatrix}\end{bmatrix} \tag{6.2}$$

Letâ€™s go through Equation 6.2 slowly. The first column of the product matrix is the sum of all columns in matrix **A** . But itâ€™s not just the columns added togetherâ€”each column in **A** is weighted according to the corresponding element from the first column of matrix **B** . Then, the second column of matrix **C** is created by again summing all of the columns in matrix **A** , except now each column is weighted by a different element of column 2 from matrix **B** . Equation 6.2 shows only two columns, but this procedure would be repeated for however many columns are in matrix **B** .

Now for the same numerical example youâ€™ve seen in the previous two perspectives:

$$\begin{bmatrix}3 & 4\\-1 & 2\\0 & 4\end{bmatrix}\begin{bmatrix}5 & 1\\3 & 1\end{bmatrix} = \begin{bmatrix} 5\begin{bmatrix}3\\-1\\0\end{bmatrix} + 3\begin{bmatrix}4\\2\\4\end{bmatrix} & 1\begin{bmatrix}3\\-1\\0\end{bmatrix} + 1\begin{bmatrix}4\\2\\4\end{bmatrix}\end{bmatrix} = \begin{bmatrix}27 & 7\\1 & 1\\12 & 4\end{bmatrix}$$

The column perspective of matrix multiplication is useful in statistics, when the columns of the left matrix contain a set of *regressors* (a simplified model of the data), and the right matrix contains *coefficients* . The coefficients encode the importance of each regressor, and the goal of statistical model-fitting is to find the best coefficients such that the weighted combination of regressors matches the data. More on this in Chapter 14!

(4) The "row perspective" You guessed itâ€”itâ€™s the same concept as the column perspective but you build up the product matrix one row at a time, and everything is done by taking weighted combinations of rows. Thus: each *row* in the product matrix is the weighted sum of all *rows* in the right matrix, where the weights are given by the elements in each *row* of the left matrix. Letâ€™s begin with the simple example:

$$\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}\begin{bmatrix}a & b\\c & d\end{bmatrix} = \begin{bmatrix}1\begin{bmatrix}a & b\end{bmatrix} + 2\begin{bmatrix}c & d\end{bmatrix}\\ \\3\begin{bmatrix}a & b\end{bmatrix} + 4\begin{bmatrix}c & d\end{bmatrix}\end{bmatrix} \tag{6.3}$$

The top row of the product matrix is created by summing together the two rows of the right matrix, but each row is weighted according to the corresponding element

of the top row of the left matrix. Same story for the second row. And of course, this would continue for however many rows are in the left matrix.

I wonâ€™t repeat the other example multiplication Iâ€™ve been showing in previous pages; thatâ€™s for you to do on your own with pencil and paper. (Hint: The result will be identical.)

The row perspective is useful, for example in principal components analysis, where the rows of the right matrix contain data (observations in rows and features in columns) and the rows of the left matrix contain weights for combining the features. Then the weighted sum of data creates the principal component scores.

Figure 6.5 (next page) visually summarizes the different perspectives. _____

Practice problems Multiply the following pairs of matrices four times, using each of four perspectives. Make sure you get the same result each time.

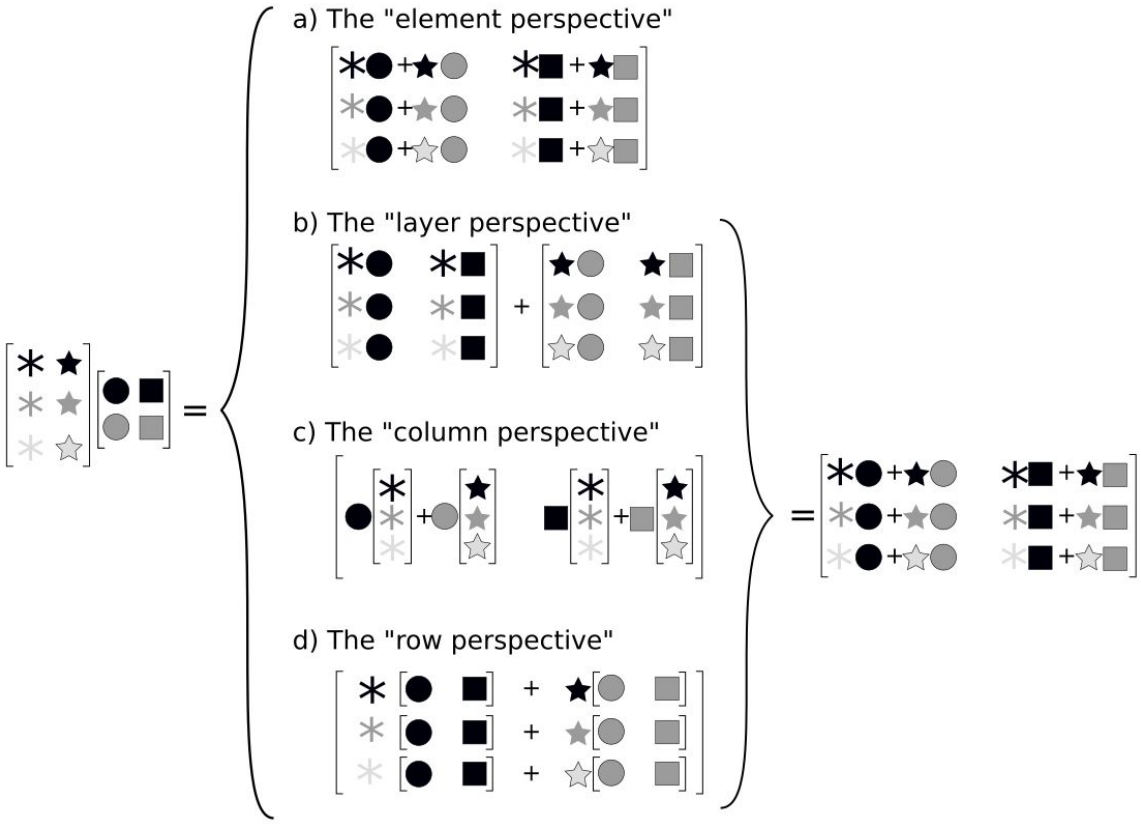
a) $\begin{bmatrix} 3 & 0 & 3 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 4 & 4 \\ 0 & 1 \\ 4 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$

Answers

a) $\begin{bmatrix} 24 & 15 \\ 4 & 5 \end{bmatrix}$

b) $\begin{bmatrix} 1a+2d+3g & 1b+2e+3h & 1c+2f+3i \\ 4a+5d+6g & 4b+5e+6h & 4c+5f+6i \\ 7a+8d+9g & 7b+8e+9h & 7c+8f+9i \end{bmatrix}$



FigureÂ 6.5: Visual representation of the four perspectives on matrix multiplication.

Practice problems Perform the following matrix multiplications. Use whichever perspective you find most confusing (thatâ€™s the perspective you need to practice!).

a) $\begin{bmatrix} 3 & 4 & 0 \\ 0 & 4 & 1 \end{bmatrix} \begin{bmatrix} 2 & -1 & 2 \\ 4 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 3 & 2 \\ 0 & 1 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 22 & -3 & 6 \\ 16 & 1 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 3 & 3 \\ 3 & 4 \end{bmatrix}$

6.2 Multiplication and equations

You know from your high-school algebra course that you are allowed to add and multiply terms to an equation, as long as you apply that operation to both sides. For example, I can multiply both sides of this equation by 7 :

$$4 + x = 5(y + 3)$$
$$7(4 + x) = 7(5(y + 3))$$
$$7(4 + x) = (5(y + 3))7$$

Notice that the bottom two equations are the same: Because scalars obey the commutative law, the 7 can go on the left or the right of the parenthetic term.

However, because matrix multiplication is not commutative, you need to be mindful to put the matrices on the same side of the equation. For example, the following is OK:

$$^{3}\mathbf{B} = \hat{I}\text{»}\,(\mathbf{C} + \mathbf{D})$$
$$\mathbf{AB} = \hat{I}\text{»}\,\mathbf{A}\,(\mathbf{C} + \mathbf{D})$$
$$\mathbf{AB} = \mathbf{A}\,(\mathbf{C} + \mathbf{D})\,\hat{I}\text{»}$$

The $\hat{I}$» can be moved around because it is a scalar, but **A** must pre-multiply both sides (or post-multiply both sides, but it must be consistent on the left- and right-hand sides of the equation). In contrast to the above, the following progression of equations is **WRONG** .

[3] Matrix sizes are not stated, so assume that the sizes make the operations valid.

$$\mathbf{B} = \hat{I}\text{»}\,(\mathbf{C} + \mathbf{D})$$
$$\mathbf{AB} = \hat{I}\text{»}\,(\mathbf{C} + \mathbf{D})\,\mathbf{A}$$

In other words, if you pre-multiply on one side of an equation, you must pre-multiply on the other side of the equation. Same goes for post-multiplication. If there are rectangular matrices in the equation, it is possible that pre- or post-multiplying isnâ€™t even valid.

Code The non-commutivity of matrix multiplication is easy to confirm in code. Compare matrices C1 and C2.

Code blockÂ 6.3: Python

```
AÂ =Â np.random.randn(2,2)Â
BÂ =Â np.random.randn(2,2)Â
C1Â =Â A@BÂ
C2Â =Â B@A
```

Code blockÂ 6.4: MATLAB

```
AÂ =Â randn(2,2);Â
BÂ =Â randn(2,2);Â
C1Â =Â A*B;Â
C2Â =Â B*A;
```

6.3 Matrix multiplication with a diagonal matrix

There is a special property of multiplication when one matrix is a diagonal matrix and the other is a dense matrix:

- Pre-multiplication by a diagonal matrix scales the *rows* of the right matrix by the diagonal elements.
- Post-multiplication by a diagonal matrix scales the *columns* of the left matrix by the diagonal elements.

Letâ€™s see two examples of 3 Ã— 3 matrices; notice how the diagonal elements appear in the rows (pre-multiply) or the columns (post-multiply). Also notice how the product matrix is the same as the dense matrix, but either the columns or the rows are scaled by each corresponding diagonal element of the diagonal matrix.

$$\begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & c \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 1a & 2a & 3a \\ 4b & 5b & 6b \\ 7c & 8c & 9c \end{bmatrix} \tag{6.4}$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & c \end{bmatrix} = \begin{bmatrix} 1a & 2b & 3c \\ 4a & 5b & 6c \\ 7a & 8b & 9c \end{bmatrix} \tag{6.5}$$

Of course, the mechanism of matrix multiplication is exactly the same as what you learned in the previous section. But all the zeros in the diagonal matrices allow us to simply things a bit.

It is worth remembering this property of diagonal matrix multiplication, because you will see applications of this in several subsequent chapters, including systems of equations (Chapter 10), diagonalization (Chapter 15), and singular-value decomposition (Chapter 16). I tried to come up with an easy mnemonic for remembering this rule. It is, admittedly, not such a great mnemonic, but you should take this as a challenge to come up with a better one. _____

Eqn. box title=**Order of matrices for modulating rows vs. columns**

P**R** e-multiply to affect **R** ows

P**O** st-multiply to affect c**O** lumns.

Practice problems Perform the following matrix multiplications. Note the differences between **a** and **c** , and between **b** and **d** .

a) $\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 5 & 0 \\ 0 & 6 \end{bmatrix} \begin{bmatrix} 4 & 3 \\ 3 & 4 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$

d) $\begin{bmatrix} 4 & 3 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 0 \\ 0 & 6 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 2 & 2 \\ 3 & 3 \end{bmatrix}$

b) $\begin{bmatrix} 20 & 15 \\ 18 & 24 \end{bmatrix}$

c) $\begin{bmatrix} 2 & 3 \\ 2 & 3 \end{bmatrix}$

d) $\begin{bmatrix} 20 & 18 \\ 15 & 24 \end{bmatrix}$

Multiplying two diagonal matrices The product of two diagonal matrices is another diagonal matrix whose diagonal elements are the products of the corresponding diagonal elements. Thatâ€™s a long sentence, but itâ€™s a simple concept. Hereâ€™s an example:

$$\begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & c \end{bmatrix} \begin{bmatrix} d & 0 & 0 \\ 0 & e & 0 \\ 0 & 0 & f \end{bmatrix} = \begin{bmatrix} ad & 0 & 0 \\ 0 & be & 0 \\ 0 & 0 & cf \end{bmatrix} \tag{6.6}$$

Take a minute to work through the mechanics of matrix multiplication to convince yourself that this is the correct answer. And then you can just remember that multiplying two diagonal matrices is easyâ€”in fact, for two diagonal matrices,

standard matrix multiplication is the same as element-wise multiplication. That becomes relevant when learning about eigendecomposition!

6.4 LIVE EVIL! (a.k.a. order of operations)

Letâ€™s start with an example to highlight the problem that we need a solution for. Implement the following matrix multiplication and then transpose the result:

$$\left(\begin{bmatrix}2 & 4\\1 & 2\end{bmatrix}\begin{bmatrix}0 & 1\\1 & 1\end{bmatrix}\right)^{\mathrm{T}} \quad = \quad ?$$

I assume you got the matrix $\begin{bmatrix}4 & 2\\6 & 3\end{bmatrix}$. Now letâ€™s try it again, but this time transpose each matrix individually before multiplying them:

$$\begin{bmatrix}2 & 4\\1 & 2\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}0 & 1\\1 & 1\end{bmatrix}^{\mathrm{T}} \quad = \quad ?$$

Did you get the same matrix as above? Well, if you did the math correctly, then you will have gotten $\begin{bmatrix}1 & 3\\2 & 6\end{bmatrix}$, which is the not the same as the previous result. Itâ€™s not even the same result but transposed. In fact, it is an entirely different matrix.

OK, now letâ€™s try it one more time. But now, instead of applying the transpose operation to each individual matrix, *swap the order of the matrices* . Thus:

$$\begin{bmatrix}0 & 1\\1 & 1\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}2 & 4\\1 & 2\end{bmatrix}^{\mathrm{T}} \quad = \quad ?$$

And now you get the same result as the first multiplication: $\begin{bmatrix}4 & 2\\6 & 3\end{bmatrix}$.

And this brings us to the main topic of this section: An operation applied to multiplied matrices gets applied to each matrix individually *but in reverse order* .

Itâ€™s a weird rule, but thatâ€™s just how it works. **"LIVE EVIL"** [4] is a mnemonic that will help you remember this important rule. Notice that LIVE spelled backwards is EVIL. Itâ€™s a palindrome.

---

[4] *n.b.* : LIVE EVIL is *not* a recommendation for how to interact with society. Please be nice, considerate, and generous.

Eqn. box title=**The LIVE EVIL rule: Reverse matrix order**

$$(\mathbf{A}\,â€¦\,\mathbf{B})^{\mathrm{T}} = \mathbf{B}^{\mathrm{T}}\,â€¦\,\mathbf{A}^{\mathrm{T}} \qquad (6.7)$$

Basically, an operation on multiplied matrices gets applied to each matrix in reverse order. Hereâ€™s how it would look for four matrices:

$$(\mathbf{ABCD})^{\mathrm{T}} = \mathbf{D}^{\mathrm{T}}\mathbf{C}^{\mathrm{T}}\mathbf{B}^{\mathrm{T}}\mathbf{A}^{\mathrm{T}}$$

The LIVE EVIL rule applies to other operations as well, such as the matrix inverse. For example:

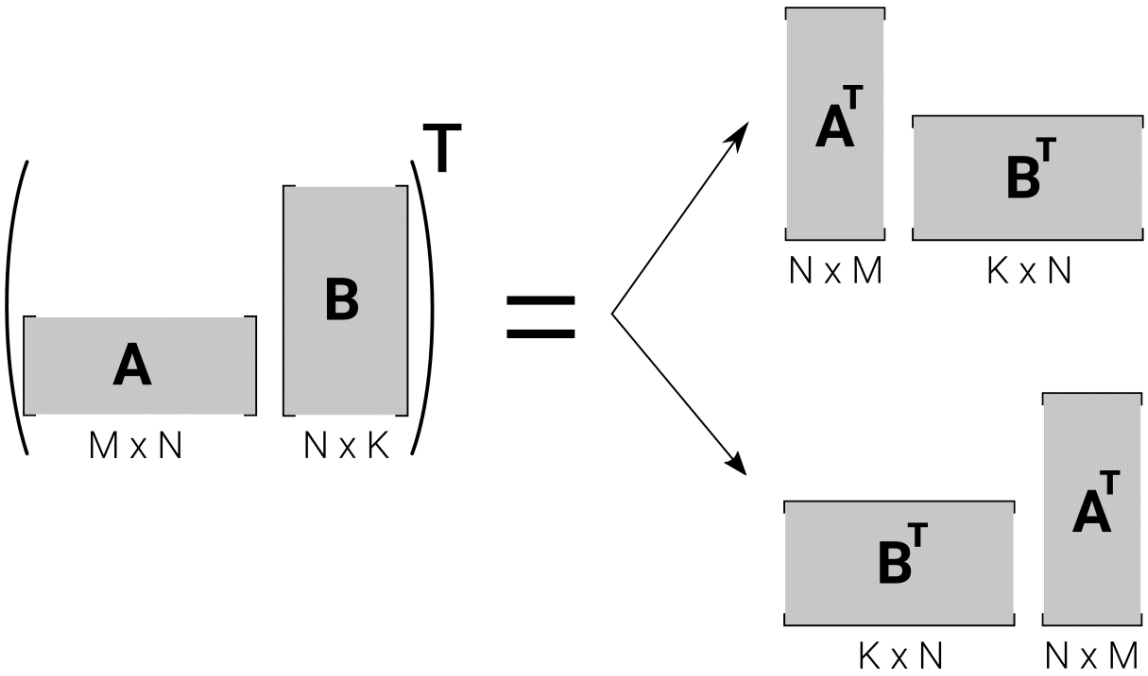
$$(\mathbf{ABC})^{-1} = \mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1}$$

LIVE EVIL is, admittedly, a strange and counter-intuitive rule. But it is important, so letâ€™s look at another example, which will also highlight why this rule needs to be the case. In the equations below, the top row implements matrix multiplication first, then transposes the result; while the second row first transposes each matrix in reverse order and then performs the matrix multiplication.

$$\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\begin{bmatrix} e & f \\ g & h \end{bmatrix}\right)^{\mathrm{T}} = \begin{bmatrix} ae+bg & af+bh \\ ce+dg & cf+dh \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} ae+bg & ce+dg \\ af+bh & cf+dh \end{bmatrix}$$

$$= \begin{bmatrix} e & g \\ f & h \end{bmatrix}\begin{bmatrix} a & c \\ b & d \end{bmatrix} = \begin{bmatrix} ae+bg & ce+dg \\ af+bh & cf+dh \end{bmatrix}$$

For square matrices, ignoring the LIVE EVIL rule still gives a result (though incorrect). However, for rectangular matrices, multiplication would be impossible when ignoring the LIVE EVIL rule. This is illustrated in Figure 6.6 .



FigureÂ 6.6: Example of the LIVE EVIL law for transposing matrix multiplication. Pay attention to the matrix sizes: Ignoring the LIVE EVIL rule and transposing without reversing leads to an invalid expression (top), whereas the multiplication remains valid when swapping matrix order (bottom).

Be careful with Equation 6.7 â€”an operation may be valid on the product of matrices but invalid on the individual matrices. This comes up frequently in statistics and the singular value decomposition. For example, the expression $(\mathbf{X}^{T} \mathbf{X})^{â^{-1}}$ may be valid whereas $\mathbf{X}^{â^{-1}} \mathbf{X}^{-T}$ may be undefined. Youâ€™ll see examples of this situation in later chapters. Fortunately, all matrices can be transposed, so for now you can always apply the LIVE EVIL rule without concern.

Practice problems Perform the following matrix multiplications. Compare with the problems and answers on page Â§ .

a) $\begin{bmatrix} 2 & -1 & 2 \\ 4 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}^{T} \begin{bmatrix} 3 & 4 & 0 \\ 0 & 4 & 1 \end{bmatrix}^{T}$

b) $\begin{bmatrix} 3 & 2 \\ 0 & 1 \end{bmatrix}^{T} \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}^{T}$

Answers

a) $\begin{bmatrix} 22 & 16 \\ -3 & 1 \\ 6 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 3 & 3 \\ 3 & 4 \end{bmatrix}$

6.5 Matrix-vector multiplication

Matrix-vector multiplication is the same thing as normal matrix-matrix multiplication, where you think of the vector as an *M* Ã— 1 or as a 1 Ã— *N* matrix. The important feature of matrix-vector multiplicationâ€”which is obvious when you think about it but is worth mentioning explicitlyâ€”is that the result is always a vector. This is important because it provides the connection between linear transformations and matrices: To apply a transform to a vector, you convert that transform into a matrix, and then you multiply the vector by that matrix. Here are two examples of matrix-vector multiplication.

$$\mathbf{Ab} = \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 5 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \cdot 5 + 2 \cdot 2 \\ 1 \cdot 5 + 3 \cdot 2 \end{bmatrix} = \begin{bmatrix} 24 \\ 11 \end{bmatrix}$$

$$\mathbf{b}^{\mathrm{T}}\mathbf{A} = \begin{bmatrix} 5 & 2 \end{bmatrix} \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 4 \cdot 5 + 1 \cdot 2 & 5 \cdot 2 + 2 \cdot 3 \end{bmatrix} = \begin{bmatrix} 24 & 16 \end{bmatrix}$$

Three observations here:

1. **bA** is not defined (assuming **b** is a column vector)
2. If **A** is rectangular, then either $\mathbf{b}^{\mathrm{T}}\mathbf{A}$ or **Ab** is undefined (depending on the sizes, but they canâ€™t both be valid)
3. **Ab** *â‰ * $\mathbf{b}^{\mathrm{T}}\mathbf{A}$ even when both are valid operations

There is an interesting exception to this third observation, which is that if the matrix is symmetric, then pre-multiplying the vector is the same as post-multiplying the transpose of the vector. (Technically, the results are not *literally the same* , because one result is a column vector while the other is a row vector, but the elements of those vectors are identical.)

Eqn. box title=**Symmetric matrix times a vector**

$$\textit{if } \mathbf{A} = \mathbf{A}^{\mathrm{T}} \textit{ then } \mathbf{Ab} = (\mathbf{b}^{\mathrm{T}}\mathbf{A})^{\mathrm{T}} \qquad (6.8)$$

Letâ€™s work through a proof of this claim. The proof works by transposing **Ab** and doing a bit of algebra (including applying the LIVE EVIL rule!) to simplify and re-arrange.

$$(\mathbf{Ab})^{\mathrm{T}} = \mathbf{b}^{\mathrm{T}}\mathbf{A}^{\mathrm{T}} = \mathbf{b}^{\mathrm{T}}\mathbf{A}$$

[5] The proof works because **A** = **A** ^{T} . If the matrix werenâ€™t symmetric, then **A** *â‰ ***A** ^{T} , in other words, **A** and **A** ^{T} would be different matrices. And of course, **b** and **b** ^{T} are the same except for orientation.

[5] Notice that our proof here involved transposing, expanding, and simplifying. This strategy underlies many linear algebra proofs.

Letâ€™s look at an example.

$$\mathbf{Ab} = \begin{bmatrix} a & b \\ b & c \end{bmatrix}\begin{bmatrix} d \\ e \end{bmatrix} = \begin{bmatrix} ad+be \\ bd+ce \end{bmatrix}$$

$$\mathbf{b}^{\mathrm{T}}\mathbf{A} = \begin{bmatrix} d & e \end{bmatrix}\begin{bmatrix} a & b \\ b & c \end{bmatrix} = \begin{bmatrix} ad+be & bd+ce \end{bmatrix}$$

Notice, as mentioned above, that the two results are not *identical vectors* because one is a column and the other is a row. However, they have *identical elements* in a different orientation.

Now watch what happens when matrix **A** is non-symmetric. In the matrices below, assume *bâ‰ f* .

$$\mathbf{Ab} = \begin{bmatrix} a & b \\ f & c \end{bmatrix}\begin{bmatrix} d \\ e \end{bmatrix} = \begin{bmatrix} ad+be \\ fd+ce \end{bmatrix}$$

$$\mathbf{b}^{\mathrm{T}}\mathbf{A} = \begin{bmatrix} d & e \end{bmatrix}\begin{bmatrix} a & b \\ f & c \end{bmatrix} = \begin{bmatrix} ad+fe & bd+ce \end{bmatrix}$$

Practice problems Perform the following matrix multiplications.

a) $\begin{bmatrix} 4 & 3 & 0 \\ 1 & 1 & 9 \\ 0 & 3 & 0 \\ 1 & 1 & 1 \end{bmatrix}\begin{bmatrix} 0 \\ 7 \\ 1 \end{bmatrix}$

b)

$$\begin{bmatrix} 4 & 0 & 2 & 0 & 8 & 0 \\ 0 & 3 & 0 & 5 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 6 \\ 1 \\ 2 \\ 1 \\ 6 \end{bmatrix}$$

Answers

a) $\begin{bmatrix} 21 \\ 16 \\ 21 \\ 8 \end{bmatrix}$

b) $\begin{bmatrix} 26 \\ 34 \end{bmatrix}$

6.6 Creating symmetric matrices

In the previous chapter, I claimed that all hope is not lost for non-symmetric matrices who aspire to the glorious status of their symmetric conspecifics, upon whom so many luxuries of linear algebra are bestowed. How can a non-symmetric matrix become a symmetric matrix?

There are two methods: Additive and multiplicative. The additive method is not widely used in practical applications (to my knowledge), but it is worth learning. The additive method to create a symmetric matrix from a non-symmetric matrix is to add the matrix to its transpose. This method is valid only for square matrices.

$$\mathbf{C} = \frac{1}{2}(\mathbf{A}^{\mathrm{T}} + \mathbf{A}) \tag{6.9}$$

An example will illustrate why Equation 6.9 works. Notice that the diagonal elements are doubled, which is why dividing by 2 is an appropriate normalization factor. (In the matrices below, assume that *bâ‰ d* , *câ‰ h* , and *fâ‰ i* .)

$$\begin{bmatrix} a & b & c \\ d & e & f \\ h & i & j \end{bmatrix} + \begin{bmatrix} a & d & h \\ b & e & i \\ c & f & j \end{bmatrix} = \begin{bmatrix} a+a & b+d & c+h \\ b+d & e+e & f+i \\ c+h & f+i & j+j \end{bmatrix} \tag{6.10}$$

This is just one example of a 3 Ã— 3 matrix. What if this is some quirk of this particular matrix? How do we know that this method will always work? This is an

important question, because there are several special properties of 2 Ã— 2 or 3 Ã — 3 matrices that do not generalize to larger matrices.

The proof that Equation 6.9 will always produce a symmetric matrix from a non-symmetric square matrix comes from the definition of symmetric matrices ($\mathbf{C} = \mathbf{C}^{T}$). Therefore, the proof works by transposing both sides of Equation 6.9 , doing a bit of algebra to simplify, and seeing what happens (Iâ€™m omitting the scalar division by 2 because that doesnâ€™t affect symmetry).

$$\mathbf{C} = \mathbf{A}^{T} + \mathbf{A} \quad (6.11)$$

$$\mathbf{C}^{T} = (\mathbf{A}^{T} + \mathbf{A})^{T} \quad (6.12)$$

$$\mathbf{C}^{T} = \mathbf{A}^{TT} + \mathbf{A}^{T} \quad (6.13)$$

$$\mathbf{C}^{T} = \mathbf{A} + \mathbf{A}^{T} \quad (6.14)$$

Because matrix addition is commutative, $\mathbf{A} + \mathbf{A}^{T} = \mathbf{A}^{T} + \mathbf{A}$. The right-hand side of Equation 6.14 is the same as the right-hand side of Equation 6.11 . And if the right-hand sides of the equations are equal, then the left-hand sides must also be equal. This proves that $\mathbf{C} = \mathbf{C}^{T}$, which is the definition of a symmetric matrix. This proof does not depend on the size of the matrix, which shows that our example above was not a fluke.

---

[6] The matrices are summed, not multiplied, so the LIVE EVIL rule does not apply.

Practice problems Create symmetric matrices from the following matrices using the additive method.

a) $\begin{bmatrix} 1 & 11 & 1 \\ -5 & -2 & 1 \\ -1 & -5 & 2 \end{bmatrix}$

b) $\begin{bmatrix} -3 & 0 & -2 & -2 \\ -1 & -3 & -2 & -6 \\ -4 & -8 & -7 & 4 \\ 6 & -2 & 5 & 4 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 1 & 3 & 0 \\ 3 & -2 & -2 \\ 0 & -2 & 2 \end{bmatrix}$

b)

$$\begin{bmatrix} -3 & -\frac{1}{2} & -3 & 2 \\ -\frac{1}{2} & -3 & -5 & -4 \\ -3 & -5 & -7 & \frac{9}{2} \\ 2 & -4 & \frac{9}{2} & 4 \end{bmatrix}$$

The multiplicative method This involves multiplying a matrix by its transpose. In fact, this is the $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ matrix that you learned about in the previous chapter.

I claimed in the previous chapter that $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ is guaranteed to be symmetric (and therefore also square), even if **A** is non-symmetricâ€”and even if **A** is non-square. Now that you know about matrix multiplication and about the LIVE EVIL rule, you are able to prove these two important claims.

First we prove that $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ is a square matrix. Letâ€™s say that **A** is size $M \times N$ (assume $M \geq N$). First note that **AA** is not a valid multiplication. Now consider that $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ means $(M \times N) \times (N \times M)$. The "inner" dimensions (N) match, and the result will be $M \times M$. So there you go: we just proved that $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ is a square matrix.

Now letâ€™s prove that $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ is symmetric. The proof follows the same strategy that we applied for the additive method: transpose $\mathbf{A}^{\mathrm{T}}\mathbf{A}$, do a bit of algebra, and see what happens.

$$(\mathbf{A}^{\mathrm{T}}\mathbf{A})^{\mathrm{T}} = \mathbf{A}^{\mathrm{T}}\mathbf{A}^{\mathrm{TT}} = \mathbf{A}^{\mathrm{T}}\mathbf{A} \tag{6.15}$$

We transposed the matrix, applied the LIVE EVIL rule (and the property that a double-transpose leaves the matrix unchanged), and got back to the original matrix. A matrix that equals its transpose is the definition of a symmetric matrix.

Are these two features unique to $\mathbf{A}^{\mathrm{T}}\mathbf{A}$? What about $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ â€”is that also square and symmetric? The answer is Yes, but I want you to get a piece of paper and prove it to yourself.

-|*|- ***Reflection*** *In practical applications, particularly in data analysis,* $\mathbf{A}^{T}\mathbf{A}$ *appears as often as* $\mathbf{A}\mathbf{A}^{T}$*. The correct form depends on how the data are stored (e.g., observations × features vs. features × observations), which usually depends on coding preferences or software format. In the written medium, I prefer* $\mathbf{A}^{T}\mathbf{A}$ *because of the aesthetic symmetry of having the* ^{T} *in the middle.* -|*|-

---

Practice problems Create two symmetric matrices from each of the following matrices, using the multiplicative method ($\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$).

a) $\begin{bmatrix} 3 & 0 & 3 \\ 0 & 7 & 0 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 6 \\ 6 & 1 \end{bmatrix}$

Answers

a) $\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 9 & 0 & 9 \\ 0 & 49 & 0 \\ 9 & 0 & 9 \end{bmatrix}$, $\mathbf{A}\mathbf{A}^T = \begin{bmatrix} 18 & 0 \\ 0 & 49 \end{bmatrix}$

b) $\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 37 & 12 \\ 12 & 37 \end{bmatrix}$, $\mathbf{A}\mathbf{A}^T = \begin{bmatrix} 37 & 12 \\ 12 & 37 \end{bmatrix}$

6.7 Multiplication of two symmetric matrices

If you multiply two symmetric matrices, will the product matrix also be symmetric? Letâ€™s try an example to find out:

$$\begin{bmatrix} a & b \\ b & c \end{bmatrix} \begin{bmatrix} d & e \\ e & f \end{bmatrix} = \begin{bmatrix} ad+be & ae+bf \\ bd+ce & be+cf \end{bmatrix} \tag{6.16}$$

Is the result symmetric? On its face, it seems like it isnâ€™t. However, this equation reveals an interesting condition on the symmetry of the result of multiplying two 2 Ã— 2 symmetric matrices: If $a = c$ and $d = f$ (in other words, the matrix has a constant diagonal), then the product of two symmetric matrices is itself symmetric. Observe and be amazed!

$$\begin{bmatrix} a & b \\ b & a \end{bmatrix} \begin{bmatrix} d & e \\ e & d \end{bmatrix} = \begin{bmatrix} ad+be & ae+bd \\ bd+ae & be+ad \end{bmatrix} \tag{6.17}$$

Is this a general rule that applies to symmetric matrices of any size? Letâ€™s repeat with 3 Ã— 3 matrices, using the constant-diagonal idea.

$$\begin{bmatrix} a & b & c \\ b & a & d \\ c & d & a \end{bmatrix} \begin{bmatrix} e & f & g \\ f & e & h \\ g & h & e \end{bmatrix} = \begin{bmatrix} ae+bf+cg & af+be+ch & ag+bh+ce \\ be+af+dg & bf+ae+dh & bg+ah+de \\ ce+df+ag & cf+de+ah & cg+dh+ae \end{bmatrix}$$

A quick glance reveals the lack of symmetry. For example, compare the element in position *2,1* with that in position *1,2* . I wonâ€™t write out the product of two 4 Ã— 4 symmetric matrices (if you want to try it, go for it), but you can take my word for it: the resulting product matrix will not be symmetric.

The lesson here is that, in general, *the product of two symmetric matrices is not a symmetric matrix* . There are exceptions to this rule, like the 2 Ã— 2 case with constant diagonals, or if one of the matrices is the identity or zeros matrix.

Is this a surprising result? Refer back to the discussion at the end of the "element perspective" of matrix multiplication (page Â§) concerning how the upper-triangle vs. the lower-triangle of the product matrix is formed from earlier vs. later rows of the left matrix. Different parts of the two multiplying matrices meet in the lower triangle vs. the upper triangle of the product matrix.

You can also see that the product matrix is not symmetric by trying to prove that it *is* symmetric (the proof fails, which is called proof-by-contradition). Assume that **A** and **B** below are both symmetric matrices.

$$(\mathbf{AB})^{\mathrm{T}} = \mathbf{B}^{\mathrm{T}}\mathbf{A}^{\mathrm{T}} = \mathbf{BA} \neq \mathbf{AB} \tag{6.18}$$

To be sure, Equation 6.18 is a perfectly valid equation, and all of the individual multiplications are valid. However, matrix multiplication is not commutative, which is why we had to put a *â‰ * sign at the end of the equation. Thus, we cannot assume that **AB** = (**AB**) ^{T} , therefore, the multiplication of two symmetric matrices is, in general, not a symmetric matrix.

-|*|- ***Reflection*** *This may seem like a uselessly academic factoid, but it leads to one of the biggest limitations of principal components analysis, and one of the most important advantages of generalized eigendecomposition, which is the computational backbone of many machine-learning methods, most prominently linear classifiers and discriminant analyses.* -|*|-

6.8 Element-wise (Hadamard) multiplication

Before having had any exposure to matrix multiplication (in this chapter or elsewhere), Hadamard multiplication is probably what you would have answered if someone asked you to guess what it means to multiply two matrices.

Hadamard multiplication involves multiplying each element of one matrix by the corresponding element in the other matrix. You have already learned about Hadamard multiplication in Chapter 3; the concept and notation is the same for matrices as for vectors. Thus:

$$\mathbf{C} = \mathbf{A} \odot \mathbf{B} \tag{6.19}$$

The formal definition of Hadamard multiplication is

Eqn. box title=**Hadamard (element-wise) multiplication**

$$c_{i,j} = a_{i,j} \text{Ã—} b_{i,j} \tag{6.20}$$

You might have already guessed that Hadamard multiplication is valid only for two matrices that are both M Ã— N , and the product matrix is also size M Ã— N .

One example will suffice for understanding.

$$\begin{bmatrix} 0 & 1 & 2 \\ -1 & 6 & 3 \end{bmatrix} \odot \begin{bmatrix} 3 & 8 & 5 \\ 4 & 1 & -5 \end{bmatrix} = \begin{bmatrix} 0 & 8 & 10 \\ -4 & 6 & -15 \end{bmatrix} \tag{6.21}$$

Because Hadamard multiplication is implemented element-wise, it obeys the commutative law just like individual numbers (scalars). That is,

$$\mathbf{A} \odot \mathbf{B} = \mathbf{B} \odot \mathbf{A}$$

There is also element-wise division, which is the same principle but for division. This operation is valid only when the divisor matrix contains all non-zero elements.

$$\begin{bmatrix} 0 & 1 & 2 \\ -1 & 6 & 3 \end{bmatrix} \oslash \begin{bmatrix} 3 & 8 & 5 \\ 4 & 1 & -5 \end{bmatrix} = \begin{bmatrix} 0/3 & 1/8 & 2/5 \\ -1/4 & 6/1 & -3/5 \end{bmatrix}$$

$\oslash$ indicates element-wise division.

One can debate whether Hadamard multiplication and division are really *matrix* operations; arguably, they are simply scalar multiplication or division, implemented *en masse* using compact notation. Indeed, element-wise matrix multiplication in computer applications facilitates convenient and efficient coding (e.g., to avoid using for-loops), as opposed to utilizing some special mathematical properties of Hadamard multiplication. That said, Hadamard multiplication does have applications in linear algebra. For example, it is key to one of the algorithms for computing the matrix inverse.

Code Here is another case where matrix multiplication is confusingly different between MATLAB and Python. In MATLAB, `A*B` indicates standard matrix multiplication and `A.*B` indicates Hadamard multiplication (note the *dot-star*). In

Python, `A@B` gives standard matrix multiplication and `A*B` gives Hadamard multiplication.

Code blockÂ 6.5: Python

```
M1Â =Â np.random.randn(4,3)Â
M2Â =Â np.random.randn(4,3)Â
CÂ =Â M1Â *Â M2
```

Code blockÂ 6.6: MATLAB

```
M1Â =Â randn(4,3);Â
M2Â =Â randn(4,3);Â
CÂ =Â M1Â .*Â M2
```

Practice problems Hadamard-multiply the following pairs of matrices.

a) $\begin{bmatrix} -4 & -3 & -9 \\ -5 & -2 & 3 \\ 0 & -7 & 7 \end{bmatrix}, \begin{bmatrix} -1 & 4 & -3 \\ -3 & -4 & 2 \\ 3 & 2 & 5 \end{bmatrix}$

b) $\begin{bmatrix} -4 & -5 & -16 \\ -4 & -1 & 2 \\ 7 & 1 & 5 \end{bmatrix}, \begin{bmatrix} -3 & 6 & -10 & 0 \\ -4 & 9 & -6 & 4 \\ 1 & 2 & -6 & 0 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 4 & -12 & 27 \\ 15 & 8 & 6 \\ 0 & -14 & 35 \end{bmatrix}$

b) Undefined!

6.9 Frobenius dot product

The Frobenius dot product, also called the Frobenius inner product, is an operation that produces a scalar (a single number) given two matrices of the same size (M Ã— N).

To compute the Frobenius dot product, you first *vectorize* the two matrices and then compute their dot product as you would for regular vectors.

Vectorizing a matrix means concatenating all of the columns in a matrix to produce a single column vector. It is a function that maps a matrix in â„ $^{M Ã— N}$ to a vector in â„ ^{MN} .

Eqn. box title=**Vectorizing a matrix**

$$\mathbf{v}_n = a_{i,j}\text{, such that}$$

$$\mathbf{v} = [\, a_{1,1}, \hat{a}\in\!\!\!

$$\langle \mathbf{A}, \mathbf{B} \rangle_F = \left\langle \begin{bmatrix} 1 & 5 & 0 \\ -4 & 0 & 2 \end{bmatrix}, \begin{bmatrix} 4 & -1 & 3 \\ 2 & 6 & 7 \end{bmatrix} \right\rangle_F = 5$$

Note the notation for the Frobenius dot product: âŸ¨ **A** , **B** âŸ© $_F$

A curious yet useful way to compute the Frobenius dot product between matrices **A** and **B** is by taking the trace of **A** ^{T} **B** . Therefore, the Frobenius dot product can also be written as follows.

Eqn. box title=**Frobenius dot product as the trace of A ^{T} B**

$$\text{âŸ¨ } \mathbf{A} \text{ , } \mathbf{B} \text{ âŸ© }_F = tr\left(\mathbf{A}^{\mathrm{T}} \mathbf{B} \right) \qquad (6.23)$$

Iâ€™ve omitted the matrix sizes in this equation, but you can tell from inspection that the operation is valid if both matrices are size M Ã— N , because the trace is defined only on square matrices.

The reason why Equation 6.23 is valid can be seen by working through a few examples, which you will have the opportunity to do in the exercises.

The Frobenius dot product has several uses in signal processing and machine learning, for example as a measure of "distance," or similarity, between two matrices.

The Frobenius inner product of a matrix with itself is the sum of all squared elements, and is called the *squared Frobenius norm* or *squared Euclidean norm* of the matrix. More on this in the next section.

Code The code below shows the trace-transpose trick for computing the Frobenius dot product.

Code blockÂ 6.9: Python

```
AÂ =Â np.random.randn(4,3)Â
BÂ =Â np.random.randn(4,3)Â
fÂ =Â np.trace(A.T@B)
```

Code blockÂ 6.10: MATLAB

```
AÂ =Â randn(4,3);Â
BÂ =Â randn(4,3);Â
fÂ =Â trace(Aâ€™*B);
```

Practice problems Compute the Frobenius dot product between the following pairs of matrices.

a) $\begin{bmatrix} 4 & 2 \\ 3 & 2 \\ -5 & -1 \end{bmatrix}, \begin{bmatrix} 7 & -2 \\ -7 & -8 \\ -1 & 8 \end{bmatrix}$

b) $\begin{bmatrix} 6 & 1 & 2 \\ 3 & 3 & -2 \end{bmatrix}, \begin{bmatrix} 4 & -11 & 1 \\ 6 & 1 & -2 \end{bmatrix}$

Answers

a) âˆ’ 16
b) 40

6.10 Matrix norms

In section 3.9 you learned that the square root of the dot product of a vector with itself is the magnitude or length of the vector, which is also called the norm of the vector.

Annoyingly, the norm of a matrix is more complicated, just like everything gets more complicated when you move from vectors to matrices. Part of the complication with matrix norms is that there are many of them! They all have some things in common, for example, all matrix norms are a single number that somehow corresponds to the "magnitude" of the matrix, but different norms correspond to different interpretations of "magnitude." In this section, you will learn a few of the common matrix norms, and weâ€™ll continue the discussion of matrix norms in Chapter 16 in the context of the singular value decomposition.

Letâ€™s start with the Frobenius norm, because itâ€™s fresh in your mind from the previous section. The equation below is an alternative way to express the Frobenius norm.

Eqn. box title=**Frobenius matrix norm**

$$\hat{a}\check{}\text{¥}\ \mathbf{A}\ \hat{a}\check{}\text{¥}_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n}(a_{i,j})^2} \qquad (6.24)$$

Now you know three ways to compute the Frobenius norm: (1) directly implementing Equation 6.24 , (2) vectorizing the matrix and computing the dot product with itself, (3) computing $tr(\mathbf{A}^T\mathbf{A})$.

If we think of a matrix space as Euclidean, then the Frobenius norm of the subtraction of two matrices provides a measure of Euclidean distance between those matrices. The right-hand side of the formula below should look familiar from computing Euclidean distance between two points.

Eqn. box title=**Euclidean distance between two matrices**

$$\hat{a}\check{}\text{¥}\ \mathbf{A}\ \hat{a}\check{}\text{'}\ \mathbf{B}\ \hat{a}\check{}\text{¥}_F = \sqrt{\sum_{i,j}(a_{i,j}-b_{i,j})^2} \qquad (6.25)$$

Of course, this formula is valid only for two matrices that have the same size.

The Frobenius norm is also called the *â„“* 2 norm (the *â„“* character is just a fancy-looking l, or a lower-case cursive L). There is also an *â„“* 1 matrix norm. To compute the *â„“* 1 norm, you sum the absolute values of all individual elements in each column, then take the largest maximum column sum.

There are many other matrix norms with varied formulas. Different applications use different norms to satisfy different criteria or to minimize different features of the matrix. Rather than overwhelm you with an exhaustive list, I will provide one general formula for the *matrix p-norm* ; you can see that for $p = 2$, the following formula is equal to the Frobenius norm.

(6.26)

$$\|\mathbf{A}\|_p = \left(\sum_{i=1}^{M} \sum_{j=1}^{N} |a_{ij}|^p \right)^{1/p}$$

Cauchy-Schwarz inequality In Chapter 3 you learned about the Cauchy-Schwarz inequality (the magnitude of the dot product between two vectors is no larger than the product of the norms of the two vectors). There is a comparable inequality for the Frobenius norm of a matrix-vector multiplication:

$$\|\mathbf{Av}\| \leq \|\mathbf{A}\|_F \|\mathbf{v}\| \tag{6.27}$$

The proof for this inequality comes from integrating (1) the row perspective of multiplication, (2) the Cauchy-Schwarz inequality for the vector dot product introduced in Chapter 3, and (3) linearity of the Frobenius norm across the rows (that is, the Frobenius norm of a matrix equals the sum of the Frobenius norms of the rows; this comes from the summation in Equation 6.24).

Letâ€™s start by re-writing the norm of the matrix-vector multiplication as the sum of vector norms coming from the dot products of each row of **A** with **v** (*m* is the number of rows and $\mathbf{a}_i$ is the ith row of **A**).

$$\|\mathbf{Av}\| = |\mathbf{a}_1^{\mathrm{T}}\mathbf{v}| + \ldots + |\mathbf{a}_n^{\mathrm{T}}\mathbf{v}| = \sum_{i=1}^{m} |\mathbf{a}_i^{\mathrm{T}}\mathbf{v}| \tag{6.28}$$

The dot-product Cauchy-Schwarz inequality allows us to write the following.

$$\sum_{i=1}^{m} |\mathbf{a}_i^{\mathrm{T}}\mathbf{v}| \leq \sum_{i=1}^{m} \|\mathbf{a}_i\| \|\mathbf{v}\| \tag{6.29}$$

Finally, we re-sum the norms of the rows back to the norm of the matrix. And that brings us back to our original conclusion in Equation 6.27 : The norm of a matrix-vector product is less than or equal to the product of the Frobenius norms of the matrix and the vector.

Code Different matrix norms can be obtained by specifiying different inputs into the `norm` functions. The code below shows Frobenius norm.

Code blockÂ 6.11: Python

```
AÂ =Â np.random.randn(4,3)Â
np.linalg.norm(A,â€™froâ€™)
```

Code blockÂ 6.12: MATLAB

```
AÂ =Â randn(4,3);Â
norm(A,â€™froâ€™)
```

Practice problems Compute Euclidean distance between the following pairs of matrices (note: compare with the exercises on page Â§).

a) $\begin{bmatrix} 4 & 2 \\ 3 & 2 \\ -5 & -1 \end{bmatrix}, \begin{bmatrix} 7 & -2 \\ -7 & -8 \\ -1 & 8 \end{bmatrix}$

b) $\begin{bmatrix} 6 & 1 & 2 \\ 3 & 3 & -2 \end{bmatrix}, \begin{bmatrix} 4 & -11 & 1 \\ 6 & 1 & -2 \end{bmatrix}$

Answers

a) $\sqrt{322}$ â‰ˆ 17 . 94

b) $\sqrt{162}$ â‰ˆ 12 . 73

6.11 What about matrix division?

All this fuss about matrix multiplications... what about division? You did learn about element-wise matrix division in section 6.8 , but thatâ€™s not really *matrix* division; thatâ€™s a compact notation for describing lots of scalar divisions. When you hear "matrix division" youâ€™re probably thinking about something like this:

$$\frac{\mathbf{A}}{\mathbf{B}}$$

This would be the equivalent of a scalar division like $\frac{2}{3}$. Well, that doesnâ€™t exist for matrices; it is not possible to *divide* one matrix by another. However, there is a conceptually comparable operation, and it is based on the idea of re-writing scalar division like this:

$$\frac{2}{3} = 2 \times 3^{-1}$$

The matrix version of this is $\mathbf{AB}^{\hat{a}\tilde{}'1}$. The matrix $\mathbf{B}^{\hat{a}\tilde{}'1}$ is called the *matrix inverse* . It is such an important topic that it merits its own chapter (Chapter 12). For now, Iâ€™ll leave you with two important facts about the matrix inverse.

1. The matrix inverse is the matrix such that $\mathbf{AA}^{-1} = \mathbf{I}$
2. Not all matrices have an inverse. A matrix has a full inverse only if it is square and full-rank.

6.12 Exercises

1.Â

Determine whether each of the following operations is valid, and, if so, the size of the resulting matrix.

$$\mathbf{A} \in \mathbb{R}^{2\times3}, \quad \mathbf{B} \in \mathbb{R}^{3\times3}, \quad \mathbf{C} \in \mathbb{R}^{3\times4}$$

a) $\mathbf{CB}$
b) $\mathbf{C}^{T}\mathbf{B}$
c) $(\mathbf{CB})^{T}$
d) $\mathbf{C}^{T}\mathbf{BC}$
e) $\mathbf{ABCB}$
f) $\mathbf{ABC}$
g) $\mathbf{C}^{T}\mathbf{BA}^{T}\mathbf{AC}$
h) $\mathbf{B}^{T}\mathbf{BCC}^{T}\mathbf{A}$
i) $\mathbf{AA}^{T}$
j) $\mathbf{A}^{T}\mathbf{A}$
k) $\mathbf{BBA}^{T}\mathbf{ABBCC}$
l) $(\mathbf{CBB}^{T}\mathbf{CC}^{T})^{T}$
m) $(\mathbf{A} + \mathbf{ACC}^{T}\mathbf{B})^{T}\mathbf{A}$
n) $\mathbf{C} + \mathbf{CA}^{T}\mathbf{ABC}$
o) $\mathbf{C} + \mathbf{BA}^{T}\mathbf{ABC}$
p) $\mathbf{B} + 3\mathbf{B} + \mathbf{A}^{T}\mathbf{A} - \mathbf{CC}^{T}$
q) $\mathbf{A}(\mathbf{ABC})$
r) $\mathbf{A}\mathbf{ABC}(\mathbf{BC})^{T}$

2.Â

Compute the following matrix multiplications. Each problem should be completed twice using the two indicated perspectives of matrix multiplication (#1: element, #2: layer, #3: column, #4: row).

a) #1,2: $\begin{bmatrix}1 & 0\\3 & 1\end{bmatrix}\begin{bmatrix}0 & 5\\2 & 2\end{bmatrix}$

b)

#2,4: $\begin{bmatrix} -3 & 2 \\ -2 & 3 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$

c) #3,4: $\begin{bmatrix} 11 & -5 \\ 9 & -13 \end{bmatrix}\begin{bmatrix} 3 & 1 \\ -8 & .5 \end{bmatrix}$

d) #1,4: $\begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}\begin{bmatrix} a & b \\ c & d \end{bmatrix}$

e) #2,3: $\begin{bmatrix} 2 & 2 \\ 1 & 3 \end{bmatrix}\begin{bmatrix} 10 & 1 \\ -5 & 4 \end{bmatrix}$

f) #1,3: $\begin{bmatrix} a & 0 \\ b & 0 \end{bmatrix}\begin{bmatrix} 2 & 3 \\ 4 & 1 \end{bmatrix}$

g) #2,3: $\begin{bmatrix} a & b \\ 0 & 0 \end{bmatrix}\begin{bmatrix} 2 & 3 \\ 4 & 1 \end{bmatrix}$

h) #1,2: $\begin{bmatrix} 1 & 0 & 4 \\ 0 & 1 & 1 \\ 3 & 3 & 0 \end{bmatrix}\begin{bmatrix} -2 & -3 & -1 \\ -1 & -9 & 3 \\ 0 & 1 & 5 \end{bmatrix}$

i) #2,3: $\begin{bmatrix} a & 0 & 1 \\ 0 & b & 0 \\ 1 & 0 & c \end{bmatrix}\begin{bmatrix} a & b & c \\ 1 & 2 & 3 \\ 0 & 0 & 1 \end{bmatrix}$

3.Â

Compute the following matrix-vector products, if the operation is valid.

a) $\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}\begin{bmatrix} 2 \\ 3 \end{bmatrix}$

b) $\begin{bmatrix} 2 \\ 3 \end{bmatrix}^{\mathrm{T}}\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$

c) $\begin{bmatrix} 2 & 1 \\ 2 & 3 \end{bmatrix}\begin{bmatrix} 2 \\ 3 \end{bmatrix}$

d) $\begin{bmatrix} 2 \\ 3 \end{bmatrix}^{\mathrm{T}}\begin{bmatrix} 2 & 1 \\ 2 & 3 \end{bmatrix}$

e) $\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}\begin{bmatrix} a \\ b \\ c \end{bmatrix}$

f) $\begin{bmatrix} 1 & 0 & 1 \\ 0 & -4 & 0 \\ 1 & 0 & 1 \end{bmatrix}\begin{bmatrix} 5 \\ 2 \\ 1 \end{bmatrix}$

g) $\begin{bmatrix} 1 \\ 2 \\ 5 \end{bmatrix}\begin{bmatrix} 1 & 0 & 1 \\ 0 & -4 & 0 \\ 1 & 0 & 1 \end{bmatrix}$

h) $\begin{bmatrix}3\\3\\2\end{bmatrix}^{\mathrm{T}}\begin{bmatrix}1&3&2\\6&1&5\\3&5&0\end{bmatrix}$

4.Â

Consider square matrices **A** and **B** , with nonzero values at all elements. What assumptions on symmetry would make the following equalities hold (note: the operations might also be impossible under any assumptions)? Provide a proof or example for each.

a) $\mathbf{AB} = \mathbf{A}^{\mathrm{T}}\mathbf{B}^{\mathrm{T}}$
b) $\mathbf{AB} = (\mathbf{AB})^{\mathrm{T}}$
c) $\mathbf{AB} = \mathbf{AB}^{\mathrm{T}}$
d) $\mathbf{AB} = \mathbf{A}^{\mathrm{T}}\mathbf{B}$
e) $\mathbf{AB} = \mathbf{B}^{\mathrm{T}}\mathbf{A}$
f) $\mathbf{AB} = (\mathbf{BA})^{\mathrm{T}}$

5.Â

In section 6.7 you learned that the product of two symmetric matrices is generally not symmetric. That was for standard multiplication; is the *Hadamard product* of two symmetric matrices symmetric? Work through your reasoning first, then test your hypothesis on the following matrix pair.

$$\begin{bmatrix}2&5&7\\5&3&6\\7&6&4\end{bmatrix}, \begin{bmatrix}a&d&f\\d&b&e\\f&e&c\end{bmatrix}$$

6.Â

For the following pairs of matrices, vectorize and compute the vector dot product, then compute the Frobenius inner product as $tr(\mathbf{A}^{\mathrm{T}}\mathbf{B})$.

a) $\begin{bmatrix}a&b\\c&d\end{bmatrix}, \begin{bmatrix}1&2\\3&4\end{bmatrix}$
b) $\begin{bmatrix}0&5\\7&-2\end{bmatrix}, \begin{bmatrix}1&0\\13&14\end{bmatrix}$
c) $\begin{bmatrix}4&-5&8\\1&-1&2\\-2&2&-4\end{bmatrix}, \begin{bmatrix}4&-5&8\\1&-1&2\\-2&2&-4\end{bmatrix}$
d) $\begin{bmatrix}a&b\\c&d\end{bmatrix}, \begin{bmatrix}a&b\\a&b\end{bmatrix}$
e)

f) $\begin{bmatrix} a & b \\ c & d \end{bmatrix}, \begin{bmatrix} a & b \\ c & d \end{bmatrix}$

$\begin{bmatrix} 1 & 1 & 7 \\ 2 & 2 & 6 \\ 3 & 3 & 5 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$

7.Â

Implement the indicated multiplications for the following matrices.

$$\mathbf{A} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & -1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 3 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 2 & 1 & 3 \\ 0 & 4 & 1 \\ 2 & 2 & 3 \end{bmatrix}$$

a) AB
b) AC
c) BC
d) CA
e) CB
f) BCA
g) ACB
h) ABC

6.13 Answers

1.Â

a) no
b) yes: 4 Ã— 3
c) no
d) yes: 4 Ã— 4
e) no
f) yes: 2 Ã— 4
g) yes: 4 Ã— 4
h) no
i) yes: 2 Ã— 2
j) yes: 3 Ã— 3
k) no
l) no
m) yes: 3 Ã— 3
n) no
o) yes: 3 Ã— 4

p) yes: 3 Ã— 3
q) no
r) yes: 2 Ã— 3

2.Â

a) $\begin{bmatrix} 0 & 5 \\ 2 & 17 \end{bmatrix}$
b) $\begin{bmatrix} -3 & 4 \\ -2 & 6 \end{bmatrix}$
c) $\begin{bmatrix} 73 & 8.5 \\ 131 & 2.5 \end{bmatrix}$
d) $\begin{bmatrix} a & b \\ 2c & 2d \end{bmatrix}$
e) $\begin{bmatrix} 10 & 10 \\ -5 & 13 \end{bmatrix}$
f) $\begin{bmatrix} 2a & 3a \\ 2b & 3b \end{bmatrix}$
g) $\begin{bmatrix} 2a+4b & 3a+b \\ 0 & 0 \end{bmatrix}$
h) $\begin{bmatrix} -2 & 1 & 19 \\ -1 & -8 & 8 \\ -9 & -36 & 6 \end{bmatrix}$
i) $\begin{bmatrix} a^2 & ab & ac+1 \\ b & 2b & 3b \\ a & b & 2c \end{bmatrix}$

3.Â

a) $\begin{bmatrix} 4 \\ 9 \end{bmatrix}$
b) $\begin{bmatrix} 4 & 9 \end{bmatrix}$
c) $\begin{bmatrix} 7 \\ 13 \end{bmatrix}$
d) $\begin{bmatrix} 10 & 11 \end{bmatrix}$
e) $\begin{bmatrix} b \\ c \\ a \end{bmatrix}$
f)

$$\begin{bmatrix} 6 \\ -8 \\ 6 \end{bmatrix}$$

g) invalid

h) $\begin{bmatrix} 27 & 22 & 21 \end{bmatrix}$

4.Â

a) Both matrices symmetric.

b) **A** = **B** , both symmetric.
Or **A** = **B** ^{T}

c) **B** = **B** ^{T}

d) **A** = **A** ^{T}

e) No general rule

f) Both symmetric.

5.Â
Yes, because the multiplication is done element-wise.

6.Â

a) $a + 2\,b + 3\,c + 4\,d$

b) 63

c) 135

d) $a^{2} + b^{2} + ca + bd$

e) $a^{2} + b^{2} + c^{2} + d^{2}$

f) undefined

7.Â

a) $\begin{bmatrix} 6 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & -3 \end{bmatrix}$

b) $\begin{bmatrix} 6 & 3 & 9 \\ 0 & 8 & 2 \\ -2 & -2 & -3 \end{bmatrix}$

c) $\begin{bmatrix} 4 & 2 & 6 \\ 0 & 20 & 5 \\ 6 & 6 & 9 \end{bmatrix}$

d)

$$\begin{bmatrix} 6 & 2 & -3 \\ 0 & 8 & -1 \\ 6 & 4 & -3 \end{bmatrix}$$

e) $\begin{bmatrix} 4 & 5 & 9 \\ 0 & 20 & 3 \\ 4 & 10 & 9 \end{bmatrix}$

f) $\begin{bmatrix} 12 & 4 & -6 \\ 0 & 40 & -5 \\ 18 & 12 & -9 \end{bmatrix}$

g) $\begin{bmatrix} 12 & 15 & 27 \\ 0 & 40 & 6 \\ -4 & -10 & -9 \end{bmatrix}$

h) $\begin{bmatrix} 12 & 6 & 18 \\ 0 & 40 & 10 \\ -6 & -6 & -9 \end{bmatrix}$

6.14 Code challenges

1.Â

Implement matrix multiplication between a 2 Ã— 4 matrix and a 4 Ã— 3 matrix, using the "layer perspective" in a for-loop. Confirm that you get the same result as when you compute matrix multiplication using @ (Python) or * (MATLAB).

2.Â

Generate a 4 Ã— 4 diagonal matrix and a 4 Ã— 4 dense matrix of random numbers. Compute both standard and Hadamard multiplication between them. You already know that the resulting product matrices are not the same, but what about the diagonals of those two product matrices?

3.Â

Consider $\mathbf{C}_1 = (\mathbf{A}^T + \mathbf{A}) \hat{a}\hat{\ }\bullet 2$ and $\mathbf{C}_2 = \mathbf{A}^T\mathbf{A}$ for some square nonsymmetric matrix $\mathbf{A}$. $\mathbf{C}_1$ and $\mathbf{C}_2$ are both symmetric and both formed from the same matrix, yet in general $\mathbf{C}_1 \hat{a}\%\circ \mathbf{C}_2$. Interestingly, if $\mathbf{A}$ is a diagonal matrix with all non-negative values, then $\mathbf{C}_1 = \mathbf{C}_2^{1\,\hat{a}\hat{\ }\bullet 2}$. Show this in code using random numbers, and then explain why you get this result.

4.Â

Letâ€™s explore the Cauchy-Schwarz inequality. Generate a random matrix $\mathbf{A}$ and a random vector $\mathbf{v}$, compute the norms of both sides of the inequality 6.27 (page Â§), and show that the right-hand side is larger than the left-hand side.

6.15 Code solutions

1.Â

Note: Showing equivalence between the two results is done by subtracting the matrices to get the zeros matrix (mathematically, this is $x = x$ â‡’ x â^’ $x = 0$). However, due to precision and rounding errors, the results might be very small numbers such as `1e-16` ($10^{\text{â^’}16}$). You can consider numbers smaller than around `1e-15` to be equal to zero.

Code blockÂ 6.13: Python

```
AÂ =Â np.random.randn(2,4)Â
BÂ =Â np.random.randn(4,3)Â
C1Â =Â np.zeros((2,3))Â
forÂ iÂ inÂ range(4):Â
Â Â Â Â C1Â +=Â np.outer(A[:,i],B[i,:])Â
Â
C1Â -Â A@BÂ #Â showÂ equality
```

Code blockÂ 6.14: MATLAB

```
AÂ =Â randn(2,4);Â
BÂ =Â randn(4,3);Â
C1Â =Â zeros(2,3);Â
forÂ i=1:4Â
Â Â Â Â C1Â =Â C1Â +Â A(:,i)*B(i,:);Â
endÂ
C1Â -Â A*BÂ %Â showÂ equality
```

2.Â

The diagonals of the two product matrices are the same.

Code blockÂ 6.15: Python

```
DÂ =Â np.diag(np.arange(1,5))Â
AÂ =Â np.random.randn(4,4)Â
C1Â =Â D*AÂ
C2Â =Â D@AÂ
print(np.diag(C1))Â
print(np.diag(C2))
```

Code blockÂ 6.16: MATLAB

```
DÂ =Â diag(1:4);Â
AÂ =Â randn(4);Â
C1Â =Â D.*A;Â
```

```
C2Â =Â D*A;Â
[diag(C1)Â diag(C2)]
```

3.Â

$\mathbf{C}_1 = \mathbf{C}_2^{1 \hat{a}\hat{\cdot} 2}$ sounds like an amazing and quirky property of matrix multiplication, but itâ€™s actually trivial because **A** is diagonal. The reason it works is the same reason that $x = \sqrt{x^2}$ for x â‰¥ 0 .

Code blockÂ 6.17: Python

```
AÂ =Â np.diag(np.random.rand(3))Â
C1Â =Â (A.T+A)/2Â
C2Â =Â A.T@AÂ
C1-np.sqrt(C2)
```

Code blockÂ 6.18: MATLAB

```
AÂ =Â diag(rand(3,1));Â
C1Â =Â (Aâ€™+A)/2;Â
C2Â =Â Aâ€™*A;Â
C1-sqrt(C2)
```

4.Â

The challenge isnâ€™t so challenging, but itâ€™s good excuse to gain experience coding with norms. My strategy for the inequality is to show that the right-hand side *minus* the left-hand side is positive. You can run the code multiples times to test different random numbers.

Code blockÂ 6.19: Python

```
importÂ numpyÂ asÂ npÂ
mÂ =Â 5Â
AÂ =Â np.random.randn(m,m)Â
vÂ =Â np.random.randn(m)Â
LHSÂ =Â np.linalg.norm(A@v)Â
RHSÂ =Â np.linalg.norm(A,ord=â€™froâ€™)Â *Â
Â Â Â Â Â Â np.linalg.norm(v)Â
RHS-LHSÂ #Â shouldÂ alwaysÂ beÂ positive
```

Code blockÂ 6.20: MATLAB

```
mÂ =Â 5;Â
AÂ =Â randn(m);Â
vÂ =Â randn(m,1);Â
LHSÂ =Â norm(A*v);Â
RHSÂ =Â norm(A,â€™froâ€™)*norm(v);Â
RHS-LHSÂ %Â shouldÂ alwaysÂ beÂ positive
```

ChapterÂ 7
Matrix rank

7.1 Six things to know about matrix rank

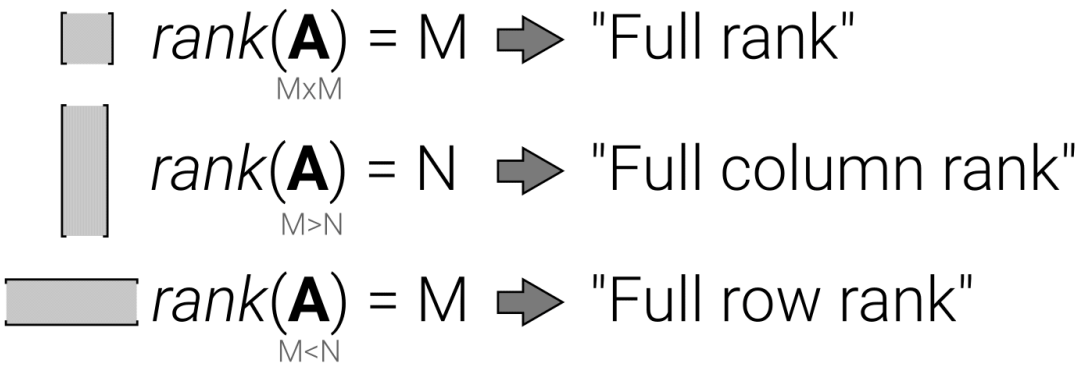
The rank of a matrix is a single number associated with that matrix, and is relevant for nearly all applications of linear algebra. Before learning about how to compute the rank, or even the formal definition of rank, itâ€™s useful to be exposed to a few key facts about matrix rank. In fact, you wonâ€™t learn the formal methods to compute rank until the next few chapters; here we focus on the concept and interpretations of rank.

1. The rank of the matrix is indicated by the letter *r* or by *rank* (**A**) , and is a non-negative integer. A matrix cannot have a rank of -2 or 4.7. A rank of 0 is possible, but most matrices have a rank > 0. If fact, only the zeros matrix can have a rank = 0.
2. The maximum possible rank of an *M* Ã— *N* matrix is the smaller of *M* or *N* . That is,

$$r \in \mathbb{N}, s.t.\ 0 \leq r \leq \min\{M, N\}$$ (7.1)

[1] $\mathbb{N}$ is the set of natural numbers, a.k.a. the counting numbers. There is some debate about whether 0 is in the set of natural numbers; it's a philosophical issue that some people choose not to worry about.

3. Rank is a property of the entire matrix; it doesnâ€™t make sense to talk about the rank of the columns of the matrix, or the rank of the rows of the matrix. In the next chapter, you will learn about matrix spaces; it also doesnâ€™t make sense to talk about the rank of the null space of the matrix.
4. The figure below shows terminology for full-rank matrices, depending on their sizes.



FigureÂ 7.1: Terminology for rank of matrices (boxes on left indicate matrix shapes).

If a matrix rank is less than the smaller of *M* or *N* , then it is variously called "reduced rank," "rank-deficient," "degenerate," "low-rank," or "singular." If you

are the kind of person who copes with lifeâ€™s challenges by insulting others, then you might also apply the epithets "dummy" or "loser" matrix.

5. The rank indicates the number of dimensions of *information* contained in the matrix. This is not the same as the total number of columns or rows in the matrix. For example, the following matrix has 3 columns and 2 rows, but a rank of 1; notice that the columns form a linearly dependent set (as do the rows), and thus that any one column is a scalar multiple of any other column (same for the rows).

$$\begin{bmatrix} 1 & 3 & 4 \\ 3 & 9 & 12 \end{bmatrix}$$

6. There are several definitions of rank that you will learn throughout this book, and several algorithms for computing the rank. However, the key definition to keep in mind is that the **rank of a matrix is the largest number of columns that can form a linearly independent set** . This is exactly the same as **the largest number of rows that can form a linearly independent set** .

-|*|- ***Reflection*** *Why all the fuss about rank? Why are full-rank matrices so important? There are some operations in linear algebra that are valid only for full-rank matrices (the matrix inverse being the most important). Other operations are valid on reduced-rank matrices (for example, eigendecomposition) but having full rank endows some additional properties. Furthermore, many computer algorithms return more reliable results when using full-rank compared to reduced-rank matrices. Indeed, one of the main goals of regularization in statistics and machine-learning is to increase numerical stability by ensuring that data matrices are full rank. So yeah, matrix rank is a big deal.* -|*|-

7.2 Interpretations of matrix rank

Algebraic interpretation As mentioned in point 6 above, if you think of a matrix as comprising a set of vectors, then the rank of the matrix corresponds to the largest number of vectors that can form a linearly independent set. Remember that a set of vectors is linearly independent if no vector can be expressed as a linear combination of the other vectors.

The phrasing that seems correct here would be "the number of linearly independent columns" in the matrix, but of course you know that linear independence is a property of a *set* of vectors, not *individual* vectors within a set.

Below are a few matrices and their ranks. Although I havenâ€™t yet taught you any algorithms for computing rank, try to understand why each matrix has its associated rank based on the description above.

$$[0pt][0pt]\begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}_{\mathrm{r=1}}, [0pt][0pt]\begin{bmatrix} 1 & 3 \\ 2 & 6 \\ 4 & 12 \end{bmatrix}_{\mathrm{r=1}}, [0pt][0pt]\begin{bmatrix} 1 & 3.1 \\ 2 & 6 \\ 4 & 12 \end{bmatrix}_{\mathrm{r=2}}, [0pt][0pt]\begin{bmatrix} 1 & 3 & 2 \\ 6 & 6 & 1 \\ 4 & 2 & 0 \end{bmatrix}_{\mathrm{r=3}}, [0pt][0pt]\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}_{\mathrm{r=1}}, [0pt][0pt]\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}_{\mathrm{r=0}}$$

Geometric interpretation Rank is the dimensionality of the subspace spanned by the columns (or the rows) of the matrix. This is not necessarily the same as the ambient dimensionality of the space containing the matrix. For example, consider the following vector (which we can also think of as a 3 Ã— 1 matrix):

$$\mathbf{v} = \begin{bmatrix} 4 & 0 & 1 \end{bmatrix}^{\mathrm{T}}$$

This object lives in â„ ^{3} , although it spans only a 1D subspace (a line). The rank of this matrix is therefore 1. In fact, all vectors have a rank of 1, except for the zeros vector, which has a rank of 0.

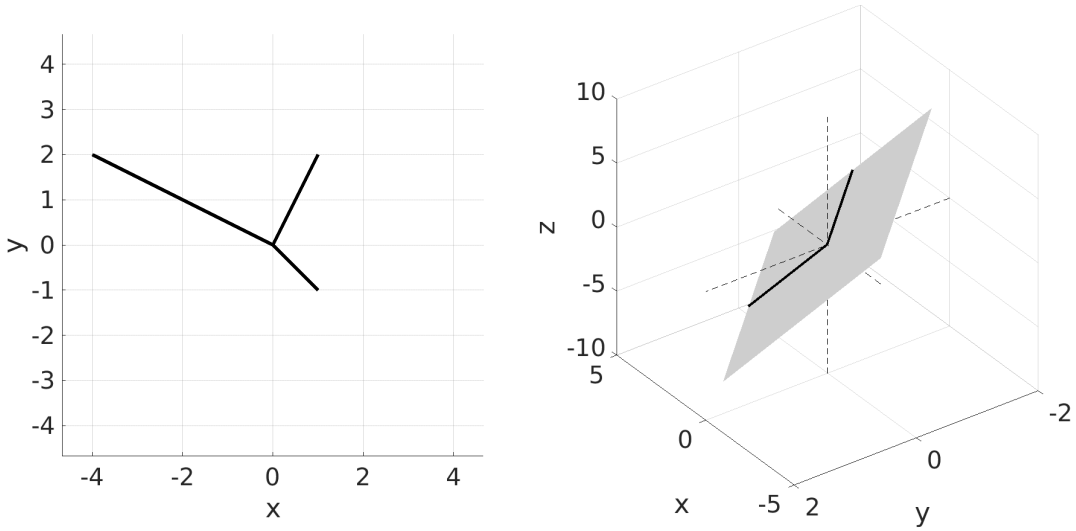
Letâ€™s reinterpret vector **v** : Instead of being a line in â„ ^{3} , letâ€™s consider it a row vector containing three elements in â„ ^{1} (three points on the real number line). In this case, either of the non-zero elements can obtained by scaling the other. Again, this leads us to the conclusion that the rank is 1.

Here is another example:

$$\begin{bmatrix} 1 & 1 & -4 \\ 2 & -1 & 2 \end{bmatrix}$$

Letâ€™s start by thinking about the matrix as comprising three column vectors in â„ ^{2} . There are three vectors in ambient dimensionality of 2, so this set of vectors spans all of â„ ^{2} , which is a 2D plane (Figure 7.2 , left panel). Thus, the rank is 2.

Now letâ€™s think about that matrix as comprising two row vectors in â„ ^{3} . Those two vectors are distinct, meaning they span a 2D plane embedded in ambient 3D space (Figure 7.2 , right panel). Thus, the rank is 2.



FigureÂ 7.2: Geometrically, the rank of a matrix is the dimensionality of the subspaces spanned by either the columns (left) or the rows (right).

The point is that regardless of the perspective you take (a matrix as comprising columns or rows), the dimensionality of the subspace spanned by those vectorsâ€”and thus, the rank of the matrixâ€”is the same. You can see in the example above that those subspaces might be very different (more on this in the next chapter!) but their dimensionalities are the same.

7.3 Computing matrix rank

Computing the matrix rank is, in modern applications, not trivial. In fact, beyond small matrices, computers cannot actually *compute* the rank of a matrix; they can only *estimate* the rank to a reasonable degree of certainty.

Nonetheless, computing the rank of small (and, if your linear algebra instructor is nice, integer-containing) matrices is not so difficult. There are several methods to compute the rank of a matrix. These methods all tap into the same underlying features of a matrix, but some methods are easier than others in some situations.

Below is a list of three methods to compute the rank of a matrix. At this point in the book, you can implement the first method; the other two methods rely on matrix operations that you will learn about in later chapters.

1. Count the largest number of columns (or rows) that can form a linearly independent set. This involves a bit of trial-and-error and a bit of educated guessing. You can follow the same tips for determining linear independence in Chapter 4 (page Â§).
2.

Count the number of pivots in the echelon or row-reduced echelon form of the matrix (Chapter 10).

3. Count the number of nonzero singular values from a singular value decomposition of the matrix (Chapter 16).

Code Except for your linear algebra exams, you will never compute matrix rank by hand. Fortunately, both Python and MATLAB have functions that return matrix rank.

Code blockÂ 7.1: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.random.randn(3,6)Â
rÂ =Â np.linalg.matrix_rank(A)
```

Code blockÂ 7.2: MATLAB

```
AÂ =Â randn(3,6);Â
rÂ =Â rank(A)
```

Practice problems Compute the rank of the following matrices based on visual inspection.

a) $\begin{bmatrix} 1 & -2 \\ 2 & -4 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 4 \\ 5 & -220 \\ 5 & 5/2 \end{bmatrix}$

d) $\begin{bmatrix} 2 & -4 & 10 \\ 2 & 3 & -4 \\ 4 & 2 & 0 \end{bmatrix}$

Answers

a) $r = 1$

b) $r = 2$

c) $r = 2$

d) $r = 2$

7.4 Rank and scalar multiplication

Iâ€™ll keep this brief: Scalar multiplication has no effect on the rank of a matrix, with one exception when the scalar is 0 (because this produces the zeros matrix, which has a rank of 0).

The reason why scalar multiplication has no effect is that the scalar simply stretches the information already present in the matrix; it does not transform, rotate, mix, unmix, change, combine, or create any new information.

Algebraically, scaling has no effect on rank because linear independence does not change if all vectors in the set are multiplied by the same scalar (of course, with the exception of 0).

I donâ€™t think this rule needs an equation, but in the interest of practicing my LaTeX skills:

$$rank(\alpha\mathbf{A}) = rank(\mathbf{A}), \;\; \alpha \neq 0 \tag{7.2}$$

Code There isnâ€™t any new code in this section, but I decided to add these code blocks to increase your familiarity with matrix rank. You should confirm that the ranks of the random matrix and the scalar-multiplied matrix are the same.

Code blockÂ 7.3: Python

```
sÂ =Â np.random.randn()Â
MÂ =Â np.random.randn(3,5)Â
r1Â =Â np.linalg.matrix_rank(M)Â
r2Â =Â np.linalg.matrix_rank(s*M)Â
print(r1,r2)
```

Code blockÂ 7.4: MATLAB

```
sÂ =Â randn;Â
MÂ =Â randn(3,5);Â
r1Â =Â rank(M);Â
r2Â =Â rank(s*M);Â
disp([r1Â r2])
```

7.5 Rank of added matrices

If you know the ranks of matrices **A** and **B** , do you automatically know the rank of **A** + **B** ? The short answer is No, you donâ€™t. However, knowing the ranks of the individual matrices will put an upper bound on the rank of **A** + **B** . Hereâ€™s the rule:

$$rank(\mathbf{A}+\mathbf{B}) \leq rank(\mathbf{A}) + rank(\mathbf{B}) \tag{7.3}$$

I remember this rule is by thinking about adding together the simplest possible rank-1 matrices to create a rank-2 matrix:

$$\begin{bmatrix}1&0&0\\0&0&0\\0&0&0\end{bmatrix}+\begin{bmatrix}0&0&0\\0&1&0\\0&0&0\end{bmatrix}=\begin{bmatrix}1&0&0\\0&1&0\\0&0&0\end{bmatrix}$$

On the other hand, adding two rank-1 matrices does not guarantee a rank-2 matrix:

$$\begin{bmatrix}1&0&0\\0&0&0\\0&0&0\end{bmatrix}+\begin{bmatrix}-1&0&0\\0&0&0\\0&0&0\end{bmatrix}=\begin{bmatrix}0&0&0\\0&0&0\\0&0&0\end{bmatrix}$$

With that in mind, here are some more examples. The ranks of the individual matrices are given in small subscripted numbers (*n.b.* non-standard notation used only in this chapter for convenience), but the ranks of the summed matrices are missing. You should compute them on your own, then check in the footnote [2] for the answers.

[2] Top to bottom: 3, 3, 2, 1

$$\begin{bmatrix}1&2&3\\3&4&1\\5&9&1\end{bmatrix}_3+\begin{bmatrix}0&3&5\\1&0&4\\3&3&0\end{bmatrix}_3=\begin{bmatrix}1&5&8\\4&4&5\\8&12&1\end{bmatrix} \tag{7.4}$$

$$\begin{bmatrix}1&2&0\\3&4&0\\5&9&0\end{bmatrix}_2+\begin{bmatrix}0&0&5\\0&0&4\\0&0&1\end{bmatrix}_1=\begin{bmatrix}1&2&5\\3&4&4\\5&9&1\end{bmatrix} \tag{7.5}$$

$$\begin{bmatrix}1&1&1\\2&2&2\\3&3&0\end{bmatrix}_2+\begin{bmatrix}0&0&0\\0&0&0\\0&0&0\end{bmatrix}_0=\begin{bmatrix}1&1&1\\2&2&2\\3&3&0\end{bmatrix} \tag{7.6}$$

$$\begin{bmatrix}-1&-4&2\\-4&2&-1\\9&4&-3\end{bmatrix}_3+\begin{bmatrix}1&4&0\\4&-2&0\\-9&-4&0\end{bmatrix}_2=\begin{bmatrix}0&0&2\\0&0&-1\\0&0&-3\end{bmatrix} \tag{7.7}$$

The rule shown in Equation 7.3 applies to matrix subtraction as well, because subtraction is the same thing as addition and multiplying the matrix by -1, and scalar multiplication doesnâ€™t change the rank of a matrix (see Equation 7.2).

Multiple constraints There are multiple constraints on the rank of a matrix. For example, "Thing 2" to know about matrix rank was that the largest possible rank is the smaller of *M* or *N* (Equation 7.1). These constraints cannot be broken.

For example, imagine two 3 Ã— 4 matrices, each with rank 3. The sum of those matrices cannot possibly have a rank of 6 (Equation 7.3), because 6 is greater than the matrix sizes. Thus, the largest possible rank of the summed matrix is 3 (the rank could be smaller than 3, depending on the values in the matrices). _____

Practice problems Determine the maximum possible rank of the following expressions, based on the following information (assume matrix sizes match for valid addition):
rank (**A**) = 4 , *rank* (**B**) = 11 , *rank* (**C**) = 14 .

a) **A** + **B**
b) (**A** + **B**) + 0 **C**
c) **C** â^' 3 **A**
d) 0((**C** + **A**) + 4 **A**)

Answers

a) 15
b) 15
c) 18
d) 0

7.6 Rank of multiplied matrices

As with summed matrices, you cannot know the exact rank of a matrix **C** = **AB** only by knowing the individual ranks of **A** and **B** . But also as with summed matrices, there is a rule for knowing the maximum possible rank of the product matrix.

$$rank(\mathbf{AB}) \leq \min\{rank(\mathbf{A}), rank(\mathbf{B})\} \tag{7.8}$$

In other words, the smallest rank of the individual matrices is the largest possible rank of the product matrix. Below are a few examples, and then Iâ€™ll explain why this rule makes sense. These are the same pairs of matrices as in the previous section, so itâ€™s interesting to compare the results. (As in the previous section, ranks are listed in the subscripts, and the ranks of the products are listed in the footnote[3].)

[3] Top to bottom: 3, 1, 0, 2

$$\begin{bmatrix}1&2&3\\3&4&1\\5&9&1\end{bmatrix}_3\begin{bmatrix}0&3&5\\1&0&4\\3&3&0\end{bmatrix}_3=\begin{bmatrix}11&12&13\\7&12&31\\12&18&61\end{bmatrix} \tag{7.9}$$

$$\begin{bmatrix}1&2&0\\3&4&0\\5&9&0\end{bmatrix}_2\begin{bmatrix}0&0&5\\0&0&4\\0&0&1\end{bmatrix}_1=\begin{bmatrix}0&0&13\\0&0&31\\0&0&61\end{bmatrix} \tag{7.10}$$

$$\begin{bmatrix}1&1&1\\2&2&2\\3&3&0\end{bmatrix}_2\begin{bmatrix}0&0&0\\0&0&0\\0&0&0\end{bmatrix}_0=\begin{bmatrix}0&0&0\\0&0&0\\0&0&0\end{bmatrix} \tag{7.11}$$

(7.12)

$$\begin{bmatrix} -1 & -4 & 2 \\ -4 & 2 & -1 \\ 9 & 4 & -3 \end{bmatrix}_3 \begin{bmatrix} 1 & 4 & 0 \\ 4 & -2 & 0 \\ -9 & -4 & 0 \end{bmatrix}_2 = \begin{bmatrix} -35 & -4 & 0 \\ 13 & -16 & 0 \\ 52 & 40 & 0 \end{bmatrix}$$

How to understand this rule? You can think about it in terms of the column space of the matrix **C** = **AB** . "Column space" is a concept youâ€™ll learn more about in the next chapter, but basically itâ€™s the subspace spanned by the columns of a matrix. Think of the *j* *th* column of **C** as being the matrix-vector product of matrix **A** and the *j* *th* column in **B** :

$$\mathbf{A}\mathbf{b}_j = \mathbf{c}_j \tag{7.13}$$

This means that each column of **C** is a linear combination of columns of **A** with the weights defined by the corresponding column in **B** . In other words, each column of **C** is in the subspace spanned by the columns of **A** .

One definition of rank is the dimensionality of the column space of a matrix. And because the column space of **C** is fully within the column space of **A** , the dimensionality of **C** â€™s column space cannot be larger than that of **A** . **C** â€™s column space could be smaller, depending on the numbers in **B** , but the rank of **A** provides an upper boundary.

The same argument can be made for matrix **B** by considering **C** to be built up row-wise instead of column-wise.

-|*|- ***Reflection*** *The rules in the past two sections will prepare you for the next two sections, which have major implications for applied linear algebra, primarily statistics and machine-learning. Moreover, the more comfortable you are with matrix rank, the more intuitive advanced linear algebra concepts will be.* -|*|- _____

Practice problems Determine the maximum possible rank of the following expressions, based on the following information (assume valid matrix sizes): *rank* (**A**) = 4 , *rank* (**B**) = 11 , *rank* (**C**) = 14 .

a) **AB**
b) (**AB**) **C**
c) 3 **CA**
d) (**C** + **A**) **A**

Answers

a) 4
b) 4
c) 4
d) 4

7.7 Rank of A , $\mathbf{A}^T$, $\mathbf{A}^T\mathbf{A}$, and $\mathbf{A}\mathbf{A}^T$

The key take-home message from this section is that these four matricesâ€”**A** , $\mathbf{A}^T$, $\mathbf{A}^T$ **A** , and $\mathbf{A}\mathbf{A}^T$ â€”all have exactly the same rank.

You already know that **A** and $\mathbf{A}^T$ have the same rank because of property 3 in the first section of this chapter: the rank is a property of the matrix; it does not reflect the columns or the rows separately. Thus, transposing a matrix does not affect its rank.

Proving that $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}\mathbf{A}^T$ have the same rank as that of **A** takes a little more work. Iâ€™m going to present two explanations here. Unfortunately, both of these explanations rely on some concepts that I will introduce later in the book. So if you find these explanations confusing, then please ear-mark this page and come back to it later. I know itâ€™s a bit uncomfortable to rely on concepts before learning about them, but it often happens in math (and in life in general) that a purely monotonic progression is impossible.

Proof 1: The rank-nullity theorem The first proof relies on a discussion of matrix null spaces, which you will learn more about in the next chapter. Briefly, the null space of a matrix is the set of all vectors **y** such that **Ay** = **0** (excluding the trivial case of **y** = **0**). The proof involves showing that $\mathbf{A}^T\mathbf{A}$ and **A** have the same null space dimensionality, which means that they must have the same rank. We start by proving that **A** and $\mathbf{A}^T\mathbf{A}$ have the same null space.

$$\mathbf{A}\mathbf{y} = \mathbf{0} \tag{7.14}$$

$$\mathbf{A}^T\mathbf{A}\mathbf{y} = \mathbf{A}^T\mathbf{0} \tag{7.15}$$

$$\mathbf{A}^T\mathbf{A}\mathbf{y} = \mathbf{0} \tag{7.16}$$

Equations 7.14 and 7.16 show that any vector in the null space of **A** is also in the null space of $\mathbf{A}^T\mathbf{A}$. This proves that the null space of $\mathbf{A}^T\mathbf{A}$ is a subset of the null space of **A** . Thatâ€™s half of the proof, because we also need to show that any vector in the null space of $\mathbf{A}^T\mathbf{A}$ is also in the null space of **A** .

$$\mathbf{A}^T\mathbf{A}\mathbf{y} = \mathbf{0} \tag{7.17}$$

$$\mathbf{y}^T\mathbf{A}^T\mathbf{A}\mathbf{y} = \mathbf{y}^T\mathbf{0} \tag{7.18}$$

$$(\mathbf{A}\mathbf{y})^T(\mathbf{A}\mathbf{y}) = 0 \tag{7.19}$$

$$\hat{\text{¥}}\,\mathbf{A}\mathbf{y}\,\hat{\text{¥}} = 0 \tag{7.20}$$

Equations 7.17 and 7.20 together show that any vector in the null space of $\mathbf{A}^T\mathbf{A}$ is also in the null space of **A** .

Now weâ€™ve proven that $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}$ have the same null spaces. Why does that matter? You will learn in the next chapter that the row space (the set of all possible weighted combinations of the rows) and the null space together span all of â„^{N}, and so if the null spaces are the same, then the row spaces must have the same dimensionality (this is called the rank-nullity theorem). And the rank of a matrix is the dimensionality of the row space, hence, the ranks of $\mathbf{A}^T\mathbf{A}$ and $\mathbf{A}$ are the same.

Proving this for $\mathbf{AA}^T$ follows the same proof as above, except you start from $\mathbf{y}^T\mathbf{A} = \mathbf{0}$ instead. I encourage you to reproduce the proof with a pen and some paper.

Proof 2: The singular value decomposition This explanation relies on another definition of rank, which is the number of non-zero singular values. Briefly, the singular value decomposition (SVD) involves representing a matrix as the product of three other matrices: $\mathbf{A} = \mathbf{U\hat{I}£V}^T$. **Î£** is a diagonal matrix that contains the singular values. The SVD of $\mathbf{A}^T\mathbf{A}$ is

$$\mathbf{A}^T\mathbf{A} = (\mathbf{U\hat{I}£V}^T)^T\mathbf{U\hat{I}£V}^T \quad (7.21)$$

$$= \mathbf{V\hat{I}£U}^T\mathbf{U\hat{I}£V}^T \quad (7.22)$$

$$= \mathbf{V\hat{I}£}^2\mathbf{V}^T \quad (7.23)$$

$\mathbf{U}$ is an orthogonal matrix, so $\mathbf{U}^T\mathbf{U} = \mathbf{I}$. Donâ€™t worry if these equations seem mysteriousâ€”thatâ€™s the whole point of Chapter 16! The important point for now is that the rank of a matrix is the number of non-zero diagonal elements in **Î£**, and that is exactly the same as the number of non-zero diagonal elements in $\mathbf{\hat{I}£}^2$. Thus, $\mathbf{A}$ and $\mathbf{A}^T\mathbf{A}$ have the same rank.

Letâ€™s see an example. The following page shows a 2 Ã— 3 rank-1 matrix, and then that matrix transposed and multiplied by its transpose. You can confirm via visual inspection that the ranks of all these matrices are 1.

$$\mathbf{A} = \begin{bmatrix} 1 & 3 & 4 \\ 3 & 9 & 12 \end{bmatrix} \qquad rank(\mathbf{A}) = 1$$

$$\mathbf{A}^T = \begin{bmatrix} 1 & 3 \\ 3 & 9 \\ 4 & 12 \end{bmatrix} \qquad rank(\mathbf{A}^T) = 1$$

$$\mathbf{A}^T\mathbf{A} = \begin{bmatrix} 10 & 30 & 40 \\ 30 & 90 & 120 \\ 40 & 120 & 160 \end{bmatrix} \quad rank(\mathbf{A}^T\mathbf{A}) = 1$$

$$\mathbf{AA}^T = \begin{bmatrix} 26 & 78 \\ 78 & 234 \end{bmatrix} \qquad rank(\mathbf{AA}^T) = 1$$

7.8 Rank of random matrices

A "random matrix" is a matrix that contains elements drawn at random. The elements of random matrices can come from various distributions, such as normal (Gaussian), uniform, Poisson, etc. Random matrices have some interesting properties, and there are entire theories built around random matrices.

The interesting property of random matrices that is most relevant for this bookâ€”and for using computers to explore concepts in linear algebraâ€”is that they are basically always full rank. Almost any time you populate a matrix with random numbers, you can assume that that matrix will have its maximum possible rank (there are some exceptions described below).

Why is this the case? When you generate random numbers on computersâ€”particularly floating-point precision random numbersâ€”it is simply mindbogglingly unlikely that linear dependencies in the columns will *just happen* to arise. Here is an example of a 4 Ã— 4 matrix of random numbers I generated in MATLAB using the function `randn`:

$$\begin{bmatrix} -0.744413799273930254507547488174 17 & 1.515314090414003067586179440695 56 & 0.421765615011302053005692869191 99 & 1.255126642192404258224769364460 37 \\ -0.726467752184593695830017168191 26 & -0.340722423993896184057916798337 83 & 0.386616833363512304000408903448 26 & 0.769576495340373534226330320962 01 \\ -1.686910932225691883705130177645 48 & 0.461062814179895974486811383030 73 & -0.158464035048825779838921334885 53 & 1.373587622180848745223613605048 74 \\ -0.189662200874316150267517855354 53 & 0.447335850049224992286411861641 68 & 0.696271659820789867723078714334 41 & -1.477562570273953942603384348331 02 \end{bmatrix}$$

Apologies for making the font so small. The point isnâ€™t for you to read the actual numbers; the point is for you to appreciate that the probability of linear dependencies leading to a reduced-rank matrix is infinitesimal. Thus, whenever you create random matrices on computers, you can assume that their rank is their *maximum possible* rank.

On the other hand, matrices are not guaranteed to be full-rank simply by virtue of having randomly drawn numbers. Reduced-rank random matrices occur when the numbers are drawn from a restricted range. For example, the following matrix was generated by random numbers drawn from the population {0,1} and has a rank of 2.

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

Thus, matrices populated with floating-point random numbers have maximum possible rank. This is useful because it allows you to create matrices with arbitrary rank, which in turn will unlock many opportunities for exploring linear algebra in code. Code challenge 1 will guide you through the process.

7.9 Full-rank matrices via â€œshiftingâ€

Full-rank square matrices are absolutely fabulous to work with. But many matrices used in practical applications are rank-deficient. So whatâ€™s a data-scientist to do?

One solution is to transform a rank-deficient matrix into a full-rank matrix through "shifting," which you learned about in section 5.8 . As a quick reminder: shifting a matrix means to add a multiple of the identity matrix ($\mathbf{A} + \hat{I}» \mathbf{I} = \mathbf{A}$) , which adds a small quantity to the diagonal elements without changing the off-diagonal elements.

In statistics and machine learning, one of the main reasons to shift a matrix is to transform a reduced-rank matrix into a full-rank matrix. Remarkably, this feat can be accomplished while making only tiny (and usually practically insignificant) changes to the information contained in the matrix.

To show how this is possible, Iâ€™ll start with an extreme example that shows how shifting a matrix can transform it into a full-rank matrix. What is the value of $\hat{I}»$ in the equation below?

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \lambda \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{7.24}$$

Iâ€™m sure you calculated that $\hat{I}» = 1$, and shifting trivially moved the matrix rank from 0 to 3. On the other hand, usually the goal of shifting a matrix is to change the information contained in the matrix as little as possible, and Iâ€™ve clearly violated that principle here.

Letâ€™s go for a less extreme example:

$$\begin{bmatrix} 1 & 3 & -19 \\ 5 & -7 & 59 \\ -5 & 2 & -24 \end{bmatrix} + .01 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1.01 & 3 & -19 \\ 5 & -6.99 & 59 \\ -5 & 2 & -23.99 \end{bmatrix} \tag{7.25}$$

The ranks of these matrices are, respectively, 2, 3, and 3. And unlike with example 7.24 , the matrices **A** and **A** are really close to each other: All the off-diagonal elements are identical, and the diagonal elements differ by a mere .01, which, for this matrix, corresponds to a change of less than 1% on the diagonal elements.

One final (again, somewhat extreme) example:

$$\begin{bmatrix} 1 & 3 & -19 \\ 5 & -7 & 59 \\ -5 & 2 & -24 \end{bmatrix} + 10^3 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1001 & 3 & -19 \\ 5 & 993 & 59 \\ -5 & 2 & 976 \end{bmatrix} \tag{7.26}$$

Again, the ranks are 2, 3, and 3. But letâ€™s think about what weâ€™ve done: By setting $\hat{I}»$ to be large relative to the values in **A** , weâ€™ve pushed the matrix to be close to a scaled version of the identity matrix (in fact, we could even say that that matrix is 10^3 **I** plus some flotsam and jetsam).

This leads to an interesting observation about matrix "shifting": When $\hat{I}»$ is close to 0, **A** is close to **A** , and as $\hat{I}»$ gets large (relative to the values in matrix **A**), **A** moves towards $\hat{I}»$ **I** .

In the context of statistics and machine learning, "shifting" is also called *regularization* or *matrix smoothing* . It is an important procedure for multivariate analyses such as principal components analysis and generalized eigendecomposition (which are the mathematical backbones of data compression and linear discriminant analyses).

An important questionâ€”and an area of ongoing research and testing in data scienceâ€”is determining the optimal $\hat{I}»$ for a given matrix and a given application. Iâ€™ll introduce one method, called "shrinkage regularization," in chapter 19.

7.10 Difficulties in computing rank in practice

Iâ€™d like to give you some sense of why it is difficult to compute the rank of large matrices, from both an algebraic and geometric interpretation.

I wrote above that one way to compute the rank of a matrix is to count the number of non-zero singular values. You havenâ€™t yet learned about singular values, but it is sufficient for now to know that an M Ã— N matrix has min { M,N } singular values.

Computers suffer from rounding errors that lead to uncertainties in distinguishing very small numbers from true zero. Oftentimes, numbers smaller than around $10^{\hat{a}^{\check{}}\ 15}$ are considered to be zero plus computer rounding error (the exact exponent depends on your computerâ€™s precision).

So letâ€™s say the computer estimates a singular value to be $3 Ã— 10^{\hat{a}^{\check{}}\ 15}$; is this a true non-zero singular value that happens to be really small, or is this actually zero but with rounding error? Your computer software (MATLAB or Python) will define some threshold for considering a small singular value equal to zero. But that threshold is arbitrary, and a rounding error could be randomly on either side of that boundary. Therefore, rounding error plus an arbitrary threshold can influence the reported rank of the matrix. Weâ€™ll come back to this issueâ€”including a discussion of how that threshold is setâ€”in Chapter 16.

Code The standard computer algorithm for computing rank is to count the number of singular values above a threshold. The purpose of this code block is for you to inspect the source code for computing rank. Even if you donâ€™t understand each line of the code, you should be able to see the following operations: (1) compute the SVD of the matrix, (2) define a "tolerance" (the threshold for identifying a number as being significantly nonzero), (3) count the number of singular values above this tolerance.

Code blockÂ 7.5: Python

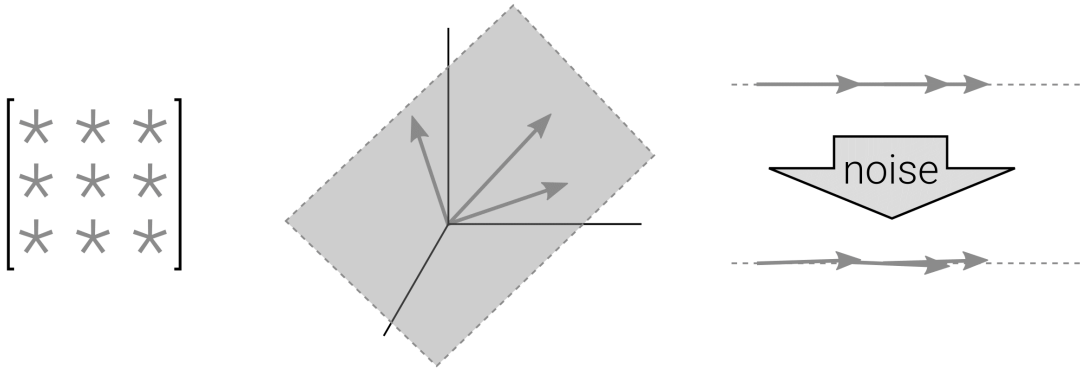
```
??np.linalg.matrix_rank
```

Code blockÂ 7.6: MATLAB

Geometry Now letâ€™s think about difficulties in computing rank geometrically. Imagine you have a 3 Ã— 3 matrix that represents some data you collected from a satellite. The columns are in â„ ^{3}, and letâ€™s imagine that you know for a fact that the three vectors all lie on a 2D plane. So you know for a fact, based on the design of the satelliteâ€™s sensors, that the rank of the data matrix must be 2.

But the sensors on the satellite are not perfect, and there is a tiny, tiny bit of noise that corrupts the signal. So in fact, if you would look at the subspace spanned by the columns of the matrix at "eye level," you would expect to see the vectors perfectly lying in a plane. Instead, however, you see the vectors pointing ever-so-slightly above or below that plane (Figure 7.3).

Your computer would tell you that the rank of this data matrix is 3, which you know is actually due to sensor noise. So you might want your rank-estimating-algorithm to ignore some small amount of noise, based on what you know about the data contained in the matrix.



FigureÂ 7.3: A 3 Ã— 3 matrix representing a 2D plane in â„ ^{3} may be counted as a rank-3 matrix in the presence of a small amount of noise. The diagrams at right illustrate three arrows on the plane (dotted line), from the perspective of looking directly across the surface of that plane.

7.11 Rank and span

In Chapter 4, you learned that an important question in linear algebra is whether a vector is in the span of another set of vectors. (Recall that a vector **v** is in the span of a set of vectors *S* if **v** can be written as a weighted combination of the vectors in *S* .)

I wrote that there are several algorithms that can answer that question, but that you hadnâ€™t yet learned the necessary concepts. We can now re-interpret this problem in the context of matrices and rank.

There is an entire section about this procedure in the next chapter (section 8.3), including a deeper explanation and diagrams.[4] However, I believe that you are now knowledgeable enough to be introduced to the augment-rank algorithm. If youâ€™re struggling with understanding why the following procedure tells us whether a vector is in the span of a set of vectors, then donâ€™t worryâ€”it means you have something to look forward to in the next chapter!

[4] Repetition facilitates comprehension, especially when knowledge increases between each repetition.

1. Put the vectors from set S into a matrix $\mathbf{S}$.
2. Compute the rank of $\mathbf{S}$. Call that rank r_1 .
3. Augment $\mathbf{S}$ by $\mathbf{v}$, thus creating $\mathbf{S}_\mathbf{v} = \mathbf{S}$ âŠ” $\mathbf{v}$
4. Compute the rank of $\mathbf{S}_\mathbf{v}$. Call that rank r_2 .
5. If $r_2 > r_1$, then $\mathbf{v}$ is *not* in the span of S .
   If $r_2 = r_1$, then $\mathbf{v}$ *is* in the span of S .
   If $r_2 < r_1$, then check your math or code for a mistake :(

7.12 Exercises

1.Â

Compute the rank of the following matrices based on visual inspection.

a) $\begin{bmatrix} 1 & 2 & 2 \\ 2 & 4 & 4 \\ 4 & 8 & 8 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 & 2 \\ 2 & 4 & 4 \\ 4 & 8 & 9 \end{bmatrix}$

c) $\begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$

d) $\begin{bmatrix} 1 & 3 & 5 \\ 0 & 8 & 3 \\ 0 & 0 & 23 \end{bmatrix}$

2.Â

Compute the rank of the result of the following operations.

$$\mathbf{A} = \begin{bmatrix} 2 & 4 & 3 \\ 0 & 1 & 3 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} -2 & -1 & 3 \\ 6 & -7 & 7 \end{bmatrix}$$

a) **A**

b) **B**

c) $\mathbf{A} + \mathbf{B}$
d) $\mathbf{A}\mathbf{A}^{\mathbf{T}}$
e) $\mathbf{A}^{\mathbf{T}}\mathbf{A}$
f) $\mathbf{A}\mathbf{B}^{\mathbf{T}}$
g) $(\mathbf{A}\mathbf{B}^{\mathbf{T}})^{\mathbf{T}}$
h) $2\,\mathbf{A}\mathbf{A}^{\mathbf{T}}$

3.Â
For the following matrices, what value of *Î»* would give each matrix rank *m* â^' 1 ?

a) $\begin{bmatrix} 1 & 3 \\ 1 & \lambda \end{bmatrix}$
b) $\begin{bmatrix} 0 & 0 \\ 0 & \lambda \end{bmatrix}$
c) $\begin{bmatrix} 1 & 2 & \lambda \\ 1 & 2 & 2 \\ 1 & 2 & 2 \end{bmatrix}$
d) $\begin{bmatrix} 0 & 1 & 0 \\ 0 & 2 & \lambda \\ 1 & 0 & 3 \end{bmatrix}$

4.Â
Determine the maximum possible rank of the following operations.
A â^^ â„ $^{2\,Ã—\,3}$, **B** â^^ â„ $^{3\,Ã—\,3}$, **C** â^^ â„ $^{3\,Ã—\,4}$, **D** â^^ â„ $^{3\,Ã—\,4}$

a) **A**
b) **B**
c) **C**
d) **D**
e) $\mathbf{C}^{\mathrm{T}}\mathbf{B}$
f) $\mathbf{C}^{\mathrm{T}}\mathbf{C}$
g) **AD**
h) **CD**
i) $\mathbf{B} + \mathbf{B}$
j) $\mathbf{C} + \mathbf{D}$
k) $\mathbf{B}\mathbf{A}^{\mathbf{T}}\mathbf{A}\mathbf{C}$
l) $\mathbf{B}\mathbf{A}^{\mathbf{T}}\mathbf{A}\mathbf{C} + \mathbf{D}$

7.13 Answers

1.Â
a) $r = 1$

b) $r = 2$
c) $r = 3$
d) $r = 3$

2.Â

a) 2
b) 2
c) 2
d) 2
e) 2
f) 2
g) 2
h) 2

3.Â

a) $\hat{I}» = 3$
b) $\hat{I}»â‰ 0$
c) $\hat{I}»â‰ 2$
d) $\hat{I}» = 0$

4.Â

a) 2
b) 3
c) 3
d) 3
e) 3
f) 3
g) 2
h) invalid!
i) 3
j) 3
k) 2
l) 3

7.14 Code challenges

1.Â

The goal of this code challenge is to create random matrices with any arbitrary rank (though still limited by the constraints presented in this chapter). In particular, combine standard matrix multiplication (previous chapter) with the rule about rank

and matrix multiplication (Equation 7.8) to create reduced-rank matrices comprising random numbers (hint: think about the "inner dimensions" of matrix multiplication).

2.Â

The goal of this code challenge is to explore the tolerance level of your computer for computing the rank of matrices with tiny values. Start by creating the 5 Ã— 5 zeros matrix and confirm that its rank is 0. Then add a 5 Ã— 5 random numbers matrix scaled by machine-epsilon, which is the computerâ€™s estimate of its numerical precision due to round-off errors. Now the rank of that summed matrix will be 5. Finally, keep scaling down the machine-epsilon until the rank of the summed matrix is 0. You can also compute the Frobenius norm to get a sense of the magnitude of the values in the matrix.

7.15 Code solutions

1.Â

When multiplying two random matrices that are (M Ã— N) Ã— (N Ã— K) where $M > N$ and $K > N$, then the product M Ã— K matrix will have rank-N . The code below shows how to create a rank-2 random matrix. I encourage you to generalize this code so that the parameters M , N , and k are soft-coded.

Code blockÂ 7.7: Python

```
AÂ =Â np.random.randn(9,2)Â
BÂ =Â np.random.randn(2,16)Â
CÂ =Â A@B
```

Code blockÂ 7.8: MATLAB

```
AÂ =Â randn(9,2);Â
BÂ =Â randn(2,16);Â
CÂ =Â A*B;
```

2.Â

On the laptop that Iâ€™m using while writing this exercise, I got rank-5 matrices down to scaling factors of around $10^{\text{âˆ’}307}$.

Code blockÂ 7.9: Python

```
ZÂ =Â np.zeros((5,5))Â
NÂ =Â np.random.randn(5,5)Â
ZNÂ =Â ZÂ +Â N*np.finfo(float).eps*1e-307Â
print(np.linalg.matrix_rank(Z))Â
print(np.linalg.matrix_rank(ZN))Â
print(np.linalg.norm(ZN,â€™froâ€™))
```

Code blockÂ 7.10: MATLAB

```
ZÂ =Â zeros(5);Â
NÂ =Â randn(5)*eps*1e-307;Â
ZNÂ =Â ZÂ +Â N;Â
rank(Z)Â %Â r=0Â
rank(ZN)Â %Â r=5Â
norm(ZN,â€™froâ€™)
```

ChapterÂ 8
Matrix spaces

"Matrix spaces" are the same concept as vector subspaces, but are defined by different features of a matrix instead of a set of vectors. This chapter ties together concepts of vector subspaces, basis, linear independence, and span.

Remarkably, this entire chapter can be summarized as trying to answer the following two questions about matrix-vector multiplication and a vector $\mathbf{b}$ *â‰ * $\mathbf{0}$:

$$\mathbf{Ax} = \mathbf{b} \; ? \tag{8.1}$$

$$\mathbf{Ay} = \mathbf{0} \; ? \tag{8.2}$$

In other words, the questions are: (8.1) Is there a weighted combination of columns of **A** that produces vector **b** with the weights given by vector **x** ? And (8.2) Is there a weighted combination of columns of **A** that produces the zeros vector with the weights given by vector **y** (for **y** *â‰ * **0**)?

As you work through this chapter, try to think about how each concept fits into these questions. At the end of the chapter, I will provide a philosophical discussion of the meaning of these two questions.

8.1 Column space of a matrix

"Column space" sounds like a fancy and exotic term, but in fact, you already know what a column space is: The *column space* of a matrix is the subspace spanned by all columns of that matrix.[1] In other words, think of a matrix as a set of column vectors, and the subspace spanned by that set of vectors as the column space of the matrix.

[1] *Column space* is also sometimes called the *range* or the *image* of a matrix.

The column space is indicated using the notation $C(\mathbf{A})$. Here is a formal definition.

Eqn. box title=**Column space of a matrix**

$$C(\mathbf{A}) = \{\beta_1 a_1 + \ldots \beta_n a_n, \quad \beta \in \mathbb{R}\} \tag{8.3}$$

$$C(\mathbf{A}) = span(a_1,\ldots,a_n) \tag{8.4}$$

The two equations above show two different, and equally acceptable, ways to express the same concept.

The main difference between a subspace spanned by a set of vectors vs. the column space of a matrix is conceptual: a set of vectors is a collection of separate objects, whereas a matrix is one unit; it can be convenient to talk about particular groups of elements in a matrix as if they were column vectors. But the matrix elements that form columns always are also part of rows, and they are also individual elements. That fluid level of reorganization isnâ€™t possible with sets.

The column space of an M Ã— N matrix is in â„ ^{M} . Donâ€™t be confused and think that the column space is in â„ ^{N} because there are N columns; each of those N columns has M elements, hence, C ($\mathbf{A}$) âˆˆ â„ ^{M} . If the rank of the matrix is $r = M$, then the column space spans all of â„ ^{M} ; if the rank is $r < M$, then the column space is an r -dimensional subspace of â„ ^{M} .

Also relevant here is the distinction between basis and span. The columns of a matrix *span* a subspace, but they may or may not be a *basis* for that subspace. Remember that a set of vectors is a basis for a subspace only if that set is linearly independent.

Consider the two matrices below; their column spaces are identical, but the columns of the left matrix form a basis for that subspace, whereas the columns of the right matrix do not.

$$\begin{bmatrix} 3 & 9 \\ 7 & 5 \\ 1 & 8 \\ 2 & 7 \\ 9 & 6 \end{bmatrix}, \quad \begin{bmatrix} 3 & 9 & 9 \\ 7 & 5 & 5 \\ 1 & 8 & 8 \\ 2 & 7 & 7 \\ 9 & 6 & 6 \end{bmatrix}$$

Letâ€™s try a few examples. For each of the matrices below, determine (1) the dimensionality of the ambient space in which the column space is embedded, (2) the dimensionality of the column space, and (3) whether the columns form a basis for the column space.

$$\mathbf{B} = \begin{bmatrix} 1 & 2 & 4 \\ 0 & 4 & 4 \\ 4 & 1 & 9 \\ 6 & 0 & 12 \\ 1 & 1 & 3 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 2 & 3 & 5 \\ 0 & 8 & 13 & 21 \\ 0 & 0 & 34 & 55 \\ 0 & 0 & 0 & 89 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 1 & 5 & -3 & 4 \\ 2 & 6 & -2 & 4 \\ 3 & 7 & -1 & 4 \\ 4 & 8 & 0 & 4 \\ 5 & 9 & 1 & 4 \\ 6 & 10 & 2 & 4 \\ 7 & 11 & 3 & 4 \end{bmatrix}$$

The column spaces of matrices **B** , **C** , and **D** live in ambient dimensionalities, respectively, of 5, 4, and 7 (an alternative phrasing is that C (**B**) â^^ â„ ^{5} , C (**C**) â^^ â„ ^{4} , C (**D**) â^^ â„ ^{7}); this is determined simply by counting the number of elements in the columns, which is M .

The dimensionalities of the column spaces is a bit more work to compute, [2] because it involves determining the largest number of columns that can form a linearly independent set of vectors, which is the same thing as the rank of the matrix. Some trial-and-error guesswork plus a bit of arithmetic, should lead you to the answers of 2, 4, and 2.

[2] If the matrix is full column rank, then the column space spans all of â„ ^{M}

Once you know the dimensionality of the column space, determining whether the columns form a basis for the column space is easy: The columns form a basis for the column space if the subspace dimensionality is the same as the number of columns. Thus, the answers for the three matrices are No, Yes, and No. It is possible to use the columns to create basis sets for the column spaces. For example, the first two columns of matrix **B** can be used as a basis set for C (**B**) .

As a quick reminder, there is an infinite number of basis vectors for the column space of a matrix; the columns themselves are an easy-to-compute basis, but they are not necessarily the best basis; in fact, one of the goals of the singular value decomposition is to provide an orthogonal basis set for the column space of the matrix.

8.2 The column space of A and AA ^{T}

Interestingly, **A** and **AA** ^{T} have the same column space. Letâ€™s first confirm that the dimension of their ambient spaces are the same: Matrix **A** is size M Ã— N , so C(**A**) â^^ â„ ^{M} . Matrix **AA** ^{T} is size (M Ã— N) Ã— (N Ã— M) = M Ã— M , and therefore is also â^^ â„ ^{M} . This doesnâ€™t prove that their column spaces

are the same, but it is a prerequisite. (For example, it should now be obvious that **A** and $\mathbf{A}^{\mathrm{T}}\mathbf{A}$ cannot possibly have the same column space if *Mâ‰ N*).

Now letâ€™s see why the column spaces must be equal. Recall from section 6.1 that the "column perspective" of matrix multiplication states that multiplication is a linear weighted combination of the columns of the left matrix, where the weights come from the columns in the right matrix. Thus, $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ is simply a linear weighted combination of the columns of **A** , which means it is in the span of the column space of **A** .

Letâ€™s see this in an example. Iâ€™m going to write out the multiplication $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ using the column perspective.

$$\begin{bmatrix} 0 & 10 \\ 3 & 7 \\ 5 & 3 \end{bmatrix} \begin{bmatrix} 0 & 3 & 5 \\ 10 & 7 & 3 \end{bmatrix} = \begin{bmatrix} 0\begin{bmatrix} 0 \\ 3 \\ 5 \end{bmatrix} + 10\begin{bmatrix} 10 \\ 7 \\ 3 \end{bmatrix} & 3\begin{bmatrix} 0 \\ 3 \\ 5 \end{bmatrix} + 7\begin{bmatrix} 10 \\ 7 \\ 3 \end{bmatrix} & 5\begin{bmatrix} 0 \\ 3 \\ 5 \end{bmatrix} + 3\begin{bmatrix} 10 \\ 7 \\ 3 \end{bmatrix} \end{bmatrix}$$

Notice that each column in $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ is simply a linear weighted combination of the columns of **A** . This is an important observation because linear combinations of vectors in some subspace always stay in that subspace (indeed, that is the definition of subspace!). This shows that the column space of $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ is a subset of the column space of **A** .

Next we rely on the fact that $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ and **A** have the same rank (see rank-nullity theorem, page Â§), which means the dimensionalities of their column spaces are the same. If the column space of $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ is a subset of the column space of $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ and those two subspaces have the same dimensionality, then they must be equal.

There is another explanation of why $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ and **A** have the same subspace, which relies on the singular value decomposition. More on this in chapter 16!

On the other hand, **A** and $\mathbf{A}\mathbf{A}^{\mathrm{T}}$ generally do not have exactly the same *columns* . So, the columns of those two matrices span the same subspace, but can have different basis sets.

8.3 Determining whether v â^^ *C* (A)

This section is a deeper discussion of the algorithm presented in section 7.11 . As a reminder, one of the important questions in linear algebra is whether a certain vector is in the column space of a matrix.

Letâ€™s start with an example. Consider the following matrix and vectors.

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 4 & 4 \\ 0 & 0 \end{bmatrix}, \qquad \mathbf{v} = \begin{bmatrix} 4 \\ 12 \\ 0 \end{bmatrix}, \ \mathbf{w} = \begin{bmatrix} 4 \\ 1 \\ 1 \end{bmatrix}$$

First, notice that the column space of **A** is a 2D plane embedded in â„ ^{3} . This is the case because the two columns form a linearly independent set (it is not possible to obtain one column by a scaled version of the other).

Now onto the question at hand: Is vector **v** in the column space of matrix **A** ? Formally, this is written as $\mathbf{v} \in C(\mathbf{A})$ or $\mathbf{v} \notin C(\mathbf{A})$. This is not a trivial question: Weâ€™re asking whether a vector in 3D happens to lie on an infinitely thin plane.

For vector **v** , the answer is Yes, $\mathbf{v} \in C(\mathbf{A})$, because it can be expressed as a linear combination of the columns in matrix **A** . A bit of guesswork will lead to you to coefficients of (1,2) for the columns to produce the vector **v** :

$$1 \begin{bmatrix} 2 \\ 4 \\ 0 \end{bmatrix} + 2 \begin{bmatrix} 1 \\ 4 \\ 0 \end{bmatrix} = \begin{bmatrix} 4 \\ 12 \\ 0 \end{bmatrix} \tag{8.5}$$

One of the beautiful features of linear algebra is that it allows expressing a large number of equations in a compact form. We can do that here by putting the column coefficients (1,2) into a vector, and then re-writing equation 8.5 as a matrix-vector multiplication of the form **Ax** = **v** :

$$\begin{bmatrix} 2 & 1 \\ 4 & 4 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 12 \\ 0 \end{bmatrix} \quad \Rightarrow \quad \mathbf{Ax} = \mathbf{v} \tag{8.6}$$

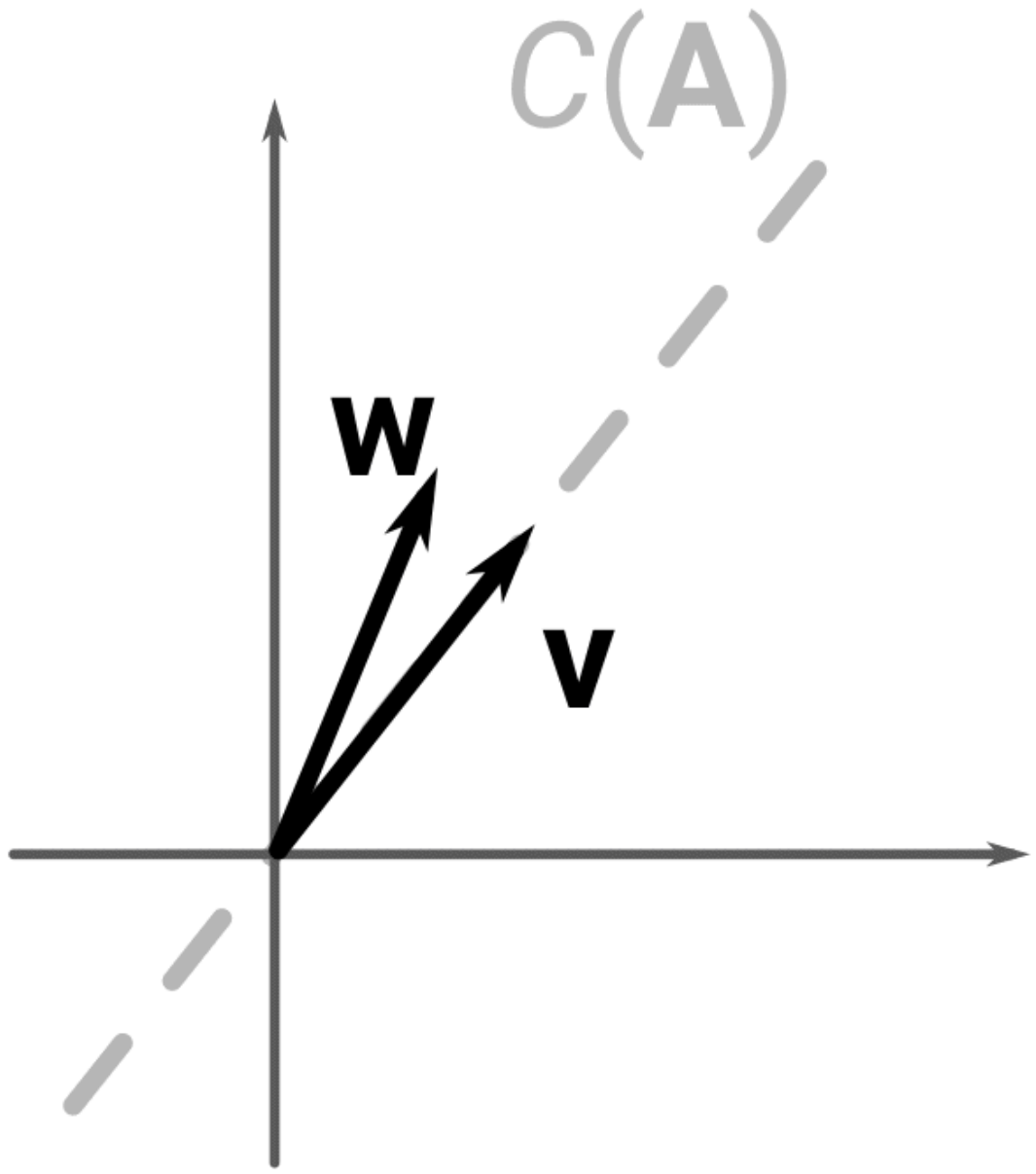
Whenever you see a matrix equation, the first thing you should do is confirm that the matrix sizes allow for a valid equation. Here we have matrix sizes (3 Ã— 2) Ã — (2 Ã— 1) = (3 Ã— 1) . That works.

Now letâ€™s consider vector **w** . It should take only a momentâ€™s inspection to see that **w** cannot be expressed as a [3] linear combination of columns in **A** , because $w_3 = 1$ cannot be created from linear combinations of 0, therefore $\mathbf{w} \notin C(\mathbf{A})$.

[3] We could also write **Axâ‰ w** , â€ **x** â ^{N}

The "augment-rank" algorithm to determine whether v â $C(\mathbf{A})$ Iâ€™ve mentioned this algorithm twice in previous chapters; the novel part here is the application to determining whether a vector is in the column space of a matrix.

Start by creating a matrix **B** = **A** âŠ” **v** (that is, augment the matrix with the vector). Then compute the ranks of these two matrices (**B** and **A**).



FigureÂ 8.1: The dashed gray line represents $C(\mathbf{M}) \subset \mathbb{R}^2$. Then $\mathbf{v} \in C(\mathbf{M})$ and $\mathbf{w} \notin C(\mathbf{M})$

There are two possible outcomes: (1) the ranks are the same, which means that $\mathbf{v} \in C(\mathbf{A})$; or (2) the rank of **B** is one higher than the rank of **A** , which means that $\mathbf{v} \notin C(\mathbf{A})$.

Why is this the case? You can think about it geometrically: If **v** is in the column space of **A** , then the vector is sitting somewhere in the subspace, hence, no new geometric directions are obtained by including vector **v** . In contrast, if **v** is outside the column space, then it points off in some other geometric dimension that is not spanned by the column space; hence, **B** has one extra geometric dimension not contained in **A** , and thus the rank is one higher. This is depicted in Figure 8.1 .

You can also think about it algebraically: If **v** can be obtained by a linear weighted combination of columns in **A** , then augmenting **A** by **v** is necessarily creating a linearly dependent set, thus the rank cannot possible increase.

A corollary of this method is that if **A** is full rank square matrix (that is, rank=*M*), then **v** is necessarily in the column space, because it is not possible to have a subspace with more than *M* dimensions in â„ M .

This algorithm only tells you *whether* a vector is in the column of a matrix. It doesnâ€™t reveal *how* to combine the columns of the matrix to express that vector. For that, you can apply a procedure called Gaussian elimination, which is a major topic of Chapter 10. In the practice problems below, you can use a bit of trial-and-error and educated guessing to find the coefficients.

Practice problems Determine whether the following vectors are in the column space of the accompanying matrices, and, if so, the coefficients on the columns to reproduce the vector.

a) $\begin{bmatrix}1\\2\end{bmatrix}, \begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}$

b) $\begin{bmatrix}1\\2\\0\end{bmatrix}, \begin{bmatrix}0 & 1 & 1\\1 & 0 & 0\\3 & 3 & 0\end{bmatrix}$

c) $\begin{bmatrix}1\\2\\0\end{bmatrix}, \begin{bmatrix}0 & 1\\1 & 0\\3 & 3\end{bmatrix}$

Answers

a) yes. (2,1)
b) yes. (2,-2,3)
c) no.

8.4 Row space of a matrix

The *row space* of a matrix, indicated by $R(\mathbf{A})$, is the same concept as the column space, except it refers to the subspace spanned by the rows. This means it is also the same thing as the column space of the matrix transpose, in other words, $R(\mathbf{A}) = C(\mathbf{A}^T)$. Therefore, there isnâ€™t really anything new in this section compared to section 8.1.

The primary difference is the way that you ask the question whether a given vector is in the row space of the matrix. In particular, this changes how you multiply the matrix and the vector: Instead of matrix-vector multiplication as with the column space ($\mathbf{Ax} = \mathbf{v}$), you have to put the row vector on the left side of the matrix, like this: $\mathbf{x}^T\mathbf{A} = \mathbf{v}^T$. Now the weighting vector $\mathbf{x}$, sometimes also called the coefficients vector, is a row vector on the left, meaning that we are taking weighted combinations of *rows* of $\mathbf{A}$ instead of columns.

An example: The row space of the matrix $\begin{bmatrix} 1 & 2 & 0 & -3 \end{bmatrix}$ is a line in â„ ^{4}. This is quite a different conceptualization from the column space of that matrix, which is all of â„ ^{1}. (Though notice that the dimensionalities are the same.)

You can apply a slightly modified version of the augment-rank algorithm to determine whether a vector is in the row space of a matrix: Place the vector as a new *row* in the matrix and then compute the rank of that row-augmented matrix.

Note that for a square full-rank matrix, $C(\mathbf{A}) = R(\mathbf{A}) =$ â„ ^{N}.

Practice problems Determine whether the following vectors are in the row space of the accompanying matrices, and, if so, the coefficients on the rows to reproduce the vector.

a) $\begin{bmatrix} \pi \\ 0 \end{bmatrix}^T, \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}^T, \begin{bmatrix} 0 & 1 & 1 \\ 2 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$

c)

$$\begin{bmatrix}1\\2\\0\end{bmatrix}^{T}, \begin{bmatrix}0 & 1\\1 & 0\\3 & 3\end{bmatrix}$$

Answers

a) no.
b) yes. (.5,.5,0)
c) wrong sizes!

8.5 Row spaces of A and A ^{T} A

Simply put, **A** and **A** ^{T} **A** have the same row spaces (*R* (**A**) = *R* (**A** ^{T} **A**)). The arguments, explanations, and implications are exactly the same as with the column spaces discussed in section 8.2 .

One new concept I will add here is that the fact that *R* (**A**) = *R* (**A** ^{T} **A**) can be an example of dimensionality reduction: both of those matrices have the same row space, but **A** ^{T} **A** might be a smaller matrix (that is, it might have fewer numbers), and therefore be computationally easier to work with.

8.6 Null space of a matrix

You might be wondering what "null" refers toâ€”you know what columns and rows are, but what is the "null" of a matrix? [4] The null space of a matrix is like the basement of a cheesy horror movie: Not every house has a basement, but if there is one, anyone who goes into it wonâ€™t come out. Likewise, not all matrices have a null space, but if a matrix has a null space and a vector goes into it, that vector can never return.

[4] The null space is also sometimes called the *kernel* of the matrix.

More formally, the null space of a matrix is indicated as *N* (**A**) and is defined as the subspace containing all of the vectors that satisfy the following equation:

$$\mathbf{Ay} = \mathbf{0} \tag{8.7}$$

This means that there is a linear combination of the columns in matrix **A** that produces a column vector of zeros, and the elements of vector **y** specify those weightings.

When can Equation 8.7 be satisfied? Let us count the ways:

1. When **y** = **0** . This is the trivial case, and we ignore trivial cases in linear algebra.
2. When not all elements in **y** are zero, and the specific numbers in **y** and **A** align in just the right way that the matrix-vector product is the zeros vector. This is the non-trivial case, and our primary concern here.

Letâ€™s see an example to make this more concrete. See if you can come up with a vector (that is, find *x* and *y*) that satisfies the equation.

$$\begin{bmatrix} 1 & 2 \\ 4 & 8 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{8.8}$$

Did you come up with the vector $[-2\ 1]^T$? That satisfies the equation and therefore is in the null space of that matrix (thus: **y** â^^ *N* (**A**)).

But thatâ€™s not the only vector that satisfies that equationâ€”perhaps your solution was $[2\ -1]^T$ or $[-1\ .5]^T$ or $[-2000\ 1000]^T$. Iâ€™m sure you see where this is going: There is an infinite number of vectors in the null space of this matrix, and all of those vectors are scaled versions of each other. In other words:

$$N\left(\begin{bmatrix} 1 & 2 \\ 4 & 8 \end{bmatrix}\right) = \left\{\sigma \begin{bmatrix} -2 \\ 1 \end{bmatrix}, \quad \sigma \in \mathbb{R}\right\} \tag{8.9}$$

As mentioned earlier, the zeros vector also satisfies the equation, but thatâ€™s a trivial solution and we ignore it.

Here is the complete definition of the null space of a matrix:

Eqn. box title=**Null space of a matrix**

$$N(\mathbf{A}) = \{\lambda \mathbf{y} \mid \mathbf{A}\mathbf{y} = \mathbf{0}, \quad \lambda \in \mathbb{R}\} \text{ â^’ } \{\mathbf{y} = \mathbf{0}\} \tag{8.10}$$

(Aside on math notation: The | indicates "such that" and the minus sign excludes **0** from the set.)

Now letâ€™s try another example:

$$\begin{bmatrix} 1 & 2 \\ 4 & 7 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{8.11}$$

Can you find a (non-trivial) vector in the null space? The answer is No, you cannot. There is no way to combine the columns to produce the zeros vector. It is colloquially said that this matrix "has no null space," although a more appropriate phrasing would be that the null space of this matrix is the empty set (again, the trivial zeros vector is ignored):

$$N\left(\begin{bmatrix}1 & 2\\ 4 & 7\end{bmatrix}\right) = \{\} \tag{8.12}$$

Did you notice anything about the two matrices in equations 8.8 and 8.11 ? Perhaps you noticed that the matrix in 8.8 was singular (rank=1) while the matrix in 8.11 was full rank (rank=2). This is no coincidenceâ€”full-rank square matrices and full-column-rank matrices necessarily have an empty null space, whereas reduced-rank and reduced-column-rank matrices necessarily have a non-empty null space. Youâ€™ll learn more about why this is in a few pages.

The previous examples used square matrices; below are two examples of rectangular matrices so you can see that null space is not just for squares. The hard work of finding the vector **y** is already done, so pay attention to the sizes: the null space is all about linear weighted combinations of the *N* columns, which are in â„ ^{M} , and the vector that contains the weightings for the columns is **y** â^^ â„ ^{N} , corresponding to the dimensionality of the row space.

$$\begin{bmatrix}1 & 2\\ 1 & 2\\ 1 & 2\\ 1 & 2\end{bmatrix}\begin{bmatrix}-2\\ 1\end{bmatrix} = \begin{bmatrix}0\\ 0\\ 0\\ 0\end{bmatrix}, \qquad \begin{bmatrix}1 & 1 & 1 & 1 & -4\\ 2 & 2 & 2 & 2 & -8\end{bmatrix}\begin{bmatrix}1\\ 1\\ 1\\ 1\\ 1\end{bmatrix} = \begin{bmatrix}0\\ 0\end{bmatrix}$$

There is a deterministic relationship between the rank of a matrix, its size, and the dimensionality of the four matrix spaces. We will return to this in section 8.10 .

Practice problems Find a vector in the null space of each matrix, if there is one.

a) $\begin{bmatrix}1 & 0\\ 4 & 0\\ 2 & 0\\ 1 & 0\end{bmatrix}$

b) $\begin{bmatrix}1 & 2 & 3\\ 3 & 1 & 4\\ 4 & 4 & 8\end{bmatrix}$

c)

$$\begin{bmatrix} 1 & 2 & 3 \\ 3 & 1 & 5 \\ 4 & 4 & 8 \end{bmatrix}$$

d) $\begin{bmatrix} -5 & 5 & 5 \\ -3 & 7 & 15 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$

b) $\begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$

c) Empty null space

d) $\begin{bmatrix} 2 \\ 3 \\ -1 \end{bmatrix}$

Practice problems For each combination of matrix and vectors, determine whether neither, one, or both vectors are in the null space of the matrix.

a) $\begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 4 & 0 \end{bmatrix}, \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \begin{bmatrix} -2 \\ -1 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 2 & 3 \\ 3 & 1 & 5 \\ 4 & 4 & 8 \\ 0 & 4 & 2 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ -2 \\ -1 \\ 0 \end{bmatrix}$

Answers

a) Neither

b) Both

c) Wrong sizes!

Left-null space There is a complementary space to the null space, called the *left-null space* , which is the same concept but with a row vector on the left of the matrix instead of a column vector on the right of the matrix. The resulting zeros vector is also a row vector. It looks like this:

$$\mathbf{y}^{\mathrm{T}}\mathbf{A} = \mathbf{0}^{\mathrm{T}} \tag{8.13}$$

The left-null space can be thought of as the "regular" null space of the matrix transpose. [5] This becomes apparent when transposing both sides of Equation 8.13 .

[5] The "regular" null space is formally the "right null space," but this is implied when referring to the "null space."

$$\begin{aligned} (\mathbf{y}^{\mathrm{T}}\mathbf{A})^{\mathrm{T}} &= \mathbf{0}^{\mathrm{TT}} \\ \mathbf{A}^{\mathrm{T}}\mathbf{y} &= \mathbf{0} \end{aligned} \tag{8.14}$$

Considering the left-null space as the (right) null space of the matrix transpose is analogous to how $R(\mathbf{A}) = C(\mathbf{A}^{\mathrm{T}})$. This also means that the two null spaces are equal when the matrix is symmetric:

[6] This result is not terribly surprising, but it is relevant for finding null space basis vectors via the SVD.

$$N(\mathbf{A}) = N(\mathbf{A}^{\mathrm{T}}), \quad \text{if } \mathbf{A} = \mathbf{A}^{\mathrm{T}} \tag{8.15}$$

Letâ€™s think about the dimensionality of the left-null space. It should be intuitive that it will mirror the null space. Iâ€™ll start by showing the left-null spaces of the two rectangular matrices used above, and then discuss the rule. Notice that the null-space vector is now on the left of the matrix.

$$\begin{bmatrix} -1 & -1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 2 \\ 1 & 2 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \end{bmatrix} \tag{8.16}$$

$$\begin{bmatrix} -2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 & -4 \\ 2 & 2 & 2 & 2 & -8 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix} \tag{8.17}$$

Hereâ€™s the rule: For an *M* Ã— *N* matrix, the row space is in ambient â„ ^{N} and the left-null space is in ambient â„ ^{M} . Again, this is sensible because the left-null space provides a weighting of all the rows to produce the zeros row vector. There are *M* rows and so a null space vector must have *M* elements.

Look closely at Equation 8.16 : Is that vector the *only* vector in the left-null space? Iâ€™m not referring to scaled version of that vector; I mean that there are more vectors in the left-null space that are separate from the one I printed. How many more (non-trivial) vectors can you identify?

Practice problems Find a vector in the left-null space of each matrix, if there is one. Notice that these matrices appeared in the practice problems on page Â§ ; are the answers the same?

a) $\begin{bmatrix} 1 & 0 \\ 4 & 0 \\ 2 & 0 \\ 1 & 0 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 & 3 \\ 3 & 1 & 4 \\ 4 & 4 & 8 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 2 & 3 \\ 3 & 1 & 5 \\ 4 & 4 & 8 \end{bmatrix}$

d) $\begin{bmatrix} -5 & 5 & 5 \\ -3 & 7 & 15 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 1 & 0 & -1 & 1 \end{bmatrix}$

b) $\begin{bmatrix} -8 & -4 & 5 \end{bmatrix}$

c) Empty left null space

d) Empty left null space

Code Python and MATLAB will return basis vectors for the null space of a matrix (if it has one). How do they solve this seemingly magical feat? They use the singular value decomposition! However, youâ€™ll have to wait several more chapters to understand why the SVD reveals bases for the null space.

Code blockÂ 8.1: Python

```
importÂ numpyÂ asÂ npÂ
fromÂ scipy.linalgÂ importÂ null_spaceÂ
AÂ =Â np.random.randn(3,4)Â
null_space(A)
```

Code blockÂ 8.2: MATLAB

```
AÂ =Â randn(3,4);Â
null(A)
```

8.7 Geometric interpretation of the null space

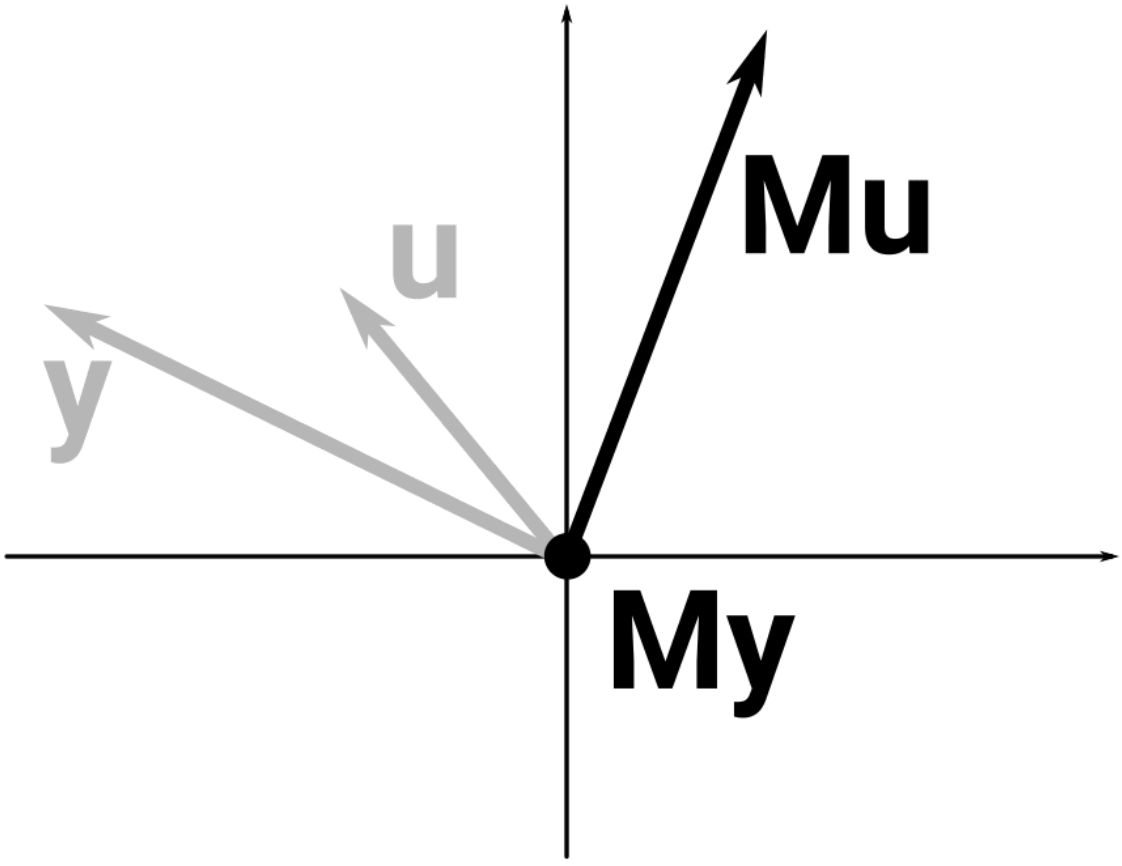
Now that you know the horror-movie analogy (the evil basement from which no unsuspecting visitors return) and the algebraic definition (**Ay** = **0**), this is a good time to learn the geometric perspective of the null space.

Recall that a matrix times a vector produces another vector. Weâ€™ll stick to â„ [2] here so that everything can be easily visualized. Consider the following matrix and vectors.

$$\mathbf{M} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}, \qquad \mathbf{u} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

Matrix **M** is singular and has a non-empty, 1-dimensional, null space. **u** is *not* in the matrixâ€™s null space, and **y** *is* in the null space (please do the matrix-vector multiplications in your head to confirm).

You can think of matrix **M** as a transformation matrix that warps vector **y** into vector **My** . Figure 8.2 shows what that looks like.



FigureÂ 8.2: Depiction of vectors **y** â^^ N (**M**) and **u** $\notin$ N (**M**) . The vectors on their own are not remarkable; however, when multiplied by the matrix, **y** becomes the zeros vector (dot at the origin), whereas **u** is some other (non-zeros) vector.

Figure 8.2 shows that when you multiply a vector in the null space by that matrix, the resulting "vector" is just a point at the origin. And thatâ€™s basically the end of the line for this matrix-vector product **My** â€”it cannot do anything else but sit in the origin. Just like the basement of a horror movie: Once you go in, you never come out. There is no escape. There is no possible other matrix **A** such that **AMy** *â‰ * **0** . In other words, the matrix entered its null space and became a singularity at the origin; no other matrix can bring it back from the abyss.

On the other hand, because matrix multiplication is not commutative, **AMy** *â‰ * **MAy** . Try this yourself: Come up with a matrix **A** and prove to yourself that **MAy** *â‰ * **0** . This will work for nearly any matrix as long as **A** is not the identity or the zeros matrices. Just pick four random integers and it will almost certainly work.

And this brings me to the final point I want to make in this section: There is nothing special about vector **y** , and there is nothing special about matrix **M** . Instead, what is special is their *combination* , that **y** â^^ N (**M**) . **MAy** *â‰ * **0** because **y** is not in the null space of matrix **A** , and vector **Ay** is not in the null space of matrix **M** (it could be if **A** is the identity matrix, the zeros matrix, or has **y** as an eigenvector, but that is galactically unlikely when populating **A** with random integers).

Practice problems For the following matrices, find a basis for the column space and a basis for the left-null space. Then draw those two basis vectors on the same Cartesian plot (one plot per matrix). Do you notice anything about the plots?

a) $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$

b) $\begin{bmatrix} -3 & 1 \\ 6 & -2 \end{bmatrix}$

Answers In both cases, the column space and the left-null space are orthogonal to each other. You can easily confirm that in these examples by computing the dot products between the basis vectors. Thatâ€™s not just some quirky effect of these specific matrices; thatâ€™s a general principle, and you will soon learn the reason why. (You can repeat this exercise for the row space and the null space and arrive at

the same conclusion.)

8.8 Orthogonal subspaces, orthogonal complements

The four subspaces of a matrixâ€”the column space, row space, null space, and left-null spaceâ€”come in two pairs of *orthogonal complements* . Before explaining what that means and why itâ€™s important, I need to take a brief aside and explain the concepts of orthogonal subspaces and orthogonal complements.

You already know what it means for two lines to be orthogonal to each other (dot product of zero; meet at a right angle); what does it mean for two *subspaces* to be orthogonal? Let me start by mentioning something thatâ€™s pretty obvious when you think about it: If a pair of vectors is orthogonal, then any scalar-vector multiplication will also be orthogonal.

$$\mathbf{v} \perp \mathbf{w} \quad \Rightarrow \quad \sigma\mathbf{v} \perp \lambda\mathbf{w}, \quad \sigma, \lambda \in \mathbb{R} \tag{8.18}$$

The reason why Equation 8.18 is obvious is that if $\mathbf{v}^T \mathbf{w} = 0$ then any scalar times 0 is still 0.

Now letâ€™s take this one step further: Imagine that two vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ are each orthogonal to $\mathbf{w}$. They donâ€™t have to be orthogonal to each other. Hereâ€™s an example:

$$\mathbf{v_1} = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}, \quad \mathbf{v_2} = \begin{bmatrix} 2 \\ 2 \\ 6 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix},$$

Take a moment to confirm that $\mathbf{v}_1$ and $\mathbf{v}_2$ are each individually orthogonal to $\mathbf{w}$, and that they are *not* orthogonal to each other.

Now we can combine this concept with Equation 8.18 to write a new equation:

$$(\alpha_1\mathbf{v_1} + \alpha_2\mathbf{v_2}) \perp \beta\mathbf{w} \tag{8.19}$$

This equation says that *any linear combination* of $\mathbf{v}_1$ and $\mathbf{v}_2$ is orthogonal to any scaled version of $\mathbf{w}$.

Each side of Equation 8.19 is consistent with the definition of a vector subspace (Equation 4.1 , page Â§). Thus, we have just extended the notion of orthogonality from vectors to subspaces (hence: V âŠ¥ W). The interpretation is that any linear combination of vectors in subspace V is orthogonal to any linear combination of vectors in subspace W .

In this example, *V* is a 2D subspace and *W* is a 1D subspace, and they are both embedded in ambient â„ [3] . Two subspaces can be orthogonal only if they are in the same ambient dimensionality, because orthogonality is defined by dot products, and dot products are defined only for vectors with the same dimensionality.

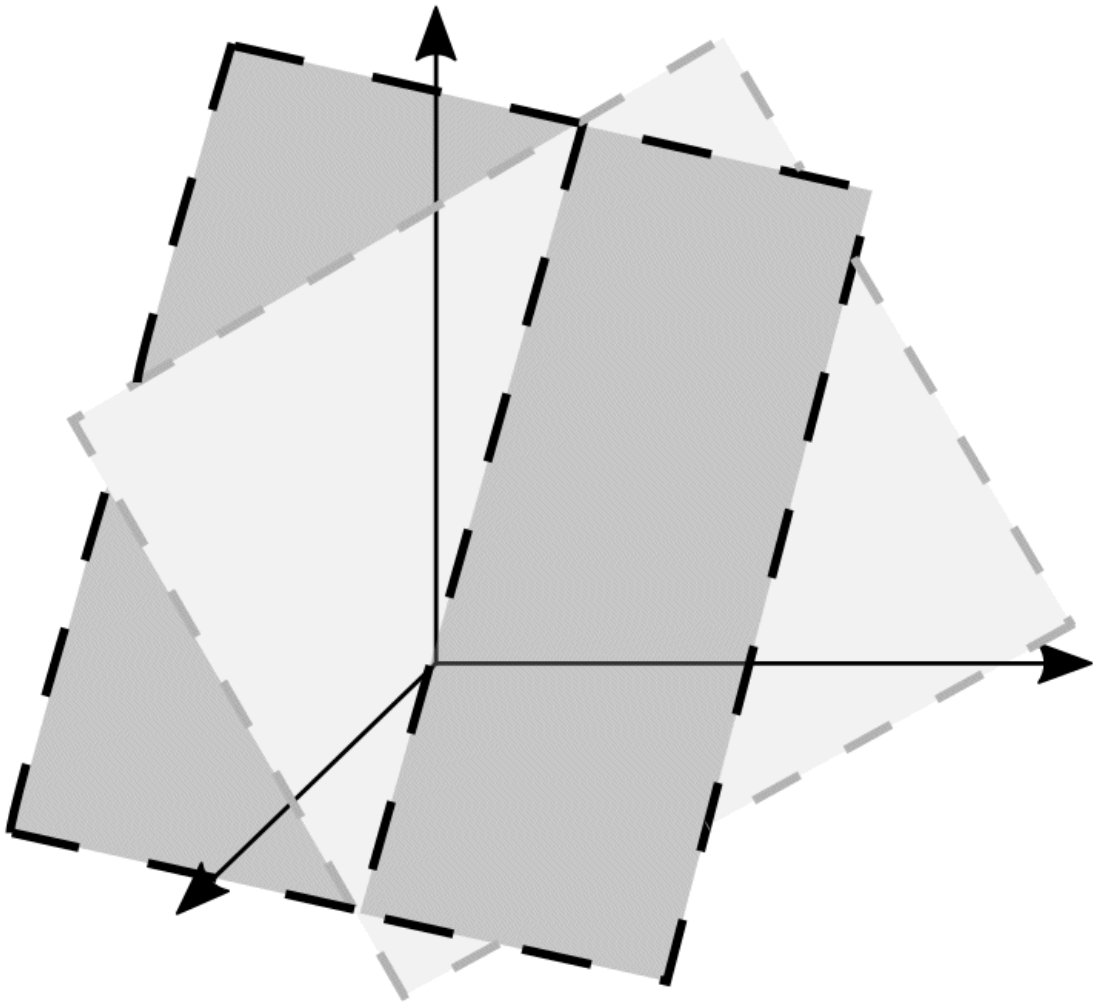
Here is the formal definition of orthogonal subspaces:

Eqn. box title=**Orthogonal subspaces**

Subspaces *S* and *M* are orthogonal subspaces if

â^€ **v** â^^ *S* and â^€ **w** â^^ *M*, **v** âŠ¥ **w** (8.20)

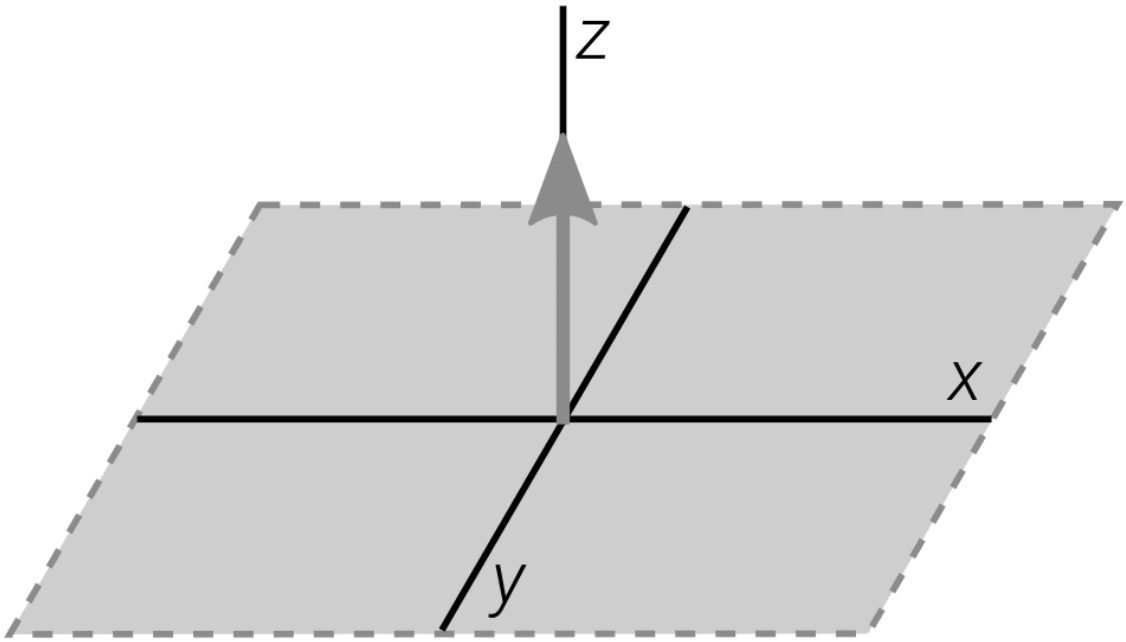
Read aloud, this definition is "For any vector **v** in subspace *S* , and for any vector **w** in subspace *W* , vector **v** is orthogonal to vector **w** .



FigureÂ 8.3: Two 2D subspaces in â„ ^{3} cannot be orthogonal subspaces.

Can you think of a geometric example of two orthogonal subspaces? You might initially think about two planes in â„ ^{3} intersecting in a perpendicular line, like Figure 8.3 .

But alas, no, these planes are not orthogonal subspaces. The line of intersection between these two planes is a vector that is contained in both subspaces. And a vector that appears is both subspaces is clearly not orthogonal to itself. Of course, there can be many vectors in one plane that are orthogonal to many vectors in the other plane, but for the two subspaces to be orthogonal, *every possible vector* needs to be orthogonal.



FigureÂ 8.4: A 1D subspace can be an orthogonal complement to a 2D subspace in â„ [3] .

In fact, in â„ [3] it is impossible for two *planes* to be orthogonal subspaces: They are 2D objects, and two 2D objects cannot possibly be orthogonal in a 3D space. A plane and a line, however, can be orthogonal complements if the line is orthogonal to the plane. The simplest example of this is the plane defined by the XY axis, and a line on the Z axis (Figure 8.4).

Orthogonal complements This leads us to *orthogonal complements* .[7] The idea of orthogonal complements is that any ambient space â„ [N] can be decomposed into two subspaces *W* and *V* such that *W* âˆª *V* spans all of â„ [N] and *W* âŠ¥ *V* . In other words, the entire ambient space is carved into subspaces that are mutually orthogonal and that meet only at the origin (all subspaces meet at the origin).

[7] Donâ€™t confuse compl *e* ment with compl *i* ment! (Although both are things you should do for your significant other.)

"Mutually orthogonal" means that a vector cannot be in both subspaces. Thus, any vector in â„ [N] that is not in *W* must be in *V* , and if itâ€™s in *V* then it must be orthogonal to any vector in *W* .

8.9 Orthogonalities of the matrix spaces

Now that you know what orthogonal subspaces and orthogonal complements are, letâ€™s get back to the task at hand, which is discussing how the four subspaces of

a matrix are paired into two orthogonal complements.

Orthogonality of the column space and the left-null space A vector is orthogonal to the column space of a matrix if the following equation is satisfied:

$$\mathbf{y} \perp C(\mathbf{A}) \tag{8.21}$$

What does it mean for vector **y** to be orthogonal to the column space of **A** ? It means that **y** is orthogonal to each column individually, and therefore that **y** is orthogonal to all possible linear combinations of the columns.

We can re-write this statement as the following matrix-vector equation:

$$^{8}\mathbf{A}^{\mathrm{T}}\mathbf{y} = \mathbf{0} \tag{8.22}$$

Remarkably, weâ€™ve just re-derived Equation 8.13 (see also Equation 8.14), which was the definition of the left-null space. Thatâ€™s pretty neat, because we started from the question of a vector that is orthogonal to the *column space* , and we ended up re-discovering the *left-null space* .

[8] Note: **0** â^ â„ [N] .

What is the justification for transposing the matrix? Because the *columns* of **A** are the *rows* of $\mathbf{A}^{\mathrm{T}}$ (Figure 8.5), the multiplication $\mathbf{A}^{\mathrm{T}}\mathbf{y}$ involves the dot product between vector **y** and each column of **A** . The question at hand here is whether we can find a vector **y** that has a dot product of zero with each column in **A** .

$$\begin{bmatrix} a & b & c \\ a & b & c \\ a & b & c \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} a & a & a \\ b & b & b \\ c & c & c \end{bmatrix}$$

FigureÂ 8.5: The columns of **A** are the rows of $\mathbf{A}^{\mathrm{T}}$.

The orthogonality of the column space and left-null space is a big deal, because weâ€™re talking about entire subspaces, not just individual vectors. A vector is a mere finite object, but subspaces are infinite expanses. Thus, the column space and the left-null space are orthogonal complements, and so together they must fill up the entire ambient space of â„ [M] .

$$C(\mathbf{A}) \cup N(\mathbf{A}^{\mathrm{T}}) \Leftrightarrow \mathbb{R}^{\mathrm{M}} \tag{8.23}$$

Let me explain that again so itâ€™s clear: For any given *M* Ã— *N* matrix, *every* vector in â„ [M] is either in the column space or in the left-null space. No vector can be in both (because they are orthogonal subspaces) except for the trivial zeros

vector. Therefore, the column space and the left-null space together span all of â„ ^{M} .

Analogously, if someone asks you if you want ice cream with sprinkles, [9] ice cream without sprinkles, or no ice cream, then those mutually exclusive options literally account for every imaginable thing in the universe.

[9] OK, perhaps not a perfect analogy, but the point is that I like sprinkles without ice cream.

One implication of Equation 8.23 is that if the column space of the matrix spans all of â„ ^{M} , then the left-null space must be empty, because only the zeros vector can be orthogonal to an entire ambient space.

A visual example This is all quite theoretical; a visualizable example should help internalize the concept. Below are two matrices; Please take a moment to find bases for the column and left-null spaces of each matrix, then graph those vectors and compute the dot product between them. Then look at the next page to see my solution. No cheating!

$$\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 1 & 1 \\ 2 & 3 \end{bmatrix}$$

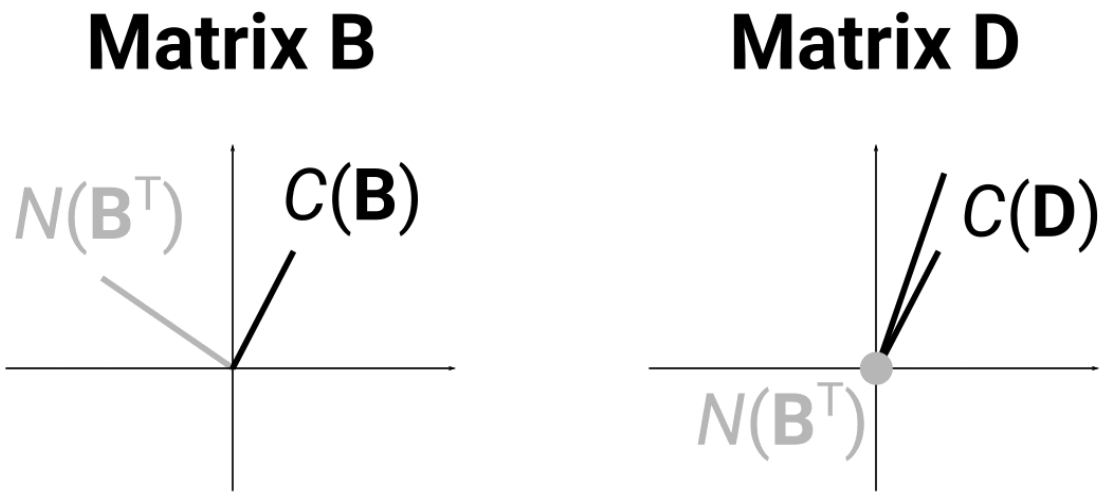
I chose the following as bases:

$$2\ C(\mathbf{B}) = \left\{ \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right\}, N(\mathbf{B}^{\mathrm{T}}) = \left\{ \begin{bmatrix} -2 \\ 1 \end{bmatrix} \right\} C(\mathbf{D}) = \left\{ \begin{bmatrix} 1 & 1 \\ 2 & 3 \end{bmatrix} \right\}, N(\mathbf{D}^{\mathrm{T}}) = \{ \square \}$$

The easiest way to choose a basis for the column space is to take the first *r* columns, where *r* is the rank of the matrix (make sure that those columns form a linearly independent set). Of course, this isnâ€™t necessarily *the best* basis set; itâ€™s just *the easiest* for small matrices.

Figure 8.6 shows these vectors, with black lines for the column space basis vectors and gray lines for the left-null space basis vectors.

Matrix **D** has an empty null space, which Iâ€™ve depicted as a dot at the origin of the graph. Itâ€™s visually apparent that the column spaces are orthogonal to their respective left-null spaces.



FigureÂ 8.6: Basis vectors for the column and null spaces of matrices **B** and **D** .

The orthogonality of C (**A**) and N ($\mathbf{A}^{\mathrm{T}}$) will lead us to an important discovery about matrix subspace dimensionalities. Thatâ€™s the main topic of the next section. But first, I want to discuss the other pair of complementary matrix spaces, which are the row space and the null space.

Orthogonality of the row space and the null space There isnâ€™t a lot to say here that isnâ€™t written above. Just swap "column" with "row" and "left-null space" with "null space." I will briefly walk through the reasoning as above but using slightly different notation, for variety.

The idea is to find a vector that is orthogonal to all rows in **A** , in other words, **y** âŠ¥ R (**A**) .

We can express this by writing that the dot product between each row of the matrix (indicated as $\mathbf{a}_m$ below) and the vector **y** is 0.

$$\begin{aligned} \mathbf{a}_1\mathbf{y} &= 0 \\ \mathbf{a}_2\mathbf{y} &= 0 \\ &\vdots \\ \mathbf{a}_m\mathbf{y} &= 0 \end{aligned}$$

And then we simply collect all of these individual equations into one compact matrix equation, which is, of course, the definition of the null space.

$$\mathbf{Ay} = \mathbf{0}$$

As with the column and left-null spaces, the row space and null space are orthogonal complements that together span all of â„ ^{N} :

$$C(\mathbf{A}^T) \cup N(\mathbf{A}) \Leftrightarrow \mathbb{R}^N \tag{8.24}$$

Any vector in â„ ^{N} is either in the row space or in the null space. The only vector that can be in both spaces is the *N* -element zeros vector.

8.10 Dimensionalities of matrix spaces

The dimensionalities of the column space, the row space, and the two null spaces are all interconnected.

First, I want to reiterate that "dimension" is not the same thing as "rank." The rank is a property of a matrix, and itâ€™s the same regardless of whether you are thinking about rows, columns, or null spaces. The ambient dimensionality differs between rows and columns for non-square matrices.

On the other hand, the dimensionality of the *subspaces spanned* by the column space and the row space is the same, and those equal the rank of the matrix. An example of this dissociation is the following 3 Ã— 3 , rank-2 matrix. To its right are the rank and dimensionalities (*dim* () indicates the dimensionality) of two of its subspaces.

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 4 & 5 \\ 2 & 0 & 6 \end{bmatrix}, \qquad \begin{array}{c} \mathbf{A} \in \mathbb{R}^{3\times 3} \\ rank(\mathbf{A}) = 2 \\ dim(N(\mathbf{A})) = 1 \\ dim(C(\mathbf{A})) = 2 \end{array}$$

The null space contains one basis vector, which means it has dimensionality of one, while the column and row spaces each has dimensionalities of 2 (notice that row 3 is a multiple of row 1). The rank of the matrix is also 2.

You can see in this example that the dimensionality of the column space plus the dimensionality of the left-null space adds up to the ambient dimensionality â„ ^{3} .

Two more examples and then Iâ€™ll present the rules: If the column space is 2D and embedded in â„ ^{2} , then the column space already covers the entire ambient space, which means thereâ€™s nothing for the left-null space to capture; the left-null space *must* therefore be the empty set. Hereâ€™s a matrix to illustrate this point; note that the left-null is empty, because there is no way to combine the rows of the matrix (or, the columns of the matrix transpose) to get a vector of zeros.

$$\begin{bmatrix} 1 & 1 \\ 2 & 3 \end{bmatrix}$$

Final example: The 2 Ã— 2 zeros matrix has columns in ambient â„ ^{2} , but the column space is empty; it contains nothing but a point at the origin. It is 0-dimensional. Therefore, its orthogonal complement must fill up the entirety of â„ ^{2} . This tells us that the left-null space must be 2-dimensional. What is a basis set for that left-null space? Literally any independent set of vectors can be a basis set. A common choice in this situation is the identity matrix, but thatâ€™s because itâ€™s convenient, not because itâ€™s the only basis set.

The story is the same for the row space and the null space, so I will just re-state it briefly: The row space lives in ambient â„ ^{N} but can span a lower-dimensional subspace depending on the elements in the matrix. The orthogonal complementâ€”the null spaceâ€”fills up whatever directions in â„ ^{N} are not already spanned by the row space. If the matrix is full-row rank, then the row space already spans all of â„ ^{N} and therefore the null space must be empty.

The formulas Here you go:

$$\underbrace{C(\mathbf{A}) \cup N(\mathbf{A}^{\mathrm{T}})}_{\text{â„}^{M}} \quad \underbrace{R(\mathbf{A}) \cup N(\mathbf{A})}_{\text{â„}^{N}}$$

And with numbered equations in a grey box:

Eqn. box title=**Subspace dimensionalities**

$$dim\,(\,C\,(\,\mathbf{A}\,))\,+\,dim\,(\,N\,(\,\mathbf{A}^{\mathrm{T}}\,))\,=\,M \qquad (8.25)$$

$$dim\,(\,R\,(\,\mathbf{A}\,))\,+\,dim\,(\,N\,(\,\mathbf{A}\,))\,=\,N \qquad (8.26)$$

One more relevant formula: The rank of the matrix is the dimensionality of the column space, which is the same as the dimensionality of the row space:

$$rank(\mathbf{A}) = dim(C(\mathbf{A})) = dim(R(\mathbf{A})) \qquad (8.27)$$

Practice problems Determine the dimensionalities of the four subspaces in the following matrices.

a) $\begin{bmatrix} 2 & 3 & 4 \\ 0 & 0 & 0 \\ 1 & 2 & 3 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 1 & 2 & 2 & 3 & 3 \\ 4 & 4 & 5 & 5 & 6 & 6 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 2 \\ 1 & 2 \\ 2 & 4 \end{bmatrix}$

Answers The four numbers below are the dimensionalities of the column space, left-null space, row space, and null space. You can confirm that the sum of the first two numbers corresponds to the number of rows, and the sum of the second two numbers corresponds to the number of columns.

a) 2,1,2,1
b) 2,0,2,4
c) 1,2,1,1

8.11 More on Ax = b and Ay = 0

These two equations are simple yet profound. It is no understatement to write that *most people learn linear algebra because they want to know how to solve these equations* . You might not realize that this is what you want to solve, but most of applied linear algebra boils down to solving one of these two equations.

Letâ€™s start with **Ax** = **b** . In this category of problems, you know *a priori* the matrix **A** and the vector **b** , and the goal of the analysis is to find the vector **x** . Basically all linear models in statistics and machine learning can be expressed using this form.

The letters might look different, though. For example, in statistics, the common form is **X** $\hat{I}^2$ = **y** , where **X** is called the "design matrix," $\hat{I}^2$ is called the "regression coefficients," and **y** is called the "observed data." You will learn more about those terms and what they mean in Chapter 14. But you can see that the general form is the same.

There are two questions that you ask with an **Ax** = **b** problem:

1. **Does it have a solution?** You already know the answer to that question: The equation has an exact solution when **b** is in the column space of **A** . In that case, the coefficients in vector **x** tell you the weightings of the columns in **A** in order to produce vector **b** . If vector **b** is not in the column space of matrix **A** , then that leads to the second question:
2. **What is the closest approximation to an exact solution?** This changes the equation to **Ax** = **b** , where **x** and **b** are selected such that (1) **b** is in the column space of **A** and **x** are the coefficients, and (2) **b** is as close as possible to the original **b** . This is obtained through the "least squares solution," which is the backbone of statistics, model fitting, machine learning, and many other areas of applied mathematics.

Now Iâ€™d like to tell you more about **Ay** = **0** . It may seem strange that someone would be so interested in finding the null space of a matrix, considering that the null space is the "black hole of no return." In practice, people are not interested in this matrix **A** *per se* ; instead, they are interested in a shifted version of this matrix, expressed as (**A** âˆ’ *Î»* **I**) **y** = **0** .

The solution to this equation (vector **y**) is called an eigenvector of the matrix, and *Î»* is its associated eigenvalue. Eigenvectors reveal directions in the matrix that have special properties, such as robustness to geometric transformations or maximizing covariance in a dataset. In different contexts, the solution to **Ay** = **0** is called Principal Components Analysis, generalized eigendecomposition, singular value decomposition, Fisher linear discriminant analysis, Rayleigh quotient, and many other names. These analyses play central roles in machine-learning applications and multivariate signal processing.

I hope this helps put things in perspective. Itâ€™s not the case that *every* problem in linear algebra boils down to one of these two equations. But as you proceed in your adventures through the jungle of linear algebra, please keep these two equations in mind; the terminology may differ across fields, but the core concepts are the same.

8.12 Exercises

1.Â

For each matrix-vector pair, determine whether the vector is in the column space of the matrix, and if so, the coefficients that map the vector into that column space.

a)

$\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}, \begin{bmatrix}2\\3\end{bmatrix}$

b) $\begin{bmatrix}1 & 0\\2 & 0\end{bmatrix}, \begin{bmatrix}3\\1\end{bmatrix}$

c) $\begin{bmatrix}1 & 0\\2 & 0\end{bmatrix}, \begin{bmatrix}3\\0\end{bmatrix}$

d) $\begin{bmatrix}1 & 1\\3 & 1\\0 & 1\end{bmatrix}, \begin{bmatrix}2\\2\\0\end{bmatrix}$

e) $\begin{bmatrix}1 & 1\\3 & 1\\0 & 1\end{bmatrix}, \begin{bmatrix}2\\2\end{bmatrix}$

f) $\begin{bmatrix}1 & 2\\2 & 1\end{bmatrix}, \begin{bmatrix}-3\\3\end{bmatrix}$

g) $\begin{bmatrix}0 & 0 & 1\\0 & 1 & 0\\1 & 0 & 0\end{bmatrix}, \begin{bmatrix}3\\6\\4\end{bmatrix}$

h) $\begin{bmatrix}-1 & 5 & 2\\-7 & 9 & 8\\-1 & 4 & \pi\end{bmatrix}, \begin{bmatrix}0\\0\\0\end{bmatrix}$

2.Â

Same as the previous exercise but for the row space.

a) $\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}, \begin{bmatrix}2\\3\end{bmatrix}^{\mathrm{T}}$

b) $\begin{bmatrix}1 & 0\\2 & 0\end{bmatrix}, \begin{bmatrix}3\\0\end{bmatrix}^{\mathrm{T}}$

c) $\begin{bmatrix}1 & 6\\2 & 12\end{bmatrix}, \begin{bmatrix}2\\9\end{bmatrix}^{\mathrm{T}}$

d) $\begin{bmatrix}1 & 0 & 1\\2 & 2 & 0\end{bmatrix}, \begin{bmatrix}1\\-2\\3\end{bmatrix}^{\mathrm{T}}$

3.Â

For each matrix-set pair, determine whether the vector set can form a basis for the column space of the matrix.

a) $\begin{bmatrix}1 & 0\\2 & 0\end{bmatrix}, \left\{\begin{bmatrix}3\\1\end{bmatrix}, \begin{bmatrix}1\\0\end{bmatrix}\right\}$

b) $\begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}, \left\{ \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \begin{bmatrix} 4 \\ 4 \end{bmatrix} \right\}$

c) $\begin{bmatrix} 3 & 6 \\ 0 & 0 \\ 1 & 2 \end{bmatrix}, \left\{ \begin{bmatrix} 1 \\ 0 \\ 1/3 \end{bmatrix} \right\}$

d) $\begin{bmatrix} 0 & 0 & 3 \\ 2 & 0 & 0 \end{bmatrix}, \left\{ \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\}$

e) $\begin{bmatrix} e^{\pi} & 3^{e} \\ \sqrt[3]{3.7} & e^{e^2} \end{bmatrix}, \left\{ \begin{bmatrix} -3 \\ -2 \end{bmatrix}, \begin{bmatrix} -1 \\ 2 \end{bmatrix} \right\}$

4.Â

Determine whether the following matrices have a null space. If so, provide basis vector(s) for that null space.

a) $\begin{bmatrix} 1 & 0 \\ 2 & 0 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 0 \\ 2 & 2 \end{bmatrix}$

c) $\begin{bmatrix} 4 & 3 \\ 1 & 1 \\ 0 & 5 \end{bmatrix}$

d) $\begin{bmatrix} 3 & 1 & 5 \\ 4 & 1 & 0 \end{bmatrix}$

5.Â

Fill in the blanks (*dim* =dimensionality) for matrix **A** â^^ â„ $^{2 \times 3}$

a) *dim* (*C* (**A**)) = 0 , *dim* (*N* ($\mathbf{A}^{T}$)) = ___

b) *dim* (*C* (**A**)) = 1 , *dim* (*N* ($\mathbf{A}^{T}$)) = ___

c) *dim* (*C* (**A**)) = 2 , *dim* (*N* ($\mathbf{A}^{T}$)) = ___

d) *dim* (*C* (**A**)) = 3 , *dim* (*N* ($\mathbf{A}^{T}$)) = ___

e) *dim* (*N* (**A**)) = 0 , *dim* (*R* (**A**)) = ___

f) *dim* (*N* (**A**)) = 1 , *dim* (*R* (**A**)) = ___

g) *dim* (*N* (**A**)) = 2 , *dim* (*R* (**A**)) = ___

h) *dim* (*N* (**A**)) = 3 , *dim* (*R* (**A**)) = ___

8.13 Answers

1.Â

a)

$\begin{bmatrix}2\\3\end{bmatrix}$

b) not in column space

c) not in column space

d) not in column space

e) sizes donâ€™t match

f) $\begin{bmatrix}3\\-3\end{bmatrix}$

g) $\begin{bmatrix}4\\6\\3\end{bmatrix}$

h) $\begin{bmatrix}0\\0\\0\end{bmatrix}$

2.Â

a) $\begin{bmatrix}2\\3\end{bmatrix}^{T}$

b) $\begin{bmatrix}1\\1\end{bmatrix}^{T}$

c) Not in the row space

d) $\begin{bmatrix}3\\-1\end{bmatrix}^{T}$

3.Â

a) No. Any column space basis must be a single vector that is a multiple of $[1\ 2]^{T}$.

b) Yes: $C(\mathbf{M}) = â„^{2}$, so any independent set of two vectors can be a basis.

c) Yes

d) Yes

e) Yes for the same reason as (b).

4.Â

a) $\begin{bmatrix}0\\1\end{bmatrix}$

b) No null space

c) No null space

d) $\begin{bmatrix} 1 \\ -4 \\ 1/5 \end{bmatrix}$

5.Â

a) 2
b) 1
c) 0
d) Trick question; *dim* (*C* (**A**)) cannot be greater than 2.
e) Trick question; *dim* (*N* (**A**)) must be > 0.
f) 2
g) 1
h) 0

8.14 Code challenges

1.Â

Create two 4 Ã— 4 random numbers matrices, each with rank=3 (consult the code challenge from Chapter 7 for how to do this). Call those matrices A and B . Then find a vector in the null space of A (vector n). Finally, show that BAn is the zeros vector while ABn is not.

2.Â

The goal of this code challenge is to confirm the subspace dimensionalities expressed on page Â§ . Create a 16 Ã— 11 matrix with rank=9. Then identify bases for the left- and right-null spaces and determine their dimensionalities. Confirm that the dimensionality of the left-null space plus the dimensionality of the column space is 16 (the ambient dimensionality of the column space), and that the dimensionality of the null space plus the dimensionality of the row space is 11 (the ambient dimensionality of the row space).

8.15 Code solutions

1.Â

The goal here is to see that once youâ€™ve entered the null space of a matrix (**An**), you can never come back; and that matrix **b** rotates **n** such that **Bn** is no longer in the null space of **A** .

Code blockÂ 8.3: Python

```
importÂ numpyÂ asÂ npÂ
fromÂ scipy.linalgÂ importÂ null_spaceÂ
```

```
AÂ =Â np.random.randn(4,3)@np.random.randn(3,4)Â
BÂ =Â np.random.randn(4,3)@np.random.randn(3,4)Â
nÂ =Â null_space(A)Â
print(B@A@n)Â #Â zerosÂ vectorÂ
print(A@B@n)Â #Â notÂ zerosÂ vector
```

Code blockÂ 8.4: MATLAB

```
AÂ =Â randn(4,3)*randn(3,4);Â
BÂ =Â randn(4,3)*randn(3,4);Â
nÂ =Â null(A);Â
B*A*nÂ %Â zerosÂ vectorÂ
A*B*nÂ %Â notÂ zerosÂ vector
```

2.Â

Recall that the dimensionality of the row space equals that of the column space, which equals the rank of the matrix.

Code blockÂ 8.5: Python

```
AÂ =Â np.random.randn(16,9)Â @Â
Â Â Â Â np.random.randn(9,11)Â
rnÂ =Â null_space(A)Â
lnÂ =Â null_space(A.T)Â
rÂ Â =Â np.linalg.matrix_rank(A)Â
print(rn.shape[1]+r)Â
print(ln.shape[1]+r)
```

Code blockÂ 8.6: MATLAB

```
AÂ =Â randn(16,9)*randn(9,11);Â
rnÂ =Â null(A);Â
lnÂ =Â null(Aâ€™);Â
rÂ Â =Â rank(A);Â
size(rn,2)+rÂ
size(ln,2)+r
```

ChapterÂ 9
Complex numbers in linear algebra

Complex numbers are used in theoretical and applied mathematics, for example in signal processing (e.g., the Fourier transform). Complex numbers play a small but important role in linear algebra, and youâ€™ll need to be at least a little bit comfortable working with complex numbers, for example when learning about eigendecomposition. The goal of this chapter is to introduce you to complex numbers and how certain linear algebra operations are adapted to matrices containing complex numbers.

If you are comfortable with the interpretation, graphing, and arithmetic of complex numbers, feel free to skim through the next several pages; section 9.5 is where we return to linear algebra.

9.1 Complex numbers and â„‚

Youâ€™ve probably heard this story before: A long, long time ago, mathematicians scratched their heads about how to solve equations like $x^2 + 1 = 0$. The answer, of course, is $x = \pm\sqrt{-1}$, but this is not a sensible answer (or so the elders thought) because no number times itself can give a negative number. Confused and concerned, they adopted the symbol $i = \sqrt{-1}$ and called it "imaginary" because it was the best term their imaginations could come up with.

The imaginary operator was, for a long time, just a quirky exception. It was Karl Friederich Gauss (yes, *that* Gauss)[1] who had the brilliant insight that the imaginary operator was not merely an exceptional case study in solving one kind of equation, but instead, that the imaginary unit was the basis of an entirely different *dimension* of numbers. These numbers were termed "complex" and had both a "real" part and an "imaginary" part. Thus was born the complex plane as well as the field of complex numbers, â„‚ .

[1] Fun fact: Gauss and many other people despised the term "imaginary," instead arguing that "lateral" would be better. I (and many others) whole-heartedly agree, but unfortunately "imaginary" remains standard terminology.

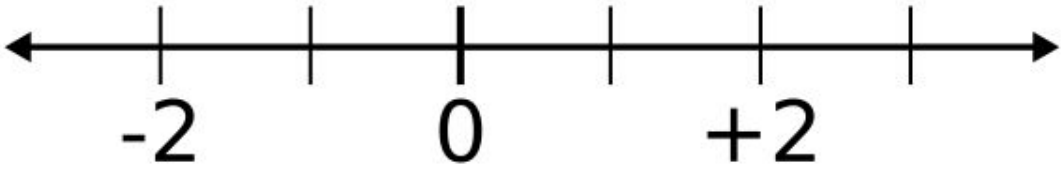
Gauss was interested in complex numbers because he was developing what is now known as the Fundamental Theory of Algebra. The FTA, as it is sometimes abbreviated, states that an nth order algebraic equation has exactly n roots. That is, an equation of the form:

$$a_0x^0 + a_1x^1 + \ldots + a_nx^n = 0 \tag{9.1}$$

has exactly *n* solutions for *x* (remember that $x^0 = 1$). The thing is that these solutions might be complex-valued. This important result is the reason why you can get complex-valued eigenvalues from real-valued matrices. More on this in Chapter 15. For now, letâ€™s review complex numbers.

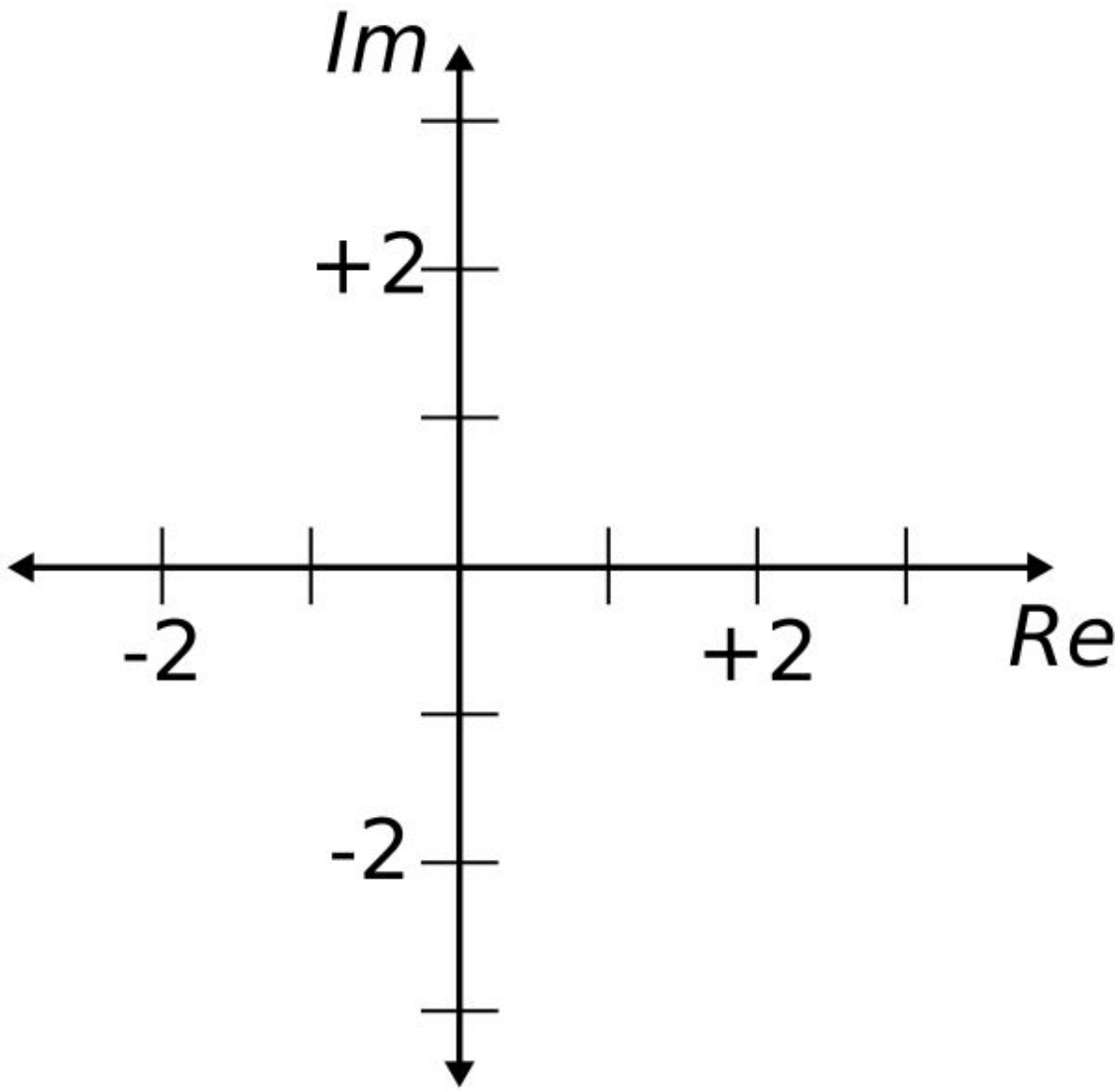
9.2 What are complex numbers?

You learned in school that numbers exist on a line, with zero in the middle, negative numbers on the left, and positive numbers on the right, like in Figure 9.1



FigureÂ 9.1: The real numbers, all lined up.

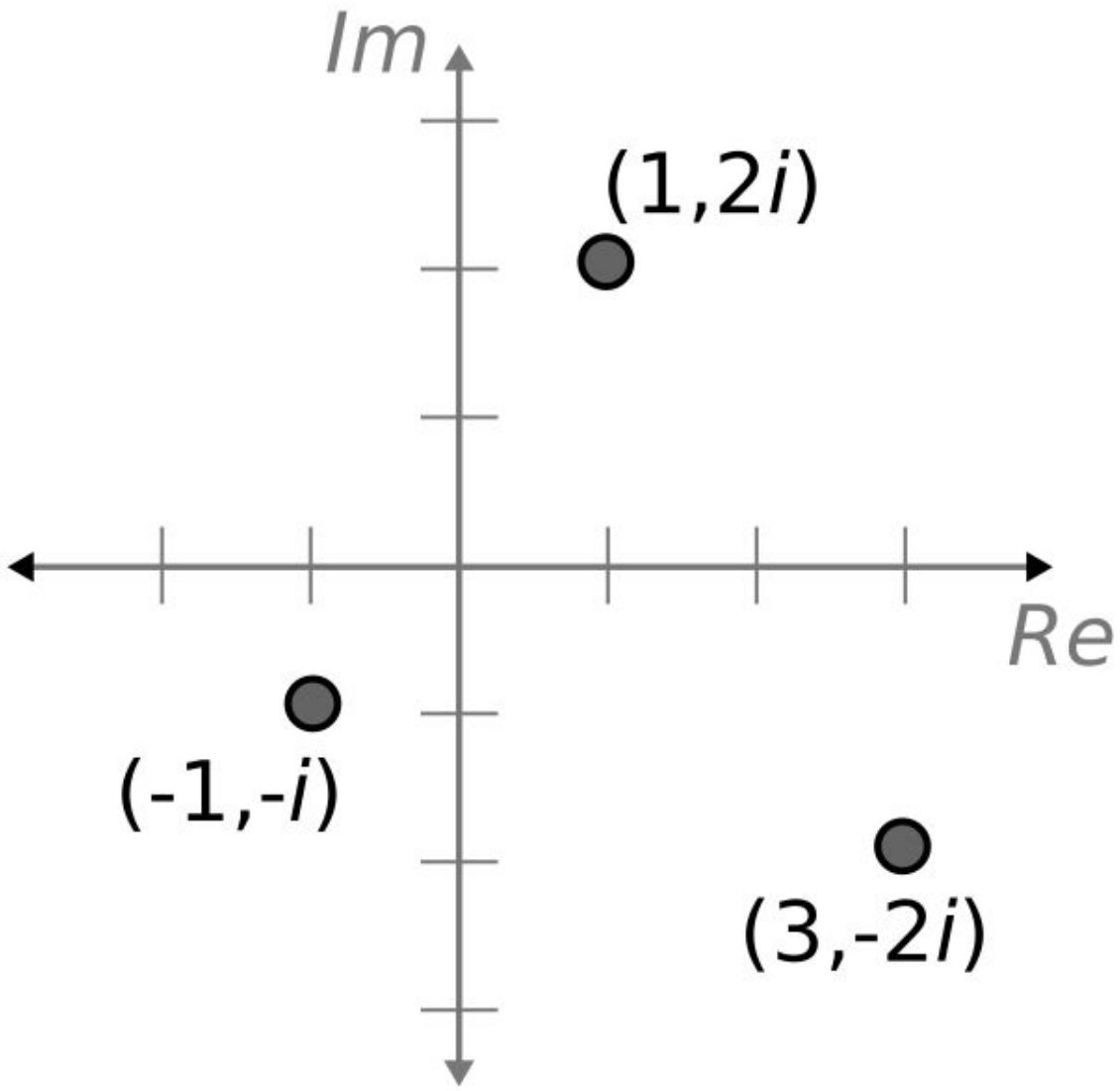
The idea of complex numbers is to extend this framework from a one-dimensional *line* to a two-dimensional *plane* . So now there are two axes for numbers, and each complex number is a coordinate in that plane. The horizontal axis is called the *real axis* (often abbreviated *Re*) and the vertical axis is called the *imaginary axis* (often abbreviated *Im*).



FigureÂ 9.2: The complex numbers, all planed up.

Donâ€™t lose sleep over what complex numbers *really mean* , whether imaginary numbers have any physical interpretation, or whether intelligent life elsewhere in the universe would also come up with imaginary numbers (I think the answer is Yes, and I hope they call them "lateral numbers"). It doesnâ€™t matter. What does matter is that complex numbers are useful from a practical perspective, and they simplify many operations in both theoretical and applied mathematics.

Complex numbers are referred to using the real and imaginary axis coordinates, just like how you would refer to XY coordinates on a Cartesian axis. Figure 9.3 shows a few examples of complex numbers as geometric coordinates and their corresponding labels.



FigureÂ 9.3: Example complex numbers on the complex plane.

A few remarks about indicating complex numbers in writing:

1. Complex numbers are always written using the real part first.
2. In between the two components could be a space, a comma, or a plus or minus sign. You might also see square brackets, parentheses, or nothing around the numbers. Variations include [a b*i*], (a,b*i*), [a+b*i*], a b*i* .
3. *z* is the go-to letter to indicate a complex number: *z* = a+*i* b. After that, *w* is the next-best-thing.
4. You can position the *i* before or after the imaginary component: [a b*i*] or [a *i* b].
5. Most people use *i* to indicate the imaginary operator. Engineers tend to use *j* because they use *i* for electrical current. On the other hand, engineers write

handwritten notes in ALL CAPS and start counting at 0, so letâ€™s not be too quick to adapt all engineering practices.

6. To avoid embarrassment at math parties, be careful to distinguish the following terms:
   - Complex number : A number that contains two parts, real and imaginary, like in the notational varieties above.
   - Imaginary number : A complex number with no real part: [0+*i* b] = *i* b.
   - Imaginary operator : The symbol (*i* or *j*) that represents the square root of minus one ($\sqrt{-1}$), without any other numbers attached to it.
   - Imaginary component : This is the real number that multiplies the imaginary operator. In the number a+*i* b, the imaginary component is b, not *i* b! Geometrically, this corresponds to the distance on the y-axis on a complex plane.

The reason why complex numbers are so useful is that they pack a lot of information into a compact representation. For the real number line, a number has only two pieces of information: its distance away from the zero and its sign (left or right of zero). A complex number contains more information: the real part; the imaginary part; the distance of the complex number to the origin; and the angle of the line from the origin to the complex number, relative to the positive real axis.

Complex vectors and complex matrices In some sense, complex matrices are really easy: They are just regular matrices except that the elements are drawn from â„‚, instead of â„ , which in practice means that at least one matrix element has a non-zero imaginary part. (Technically, the number 4 *i* 0 â^ˆ â„‚, , but for practical purposes, we can say that there should be some non-zero imaginary components.) Below are a few examples.

$$\begin{bmatrix} 6-i2 \\ 3+i4 \end{bmatrix}, \quad \begin{bmatrix} 1 \\ 2 \\ 3+i4 \\ i \end{bmatrix}, \quad \begin{bmatrix} 1+3i & 2-7i & -4+i \\ 4 & 7+2i & 4i \end{bmatrix}, \quad \begin{bmatrix} 7-4i & 1+i \\ 1+i & 2+2i \end{bmatrix}$$

There is a minor abuse of notation in the matrices above: the number 1 is actually 1 + 0 *i* ; likewise, the number 4 *i* is really the number 0 + 4 *i* . But I and most other people omit the 0-valued components for visibility.

Complex vectors and matrices are confusing at firstâ€”many students mistakenly think that the left-most column *vector* above is a 2 Ã— 2 *matrix* , and that the

right-most 2 Ã— 2 matrix is a 2 Ã— 4 matrix. If ever you see *i* in a vector or matrix, be especially careful to inspect and correctly interpret the matrix.

Code There are several ways to create complex numbers in Python and MATLAB. The complex operator in Python is `1j` (or any other number and `j`). MATLAB accepts `1i` and `1j` . Be mindful that Python is more stringent about data types than is MATLAB. For example, try running the code below without the second input into the `np.zeros()` function.

Code blockÂ 9.1: Python

```
z Â  = Â  np . complex (3,4) Â
Z Â  = Â  np . zeros (2, dtype = complex ) Â
Z [0] Â  = Â  3+4 j
```

Code blockÂ 9.2: MATLAB

```
z Â  = Â  complex (3,4); Â
Z Â  = Â  zeros (2,1); Â
Z (1) Â  = Â  3+4 i ;
```

9.3 The complex conjugate

The *conjugate* of a complex number is simply that number with the sign of the imaginary part flipped. It is indicated using an overbar (z) or using a superscripted asterisk ($z^{â^—}$). The equation below shows that the polar form of a complex number has the exponential multiplied by -1. If you are not familiar with polar notation, then donâ€™t worry about itâ€”I include it here in the interest of completeness but it is not used in the rest of the chapter.

$$z^* = z = \text{a}+\text{b}i = \text{a-b}i \tag{9.2}$$

$$z = re^{i\theta} = re^{-i\theta} \tag{9.3}$$

Be mindful that the complex conjugate means to flip the sign of the imaginary component, not set it to be negative. Thus, if the imaginary component is already negative, then its conjugate would have a positive imaginary component: a âˆ’ b*i* = a + b*i* .

SVG-Viewer needed.

FigureÂ 9.4: A complex number ([1 2i] and its complex conjugate ([1 -2i]).

Figure 9.4 depicts the geometric interpretation of the complex conjugate, which is to reflect the complex number across the real axis.

Complex conjugate pairs A complex conjugate pair is a complex number and its conjugate, together forever, just like penguins. Here are a few examples; note that the magnitudes of the real and imaginary parts are the same; the only difference is the sign of the imaginary part.

$$z,z = 4 + 2i, 4 \hat{a}\check{c}\check{a} 2i$$

$$w,w = 72 \hat{a}\check{c}\check{a} 32i, 72 + 32i$$

$$u,u = a + bi, a \hat{a}\check{c}\check{a} bi$$

Complex conjugate pairs are featured in the "Complex conjugate root theorem," which states that if a polynomial with real-valued coefficients has at least one complex-valued root, then it has 2N complex roots that come in conjugate pairs. For example:

$$x^2 + 9 = 0 \; x = +3i, \hat{a}\check{c}\check{a}3i$$

This should also be familiar from the quadratic equation [2]: When $b^2 < 4ac$ then the roots come in conjugate pairs.

[2] The quadratic equation: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

Code The complex conjugate is easy to implement. Iâ€™m using the opportunity to show you an additional way to create complex matrices.

Code blockÂ 9.3: Python

```
rÂ =Â np.random.randint(-3,4,size=3)Â
iÂ =Â np.random.randint(-3,4,size=3)Â
ZÂ =Â r+i*1jÂ
print(Z)Â
print(Z.conj())
```

Code blockÂ 9.4: MATLAB

```
rÂ =Â randi([-3,3],1,3);Â
iÂ =Â randi([-3,3],1,3);Â
ZÂ =Â r+i*1jÂ
conj(Z)
```

–

Practice problems Take the complex conjugate of the following numbers.

a) 3 + 4i

b) âˆ’6(âˆ’5 âˆ’ i)
c) j
d) 17

Answers

a) 3 âˆ’ 4i
b) âˆ’6(âˆ’5 + i)
c) âˆ’j
d) 17

-|*|- **Reflection** Complex conjugate pairs are used in many areas of applied mathematics. For example, the most efficient way to compute a power spectrum from the Fourier transform is to multiply the complex-valued spectrum by its complex conjugate. More germane to this book: A matrix with entirely real-valued entries can have complex-valued eigenvalues; and when there are complex-valued eigenvalues, they always come in conjugate pairs. -|*|-

9.4 Arithmetic with complex numbers

In this section, the scalars z and w refer to complex numbers, such that:

$$z = a + ib$$
$$w = c + id$$

Addition and subtraction Adding two complex numbers works the way you would think it should:

$$z + w = a + ib + c + id$$
$$= (a+c) + i(b+d)$$

In other words, sum the real parts, then sum the imaginary parts and attach the i. The only potential source of error here is misplacing the parentheses (ib + d).

Subtraction works exactly the same way; just be careful that you are replacing the correct plus signs with a minus signs. Pay attention to the pluses and minuses here:

$$z - w = a + ib - (c + id)$$
$$= (a-c) + i(b-d)$$

Multiplication Multiplication of complex numbers, unfortunately, does not work the way you might initially expect. You might have expected (hoped) that you would separately multiply the two real parts and the two imaginary parts, and

then put them together. Instead, you have to incorporate the cross-terms. Fortunately, though, the multiplication does follow algebraic rules you already know for expanding grouped terms.

$$zw = (a + ib)(c + id) \quad (9.4)$$

$$= ac + iad + ibc + i^2 bd \quad (9.5)$$

$$= ac\text{-}bd + i(ad+bc) \quad (9.6)$$

Notice that $i^2 bd = \hat{a}\hat{}'bd$.

An interesting thing happens when you multiply a complex number by its conjugate. Observe:

$$z^{\hat{a}\hat{}—} z = (a+bi)(a-bi)$$

$$= a^2 \hat{a}\hat{}' (bi)^2 \hat{a}\hat{}' abi + abi$$

$$= a^2 + b^2 \quad (9.7)$$

In addition to being practical, this is an important result, and equation 9.7 is used throughout complex linear algebra and signal processing. In the next section, youâ€™ll see how this property is related to the dot product with complex vectors.

Division Division is even weirder. There is a trick for doing division with complex numbers[3]. I have no idea who invented it, but I suspect it was Gauss. That trick is to multiply both the numerator and the denominator by the complex conjugate of the denominator.

[3] A rule in mathematics: when in doubt, credit Gauss.

$$\frac{z}{w} = \frac{a + ib}{c + id}$$

$$= \frac{(c - id)(a + ib)}{(c - id)(c + id)}$$

$$= \frac{(c - id)(a + ib)}{c^2 + d^2}$$

$$= \frac{(ca+db) + i(cb\text{-}da)}{c^2 + d^2}$$

Notice that the denominator becomes real-valued, which makes the fraction easier to work with. In the interest of learning-by-repetition, here is the concept again in compact form:

$$\frac{z}{w} = \frac{wz}{ww} \quad (9.8)$$

-|*|- **Reflection** Why does the denominator have to be real-valued? I honestly have no idea. But all my math teachers and everyone who writes about math in books and on the Internet says that we should avoid having complex numbers in the denominator of a fraction. I donâ€™t condone conformity and I think itâ€™s important to question the status-quo, but on the other hand, you gotta pick your battles. This one isnâ€™t worth fighting. -|*|-

---

Practice problems For the following two complex numbers, implement the indicated arithmetic operations.

$$z = 3 + 6i, \;\; w = -2 - 5i$$

a) 2z + wz
b) w(z + z)
c) 5z + 6w
d) 5z + 6w
e) $\frac{(w+2)z}{w}$

Answers

a) 30 âˆ’ 15i
b) âˆ’12 âˆ’ 30i
c) 3
d) 3 + 60i
e) (1 + 12i)$\frac{15}{29}$

9.5 The Hermitian and complex dot products

The Hermitian transpose, often called simply the Hermitian, is a fancy term for a conjugate-and-transpose operation. It is indicated with a superscripted H instead of a T (v^H). Sometimes you will also see the Hermitian indicated with a superscripted asterisk ($v^{\hat{a}\hat{}—}$) but this notation can lead to confusion because it can also indicate the conjugate (without transpose) of a vector. To be on the safe side, I will use an overbar on top of a vector (v) to indicate the conjugate of the elements in the vector without transposing that vector. Below is an example of a vector and its Hermitian.

$$\begin{bmatrix} 2 & 4-i5 & 1 & 2+i9 \end{bmatrix}^{\mathrm{H}} = \begin{bmatrix} 2 \\ 4+i5 \\ 1 \\ 2-i9 \end{bmatrix}$$

Notice that nothing happened to the real-valued elements (first and third entries). For this reason, the Hermitian and "regular" transpose are identical operations for real-valued matrices.

Dot product with complex vectors The dot product with complex vectors is exactly the same as as the dot product with real-valued vectors: element-wise multiply and sum.

However, in nearly all cases, the "regular" dot product is replaced with the Hermitian dot product, which simply means to implement the dot product as z^H w instead of z^T w.

SVG-Viewer needed.

FigureÂ 9.5: The vector [0 i] has length=1, which the dot product formula must produce.

Why are complex vectors conjugated when computing the dot product? The answer will be obvious from a geometric perspective: Recall that the dot product of a vector with itself is the squared length of the line represented by that vector. Consider what happens when we compute the length of a complex vector that we know has length=1 (Figure 9.5):

$$\mathbf{v} = \begin{bmatrix} 0 \\ i \end{bmatrix}. \quad \begin{cases} \mathbf{v}^{\mathrm{T}}\mathbf{v} = 0^2 + i^2 & = -1 \\ \mathbf{v}^{\mathrm{H}}\mathbf{v} = 0^2 + (-i)(-i) & = -1 \\ \mathbf{v}^{\mathrm{H}}\mathbf{v} = 0^2 + (-i)(i) & = 1 \\ \mathbf{v}^{\mathrm{T}}\mathbf{v} = 0^2 + (i)(-i) & = 1 \end{cases}$$

Clearly, the third and fourth options provide accurate results. This example also shows that it doesnâ€™t matter which vector is conjugated, although the third option (v^H v) is generally preferred for typographical reasons.

Code The MATLAB dot() function always implements the Hermitian dot product, because conjugating real-valued numbers has no effect. In Python, however, you need to use vdot() instead of dot().

Code blockÂ 9.5: Python

```
importÂ numpyÂ asÂ npÂ
vÂ =Â [0,1j]Â
print(np.dot(v,v))Â
print(np.vdot(v,v))
```

Code blockÂ 9.6: MATLAB

```
vÂ =Â [0Â 1i];Â
dot(v,v)
```

–

Practice problems For the following vectors, implement the specified operations.

$$\mathbf{z} = \begin{bmatrix} 1 \\ 3i \\ 4-2i \end{bmatrix}, \mathbf{w} = \begin{bmatrix} i \\ 1-i \\ 0 \end{bmatrix},$$

a) $z^H (z + w)$
b) $\frac{\mathbf{z}^H \mathbf{z}}{\mathbf{w}^H \mathbf{w}}$
c) 2z + w âŠ™ z

Answers

a) 27 âˆ’ 2i
b) âˆ’10
c) $\begin{bmatrix} 2+i \\ 3+9i \\ 8-4i \end{bmatrix}$

–

9.6 Special complex matrices

Special complex matrices sometimes have different names from their real-valued counterparts. Below are two such matrices that you may encounter.

Hermitian matrices A Hermitian matrix is the complex-valued equivalent of something between a symmetric matrix ($A = A^T$) and a skew-symmetric matrix

($A = \hat{}'A^T$).

A Hermitian matrix is defined as $A = A^H$. Thus, the magnitudes of the real and imaginary parts are the same, but the signs of the imaginary parts are swapped. Below is an example; Please take a moment and mentally confirm that $A^H = A$.

$$\begin{bmatrix} 2 & 3-2i & 2+2i \\ 3+2i & 5 & 8 \\ 2-2i & 8 & 9 \end{bmatrix} \tag{9.9}$$

Notice that the diagonal elements must be real-valued, because only real-valued numbers are equal to their complex conjugate (a + 0i = a â^' 0i). Notice also that Hermitian matrices may contain numbers with non-zero-valued imaginary parts.

Unitary matrix For real-valued matrices, an "orthogonal matrix" is one for which its transpose is its inverse; thus, multiplying the matrix by its transpose gives the identity matrix ($QQ^T = I$). Another way to phrase this is that each column in an orthogonal matrix is orthogonal with each other column, and that each column has unit magnitude. Youâ€™ll learn more about these special matrices in Chapter 13.

A complex-valued matrix that has such properties is called a unitary matrix. Here is the typical example that is used to illustrate a unitary matrix:

$$\frac{1}{2}\begin{bmatrix} 1+i & 1-i \\ 1-i & 1+i \end{bmatrix}$$

You will have the opportunity to confirm that this matrix is indeed unitary in the code challenges. There is a lot more that could be said about complex matrices in linear algebra. However, the topics introduced in this chapter cover what you will need to know for most linear algebra applications, including everything you need to know for the rest of this book.

9.7 Exercises

1.Â

Implement the specified operations using the following variables.

$$w = 2+5i, \quad z = -i, \quad \mathbf{d} = \begin{bmatrix} 4 \\ 2+2i \\ -4i \end{bmatrix}, \quad \mathbf{R} = \begin{bmatrix} 3 & 4 & i \\ -2i & 0 & 0 \\ 4 & 1 & 1 \end{bmatrix}$$

a) wd
b) d^H wd
c) Rd
d) R^H Rd
e) wz
f) $wz^{\hat{a}\hat{}—}$
g) wdR
h) wd^H R

9.8 Answers

1.Â

a) $\begin{bmatrix} 8+20i \\ -6+14i \\ 20-8i \end{bmatrix}$

b) 80 + 200i

c) $\begin{bmatrix} 24+8i \\ -8i \\ 18-2i \end{bmatrix}$

d) $\begin{bmatrix} 160+16i \\ 114+30i \\ 26-26i \end{bmatrix}$

e) 5 â^’ 2i

f) â^’5 + 2i

g) Wrong sizes!

h) $\begin{bmatrix} -44+64i & 12+88i & -40+16i \end{bmatrix}$

9.9 Code challenges

1.Â

On page Â§ I showed an example of a unitary matrix. Confirm that this is unitary by showing that U^H U = I. Also confirm that U^T Uâ‰ I.

2.Â

In Chapter 6 you learned two methods (additive and multiplicative) to create a symmetric matrix from a non-symmetric matrix. What happens when you apply those methods to complex matrices? To find out, generate a 3Ã—3 matrix of complex random numbers. Then apply those two methods to generate two new matrices, and test whether those matrices are (1) symmetric, (2) Hermitian, or (3) neither.

9.10 Code solutions

1.Â

Both Python and MATLAB allow you to implement Hermitian and regular transpose operations. Be careful that you are implementing the correct version! Notice that the method .H in Python requires converting the matrix into a matrix object.

Code blockÂ 9.7: Python

```
UÂ =Â .5*np.array([Â [1+1j,1-1j],[1-1j,1+1j]Â ])Â
print(U@np.matrix(U).H)Â #Â HermitianÂ
U@U.TÂ #Â notÂ Hermitian
```

Code blockÂ 9.8: MATLAB

```
UÂ =Â .5*[1+1iÂ 1-1i;Â 1-1iÂ 1+1i];Â
Uâ€™*UÂ %Â HermitianÂ
transpose(U)*UÂ %Â notÂ Hermitian
```

2.Â

Both methods for creating symmetric matrices ($A + A^T$ and $A^T A$) work for complex matrices, except that the resulting matrices are Hermitian, not symmetric. As mentioned earlier, be mindful that you are taking the Hermitian transpose, not the "regular" transpose. Python does not have a built-in function to test whether a matrix is Hermitian, but subtracting two equal matrices will produce the zeros matrix.

Code blockÂ 9.9: Python

```
rÂ =Â np.random.randn(3,3)Â
iÂ =Â np.random.randn(3,3)Â
AÂ =Â np.matrix(Â r+i*1jÂ )Â
A1Â =Â A+A.HÂ
A2Â =Â A@A.HÂ
print(A1-A1.H)Â
print(A2-A2.H)
```

Code blockÂ 9.10: MATLAB

```
AÂ =Â complex(randn(3,3),randn(3,3));Â
A1Â =Â A+Aâ€™;Â
A2Â =Â A*Aâ€™;Â
ishermitian(A1)Â %Â issymmetric(A1)Â isÂ false!Â
ishermitian(A2)
```

ChapterÂ 10
Systems of equations

One of the many great things about linear algebraâ€”which Iâ€™ve written in previous chapters but is particularly germane to this chapterâ€”is that it can provide compact notation for large collections of mathematical expressions, equations, variables, data, and so on.

In this chapter you will learn how to represent systems of equations using matrices and vectors, and how to solve those systems using linear algebra operations. This knowledge will be central to a range of applications including matrix rank, the inverse, and least-squares statistical model fitting.

What is an equation? It is a statement of equality, typically with one or more unknown. For example:

$$2x = 6$$

To be sure, this single equation does not need matrices and vectors. In fact, using matrices to solve one equation simply creates gratuitous confusion. On the other hand, consider the following system of equations.

$$\begin{aligned} 2x + 3y \hat{a}\check{} 5z &= 8 \\ \hat{a}\check{} 2y + 2z &= \hat{a}\check{}3 \\ 5x \hat{a}\check{} 4z &= 3 \end{aligned}$$

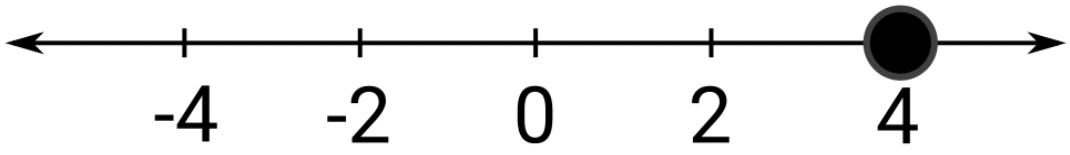
You can imagine an even larger system, with more variables and more equations. It turns out that this system can be represented compactly using the form Ax = b. And this isnâ€™t just about saving space and inkâ€”converting a system of many equations into one matrix equation leads to new and efficient ways of solving those equations. Are you excited to learn? Letâ€™s begin.

10.1 Algebra and geometry of equations

Algebraic equations have an associated picture. The number of variables (unknowns) in the equation determines the number of dimensions in the corresponding graph. For example, the equation

$$2x + 3 = 11 \tag{10.1}$$

has the solution x = 4, which can be visualized as a point on the number line:

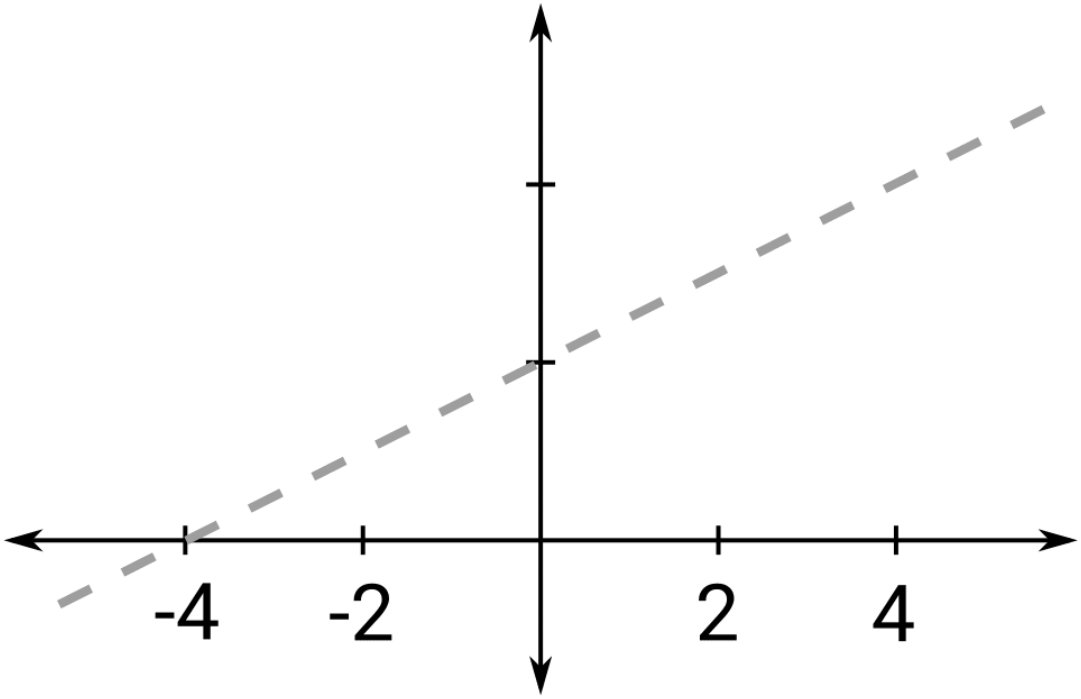


FigureÂ 10.1: A graph of the solution to Equation 10.1 .

Here is an equation with two variables:

$$2y = x + 4 \tag{10.2}$$

And Iâ€™m sure you know that the geometric translation of this equation is a line in a 2D space. You probably also know that any point on this line is a valid solution to the equation.



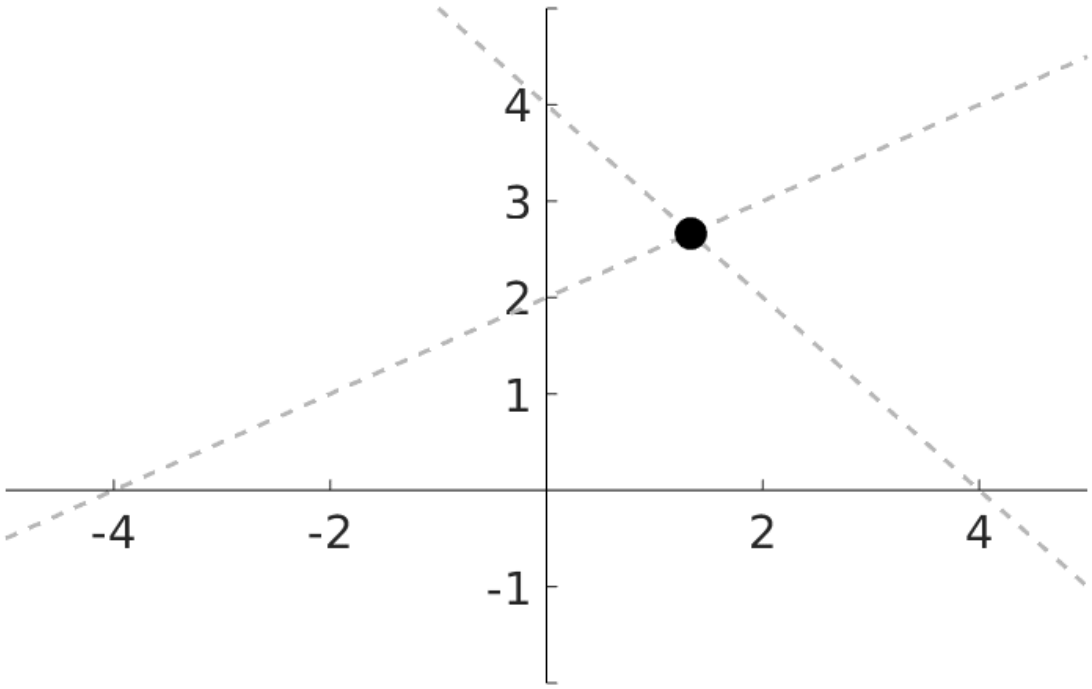
FigureÂ 10.2: A graph of the solution to Equation 10.2 .

An equation with 3 variables has an associated 3D graph, and so on.

So far, we been talking about individual equations. What do systems of equations look like? Consider the following system of two equations and two variables.

$$\left\{ \begin{array}{c} y = x/2 + 2 \\ y = -x + 4 \end{array} \right\} \tag{10.3}$$

If one 2-variable equation is a line, then a system of two 2-variable equations is two lines. Figure 10.3 shows the picture.



FigureÂ 10.3: A graph of the solution to the system of equations 10.3 .

Now things are starting to get interesting. The point on the graph where the two lines intersect is the solution to both equations. In this case, that point is (x,y) = (4âˆ•3,8âˆ•3). Try this yourself by plugging those values into both equations in system 10.3 . You can also try points that are on one line but not the other; you will find that those pairs of numbers will solve only one of the equations. Try, for example, (0,4) and (-4,0).

When learning about equations in high school, you learned that any arithmetic operation performed on one side of the equation must be done to the other. For example, in the equation x = 4, you may multiply both sides by 8 (8Ã—x = 8Ã—4), but it is not allowed to multiply only one side by 8.

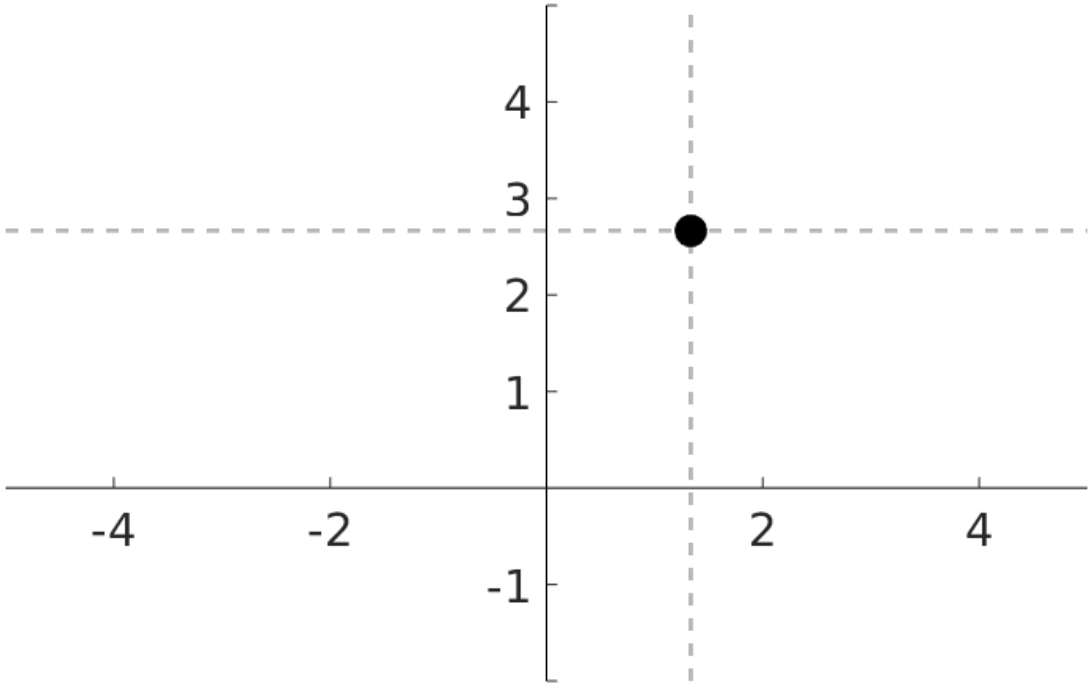
With a system of equations, you still have the same rule, although you donâ€™t have to apply the same operation to all equations in the system. But having a system of equations allows you to do something more: You may add and subtract entire equations from each other (this is analogous to multiplying the left-hand side by "8" and the right-hand side by "4Ã—2"). Letâ€™s try this with Equation 10.3 . I will transform the first equation to be itself minus the second equation:

$$\left\{ \begin{array}{c} 0y = 3x/2 - 2 \\ y = -x + 4 \end{array} \right\} \tag{10.4}$$

Next, I will replace the second equation by itself plus two times the original first equation:

$$\left\{ \begin{array}{l} 0y = 3x/2 - 2 \\ 3y = 0x + 8 \end{array} \right\} \tag{10.5}$$

At a superficial glance, system 10.5 looks really different from system 10.3 . What does the graph of this "new" system look like? Letâ€™s see:



FigureÂ 10.4: A graph of the solution to the system of equations 10.5 . Compare with Figure 10.3 .

The amazing thing here is that the solution stays the same, even though the lines are different. The same can be said of the algebraic equations (10.3 and 10.5): They look different, but the solution to the system remains the same before and after we subtracted equations from each other.

So the conclusion here is that you can take a system of equations, scalar multiply individual equations, and add and subtract equations from each other, to your heartâ€™s delight. The individual equationsâ€”and the lines representing those equationsâ€”will change, but the point of intersection will remain exactly the same (of course, this statement breaks down if you scalar-multiply by 0, so letâ€™s exclude this trivial annoyance).

Now let me explain why this concept is powerful. Consider the system of equations 10.6 . Take a moment to try to find the solution to this system (that is, the (x,y) pair

that satisfies both equations); try it based on visual inspection without writing anything down and without adding or subtracting the equations from each other.

$$\left\{ \begin{array}{rl} 2x + 3y = & 8 \\ -2x + 2y = & -3 \end{array} \right\} \tag{10.6}$$

Did you get it? Itâ€™s actually pretty difficult to solve in your head. Now I will add the first equation to the second; try again to solve the system without writing anything down.

$$\left\{ \begin{array}{rl} 2x + 3y = & 8 \\ 5y = & 5 \end{array} \right\} \tag{10.7}$$

Suddenly, this new second equation is much easier to solve. You see immediately that y = 1, and then you can plug that into the first equation to get x = 5âˆ•2 = 2.5. Thus, the solution is now easier to calculate because the second equation decouples y from x. This is the principle of row reduction and Gaussian elimination, which you will learn about later in this chapter.

Do all systems have a common algebraic solution and unique geometric crossing? No, definitely not. A system of equations will have (1) one intersecting point like in the examples above, (2) no points in common, or (3) an infinite number of points in common. Weâ€™ll get to this discussion later on in this chapter. First you need to learn how to convert a system of equations into a matrix-vector equation.

10.2 Matrices representing systems of equations

Converting a system of equations into a matrix equation is straightforward, and requires understanding the three components that together define a system of equations:

1. Variables: These are the unknowns that you want to solve for. They are typically labeled x, y, ..., or x_1 , x_2 , ...
2. Coefficients: These are the numbers that multiply the variables. There is one coefficient per variable. If the variable is sitting by itself, then the coefficient is 1; if the variable is not present, then the coefficient is 0.
3. Constants: These are the numbers that do not multiply variables. Every equation has one constant (which might be 0).

Sometimes, these components are easy to spot. For example, in the equation

$$2x + 3y - 4z = 5 \tag{10.8}$$

The variables are (x,y,z), the corresponding coefficients are (2,3,âˆ’4), and the constant is 5. In other equations, you might need to do a bit of arithmetic to separate the components:

$$2(x + 3y) - z - 5 = 1 \tag{10.9}$$

The variables are still (x,y,z), the coefficients are (2,6,âˆ’1), and the constant is 6.

Components to matrices Once youâ€™ve identified the components of a system, you put those components into matrices:

1. The coefficients go into the coefficients matrix, with columns corresponding to variables and with rows corresponding to equations.
2. The variables go into a column vector that right-multiplies the coefficients matrix. Importantly, the order of variables in this vector must match the order of variables in the columns of the matrix.
3. The coefficients matrix and variables vector are on the left-hand side of the equation. The constants go into a column vector on the right-hand side of the equation, with the number of elements in the vector corresponding to the number of equations. Of course, the nth element in the constants vector must correspond to the nth equation in the coefficients matrix.

For an example, letâ€™s work with the following system of equations:

$$\left\{ \begin{array}{rl} 2x+3y-5z= & 8 \\ -2y+2z+2= & -1 \end{array} \right\} \tag{10.10}$$

Observe how this system is translated into matrices and vectors. In particular, observe how the coefficients, variables, and constants are organized:

$$\begin{bmatrix} 2 & 3 & -5 \\ 0 & -2 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 8 \\ -3 \end{bmatrix} \tag{10.11}$$

Be mindful that one or more variables might be missing from an equation, in which case you must put zeros in the correct locations of the coefficients matrix. Also note that all numbers unattached to variables are combined into one constant.

Take a minute to work through the matrix-vector multiplication to convince yourself that Equation 10.11 is an accurate representation of the system of equations in 10.10 .

You can see that converting a system of equations into a matrix equation is not terribly difficult. But it does require vigilance, because a simple mistake here can have dire consequences later on. For example, consider the following system:

$$\left\{ \begin{array}{r} 2x+3y-5z=8 \\ 5x=2 \end{array} \right\} \tag{10.12}$$

If you are converting this system into matrices [1] while simultaneously Facebooking, savoring Vietnamese coconut coffee, and watching reruns of Rick and Morty, you might end up with the following matrix:

[1] Life pro tip: Vietnamese coconut coffee is really delicious.

$$\begin{bmatrix} 2 & 3 & -5 \\ 0 & 0 & 5 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$$

This is wrong because the second equation is actually now implemented as 5z = 2 whereas the original equation was 5x = 2. When there is potential for confusion, re-write the set of equations with all the variables vertically aligned, using 0â€™s where appropriate:

$$\begin{aligned} 2x + 3y - 5z &= 8 \\ 5x &= 2 \end{aligned} \quad \Rightarrow \quad \begin{aligned} 2x + 3y - 5z &= 8 \\ 5x + 0y + 0z &= 2 \end{aligned}$$

Practice problems (1 of 2) Convert the following systems of equations into their matrix form.

a) $\left\{ \begin{aligned} 2x + 3y + 75z &= 8 \\ -2y + 2z &= -3 \end{aligned} \right\}$

b) $\left\{ \begin{aligned} x - z/2 &= 1/3 \\ 3y + 6z &= 4/3 \end{aligned} \right\}$

c) $\left\{ \begin{aligned} s - t &= 6 \\ u + v &= 1 \\ t + u &= 0 \\ 2v + 3t &= 10 \end{aligned} \right\}$

d) $\left\{ \begin{aligned} x + y &= 2 \\ x - y &= 0 \end{aligned} \right\}$

Answers

a) $\begin{bmatrix} 2 & 3 & 75 \\ 0 & -2 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 8 \\ -3 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 0 & -1/2 \\ 0 & 3 & 6 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \frac{1}{3} \begin{bmatrix} 1 \\ 4 \end{bmatrix}$

c) $\begin{bmatrix} 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 3 & 0 & 2 \end{bmatrix} \begin{bmatrix} s \\ t \\ u \\ v \end{bmatrix} = \begin{bmatrix} 6 \\ 1 \\ 0 \\ 10 \end{bmatrix}$

d) $\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$

Practice problems (2 of 2) Convert the following matrix-vector products into their "long-form" equations (i.e., the opposite of the previous exercises).

a) $\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 \\ 5 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} j \\ k \end{bmatrix} = \begin{bmatrix} 10 \\ 9 \end{bmatrix}$

c) $\begin{bmatrix} 7 & 7 & 8 & 8 & 6 & 7 \\ 1 & 0 & 9 & 1 & 2 & 0 \end{bmatrix} \begin{bmatrix} q \\ w \\ e \\ r \\ t \\ y \end{bmatrix} = \begin{bmatrix} 9 \\ 9 \end{bmatrix}$

d) $\begin{bmatrix} 1 & 3 \\ 2 & 4 \\ 3 & 4 \\ 4 & 2 \end{bmatrix} \begin{bmatrix} s \\ t \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \\ 6 \\ 2 \end{bmatrix}$

Answers

a) Not a valid equation!

b) $\begin{Bmatrix} j & = & 10 \\ k & = & 9 \end{Bmatrix}$

c) $\begin{Bmatrix} 7q + 7w + 8e + 8r + 6t + 7y & = & 9 \\ 1q + 9e + r + 2t & = & 9 \end{Bmatrix}$

d) $\begin{Bmatrix} s + 3t & = & 5 \\ 2s + 4t & = & 4 \\ 3s + 4t & = & 6 \\ 4s + 2t & = & 2 \end{Bmatrix}$

-|*|- **Reflection** Sometimes, the biggest challenge in data analysis and modeling is figuring out how to represent a problem using equations; the rest is usually just a matter of algebra and number-crunching. Indeed, the translation from real-world problem to matrix equation is rarely trivial and sometimes impossible. In this case, representing a system of equations as matrix-vector multiplication leads to the compact and simplified notation: Ax = b. And this form leads to an equally compact solution via the least-squares algorithm, which is a major topic of Chapter 14. -|*|-

10.3 Row reduction, echelon form, and pivots

Row reduction may initially seem like a tangent from representing and solving systems of equations, however, it provides the computational backbone of solving systems of equations.

Row reduction involves modifying rows of a matrix while leaving many key properties of the matrix intact. It is based on the principle that you learned about in section 10.1 : in a system of equations, individual equations can be scalar-multiplied, added, and subtracted. Thus, row reduction involves linearly combining the rows of a matrix, with the goal of transforming a matrix into a form that facilitates subsequent inspection and analyses.

"Transforming the matrix to facilitate analyses" basically means increasing the number of zeros in the matrix. Zeros are great in matrices: the more zeros a matrix has, the easier and faster it is to work with. Thus, think of row reduction as a method to re-organize the information in a matrix in a way that increases the number of zero-valued entries.

So how does row reduction work? Itâ€™s exactly the same procedure we applied in Equation 10.6 (page Â§): Replace rows in the matrix with linear combinations of other rows in the same matrix. Letâ€™s start with an example.

Consider the matrix defined by the coefficients in the equation system 10.6 (the constants vector is omitted for now; after learning the mechanism of row reduction, youâ€™ll learn how to incorporate the right-hand side of the equations). Notice that by adding the first row to the second row, we get a zero in the (2,1) position.

$$\begin{bmatrix} 2 & 3 \\ -2 & 2 \end{bmatrix} \xrightarrow{R_1+R_2} {}^{[c]x} \quad (10.13)$$

[2] The arrow with $R_1 + R_2$ indicates the operation that was performed, in this case, adding row 1 to row 2.

$$\begin{bmatrix} 2 & 3 \\ 0 & 5 \end{bmatrix}$$

These two matrices are not the same, but they are related to each other by a simple linear operation that could be undone if we keep track of which rows were added to which other rows. That linear operation increased the number of zeros in the matrix from none to one, and so is consistent with our goal. In this case, the linear transformation converted the original matrix into an upper-triangular matrix, which, in the parlance of row reduction, is called the echelon form of the matrix.

Echelon form One of the main goals of row reduction is to convert a dense matrix into its echelon form. [3] A matrix is in its echelon form when the following two criteria are satisfied:

[3] Sometimes also called row echelon form.

1. The first non-zero number in each row is to the right of the first non-zero numbers in the rows above.
2. Rows of all zeros are below rows with at least one non-zero element.

The matrices below are all in echelon form:

$$\begin{bmatrix} 2 & 4 & 5 \\ 0 & 1 & 3 \\ 0 & 0 & 9 \end{bmatrix}, \begin{bmatrix} 4 & 3 & 0 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 5 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 9 \end{bmatrix}, \begin{bmatrix} 4 & 1 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Notice how each of these matrices conforms to the two criteria: the first non-zero term in each row is to the right of the first non-zero term in the rows above, and rows of all zeros are on the bottom.

Obviously, most matrices are not already in their echelon form. This is where row reduction comes in: Apply row reduction to a matrix until it reaches echelon form. (Some matrices need to have rows swapped; weâ€™ll deal with this situation later.)

Letâ€™s try an example with a 3Ã—3 matrix. The goal is to find the multiples of some rows to add to other rows in order to obtain the echelon form of the matrix. Itâ€™s often easiest to start by clearing out the bottom row using multiples of the top row(s).

$$\begin{bmatrix} 1 & 2 & 2 \\ -1 & 3 & 0 \\ 2 & 4 & -3 \end{bmatrix} \xrightarrow[-2R_1+R_3]{\substack{[c]x \\ x}} \begin{bmatrix} 1 & 2 & 2 \\ -1 & 3 & 0 \\ 0 & 0 & -7 \end{bmatrix} \xrightarrow[x]{\substack{x \\ R_1+R_2}} \begin{bmatrix} 1 & 2 & 2 \\ 0 & 5 & 2 \\ 0 & 0 & -7 \end{bmatrix}$$

As with many operations in linear algebra (and math in general), the procedure is easy to implement in small examples with carefully chosen integer numbers. Computers can take care of the arithmetic for harder problems, but itâ€™s important that you understand the procedure.

One of the nice features of the echelon form is that linear dependencies in the columns and rows reveal themselves. In the example above, you can immediately see that the third column cannot be formed from a linear combination of the first two. It is similarly straightforward to use the presence of zeros to convince yourself that the first row cannot be created by combinations of the other two rows (for example, nothing times zero can produce the 1 in the top-left element). So, this matrix comprises a set of linearly independent columns (and rows), which means this is a rank-3 (full-rank) matrix.

Watch what happens to the echelon form when there are linear dependencies.

$$\begin{bmatrix} 1 & 2 & 2 \\ -1 & 3 & 0 \\ 2 & 4 & 4 \end{bmatrix} \xrightarrow[-2R_1+R_3]{[c]x \\ x} \begin{bmatrix} 1 & 2 & 2 \\ -1 & 3 & 0 \\ 0 & 0 & 0 \end{bmatrix} \xrightarrow[x]{x \\ R_1+R_2} \begin{bmatrix} 1 & 2 & 2 \\ 0 & 5 & 2 \\ 0 & 0 & 0 \end{bmatrix}$$

When the columns (or rows) of a matrix form a linearly dependent set, the echelon form of the matrix has at least one row of zeros.

In fact, this is one way to compute the rank of a matrix: Transform it to its echelon form, and count the number of rows that contain at least one non-zero number. That count is the rank of the matrix. Iâ€™ll have more to say about the relationship between rank and row reduction in a later section, but this statement (rank is the number of non-zeros rows in the echelon form) is the main idea.

Practice problems Convert the following matrices into their echelon form.

a) $\begin{bmatrix} 1 & 0 & 2 \\ 2 & 1 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 2 & 1 \end{bmatrix}$

c) $\begin{bmatrix} 2 & 2 & 1 \\ 4 & 0 & 1 \\ 3 & 1 & 1 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & -3 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}$

c) $\begin{bmatrix} 2 & 2 & 1 \\ 0 & -4 & -1 \\ 0 & 0 & 0 \end{bmatrix}$

A few tips for row reduction Row reduction is admittedly kindof a weird procedure when you first start doing it. But after you solve several problems, it will start to feel more natural. Here are a few tips that might help you avoid difficult arithmetic. These are not steps that you always implement; these are strategies to keep in mind that might make things easier.

1) Divide an entire row by a scalar to make the left-most non-zero number equal 1.

$$\begin{bmatrix} 3 & 6 & 9 \\ 1 & 1 & 1 \\ 2 & 1 & 1 \end{bmatrix} \begin{matrix} [c] \\ x \\ x \end{matrix} \xrightarrow{R_1/3} \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \\ 2 & 1 & 1 \end{bmatrix}$$

2) Multiply a row by a scalar to facilitate eliminating elements.

$$\begin{bmatrix} 3 & 6 & 9 \\ 1 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix} \begin{matrix} [c]x \\ x \\ -R_1+3R_3 \end{matrix} \xrightarrow{} \begin{bmatrix} 3 & 6 & 9 \\ 1 & 1 & 1 \\ 0 & -3 & -3 \end{bmatrix}$$

3) Multiply a row by a scalar to get rid of difficult fractions.

$$\begin{bmatrix} 3 & 6 & 9 & 2 \\ 1 & 1 & 2 & 4 \\ 0 & 0 & 2/3 & 17/6 \end{bmatrix} \begin{matrix} [c]x \\ x \\ 6R_3 \end{matrix} \xrightarrow{} \begin{bmatrix} 3 & 6 & 9 & 2 \\ 1 & 1 & 2 & 4 \\ 0 & 0 & 4 & 17 \end{bmatrix}$$

Keeping track of row reduction Although the matrices change during row reduction, those changes are reversible if you keep track of them. Letâ€™s continue working with the 2Ã—2 matrix from a few pages ago:

$$\mathbf{A} = \begin{bmatrix} 2 & 3 \\ -2 & 2 \end{bmatrix}$$

Itâ€™s trivial that IA = A. But what would happen if we change the identity matrix just a bit? Letâ€™s see: [4]

[4] This is no longer the identity matrix, so Iâ€™ll call it R for reduction.

$$\mathbf{RA} = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ -2 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ -4 & 4 \end{bmatrix}$$

What weâ€™ve done here is keep row 1 the same and double row 2. But how do we linearly combine rows? Well, if changing the diagonal elements affects only the corresponding row, then we need to change the off-diagonal elements to combine rows. To replace row-2 with row-1 plus row-2, you put a 1 in the (2,1) position of the R matrix:

$$\mathbf{RA} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ -2 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 0 & 5 \end{bmatrix}$$

More generally, to multiply the ith row by Ïf and add it to the jth row, you enter Ïf into the (i,j) entry of the identity matrix to form an R matrix.

Each nth step of row reduction has its own R_n matrix, each one to the left of A and any previously applied R matrices. Depending on why you are implementing row reduction, you might not need to keep track of the transformation matrices, but it is important to understand that every echelon matrix E is related to its original form A through a series of transformation matrices:

$$\mathbf{E} = \mathbf{R_n R_{n-1} ... R_1 A} \tag{10.14}$$

This also shows how row reduction does not actually entail losing information in the matrix, or even fundamentally changing the information in the matrix. Instead, we are merely applying a sequence of linear transformations to reorganize the information in the matrix. (That said, row reduction without storing the R matrices does involve information-loss; whether that matters depends on the goal of row-reduction. More on this point later.)

Exchanging rows in a matrix In the examples of row reduction thus far, the matrices "just so happened" to be constructed such that each row had nonzero elements to the right of nonzero elements in higher rows (criteria #1 of an echelon form matrix). This is not guaranteed to happen, and sometimes rows need to be exchanged. Row exchanges have implications for several matrix properties.

Exchanging rows of a matrix is a linear transformationâ€”sometimes called a permutation because we are permuting the row orderâ€”which means it can be expressed as a matrix multiplication. The way to accomplish it is by first manipulating the identity matrix with how you want the rows to be exchanged (letâ€™s call this matrix P for permutation), then left-multiplying the to-be-permuted matrix by P.

For example, if you want to exchange the first and second rows of a 3Ã—3 matrix, create matrix P, which is the identity matrix with the first two rows exchanged, then left-multiply by this permutation matrix.

$$\mathbf{PA} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 4 & 5 & 6 \\ 1 & 2 & 3 \\ 7 & 8 & 9 \end{bmatrix} \tag{10.15}$$

Important: This is a good illustration of how matrix multiplication is non-commutative. Watch what happens when the same permutation matrix right-multiplies A.

$$\mathbf{AP} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 3 \\ 5 & 4 & 6 \\ 8 & 7 & 9 \end{bmatrix} \tag{10.16}$$

Thus: left-multiply for row exchanges, right-multiply for column-exchanges (this is also why the Râ€™s are on the left). To help you remember this, Iâ€™m going to recycle a mnemonic I introduced in the chapter on matrix multiplications (box ??, page ??):

Eqn. box title=Order of matrices for transforming rows vs. columns

PRe-multiply to transform Rows

POst-multiply to transform cOlumns.

If you are confused about why PA and AP are so different, you might consider reviewing section 6.1 on thinking about matrix multiplication as linear combinations of rows vs. columns.

Now we can add to our general formula for transforming a matrix to its echelon form (Equation 10.14) to include row exchanges.

$$\mathbf{E} = \mathbf{R_n R_{n-1} \ldots R_1 P A} \tag{10.17}$$

(You might need additional permutation matrices, in which case they would be called P_1 , P_2 , and so on.)

The permutation matrix is the mechanism to keep track of the transformations you apply to the matrix. As I wrote above, in many cases, you donâ€™t need to keep track of these transformation matrices; you can simply swap rows as needed.

Letâ€™s see an example of reducing a matrix to its echelon form that requires a row swap.

$$\begin{bmatrix} 2 & 1 & 4 \\ 4 & 2 & 9 \\ 8 & 5 & 3 \end{bmatrix} \xrightarrow[-4R_1+R_3]{[c]x} \begin{bmatrix} 2 & 1 & 4 \\ 4 & 2 & 9 \\ 0 & 1 & -9 \end{bmatrix} \xrightarrow[x]{-R_1+R_2} \begin{bmatrix} 2 & 1 & 4 \\ 0 & 0 & 1 \\ 0 & 1 & -9 \end{bmatrix}$$

Notice that the right-most matrix is not in its echelon form, because the leading non-zero term in the third row is to the left of the leading non-zero term in the second row. Swapping the second and third rows will set things right, and weâ€™ll have our proper echelon form.

Row swapping has implications for the sign of the determinant , which is a number associated with a square matrix that is zero for all singular square matrices and non-zero for full-rank matrices. Youâ€™ll learn more about determinants in the next chapter, including the effect of row swaps. I mention it here to create a space in your

brain for this information to be slotted in. To quell suspense: Each row swap flips the sign of the determinant but does not change its magnitude.

Pivots In mechanics, a pivot is a point around which something rotates or moves. The pivot is an important part of a moving system; think of the fulcrum of a see-saw. Pivots in a matrix are similarly important, because they reveal several key properties of a matrix.

After putting the matrix into echelon form, the pivots are the left-most non-zero elements in each row. Not every row has a pivot, and not every column has a pivot. A zero cannot be a pivot even if itâ€™s in a position that could be a pivot. Thatâ€™s because pivots are used as a denominator in row-reduction, and terrible things happen when you divide by zero. In the following matrix, the gray boxes highlight the pivots.

$$\begin{bmatrix} a & b & c & d & e \\ 0 & f & g & h & i \\ 0 & 0 & 0 & j & k \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

The pivots of a matrix are generally not visible before transforming to echelon form. That is to say, you cannot determine the pivots of a matrix without first converting it into its echelon form.

Code There are no dedicated functions in Python or in MATLAB to apply row reduction to obtain the echelon form of a matrix. This is because the echelon form of a matrix is non-unique, meaning you can obtain many distinct E matrices for any given A matrix (of course, the R and P matrices will differ for different E matrices). Youâ€™ll learn more about the non-uniqueness of the echelon form in a few pages.

However, there is a way to obtain an echelon form of a matrixâ€”and the corresponding R and P matricesâ€”through a procedure called LU decomposition . "LU" stands for lower-upper, and is a decomposition that represents a matrix as the product of two triangular matrices. There are several algorithms of LU decomposition, one of which is conceptually similar to Gaussian elimination. I wonâ€™t go into a lengthy discussion of LU decomposition here, but the idea in the context of the echelon form of a matrix is the following:

$$LU = PA \tag{10.18}$$

$$E = U \tag{10.19}$$

$$R_n R_{nâ^'1} ...R_1 = L^{â^'1} \tag{10.20}$$

Code blockÂ 10.1: Python

```
fromÂ scipy.linalgÂ importÂ luÂ
P,L,UÂ =Â lu(A)
```

Code blockÂ 10.2: MATLAB

```
[L,U,P]Â =Â lu(A);
```

Practice problems Use row reduction to transform the matrix into its echelon form, and then identify the pivots.

a) $\begin{bmatrix} 1 & 2 & 2 & 4 \\ 2 & 0 & 3 & 2 \\ 8 & 0 & 4 & 0 \end{bmatrix}$

b) $\begin{bmatrix} 2 & 0 & 1 \\ 1 & 1 & 2 \\ 3 & 1 & 3 \end{bmatrix}$

c) $\begin{bmatrix} 3 & 2 & 1 \\ 3 & 2 & 7 \\ 9 & 6 & 3 \\ 1 & 1 & 1 \end{bmatrix}$

Answers Echelon form is shown; pivots are highlighted.

a) $\begin{bmatrix} 1 & 2 & 2 & 4 \\ 0 & -4 & -1 & -6 \\ 0 & 0 & -8 & -8 \end{bmatrix}$

b) $\begin{bmatrix} 2 & 0 & 1 \\ 0 & 1 & 1.5 \\ 0 & 0 & 0 \end{bmatrix}$

c) $\begin{bmatrix} 3 & 2 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & 6 \\ 0 & 0 & 0 \end{bmatrix}$

Pivot-counting and rank The rank of a matrix is the number of pivots in the echelon form of that matrix. Letâ€™s think about why this is the case.

During row reduction, any row that can be created using a linear weighted combination of other rows in the matrix will turn into a row of zeros. We could phrase this another way: If row reduction produces a zeros row, then it means that

some linear combination of row vectors equals the zeros vector. This is literally the definition of linear dependence (Equation 4.6 , page Â§).

But this only proves that row reduction can distinguish a reduced-rank matrix from a full-rank matrix. How do we know that the number of pivots equals the rank of the matrix?

One way to think about this is using the theorem presented in Chapter 8, that the rank of a matrix is the dimensionality of the row space. Row reduction makes the dimensionality of the row space crystal clear, because all rows that can be formed by other rows are zeroed out. Any remaining rows necessarily form a linearly independent set, and the rank of a matrix is the largest number of rows that can form a linearly independent set. Each non-zeros row in the echelon form contains exactly one pivot; hence, the number of pivots in a matrix equals the number of non-zeros rows in the echelon form of the matrix, which equals the dimensionality of the row space (and thus also the dimensionality of the column space), which equals the rank.

For this same reason, row reduction reveals a basis set for the row space: You simply take all the non-zeros rows.

So the echelon form "cleans up" the matrix to reveal the dimensionality of the row space and therefore the rank of the matrix. And the "cleaning" process happens by moving information around in the matrix to increase the number of zero-valued elements. _____

Practice problems Use row reduction to compute the rank of the following matrices by counting pivots.

a) $\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$

b) $\begin{bmatrix} 3 & 3 & 5 \\ -4 & 4 & 0 \\ 5 & 1 & 5 \\ 6 & 2 & 1 \end{bmatrix}$

c) $\begin{bmatrix} 2 & 4 & 6 \\ 5 & 3 & 2 \end{bmatrix}$

Answers Echelon form is shown; pivots are highlighted.

a) Easy: no pivots!
r = 0

b) $\begin{bmatrix} 1 & 1 & 5/3 \\ 0 & 6 & 5 \\ 0 & 0 & 30 \\ 0 & 0 & 0 \end{bmatrix}$,r = 3

c) $\begin{bmatrix} 1 & 2 & 3 \\ 0 & 7 & 13 \end{bmatrix}$,r = 2

Non-uniqueness of the echelon form Did you always get the exact same echelon matrices that I got in all the exercises above? You might have, but you might also have gotten some slightly different results.

The echelon form of a matrix is non-unique. This means that a given matrix can have multipleâ€"equally validâ€"associated echelon-form matrices. Thatâ€™s because of row-swapping and row-scalar multiplications. Take any echelon matrix, pick a row at random, and multiply that randomly selected row by some random scalar. There is literally no end to the possible matrices you can create this way, and they will all be the echelon form of the original matrix (assuming no arithmetic mistakes).

On the other hand, some features of the infinite number of echelon form matrices for a given matrix are constant. In particular, the number of pivots will always be the same (although the numerical values of the pivots may differ), as will the number of zeros rows (this is obvious, because the number of zeros rows is M âˆ’ r, where M is the number of rows and r is the rank).

10.4 Gaussian elimination

Gaussian elimination is an application of row reduction to solving a system of equations. Let us return to the system presented in Equation 10.6 , which Iâ€™ll reproduce here:

$$\left\{ \begin{array}{rl} 2x + 3y = & 8 \\ -2x + 2y = & -3 \end{array} \right\} \qquad (10.6)$$

We discovered that itâ€™s difficult to solve this system because of the multiple x and y terms. So far in this chapter, you learned how to convert this system into a matrix equation, and then you learned how to apply row reduction to transform a matrix into its echelon form.

Now weâ€™re going to extend this procedure by a little bit. In particular, we will do the following:

1. Augment the coefficients matrix by the constants vector.
2. Row-reduce to echelon form.

3. Apply back-substitution to solve the system.

Step 1 produces the following matrix:

$$\left[\begin{array}{cc|c} 2 & 3 & 8 \\ -2 & 2 & -3 \end{array}\right] \tag{10.21}$$

When implementing row reduction, make sure to apply the row operations to the entire row, not just the numbers from the coefficients matrix. Step 2 will produce the following matrix:

$$\left[\begin{array}{cc|c} 2 & 3 & 8 \\ 0 & 5 & 5 \end{array}\right] \tag{10.22}$$

Before moving on to step 3, Iâ€™m going to simplify this matrix by replacing row 2 with itself divided by 5.

$$\left[\begin{array}{cc|c} 2 & 3 & 8 \\ 0 & 1 & 1 \end{array}\right] \tag{10.23}$$

Now weâ€™re ready for step 3. What is "back substitution"? This means mapping the matrix-vector equation back into a "long-form" system of equations, and then solving the system from the bottom row to the top row.

$$\left\{\begin{array}{r} 2x + 3y = 8 \\ y = 1 \end{array}\right\} \tag{10.24}$$

Take a moment to compare system 10.24 to matrix 10.23 , and make sure you see how they map onto each other.

Now the solution to this system is easy to compute, and you can understand where the name "back-substitution" comes from: Starting from the bottom row and moving upwards, substitute variables in each equation until youâ€™ve solved for all the variables. In this case, we already see that y = 1, and substituting that into the first equation leads to x = 5âˆ•2.

As I mentioned in the outset of this chapter, row manipulations change the equations and the corresponding graphs, but the solution stays the same. Therefore, itâ€™s a good idea to substitute the solution we discovered here back into the original system (Equation 10.6) to confirm that the solution solves the system. This is an important test, because if the solution from Gaussian elimination does not work for the original system, then youâ€™ve made a mistake somewhere, either in row reduction or in the arithmetic during back-substitution.

Are you impressed by Gaussian elimination? Perhaps you are now thinking that this is the pinnacle of using linear algebra to solve systems of equations, and it couldnâ€™t possibly get better than this. Well, prepare to be proven wrong!

Practice problems Perform Gaussian elimination to solve the following systems of equations.

a) $\left\{\begin{array}{c} 2x-3z=-8 \\ y+4z+3x=25 \\ -z+x=-2 \end{array}\right\}$

b) $\left\{\begin{array}{c} 3y+x+2z=36.5 \\ 2x+6y-2z=-29 \\ y+z=18 \\ 14x-z=-24 \end{array}\right\}$

Answers

a) $\left\{\begin{array}{c} x=2 \\ y=3 \\ z=4 \end{array}\right\}$

b) $\left\{\begin{array}{c} x=-.5 \\ y=1 \\ z=17 \end{array}\right\}$

-|*|- **Reflection** Before the Internet age, communication among scientists was slow, difficult, and limited. Geographical distances and language barriers exacerbated the problem. Iâ€™m not a math historian, but I did read up a bit about the history of the term "Gaussian elimination." Apparently, the method was known to Chinese mathematicians even before Christ was born. Gauss himself did not discover the method, but improved it, along with better notation, from Newton (who might have discovered it independently of the Chinese). The attribution to Gauss was made in the 20th century, 100 years after his death. He could neither correct the record nor inappropriately take credit. -|*|-

10.5 Row-reduced echelon form

Often abbreviated RREF, this is row reduction taken to the extreme. The goal here is to continue with row reduction after converting a matrix to its echelon form, until it becomes a matrix where all pivots have a value of 1, and each pivot is the only non-zero element in its column. Here is the procedure to get a matrix into its row-reduced echelon form.

1. Transform a matrix into its echelon form as described earlier.

2. For each row that contains a pivot:

(a) Divide that row by its pivot, which converts the pivot into the number 1.

(b) Apply row-reduction but work upwards instead of downwards. That is, use row-reduction to produce zeros in the elements above the pivot. Continue "upwards row-reduction" until the pivot is the only non-zero element in its column.

It is often useful to apply step 2 from the bottom row of the matrix to the top. Thus, to obtain the echelon form of the matrix, you work from the top row down to the bottom; and to obtain the RREF, you work from the bottom row (or the last row with pivots if there are rows of zeros) back up to the top.

Below are a few examples of matrices and their row-reduced echelon forms; notice that all pivots have a value of one and that the pivot is the only non-zero entry in its column.

$$\begin{bmatrix} 5 & \begin{matrix} 1 \\ \text{[5] Linear dependencies in the columns produce zeros row(s) and pivotless column(s).} \\ 7 \end{matrix} & \begin{matrix} 4 & \begin{matrix} 2 & 3 \\ 5 & 6 \\ 8 & 9 \end{matrix} \end{matrix} \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 5 & 2 & 1 & 0 & 4 \\ 4 & 1 & 5 & 6 & 3 \\ 2 & 1 & 9 & 0 & 4 \end{bmatrix} \Rightarrow \begin{bmatrix} 6 & \begin{matrix} 1 \\ \text{[6] A wide matrix with independent rows becomes } \mathbf{I} \text{ augmented by other numbers.} \\ 0 \end{matrix} & \begin{matrix} 0 & 0 & 3.4 & -.03 \\ 1 & 0 & -8.6 & 1.967 \\ 0 & 1 & .2 & .233 \end{matrix} \end{bmatrix}$$

$$\begin{bmatrix} 7 & \begin{matrix} 1 \\ \text{[7] A tall matrix with linearly independent columns has } \mathbf{I} \text{ on top and zeros underneath.} \\ 4 \\ 1 \end{matrix} & \begin{matrix} 6 & \begin{matrix} 2 \\ 2 \\ 7 \\ 7 \end{matrix} \end{matrix} \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 8 & \begin{matrix} 1 \\ \text{[8] A full-rank square matrix transforms into } \mathbf{I}. \\ 6 \end{matrix} & \begin{matrix} 4 & \begin{matrix} 2 & 3 \\ 1 & 2 \\ 4 & 2 \end{matrix} \end{matrix} \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

At this point, you might be concerned that transforming a matrix into its RREF is non-unique, meaning that many different matrices will become the same row-reduced echelon matrix. Thatâ€™s true: although each matrix has a unique RREF (that is, there is exactly one RREF associated with a matrix), many different matrices can lead to the same RREF. For example, all square full-rank matrices have the identity matrix as their RREF.

However, you already know how to keep track of the progression from the original matrix to its RREF: Put each step of row reduction and row swaps into transformation matrices, as described earlier in this chapter. On the other hand, the usual goal of RREF is to solve a system of equations, and that solution is the same for the original matrix, the echelon form, and the RREF. Thus, in practice, you will probably not need to worry about saving the intermediate transformation matrices.

Code Computing the RREF of a matrix is easy. In Python, you must first convert the matrix (which is likely to be stored as a numpy array) into a sympy matrix.

Code blockÂ 10.3: Python

```
importÂ numpyÂ asÂ npÂ
importÂ sympyÂ asÂ symÂ
AÂ =Â np.random.randn(2,4)Â
sym.Matrix(A).rref()
```

Code blockÂ 10.4: MATLAB

```
AÂ =Â randn(2,4);Â
rref(A)
```

Practice problems Reduce these matrices to their RREF.

a) $\begin{bmatrix} 4 & 8 & 4 & 3 \\ 0 & 1 & 8 & 0 \\ 2 & 7 & 0 & 8 \end{bmatrix}$

b) $\begin{bmatrix} 4 & 0 & 2 \\ 8 & 1 & 7 \\ 4 & 8 & 0 \\ 3 & 0 & 8 \end{bmatrix}$

c) $\begin{bmatrix} 13 & 7 & 4 & 7 \\ 5 & 3 & 2 & 3 \\ 2 & 1 & 1 & 1 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 1 & 0 & 0 & -3 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & -1/4 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$

10.6 Gauss-Jordan elimination

Gauss-Jordan elimination is like a super-charged version of Gaussian elimination. Itâ€™s basically the same procedure that was described on page Â§ , except you modify step 2: Instead of row-reducing the coefficients-constants augmented matrix to its echelon form, you row-reduce the matrix to its RREF. The idea is that the row reduction takes a bit longer, but the back-substitution becomes much easier. I will continue with the example show previously. We arrived at the following matrix:

$$\left[\begin{array}{cc|c} 2 & 3 & 8 \\ 0 & 1 & 1 \end{array}\right] \tag{10.25}$$

Now letâ€™s continue with upwards row reduction to get all pivots equal to 1 and the only non-zero elements in their columns:

$$\left[\begin{array}{cc|c} 2 & 3 & 8 \\ 0 & 1 & 1 \end{array}\right] \begin{matrix}[c] \\ x \end{matrix} \xrightarrow{-3R_2+R_1} \left[\begin{array}{cc|c} 2 & 0 & 5 \\ 0 & 1 & 1 \end{array}\right] \begin{matrix}[c] \\ x \end{matrix} \xrightarrow{.5R_1} \left[\begin{array}{cc|c} 1 & 0 & 5/2 \\ 0 & 1 & 1 \end{array}\right]$$

Now we map the right-most matrix above back onto their equations:

$$\left\{\begin{array}{c} x = 5/2 \\ y = 1 \end{array}\right\}$$

OK, so now itâ€™s time for back-substitution... except you donâ€™t need to do any back-substitution! All the work is done for you; you simply read off the solutions right from the equations. _____

Practice problems Implement Gauss-Jordan elimination to solve the following systems of equations.

a) $\left\{\begin{array}{l} 3x + 10y = 62 \\ -x + 3y = 11 \end{array}\right\}$

b) $\left\{\begin{array}{l} 3x + 10y = 11.5 \\ -x + 3y = -1.3 \end{array}\right\}$

c) $\left\{\begin{array}{l} 4x - 6y + 3z = 1 \\ 5z + 2y + x = 8 \\ 3y + x - 4z = 0 \end{array}\right\}$

Answers

a) $\left\{\begin{array}{l} x = 4 \\ y = 5 \end{array}\right\}$

b)

$$\begin{Bmatrix} x = 5/2 \\ y = 2/5 \end{Bmatrix}$$

c) x = y = z = 1

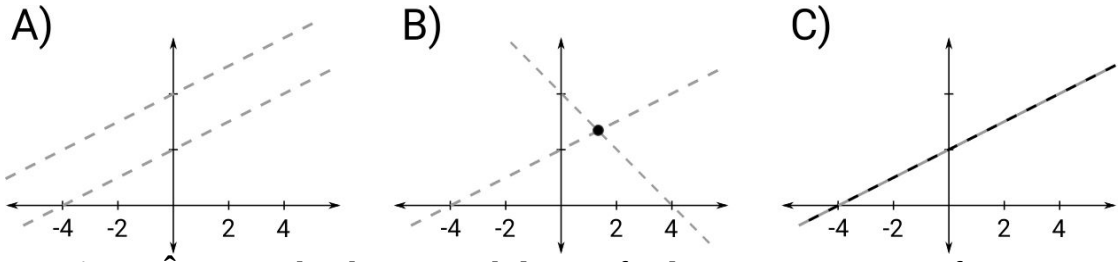
-|*|- **Reflection** Math has many life-lessons if you look at it in the right way. The wisdom in this chapter is the following: Try to solve a problem (system of equations) with no preparation or organization, and you will spend a lot of time with little to show except stress and frustration. Do some preparation and organization (Gaussian elimination) and you can solve the problem with a bit of dedication and patience. And if you spend a lot of time preparing and strategizing before even starting to solve the problem (Gauss-Jordan elimination), the solution may simply reveal itself with little additional work. Youâ€™ll be left with a deep sense of satisfaction and you will have earned your Mojito on the beach. -|*|-

10.7 Possibilities for solutions

All of the example systems shown so far in this chapter have had a unique solution. But this is not always the case. In fact, there are three possibilities:

1. The system has no solutions.
2. The system has exactly one unique solution.
3. The system has an infinite number of solutions.

This might be more intuitive to understand geometrically. Figure 10.5 shows how the three possibilities would look (this is for a system with two equations and two variables, but this generalizes to higher dimensions).



FigureÂ 10.5: The three possibilities of solutions to systems of equations.

Letâ€™s consider each possibility in turn.

No solution (Figure 10.5A) Geometrically, the lines for the different equations never touch. In 2D that means that lines are parallel; in higher dimensions there are other

possibilities, but still the lines do not have a common meeting point. Here is an example of a system without a solution:

$$\left\{ \begin{array}{r} y = x/2 + 2 \\ y = x/2 + 4 \end{array} \right\} \tag{10.26}$$

Itâ€™s clear from the equations that there is no common solution. Of course, each equation on its own can be solved, but the system of equations is inconsistent. Hereâ€™s what the coefficients-constants matrix and its RREF look like:

$$\left[\begin{array}{cc|c} -.5 & 1 & 2 \\ -.5 & 1 & 4 \end{array}\right] \quad \Rightarrow \quad \begin{bmatrix} 1 & -2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Do you spot the problem? Letâ€™s map the RREF back onto a system of equations:

$$\left\{ \begin{array}{r} y - 2x = 0 \\ 0 = 1 \end{array} \right\} \tag{10.27}$$

Notwithstanding some silly "proofs" that require division by zero [9], we generally do not accept the conclusion that 0 = 1. Therefore, this system is inconsistent; it has no solution.

[9] For example: https://www.pleacher.com/mp/mhumor/onezero2.html

Unique solution (Figure 10.5B) This is the case for the examples weâ€™ve worked through earlier in this chapter. The system has exactly one solution and you can use Gaussian elimination to find it.

Infinite solutions (Figure 10.5C) Geometrically, this means that the two equations are collinear; they are in the same subspace. Hereâ€™s a system that would produce Figure 10.5 C:

$$\left\{ \begin{array}{r} y = x/2 + 2 \\ 2y = x + 4 \end{array} \right\} \tag{10.28}$$

Notice that the second equation is the first multiplied by 2. And here is that systemâ€™s corresponding matrix and RREF, and below that, the mapping back into a system of equations:

$$\left[\begin{array}{cc|c} -.5 & 1 & 2 \\ -1 & 2 & 4 \end{array}\right] \quad \Rightarrow \quad \begin{bmatrix} 1 & -2 & -4 \\ 0 & 0 & 0 \end{bmatrix}$$
$$\left\{ \begin{array}{r} y - 2x = 4 \end{array} \right\} \tag{10.29}$$

Yeah, itâ€™s not really a "system" anymore, unless you want to include an extra equation that reads 0 + 0 = 0.

So what is the solution to this equation? There are many. Pick whatever you want for x and then solve for y. Or pick whatever you want for y and solve for x.

-|*|- **Reflection** An important aspect of life is balancing priorities, which is basically like trying to find a solution that satisfies multiple equations. As you now know, a system of equations has (1) zero solutions, (2) exactly one solution, or (3) an infinite number of equally good solutions. So if life is all about balancing equations, then it seems that one of these three possibilities is correct:

1. Life has no purpose.
2. Life has exactly one purpose.
3. Life has an infinite number of equally good purposes.

-|*|-

10.8 Matrix spaces after row reduction

In Chapter 8 you learned about matrix spaces and subspaces, and in this chapter you learned about how to reorganize the contents of a matrix by linear combinations of rows. Row-reduction has implications for the span of matrix subspaces, which you will learn in this section.

To quell suspense, the main take-home message here is that row reduction does not affect the row space of the matrix, but it can drastically affect the column space of the matrix; on the other hand, the dimensionalities of the matrix subspaces do not change. Now letâ€™s work through this methodically.

Letâ€™s talk about rank first. You already know that rank is a property of the matrix, not of the rows or the columns. The rank doesnâ€™t change before vs. after row-reduction. In fact, row-reduction makes the rank easier to compute (count the pivots).

Next letâ€™s think about the row space. That doesnâ€™t change after row reduction. To understand why, think about the definition of a subspace and the process of row reduction: A subspace is defined as all possible linear combinations of a set of vectors. That means that you can take any constant times any vector in the subspace, add that to any other constant times any other vector in the subspace, and the resulting vector will still be in the subspace.

Now think about the process of row reduction: you take some constant times one row and add it to another row. That is entirely consistent with the algebraic definition of a subspace. So you can do row-reduction to your heartâ€™s delight and you will never leave the initial subspace.

The only characteristic of the row space that could change is the basis vectors: If you take the rows of the matrix (possibly sub-selecting to get a linearly independent set)

as a basis for the row space, then row reduction will change the basis vectors. But those are just different vectors that span the same space; the subspace spanned by the rows is unchanged before, during, and after row reduction.[10]

[10] Anyway, the rows tend to be a poor choice for basis vectors. You will understand why when you learn about the SVD.

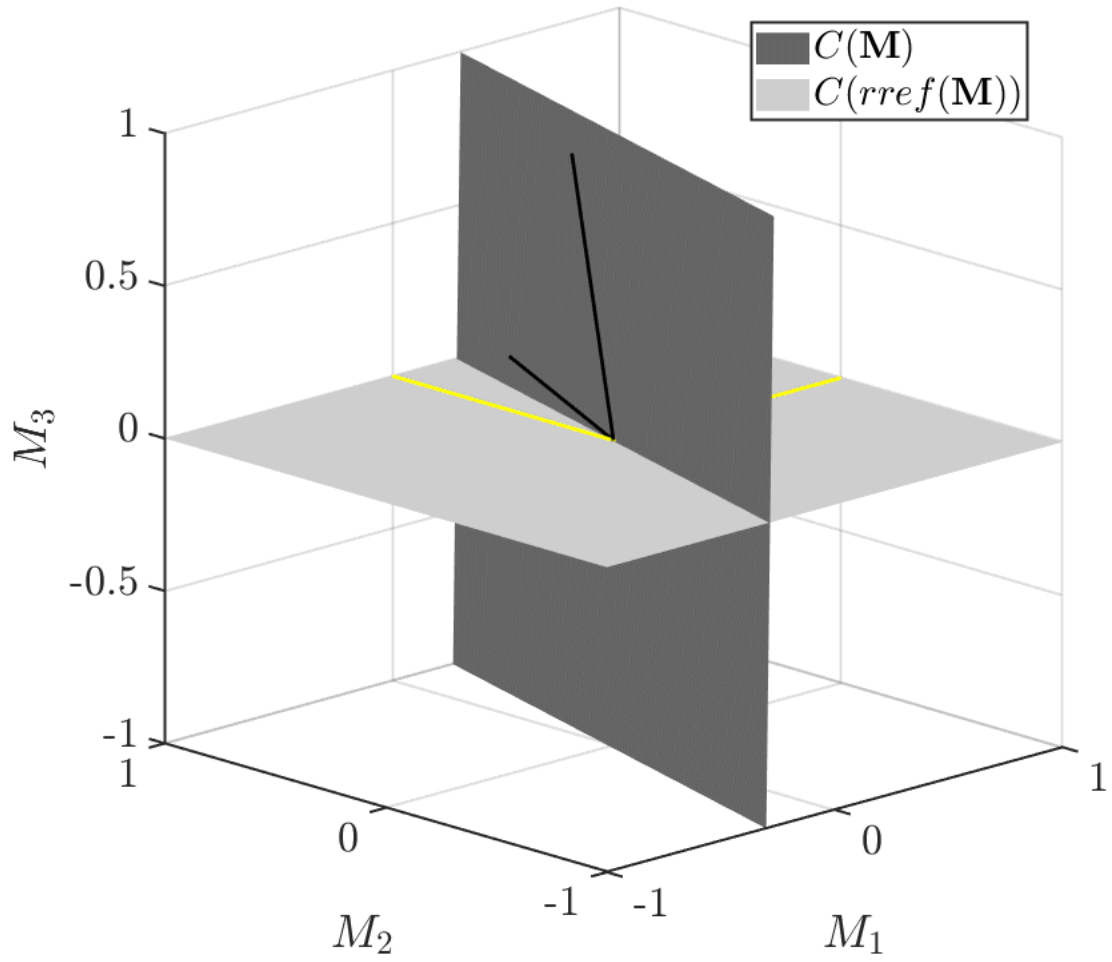
Now letâ€™s talk about the column space. The column space actually can change during row reduction. Let me first clarify that the dimensionality of the column space does not change with row reduction. The dimensionality will stay the same because the dimensionality of the row space, and the rank of the matrix, are unaffected by row reduction.

But what can change is the actual subspace that is spanned by the columns. This can happen when the column space occupies a lower-dimensional subspace of the ambient space in which the columns live. The reason is that row reduction involves changing entire rows at a time; individual elements of a column will change while other elements in the same column stay the same. I believe that one clear example will suffice for understanding. Below you can see a matrix and its RREF.

$$\mathbf{M} = \begin{bmatrix} 1 & 2 \\ 3 & 7 \\ 9 & 1 \end{bmatrix}, \quad rref(\mathbf{M}) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \tag{10.30}$$

The two columns form a linearly independent set in both matrices (clearly seen in both, though easier to see in the RREF), so we can take the columns as bases for the column spaces. In both matrices, the column space is a 2D plane embedded in a 3D ambient space.

But are they the same plane, i.e., the same subspace? Not at all! They intersect at the origin, and they have a line in common (that is, there is a 1D subspace in both 2D subspaces), but otherwise they are very different from each otherâ€”before RREF it was a tilted plane and after RREF itâ€™s the XY plane at Z=0 (Figure 10.6).



FigureÂ 10.6: A graph of the column spaces before and after RREF for matrix M (Equation 10.30). Column vectors are drawn on top of the planes (unit-normalized for visualization).

On the other hand, keep in mind that row reduction is not necessarily guaranteed to change the column space. Consider the following matrix and its RREF:

$$\begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix} \quad \Rightarrow \quad \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Itâ€™s a full-rank matrix and its RREF is the identity matrix. The column space spans all of â„ ^{2} both before and after row reduction. Now this situation is analogous to the row space: The elements in the columns are different but the subspaces spanned by those columns are exactly the same.

Again, the take-home messages in this section are that row reduction (1) does not affect the rank or the dimensionalities of the matrix subspaces, (2) does not change

the row space, and (3) can (but does not necessarily) change the column space.

10.9 Exercises

1.Â

Reduce the following matrices to their echelon form. Highlight the pivots.

a) $\begin{bmatrix} 2 & 0 & -1 & -1 \\ 1 & -2 & -2 & 5 \\ -3 & 5 & -4 & -1 \\ 0 & -3 & -1 & -3 \end{bmatrix}$

b) $\begin{bmatrix} 2 & 1 & 3 & -1 \\ -5 & -6 & -5 & 3 \\ -3 & -4 & -3 & 2 \\ -1 & -2 & -1 & 1 \end{bmatrix}$

2.Â

Given the following matrix sizes and ranks, determine the number of zeros rows in the row-reduced echelon form.

a) â„ 2Ã—3 ,r = 2

b) â„ 3Ã—2 ,r = 2

c) â„ 7Ã—7 ,r = 7

d) â„ 7Ã—7 ,r = 3

e) â„ 7Ã—2 ,r = 2

f) â„ 2Ã—7 ,r = 2

g) â„ 6Ã—7 ,r = 0

h) â„ 4Ã—4 ,r = 4

3.Â

Use row reduction to determine whether the following systems have zero, one, or an infinite number of solutions. You donâ€™t need to solve the system.

a) $\left\{ \begin{array}{c} 4x - 3y + 3z = 4 \\ 5x + y + z = 7 \\ x + 4y - 2z = 1 \end{array} \right\}$

b) $\left\{ \begin{array}{c} 4x - 3y + 3z = 4 \\ 5x + y + z = 7 \\ x + 4y - 2z = 3 \end{array} \right\}$

c)

$$\left\{\begin{array}{c} 4x-3y+3z=4 \\ 5x+y+z=7 \\ 2x+4y-2z=1 \end{array}\right\}$$

4.Â
Solve the following systems of equations, by converting the equations in a matrix equation and then apply Gaussian elimination and back-substitution (or Gauss-Jordan elimination).

a) $\left\{\begin{array}{c} 2x+4y-6z=-15 \\ y-4z=-8 \\ x+6z=10 \end{array}\right\}$

b) $\left\{\begin{array}{c} 3z+y-x=12 \\ -2y+10x-z=12 \\ 9x-y+2z=24 \\ 4x+3y+2z=22 \end{array}\right\}$

10.10 Answers

1.Â

a) $\begin{bmatrix} 1 & -2 & -2 & 5 \\ 0 & 4 & 3 & -11 \\ 0 & 0 & -37 & 45 \\ 0 & 0 & 0 & \frac{1554}{29} \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 & 1 & -1 \\ 0 & -3 & 1 & 1 \\ 0 & 0 & 2 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$

2.Â

a) 0
b) 1
c) 0
d) 4
e) 5
f) 0
g) 6
h) 0

3.Â

a) No solutions
b) âˆž solutions
c) One solution

4.Â

a) x = 1,y = â€’2,z = 3â€•2
b) x = 2,y = 2,z = 4

10.11 Coding challenges

1.Â

How do you think I came up with the sample systems of equations in this chapter? Is it because Iâ€™m a super-duper-math-genius? Unfortunately, thatâ€™s definitely not the case. But I am a decent coder. And in this code challenge, you will learn the age-old secret of creating your own systems of equations. Below is an explanation of the procedure; your task is to write code to implement it. To start with, use three equations and three variables (x, y, and z).

1. Create a vector that contains the values of the variables, which is actually the numbers that you want to solve for in the equations. For example, if you pick x = 3 and y = â€’4, then the vector of solutions is x = [3 â€’ 4].
2. Make up the expressions with random coefficientsâ€”but donâ€™t worry about the solution (the constants). For example, 2x â€’ 3z.
3. Create the coefficients matrix like what you learned in section 10.2 . Letâ€™s call this matrix A.
4. Compute the constants as Ax.

And voilÃ ! You now have all the pieces of a system of equations. If you want to create systems that have zero, one, or infinite solutions, you will need to make some minor adjustments to the above procedure (e.g., by changing a coefficient after already solving for the constants).

2.Â

The margin notes in section 10.5 described patterns in the RREF of a matrix that depend on the size of the matrix. The idea is that every RREF is essentially the identity matrix, possibly with some additional rows of zeros or columns of non-zero numbers. Write code to explore these possibilities. In particular, create random numbers matrices that are (1) square, (2) wide, or (3) tall; and then compute and examine the RREF of those matrices.

10.12 Code solutions

1.Â

Hint: You can actually skip from step 1 directly to step 3; step 2 is useful if you plan on writing out the full equations. To create a system with no solutions, start by creating a system with one solution, and then manually change one of the numbers. To create a system with infinite solutions, start by creating a system with one solution, and remove at least one equation from the system.

Code blockÂ 10.5: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.array([[2,0,-3],[3,1,4],[1,0,-1]])Â
xÂ =Â [2,3,4]Â
bÂ =Â A@x
```

Code blockÂ 10.6: MATLAB

```
AÂ =Â [Â 2Â 0Â -3;Â 3Â 1Â 4;Â 1Â 0Â -1];Â
xÂ =Â [2Â 3Â 4]â€™;Â %Â note:Â columnÂ vector!Â
bÂ =Â A*x;
```

2.Â

The code below shows an example of a wide matrix (3Ã—6).

Code blockÂ 10.7: Python

```
importÂ numpyÂ asÂ npÂ
importÂ sympyÂ asÂ symÂ
AÂ =Â np.random.randn(3,6)Â
sym.Matrix(A).rref()[0]
```

Code blockÂ 10.8: MATLAB

```
rref(randn(3,6))
```

ChapterÂ 11
Matrix determinant

11.1 Four things to know about determinants

Thing 1: only for square matrices The determinant is defined only for square matrices. Any time you hear or read the word "determinant," you can immediately start thinking about square (MÃ—M) matrices. Therefore, all of the matrices in this chapter are square.

Thing 2: a matrix has one determinant The determinant is a scalar that contains information about the matrix. The determinant is unique in the sense that a matrix has exactly one determinant (of course, many different matrices can have the same numerical value as their determinant, but it is not possible for one matrix to have multiple distinct determinants). You might already be thinking about other numbers that characterize a matrix, like rank or dimensionality. The determinant is not the same thing as the rank, although it is related to the rank, which is the next thing to know.

Thing 3: zero for singular matrices The determinant is zero for a non-invertible matrixâ€” a matrix with rank r < M. You can also say that a matrix with linear dependencies in the columns or in the rows has a determinant of zero. In fact, when you learn about computing the inverse in the next chapter, youâ€™ll learn that you should compute the determinant of a matrix before even starting to compute the inverse.

Thing 4: notation The determinant is indicated using single-vertical lines around the matrix letter, by printing the contents of the matrix with vertical lines instead of square brackets, or by using det(A). Single vertical lines are used to disambiguate the determinant (|A|) from the magnitude or norm (â¥Aâ¥). The Greek character Î” is also commonly used to indicate the determinant. Thus:

$$det(\mathbf{A}) \quad = \quad |\mathbf{A}| \quad = \quad \begin{vmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{vmatrix} \quad = \quad \Delta$$

There are several applications of the determinant. The two applications used in this book are computing the inverse (and determining whether a matrix has an inverse) (Chapter 12), and uncovering the eigenvalues of a matrix (Chapter 15). A third application is in geometry: If a matrix contains the vertices of a polygon, then the absolute value of the determinant measures the volume of that polygon. I wonâ€™t discuss this application here.

The determinant is really difficult to compute for large matrices ("large" is a subjective term, but for calculating the determinant, Iâ€™d consider anything larger than 4Ã—4 to be large). In fact, computers are unable to compute the determinant reliably for large matrices. Youâ€™ll see this yourself in the code challenges for this chapter.

However, there are handy short-cuts for computing the determinant of 2Ã—2 and 3Ã—3 matrices. Therefore, I will introduce these short-cuts first, and then present the "full" determinant formula thereafter.

11.2 Determinant of a 2Ã—2 matrix

The determinant of a 2Ã—2 matrix equals the product of the diagonal minus the product of the off-diagonal.

[1]

---

[1] Donâ€™t be seduced by the simplicity of this equationâ€”this short-cut works only for 2Ã—2 matrices.

Eqn. box title=Equation for the determinant of a 2Ã—2 matrix.

$$\det\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = \text{ad â^’ bc} \qquad (11.1)$$

Here are a few examples with real numbers.

$$\begin{vmatrix} 1 & 2 \\ 0 & 1 \end{vmatrix} = 1\text{Ã—}1 - 2\text{Ã—}0 = 1 \tag{11.2}$$

$$\begin{vmatrix} 5 & -3 \\ -4 & 2 \end{vmatrix} = 5\text{Ã—}2 - (-3\text{Ã—}-4) = -2 \tag{11.3}$$

$$\begin{vmatrix} 1 & 2 \\ 2 & 4 \end{vmatrix} = 1\text{Ã—}4 - 2\text{Ã—}2 = 0 \tag{11.4}$$

Notice that the first two matrices have a rank of 2 (and, thus, the columns form a linearly independent set), while the third matrix has a rank of 1. And that the determinant was non-zero for the first two matrices and zero for the third matrix.

That is no coincidence: Every reduced-rank matrix has a determinant of zero. Indeed, one definition of a singular matrix is a matrix with a determinant of zero. Thus, ad = bc for every singular 2Ã—2 matrix.

To understand why a 2Ã—2 matrix must have Î” = 0, consider a matrix of letters with one column a scalar multiple of the other, i.e., linear dependence in the columns:

$$\begin{bmatrix} a & \lambda a \\ c & \lambda c \end{bmatrix}$$

And now we compute the determinant of this matrix:

$$\begin{aligned} \Delta &= ac\lambda - a\lambda c \\ &= 0 \end{aligned} \tag{11.5}$$

Thus, the determinant of a 2Ã—2 matrix with linear dependencies in the columns is necessarily 0. Take a moment to re-do this example but setting the linear dependency in the rows instead of the columns.

Practice problems Determine whether the following determinants are correct, and, if not, compute the true determinant.

a) $\begin{vmatrix} 3 & 1 \\ 2 & 1 \end{vmatrix} = 1$

b) $\begin{vmatrix} 3 & 4 \\ 1 & 1 \end{vmatrix} = -1$

c) $\begin{vmatrix} 1 & 9 \\ 6 & 1 \end{vmatrix} = -3$

d) $\begin{vmatrix} 9 & 1 \\ 6 & 1 \end{vmatrix} = -3$

e) $\begin{vmatrix} 1 & 9 \\ 0 & 0 \end{vmatrix} = 0$

f) $\begin{vmatrix} 1 & 5 \\ 3 & 15 \end{vmatrix} = 15$

g) $\begin{vmatrix} 1 & 5 \\ 3 & 0 \end{vmatrix} = -8$

h) $\begin{vmatrix} -1 & 2 \\ 4 & -5 \end{vmatrix} = -3$

Answers

a) Correct
b) Correct
c) No, Î” = −53
d) No, Î” = 3
e) Correct

f) No, Î” = 0
g) No, Î” = â^'15
h) Correct

Determinant and transpose The determinant is robust to the transpose operation. In other words, det(A) = det(A^T). This is because the determinant is a property of the matrix, not of the rows or the columns. This is easy to prove:

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix} = \text{ad â^' bc} \tag{11.6}$$

$$\begin{vmatrix} a & c \\ b & d \end{vmatrix} = \text{ad â^' cb} \tag{11.7}$$

Notice that the difference between the matrix and its transpose is simply the swapping of c and d. Scalar multiplication is commutative, and thus the determinant is unaffected by the transpose.

This proof is valid only for 2Ã—2 matrices because Equation 11.1 is a short-cut for 2Ã—2 matrices. However, this property does generalize to any sized square matrix. We will revisit this concept when learning about 3Ã—3 matrices.

11.3 The characteristic polynomial

An interpretation of Equation 11.1 is that we transform a matrix into an equation with the matrix elements on one side of the equation, and the determinant on the other side. Let me write that explicitly, using Î” to indicate the determinant:

$$ab - cd = \Delta \tag{11.8}$$

This is interesting because it means that we can solve for a particular matrix element if we know the determinant. Letâ€™s see an example:

$$\begin{vmatrix} 2 & 7 \\ 4 & \lambda \end{vmatrix} = 4 \quad \Rightarrow \quad 2\lambda - 28 = 4 \quad \Rightarrow \quad 2\lambda = 32 \quad \Rightarrow \quad \lambda = 16$$

So we start with the knowledge that the matrix determinant is 4 and then we can solve for a unique matrix element Î» that makes the equation true. Letâ€™s try another example:

$$\begin{vmatrix} \lambda & 1 \\ 3 & \lambda \end{vmatrix} = 1 \quad \Rightarrow \quad \lambda^2 - 3 = 1 \quad \Rightarrow \quad \lambda^2 = 4 \quad \Rightarrow \quad \lambda = \pm 2$$

Notice how the equation worked out: There were two Î»s on the diagonal, which produced a second-order polynomial equation, and thus there were two solutions. So in this case, there is no single Î» that satisfies our determinant equation; instead, there are two equally acceptable solutions. Thatâ€™s no surprise: The Fundamental Theorem of Algebra tells us that an nth order polynomial has exactly n roots (though they might be complex-valued).

Now I want to push this idea one step further. Instead of having only Î» on the diagonal of the matrix, letâ€™s have both a real number and an unknown. Iâ€™m also assuming that the matrix below is singular, which means I know that its determinant is zero.

$$\begin{vmatrix} 1-\lambda & 3 \\ 3 & 1-\lambda \end{vmatrix} = 0 \quad \text{â‡' } (1 \text{ â^' } \lambda)^2 \text{ â^' } 9 = 0 \tag{11.9}$$

$$\text{â‡' } \lambda^2 \text{ â^' } 2\lambda \text{ â^' } 8 = 0$$

$$\text{â‡' } (\lambda + 2)(\lambda \text{ â^' } 4) = 0$$

$$\text{â‡' } \lambda = \text{â^'2,4}$$

There are two possible values of Î», which means there are two distinct matrices that solve this equation. Those two matrices are shown below (obtained by plugging in Î»). It should take only a quick glance to confirm that both matrices have Î” = 0 (which is the same thing as saying that both matrices are reduced-rank).

$$\lambda = -2 \ddagger \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix}$$

$$\lambda = 4 \ddagger \begin{bmatrix} -3 & 3 \\ 3 & -3 \end{bmatrix}$$

Does subtracting a constant from the diagonal of a square matrix sound familiar? In fact, what we did in this example is shift the matrix by $-\lambda$I. Therefore, we could rewrite Equation 11.9 as

$$det\left(\begin{bmatrix} 1 & 3 \\ 3 & 1 \end{bmatrix} - \lambda\mathbf{I}\right) = 0 \tag{11.10}$$

Now I will replace that specific matrix with a letter so that it generalizes to any square matrix of any size. That gives us the characteristic polynomial of the matrix.

Eqn. box title=The characteristic polynomial of a matrix.

$$\det(\mathbf{A} - \lambda\mathbf{I}) \tag{11.11}$$

The characteristic polynomial is a big deal. For one thing, it allows us to represent a matrix in terms of a polynomial expression, and polynomials have a lot of great properties in mathematics.

But more importantly for our purposes in this book, when the characteristic polynomial is set to zero (that is, when we assume the determinant of the shifted matrix is 0), the λ's—the roots of the polynomial—are the eigenvalues of the matrix. Pretty neat, eh? More on this in Chapter 15. For now, we're going to continue our explorations of determinants of larger matrices.

Practice problems Which value(s) of λ would make the following matrices singular (that is, have a determinant of zero)?

a) $\begin{bmatrix} 1 & 2 \\ 4 & \lambda \end{bmatrix}$
b) $\begin{bmatrix} 0 & 14 \\ \lambda & 4 \end{bmatrix}$
c) $\begin{bmatrix} 4 & 1 \\ 1 & \lambda \end{bmatrix}$
d) $\begin{bmatrix} 10 & 1 \\ 3-\lambda & 1 \end{bmatrix}$
e) $\begin{bmatrix} 6 & 2 \\ 5 & 3\lambda \end{bmatrix}$
f) $\begin{bmatrix} \lambda & 4 \\ 1 & \lambda \end{bmatrix}$
g) $\begin{bmatrix} \lambda & 18 \\ 1/2 & \lambda \end{bmatrix}$
h) $\begin{bmatrix} 5-\lambda & -1/3 \\ -3 & 5-\lambda \end{bmatrix}$

Answers

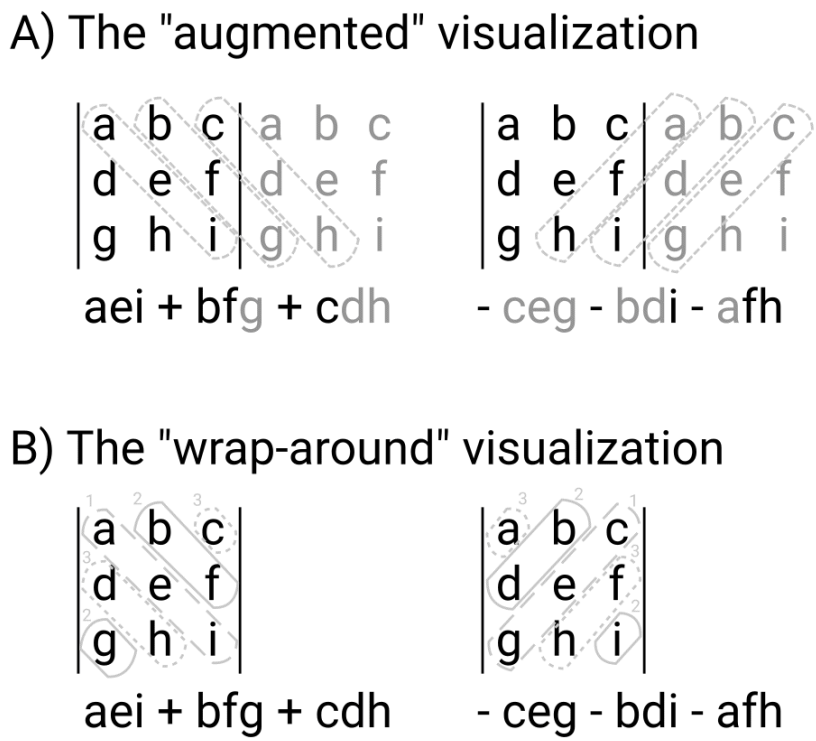
a) λ = 8
b) λ = 0
c) λ = 1, 4
d) λ = −7
e) λ = 5, 9

f) Î» = Â±2
g) Î» = Â±3
h) Î» = 6,4

11.4 Determinant of a 3Ã—3 matrix

There is also a short-cut to determine the determinant of a 3Ã—3 matrix, which involves adding the products of the right-going diagonals and subtracting the products of the left-going diagonals.

Itâ€™s easiest to understand visually. There are two ways to think about the procedure, one that "wraps around" the matrix and one that augments the matrix with the first two columns (Figure 11.1). Of course, these arenâ€™t really different methods, just different ways of interpreting the same procedure. Whichever picture you find easier to remember is the one you should focus on.



FigureÂ 11.1: Two visualizations of the short-cut for computing the determinant of a 3Ã—3 matrix.

I hope you find the pictures intuitive and memorable. Trying to memorize the algebraic equation is a terrible idea, but itâ€™s printed below for completeness.

Eqn. box title=Determinant of 3Ã—3 matrix.

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = \text{aei + bfg + cdh âˆ’ ceg âˆ’ bdi âˆ’ afh} \qquad (11.12)$$

Letâ€™s see a few examples with numbers.

$$\begin{vmatrix} 1 & 3 & 5 \\ 2 & 4 & 2 \\ 1 & 2 & 3 \end{vmatrix} = 1\cdot4\cdot3 + 3\cdot2\cdot1 + 5\cdot2\cdot2 - 5\cdot4\cdot1 - 3\cdot2\cdot3 - 1\cdot2\cdot2 = 38 - 44 = -4$$

$$\begin{vmatrix} 0 & 3 & -9 \\ -6 & 0 & 1 \\ 8 & 0 & 1 \end{vmatrix} = 0\cdot0\cdot1 + 3\cdot1\cdot8 + 9\cdot6\cdot0 - 9\cdot0\cdot8 + 3\cdot6\cdot1 - 0\cdot1\cdot0 = 24 + 18 = 42$$

$$\begin{vmatrix} 1 & 3 & 5 \\ 2 & 4 & 2 \\ 3 & 7 & 7 \end{vmatrix} = 1\cdot4\cdot7 + 3\cdot2\cdot3 + 5\cdot2\cdot7 - 5\cdot4\cdot3 - 3\cdot2\cdot7 - 1\cdot2\cdot7 = 116 - 116 = 0$$

The third example has a determinant of zero, which means the matrix is singular (reduced-rank). Itâ€™s singular because the columns (or rows) form a dependent set. Can you guess by visual inspection how the columns are linearly related to each other? It might be easier to see from looking at the rows.

Letâ€™s now demonstrate that the determinant of a 3Ã—3 reduced-rank matrix must be 0. Like with the example of the 2Ã—2 matrix, weâ€™ll accomplish this by creating a matrix with letters such that one column is a multiple of another column.

$$\begin{bmatrix} a & b & \lambda a \\ d & e & \lambda d \\ g & h & \lambda g \end{bmatrix}$$

The determinant, after some algebraic handiwork, is found to be zero (parentheses added for visual clarity).

Î” = aeÎ»g + bÎ»dg + Î»adh â€“ Î»aeg â€“ bdÎ»g â€“ aÎ»dh (11.13)
= (Î»aeg â€“ Î»aeg) + (Î»bdg â€“ Î»bdg) + (Î»adh â€“ Î»adh)
= 0

It is worth taking a few moments to go through this demonstration assuming a linear dependency in the rows.

Practice problems Determine whether the following determinants are correct.

a) $\begin{vmatrix} 1 & 1 & 2 \\ 2 & 2 & 1 \\ 4 & 3 & 0 \end{vmatrix}$ = â€“2

b) $\begin{vmatrix} 1 & 1 & 2 \\ 4 & 3 & 0 \\ 2 & 2 & 1 \end{vmatrix}$ = â€“3

c) $\begin{vmatrix} 1 & 2 & 4 \\ 1 & 2 & 3 \\ 2 & 1 & 0 \end{vmatrix}$ = â€“3

d) $\begin{vmatrix} 1 & -2 & 3 \\ -4 & 5 & -6 \\ 7 & -8 & 9 \end{vmatrix}$ = 8

Answers

a) No, Î” = â€“3
b) No, Î” = +3
c) Correct
d) No, Î” = 0

Practice problems Which value(s) of Î» would make the following matrices singular?

a) $\begin{bmatrix} 0 & 1 & 1 \\ 0 & 4 & 7 \\ 0 & 3 & \lambda \end{bmatrix}$

b) $\begin{bmatrix} 1 & 0 & -3 \\ -4 & -11 & 1 \\ 3 & \lambda & 0 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 3 \\ 3 & \lambda & 4 \end{bmatrix}$

d) $\begin{bmatrix} 1-\lambda & 0 & 4 \\ -4 & 3-\lambda & 4 \\ 0 & 0 & 5-\lambda \end{bmatrix}$

Answers

a) any Î»
b) Î» = 9
c) Î» = 3
d) Î» = 1,3,5

Determinant and transpose We can now show that the determinant of a 3Ã—3 matrix is the same before and after transposing. It takes slightly longer to prove than for the 2Ã—2 case, but is still worth the effort. The ordering of the terms is different, but the individual products are the same, as are their signs.

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = \text{aei + bfg + cdh â^’ ceg â^’ bdi â^’ afh} \tag{11.14}$$

$$\begin{vmatrix} a & d & g \\ b & e & h \\ c & f & i \end{vmatrix} = \text{aei + dhc + gbf â^’ gec â^’ dbi â^’ ahf} \tag{11.15}$$

11.5 Full procedure to compute the determinant

It turns out that the aforediscussed tricks for computing the determinants of 2Ã—2 and 3Ã—3 matrices are simplifications of the full procedure to compute the determinant of any sized matrix. In this section you will learn that full procedure. It gets really complicated really quickly, so I will illustrate it using a 4Ã—4 matrix, and then youâ€™ll see how this simplifies to the smaller matrices.

In general, the procedure is to multiply the ith element of the first row of the matrix by the determinant of the 3Ã—3 submatrix created by excluding the ith row and i^{th} column. That gives four numbers. You then add the 1st and 3rd, and subtract the 2nd and 4th. Figure 11.2 shows the operation.

$$\begin{vmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{vmatrix} = a \times \begin{vmatrix} f & g & h \\ j & k & l \\ n & o & p \end{vmatrix} - b \times \begin{vmatrix} e & g & h \\ i & k & l \\ m & o & p \end{vmatrix} + c \times \begin{vmatrix} e & f & h \\ i & j & l \\ m & n & p \end{vmatrix} - \times d \begin{vmatrix} e & f & g \\ i & j & k \\ m & n & o \end{vmatrix}$$

FigureÂ 11.2: The procedure for computing the determinant of a 4Ã—4 matrix. Note the alternating plus and minus signs.

Thus, computing the determinant of a 4Ã—4 matrix actually requires computing four 3Ã—3 determinants. Interestingly, you can modify this procedure to go column-wise rather than row-wise. [2] That is, multiply each

element of the first column by the determinant of the submatrix created by excluding that elementâ€™s row. Alternatively: Transpose the matrix and apply the procedure illustrated in Figure 11.2 .

[2] Note: There are some fancy linear algebra terms here like minors matrix and cofactors that will be introduced in the next chapter.

Now letâ€™s see what happens when we apply this same procedure to a 3Ã—3 matrix (Figure 11.3).

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix}$$

$$= a(ei-fh) - b(di-fg) + c(dh-eg)$$

$$= aei-afh - (bdi-dfg) + cdh-ceg$$

$$= aei + bfg + cdh - ceg - bdi - afh$$

FigureÂ 11.3: The procedure for computing the determinant of a 3Ã—3 matrix. Note the alternating plus and minus signs. And notice that the end result is the same as the shortcut in Equation 11.12 .

Now itâ€™s your turn: apply the full procedure to a 2Ã—2 matrix. (The determinant of a scalar is simply that scalar, thus, det(â^'4) = â^'4.) You will arrive at the familiar ad â^' bc.

The full procedure scales up to any sized matrix. But that means that computing the determinant of a 5Ã—5 matrix requires computing 5 determinants of 4Ã—4 submatrices, and each of those submatrices must be broken down into 3Ã—3 submatrices. Honestly, I could live 1000 years and die happy without ever computing the determinant of a 5Ã—5 matrix by hand. _____

Practice problems Compute the determinant of the following matrices (lots of zeros in these matrices... youâ€™re welcome).

a) $\begin{bmatrix} 0 & -1 & 2 & 0 \\ 1 & 2 & 0 & 5 \\ 0 & 3 & -4 & 0 \\ 1 & 0 & 2 & 0 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 0 & -3 & 3 \\ 6 & 0 & 0 & -1 \\ 0 & 0 & -4 & 0 \\ -1 & -1 & -3 & 2 \end{bmatrix}$

c) $\begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 2 & 0 & 5 \\ 0 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$

Answers

a) -10
b) 72
c) 0

11.6 Determinant of a triangular matrix

Hereâ€™s the key take-home of this section: The determinant of a triangular matrix is equal to the product of its diagonal elements.

There are three kinds of triangular matrices: upper-triangular, lower-triangular, and diagonal. Thus, computing the determinant of even a large triangular matrix is pretty easy.

In fact, for some matrices, itâ€™s easier to apply row reduction to get [3] the matrix into its echelon form and then compute the determinant as the product of the diagonal. HOWEVER, be aware that row exchanges and row-scalar-multiplication affect the determinant, as you will learn in subsequent sections.

[3] Useful tip for exams!

Why is this shortcut true? Letâ€™s start by proving it for upper-triangular 2Ã—2 matrices:

$$\begin{vmatrix} a & b \\ 0 & d \end{vmatrix} = ad - b0 = ad \tag{11.16}$$

You can now prove this for lower-triangular and diagonal matrices.

The proof for 3Ã—3 matrices is comparable but involves slightly more algebra. The expression below draws from Equation 11.12 but sets the lower-triangular elements to zero.

$$\begin{vmatrix} a & b & c \\ 0 & e & f \\ 0 & 0 & i \end{vmatrix} = aei + bf0 + c00 - ce0 - b0i - af0 = aei \tag{11.17}$$

Again, I will leave it to you to adapt Equation 11.17 to lower-triangular and diagonal matrices. You will discover that the result is the same: The determinant equals the product of the diagonals.

Finally, letâ€™s inspect the 4Ã—4 case. Figure 11.4 is a modified version of Figure 11.2 with e = i = f = m = n = o = 0.

$$\begin{vmatrix} a & b & c & d \\ 0 & f & g & h \\ 0 & 0 & k & l \\ 0 & 0 & 0 & p \end{vmatrix} = a \times \begin{vmatrix} f & g & h \\ 0 & k & l \\ 0 & 0 & p \end{vmatrix} - b \times \begin{vmatrix} 0 & g & h \\ 0 & k & l \\ 0 & 0 & p \end{vmatrix} + c \times \begin{vmatrix} 0 & f & h \\ 0 & 0 & l \\ 0 & 0 & p \end{vmatrix} - d \times \begin{vmatrix} 0 & f & g \\ 0 & 0 & k \\ 0 & 0 & 0 \end{vmatrix}$$

FigureÂ 11.4: The full determinant formula with an upper-triangular 4Ã—4 matrix.

The remarkable result here is that three of the four submatrices are singular (notice the columns of zeros). The only non-singular matrix is the first, which itself is a diagonal matrix, and which multiplies the first diagonal element of the original matrix. In other words, after following this rather complicated formula, the determinant of the matrix simplifies to Î” = afkp.

In mathematics, proofs donâ€™t necessarily lead to intuition. The reason why the determinant equals the product of the diagonals for any triangular matrix is this: All-but-one of the individual terms in the determinant formula have at least one element from the upper-triangle and at least one element from the lower-triangle. The one exception is the term defined by the main diagonal.

Code Computing the determinant is simple in Python and in MATLAB. However, the determinant is a numerically unstable computation for large matrices, which will be demonstrated in the code challenges.

Code blockÂ 11.1: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.random.randn(3,3)Â
np.linalg.det(A)
```

Code blockÂ 11.2: MATLAB

```
AÂ =Â randn(3,3);Â
det(A)
```

11.7 Determinant and row reduction

The procedures involved in row reduction (swapping rows, adding multiples of rows to other rows, and multiplying a row by a constant) have implications for the determinant of the matrix. Letâ€™s explore each of these three operations in turn.

Row swapping Curiously enough, swapping the rows of a matrix flips the sign of the determinant, without affecting the magnitude of the determinant. Letâ€™s start with a simple example of the identity matrix with and without rows swapped: [4]

[4] As sure as the sun rises, the determinant of the identity matrix of any size is 1.

$$\begin{vmatrix} 1 & 0 \\ 0 & 1 \end{vmatrix} = 1\text{Ã—}1 \text{ â^’ } 0\text{Ã—}0 = 1$$

$$\begin{vmatrix} 0 & 1 \\ 1 & 0 \end{vmatrix} = 0\text{Ã—}0 \text{ â^’ } 1\text{Ã—}1 = \text{â^’}1$$

Now letâ€™s try with a full matrix comprising letters.

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix} = \text{ad â^’ cb}$$

$$\begin{vmatrix} c & d \\ a & b \end{vmatrix} = \text{cb â^’ da} = \text{â^’(ab â^’ cd)}$$

With a 2Ã—2 matrix, there is only one possible way to swap rows. Letâ€™s try this again with the 3Ã—3 identity matrix. We can see what happens with multiple row swaps.

$$\begin{vmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix} = \text{1â‹…1â‹…1 + 0â‹…0â‹…0 + 0â‹…0â‹…0 â^’ 0â‹…1â‹…0 â^’ 0â‹…0â‹…1 â^’ 1â‹…0â‹…0 = 1}$$

$$\begin{vmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{vmatrix} = \text{0â‹…0â‹…1 + 1â‹…0â‹…0 + 0â‹…1â‹…0 â^’ 0â‹…0â‹…0 â^’ 1â‹…1â‹…1 â^’ 0â‹…0â‹…0 = â^’1}$$

$$\begin{vmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{vmatrix} = \text{0â‹…0â‹…0 + 1â‹…1â‹…1 + 0â‹…0â‹…0 â^’ 0â‹…0â‹…1 â^’ 1â‹…0â‹…0 â^’ 0â‹…1â‹…0 = 1}$$

Thus, each row swap reverses the sign of the determinant. Two row swaps therefore "double-reverses" the sign. More generally, an odd number of row swaps effectively multiplies the sign of the determinant by â^’1, whereas an even number of row swaps preserves the sign of the determinant.

One more example with a matrix of letters.

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = \text{aei + bfg + cdh â^’ ceg â^’ bdi â^’ afh} \qquad (11.18)$$

$$\begin{vmatrix} d & e & f \\ a & b & c \\ g & h & i \end{vmatrix} = \text{bdi + ceg + afh â^’ bfg â^’ aei â^’ cdh} \qquad (11.19)$$

$$= \text{â^’(aei + bfg + cdh â^’ ceg â^’ bdi â^’ afh)}$$

$$\begin{vmatrix} d & e & f \\ g & h & i \\ a & b & c \end{vmatrix} = \text{cdh + aei + bfg â^’ afh â^’ ceg â^’ bdi} \qquad (11.20)$$

Adding multiples of rows Adding a multiple of one row to another is the most common operation in row reduction. Letâ€™s see what happens when we replace row 1 by row 1 plus row 2 (spoiler alert: the determinant is unchanged):

$$\begin{vmatrix} a+c & b+d \\ c & d \end{vmatrix} = \text{(a + c)d â^’ (b + d)c}$$

$$= \text{ad + cd â^’ (bc + dc)}$$

$$= \text{ad â^’ bc} \qquad (11.21)$$

Letâ€™s try this again with a 3Ã—3 matrix to make sure our conclusion isnâ€™t based on a quirk of the 2Ã—2 case:

$$\begin{vmatrix} a+g & b+h & c+i \\ d & e & f \\ g & h & i \end{vmatrix}$$

= (a + g)ei + (b + h)fg + (c + i)dh â^' (c + i)eg â^' (b + h)di â^' (a + g)fh

= aei + gei + bfg + hfg + cdh + idh â^' ceg â^' ieg â^' bdi â^' hdi â^' afh â^' gfh

= aei + bfg + cdh â^' ceg â^' bdi â^' afh (11.22)

Itâ€™s not immediately visually obvious that the "extra" terms will cancel, but notice that there are identical terms with opposite signs and with letters in a different order (for example: +gei and â^'ieg). Canceling those terms will bring us back to the familiar determinant formula. Itâ€™s as we never added any rows in the first place.

Now you know that the determinant is unaffected by adding rows to each other. Why does this happen? In Equation 11.21 , the copy of row 2 embedded in row 1 adds, and then subtracts, the added elements (+cd and â^'dc). Same story for Equation 11.22 : The "extra terms" come from the products of the right-going diagonals with the added elements (gei, hfg, idh) (consult back to Figure 11.1 on page Â§), and then those same "extra terms" are subtracted off from the left-going diagonals (ieg, hdi, gfh).

The previous paragraph will provide an answer to the next question: What about adding multiples of one row to another? In the two examples above, I only showed a row being added to another row, but row reduction often involves scalar-multiplying rows and adding them. It turns out that scalar multiplying does not change our conclusion: combining rows has no effect on the determinant. The reason is that the multiple is simply a different constant that gets added and then subtracted to the determinant formula.

Letâ€™s think through this using example 11.22 . Instead of adding row 3 to row 1, letâ€™s add 4 times row 3 to row 1. The first row of the new matrix then becomes [(a + 4g) (b + 4h) (c + 4i)]. Well, 4g is still just a constant, so we might as well give it a new variable name gâ€². Thus, the first element becomes (a + gâ€²). Same for the other two elements of the first row: (b + hâ€²) and (c + iâ€²). Then the opposite-signed terms in Equation 11.22 are, for example, gâ€²ei and â^'iegâ€². Those still cancel out.

Finally, we can address subtracting rows: Subtracting rows is the same thing as adding rows with one row scalar-multiplied by â^'1. For this reason, subtracting rows has the same impact on the determinant as adding rows: None whatsoever.

Row-scalar multiplication What happens to the determinant when you multiply an entire row by a scalar? Notice the difference from the discussion above: Here we are concerned with multiplying only one row by a scalar; we are not combining multiple rows by addition or subtraction.

The answer is that multiplying a row by some scalar Î² scales the determinant by Î². Letâ€™s see an example in our favorite lettered 2Ã—2 matrix.

$$\begin{vmatrix} \beta a & \beta b \\ c & d \end{vmatrix} = \beta ad - \beta bc = \beta(ad - bc) \quad (11.23)$$

How does this generalize to scaling rows by different values? Letâ€™s find out:

$$\begin{vmatrix} \beta a & \beta b \\ \gamma c & \gamma d \end{vmatrix} = \beta a \gamma d - \beta b \gamma c = \beta\gamma(ad - bc) \quad (11.24)$$

In other words, any scalar that multiplies an entire row, scales the determinant by the same amount.

This scales up to any sized matrix, which is easy to confirm in the 3Ã—3 case:

$$\begin{vmatrix} a & b & c \\ d & e & f \\ \zeta g & \zeta h & \zeta i \end{vmatrix}$$ = aeÎ¶i + bfÎ¶g + cdÎ¶h â^' ceÎ¶g â^' bdÎ¶i â^' afÎ¶h

= Î¶(aei + bfg + cdh â^' ceg â^' bdi â^' afh) (11.25)

The reason why this happens is that each term in the determinant formula contains exactly one element from each row in the matrix. Therefore, multiplying an entire row by a scalar means that every term in the formula contains

that scalar, and it can thus be factored out.

Summary Following is the high-level summary of this section; itâ€™s really easy to memorize, but of course you needed the previous pages to explain why these are true.

$$\begin{aligned} \text{Row swap} &\rightarrow \det = -\det \\ \text{Add rows} &\rightarrow \det = \det \\ \text{Scale row by } \beta &\rightarrow \det = \det\beta \end{aligned}$$

11.8 Determinant & matrix-scalar multiplication

The previous section might have led you to wonder what happens to the determinant of a scalar-multiplication over the entire matrix. That is, what is the relationship between |A| and |Î²A|?

One way to think about this is that matrix-scalar multiplication is the same thing as row-scalar multiplication, repeated for all rows and using the same scalar. So if multiplying one row by a scalar Î² scales the determinant by Î², then multiplying two rows by Î² scales the determinant by Î²Î² = Î²^2 . Generalizing this to all M rows of a matrix leads to the following formula:

$$det(\beta\mathbf{A}) = \beta^M det(\mathbf{A}), \quad \mathbf{A} \in \mathbb{R}^{M\times M} \tag{11.26}$$

It is also insightful to think about this from an element perspective of matrix-scalar multiplication.

$$\beta\begin{vmatrix} a & b \\ c & d \end{vmatrix} = \begin{vmatrix} \beta a & \beta b \\ \beta c & \beta d \end{vmatrix} = \beta a\beta d - \beta b\beta c = \beta^2 (ad - bc) \tag{11.27}$$

You can see that each term in the determinant formula brings Î²^2 , which can be factored out. Letâ€™s see how this looks for a 3Ã—3 matrix.

$$\begin{aligned} \beta\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} &= \begin{vmatrix} \beta a & \beta b & \beta c \\ \beta d & \beta e & \beta f \\ \beta g & \beta h & \beta i \end{vmatrix} \\ &= \beta^3 aei + \beta^3 bfg + \beta^3 cdh - \beta^3 ceg - \beta^3 bdi - \beta^3 afh \\ &= \beta^3 (aei + bfg + cdh - ceg - bdi - afh) \end{aligned} \tag{11.28}$$

You can consider how this works for the 4Ã—4 case (refer to Figure 11.2): The determinant of each submatrix has a scalar of Î²^3 , which then multiplies Î² times the element in the first row, leading to Î²^4 .

Generalizing this to higher dimensions leads back to Equation 11.26 .

11.9 Determinant in theory and in practice

As mentioned at the outset of this chapter, the determinant has several applications. Two of the most important applicationsâ€”and the primary applications in this bookâ€”are computing the matrix inverse, and discovering the eigenvalues. Clearly, the determinant has an important role in linear algebra.

On the other hand, the determinant is difficult to compute for "large" matrices, due to numerical instabilities. Numerical instabilities in computers result from multiplying or dividing by very very small numbers, or very very large numbers. This is particularly problematic when the matrices have a high condition number (a concept you will learn about in Chapter 16 on the singular value decomposition), are close to singular, or are singular.

You will have the opportunity to examine this yourself in the code challenges at the end of this chapter. The main point is that the determinant is an important concept to understand for the theory of linear algebra, but you should avoid computing or using it directly when implementing linear algebra concepts on computers.

11.10 Exercises

1.Â

Compute the determinant of the following matrices.

a) $\begin{bmatrix} a & a \\ b & c \end{bmatrix}$

b) $\begin{bmatrix} 5 & 2 \\ 10 & 4 \end{bmatrix}$

c) $\begin{bmatrix} 5 & 10 \\ 2 & 4 \end{bmatrix}$

d) $\begin{bmatrix} 50 & 7 \\ 7 & 1 \end{bmatrix}$

e) $\begin{bmatrix} 2 & \sqrt{e^{-\pi/314}} \\ 0 & 8 \end{bmatrix}$

f) $\begin{bmatrix} -2 & 6 \\ -4 & 11 \end{bmatrix}$

g) $\begin{bmatrix} 100 & \sqrt{599} \\ \sqrt{599} & 6 \end{bmatrix}$

h) $\begin{bmatrix} 13 & 34 \\ 5 & 13 \end{bmatrix}$

i) $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 13 & 34 \\ 0 & 5 & 13 \end{bmatrix}$

j) $\begin{bmatrix} 1 & 1 & 0 \\ 6 & -5 & 4 \\ 2 & 0 & -9 \end{bmatrix}$

k) $\begin{bmatrix} 0 & 2 & 3 \\ 1 & -78 & 17 \\ 0 & 1 & 2 \end{bmatrix}$

l) $\begin{bmatrix} 1 & -78 & 17 \\ 0 & 2 & 3 \\ 0 & 1 & 2 \end{bmatrix}$

m) $\begin{bmatrix} 0 & 1 & 2 \\ 1 & -78 & 17 \\ 0 & 2 & 3 \end{bmatrix}$

n) $\begin{bmatrix} 1 & -2 & 3 \\ -4 & 5 & -6 \\ 7 & -8 & 9 \end{bmatrix}$

o) $\begin{bmatrix} 0 & 1 & 2 & 0 \\ -4 & 0 & 3 & 1 \\ 3 & -3 & 0 & 0 \\ 6 & 0 & -2 & -2 \end{bmatrix}$

2.Â

Apply row-reduction to obtain the echelon form of these matrices, and then compute the determinant as the product of the diagonal. Keep track of row-swaps and row-scalar multiplications.

a) $\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$

b) $\begin{bmatrix} 3 & 4 \\ 1 & 3 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 3 & -2 \\ 3 & 5 & 6 \\ 2 & 4 & 3 \end{bmatrix}$

d) $\begin{bmatrix} 0 & 2 & -4 \\ 3 & -2 & 7 \\ 3 & 2 & -1 \end{bmatrix}$

3.Â
Does having a zero in the main diagonal necessarily give a matrix Î” = 0? The correct answer is "it depends"; now you need to figure out what it depends on, and why that is. Then create an example matrix with all zeros on the diagonal but with Î”â‰ 0.

11.11 Answers

1.Â
a) a(c âˆ’ b)
b) 0
c) 0
d) 1
e) 16
f) 2
g) 1
h) âˆ’1
i) âˆ’1
j) 107
k) âˆ’1
l) +1
m) 1
n) 0
o) âˆ’24

2.Â
These answers show the echelon form and the determinant of the original matrix. Notice that the determinant of the original matrix is not necessarily the same thing as the the product of the diagonals of the echelon form of the matrix (depending on row manipulations).

a) $\begin{bmatrix} 2 & 3 \\ 0 & -1 \end{bmatrix}$,Î” = âˆ’2
b) $\begin{bmatrix} 1 & 3 \\ 0 & -5 \end{bmatrix}$,Î” = 5
c) $\begin{bmatrix} 1 & 3 & -2 \\ 0 & -2 & -6 \\ 0 & 0 & 1 \end{bmatrix}$,Î” = âˆ’4
d) $\begin{bmatrix} 3 & 2 & -1 \\ 0 & -4 & 8 \\ 0 & 0 & 0 \end{bmatrix}$,Î” = 0

3.Â
The determinant of a matrix is zero if at least one diagonal element is zero only if the matrix is triangular (upper-triangular, lower-triangular, or diagonal). This is because the determinant of a triangular matrix is equal to the product of the diagonal elements. Here is an example of a matrix with a diagonal of zeros and Î” = 4:

$$\begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 2 & 1 & 0 \end{bmatrix}$$

11.12 Code challenges

1.Â
Write code that illustrates Equation 11.26 . Your code should generate a 4Ã—4 matrix comprising integers between 0 and 10, then generate a random integer Î²² between -10 and -1, and then print out the left- and right-hand sides of Equation 11.26 .

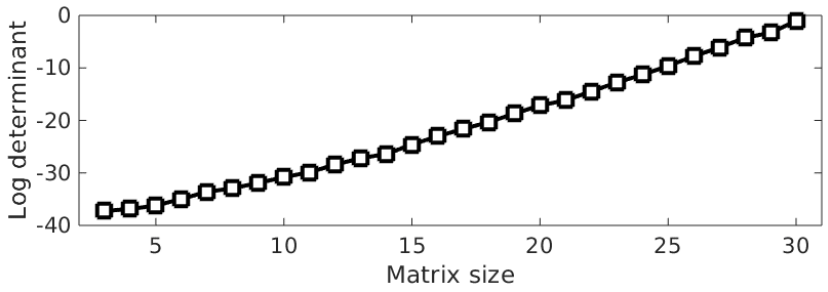
2.Â

We will now demonstrate the numerical instabilities of the determinant. Implement the following in code:

1. Create a matrix of normally distributed random numbers.
2. Ensure that the matrix is reduced-rank.
3. Compute the absolute value of the determinant (we are interested in whether the determinant deviates from zero; the sign doesnâ€™t matter).

Run these three steps in a double for-loop: One over matrix sizes ranging from 3Ã—3 to 30Ã—30, and a second that repeats the three steps 100 times. This is equivalent to repeating a scientific experiment multiple times.

Finally, make a plot of the average determinant as a function of the matrix size. In theory, all of these determinants should be zero! The figure below shows my result. I show the y-axis in log-scale because the determinants grow exponentially.



figureAverage determinant magnitudes of singular matrices of different sizes.

11.13 Code solutions

1.Â

Note that upper bounds in Python are usually (though not always) specified as exclusive, meaning that an upper bound parameter of "11" will produce random numbers up to 10.

Code blockÂ 11.3: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.random.randint(0,11,(4,4))Â
bÂ =Â np.random.randint(-10,1)Â
print(np.linalg.det(b*A),Â
Â Â Â Â Â Â b**4*np.linalg.det(A))
```

Code blockÂ 11.4: MATLAB

```
AÂ =Â randi([0Â 10],4);Â
bÂ =Â randi([-10Â -1],1);Â
[det(b*A)Â b^4*det(A)]
```

2.Â

One easy way to create a singular matrix is to set column 1 equal to column 2.

Code blockÂ 11.5: Python

```
importÂ numpyÂ asÂ npÂ
importÂ matplotlib.pyplotÂ asÂ pltÂ
Â
nsÂ =Â np.arange(3,31)Â
itersÂ =Â 100Â
detsÂ =Â np.zeros((len(ns),iters))Â
Â
forÂ niÂ inÂ range(len(ns)):Â
Â Â forÂ iÂ inÂ range(iters):Â
Â Â Â Â AÂ =Â np.random.randn(ns[ni],ns[ni])#step1Â
```

```
Â Â Â Â A[:,0]Â =Â A[:,1]Â #Â stepÂ 2Â
Â Â Â Â dets[ni,i]=np.abs(np.linalg.det(A))#st3Â
Â
plt.plot(ns,np.log(np.mean(dets,axis=1)))Â
plt.xlabel(â€™MatrixÂ sizeâ€™)Â
plt.ylabel(â€™LogÂ determinantâ€™);
```

Code blockÂ 11.6: MATLAB

```
nsÂ =Â 3:30;Â
itersÂ =Â 100;Â
detsÂ =Â zeros(length(ns),iters);Â
Â
forÂ ni=1:length(ns)Â
Â Â Â Â forÂ i=1:itersÂ
Â Â Â Â Â Â Â Â AÂ =Â randn(ns(ni));Â %Â stepÂ 1Â
Â Â Â Â Â Â Â Â A(:,1)Â =Â A(:,2);Â %Â stepÂ 2Â
Â Â Â Â Â Â Â Â dets(ni,i)Â =Â abs(det(A));Â %Â stepÂ 3Â
Â Â Â Â endÂ
endÂ
Â
plot(ns,log(mean(dets,2)),â€™s-â€™)Â
xlabel(â€™MatrixÂ sizeâ€™)Â
ylabel(â€™LogÂ determinantâ€™)
```

ChapterÂ 12
Matrix inverse

As usual, I will begin this chapter with some general conceptual introductions, and then weâ€™ll get into the details.

12.1 Concepts and applications

In section 6.11 I wrote that matrix division per se doesnâ€™t exist, however, there is a conceptually similar operation, which involves multiplying a matrix by its inverse. By way of introduction to the matrix inverse, Iâ€™m going to start with the "scalar inverse." Solve for x in this equation.

$$3x = 1 \tag{12.1}$$

Obviously, x = 1âˆ•3. How did you solve this equation? I guess you divided both sides of the equation by 3. But let me write this in a slightly different way:

$$\begin{aligned} 3x &= 1 \\ 3^{âˆ’1}\,3x &= 3^{âˆ’1}\,1 \\ 1x &= 3^{âˆ’1} \\ x &= \frac{1}{3} \end{aligned}$$

I realize that this is a gratuitously excessive number of steps to write out explicitly, but it does illustrate a point: To separate the 3 from the x, we multiplied by $3^{âˆ’1}$, which we might call the "inverse of 3." And of course, we had to do this to both sides of the equation. Thus, 3Ã—$3^{âˆ’1}$ = 1. A number times its inverse is 1. The number 1 is special because it is the multiplicative identity.

What about the following equation; can you solve for x here?

$$0x = 1$$

Nope, you cannot solve for x here, because you cannot compute 0/0. There is no number that can multiply 0 to give 1. The lesson here is that not all numbers have inverses.

OK, now letâ€™s talk about the matrix inverse. The matrix inverse is a matrix that multiplies another matrix such that the product is the identity matrix. The identity matrix is important because it is the matrix analog of the number 1â€”the multiplicative identity (AI = A). Here it is in math notation:

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{I} \tag{12.2}$$

Now let me show you an example of using the matrix inverse. In the equations below, imagine that A and b are known and x is the vector of unknowns that we want to solve for.

$$\mathrm{Ax} = \mathrm{b} \tag{12.3}$$

$$\mathrm{A^{âˆ’1}\,Ax} = \mathrm{A^{âˆ’1}\,b} \tag{12.4}$$

$$\mathrm{Ix} = \mathrm{A^{âˆ’1}\,b} \tag{12.5}$$

$$\mathrm{x} = \mathrm{A^{âˆ’1}\,b} \tag{12.6}$$

IMPORTANT:[1] Because matrix multiplication is non-commutative (that is, ABâ‰ BA), you need to be mindful to multiply both sides of the equation by the matrix inverse on the same side. For example, the following equation is invalid:

[1] This was already mentioned in section 6.2 but itâ€™s important enough to repeat.

$$\mathbf{A}^{-1}\mathbf{Ax} = \mathbf{bA}^{-1} \tag{12.7}$$

This equation is WRONG because the inverse pre-multiplies on the left-hand side of the equation but post-multiplies on the right-hand side of the equation. As it turns out, post-multiplying by $\mathrm{A^{âˆ’1}}$:

$$\mathbf{AxA}^{-1} = \mathbf{bA}^{-1} \tag{12.8}$$

is invalid for this equation, because both Ax and b are column vectors. Thus, the sizes do not permit matrix post-multiplication.

Inverting the inverse Because the inverse is unique (youâ€™ll learn the proof of this claim later), it can be undone. Thus, the inverse of an inverse is the original matrix. That is:

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A} \tag{12.9}$$

Transpose and inverse Equation 12.9 is reminiscent of double-transposing a matrix ($\mathrm{A^{TT}} = \mathrm{A}$). Although the transpose and inverse are completely different operations, they do have a special relationship: The transpose of the inverse equals the inverse of the transpose.

$$(\mathbf{A}^{-1})^{\mathrm{T}} = (\mathbf{A}^{\mathrm{T}})^{-1} = \mathbf{A}^{-\mathrm{T}} \tag{12.10}$$

Actually, what I wrote above ("transpose and inverse are completely different") is true only for most matrices. There is a special kind of matrix, called an orthogonal matrix, for which the inverse equals the transpose. More about this in the next chapter!

Conditions for invertibility You saw above that not all numbers have an inverse. Not all matrices have inverses either. In fact, many (or perhaps most) matrices

that you will work with in practical applications are not invertible. Remember that square matrices without an inverse are variously called singular, reduced-rank, or rank-deficient.

A matrix has a full inverse matrix if the following criteria are met:

1. It is square
2. It is full-rank

So the full matrix inverse exists only for square matrices. What does a "full" matrix inverse mean? It means you can put the inverse on either side of the matrix and still get the identity matrix:

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I} \tag{12.11}$$

Thus, the full matrix inverse is one of the few exceptions to matrix multiplication commutivity.

Some rectangular matrices have a "one-sided" inverse, if certain conditions are met. One-sided inverses are non-commutative, for example AA$^{\hat{a}\tilde{}'1}$ = I but A$^{\hat{a}\tilde{}'1}$ Aâ‰ I. For this reason, the "full inverse" is also sometimes called the "two-sided inverse." Youâ€™ll learn about one-sided inverses starting on page Â§ ; until then, please assume that all matrices are square.

Remember to LIVE EVIL As a quick reminder:

$$(\mathbf{ABC})^{-1} = \mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1} \tag{12.12}$$

However, this is not as simple as it sounds: It is possible for the matrix product (ABC) to be invertible while the individual matrices are not invertible.

Uniqueness of the matrix inverse Every inverse is unique, meaning that if a matrix has an inverse, it has exactly one inverse. Iâ€™ll show two proofs for this claim; both proofs rely on the same assumptions, and both work by contradiction. The assumptions are:

1. Matrix A is invertible.
2. B is an inverse of A.
3. C is also an inverse of A, distinct from B (thus, Bâ‰ C).

Assumptions 2 and 3 can be written out mathematically:

$$\mathrm{AB} = \mathrm{I} \tag{12.13}$$

$$\mathrm{AC} = \mathrm{I} \tag{12.14}$$

Next Iâ€™m going to pre-multiply both equations by A$^{\hat{a}\tilde{}'1}$:

$$A^{\hat{}\text{'}1}AB = A^{\hat{}\text{'}1}I\hat{a}\ddagger\text{'}B = A^{\hat{}\text{'}1} \quad (12.15)$$

$$A^{\hat{}\text{'}1}AC = A^{\hat{}\text{'}1}I\hat{a}\ddagger\text{'}C = A^{\hat{}\text{'}1} \quad (12.16)$$

From the equations to the right of the arrow, we must conclude that B = C. However, our assumption was that Bâ‰ C. The conclusion was inconsistent with the assumption, therefore the assumption is wrong. This proves that all inverses of a given matrix are equal.

The second proof is shorter; notice that each subsequent expression is based on adding or removing the identity matrix, expressed as the matrix times its inverse:

$$\mathbf{C = CI = CAB = IB = B} \quad (12.17)$$

Again, the conclusion is that any two matrices that claim to be the inverse of the same matrix must be equal to each other.

Inverse of a symmetric matrix The inverse of a symmetric matrix is itself symmetric. That is, if $A = A^T$ then $A^{\hat{}\text{'}1} = A^{\hat{}\text{'}T}$.

To prove this claim, we start from two assumptions: That the matrix A is symmetric, and that it has an inverse. The strategy here is to transpose the inverse equation and then do a bit of algebra. Hereâ€™s math tip: Write down the first equation, close the book, and see if you can discover the proof on your own.

$$A^{\hat{}\text{'}1}A = I \quad (12.18)$$

$$(A^{\hat{}\text{'}1}A)^T = I^T \quad (12.19)$$

$$A^T A^{\hat{}\text{'}T} = I \quad (12.20)$$

$$AA^{\hat{}\text{'}T} = I \quad (12.21)$$

$$AA^{\hat{}\text{'}T} = AA^{\hat{}\text{'}1} \quad (12.22)$$

In the previous page, we proved that if a matrix has an inverse, it has one unique inverse. Therefore, Equation 12.22 brings us to our final conclusion that if the matrix is symmetric, its inverse is also symmetric.

Avoid the inverse when possible! The last thing I want to discuss before teaching you how to compute the matrix inverse is that the matrix inverse is great in theory. When doing abstract paper-and-pencil work, you can invert matrices as much as you want, regardless of their size and content (assuming they are square and full-rank). But in practice, computing the inverse of a matrix on a computer is difficult and can be wrought with numerical inaccuracies and rounding errors.

Thus, in practical computer applications of linear algebra, you should avoid using the explicit inverse unless it is absolutely necessary.

Computer scientists have worked hard over the past several decades to develop algorithms to solve problems thatâ€”on paperâ€”require the inverse, without actually computing the inverse. The details of those algorithms are beyond the scope of this book. Fortunately, they are implemented in low-level libraries called by MATLAB, Python, C, and other numerical processing languages. This is good news, because it allows you to focus on understanding the conceptual aspects of the inverse, while letting your computer deal with the number-crunching.

Computing the matrix inverse There are several algorithms to compute the matrix inverse. In this book, you will learn three: MCA (minors, cofactors, adjugate), row-reduction, and SVD. You will learn about the first two algorithms in this chapter, and youâ€™ll learn about the SVD method in Chapter 16. But there are convenient short-cuts for computing the inverses of a diagonal matrix and a 2Ã—2 matrix, and thatâ€™s where weâ€™ll start.

12.2 Matrix inverse of a diagonal matrix

Diagonal matrices have an extremely easy-to-compute inverse: Simply invert each diagonal element and ignore the off-diagonal zeros.

$$\mathrm{A} = \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & c \end{bmatrix} \tag{12.23}$$

$$\mathrm{A}^{\hat{}\text{'}1} = \begin{bmatrix} 1/a & 0 & 0 \\ 0 & 1/b & 0 \\ 0 & 0 & 1/c \end{bmatrix} \tag{12.24}$$

$$\mathrm{A}\mathrm{A}^{\hat{}\text{'}1} = \mathrm{A}^{\hat{}\text{'}1}\,\mathrm{A} = \begin{bmatrix} a/a & 0 & 0 \\ 0 & b/b & 0 \\ 0 & 0 & c/c \end{bmatrix} \tag{12.25}$$

This example is for a 3Ã—3 matrix for visualization, but the principle holds for any number of dimensions.

This inverse procedure also shows one reason why singular matrices are not invertible: A singular diagonal matrix has at least one diagonal element equal to zero. If you try to apply the above short-cut, youâ€™ll end up with an element of 0/0.

The universe would be a lovely placeâ€”and linear algebra would be a much easier subjectâ€”if all matrix inverses were as easy and intuitive to compute as the short-cut for diagonal matrices (invert each matrix element). However, it takes

only one example to show that this doesnâ€™t work when there is a single off-diagonal non-zero element. We start with an almost-diagonal matrix A, and another matrix $\tilde{A}f$ that is obtained by reciprocating each non-zero matrix element.

$$\mathbf{A} = \begin{bmatrix} 3 & 0 & 2 \\ 0 & 1/2 & 0 \\ 0 & 0 & 4 \end{bmatrix}, \qquad \tilde{\mathbf{A}} = \begin{bmatrix} 1/3 & 0 & 1/2 \\ 0 & 2 & 0 \\ 0 & 0 & 1/4 \end{bmatrix}$$

$$A\tilde{A}f = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{12.26}$$

$$\tilde{A}fA = \begin{bmatrix} 1 & 0 & 8/3 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{12.27}$$

You can see that $\tilde{A}f$ is definitely not the inverse of A, because their product is not the identity matrix.

Before learning the full formula for computing the matrix inverse, letâ€™s spend some time learning another short-cut for the inverse that works on 2Ã—2 matrices.

12.3 Matrix inverse for a 2Ã—2 matrix

The famous shortcut for computing the inverse of a 2Ã—2 matrix has four steps:

1. Compute the determinant and check whether Î” = 0.
2. Swap the diagonal elements.
3. Multiply the off-diagonal elements by -1.
4. Divide all matrix elements by Î”.

The reason why you start the procedure by computing the determinant is that the matrix has no inverse if the determinant is zero. Thus, if step 1 gives an answer of zero, then you donâ€™t need to apply the remaining steps. Youâ€™ll see an example of this soon, but letâ€™s start with an invertible matrix.

$$\begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}^{-1} = \frac{1}{-1} \begin{bmatrix} 3 & -2 \\ -2 & 1 \end{bmatrix} = \begin{bmatrix} -3 & 2 \\ 2 & -1 \end{bmatrix}$$

Multiplying the right-most and left-most matrices above will prove that one is the inverse of the other. As I wrote earlier, a square invertible matrix has a full inverse, meaning the order of multiplication doesnâ€™t matter.

$$\begin{bmatrix}1 & 2\\ 2 & 3\end{bmatrix}\begin{bmatrix}-3 & 2\\ 2 & -1\end{bmatrix}=\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$$
$$\begin{bmatrix}-3 & 2\\ 2 & -1\end{bmatrix}\begin{bmatrix}1 & 2\\ 2 & 3\end{bmatrix}=\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$$

Here is the general formula for the 2Ã—2 inverse.

–

Eqn. box title=Formula for the inverse of a 2Ã—2 matrix

$$\begin{bmatrix}a & b\\ c & d\end{bmatrix}=\frac{1}{ad-bc}\begin{bmatrix}d & -b\\ -c & a\end{bmatrix} \qquad (12.28)$$

Now watch what happens when we try to invert a rank-1 matrix.

$$\begin{bmatrix}1 & 2\\ 2 & 4\end{bmatrix}=\frac{1}{4-4}\begin{bmatrix} & \\ & \end{bmatrix}$$

You can stop the computation as soon as you see that the determinant, which goes into the denominator, is zero. This is one explanation for why a singular matrix (with Î” = 0) has no inverse.

Below are two more examples of matrices and their inverses, and a check that the multiplication of the two yields I.

$$\begin{bmatrix}3 & 1\\ 2 & 1\end{bmatrix}^{-1}=\frac{1}{1}\begin{bmatrix}1 & -1\\ -2 & 3\end{bmatrix} \quad\Rightarrow\quad \begin{bmatrix}3 & 1\\ 2 & 1\end{bmatrix}\begin{bmatrix}1 & -1\\ -2 & 3\end{bmatrix}=\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$$

[2] $\begin{bmatrix}2 & 7\\ 3 & -8\end{bmatrix}$ â^'1 = $\frac{1}{-37}\begin{bmatrix}-8 & -7\\ 3 & 2\end{bmatrix}$ â‡' $\frac{1}{-37}\begin{bmatrix}-8 & -7\\ 3 & 2\end{bmatrix}\begin{bmatrix}2 & 7\\ 3 & -8\end{bmatrix}=\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}$

---

[2] Hint: Sometimes itâ€™s easier to leave the determinant as a scalar, rather than to divide each element.

Practice problems Compute the inverse (if it exists) of the following matrices.

a) $\begin{bmatrix}1 & 4\\ 4 & 4\end{bmatrix}$

b) $\begin{bmatrix}2 & 1/2\\ 3 & 1/3\end{bmatrix}$

c)

$\begin{bmatrix} 2 & 1/2 \\ 3 & 3/4 \end{bmatrix}$

d) $\begin{bmatrix} -9 & 3 \\ -9 & -8 \end{bmatrix}$

Answers

a) $\frac{1}{3}\begin{bmatrix} -1 & 1 \\ 1 & -1/4 \end{bmatrix}$

b) $\frac{-1}{6/5}\begin{bmatrix} 1/3 & -1/2 \\ -3 & 2 \end{bmatrix}$

c) No inverse.

d) $\frac{1}{99}\begin{bmatrix} -8 & -3 \\ 9 & -9 \end{bmatrix}$

_

12.4 The MCA algorithm

The short-cut for a 2Ã—2 matrix above is actually just a special case of the MCA algorithm. This full procedure for computing the matrix inverse is not really difficult, but it is rather time-consuming. There are three steps.

[3] M: Compute the minors matrix, a matrix of determinants of submatrices.

[3] The MCA algorithm bears no explicit connection to the late-and-great Adam Yauch .

C: Compute the cofactors matrix, the Hadamard multiplication of the minors matrix with an alternating grid of +1 and â^'1.

A: Compute the adjugate matrix, the transpose of the cofactors matrix, divided by the determinant.

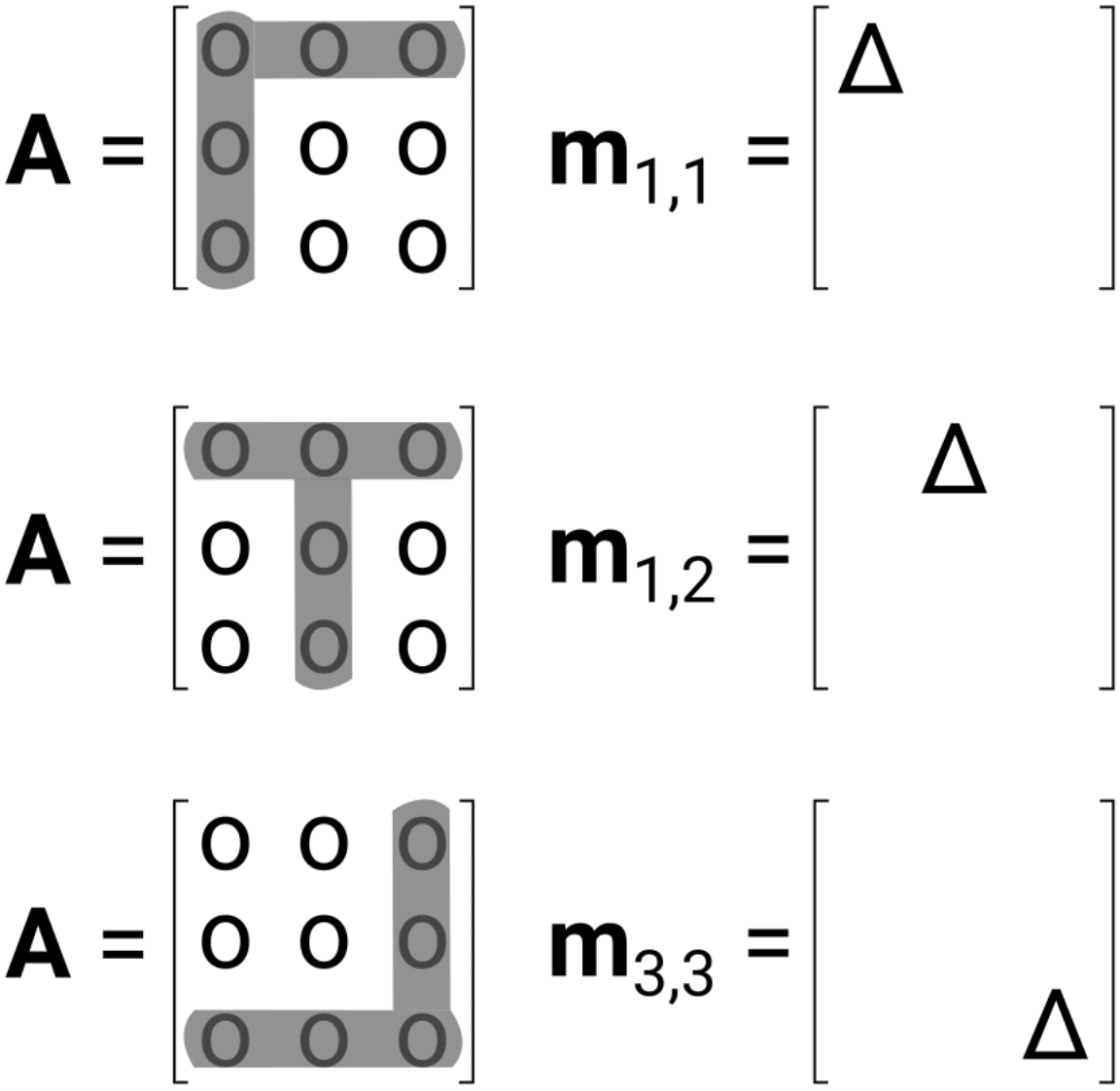
(OK, itâ€™s technically 4 steps if you consider dividing by the determinant its own full step. But MCA sounds better than MCAD.)

Letâ€™s go through each of these steps in more detail.

Minors matrix The minors matrix is a matrix in which each element $m_{i,j}$ of the matrix is the determinant of the matrix excluding the ith row and the jth column. [4]

Thus, for each element in the matrix, cross out that row and that column, and compute the determinant of the remaining matrix. That determinant goes into the matrix element under consideration. Itâ€™s easier to understand visually (Figure 12.1).

[4] This should sound familiar from the formula for computing the determinant of a 4Ã—4 matrix.



FigureÂ 12.1: Each element $m_{i,j}$ of the minors matrix is the determinant of the submatrix formed from excluding row i and column j from the original matrix.

The matrices on the following page illustrate creating a minors matrix M from a 3Ã—3 matrix A. Pick a few elements of the minors matrix to confirm that you can reproduce those values from the determinants of submatrices in matrix A.

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 4 & 2 \\ 1 & 3 & 2 \end{bmatrix}, \quad m_{1,1} = \begin{bmatrix} 2 & & \\ & & \\ & & \end{bmatrix} \tag{12.29}$$

$$m_{1,2} = \begin{bmatrix} & -2 & \\ & & \\ & & \end{bmatrix} \tag{12.30}$$

$$M = \begin{bmatrix} 2 & -2 & -4 \\ -1 & 3 & 5 \\ -2 & 4 & 8 \end{bmatrix} \tag{12.31}$$

The minors matrix is the most time-consuming part of the MCA algorithm. Itâ€™s also the most tedious. Donâ€™t rush through it.

Practice problems Compute the minors matrix of the following matrices.

a) $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

c) $\begin{bmatrix} 3 & 4 & 1 \\ 0 & 2 & 3 \\ 5 & 4 & 1 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 4 & 3 \\ 2 & 1 \end{bmatrix}$

b) $\hat{a}^{\wedge}\text{'}3\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$

c) $\begin{bmatrix} -10 & -15 & -10 \\ 0 & -2 & -8 \\ 10 & 9 & 6 \end{bmatrix}$

Cofactors matrix The cofactors matrix is the Hadamard product of the minors matrix with a matrix of alternating signs. Letâ€™s call that matrix G for grid:

The matrix is a grid of +1s and âˆ’1s, starting with +1 for the upper-left element. Below are a few examples of G matrices. They look like checkerboards (or chessboards, depending on your level of board game sophistication) (Figure 12.2).

$$\begin{bmatrix} + & - \\ - & + \end{bmatrix}, \begin{bmatrix} + & - & + \\ - & + & - \\ + & - & + \end{bmatrix}, \begin{bmatrix} + & - & + & - \\ - & + & - & + \\ + & - & + & - \\ - & + & - & + \end{bmatrix}$$

The formula that defines each element of the G matrix is

$$g_{i,j} = -1^{i+j} \tag{12.32}$$

where i and j refer, respectively, to row and column index. Try out a few examples to see how this formula produces the alternating grids above. For example, the (1,1) position is $\hat{a}\check{}'1^{1+1} = 1$ while the (1,2) position is $\hat{a}\check{}'1^{1+2} = \hat{a}\check{}'1$. That said, I think itâ€™s easier to remember what the matrix looks like rather than memorizing the formula.

+0		1	+0
	1	+0	1
+0		1	+0

FigureÂ 12.2: Simple and elegant. Linear algebra is truly inspiring.

Finally, the cofactors matrix: C = G âŠ™ M. Using the example matrix from the previous page,

$$\mathbf{C} = \begin{bmatrix} 2 & 2 & -4 \\ 1 & 3 & -5 \\ -2 & -4 & 8 \end{bmatrix}$$

Be mindful of the signs: The sign of each cofactors matrix element depends both on the sign of the determinant, and on the sign of the G matrix. That is, of course, obvious from your elementary school arithmetic, but most mistakes in higher mathematics are arithmetic... _____

Practice problems Compute the cofactors matrix of the following matrices.

a) $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

c) $\begin{bmatrix} 3 & 4 & 1 \\ 0 & 2 & 3 \\ 5 & 4 & 1 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 4 & -3 \\ -2 & 1 \end{bmatrix}$

b) $\hat{a}\check{}'3\begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}$

c) $\begin{bmatrix} -10 & 15 & -10 \\ 0 & -2 & 8 \\ 10 & -9 & 6 \end{bmatrix}$

Adjugate matrix At this point, all the hard work is behind you. The adjugate matrix is simply the transpose of the cofactors matrix, scalar-multiplied by the inverse of the determinant of the matrix (note: itâ€™s the determinant of the original matrix, not the minors or cofactors matrices). Again, if the determinant is zero, then this step will fail because of the division by zero.

Assuming the determinant is not zero, the adjugate matrix is the inverse of the original matrix.

Letâ€™s continue working with the matrix in the examples above.

$$\mathbf{A}^{-1} = \frac{1}{2}\begin{bmatrix} 2 & 1 & -2 \\ 2 & 3 & -4 \\ -4 & -5 & 8 \end{bmatrix}$$

And now letâ€™s test that this matrix really is the inverse of our original matrix:

[5]Here is another example of how it's often easier to leave scalars outside the matrix, rather than dealing with fractions. $\begin{bmatrix}2&1&1\\0&4&2\\1&3&2\end{bmatrix}\frac{1}{2}\begin{bmatrix}2&-1&-2\\-2&3&4\\-4&5&8\end{bmatrix}=\frac{1}{2}\begin{bmatrix}2&0&0\\0&2&0\\0&0&2\end{bmatrix}$

Now that you know the full MCA formula, we can apply this to a 2Ã—2 matrix. You can see that the "short-cut" in the previous section is just a simplification of this procedure.

$$\text{Original matrix : } \begin{bmatrix}4&2\\3&2\end{bmatrix}$$

$$\text{Minors matrix : } \begin{bmatrix}2&3\\2&4\end{bmatrix}$$

$$\text{Cofactors matrix : } \begin{bmatrix}2&-3\\-2&4\end{bmatrix}$$

$$\text{Adjugate matrix : } \frac{1}{2}\begin{bmatrix}2&-2\\-3&4\end{bmatrix}$$

–

Practice problems Compute the adjugate matrices of the following matrices (in case you havenâ€™t noticed: youâ€™ve already computed the minors and cofactors matrices above).

a) $\begin{bmatrix}1&2\\3&4\end{bmatrix}$

b) $\begin{bmatrix}1&2&3\\4&5&6\\7&8&9\end{bmatrix}$

c) $\begin{bmatrix}3&4&1\\0&2&3\\5&4&1\end{bmatrix}$

Answers

a) $\frac{1}{-2}\begin{bmatrix}4&-2\\-3&1\end{bmatrix}$

b) $\frac{-3}{0}\begin{bmatrix}1&-2&1\\-2&4&-2\\1&-2&1\end{bmatrix}$

c)

$$\frac{1}{20}\begin{bmatrix} -10 & 0 & -10 \\ 15 & -2 & -9 \\ 10 & 8 & 6 \end{bmatrix}$$

Practice problems Apply the MCA algorithm to the following matrices to derive their inverses, when they exist.

a) $\begin{bmatrix} 1 & 1 \\ 1 & 4 \end{bmatrix}$

b) $\begin{bmatrix} 2 & 2 & 4 \\ 4 & 2 & 5 \\ 1 & 2 & 1 \end{bmatrix}$

c) $\begin{bmatrix} 1 & -5 & 4 \\ -1 & -15 & 6 \\ 2 & 0 & 3 \end{bmatrix}$

Answers

a) $3\begin{bmatrix} 4 & -1 \\ -1 & 1 \end{bmatrix}$

b) $10\begin{bmatrix} -8 & 6 & 2 \\ 1 & -2 & 6 \\ 6 & -2 & -4 \end{bmatrix}$

c) No inverse!

Code The code challenges at the end of this chapter will provide the opportunity to implement the MCA algorithm in code. Below you can see how easy it is to call the inverse functions in Python and MATLAB.

Code blockÂ 12.1: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.random.randn(3,3)Â
AiÂ =Â np.linalg.inv(A)Â
A@Ai
```

Code blockÂ 12.2: MATLAB

```
AÂ =Â randn(3,3);Â
AiÂ =Â inv(A);Â
A*Ai
```

12.5 Inverse via row reduction

This is a conceptually very different method for obtaining the inverse of a square matrix, but the result will be the same as the MCA method. The idea is to augment the matrix with the identity matrix and then perform row reduction to get the matrix into its RREF form. This will lead to one of two possible outcomes:

- Row reduction transforms the original matrix to the identity matrix, in which case the augmented matrix is the inverse (Equation 12.33).
- Row reduction does not produce the identity matrix, in which case the matrix is singular and therefore has no inverse.

_____

Eqn. box title=Row reduction method of computing the inverse

$$\text{rref}\left(\left[\mathbf{A} \mid \mathbf{I}\right]\right) \text{â‡’} \left[\mathbf{I} \mid \mathbf{A}^{-1}\right] \qquad (12.33)$$

_____

_____

Letâ€™s start with an example:

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 3 & 4 & 0 & 1 \end{array}\right]$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \text{â‡’} \begin{array}{c} [c]\text{Â} \\ \xrightarrow{-3R_1+R_2} \end{array} \left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & -2 & -3 & 1 \end{array}\right]$$

$$\left[\begin{array}{cc|cc} 1 & 0 & -2 & 1 \\ 0 & -2 & -3 & 1 \end{array}\right]$$

$$\begin{array}{c} [c]\xrightarrow{R_2+R_1} \\ \text{Â} \end{array} \qquad \begin{array}{c} [c]\xrightarrow{-1/2R_2} \\ \text{Â} \end{array}$$

$$\left[\begin{array}{cc|cc} 1 & 0 & -2 & 1 \\ 0 & 1 & 3/2 & -1/2 \end{array}\right]$$

You can confirm that the augmented part of the final matrix is the same as the inverse we computing from the MCA algorithm in the practice problems in the previous section.

We know that a singular matrix has no inverse. Letâ€™s see what happens when we apply the row-reduction method to a rank-1 matrix.

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 3 & 6 & 0 & 1 \end{array}\right]$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \text{â‡’} \quad \begin{array}{c} \text{[c]Â} \\ \xrightarrow{-3R_1+R_2} \end{array}$$

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & 0 & -3 & 1 \end{array}\right]$$

Thatâ€™s the end of the line. We cannot row reduce anymore, and yet we have not gotten the identity matrix on the left. This system is inconsistent, and ergo, this matrix has no inverse.

Practice problems Use row reduction to compute the inverse of the following matrices, or to discover that the matrix is singular. Then multiply each matrix by its inverse to make sure youâ€™ve gotten the correct result.

a) $\begin{bmatrix} -4 & 7 \\ 3 & -8 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 0 & 2 \\ 1 & 2 & 1 \\ 1 & 0 & 1 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 0 & 2 & 5 \\ 3 & 0 & 0 & 1 \\ 0 & 4 & 5 & 0 \\ 5 & 8 & 2 & -17 \end{bmatrix}$

d) $\begin{bmatrix} 1 & 0 & 2 & 5 \\ 3 & 0 & 0 & 1 \\ 0 & 4 & 5 & 0 \\ 0 & 3 & 4 & 0 \end{bmatrix}$

Answers

a) $\frac{1}{11}\begin{bmatrix}-8 & -7\\ -3 & -4\end{bmatrix}$

b) $\begin{bmatrix}-1 & 0 & 2\\ 0 & 1/2 & -1/2\\ 1 & 0 & -1\end{bmatrix}$

c) No inverse.

d) $14\begin{bmatrix}-1 & 5 & -6 & 8\\ 0 & 0 & 56 & -70\\ 0 & 0 & -42 & 56\\ 3 & -1 & 18 & -24\end{bmatrix}$

_

Code The code below shows the row-reduction method, and then compares that to the inverse function.

Code blockÂ 12.3: Python

```
importÂ numpyÂ asÂ npÂ
importÂ sympyÂ asÂ symÂ
AÂ Â =Â np.random.randn(3,3)Â
AcatÂ =Â np.concatenate((A,np.eye(3,3)),axis=1)Â
ArÂ =Â sym.Matrix(Acat).rref()[0]Â #Â RREFÂ
ArÂ =Â Ar[:,3:]Â #Â keepÂ inverseÂ
AiÂ =Â np.linalg.inv(A)Â
Ar-Ai
```

Code blockÂ 12.4: MATLAB

```
AÂ =Â randn(3);Â
ArÂ =Â rref([AÂ eye(3)]);Â %Â RREFÂ
ArÂ =Â Ar(:,4:end);Â %Â keepÂ inverseÂ
AiÂ =Â inv(A);Â
Ar-Ai
```

Why does it work? Equation 12.33 almost seems like magic. (In fairness, much of mathematics seems like magic before you become familiar with it... and a lot of it continues to seem like magic even after years of experience.) In fact, the reason why this method works is fairly straightforward, and involves thinking about Equation 12.33 in terms of solving a system of equations.

In Chapter 10, you learned that you can solve the equation Ax = b by performing Gauss-Jordan elimination on the augmented matrix [A|b]. If there is a solutionâ€”that is, if b is in the column space of Aâ€”then row reduction produces the augmented matrix [I|x].

Here we follow the same reasoning, but the vector b is expanded to the matrix I. That is, we want to solve AX = I, where X is the inverse of A. It might be easier to think about this in terms of columns of the identity matrix. Iâ€™ll use e_i to indicate the ith column of the identity matrix. And Iâ€™ll use a 3Ã—3 matrix in the interest of concreteness, but the procedure is valid for a matrix of any size.

$$Ax_1 = \begin{bmatrix}1\\0\\0\end{bmatrix} \text{â‡’ rref}([\ A \mid e_1\ ]) \text{â‡’ } [\ I \mid x_1\ ] \tag{12.34}$$

$$Ax_2 = \begin{bmatrix}0\\1\\0\end{bmatrix} \text{â‡’ rref}([\ A \mid e_2\ ]) \text{â‡’ } [\ I \mid x_2\ ] \tag{12.35}$$

$$Ax_3 = \begin{bmatrix}0\\0\\1\end{bmatrix} \text{â‡’ rref}([\ A \mid e_3\ ]) \text{â‡’ } [\ I \mid x_3\ ] \tag{12.36}$$

Each equation individually finds x_i , which is the vector that represents the weighting of the columns in A in order to obtain the vector e_i , which is one column of the identity matrix. As long as the column space of A spans all of R^M , there is guaranteed to be a solution to each of the systems of equations. And C(A) spans the entire ambient space when A is full-rank.

Thus, when breaking the problem down into individual column vectors, there is nothing new compared to what you learned in Chapter 10. The only addition here is to collect all of these separate steps together into one matrix equation: AX = I.

-|*|- **Reflection** The matrix inverse is a funny thing. Conceptually, itâ€™s one of the most important matrix operations in linear algebra and its applications. And yet, computer programs go to great lengths to avoid explicitly computing it unless absolutely necessary. So why, you might wonder, should I suffer through learning how to compute it when I can type inv on a computer? For the same reason that you need to learn how to compute 3+4 without a calculator: You will never really learn math unless you can do it without a computer. Frustrating but true. -|*|-

12.6 The left inverse for tall matrices

I wrote on page Â§ that only square matrices can have a full inverse. Thatâ€™s true, but it applies only to a full, a.k.a. two-sided, inverse. Rectangular matrices can have a one-sided inverse. The goal of this section is to derive the one-sided left inverse, explain how to interpret it, and define the conditions for a rectangular matrix to have a left inverse.

Letâ€™s start with a tall matrix, so dimensions M > N. Weâ€™ll call this matrix T for tall. Although this matrix is not invertible, we can, with a bit of creativity, come up with another matrix (actually, the product of several matrices) that will left-multiply T to produce the identity matrix. The key insight to get us started is that $T^T T$ is a square matrix. [6] In fact, $T^T T$ is invertible if rank(T) = N (more on this condition later). If $T^T T$ is invertible, then it has an inverse.

[6] Yes, I realize that there is an absurd number of "T"s in these equations.

$$(\mathbf{T}^{\mathrm{T}}\mathbf{T})^{-1}(\mathbf{T}^{\mathrm{T}}\mathbf{T}) = \mathbf{I} \tag{12.37}$$

If this expression looks strange, then just imagine rewriting it as:

$$C = T^T T$$

$$C^{â^'1} C = I \tag{12.38}$$

Hereâ€™s the thing about Equation 12.37 : The first set of parentheses is necessary because we are inverting the product of two matrices (neither of those matrices is individually invertible!). However, the second set of parentheses is not necessary; theyâ€™re there just for aesthetic balance. By removing the unnecessary parentheses and re-grouping, some magic happens: the product of three matrices that can left-multiply T to produce the identity matrix.

Eqn. box title=The left inverse

$$\underbrace{(\mathbf{T}^{\mathrm{T}}\mathbf{T})^{-1}\mathbf{T}^{\mathrm{T}}}_{\text{Left inverse}}\ \mathrm{T} = \mathrm{I} \tag{12.39}$$

Here is another way to look at it:

$$T^{-L} = (T^T T)^{â^'1} T^T$$

$$T^{-L} T = I$$

where T^{-L} indicates the left inverse. (This is non-standard notation, used here only to facilitate comprehension.)

Why is this called a "one-sided" inverse? Letâ€™s see what happens when we put the left inverse on the right side of the matrix:

$$\mathbf{T}(\mathbf{T}^{\mathrm{T}}\mathbf{T})^{-1}\mathbf{T}^{\mathrm{T}} \neq \mathbf{I}$$

The left-hand side of this equation is actually valid, in the sense that all the sizes match to make the multiplications work. But the result will not be the identity matrix. Youâ€™ll see numerical examples later. The point is that this is a left inverse.

Conditions for validity Now letâ€™s think about the conditions for the left inverse to be valid. Looking back to Equation 12.38 , matrix C (which is actually T^T T) is invertible if it is full rank, meaning rank(C) = N. When is T^T T a full-rank matrix? Recall from section 7.7 that a matrix times its transpose has the same rank as the matrix on its own. Thus, T^T T is full-rank if T has a rank of N, meaning it is full column-rank.

And this leads us to the two conditions for a matrix to have a left inverse:

1. It is tall (more rows than columns, M > N).
2. It is full column rank (rank=N).

Enough talk. Letâ€™s see an example.

$$\mathrm{T} = \begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \tag{12.40}$$

$$\mathrm{T}^{\mathrm{T}}\,\mathrm{T} = \begin{bmatrix} 3 & 9 \\ 9 & 29 \end{bmatrix} \tag{12.41}$$

$$(\mathrm{T}^{\mathrm{T}}\,\mathrm{T})^{\hat{a}\tilde{}'1} = \frac{1}{6}\begin{bmatrix} 29 & -9 \\ -9 & 3 \end{bmatrix} \tag{12.42}$$

$$\mathrm{T}^{\text{-L}} = (\mathrm{T}^{\mathrm{T}}\,\mathrm{T})^{\hat{a}\tilde{}'1}\,\mathrm{T}^{\mathrm{T}} = \frac{1}{6}\begin{bmatrix} 11 & 2 & -7 \\ -3 & 0 & 3 \end{bmatrix} \tag{12.43}$$

$$\mathrm{T}^{\text{-L}}\,\mathrm{T} = \frac{1}{6}\begin{bmatrix} 6 & 0 \\ 0 & 6 \end{bmatrix} \tag{12.44}$$

$$\mathrm{T}\mathrm{T}^{\text{-L}} = \begin{bmatrix} 5 & 2 & -1 \\ 2 & 2 & 2 \\ -1 & 2 & 5 \end{bmatrix} \tag{12.45}$$

Equation 12.44 shows that the pre-multiplying by the left inverse produces the identity matrix. In contrast, Equation 12.45 shows that post-multiplying by the

left inverse definitely does not produce the identity matrix. This example also highlights that it is often useful to leave the determinant as a scalar outside the matrix to avoid dealing with difficult fractions during matrix multiplications.

Practice problems Compute the left inverse of the following matrices.

a) $\begin{bmatrix} 3 & 0 \\ 0 & 2 \\ 0 & 4 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 2 & 3 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 2 \\ 1 & 2 \\ 1 & 2 \end{bmatrix}$

Answers Matrices below are $(T^T T)^{âˆ’1} T^T$

a) $\frac{1}{10}\begin{bmatrix} 10/3 & 0 & 0 \\ 0 & 1 & 2 \end{bmatrix}$

b) $\frac{1}{3}\begin{bmatrix} 5 & -4 & 1 \\ -3 & 3 & 0 \end{bmatrix}$

c) No left inverse!

12.7 The right inverse for wide matrices

You can probably guess where weâ€™re going in this section: wide matrices (that is, more columns than rows) do not have a left inverse. However, they can have a right inverse. I encourage you to try to discover the right inverse, as well as the conditions for a matrix to have a right inverse, on your own before continuing to read the rest of this section. To make things easier for you, Iâ€™ll leave the rest of this page blank...

Did you figure it out? The reasoning is the same as for the left inverse. The key difference is that you post-multiply by the transposed matrix instead of pre-multiplying by it. Letâ€™s call this matrix W for wide.

Briefly: We start from WW^T , and then right-multiply by $(WW^T)^{-1}$. That leads us to the following:

Eqn. box title=The right inverse
Right inverse = I

Again, if you find the multitude of matrices confusing, then you can substitute the three-matrix product for another matrix:

$$W \underbrace{\mathbf{W}^{\mathrm{T}}(\mathbf{W}\mathbf{W}^{\mathrm{T}})^{-1}}$$ (12.46)

$$W^{-R} = W^T (W^T W)^{â^'1}$$

$$WW^{-R} = I$$

The conditions for a matrix to have a right inverse are:

1. It is wide (more columns than rows, M < N).
2. It is full row rank (rank=M).

As with the left-inverse, putting the right inverse on the left of matrix W is a valid matrix multiplication, but will not produce the identity matrix.

And finally, an example with numbers.

$$W = \begin{bmatrix} 1 & 0 & 4 \\ 2 & -3 & 1 \end{bmatrix} \quad (12.47)$$

$$WW^T = \begin{bmatrix} 17 & 6 \\ 6 & 14 \end{bmatrix} \quad (12.48)$$

$$(WW^T)^{â^'1} = \frac{1}{202}\begin{bmatrix} 14 & -6 \\ -6 & 17 \end{bmatrix} \quad (12.49)$$

$$W^{-R} = W^T (WW^T)^{â^'1} = \frac{1}{202}\begin{bmatrix} 2 & 28 \\ 18 & -51 \\ 50 & -7 \end{bmatrix} \quad (12.50)$$

$$W^{-R} W = \frac{1}{202}\begin{bmatrix} 58 & -84 & 36 \\ -84 & 153 & 21 \\ 36 & 21 & 193 \end{bmatrix} \quad (12.51)$$

$$WW^{-R} = \frac{1}{202}\begin{bmatrix} 202 & 0 \\ 0 & 202 \end{bmatrix} \quad (12.52)$$

Practice problems Compute the right inverse of the following matrices.

a) $\begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 4 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 0 & 3 \\ -2 & 0 & 4 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 3 & 2 \\ 2 & 6 & 4 \end{bmatrix}$

Answers Matrices below are $W^T (W^T W)^{â^'1}$

a) $\frac{1}{10}\begin{bmatrix} 10/3 & 0 \\ 0 & 1 \\ 0 & 2 \end{bmatrix}$

b) $\frac{1}{10}\begin{bmatrix} 4 & -3 \\ 0 & 0 \\ 2 & 1 \end{bmatrix}$

c) No right inverse!

Code The code below shows the left inverse for a tall matrix; itâ€™s your job to modify the code to produce the right-inverse for a wide matrix! (Note: In practice, itâ€™s better to compute the one-sided inverses via the Moore-Penrose pseudoinverse algorithm, but itâ€™s good practice to translate the formulas directly into code.)

Code blockÂ 12.5: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.random.randn(5,3)Â
AlÂ =Â np.linalg.inv(A.T@A)@A.TÂ
Al@A
```

Code blockÂ 12.6: MATLAB

```
AÂ =Â randn(5,3);Â
AlÂ =Â inv(Aâ€™*A)*Aâ€™;Â
Al*A
```

12.8 The pseudoinverse, part 1

This section is called "part 1" because Iâ€™m going to introduce you to the pseudoinverse here, but Iâ€™m not going to teach you how to compute it until section 16.12 . Thatâ€™s not because Iâ€™m mean, and itâ€™s not because Iâ€™m testing your patience (which is, by the way, a virtue). Instead, itâ€™s because the algorithm to compute the Moore-Penrose pseudoinverse will seem incomprehensibly complicated if I present it here, whereas it will be intuitive and almost obvious in Chapter 16 after learning about the SVD.

The pseudoinverse is used when a matrix does not have a full inverse, for example if the matrix is square but rank-deficient. As I mentioned in the beginning of this chapter, a rank-deficient matrix does not have a true inverse. However, all matrices have a pseudoinverse, which is a matrix that will transform the rank-deficient matrix to something close-to-but-not-quite the identity matrix.

There are several algorithms to compute a pseudoinverse, but the most commonly used method is called the Moore-Penrose pseudoinverse. The original author names are often dropped, therefore, when you read or hear the term "pseudoinverse," itâ€™s safe to assume that itâ€™s the MP pseudoinverse.

Below is a list of important concepts about the pseudoinverse. This list contains claims that are justified and proven in Chapter 16.

1. It is indicated using a "dagger," asterisk, or plus sign in the superscript: $A^{â€}$, $A^{â^—}$, or A^{+} .
2. The pseudoinverse multiplies the original matrix to approximate the identity matrix: $AA^{â€}$ â‰ˆ I.
3. There are several ways to create a matrix pseudoinverse, which means that a singular matrix can have several pseudoinverses (unlike the true inverse, which is unique). However, the MP pseudoinverse is unique, meaning that every matrix has exactly one MP pseudoinverse. The uniqueness of the MP pseudoinverse contributes to its popularity.
4. The pseudoinverse is sided, thus $AA^{â€}$ â‰ $A^{â€}$ A. However, the pseudoinverse has the neat property that $AA^{â€}$ A = A (for square matrices).
5. For a full-rank matrix, the pseudoinverse is the same as the full inverse, that is, $A^{â€} = A^{â^’1}$.
6. For a tall full column-rank matrix, the pseudoinverse equals the one-sided left inverse. Same story for a wide full row-rank matrix and the right inverse.

Here is an example of a non-invertible square matrix, its pseudoinverse, and the products of those matrices.

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 3 & 1 \\ 2 & 5 & 2 \end{bmatrix}, \quad \mathbf{A}^{\dagger} = \frac{1}{6} \begin{bmatrix} 8 & -7 & 1 \\ -6 & 6 & 0 \\ 8 & -7 & 1 \end{bmatrix}$$

$$\mathbf{A}^{\dagger}\mathbf{A} = \frac{1}{2} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \quad \mathbf{A}\mathbf{A}^{\dagger} = \frac{1}{3} \begin{bmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}$$

You can see several properties of the pseudoinverse in this example, including that AA$^{â€}$ â‰ A$^{â€}$ A and that neither of these produces the identity matrix, although they both produce a matrix is that is sorta-kinda close to the identity.

Code The MP pseudoinverse is so important in applied linear algebra that numerical software developers have made it super-easy to implement. Notice that the matrix below is singular.

Code blockÂ 12.7: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.random.randn(3,3)Â
A[1,:]Â =Â A[0,:]Â
ApiÂ =Â np.linalg.pinv(A)Â
Api@A
```

Code blockÂ 12.8: MATLAB

```
AÂ =Â randn(3,3);Â
A(2,:)Â =Â A(1,:);Â
ApiÂ =Â pinv(A);Â
Api*A
```

12.9 Exercises

1.Â

Compute the inverse of the following matrices (when possible). For problems with a *, additionally compute AA$^{â^'1}$ and A$^{â^'1}$ A.

a) $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 4 & 0 \\ 0 & -2 \end{bmatrix}$

c) â^—$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$

d) $\begin{bmatrix} 3 & 2 \\ 6 & 4 \end{bmatrix}$

e) âˆ’$\begin{bmatrix} 4 & -4 \\ 1 & 6 \end{bmatrix}$

f) $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 2 & 1 \end{bmatrix}$

g) âˆ’$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

h) $\begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 0 \\ 7 & 6 & 0 \end{bmatrix}$

i) $\begin{bmatrix} 2 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 3 \end{bmatrix}$

j) âˆ’$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$

k) $\begin{bmatrix} 3 & 1 \\ 0 & b \end{bmatrix}$

l) $\begin{bmatrix} 2 & 1 & 0 & 6 \\ -1 & 0 & 4 & 0 \\ 2 & 0 & 3 & -4 \\ 0 & 1 & 0 & 4 \end{bmatrix}$

2.Â

[7] The inverse of the inverse is the original matrix. Is that $A^{â^'1}$ $A^{â^'1}$ or $(A^{â^'1})^{â^'1}$? Think of an answer and then confirm it empirically using the matrix $\begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix}$.

[7] Hint: Law of Exponents:

$$x^n\, x^m = x^{n+m}$$

$$(x^n)^m = x^{nm}$$

3.Â

Use the row-reduction method to compute the inverse of the following matrices.

a) $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$

b)

$$\begin{bmatrix} 1 & 7 \\ -2 & 4 \end{bmatrix}$$

c) $\begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 4 \\ 1 & 2 & 5 \end{bmatrix}$

d) $\begin{bmatrix} 1 & 2 & 3 \\ 1 & 3 & 4 \\ 2 & 5 & 7 \end{bmatrix}$

4.Â

For the following matrices and vectors, compute and use A_n $^{â^{\wedge}'1}$ to solve for x in A_n x = b_n .

$$\mathbf{A}_1 = \begin{bmatrix} 1 & 4 \\ 5 & 2 \end{bmatrix}, \mathbf{A}_2 = \begin{bmatrix} 3 & -3 \\ 2 & 2 \end{bmatrix}, \mathbf{A}_3 = \begin{bmatrix} 1 & 3 & 4 \\ 0 & 0 & 4 \\ 2 & 2 & 1 \end{bmatrix} \mathbf{A}_4 = \begin{bmatrix} 9 & 0 & 1 \\ 3 & 1 & 0 \\ 0 & 6 & 1 \end{bmatrix}$$

$$\mathbf{b}_1 = \begin{bmatrix} 4 \\ 1 \end{bmatrix}, \mathbf{b}_2 = \begin{bmatrix} 6 \\ -3 \end{bmatrix}, \mathbf{b}_3 = \begin{bmatrix} 1 \\ 3 \\ 8 \end{bmatrix} \mathbf{b}_4 = \begin{bmatrix} 8 \\ 3 \\ 1 \end{bmatrix}$$

a) A_1 x = b_1
b) A_1 x = b_2
c) A_2 x = b_1
d) A_2 x = b_2
e) A_3 x = b_3
f) A_3 x = b_4
g) A_4 x = b_2
h) A_4 x = b_4

12.10 Answers

1.Â

a) $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 1/4 & 0 \\ 0 & -1/2 \end{bmatrix}$

c)

$\frac{1}{3}\begin{bmatrix} -1 & 2 \\ 2 & -1 \end{bmatrix}$

d) No inverse!

e) $\frac{1}{28}\begin{bmatrix} 6 & 4 \\ -1 & 4 \end{bmatrix}$

f) $\frac{1}{3}\begin{bmatrix} 3 & 0 & 0 \\ 0 & -1 & 2 \\ 0 & 2 & -1 \end{bmatrix}$

g) $\frac{1}{3}\begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$

h) No inverse!

i) $\begin{bmatrix} 1/2 & 0 & 0 \\ 0 & 1/4 & 0 \\ 0 & 0 & 1/3 \end{bmatrix}$

j) $\frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$

k) $\frac{1}{3b}\begin{bmatrix} b & -1 \\ 0 & 3 \end{bmatrix}$

l) $\frac{1}{-54}\begin{bmatrix} -16 & 6 & -8 & 16 \\ 44 & 24 & -32 & -98 \\ -4 & -12 & -2 & 4 \\ -11 & -6 & 8 & 11 \end{bmatrix}$

2.Â

The correct expression for the inverse of the inverse is $(A^{â^'1})^{â^'1}$. $A^{â^'1}$ $A^{â^'1}$ would mean to matrix-multiply the inverse by itself, which is a valid mathematical operation but is not relevant here.

3.Â

Matrices below are the inverses (the augmented side of the matrix).

a) $\frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$

b) $\frac{1}{18}\begin{bmatrix} 4 & -7 \\ 2 & 1 \end{bmatrix}$

c) $\frac{1}{2}\begin{bmatrix} 7 & -4 & -1 \\ -1 & 2 & -1 \\ -1 & 0 & 1 \end{bmatrix}$

d) Not invertible!

4.Â

The strategy here is to compute the inverses of the four A matrices, then implement matrix-vector multiplication. Below are the inverses, and below that, the solutions to x.

$$\mathbf{A}_1^{-1} = \frac{1}{-18}\begin{bmatrix} 2 & -4 \\ -5 & 1 \end{bmatrix}, \quad \mathbf{A}_2^{-1} = \frac{1}{12}\begin{bmatrix} 2 & 3 \\ -2 & 3 \end{bmatrix},$$
$$\mathbf{A}_3^{-1} = \frac{1}{16}\begin{bmatrix} -8 & 5 & 12 \\ 8 & -7 & -4 \\ 0 & 4 & 0 \end{bmatrix}, \quad \mathbf{A}_4^{-1} = \frac{1}{27}\begin{bmatrix} 1 & 6 & -1 \\ -3 & 9 & 3 \\ 18 & -54 & 9 \end{bmatrix}$$

a) $18\begin{bmatrix} -4 & 19 \end{bmatrix}^T$
b) $18\begin{bmatrix} -24 & 33 \end{bmatrix}^T$
c) $12\begin{bmatrix} 11 & -5 \end{bmatrix}^T$
d) $12\begin{bmatrix} 3 & -21 \end{bmatrix}^T$
e) $16\begin{bmatrix} 103 & -45 & 12 \end{bmatrix}^T$
f) $16\begin{bmatrix} -37 & 39 & 12 \end{bmatrix}^T$
g) Invalid operation!
h) $27\begin{bmatrix} 25 & 6 & -9 \end{bmatrix}^T$

12.11 Code challenges

1.Â

Your task here is "simple": Implement the MCA algorithm to compute the matrix inverse. Consult the description in section 12.4 . Test your algorithm by computing the inverse of random matrices and compare against the Python/MATLAB inv functions.

2.Â

I wrote earlier that the algorithm underlying the MP pseudoinverse is only understandable after learning about the SVD.[8] But that neednâ€™t stop us from exploring the pseudoinverse with code! The goal of this challenge is to illustrate that the pseudoinverse is the same as (1) the "real" inverse for a full-rank square matrix, and (2) the left inverse for a tall full-column-rank matrix.

[8] Reminder: Demonstrations in code help build insight but are no substitute for a formal proof.

12.12 Code solutions

1.Â

This one might have been challenging, particularly if you are not an experienced coder.

Code blockÂ 12.9: Python

```
importÂ numpyÂ asÂ npÂ
Â
#Â createÂ matrixÂ
mÂ =Â 4Â
AÂ =Â np.random.randn(m,m)Â
MÂ =Â np.zeros((m,m))Â
GÂ =Â np.zeros((m,m))Â
Â
#Â computeÂ minorsÂ matrixÂ
forÂ iÂ inÂ range(m):Â
Â Â forÂ jÂ inÂ range(m):Â
Â
Â Â Â Â #Â selectÂ rowsÂ andÂ colsÂ
Â Â Â Â rowsÂ =Â [True]*mÂ
Â Â Â Â rows[i]Â =Â FalseÂ
Â
Â Â Â Â colsÂ =Â [True]*mÂ
Â Â Â Â cols[j]Â =Â FalseÂ
Â
Â Â Â Â M[i,j]=np.linalg.det(A[rows,:][:,cols])Â
Â
Â Â Â Â #Â computeÂ GÂ
Â Â Â Â G[i,j]Â =Â (-1)**(i+j)Â
Â
#Â computeÂ CÂ
CÂ =Â MÂ *Â GÂ
Â
#Â computeÂ AÂ
AinvÂ =Â C.TÂ /Â np.linalg.det(A)Â
AinvIÂ =Â np.linalg.inv(A)Â
AinvI-AinvÂ #Â compareÂ againstÂ inv()
```

Code blockÂ 12.10: MATLAB

```
%Â createÂ matrixÂ
mÂ =Â 4;Â
AÂ =Â randn(m);Â
[M,G]Â =Â deal(Â zeros(m)Â );Â
Â
%Â computeÂ matricesÂ
forÂ i=1:mÂ
Â Â forÂ j=1:mÂ
Â
Â Â Â Â %%Â selectÂ rows/colsÂ
Â Â Â Â rowsÂ =Â true(1,m);Â
Â Â Â Â rows(i)Â =Â false;Â
Â
Â Â Â Â colsÂ =Â true(1,m);Â
Â Â Â Â cols(j)Â =Â false;Â
Â
Â Â Â Â %Â computeÂ MÂ
Â Â Â Â M(i,j)Â =Â det(Â A(rows,cols)Â );Â
Â
Â Â Â Â %Â computeÂ GÂ
Â Â Â Â G(i,j)Â =Â (-1)^(i+j);Â
Â Â endÂ
endÂ
Â
%Â computeÂ CÂ
CÂ =Â MÂ .*Â G;Â
Â
%Â computeÂ AÂ
AinvÂ =Â Câ€™/det(A);Â
AinvIÂ =Â inv(A);Â
AinvI-AinvÂ %Â compareÂ againstÂ inv()
```

2.Â

.

Code blockÂ 12.11: Python

```
importÂ numpyÂ asÂ npÂ
Â
#Â squareÂ matrixÂ
AÂ Â Â =Â np.random.randn(5,5)Â
AiÂ Â =Â np.linalg.inv(A)Â
```

```
ApiÂ =Â np.linalg.pinv(A)Â
AiÂ -Â ApiÂ #Â testÂ equivalenceÂ
Â
#Â tallÂ matrixÂ
TÂ Â Â =Â np.random.randn(5,3)Â
TlÂ Â =Â np.linalg.inv(T.T@T)@T.TÂ #Â leftÂ invÂ
TpiÂ =Â np.linalg.pinv(T)Â #Â pinvÂ
TlÂ -Â TpiÂ #Â testÂ equivalance
```

Code blockÂ 12.12: MATLAB

```
%Â squareÂ matrixÂ
AÂ Â Â =Â randn(5);Â
AiÂ Â =Â inv(A);Â
ApiÂ =Â pinv(A);Â
AiÂ -Â ApiÂ %Â testÂ equivalenceÂ
Â
%Â tallÂ matrixÂ
TÂ Â Â =Â randn(5,3);Â
TlÂ Â =Â inv(Tâ€™*T)*Tâ€™;Â %Â leftÂ invÂ
TpiÂ =Â pinv(T);Â %Â pinvÂ
TlÂ -Â TpiÂ %Â testÂ equivalance
```

ChapterÂ 13
Projections and orthogonalization

The goal of this chapter is to introduce a framework for projecting one space onto another space (for example, projecting a point onto a line). This framework forms the basis for orthogonalization and for an algorithm called linear least-squares, which is the primary method for estimating parameters and fitting models to data, and is therefore one of the most important algorithms in applied mathematics, including control engineering, statistics, and machine learning. Along the way, weâ€™ll also re-discover the left inverse.

13.1 Projections in â„^{2}

We are going to discover a formula for projecting a point onto a line. From there, we can generalize the formula to projecting one space onto another space. Donâ€™t worry, itâ€™s easier than you might think.

We start with a vector a, a point b not on a, and a scalar Î² such that Î²a is as close to b as possible without leaving a. Figure 13.1 shows the situation. (Because we are working with standard-position vectors, it is possible to equate coordinate points with vectors.)

1

SVG-Viewer needed.

FigureÂ 13.1: We need a formula to obtain the scalar Î² such that point Î²a is as close as possible to point b without leaving vector a.

The question is: Where do we place Î² so that the point Î²a is as close as possible to point b? You might have an intuition that Î² should be placed such that the line from Î²a to b is at a right angle to a. Thatâ€™s the right intuition (yes, thatâ€™s a pun).

One way to think about this is to imagine that the line from Î²a to b is one side of a right triangle. Then the line from b to a is the hypotenuse of that triangle. Any hypotenuse is longer than the adjacent side, and so the shortest hypotenuse (i.e., the shortest distance from b to a) is the adjacent side.

We need an expression for the line from point b to point Î²a. We can express this line as the subtraction of vector b (the vector from the origin to point b) and vector Î²a (the scaled version of vector a). Thus, the expression for the line is (b âˆ’ Î²a).

Letâ€™s recap: We have established that the closest projection of b onto a involves the line that meets a at a right angle, and we have an expression for that line. Now our goal is to figure out the value of Î². The geometric approach already provided the key insight, we just need to translate that into algebra. In particular, vectors (b âˆ’ Î²a) and a are orthogonal, meaning they are perpendicular:

$$(\mathrm{b} - \beta\mathbf{a}) \perp \mathbf{a} \tag{13.1}$$

And that in turns means that the dot product between them is zero. Thus, we can rewrite Equation 13.1 as

$$(\mathrm{b} - \beta\mathbf{a})^{\mathrm{T}}\mathbf{a} = 0 \tag{13.2}$$

And that is the key insight that geometry provides us. From here, solving for Î² just involves a bit of algebra. Itâ€™s a beautiful and important derivation, so Iâ€™ll put all of it into a math box.

Eqn. box title=Orthogonal projection of a point onto a line

$$\mathrm{a}^{\mathrm{T}}(\mathrm{b} - \beta\mathrm{a}) = 0 \tag{13.3}$$

$$\mathrm{a}^{\mathrm{T}}\mathrm{b} - \beta\mathrm{a}^{\mathrm{T}}\mathrm{a} = 0$$

$$\beta\mathrm{a}^{\mathrm{T}}\mathrm{a} = \mathrm{a}^{\mathrm{T}}\mathrm{b} \tag{13.4}$$

$$\hat{I}^2 = \frac{\mathbf{a}^T\mathbf{b}}{\mathbf{a}^T\mathbf{a}}$$

Note that dividing both sides of the equation by $a^T a$ is valid because it is a scalar quantity.

The technical term for this procedure is projection: We are projecting b onto the subspace defined by vector a. This is commonly written as $proj_a$ (b).

(As an aside: Itâ€™s tricky to remember whether projecting b onto a is written as $proj_a$ (b) or $proj_b$ (a). A memory trick here is that the ð •Šubspace goes in the ð •Šubscript. Youâ€™ve seen now several of these weird little mnemonics in this book, and there will be more. I hope they help you learn, or at least make you smile.)

Note that Equation 13.4 gives the scalar value $\hat{I}^2$, not the actual point represented by the open circle in Figure 13.1 . To calculate that vector $\hat{I}^2 a$, simply replace $\hat{I}^2$ with its definition: [2]

---

[2] In the fraction, one a in the denominator doesnâ€™t cancel the multiplying a, because the vector in the denominator is part of a dot product.

Eqn. box title=Projection of a point onto a line

$$proj_a(b) = \frac{\mathbf{a}^T\mathbf{b}}{\mathbf{a}^T\mathbf{a}}\,a \qquad (13.5)$$

Letâ€™s work through an example with real numbers. Weâ€™ll use the following vector and point.

$$\mathbf{a} = \begin{bmatrix}-2\\-1\end{bmatrix}, \quad b = (3,-1)$$

If you draw these two objects on a 2D plane, youâ€™ll see that this example is different from that presented in Figure 13.1 : The point is "behind" the line, so it will project negatively onto the vector. Letâ€™s see how this works out algebraically.

$$proj_a(b) = \frac{\begin{bmatrix}-2\\-1\end{bmatrix}^T\begin{bmatrix}3\\-1\end{bmatrix}}{\begin{bmatrix}-2\\-1\end{bmatrix}^T\begin{bmatrix}-2\\-1\end{bmatrix}}\begin{bmatrix}-2\\-1\end{bmatrix} = \frac{-6+1}{4+1}\begin{bmatrix}-2\\-1\end{bmatrix} = \hat{a}\text{'}1\begin{bmatrix}-2\\-1\end{bmatrix}$$

Notice that $\hat{I}^2 = \hat{a}\text{'}1$. Thus, we are projecting "backwards" onto the vector (Figure 13.2). This makes sense when we think of a as being a basis vector for a 1D subspace that is embedded in â„ [2] .

SVG-Viewer needed.

FigureÂ 13.2: Example of projecting a point onto a line. The intersection $\hat{I}^2 a$ is coordinate (2,1).

Practice problems Draw the following lines (a) and points (b). Draw the approximate location of the orthogonal projection of b onto a. Then compute the exact $proj_a$ (b) and compare with your guess.

a) $a = \begin{bmatrix}-.5\\2\end{bmatrix}$, b = (.5,2)

b) $a = \begin{bmatrix}0\\2\end{bmatrix}$, b = (0,1)

c) $a = \begin{bmatrix}2\\0\end{bmatrix}$, b = (0,â€™1)

d) $a = \begin{bmatrix}1\\2\end{bmatrix}$, b = (2,1)

Answers Answers below are the Î²; proj

a(b)isÎ²a.

a) 3.75/4.25
b) .5
c) 0
d) 4/5

-|*|- Reflection Mapping over magnitude: Meditating on Equation 13.4 will reveal that it is a mapping between two vectors, scaled by the squared length of the "target" vector. Itâ€™s useful to understand this intuition (mapping over magnitude), because many computations in linear algebra and its applications (e.g., correlation, convolution, normalization) involve some kind of mapping divided by some kind of magnitude or norm. -|*|-

13.2 Projections in â„ ^{N}

Now you know how to project a point onto a line, which is a 1D subspace. Weâ€™re now going to extend this to projecting a point onto an N-D subspace.

We begin simply by replacing vector a (which is, by definition, a 1D subspace) to matrix A, the columns of which form a subspace with some dimensionality between 1 and the matrix rank (excluding the case of a zero-dimensional subspace from the zeros matrix; that would be boring).

Point b is still the same: We can conceptualize it as a coordinate in space, but we represent it using vector b, which is in â„ ^{M} , corresponding to the ambient dimensionality of the column space of A.

Because A has multiple columns, we also need to replace scalar Î² with a vector. Iâ€™ll call that vector x.

$$A^T(b - Ax) = 0 \quad (13.6)$$

$$A^T b - A^T Ax = 0$$

$$A^T Ax = A^T b \quad (13.7)$$

Hmm... are we stuck here? When we restricted ourselves to a 1D subspace, a was a vector and therefore $a^T a$ was a scalar, so we could divide both sides of the equation by that dot product. But now we have $A^T A$, and as you know, there is no matrix division. Can you guess the solution that allows us to solve for x? The secret is revealed on the next page; see if you can derive a solution to our conundrum before looking ahead!

[3] Note the zeros vector here instead of the scalar 0 used above.

Iâ€™m sure you guessed it: Left-multiply by $(A^T A)^{-1}$.

$$(A^T A)^{-1} A^T Ax = (A^T A)^{-1} A^T b$$

$$x = (A^T A)^{-1} A^T b \quad (13.8)$$

I hope you see that we "accidentally" re-discovered the formula for the left inverse in Equation 13.8 . This never ceases to amaze me: We started off with a completely different goal that seems to have nothing whatsoever to do with computing an inverse of a tall matrix, and yet we arrived at the left inverse as the way to project a point onto the column space of A.

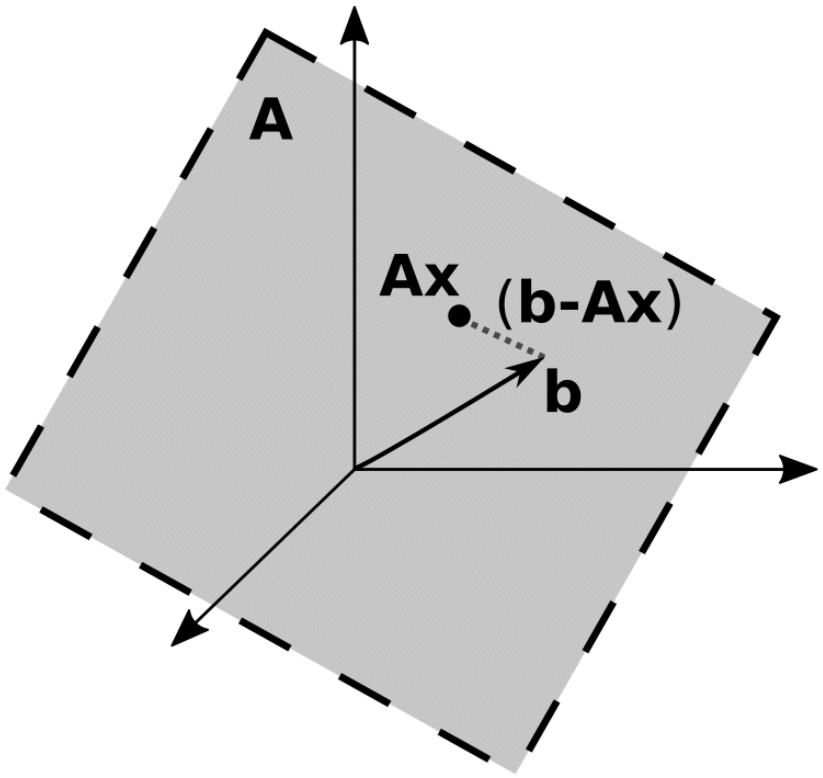
Indeed, one way of thinking about the one-sided inverse is that it projects a rectangular matrix onto the (square) identity matrix.

You can see that this equation involves inverting a matrix, which should immediately raise the question in your mind: What is the condition for this equation to be valid? The condition, of course, is that $A^T A$ is square and full-rank. And you know from the previous chapter that this is the case if A is already square and full-rank, or if it is tall and full column-rank.

It is insightful to think back to the geometric perspective: we are trying to project a point (also represented as the end-point of a vector in its standard position) b onto the column space of matrix A. If b is in the column space of

A, then x indicates the combinations of columns in A that produce b. Figure 13.3 shows a visualization of this idea for a 2D column space embedded in 3D ambient space (in other words, the matrix A has three rows and rank=2).

What happens if b is already inside the column space of A? Geometrically, it means that â¯bâ’Axâ¯ = 0, and consequently, that b = Ax. Plugging this into Equation 13.6 also makes sense algebraically.



FigureÂ 13.3: Visualization of projecting point b onto the 2D column subspace of A (grey patch). Assume that b is not in the column space of A, and vector (b â’ Ax), represented as the dotted line, is orthogonal to the column space. Thus, Ax is the vector inside the column space of A that minimizes the distance to b.

We can also consider what happens when A is a square full-rank matrix: This guarantees that b is already in its column space, because the column space of A spans all of â„ ^{M} . In that case, Equation 13.8 simplifies a bit:

$$x = (A^T A)^{â^’1} A^T b$$
$$x = A^{â^’1} A^{â^’T} A^T b$$
$$x = A^{â^’1} b \quad (13.9)$$

Practice problems Solve for x using Equation 13.8 .

a) $\begin{bmatrix} 1 & 2 \\ 3 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 5.5 \\ -3.5 \\ 1.5 \end{bmatrix}$

b) $\begin{bmatrix} -2 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 12 \\ 46 \end{bmatrix}$

c) $\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 1.2 \end{bmatrix}$

d)

$$\begin{bmatrix}1 & 3\\ 2 & 6\\ 3 & 9\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix}=\begin{bmatrix}4\\ 8\\ 11\end{bmatrix}$$

Answers

a) $\begin{bmatrix}-2.5 & 4\end{bmatrix}^T$
b) $\begin{bmatrix}7 & 13\end{bmatrix}^T$
c) $\begin{bmatrix}2 & 1\end{bmatrix}^T$
d) Doesnâ€™t work! ($A^T A$ is singular)

-|*|- **Reflection** Figures 13.2 and 13.3 provide the geometric intuition underlying the least-squares formula, which is the mathematical backbone of many analyses in statistics, machine-learning, and AI. Stay tuned... -|*|-

Code You can implement Equation 13.8 based on what you learned in the previous chapter. But that equation is so important and is used so often that numerical processing software packages have short-cuts for solving it. The Python code uses the numpy function lstsq, which stands for least-squares.

Code blockÂ 13.1: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â [[1,2],[3,1],[1,1]]Â
bÂ =Â [5.5,-3.5,1.5]Â
np.linalg.lstsq(A,b)[0]
```

Code blockÂ 13.2: MATLAB

```
AÂ =Â [1Â 2;Â 3Â 1;Â 1Â 1];Â
bÂ =Â [5.5Â -3.5Â 1.5]â€™;Â
A\b
```

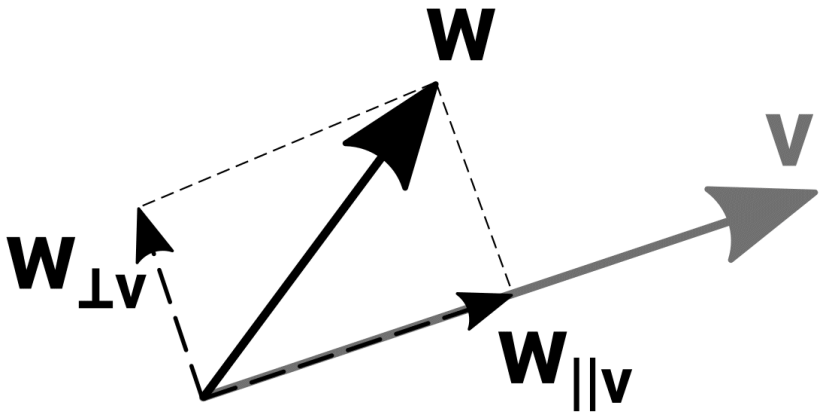
13.3 Orthogonal and parallel vector components

In this section, you will learn how to decompose one vector into two separate vectors that are orthogonal to each other, and that have a special relationship to a third vector. This procedure will draw on concepts you learned in Chapter 2 (adding and subtracting vectors) and in the previous section (projecting onto vectors). And in turn, it forms the basis for orthogonalization, which youâ€™ll learn about in the following section.

Letâ€™s start with a picture so you understand our goal (in â„ ^{2} , of course, because linear algebra always starts in â„ ^{2}). We begin with two vectors, w and v. Letâ€™s call w the "target" vector, and v the "reference" vector.

The idea is that we want to break up w into two separate vectors, one of which is parallel to w and the other is perpendicular to w. In Figure 13.4 , the component of vector w that is parallel to v is labeled $w_{â^¥v}$ and the component that is perpendicular to v is labeled $w_{âŠ¥v}$. The thin gray dashed lines illustrate that these two vector components sum to form the original vector w. In other words:

$$\mathbf{w} = \mathbf{w}_{\perp \mathbf{v}} + \mathbf{w}_{||\mathbf{v}} \tag{13.10}$$



FigureÂ 13.4: Decomposing vector w into components that are parallel and orthogonal to vector v.

Now that you have the picture, letâ€™s start deriving formulas and proofs. Donâ€™t worry, itâ€™ll be easier than you might think.

Parallel component If w and v have their tails at the same point, then the component of w that is parallel to v is simply collinear with v. In other words, $w_{||v}$ is a scaled version of v. Does this situation look familiar? If not, then you were either sleep-reading section 13.1 , or you just opened to book to this page. Either way, it would behoove you to go back and re-read section 13.1 . All we need to do here is project w onto v. Hereâ€™s what that formula looks like:

[4] [4]This is just a modified version of Equation 13.5.

$$\mathbf{w}_{||\mathbf{v}} = \text{proj}_{\mathbf{v}}\mathbf{w} = \frac{\mathbf{w}^{\mathrm{T}}\mathbf{v}}{\mathbf{v}^{\mathrm{T}}\mathbf{v}}\mathbf{v} \tag{13.11}$$

Perpendicular component How do we solve for $w_{âŠ¥v}$? We donâ€™t need to do anything fancy; we simply revisit the fact that the vector w is exactly decomposed into the sum of its perpendicular and parallel components (Equation 13.10). In particular, we already know w, and we just computed $w_{||v}$. A bit of middle-school algebra leads us to the solution:

$$\mathbf{w}_{\perp\mathbf{v}} = \mathbf{w} - \mathbf{w}_{||\mathbf{v}} \tag{13.12}$$

And this brings us to the full set of equations for decomposing a vector into two components relative to a target vector. _____

Eqn. box title=Decomposing a vector relative to another vector
vw = v_____ The result of this procedure is a set of orthonormal vectors that, when placed as column vectors in a matrix, yield an orthogonal matrix Q. Letâ€™s start with an example using three vectors in â„ [2] . Before looking at the math, draw these three vectors on a 2D plane, and try to visualize how the resulting orthogonalized vectors might look.

w = $w_{||v}$ + $w_{âŠ¥v}$

$w_{||v}$ = proj (13.14) w_âŠ¥ v = w â^’ wâ^¥v(13.15)

_____

Hereâ€™s a tip that might help you remember these formulas: Because $w_{â^¥v}$ is parallel to v,

We need to prove that $w_{â^¥v}$ and $w_{âŠ¥v}$ really are orthogonal to each other. The proof comes f

That looks on first glance like a messy set of equations, but when keeping in mind that the dc

[5] If you squint, this proof looks like a flock of birds.

Letâ€™s work through an example in â„ [2] , using vectors v=[2 3]^T and w=[4 0]^T . And to

Itâ€™s easily seen in this example that $v_{\|w}$ and $v_{â\check{S}¥w}$ are orthogonal and that $v_{\|w} + v_{â\check{S}¥w} = v$

Now letâ€™s try an example in â„ [6] . We wonâ€™t be able to visualize the results, but the

$$a = \begin{bmatrix} 2 \\ 3 \\ 5 \\ 0 \\ 1 \\ 2 \end{bmatrix}, \; b = \begin{bmatrix} 0 \\ -2 \\ -2 \\ 5 \\ 5 \\ -6 \end{bmatrix}$$

You can confirm (on paper or computer) that the two vector components (1) sum to produce

Practice problems For the following pairs of vectors (a,b), decompose a into parallel and pe

a) $\begin{bmatrix} 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 0 \\ -3 \end{bmatrix}$

b) $\begin{bmatrix} 1/2 \\ 0 \end{bmatrix}, \begin{bmatrix} -1 \\ 2 \end{bmatrix}$

c) $\begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix}, \begin{bmatrix} 2 \\ 4 \\ 0 \end{bmatrix}$

Answers Answers below are $a_{â\tilde{}¥b}$ and $a_{â\check{S}¥b}$.

a) $\begin{bmatrix} 0 \\ 2 \end{bmatrix}, \begin{bmatrix} 2 \\ 0 \end{bmatrix}$

b) $\begin{bmatrix} .1 \\ -.2 \end{bmatrix}, \begin{bmatrix} .4 \\ .2 \end{bmatrix}$

c) $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 3 \end{bmatrix}$

-|*|- **Reflection** A quick glance at the geometric representations of vectors in â„ ^{2} , particul

13.4 Orthogonal matrices

I introduced you to orthogonal matrices in section 5.5 . Transforming a matrix into an orthog

Orthogonal matrices are typically indicated using the letter Q. Thatâ€™s not the only letter g

There are two key properties of an orthogonal matrix:

1. All columns are pairwise orthogonal.
2. All columns have a magnitude of 1.

Here are these two properties translated into math:

In a linear algebra sense, Equation 13.17 is ugly because it can be written in a more compact

Eqn. box title=Orthogonal matrix definition

If youâ€™re struggling to see the link between Equations 13.17 and 13.18 , then think about

This is a remarkable definition, because it matches the definition of the matrix inverse. That

Thatâ€™s really convenient, because computing the inverse is a hassle and can be numerical

Now for some examples. The trivial example of an orthogonal matrix is the identity matrix, I

But wait a minuteâ€”it that really an orthogonal matrix? It conforms to the first property (ort

In fact, this is a special case of the general form for 2Ã—2 Q matrices:

A bit of arithmetic will confirm the first property, and a bit of trigonometry will confirm the

Rectangular Q The quick version here is that an orthogonal matrix does not need to be squa

Consider the following matrix. Take a moment to prove to yourself that it is an orthogonal m

This example is for a tall orthogonal matrix. Can we have a wide orthogonal matrix? Letâ€™

Well, the second requirement (norm of 1) already fails. A scalar of $(3\sqrt{5})$â^•5 will work for

But this example doesnâ€™t show that a wide matrix cannot be orthogonal; it merely shows

In a strict sense, the answer is No: A wide matrix cannot be an orthogonal matrix. Thatâ€™s

A wide matrix has size M < N and its maximum possible rank is M, corresponding to the nu

That said, wide matrices can be "almost orthogonal"â€”they can have interesting properties t

You can see that all columns are pairwise orthogonal, but the third column has a norm of 0. T

Orthogonal matrices seem a bit magical. Are they exceedingly rare and nearly impossible to

Practice problems Determine the value of Î¶ that will produce orthogonal matrices.

a) Î¶$\begin{bmatrix} -1/2 & 1/2 \\ 1/2 & 1/2 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 2^{-.5} & 2^{-.5} \\ 0 & 2^{-.5} & -2^{-.5} \\ \zeta & 0 & 0 \end{bmatrix}$

c) $\begin{bmatrix} \cos(\zeta) & -\sin(\zeta) & 0 \\ \sin(\zeta) & \cos(\zeta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Answers Answers below are Î¶.

a) $\sqrt{2}$
b) 1
c) any Î¶

13.5 Orthogonalization via Gram-Schmidt

Letâ€™s say you have a set of vectors in â„ ^{N} that is independent but not orthogonal. You

---

Eqn. box title=Procedure for creating a set of orthonormal vectors
(13.31) For all remaining vectors in the set:
2) Orthogonalize v_k^* to all previous vectors (eq. 13.15)
3) Normalize v_k^* to unit length

1) Start with v_1 and normalize to unit length: v_1 âˆ’

v_1 âˆ’ =

v_2 âˆ’ =

v_2 âˆ’ =

v_3 âˆ’ =

=

Putting these vectors into a matrix yields

[6] Normalize to unit vector

[7] Orthogonalize to v_1

[8] Normalize to unit vector

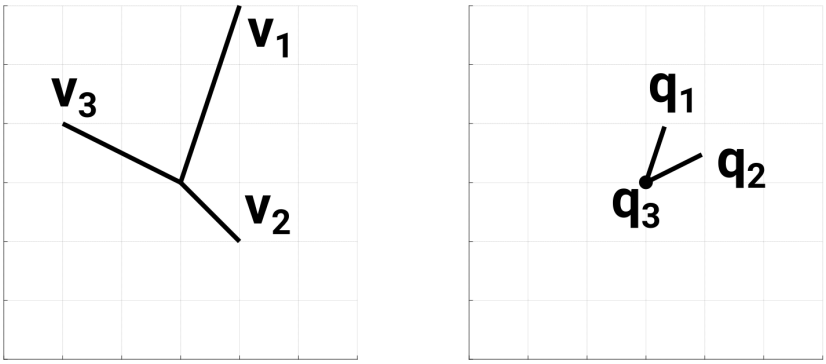
[9] Orthogonalize to v_1 âˆ’ and to v_2 âˆ’

$$Q = \begin{bmatrix} \frac{1}{\sqrt{10}} & \frac{3\sqrt{5}}{5\sqrt{2}} & 0 \\ \frac{3}{\sqrt{10}} & \frac{3\sqrt{5}}{10\sqrt{2}} & 0 \end{bmatrix}$$

A few notes about this example:

1. The arithmetic gets difficult quickly, doesnâ€™t it? I could have come up with numbers that worked out easier, but there are several important lessons here: (1) even seemingly simple problems can become complicated (a general challenge in life), (2) donâ€™t be intimidated by the complexityâ€”take a deep breath, get some paper and a pencil, and work through your (math) problems with a positive and assertive attitude.
2. Note that each vector is orthogonalized to $v_{kâ€™1}$ âˆ’ â€”the already-orthogonalized vectorâ€”not to the original $v_{kâ€™1}$.
3. Are you surprised with the result v_3 âˆ’ = 0? You shouldnâ€™t beâ€”itâ€™s not possible to have 3 orthogonal vectors in â„ [2] or, in general, n+1 orthogonal vectors in â„ [N] .
4. Numerical computing programs like MATLAB and Python will omit the final column of zeros, thus returning a true orthogonal matrix, even though it is not the same size as the original matrix. Iâ€™ll write more about the sizes of orthogonalized matrices in a later section.

Figure 13.6 shows the column vectors of V and Q. Aside from the intense arithmetic on the previous page, it is also geometrically obvious that q_3 must be zeros, because it is not possible to have more than two orthogonal vectors in â„ [2] . This is an application of the theorem about the maximum number of vectors that can form a linearly independent set, which was introduced in section 4.6 .



FigureÂ 13.6: Lines show column vectors in matrix V (left) and Q (right). Note that vector q_3 is a dot at the origin.

Practice problems Produce orthogonal matrices using the Gram-Schmidt procedure.

a) $\begin{bmatrix} 1 & 3 \\ 1 & 4 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 3 \\ -3 & 4 \end{bmatrix}$

c) $\begin{bmatrix} 3 & 3 \\ 4 & 4 \end{bmatrix}$

Answers

a) $\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$

b) $\frac{1}{\sqrt{10}} \begin{bmatrix} 1 & 3 \\ -3 & 1 \end{bmatrix}$

c) $\frac{1}{5} \begin{bmatrix} 3 & 0 \\ 4 & 0 \end{bmatrix}$

-|*|- **Reflection** The Gram-Schmidt procedure is numerically unstable, due to round-off errors that propagate forward to each subsequent vector and affect both the normalization and the orthogonalization. Youâ€™ll see an example of this in the code challenges. Computer programs therefore use numerically stable algorithms that achieve the same conceptual result, based on modifications to the standard Gram-Schmidt procedure or alternative methods such as Givens rotations or Gaussian elimination. -|*|-

13.6 QR decomposition

The Gram-Schmidt procedure transforms matrix A into orthogonal matrix Q. Unless A is already an orthogonal matrix, Q will be different than A, possibly very different. Thus, information is lost when going from A â†’ Q. Is is possible to recover the information that was lost in translation? Obviously, the answer is yes. And thatâ€™s the idea of QR decomposition:

$$\mathbf{A} = \mathbf{QR} \tag{13.32}$$

The Q here is the same Q that you learned about above; itâ€™s the result of Gram-Schmidt orthogonalization (or other comparable but more numerically stable algorithm).[10] R is like a "residual" matrix that contains the information that was orthogonalized out of A. You already know how to create matrix Q; how do you compute R? Easy: Take advantage of the definition of orthogonal matrices:

[10] QR decomposition is unrelated to QR codes.

$$Q^T A = Q^T QR \tag{13.33}$$

$$= R \tag{13.34}$$

$Q^T A$

Iâ€™d like to show you an example, but because you already know how to compute Q, Iâ€™ll simply write the answer without details or fanfare; that will allow us to focus the discussion on R.

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad \Rightarrow \quad \mathbf{Q} = \begin{bmatrix} \sqrt{2}^{-1} & \sqrt{6}^{-1} \\ \sqrt{2}^{-1} & -\sqrt{6}^{-1} \\ 0 & -2\sqrt{6}^{-1} \end{bmatrix} \tag{13.35}$$

Following Equation 13.34 , we compute R.

$$\mathbf{R} = \mathbf{Q}^{\mathrm{T}}\mathbf{A} = \begin{bmatrix} \sqrt{2}^{-1} & \sqrt{2}^{-1} & 0 \\ \sqrt{6}^{-1} & -\sqrt{6}^{-1} & -2\sqrt{6}^{-1} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \sqrt{2} & \sqrt{2}^{-1} \\ 0 & -\sqrt{6}/2 \end{bmatrix} \tag{13.36}$$

In this example, R is an upper-triangular matrix. Thatâ€™s not just some happy coincidence. In fact, R will always be an upper-triangular matrix.

To understand why R must be upper-triangular, itâ€™s useful to keep your eye on Equation 13.34 while reviewing the mechanics of matrix multiplication. In particular, on page Â§ I stated that the upper triangle of the product matrix (here thatâ€™s R) comprises dot products between earlier rows in the left matrix (here thatâ€™s Q^T) with later columns in the right matrix (A). In contrast, the lower triangle of the product matrix comprises dot products between later rows of the left matrix and earlier columns of the right matrix. (Also keep in mind that the rows of Q^T are the columns of Q.)

Because orthogonalization works column-wise from left-to-right, later columns in Q (i.e., rows of Q^T) are orthogonalized to earlier columns of A, hence the lower triangle comes from pairs of vectors that have been orthogonalized. But the other way around is not the case: earlier columns in Q (earlier rows in Q^T) have not yet been orthogonalized to later columns of A. And thatâ€™s why R will always be upper-triangular.

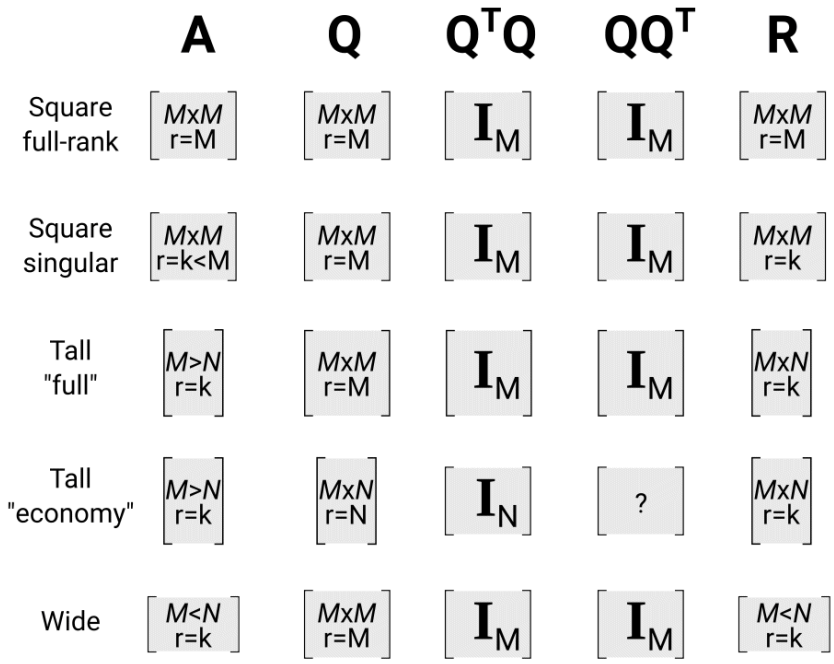
Sizes of Q and R, given A The sizes of Q and R depend on the size of A, and on a parameter of the implementation.

Letâ€™s start with the square case, because thatâ€™s easy: If A is a square matrix, then Q and R are also square matrices, of the same size as A. This is true regardless of the rank of A (more on rank and QR decomposition below).

Now letâ€™s consider a tall matrix A (M > N). Computer algorithms can implement the "economy QR" or "full QR" decomposition. The economy QR is what you might expect based on the example in the previous page: Q will be the same size as A, and R will be NÃ—N. However, it is possible to create a square Q from a tall A. Thatâ€™s because the columns of A are in â„ ^{M} , so even if A has only N columns, there are M â€“ N more possible columns to create that will be orthogonal to the first M. Thus, the full QR decomposition of a tall matrix will have Q â€¦ â„ $^{M×M}$ and R â€¦ â„ $^{M×N}$. In other words, Q is square and R is the same size as A.

Finally, we consider wide matrices. There is no economy QR for wide matrices, because A already has more columns than could form a linearly independent set. Thus, for a wide matrix A, the story is the same as for the full QR decomposition of a tall matrix: Q â€¦ â„ $^{M×M}$ and R â€¦ â„ $^{M×N}$.

Figure 13.7 provides a graphical overview of these sizes.



FigureÂ 13.7: Representation of matrix sizes and ranks from QR decomposition for different possibilities of A (the different rows). When k is not specified relative to M, then k â‰¤ M. The "?" indicates that the matrix elements depend on the values in A and Q, i.e., it is not the identity matrix.

Ranks of Q, R, and A The Q matrix will always have its maximum possible rank (M or N depending on its size, as described above), even if A is not full rank.

It may seem surprising that the rank of Q can be higher than the rank of A, considering that Q is created from A. But consider this: If I give you a vector [-1 1], can you give me a different, orthogonal, vector? Of course you could: [1 1] (among others). Thus, new columns in Q can be created that are orthogonal to all previous columns. You can try this yourself in Python or MATLAB by taking the QR decomposition of 1 (the matrix of all 1â€™s).

On the other hand, R will have the same rank as A. First of all, because R is created from the product Q^T A, the maximum possible rank of R will be the rank of A, because the rank of A is equal to or less than the rank of Q. Now letâ€™s think about why the rank of R equals the rank of A: Each diagonal element in R is the dot product between corresponding columns in A and Q. Because each column of Q is orthogonalized to earlier columns in A, the dot products forming the diagonal of R will be non-zero as long as each column in A is linearly independent of its earlier columns. On the other hand, if column k in A can be expressed as a linear combination of earlier columns in A, then column k of Q is orthogonal to column k of A, meaning that matrix element $R_{k,k}$ will be zero.

Code QR decomposition is easy to implement in Python and MATLAB. Computing the full vs. economy version requires an additional optional input. Confusingly, the default is Python is economy whereas the default in MATLAB is full.

Code blockÂ 13.3: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.random.randn(4,3)Â
Q,RÂ =Â np.linalg.qr(A)
```

Code blockÂ 13.4: MATLAB

```
AÂ =Â randn(4,3);Â
[Q,R]Â =Â qr(A);
```

13.7 Inverse via QR

Iâ€™ve mentioned before that you should avoid having computers explicitly compute the inverse unless itâ€™s necessary, because of the risk of inaccurate results due to numerical instabilities. The QR decomposition provides a more stable algorithm to compute the matrix inverse, compared to the MCA algorithm you learned about in the previous chapter. To begin, we simply write out the QR decomposition formula and then invert both sides of the equation:

$$\begin{aligned} A &= QR \\ A^{â^'1} &= (QR)^{â^'1} \\ A^{â^'1} &= R^{â^'1} Q^{â^'1} \\ A^{â^'1} &= R^{â^'1} Q^T \end{aligned} \tag{13.37}$$

Well, we still need to compute the explicit inverse of R, but if A is a dense matrix, then $R^{â^'1}$ is easier and more stable to calculate than is $A^{â^'1}$, because nearly half of the elements in R are zeros, and matrices with a lot of zeros are easy to work with.

13.8 Exercises

1.Â

Without looking back at page Â§ , derive $w_{â^¥v}$ and $w_{âš¥v}$ and prove that they are orthogonal.

2.Â

Simplify the following operations, assuming Q,A â^ â„ MÃ—M .

a) $Q^{â^'T}$
b) $QQ^T Q^{-T} Q^T Q^{â^'1} QQ^{â^'1} Q^T$
c) $Q^{-T} QQQ^{â^'1} AAQ^T QA^{â^'1}$

3.Â

In section 13.1 , I defined the difference vector to be (b â^'Î²a). What happens if you define this vector as (Î²a â^' b)? Do the algebra to see if the projection formula changes.

4.Â

Why are orthogonal matrices called "pure rotation" matrices? Itâ€™s not because theyâ€™ve never committed sins; itâ€™s because they rotate but do not scale any vector to which they are applied. Prove this claim by comparing â^¥xâ^¥ to â^¥Qxâ^¥. (Hint: Itâ€™s easier to prove that the magnitudes-squared are equal.)

5.Â

Determine whether the following are orthogonal matrices.

a) $\begin{bmatrix} 1 & -\sqrt{3} \\ \sqrt{3} & 1 \end{bmatrix}$

b) $\frac{1}{5}\begin{bmatrix} 3 & 4 & 0 \\ -4 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix}$

c) $\begin{bmatrix} 1/\sqrt{2} & 1\sqrt{2} & 0 \\ -\sqrt{2}/6 & \sqrt{2}/6 & 2\sqrt{2}/3 \\ 2/3 & -2/3 & 2/3 \end{bmatrix}$

13.9 Answers

1.Â

You can check your proof on page Â§ .

2.Â

a) Q
b) $(Q^T)^2$
c) $Q^2 A$

3.Â

It doesnâ€™t; youâ€™ll arrive at the same projection formula.
4.Â

Hereâ€™s the proof; the key result at the end is that âˆ¥Qxâˆ¥ = âˆ¥xâˆ¥.

$$\begin{aligned} \text{âˆ¥Qxâˆ¥}^2 &= (Qx)^T (Qx) \\ &= x^T Q^T Qx \\ &= x^T x = \text{âˆ¥xâˆ¥}^2 \end{aligned}$$

5.Â

a) No (but try a scaling factor)
b) Yes
c) No (but change the final element to 1/3)

13.10 Code challenges

1.Â
Use code to confirm the sizes of Q and R based on Figure 13.7 .
2.Â
Implement the Gram-Schmidt algorithm to compute Q from a matrix A. You can check that your code is correct by (1) Confirming that $Q^T Q = I$ and (2) by computing Q from QR decomposition. I recommend starting with a square A so you donâ€™t need to worry about economy vs. full decomposition.

Note: When you compare your Q against the output of QR decomposition, you might find that some columns have the same values but opposite sign. For example, perhaps your Q will have a column [1 2 3] whereas the Q from Python/MATLAB is [-1 -2 -3]. There are fundamental sign uncertainties in many decompositions. The short version of the reason is that the vectors are crafted to be basis vectors for 1D subspaces, and thus the sign of the vector is simply a scalar. This will be more clear after learning about eigenvectors.
3.Â
Using your code above, compute Q on the following matrix (this was the example in section 13.5):

$$\begin{bmatrix} 1 & 1 & -2 \\ 3 & -1 & 1 \end{bmatrix}$$

The first two columns of your Q should match v_1 âˆ— and v_2 âˆ— in the text. But you will not get a third column of zeros; it will be some other vector that has magnitude of 1 and is not orthogonal to the previous two columns (easily confirmed: $Q^T Q$â‰ˆ I). What on Earth is going on here?!?!

13.11 Code solutions

1.Â
This oneâ€™s easy, so Iâ€™m not giving you code! You simply need to create random matrices of various sizes, and check the sizes of the resulting Q and R matrices.
2.Â
This is a tricky algorithm to implement. In fact, this exercise is one of the most difficult in this book.

Code blockÂ 13.5: Python

```
importÂ numpyÂ asÂ npÂ
Â
mÂ =Â 4Â
nÂ =Â 4Â
AÂ =Â np.random.randn(m,n)Â
QÂ =Â np.zeros((m,n))Â
Â
forÂ iÂ inÂ range(n):Â
Â
```

```
Â Â Q[:,i]Â =Â A[:,i]Â
Â
Â Â #Â orthogonalizeÂ
Â Â aÂ =Â A[:,i]Â #Â convenienceÂ
Â Â forÂ jÂ inÂ range(i):Â #Â onlyÂ toÂ earlierÂ colsÂ
Â Â Â Â qÂ =Â Q[:,j]Â #Â convenienceÂ
Â Â Â Â Q[:,i]=Q[:,i]-np.dot(a,q)/np.dot(q,q)*qÂ
Â
Â Â #Â normalizeÂ
Â Â Q[:,i]Â =Â Q[:,i]Â /Â np.linalg.norm(Q[:,i])Â
Â
#Â QRÂ
Q2,RÂ =Â np.linalg.qr(A)
```

Code blockÂ 13.6: MATLAB

```
mÂ =Â 4;Â
nÂ =Â 4;Â
AÂ =Â randn(m,n);Â
QÂ =Â zeros(m,n);Â
Â
forÂ i=1:nÂ %Â loopÂ throughÂ columnsÂ (n)Â
Â
Â Â Â Â Q(:,i)Â =Â A(:,i);Â
Â
Â Â Â Â %Â orthogonalizeÂ
Â Â Â Â ifÂ i>1Â
Â Â Â Â Â Â aÂ =Â A(:,i);Â %Â convenienceÂ
Â Â Â Â Â Â forÂ j=1:i-1Â %Â onlyÂ toÂ earlierÂ columnsÂ
Â Â Â Â Â Â Â Â qÂ =Â Q(:,j);Â %Â convenienceÂ
Â Â Â Â Â Â Â Â Q(:,i)Â =Â Q(:,i)Â -Â (aâ€™*q/(qâ€™*q))Â *Â q;Â
Â Â Â Â Â Â endÂ
Â Â Â Â endÂ
Â
Â Â Â Â %Â normalizeÂ
Â Â Â Â Q(:,i)Â =Â Q(:,i)Â /Â norm(Q(:,i));Â
endÂ
Â
%Â QRÂ
[Q2,R]Â =Â qr(A);
```

3.Â

The issue here is normalization of computer rounding errors. To see this, modify your algorithm so that column 3 of Q is not normalized.

You will find that both components of the third column have values close to zero, e.g., around $10^{âˆ’15}$. Thatâ€™s basically zero plus computer round error. The normalization step is making mountains out of microscopic anthills. Congratulations, you have just discovered one of the reasons why the "textbook" Gram-Schmidt algorithm is avoided in computer applications!

ChapterÂ 14
Least-squares

14.1 Introduction

The physical and biological world that we inhabit is a really, really, really complicated place. There are uncountable dynamics and processes and individuals with uncountable and mind-bogglingly complex interactions. How can we make sense of this complexity? The answer is we canâ€™t: Humans are terrible at understanding such enormously complex systems.

So we do the next-best thing, which is to develop simplified models of the most important aspects of the system under investigation, while ignoring or abstracting away the aspects that are less relevant.

This process leads to a model , which is a set of equations that allows scientists to isolate and understand the principles of the system under investigation. Thus, the goal of building models is not to replicate exactly the system under investigation, but instead to identity a simplified and low-dimensional representation that actually can be understood by humans or simulated by computers.

On the other hand, there is a lot of diversity in nature, and models should be sufficiently generic that they can be applied to different datasets. This means that the models must be flexible enough that they can be adjusted to different datasets without having to create a brand new model for each particular dataset.

This is why models contain both fixed features and free parameters. The fixed features are components of the model that the scientist imposes, based on scientific evidence, theories, and intuition (a.k.a. random guesses). Free parameters are variables that can be adjusted to allow the model to fit any particular data set. And this brings us to the primary goal of model-fitting: Find values for these free parameters that make the model match the data as closely as possible.

Here is an example to make this more concrete. Letâ€™s say you want to predict how tall someone is. Your hypothesis is that height is a result of the personâ€™s sex (that is, male or female), their parentsâ€™ height, and their childhood nutrition rated on a scale from 1-10. So, males tend to be taller, people born to taller parents tend to be taller, and people who ate healthier as children tend to be taller. Obviously, what really determines an adultâ€™s height is much more complicated than this, but we are trying to capture a few of the important factors in a simplistic way. We can then construct our model of height:

$$h = \beta_1 s + \beta_2 p + \beta_3 n + \epsilon \tag{14.1}$$

where h is the personâ€™s height, s is the sex, p is the parentsâ€™ height, and n is the childhood nutrition score. These are the fixed features of the modelâ€”they are in the model because I put them there, because I believe that they are important. On the other hand, it is obvious that these three factors do not exactly determine someoneâ€™s height; hence, we include the ð œ– as an "error term" or "residual" that captures all of the variance in height that the three other variables cannot explain.

But hereâ€™s the thing: I donâ€™t know how important each of these factors is. So I specify that the model needs to include these terms, and I will let the data tell me how important each term is. Thatâ€™s why each fixed feature is scaled by a Î² parameter. Those Î²s are the free parameters. For

example, if $\hat{I}^2_3$ is large, then nutrition is very important for determining someoneâ€™s adult height; and if $\hat{I}^2_3$ is close to zero, then the data are telling me that childhood nutrition is not important for determining height.

Of course, that leads to the question of how you find the model parameters that make the model match the data as closely as possible. Randomly picking different values for the $\hat{I}^2$s is a terrible idea; instead, we need an algorithm that will find the best parameters given the data. This is the idea of model fitting. The most commonly used algorithm for fitting models to data is linear least-squares, and that is what you will learn in this chapter.

14.2 The five steps of model-fitting

The process of model-fitting in statistics can be broken down into five steps.

Step 1 Determine an equation or set of equations that comprise your model. In practice, you would often use existing equations or slight modifications of existing equations. I already showed you an example above: Our hypothesis about important factors that predict height were converted into Equation 14.1 .

Step 2 Work the data into the model. You get existing data from a database or by collecting data in a scientific experiment. Or you can simulate your own data. I made up the data in Table 14.1 for illustration purposes.

lightgray ID	Height	Sex	Parentsâ€™ height	Nutrition
1	180	M	175	8
2	170	F	172	6
⋮	⋮	⋮	⋮	⋮
10	176	F	189	7

TableÂ 14.1: Example data table. Height is in cm.

But this is not the format that we need the data in. We need to map the data in this table into the model. That means putting these numbers inside the equation from step 1. The first row of data, converted into an equation, would look like this:

[1]Binary variables are often "dummy-coded," meaning that female=0 and male=1 (this mapping will make positive β correspond to taller adults).

$$180 = \beta_1 1 + \beta_2 175 + \beta_3 8$$

One row of data gives us one equation. But we have multiple rows, corresponding to multiple individuals, and so we need to create a set of equations. And because those equations are all linked, we can consider them a system of equations. That system will look like this:

$$\left\{ \begin{array}{rcl} 180 &=& \beta_1 1 + \beta_2 175 + \beta_3 8 \\ 170 &=& \beta_1 0 + \beta_2 172 + \beta_3 6 \\ & \vdots & \\ 176 &=& \beta_1 0 + \beta_2 189 + \beta_3 7 \end{array} \right\} \quad (14.2)$$

Notice that each equation follows the same "template" from Equation 14.1 , but with the numbers taken from the data table. (The statistically astute reader will notice the absence of an intercept term, which captures expected value of the data when the predictors are all equal to zero. Iâ€™m omitting that for now, but will discuss it later.)

Step 3 The system of equations above should active your happy memories of working through Chapter 10 on solving systems of equations. Armed with those memories, you might have already guessed what step 3 is: convert the system of equations into a single matrix-vector equation. Itâ€™s exactly the same concept that you learned previously: Split up the coefficients, variables, and constants, and put those into matrices and vectors.

$$\begin{bmatrix} 1 & 175 & 8 \\ 0 & 172 & 6 \\ 0 & 189 & 7 \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix} = \begin{bmatrix} 180 \\ 170 \\ 176 \end{bmatrix} \tag{14.3}$$

In previous chapters, I would have called this Ax = b. The standard statistics lingo uses different letters, although the concepts and algorithms are the same. Iâ€™ll explain the statistics terminology in a few pages, but for now, suffice it to say that this equation is now called XÎ² = y.[2] X is the matrix of coefficients, Î² is the vector of the unknown free parameters that we want to solve for, and y is the vector of constants (in this example, height measured in cm).

[2] Note that Î² here is a vector, although I have previously written that Greek characters indicate scalars.

Step 4 Solve for Î², which is also called fitting the model, estimating the parameters, computing the best-fit parameters, or some other related terminology. The rest of the chapter is focused on this step.

Step 5 Statistically evaluate the model: is it a good model? How well does it fit the data? Does it generalize to new data or have we over-fit our sample data? Do all model terms contribute or should some be excluded? This step is all about inferential statistics, and it produces things like p-values and t-values and F-values. Step 5 is important for statistics applications, but is outside the scope of this book, in part because it relies on probability theory, not on linear algebra. Thus, step 5 is not further discussed here.

14.3 Terminology

It is unfortunate and needlessly confusing that statisticians and linear algebrists use different terminology for the same concepts. Terminological confusion is a common speed-bump to progress in science and cross-disciplinary collaborations. Nonetheless, there are worse things in human civilization than giving a rose two names. The table below introduces the terminology that will be used only in this chapter.

lightgray LinAlg	Stats	Description
Ax = b	XÎ² = y	General linear model (GLM)
A	X	Design matrix (columns = independent variables, predictors, regressors)
x	Î²	Regression coefficients or beta parameters
b	y	Dependent variable, outcome measure, data

TableÂ 14.2: Translating between linear algebra and statistics terminology.

The final point I want to make here is about the term "linear" in linear least-squares. Linear refers to the way that the parameters are estimated; it is not a restriction on the model. The model may contain nonlinear terms and interactions; the restriction for linearity is that the coefficients (the free parameters) scalar-multiply their variables and sum to predict the dependent variable. Basically that just means that itâ€™s possible to transform the system of equations into a matrix equation. That restriction allows us to use linear algebra methods to solve for Î²; there are also nonlinear methods for estimating parameters in nonlinear models.

To make sure this point is clear, consider the following model, which contains two nonlinearities:

$$h = \beta_1 s + \beta_2 \sqrt{\beta_3} p + \beta_3 n^3 + \epsilon \quad (14.4)$$

One nonlinearity is in the regressors ($\hat{I}^2_2 \sqrt{\beta_3}$), and one is in a predictor (n^3). The latter is no problem; the former prevents linear least-squares from fitting this model (there are nonlinear alternatives that you would learn about in a statistics course).

14.4 Least-squares via left inverse

There is actually one solution to the general linear model equation, but it is insightful to arrive at that solution from different perspectives. In this and the next sections, we will start from algebra (this section), geometry (next section), and row-reduction (next-next section), and arrive at the same conclusion.

The problem we face is simple: Solve for $\hat{I}^2$ in $X\hat{I}^2 = y$. If X were a square matrix, we would obtain the solution by pre-multiplying both sides by $X^{â^'1}$. However, design matrices are typically tall matrices, because there are typically many more observations than there are variables. Using our example of predicting height, we might have data from 1000 individuals and our model contains 3 predictors. Thus, X would be a 1000Ã—3 matrix. (Deep learning applications often have wide matricesâ€”more variables than observationsâ€”and this requires regularization and other tricks to make the problem solvable. But nearly all statistics applications involve tall design matrices.)

You know what the solution is: instead of the full inverse, we use the left-inverse. Thus, the solution to the least squares problem is:

$$\begin{aligned} X\hat{I}^2 &= y \\ (X^T X)^{â^'1} X^T X\hat{I}^2 &= (X^T X)^{â^'1} X^T y \\ \hat{I}^2 &= (X^T X)^{â^'1} X^T y \end{aligned} \quad (14.5)$$

When is Equation 14.5 a valid solution? Iâ€™m sure you guessed itâ€”when X is full column-rank, which means the design matrix contains a linearly independent set of predictor variables. If the design matrix has repeated columns, or if one predictor variable can be obtained from a linear combination of other predictor variables, then X will be rank-deficient and the solution is not valid. In statistics lingo, this situation is called "multicollinearity."

When is the solution exact? I wrote in section 8.11 that one of the fundamental questions in linear algebra is whether some vector is in the column space of some matrix. Here you can see an application of that question. We want to know whether vector y is in the column space of matrix X. Theyâ€™re both in $â„^M$, because they both have M elements in the columns, so itâ€™s a valid question to ask. But hereâ€™s the thing: y has M elements and X has N columns, where N << M (e.g., M = 1000 vs. N = 3). In this example, we are asking whether a specific vector is inside a 3D subspace embedded in ambient $â„^{1000}$.

The answer is nearly always No, $y \notin C(X)$. Even for a good model, a tiny bit of noise in the data would push vector y out of the column space of X. What to do here?

The solution is to think about a slightly different question: Is there a y such that y really is in the column space of X? Then we would call the parameters $\hat{I}^2$. Obviously, some $\hat{I}^2$ and y exist, and also obviously, we want to find the y that is as close to y as possible. That leads to the full formulation of the general linear model and some related equations.

$$y = X\hat{I}^2 + ð \quad œ– \quad (14.6)$$

$$y = y + \delta_{\text{œ–}} \quad (14.7)$$

$$y = X\hat{I}^2 \quad (14.8)$$

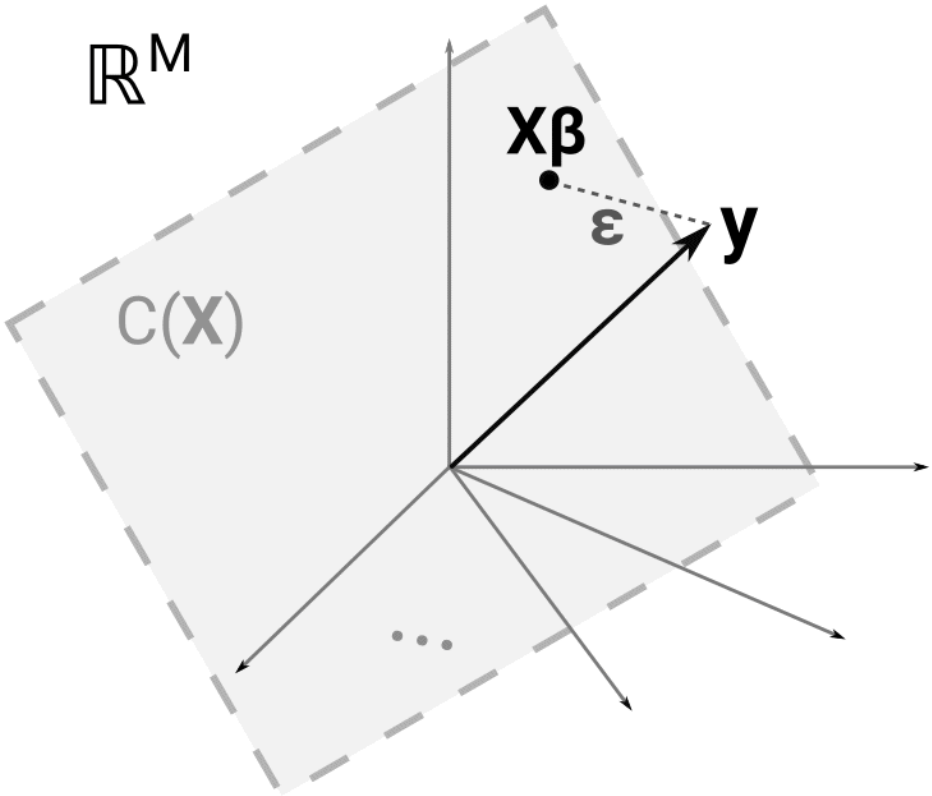
ð œ– is the residual, what you need to add to y to get it into the column space of the design matrix. The better the model fits the data, the closer y is to the column space of X, and thus the smaller the ð œ–. Keep in mind that ð œ– is not in the data that you measured, and itâ€™s not in the model that you specify. Instead, itâ€™s what you need to add to y to reach the column space of X.

There is more to say about this ð œ– vector (section 14.7), but first I want to prove to you that the least squares equation minimizes ð œ–. And to do that, we need to re-derive the least-squares solution from a geometric perspective.

14.5 Least-squares via orthogonal projection

Imagine an ambient â„ ^{M} space, and a subspace inside that ambient space, whose basis vectors are the columns of the design matrix. I will illustrate this using a 2D subspace because it is easy to visualize (Figure 14.1), but the actual dimensionality of this subspace corresponds to the number of columns in the design matrix, which means the number of predictor variables in your statistical model.

The measured data in y form a vector in â„ ^{M} . As discussed in the previous chapter, it is sometimes easier to conceptualize y as a coordinate in space rather than a vector from the origin.



FigureÂ 14.1: The geometry of least-squares via orthogonal projection.

When working with real data, it is incredibly unlikely that y is exactly in the column space of X. If fact, if y âˆˆ C(X), then probably the model is way too complicated or the data are way too simple. The goal of science is to understand things that are difficult to understand; [3] if your model explains a phenomenon 100%, then itâ€™s not difficult to understand, and you should try to work on a harder problem.

[3] n.b. Iâ€™m kindof joking here. Sortof.

But the column space is important because that is the mathematical representation of our theory of the world. So then the question is, What is the coordinate that is as close as possible to the data vector while still being inside the subspace? The answer to that question comes from the orthogonal projection of the vector onto the subspace. That orthogonal projection is vector ðœ–= y âˆ’ XÎ², which is orthogonal to the column space of X.

$$X^T \text{ðœ–} = 0 \quad (14.9)$$

$$X^T (y - X\hat{\beta}) = 0 \quad (14.10)$$

$$X^T y - X^T X\hat{\beta} = 0 \quad (14.11)$$

$$X^T X\hat{\beta} = X^T y \quad (14.12)$$

$$\hat{\beta} = (X^T X)^{-1} X^T y \quad (14.13)$$

And amazingly (though perhaps not surprisingly), weâ€™ve arrived back at the same solution as in the previous section.

We also see that the design matrix X and the residuals vector ðœ– are orthogonal. Geometrically, that makes sense; statistically, it means that the prediction errors should be unrelated to the model, which is an important quality check of the model performance.

-|*|- **Reflection** You might have noticed that Iâ€™m a bit loose with the plus signs and minus signs. For example, why is ðœ– defined as y âˆ’ XÎ² and not XÎ² âˆ’ y? And why is ðœ– added in Equation 14.6 instead of subtracted? Sign-invariance often rears its confusing head in linear algebra, and in many cases, it turns out that when the sign seems like itâ€™s arbitrary, then the solution ends up being the same regardless. Also, in many cases, there are coefficients floating around that can absorb the signs. For example, you could flip the signs of all the elements in ðœ– to turn vector âˆ’ðœ– into +ðœ–. -|*|-

14.6 Least-squares via row-reduction

Now weâ€™re going to derive the least-squares formula again, this time using row-reduction and Gauss-Jordan elimination. It may seem gratuitous to have yet another derivation of least-squares, but this section will help link concepts across different chapters of this book, and therefore has high conceptual/educational value (IMHO).

Recall from Chapter 10 that we can solve a system of equations by performing row-reduction on a matrix of coefficients augmented by the constants. And then in Chapter 12 you saw how to apply that method to computing the matrix inverse. Letâ€™s see if we can apply that same concept here to solve for Î² in our statistical model.

$$\text{rref}([\,\mathbf{X} \mid \mathbf{y}\,]) \quad \Rightarrow \quad [\,\mathbf{I} \mid \beta\,] \quad (14.14)$$

Unfortunately, life does not always conform to expectations. Take a moment to think about the problem with the conclusion above.

The problem is not the augmented matrix: X and y both have M rows, so the augmenting is valid. And row-reduction is also valid. However, [X | y] will have N + 1 columns and will have a rank of N + 1 < M. Thus, the actual outcome of RREF will be

$$\text{rref}([\,\mathbf{X} \mid \mathbf{y}\,]) \quad \Rightarrow \quad \begin{bmatrix}\mathbf{I}\\ \mathbf{0}\end{bmatrix} \tag{14.15}$$

In other words, a tall matrix with the identity matrix I_N on top and all zeros underneath. This is not useful; we need to re-think this.

The solution comes from pre-multiplying the general linear model equation by X^T :

$$\mathbf{X}^{\mathrm{T}}\mathbf{X}\beta = \mathbf{X}^{\mathrm{T}}\mathbf{y} \tag{14.16}$$

Equation 14.16 is often called the "normal equation." (I know what youâ€™re thinking and I agree: it looks less normal than the equation we derived it from. But thatâ€™s the terminology.)

Now we can go back to Gauss-Jordan and get a sensible result.

$$\text{rref}(\,[\,\mathbf{X}^{\mathrm{T}}\mathbf{X} \mid \mathbf{X}^{\mathrm{T}}\mathbf{y}\,]) \quad \Rightarrow \quad [\,\mathbf{I} \mid \beta\,] \tag{14.17}$$

Letâ€™s think about the sizes. $X^T X$ will be NÃ—N and thus the augmented matrix will be of size NÃ—(N + 1), in other words, a wide, full row-rank, matrix. That will give us our desired Î² on the right.

The reason why this works comes from thinking about the row spaces of X and $X^T X$. Remember from Chapter 8 (section 8.5) that these two matrices have the same row space, and that $X^T X$ is a more compact representation of the space spanned by rows of X. Furthermore, assuming that X has linearly independent columns (which is an assumption weâ€™ve been making in this entire chapter), then $X^T X$ spans all of â„ ^{N} , which means that any point in â„ ^{N} is guaranteed to be in the column space of $X^T X$. And $X^T y$ is just some point in â„ ^{N} , so itâ€™s definitely going to be in the column space of $X^T X$. Therefore, Î² contains the coefficients on the columns of matrix $X^T X$ to get us exactly to $X^T y$. Note that with the Gauss-Jordan approach to solving least-squares using the normal equations, we never leave the N-dimensional subspace of â„ ^{N} , so the question of whether y â^ˆ C(X) doesnâ€™t even come up.

However, we still need to compute ð œ– to make the GLM equation correct. The reason is that Î² is a solution to the normal equation, but we want to find the solution to y, which is not the same thing as $X^T y$. Fortunately, thatâ€™s an easy rearrangement of Equation 14.6 :

$$\epsilon = \mathbf{X}\beta - \mathbf{y} \tag{14.18}$$

The statistical interpretation of Equation 14.18 is that the residual is the difference between the model-predicted values (XÎ²) and the observed data (y).

By the way, we can also arrive at the normal equations by starting from the known solution of the left-inverse, and then pre-multiplying by $X^T X$:

$$\begin{aligned} \hat{\beta} &= (X^T X)^{-1} X^T y \\ X^T X\hat{\beta} &= (X^T X)(X^T X)^{-1} X^T y \qquad (14.19)\\ X^T X\hat{\beta} &= X^T y \qquad (14.20) \end{aligned}$$

14.7 Model-predicted values and residuals

Let me bring your attention back to Figure 14.1 . The data vector y is unlikely to be exactly in the column space of the design matrix X, and vector ð œ– gets us from the data into the subspace spanned by the design matrix. We also know that ð œ– is defined as being orthogonal to the column space of X.

Statistically, ð œ– is the residual variance in the data that the model cannot account for. We can also write this as follows:

$$\epsilon = \mathbf{X}\beta - \mathbf{y} \tag{14.21}$$

When is the model a good fit to the data? It is sensible that the smaller ð œ– is, the better the model fits the data. In fact, we donâ€™t really care about the exact values comprising ð œ– (remember, there is one ð œ–$_i$ for each data value y_i); instead, we care about the norm of ð œ–.

$$\|\epsilon\|^2 = \|\mathbf{X}\beta - \mathbf{y}\|^2 \tag{14.22}$$

Now we can re-frame the goal of model-fitting in a slightly different way: Find the values in vector Î² that minimize both sides of Equation 14.22 . This is a standard optimization problem, and can be expressed as

$$\min_{\beta} \|\mathbf{X}\beta - \mathbf{y}\|^2 \tag{14.23}$$

The solution to this problem is obtained by computing the derivative and setting that equal to zero:

$$0 = \frac{d}{d\beta} \hat{a}\check{\text{¥}}X\hat{I}^2 \hat{a}\check{\text{ }}' y\hat{a}\check{\text{¥}}^2 \qquad = 2X^T (X\hat{I}^2 \hat{a}\check{\text{ }}' y) \tag{14.24}$$

$$0 = X^T X\hat{I}^2 \hat{a}\check{\text{ }}' X^T y \tag{14.25}$$

$$X^T X\hat{I}^2 = X^T y \tag{14.26}$$

$$\hat{I}^2 = (X^T X)^{\hat{a}\check{\text{ }}'1} X^T y \tag{14.27}$$

And here we are, back at the same solution to least-squares that weâ€™ve obtained already using three different approaches. And now Iâ€™ve slipped in a fourth method to derive the least-squares equation, using a bit of calculus and optimization. (If your calculus is rusty, then donâ€™t worry if the first equation is nebulous. The idea is to take the derivative with respect to Î² using the chain rule.)

14.8 Least-squares example

Doing statistics in the real-world is usually not so simple as defining the model, fitting the parameters to data, and then going out for sushi to celebrate a job-well-done. In practice, statisticians will often begin with simple models and then add complexity to the models as it becomes clear that the simple models are inappropriately simple for the data, and according to the available theoretical frameworks. I want to give you a taste of that process.

We begin with a set of numbers. Letâ€™s call it set D for data. We assume that the order is meaningful; perhaps these are data values from a signal being recorded over time.

$$D = \{-4, 0, -3, 1, 2, 8, 5, 8\}$$

Iâ€™m going to start by assuming that the signal has a constant value. Thus, my hypothesis at this point is that there is only one meaningful signal value, and the divergences at each time point reflect errors (for example, sensor noise). Thus, my model is:

$$d = \beta 1 + \epsilon \tag{14.28}$$

That was step 1. Step 2 is to work the data into the model. That will give a series of equations that looks like this:

$$\left\{\begin{array}{rcl} -4 &=& \beta 1 \\ 0 &=& \beta 1 \\ -3 &=& \beta 1 \\ 1 &=& \beta 1 \\ 2 &=& \beta 1 \\ 8 &=& \beta 1 \\ 5 &=& \beta 1 \\ 8 &=& \beta 1 \end{array}\right\} \tag{14.29}$$

Why is there a "1" next to the Î²? My hypothesis is that a single number explains the data. But I donâ€™t know a priori what that number should be. Thatâ€™s what the Î² is for. Now, that number could be anything else, and Î² would scale to compensate. For example, letâ€™s say the average data value is 2. With the coefficient set to 1, then Î² = 2, which is easy to interpret. However, if I were to set the coefficient to 9.8, then Î² = .20408.... And then the interpretation would be that the average value of the signal is Î²Ã—9.8. That is mathematically correct, but difficult and awkward to interpret. Thatâ€™s why we set the coefficient value to 1 (in a few pages, I will start calling this column the "intercept").

Now for step 3: Convert the set of equations into a matrix-vector equation.

$$\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} [\beta] = \begin{bmatrix} -4 \\ 0 \\ -3 \\ 1 \\ 2 \\ 8 \\ 5 \\ 8 \end{bmatrix} \tag{14.30}$$

Step 4 is to fit the model.

$$\hat{\beta} = (X^T X)^{â^'1} X^T y \tag{14.31}$$

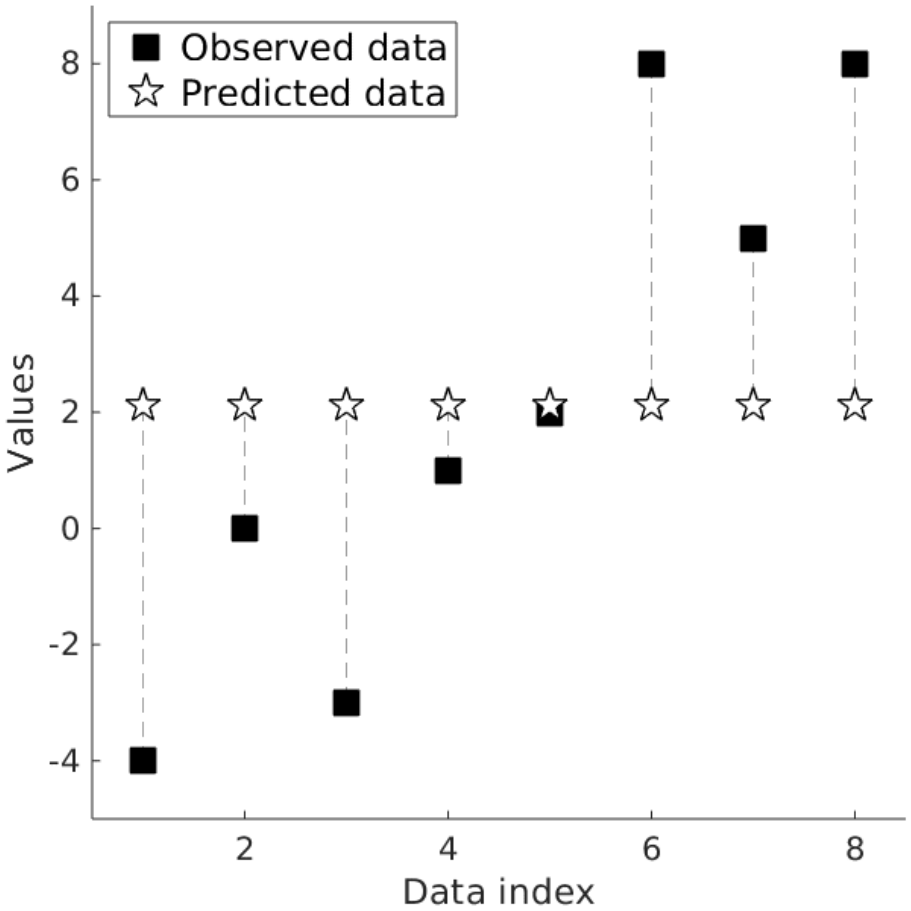
$$\hat{\beta} = 8^{â^'1} Ã— 17 \tag{14.32}$$

$$\hat{\beta} = \frac{17}{8} = 2.125 \tag{14.33}$$

Did you notice what happened here? We ended up summing all the data points (dot product between the numbers and a vector of 1â€™s) and dividing by the number of elements.[4] Thatâ€™s literally the arithmetic mean. So weâ€™ve re-derived the average from a statistics/linear algebra perspective, where our model is that the data are meaningfully characterized by a single number.

[4] Another good reason to set the coefficients to 1.

Itâ€™s always a good idea to visualize data and models. Letâ€™s see what they look like (Figure 14.2).



FigureÂ 14.2: Observed and predicted data. Black squares show the data (y) and white stars show the model-predicted data (Å· = XÎ²). The dashed gray lines were added to facilitate comparing data values to their predicted values.

What do you think of the comparison of predicted and actual data shown in Figure 14.2 ? I think the model looks pretty awful, to be honest. The data clearly show an upward trend, which the model cannot capture. Letâ€™s try changing the model. (Important note: Iâ€™m skipping step 5, which would involve formal tests of the model.)

Instead of predicting a single data value, letâ€™s predict that the data values change over the x-axis (the statistics term for this is a "linear trend"). The new model equation from step 3 will look like this:

$$\begin{bmatrix}1\\2\\3\\4\\5\\6\\7\\8\end{bmatrix}[\beta] = \begin{bmatrix}-4\\0\\-3\\1\\2\\8\\5\\8\end{bmatrix} \tag{14.34}$$

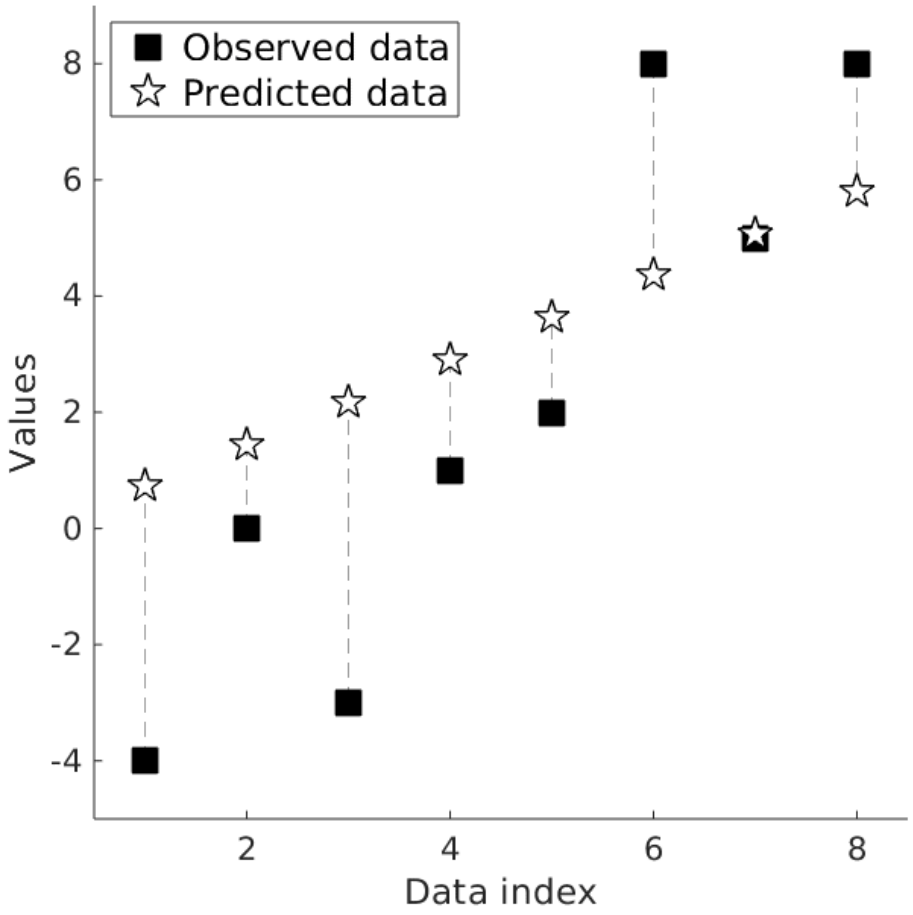
Note that we are no longer predicting the average value; we are now predicting a slope. Letâ€™s see what the parameter is:

$$\hat{\beta} = (X^T X)^{-1} X^T y \quad (14.35)$$

$$\hat{\beta} = 204^{-1} \times 148 \quad (14.36)$$

$$\hat{\beta} = \frac{204}{148} = .7255 \quad (14.37)$$

Figure 14.3 shows the same data with the new model-predicted data values.



FigureÂ 14.3: Our second modeling attempt.

It looks better compared to Figure 14.2 , but still not quite right: The predicted data are too high (over-estimated) in the beginning and too low (under-estimated) in the end. The problem here is that the model lacks an intercept term.

The intercept is the expected value of the data when all other parameters are set to 0. Thatâ€™s not the same thing as the average value of the data. Instead, it is the expected value when all model parameters have a value of zero. Thinking back to the example model in the beginning of this chapter (predicting adult height as a function of sex, parentsâ€™ height, and childhood nutrition), the intercept term captures the expected height of an adult female whose parentsâ€™ height is zero cm and who had a score of 0 on childhood nutrition. Not exactly a plausible situation. So why do these models need an intercept term?

The reason is that without an intercept, the model is forced to include the origin of the graph (X,Y=0,0). This caused the model in Figure 14.3 to be a poor fit. The intercept term allows the best-fit line to shift off the origin (that is, to intersect the Y-axis at any value other than zero). Thus, in many statistical models, the intercept is not necessarily easily interpretable, but is necessary for proper model fitting (unless all data and regressors are mean-subtracted, in which case the best-fit line touches the origin).

Now letâ€™s expand the model to include an intercept term, which involves including an extra column of 1â€™s in the design matrix. Weâ€™ll also need two Î²s instead of one.

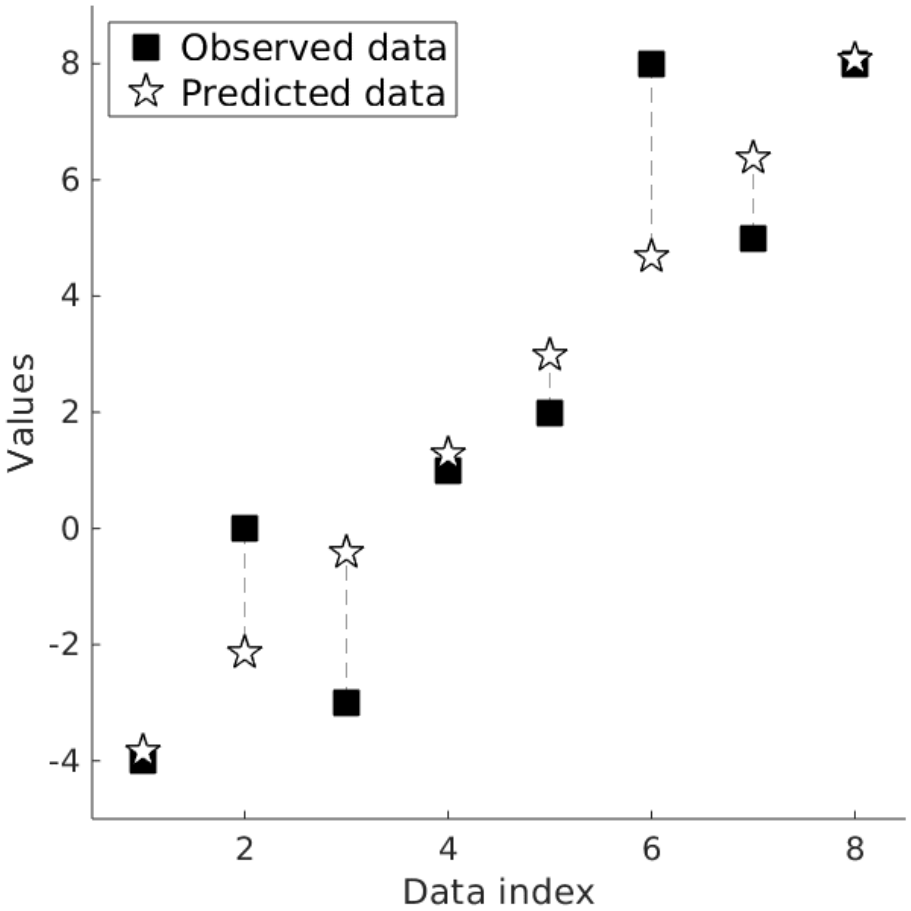
$$\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \\ 1 & 6 \\ 1 & 7 \\ 1 & 8 \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} -4 \\ 0 \\ -3 \\ 1 \\ 2 \\ 8 \\ 5 \\ 8 \end{bmatrix} \tag{14.38}$$

The arithmetic gets a bit more involved, but produces the following.

$$\hat{\beta} = (X^T X)^{-1} X^T y \tag{14.39}$$

$$\hat{\beta} = \begin{bmatrix} 8 & 36 \\ 36 & 204 \end{bmatrix}^{-1} \begin{bmatrix} 17 \\ 148 \end{bmatrix} \tag{14.40}$$

$$\hat{\beta} = \begin{bmatrix} -5.5357 \\ 1.7024 \end{bmatrix} \tag{14.41}$$



FigureÂ 14.4: Our final model of this dataset.

Iâ€™m sure you agree that this third model (linear trend plus intercept) fits the data reasonably well (Figure 14.4). There are still residuals that the model does not capture, but these seem like they could be random fluctuations. Notice that the best-fit line does not go through the origin of the graph. We donâ€™t see exactly where the line will cross the y-axis because the x-axis values start at 1. However, $\hat{I}^2_1$, the intercept term, predicts that this crossing would be at y = âˆ’5.5357, which looks plausible from the graph.

The final thing I want to do here is confirm that ð œ– âŠ¥ XÎ². The two columns below show the residual and predicted values, truncated at 4 digits after the decimal point.

ϵ	$\mathbf{X}\beta$
0.1667	−3.8333
−2.1310	−2.1310
2.5714	−0.4286
0.2738	1.2738
0.9762	2.9762
−3.3214	4.6786
1.3810	6.3810
0.0833	8.0833

$$\epsilon^{\mathrm{T}}(\mathbf{X}\beta) = 0.0000000000000142...$$

It may seem like the dot product is not exactly zero, but it is 14 orders of magnitude smaller than the data values, which we can consider to be zero plus computer rounding error.

14.9 Code challenges

Imagine you work as a data scientist in a company that sells widgets online. The company gives you a dataset of information from 1000 sales, which includes the time of the sale (listed in hours of the day using a 24-hour clock, e.g., 15 = 3pm), the age of the buyer (in years), and the number of widgets sold. The data are available at sincxpress.com/widget_data.txt (Note: these are data I made up for this exercise.)

1.Â Explain and write down a mathematical model that is appropriate for this dataset.

2.Â Write the matrix equation corresponding to the model, and describe the columns in the design matrix.

3.Â Compute the model coefficients using the least-squares algorithm in MATLAB or Python. You can also divide the Î² coefficients by the standard deviations of the corresponding independent variables, which puts the various model terms in the same scale and therefore more comparable.

4.Â Produce scatter plots that visualize the relationship between the independent and dependent variables.

5.Â One measure of how well the model fits the data is called R^2 ("R-squared"), and can be interpreted as the proportion of variance in the dependent variable that is explained by the design matrix. Thus, an R^2 of 1 indicates a perfect fit between the model. The definition of R^2 is

$$R^2 = 1 - \frac{\sum_i \epsilon_i^2}{\sum_i (y_i - \bar{y})^2} \tag{14.42}$$

ð œ–$_i$ is the error (residual) at each data point that was introduced earlier, and È³ is the average of all elements in y.

Compute R^2 for the model to see how well it fits the data. Are you surprised, based on what you see in the scatterplots?

14.10 Code solutions

1.Â Note: There are multiple correct answers to this problem.

A simple model would predict that time and age are both linear predictors of widget purchases. There needs to be an intercept term, because the average number of widgets purchased is greater than zero. The variables could interact (e.g., older people buy widgets in the morning while younger people buy widgets in the evening), but Iâ€™ve excluded interaction terms in the interest of brevity.

Thus, a basic model would look like this:

$$y = \beta_1 + \beta_2 \mathbf{t} + \beta_3 \mathbf{a}$$

y is the number of widgets sold, $\hat{I}^2_1$ is the intercept, t is the time of day, and a is the age.

2.Â

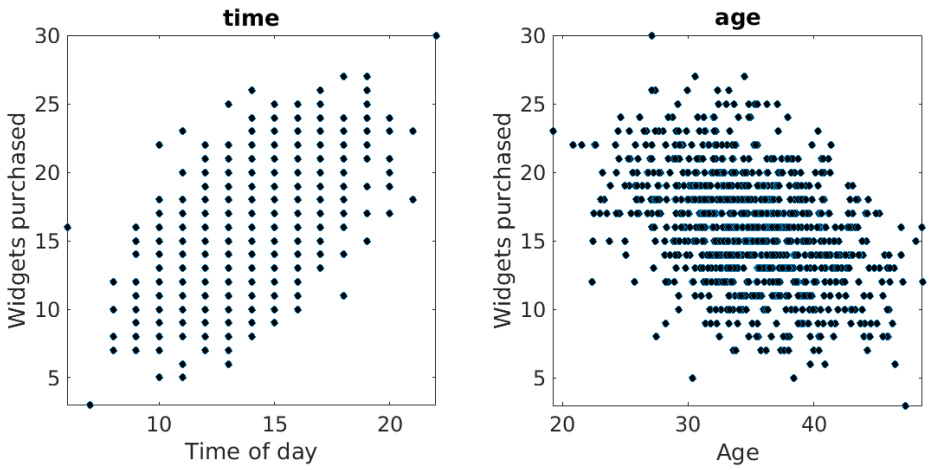
The matrix equation is XÎ² = y. X has three columns: intercept, which is all 1â€™s; time of day; age.

3.Â

Python and MATLAB code are on the following page.

Code blockÂ 14.1: Python

```
importÂ numpyÂ asÂ npÂ
Â
#Â loadÂ theÂ dataÂ
dataÂ =Â np.loadtxt(fname=â€™widget_data.txtâ€™,Â
Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â delimiter=â€™,â€™)Â
Â
#Â designÂ matrixÂ
XÂ =Â np.concatenate((np.ones((1000,1)),Â
Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â
```



figureImage

for question 1

Code blockÂ 14.3: Python

```
importÂ matplotlib.pyplotÂ asÂ pltÂ
fig,axÂ =Â plt.subplots(1,2,figsize=(8,4))Â
ax[0].plot(X[:,1],y,â€™oâ€™,markerfacecolor=â€™kâ€™)Â
ax[0].set_title(â€™TimeÂ variableâ€™)Â
ax[0].set_xlabel(â€™TimeÂ ofÂ dayâ€™)Â
ax[0].set_ylabel(â€™WidgetsÂ purchasedâ€™)Â
Â
ax[1].plot(X[:,2],y,â€™oâ€™,markerfacecolor=â€™kâ€™)Â
ax[1].set_title(â€™AgeÂ variableâ€™)Â
ax[1].set_xlabel(â€™Ageâ€™)Â
ax[1].set_ylabel(â€™WidgetsÂ purchasedâ€™)Â
plt.show()
```

Code blockÂ 14.4: MATLAB

```
subplot(121)Â
plot(X(:,2),y,â€™oâ€™,â€™markerfacecolorâ€™,â€™kâ€™)Â
axisÂ square,Â title(â€™TimeÂ variableâ€™)Â
xlabel(â€™TimeÂ ofÂ dayâ€™)Â
ylabel(â€™WidgetsÂ purchasedâ€™)Â
subplot(122)Â
plot(X(:,3),y,â€™oâ€™,â€™markerfacecolorâ€™,â€™kâ€™)Â
axisÂ square,Â title(â€™AgeÂ variableâ€™)Â
xlabel(â€™Ageâ€™),Â ylabel(â€™WidgetsÂ purchasedâ€™)
```

5.Â

The model accounts for 36.6% of the variance of the data. That seems plausible given the variability in the data that can be seen in the graphs.

Code blockÂ 14.5: Python

```
yHatÂ =Â X@betaÂ
r2Â =Â 1Â -Â np.sum((yHat-y)**2)Â
Â Â Â Â Â Â Â /Â np.sum((y-np.mean(y))**2)
```

Code blockÂ 14.6: MATLAB

```
yHatÂ =Â X*beta;Â
r2Â =Â 1Â -Â sum((yHat-y).^2)Â ...Â
Â Â Â Â Â /Â sum((y-mean(y)).^2);
```

ChapterÂ 15
Eigendecomposition

15.1 What are eigenvalues and eigenvectors?

A typical interaction between a student and a teacher about eigendecomposition goes something like this:

- Student: Honored and respected professor: What is an eigenvalue?
- Teacher: Courteous and ambitious student: Itâ€™s the thing that you use to find an eigenvector.
- Student: Oh. OK, but then whatâ€™s an eigenvector?
- Teacher: Itâ€™s the thing you get when you know the eigenvalue.
- Student:
- Teacher:
- Student: "Crystal clear." Iâ€™ll go spend some time on YouTube.

---

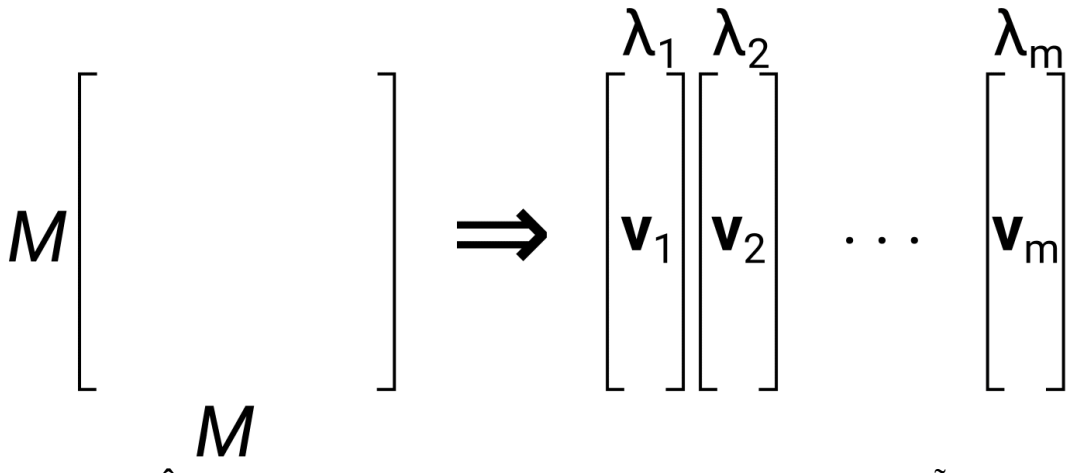
[1] There are myriad explanations of eigenvectors and eigenvalues, and in my experience, most students find most explanations incoherent on first impression. In this section, I will provide three explanations that I hope will build intuition; additional explanations and insights will come in subsequent sections as well as subsequent chapters.

[1] The terms eigenvalue decomposition, eigenvector decomposition, and eigendecomposition are all used interchangeably; here I will use eigendecomposition.

But there are two key properties to know about eigendecomposition before we get to interpretations. First, eigendecomposition is defined only for square matrices. They can be singular or invertible, symmetric or triangular or diagonal; but eigendecomposition can be performed only on square matrices. All matrices in this chapter are square (the singular value decomposition is defined for any sized matrix, and is the main topic of the next chapter).

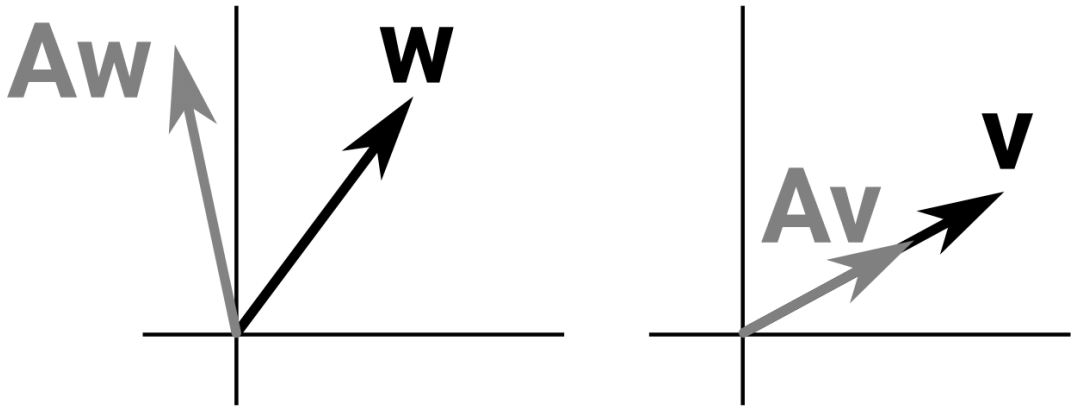
Second, the purpose of eigendecomposition is to extract two sets of features from a matrix: eigenvalues and eigenvectors. Needless to say, eigenvalues are scalars and eigenvectors are vectors. Typically, eigenvalues are labeled Î» and eigenvectors are labeled v.

An MÃ—M matrix has M eigenvalues and M eigenvectors. The eigenvalues and eigenvectors are paired, meaning that each eigenvalue has an associated eigenvector. There are occasional points of additional complexity, for example with repeated eigenvalues; weâ€™ll get to that later. But in general, each eigenvalue has an associated eigenvector (Figure 15.1).



FigureÂ 15.1: The big picture of eigendecomposition. Each MÃ—M matrix "hides" M eigenvalues and M associated eigenvectors; eigendecomposition is the process of revealing the matrixâ€™s deepest secrets.

Geometric interpretation One way to think about matrix-vector multiplication is that matrices act as input-output transformers (section 6.5): Vector w goes in, and vector Aw = y comes out. In the overwhelming vast majority of possible combinations of different matrices A and vectors w, the resulting vector y will point in a different direction than w. In other words, A rotates the vector. Figure 15.1 (left panel) shows an example.



FigureÂ 15.2: Vector w is not an eigenvector of matrix A, but vector v is an eigenvector of A. Its associated eigenvalue is Î» = .5.

But something is different in the right-side panel of Figure 15.1 : The output vector points in the same direction as the input vector. The matrix did not change the direction of the vector; it merely scaled it down.

In other words, the matrix-vector multiplication had the same effect as scalar-vector multiplication. The entire matrix is acting on the vector as if that matrix were a single number. That might not sound so impressive in a 2Ã—2 example, but the same concept

applies to a 10^{10} Ã—10^{10} (total of 10^{100} [a googel] elements!) matrix. If we could find that single number that has the same effect on the vector as does the matrix, we would be able to write the following equality, which is also known as the fundamental eigenvalue equation: _____

Eqn. box title=Eigenvalue equation

$$Av = Î»v \quad (15.1)$$

_____

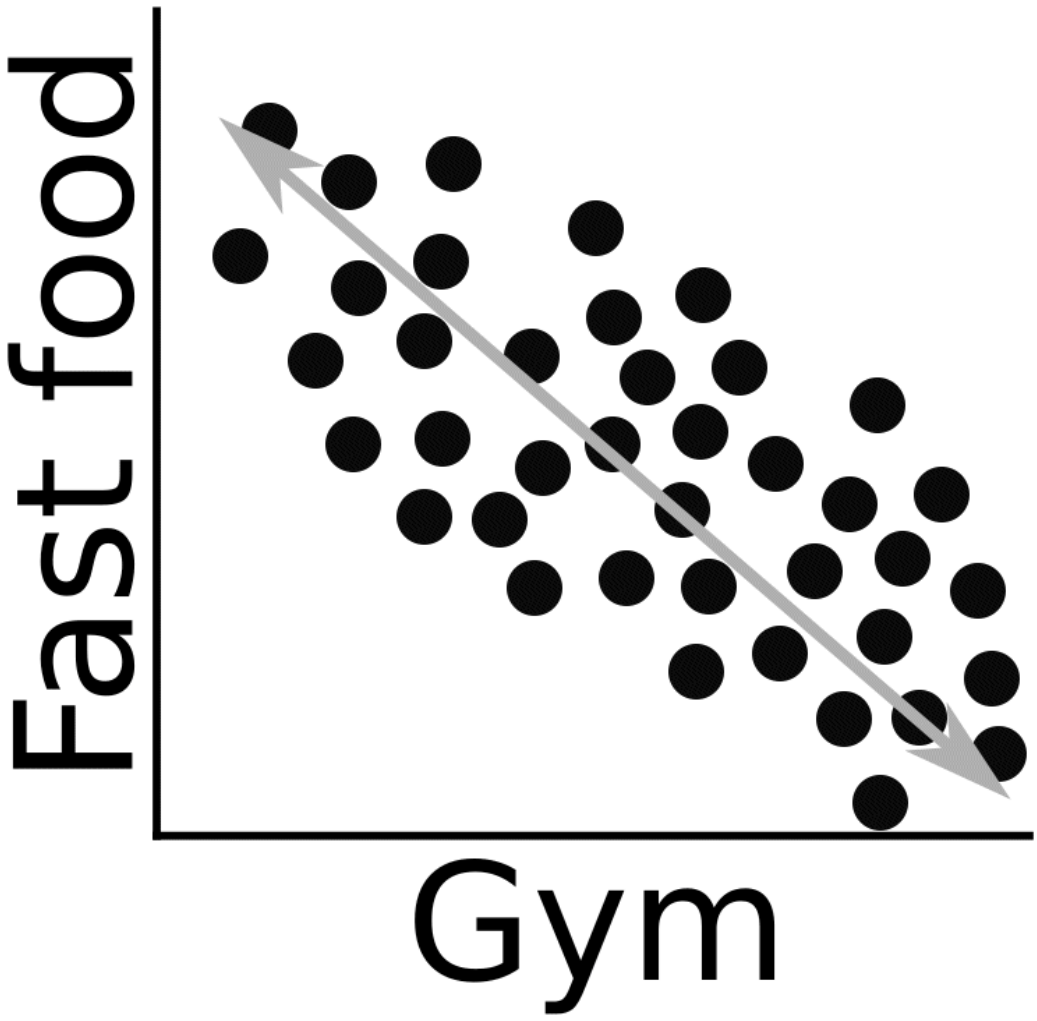
_____

When the above equation is satisfied, then v is an eigenvector and Î» is its associated eigenvalue (geometrically, Î» is the amount of stretching). Letâ€™s be clear about this equation: It is not saying that the matrix equals the scalar; both sides of this equation produce a vector, and we cannot simply divide by v. Instead, this equation is saying that the effect of the matrix on the vector is the same as the effect of the scalar on the vector.

The example shown in Figure 15.1 has Î» = .5. That means that Av = .5v. In other words, the matrix shrunk the vector by half, without changing its direction.

Importantly, eigenvectors/values are not special properties of the vector alone, nor are they special properties of the matrix alone. Instead, it is the combination of a particular matrix, a particular vector, and a particular scalar. Changing any of these quantities (even a single element) will likely destroy this special relationship. Thus, the vector v shown in Figure 15.1 is unlikely to be an eigenvector of a different matrix Bâ‰ A.

Statistical interpretation Do you make New Yearâ€™s resolutions? Many people make a New Yearâ€™s resolution to eat healthier and go to the gym more often. Unfortunately, most people break their resolutions after a few days or weeks. Imagine you are a researcher, and you want to know how long it takes people to fall back to their old habits. Lucky for you, your town has one fast-food restaurant and one gym, next to each other. So you sit across the street each day and count how many people walk into the fast-food place vs. the gym. The data you collect are organized in a 40Ã—2 table, with each row corresponding to a day and each column corresponding to the number of people that walked into the fast-food place, and the number of people who walked into the gym. You make a plot of the data with a "fast-food axis" and a "gym axis," and each dot in the graph corresponds to the data recorded on one day (Figure 15.1).



I0.5 Each dot corresponds to one dayâ€™s measurement of the number of people going into the fast-food place (y-axis) vs. the gym (x-axis).

Clearly there is meaningful structure in this dataset; there is some relationship between the axes, such that on days when more people walk into the gym, fewer people walk into the fast-food place.

Our concern is that neither axis seems optimal for understanding the structure in the data. Clearly, a betterâ€”more efficient and more informativeâ€”axis would reflect this structure. Iâ€™m sure you agree that the grey line drawn on top of the dots is an appropriate axis.

Why am I writing all of this? It turns out that that the gray line is an eigenvector of the data matrix times its transpose, which is also called a covariance matrix. In fact, the gray line shows the first principal component of the data. Principal components analysis (PCA) is one of the most important tools in data science (for example, it is a primary

method used in unsupervised machine learning), and it is nothing fancier than an eigendecomposition of a data matrix. More on this in Chapter 19.

Rubikâ€™s cube This explanation is the most conceptual of the three introductions to eigendecomposition, but I think it provides a nice mental image of the procedure, and of the outcome.

Think of a Rubikâ€™s cube as a matrix (I know, technically it would be a sparse tensor; as I wrote, this is a conceptual not a literal analogy). When you pick up the Rubikâ€™s cube, all of the colors are randomized; in other words, the information in the cube is randomly scattered around. Likewise, patterns of information in a data matrix are often distributed across different rows and columns.

To solve the Rubikâ€™s cube, you apply a specific sequence of rotations. You twist the rows and columns in a specific way to get each side of the cube filled with only one color.

That specific sequence of rotations is like the eigenvectors: The eigenvectors provide a set of instructions for how to rotate the information in the matrix, and once you apply all of the rotations, the information in the matrix becomes "ordered," with all of the similar information packed into one eigenvalue. Thus, the eigenvalue is analogous to a color.

The completed Rubikâ€™s cube is analogous to a procedure called "diagonalization," which means to put all of the eigenvectors into a matrix, and all of the eigenvalues into a diagonal matrix. That diagonal matrix is like the solved Rubikâ€™s cube.

If this analogy is confusing, then hopefully it will make more sense by the end of the chapter. And if itâ€™s still confusing after this chapter, then please accept my apologies and I hope the previous two explanations were sensible.

15.2 Finding eigenvalues

Eigenvectors are like secret passages that are hidden inside the matrix. In order to find those secret passages, we first need to find the secret keys. Eigenvalues are those keys. Thus, eigendecomposition requires first finding the eigenvalues, and then using those eigenvalues as "magic keys" to unlock the eigenvectors.

The way to discover the eigenvalues of a matrix is to re-write Equation 15.1 so that we have some expression equal to the zeros vector.

$$\mathbf{Av} - \lambda\mathbf{v} = \mathbf{0} \tag{15.2}$$

Notice that both terms of the left-hand side of the equation contain vector v, which means we can factor out that vector. In order to make this a valid operation, we need to insert an identity matrix before the vector.

$$\text{Av â^’ Î»Iv} \quad = 0 \tag{15.3}$$

$$(A - \lambda I)v = 0 \qquad (15.4)$$

Equation 15.4 is important. It means that a matrix shifted by λ times a vector v gives us the zeros vector. Where have you seen something like this before? Yes, you saw it in section 8.6 (page Â§): this is the definition of the null space of a matrix.

Of course, we could set v = 0 to make Equation 15.4 true for any matrix and any λ. But this is a trivial solution, and we do not consider it. Thus, the eigenvalue equation tells us that (A â^' λI)â€"the matrix shifted by λâ€"has a non-trivial null space.

Thus, weâ€™ve discovered that when shifting a matrix by an eigenvalue, the eigenvector is in its null space. That becomes the mechanism for finding the eigenvector, but itâ€™s all very theoretical at this pointâ€"we still donâ€™t know how to find λ!

The key here is to remember what we know about a matrix with a non-trivial null space, in particular, about its rank: We know that any square matrix with a non-trivial null space is reduced rank. And we also know that a reduced-rank matrix has a determinant of zero. And this leads to the mechanism for finding the eigenvalues of a matrix.

Eqn. box title=Finding eigenvalues

$$|A - \lambda I| = 0 \qquad (15.5)$$

In section 11.3 you learned that the determinant of a matrix is computed by solving the characteristic polynomial. And you saw examples of how a known determinant can allow us to solve for some unknown variable inside the matrix. Thatâ€™s exactly the situation we have here: We have a matrix with one missing parameter (λ) and we know that its determinant is zero. And thatâ€™s how you find the eigenvalues of a matrix. Letâ€™s work through some examples.

Eigenvalues of a 2Ã—2 matrix For a 2Ã—2 matrix, the characteristic polynomial is a quadratic equation. Letâ€™s start with a lettered example.

$$\begin{aligned}
\left| \begin{bmatrix} a & b \\ c & d \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right| &= 0 \\
\begin{vmatrix} a-\lambda & b \\ c & d-\lambda \end{vmatrix} &= 0 \\
(a - \lambda)(d - \lambda) - bc &= 0 \\
\lambda^2 - (a+d)\lambda + (ad - bc) &= 0 \qquad (15.6)
\end{aligned}$$

The characteristic polynomial of a 2Ã—2 matrix is a 2nd degree algebraic equation, meaning there are two λâ€™s that satisfy the equation (they might be complex). The

solutions can be found using the quadratic formula, which is one of those things that people try but fail to memorize in high-school math class.

$$\lambda = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \tag{15.7}$$

WARNING. a, b, and c in Equation 15.7 are not the same as a, b, and c in Equation 15.6 ; the former are polynomial coefficients whereas the latter are matrix elements. Let me rewrite these two equations using distinct variables.

$$\hat{I}\pm\hat{I}\text{»}^2 \text{â}\text{^}\text{'} \hat{I}^2\hat{I}\text{»} + \hat{I}^3 = 0 \tag{15.8}$$

$$\hat{I}\pm = 1$$

$$\hat{I}^2 = \text{â}\text{^}\text{'}(a + d)$$

$$\hat{I}^3 = ad \text{â}\text{^}\text{'} bc$$

$$\hat{I}\text{»} = \frac{-\beta \pm \sqrt{\beta^2 - 4\gamma}}{2} \tag{15.9}$$

$$\hat{I}\text{»} = \frac{(a+d) \pm \sqrt{(a+d)^2 - 4(ad-bc)}}{2} \tag{15.10}$$

If youâ€™re lucky, then your teacher or linear algebra textbook author designed matrices with entries that can be factored into the form $(\hat{I}\text{»}_1 \text{â}\text{^}\text{'} \hat{I}\text{»})(\hat{I}\text{»}_2 \text{â}\text{^}\text{'} \hat{I}\text{»}) = 0$. Otherwise, youâ€™ll have to try again to memorize the quadratic formula, or at least keep it handy for quick reference.

Letâ€™s solve a few numerical examples.

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \quad \Rightarrow \quad \begin{vmatrix} 1-\lambda & 2 \\ 2 & 1-\lambda \end{vmatrix} = 0 \tag{15.11}$$

$$(1 \text{â}\text{^}\text{'} \hat{I}\text{»})(1 \text{â}\text{^}\text{'} \hat{I}\text{»}) \text{â}\text{^}\text{'} 4 = 0$$

$$\hat{I}\text{»}^2 \text{â}\text{^}\text{'} 2\hat{I}\text{»} \text{â}\text{^}\text{'} 3 = 0$$

$$(\hat{I}\text{»} \text{â}\text{^}\text{'} 3)(\hat{I}\text{»} + 1) = 0$$

$$\text{â}\ddagger\text{'} \hat{I}\text{»}_1 = 3, \hat{I}\text{»}_2 = \text{â}\text{^}\text{'}1$$

You should always check to see whether the characteristic polynomial can be factored before applying the quadratic formula. In the next example, you wonâ€™t be so lucky.

$$\begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix} \quad \Rightarrow \quad \begin{vmatrix} 1-\lambda & 2 \\ 2 & 3-\lambda \end{vmatrix} = 0$$

$$(1 \text{â}\text{^}\text{'} \hat{I}\text{»})(3 \text{â}\text{^}\text{'} \hat{I}\text{»}) \text{â}\text{^}\text{'} 4 = 0$$

$$\hat{I}\text{»}^2 \text{â}\text{^}\text{'} 4\hat{I}\text{»} \text{â}\text{^}\text{'} 1 = 0$$

$$\hat{I}\text{»} = \frac{4 \pm \sqrt{20}}{2}$$

$$\hat{I}\text{»} = 2 \hat{A}\pm\sqrt{5}$$

Practice problems Find the eigenvalues of the following matrices.

a) $\begin{bmatrix} -4 & 2 \\ 0 & 4 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 3 \\ 5 & 0 \end{bmatrix}$

c) $\begin{bmatrix} 3 & 0 \\ 0 & 5 \end{bmatrix}$

d) $\begin{bmatrix} 2 & 5 \\ 6 & 3 \end{bmatrix}$

Answers

a) 4,âˆ’4

b) Â±$\sqrt{15}$

c) 3,5

d) âˆ’3,8

There is a small short-cut you can take for finding eigenvalues of a 2Ã—2 matrix. It comes from careful inspection of Equation 15.6 . In particular the Î»^{1} coefficient is the trace of the matrix and the Î»^{0} coefficient is the determinant of the matrix. Hence, we can rewrite Equation 15.6 as follows:

$$\lambda^2 - tr(\mathbf{A})\lambda + |\mathbf{A}| = 0 \qquad (15.12)$$

You still need to solve the equation for Î», so this admittedly isnâ€™t such a brilliant short-cut. But it will get to you the characteristic polynomial slightly faster.

Be mindful that this trick works only for 2Ã—2 matrices; donâ€™t try to apply it to any larger
matrices.

Practice problems Use the trace-determinant trick to find the eigenvalues of the following matrices.

a) $\begin{bmatrix} -4 & 1 \\ 1 & 3 \end{bmatrix}$

b) $\begin{bmatrix} -2 & 2 \\ -3 & 2 \end{bmatrix}$

c) $\begin{bmatrix} 6 & -6 \\ 0 & -3 \end{bmatrix}$

d) $\begin{bmatrix} 6 & 3 \\ 3 & 1.5 \end{bmatrix}$

Answers

a) (â^'1 Â±$\sqrt{53}$)â^•2
b) Â±$\sqrt{2}$ i
c) 6, -3
d) 0, 7.5

Eigenvalues of a 3Ã—3 matrix The algebra gets more complicated, but the principle is the same: Shift the matrix by â^'Î» and solve for Î” = 0. The characteristic polynomial produces a third-order equation, so you will have three eigenvalues as roots of the equation. Here is an example.

$$\begin{bmatrix} 9 & 0 & -8 \\ 15 & 3 & -15 \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{vmatrix} 9-\lambda & 0 & -8 \\ 15 & 3-\lambda & -15 \\ 0 & 0 & 1-\lambda \end{vmatrix} = 0$$

$$(9 - \lambda)(3 - \lambda)(1 - \lambda) = 0$$

$$\lambda_1 = 9, \lambda_2 = 3, \lambda_3 = 1$$

That example was easy because the characteristic polynomial was already factored due to the zeros in the matrix. Itâ€™s not always so simple.

$$\begin{bmatrix} -2 & 2 & -3 \\ -4 & 1 & -6 \\ -1 & -2 & 0 \end{bmatrix} \Rightarrow \begin{vmatrix} -2-\lambda & 2 & -3 \\ -4 & 1-\lambda & -6 \\ -1 & -2 & 0-\lambda \end{vmatrix} = 0$$

$$(-2 - \lambda)(1 - \lambda)(-\lambda) + 12 - 24 - 3(1 - \lambda) - 8\lambda + 12(2 + \lambda) = 0$$

$$-\lambda^3 + \lambda^2 + 5\lambda + 9 = 0$$

$$(3 - \lambda)(1 - \lambda)(3 - \lambda) = 0$$

The three Î»â€™s are 3, 1, and -3. _____

Practice problems Find the eigenvalues of the following matrices.

a) $\begin{bmatrix} 3 & 0 & 0 \\ -2 & 2 & 1 \\ 1 & 0 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 3 & -2 & -2 \\ 2 & -2 & -2 \\ -2 & 3 & 3 \end{bmatrix}$

c) $\begin{bmatrix} 3 & 0 & 0 \\ -2 & 4 & 0 \\ -4 & 2 & 3 \end{bmatrix}$

Answers

a) 1, 2, 3
b) 0, 1, 3
c) 4, 3, 3

M columns, M Î»s It is no coincidence that the 2Ã—2 matrices had 2 eigenvalues and the 3Ã—3 matrices had 3 eigenvalues. Indeed, the Fundamental Theorem of Algebra states that any m-degree polynomial has m solutions. And because an MÃ—M matrix has an Mth order characteristic polynomial, it will have M roots, or M eigenvalues. Hence, an MÃ—M matrix has M eigenvalues (some of those eigenvalues may be repeated, complex numbers, or zeros).

-|*|- **Reflection** You can already see from the examples above that eigenvalues have no intrinsic sorting. We can come up with sensible sorting, for example, ordering eigenvalues according to their position on the number line or magnitude (distance from zero), or by a property of their corresponding eigenvectors. Sorted eigenvalues can facilitate data analyses, but eigenvalues are an intrinsically unsorted set. -|*|-

15.3 Finding eigenvectors

The eigenvectors of a matrix reveal important "directions" in that matrix. You can think of those directions as being invariant to rotations, like in the illustration in Figure 15.1 . The eigenvectors are encrypted inside the matrix, and each eigenvalue is the decryption key for each eigenvector. Once you have the key, slot it into the matrix, turn it, and the eigenvector will be revealed.

In particular, once youâ€™ve identified the eigenvalues, shift the matrix by each â^’Î»$_i$ and find a vector v_i in the null space of that shifted matrix. This is the eigenvector associated with eigenvalue Î»$_i$. This can be written down in two alternative ways; both are presented below to make sure you get the idea. [2]

---

[2] Remember that the trivial zeros vector is not considered a solution.

Eqn. box title=Finding eigenvectors

$$v_i \hat{} N(A \hat{} \hat{I}»_i I) \quad (15.13)$$

$$(A \hat{} \hat{I}»_i I)v_i = 0 \quad (15.14)$$

Letâ€™s work through a practice problem. Iâ€™ll use the matrix presented on page Â§ . As a quick reminder, that matrix and its eigenvalues are:

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \quad \Rightarrow \quad \lambda_1 = 3,\ \lambda_2 = -1$$

Before looking at the numbers below, I encourage you to work through this on your own: Shift the matrix by each Î» and find a vector in its null space.

Here is my solution for $\hat{\lambda}_1$:

$$\begin{bmatrix} 1-3 & 2 \\ 2 & 1-3 \end{bmatrix} = \begin{bmatrix} -2 & 2 \\ 2 & -2 \end{bmatrix} \quad \Rightarrow \quad \begin{bmatrix} -2 & 2 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

[3] Now letâ€™s repeat for the other eigenvalue.

[3] I know, it looks weird to write â^'â^'instead of +. If you donâ€™t like it, then feel free to write on the page or screen.

$$\begin{bmatrix} 1--1 & 2 \\ 2 & 1--1 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \quad \Rightarrow \quad \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

That gives us our eigendecomposition for this matrix:

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{cases} \lambda_1 = 3, & \mathbf{v}_1 = \begin{bmatrix} 1 & 1 \end{bmatrix}^{\mathrm{T}} \\ \lambda_2 = -1, & \mathbf{v}_2 = \begin{bmatrix} -1 & 1 \end{bmatrix}^{\mathrm{T}} \end{cases} \tag{15.15}$$

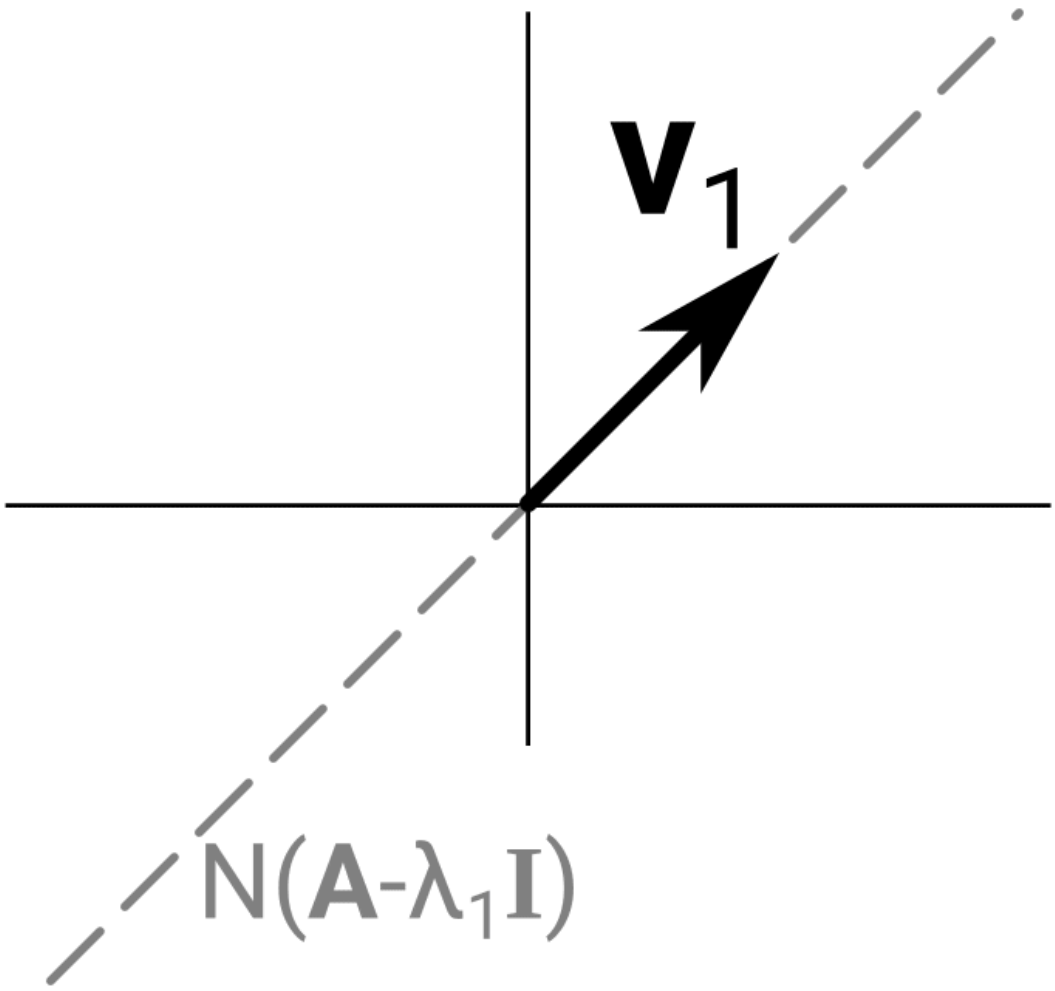
I wonder whether you got the same answers that I wrote above. The eigenvalues are uniqueâ€”there are no other possible solutions to the characteristic polynomial. But are the eigenvectors unique? Are the eigenvectors I printed above the only possible solutions? Take a moment to test the following possible eigenvectors to see whether they are also in the null space of the shifted matrix.

$$\mathbf{v}_1 \stackrel{?}{=} \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \begin{bmatrix} \pi \\ \pi \end{bmatrix}$$

$$\mathbf{v}_2 \stackrel{?}{=} \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \begin{bmatrix} 13 \\ -13 \end{bmatrix}, \begin{bmatrix} -a \\ a \end{bmatrix}$$

I believe you see where this is going: There is an infinite number of equally good solutions, and they are all connected by being scaled versions of each other. Thus, the true interpretation of an eigenvector is a basis vector for a 1D subspace. Therefore, we could re-imagine solution 15.15 as

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{cases} \lambda_1 = 3, & \mathbf{v}_1 = \alpha \begin{bmatrix} 1 & 1 \end{bmatrix}^{\mathrm{T}}, \quad \alpha \in \mathbb{R} \\ \lambda_2 = -1, & \mathbf{v}_2 = \beta \begin{bmatrix} -1 & 1 \end{bmatrix}^{\mathrm{T}}, \quad \beta \in \mathbb{R} \end{cases} \tag{15.16}$$



FigureÂ 15.3: The "preferred" eigenvector is the unit-length basis vector for the null space of the matrix shifted by its eigenvalue.

Given that we have an infinite number of equally accurate eigenvectors for each eigenvalue, how can we pick just one? For a variety of practical reasons (which you will learn about later), the best eigenvector to pick is the one that has a unit norm. Thus, out of the infinity of possible eigenvectors, we generally prefer the one such that âˆ¥vâˆ¥ = 1 (Figure 15.3). Thatâ€™s what numerical processing software programs like MATLAB and Python will do. This leads to our final re-expression of solution 15.15 :

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{cases} \lambda_1 = 3, & \mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ \lambda_2 = -1, & \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \end{cases} \tag{15.17}$$

That said, when solving problems by hand, itâ€™s usually easier to have eigenvectors with integer elements, rather than having to worry about all the fractions and square roots that arise in normalization. _____

Practice problems Find the eigenvectors of the following matrices.

a) $\begin{bmatrix} 3 & 1 \\ 4 & 6 \end{bmatrix}$

b) $\begin{bmatrix} 3 & -2 \\ 1 & 0 \end{bmatrix}$

Answers Vectors are not normalized for computational ease.

a) $\hat{I}»_1 = 2, v_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$, $\hat{I}»_2 = 7, v_2 = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$

b) $\hat{I}»_1 = 1, v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $\hat{I}»_2 = 2, v_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$

_____

Iâ€™d like to show another example of eigendecomposition of a 3Ã—3 matrix. Youâ€™ll see that the concept is the same, but the arithmetic quickly gets challenging. Itâ€™s important to work through enough examples that you understand the concept, and then you let computers do the hard work for you.

Again, the concept is that we find the 3 eigenvalues of the 3Ã—3 matrix by shifting by â^'Î», setting the determinant to zero, and solving for Î». Then we shift the matrix by each solved Î» and find a vector in the null space of that shifted matrix. Easy peasy, right? Letâ€™s see...

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 3 & 7 \\ 3 & 3 & 6 \end{bmatrix} \Rightarrow \begin{vmatrix} 1-\lambda & 2 & 3 \\ 4 & 3-\lambda & 7 \\ 3 & 3 & 6-\lambda \end{vmatrix} = 0$$

$$(1 â^' Î»)(3 â^' Î»)(6 â^' Î») + 42 + 36 â^' 9(3 â^' Î») â^' 8(6 â^' Î») â^' 21(1 â^' Î») = 0$$

$$â^' Î»^3 + 10Î»^2 â^' 27Î» + 18 + 42 + 33 â^' 27 + 9Î» â^' 48 + 8Î» â^' 21 + 21Î» = 0$$

$$â^' Î»^3 + 10Î»^2 + 11Î» = 0$$

$$â^' Î»(Î»^2 â^' 10Î» â^' 11) = 0$$

$$â^' Î»(Î» + 1)(Î» â^' 11) = 0$$

Our three eigenvalues are 0, -1, and 11. Notice that we got a Î» = 0, and the matrix is reduced-rank (the third column is the sum of the first two). Thatâ€™s not a coincidence, but a deeper discussion of the interpretation of zero-valued eigenvalues will come later.

Now we can solve for the three eigenvectors, which are the vectors just to the left of the equals sign (non-normalized).

$$\hat{I}\text{»}_1: \begin{bmatrix} 1-0 & 2 & 3 \\ 4 & 3-0 & 7 \\ 3 & 3 & 6-0 \end{bmatrix} \text{â‡'} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 3 & 7 \\ 3 & 3 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\hat{I}\text{»}_2: \begin{bmatrix} 1-1 & 2 & 3 \\ 4 & 3-1 & 7 \\ 3 & 3 & 6-1 \end{bmatrix} \text{â‡'} \begin{bmatrix} 2 & 2 & 3 \\ 4 & 4 & 7 \\ 3 & 3 & 7 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$[4]\ \hat{I}\text{»}_3: \begin{bmatrix} 1-11 & 2 & 3 \\ 4 & 3-11 & 7 \\ 3 & 3 & 6-11 \end{bmatrix} \text{â‡'} \begin{bmatrix} -10 & 2 & 3 \\ 4 & -8 & 7 \\ 3 & 3 & -5 \end{bmatrix} \begin{bmatrix} 19 \\ 41 \\ 36 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

[4] You can appreciate why linear algebra was mostly a theoretical branch of mathematics before computers came to the rescue.

15.4 Diagonalization via eigendecomposition

It is time to take a step back and look at the big picture. An MÃ—M matrix contains M eigenvalues and M associated eigenvectors. The set of eigenvalue/vector pairs produces a set of similar-looking equations:

$$\begin{aligned} \mathbf{A}\mathbf{v}_1 &= \lambda_1\mathbf{v}_1 \\ \mathbf{A}\mathbf{v}_2 &= \lambda_2\mathbf{v}_2 \\ &\vdots \\ \mathbf{A}\mathbf{v}_m &= \lambda_m\mathbf{v}_m \end{aligned} \tag{15.18}$$

This is clunky and ugly, and therefore violates an important principle in linear algebra: Make equations compact, elegant, and simple. Fortunately, all of these equations can be simplified by having each eigenvector be a column in a matrix, and having each eigenvalue be an element in a diagonal matrix. [5]

[5] Î› is the capital version of Î». Be careful not to confuse the Greek Î› with the Roman A. We donâ€™t want to start any new Greek-Roman conflicts.

$$\mathbf{AV} = \mathbf{V\Lambda} \tag{15.19}$$

On first glance, it may seem inconsistent that I wrote Î»v in Equation 15.18 but VÎ› in the matrix Equation 15.19 . There are two reasons why it must be VÎ› and cannot be Î›V. Both reasons can be appreciated by seeing the right-hand side of Equation 15.19 written out for a 3Ã—3 matrix. Before reading the text below the equation, try to understand why (1) the Î»â€™s have to be the diagonal elements and (2) why we need to post-multiply by Î›. In the eigenvectors below, the first subscript number corresponds to the eigenvector, and the second subscript number corresponds to the eigenvector element. For example, v_{12} is the second element of the first eigenvector.

(15.20)

$$\begin{bmatrix} v_{11} & v_{21} & v_{31} \\ v_{12} & v_{22} & v_{32} \\ v_{13} & v_{23} & v_{33} \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{bmatrix} = \begin{bmatrix} \lambda_1 v_{11} & \lambda_2 v_{21} & \lambda_3 v_{31} \\ \lambda_1 v_{12} & \lambda_2 v_{22} & \lambda_3 v_{32} \\ \lambda_1 v_{13} & \lambda_2 v_{23} & \lambda_3 v_{33} \end{bmatrix}$$

Those two questions are closely related. The reason why the eigenvalues go in the diagonal of a matrix that post-multiplies the eigenvectors matrix is that the eigenvalues must scale each column of the V matrix, not each row (refer to page ?? for the rule about pre- vs. post-multiplying a diagonal matrix). If Î› pre-multiplied V, each element of each eigenvector would be scaled by a different Î».

There is another reason why itâ€™s VÎ› and not Î›V. If the equation read AV = Î›V, then we could multiply both sides by $V^{â^'1}$, producing the statement A = Î›, which is not generally true.

So then why did I write Î»v for the single equation instead of the more-consistent vÎ»? For better or worse, Î»v is the common way to write it, and thatâ€™s the form you will nearly always see.

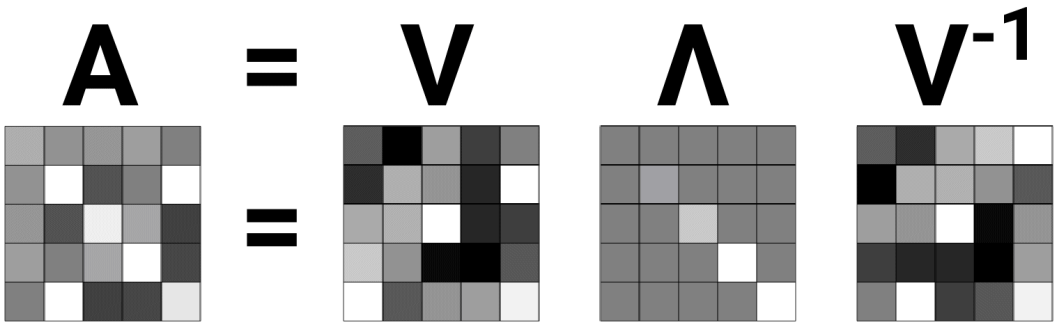
Diagonalization The elegant Equation 15.19 is not only a practical short-hand for a set of equations; it provides an important conceptual insight into one of the core ideas of eigendecomposition: Finding a set of basis vectors such that the matrix is diagonal in that basis space. This can be seen by left-multiplying both sides of the equation by $V^{â^'1}$. This is a valid operation if we assume that all eigenvectors form an independent set. This assumption is valid when there are M distinct eigenvalues (more on this in the next section).

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1} \tag{15.21}$$

Thus, matrix A is diagonal in basis V. Thatâ€™s why eigendecomposition is also sometimes called diagonalization. To diagonalize a matrix A means to find some matrix of basis vectors such that A is a diagonal matrix in that basis space.

Letâ€™s now revisit the Rubikâ€™s cube analogy from the beginning of this chapter: In Equation 15.21 , A is the scrambled Rubikâ€™s cube with all sides having inter-mixed colors; V is the set of rotations that you apply to the Rubikâ€™s cube in order to solve the puzzle; Î› is the cube in its "ordered" form with each side having exactly one color; and $V^{â^'1}$ is the inverse of the rotations, which is how you would get from the ordered form to the original mixed form.

Figure 15.4 shows what diagonalization looks like for a 5Ã—5 matrix; the gray-scale intensity corresponds to the matrix element value.



FigureÂ 15.4: Diagonalization of a matrix in pictures.

Practice problems Diagonalize the following matrices.

a) $\begin{bmatrix} 2 & 6 \\ 6 & 7 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 0 \\ -7 & 5 \end{bmatrix}$

Answers Matrices below are V,Î›,V$^{â^'1}$.

a) $\frac{1}{\sqrt{13}}\begin{bmatrix} -3 & 2 \\ 2 & 3 \end{bmatrix}, \begin{bmatrix} -2 & 0 \\ 0 & 11 \end{bmatrix}, \frac{1}{\sqrt{13}}\begin{bmatrix} -3 & 2 \\ 2 & 3 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 4 \\ 1 & 7 \end{bmatrix}, \begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}, \frac{1}{-4}\begin{bmatrix} 7 & -4 \\ -1 & 0 \end{bmatrix}$

-|*|- **Reflection** The previous reflection box mentioned sorting eigenvalues. Figure 15.4 shows that the eigenvalues are sorted ascending along the diagonal. Re-sorting eigenvalues is fine, but you need to be diligent to apply the same re-sorting to the columns of V, otherwise the eigenvalues and their associated eigenvectors will be mismatched. -|*|-

Code Obtaining eigenvalues and eigenvectors is really easy in Python and in MATLAB. Annoyingly, the outputs are different between the programs: Python returns the eigenvalues in a vector and the eigenvectors in a matrix (note that I create the diagonal Î› in a separate line); in contrast, MATLAB outputs the eigenvectors and the eigenvalues in the diagonal of a matrix, unless you request only one output, in which case you get the vector of eigenvalues.

Code blockÂ 15.1: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.array([[2,3],[3,2]])Â
```

```
L,VÂ =Â np.linalg.eig(A)Â
LÂ =Â np.diag(L)
```

Code blockÂ 15.2: MATLAB

```
AÂ =Â [Â 2Â 3;Â 3Â 2Â ];Â
LÂ =Â eig(A);Â %Â vectorÂ ofÂ eigenvaluesÂ
[V,L]Â =Â eig(A);
```

15.5 Conditions for diagonalization

Not all square matrices are diagonalizable. It might seem circular, but the definition of a diagonalizable matrix is a matrix that can be diagonalized, that is, that Equation 15.21 ($A = VÎ›V^{âˆ’1}$) is true.

Fortunately, if you are interested in applied linear algebra, then "most" matrices are diagonalizable. "Most" is a difficult word in math, because there is an infinite number of diagonalizable and non-diagonalizable matrices. But what I mean is that most of the square matrices that show up in statistics, machine-learning, data science, computational simulations, and other applications, are likely to be diagonalizable. Importantly, all symmetric matrices are diagonalizable.

But there are matrices for which no matrix V can make that decomposition true. Hereâ€™s an example of a non-diagonalizable matrix.

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \quad \lambda = \{0, 0\}, \quad \mathbf{V} = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Notice that the matrix is rank-1 and yet has two zero-valued eigenvalues. This means that our diagonal matrix of eigenvalues would be the zeros matrix, and it is impossible to reconstruct the original matrix using $Î› = 0$.

There is an entire category of matrices that is non-diagonalizable, called nilpotent matrices [6]. A nilpotent matrix means that for some matrix power k, $A^k = 0$. In other words, keep multiplying the matrix by itself and eventually youâ€™ll get the zeros matrix; below is an example of a rank-1 nilpotent matrix with k = 2. I encourage you to take a moment to confirm that AA = 0.

[6] A proof for this statement is given in section 15.11 .

$$\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

All triangular matrices that have zeros on the diagonal are nilpotent, have all zero-valued eigenvalues, and thus cannot be diagonalized.

All hope is not lost, however, because the singular value decomposition is valid on all matrices, even the non-diagonalizable ones. The two non-diagonalizable example matrices above have singular values, respectively, of {2,0} and {1,0}.

15.6 Distinct vs. repeated eigenvalues

Distinct eigenvalues [7] Many square matrices have M distinct eigenvalues. Thatâ€™s nice because distinct eigenvalues always lead to distinct eigenvectors. You might already have the intuition that plugging different Î»â€™s into a matrix should reveal different eigenvectors. But we need to prove that this is the case.

[7] Distinct eigenvalues means $\lambda_i \neq \lambda_j$; distinct eigenvectors means they form a linearly independent set.

We are going to prove the claim that distinct eigenvalues lead to distinct eigenvectors by trying to disprove it and failing (a good strategy in scientific research).

We start with the following assumptions:

1. Matrix A has distinct eigenvalues λ_1 and λ_2 ($\lambda_1 \neq \lambda_2$).
2. Their associated eigenvectors v_1 and v_2 are linearly dependent, meaning that one can be expressed as a scaled version of the other. This assumption can be written as follows.

$$\beta_1 \mathbf{v}_1 + \beta_2 \mathbf{v}_2 = \mathbf{0}, \qquad \beta_1, \beta_2 \neq 0 \tag{15.22}$$

Equation 15.22 is the starting point for our proof. We proceed by multiplying the entire equation by matrix A. And from there we can replace Av with λv.

$$\begin{aligned} \lambda^2_1 \, Av_1 + \lambda^2_2 \, Av_2 &= \quad A0 \\ \lambda^2_1 \, \lambda_1 \, v_1 + \lambda^2_2 \, \lambda_2 \, v_2 &= \quad 0 \end{aligned} \tag{15.23}$$

Next I will multiply Equation 15.22 by λ_1 , and then subtract equations 15.23 from 15.22 .

$$\begin{array}{rl} & \beta_1 \lambda_1 \mathbf{v}_1 + \beta_2 \lambda_1 \mathbf{v}_2 = \mathbf{0} \\ - & \beta_1 \lambda_1 \mathbf{v}_1 + \beta_2 \lambda_2 \mathbf{v}_2 = \mathbf{0} \\ \hline & \beta_2 \lambda_1 \mathbf{v}_2 - \beta_2 \lambda_2 \mathbf{v}_2 = \mathbf{0} \end{array}$$

The two terms in the left-hand side of the difference equation both contain $\lambda^2_2 \, v_2$, so that can be factored out, revealing the final nail in the coffin of our to-be-falsified hypothesis:

$$(\lambda_1 - \lambda_2)\beta_2 \mathbf{v}_2 = \mathbf{0} \tag{15.24}$$

Why is this the key equation? It says that we multiply three terms to obtain the zeros vector, which means at least one of these terms must be zero. (λ_1 â^’λ_2) cannot equal zero because we began from the assumption that they are distinct. v_2 â‰ 0 because we do not consider zero eigenvectors.

Thus, we are forced to conclude that $\hat{I}^2_2 = 0$. This violates the assumption in Equation 15.22 , which proves that the assumption is invalid, which proves that distinct eigenvalues lead to linearly independent eigenvectors.

However, when we consider a set with more than two eigenvectors, it is possible to have a linearly dependent set with one $\hat{I}^2 = 0$. So letâ€™s allow $\hat{I}^2_2 = 0$ and plug that into Equation 15.22 . This forces us to conclude that $\hat{I}^2_1 = 0$. Thus, the only way to combine the two vectors to produce the zeros vector is by weighting both vectors by zero; in other words, there is no nontrivial combination of the two vectors that produces the zeros vector, which means the two vectors are linearly independent. If our set contained more than two eigenvectors, then repeating the above logic (setting each $\hat{I}^2 = 0$) ultimately leads to the conclusion that all $\hat{I}^2$s must be zero.

Either way you look at it, the conclusion is that distinct eigenvalues lead to linearly independent eigenvectors.

We did not need to impose any assumptions about the field from which the Î»s are drawn; they can be real or complex-valued, rational or irrational, integers or fractions. The only important quality is that each Î» is distinct.

Repeated eigenvalues Repeated eigenvalues complicate matters because they sometimes have distinct eigenvectors and sometimes not. Iâ€™ll start by showing two examples before discussing the more general principle. Consider the following matrix and its eigenvalues.

$$\begin{bmatrix} 6 & -4 \\ 1 & 2 \end{bmatrix} \Rightarrow \begin{vmatrix} 6-\lambda & -4 \\ 1 & 2-\lambda \end{vmatrix} \Rightarrow \lambda^2 - 8\lambda + 16 = 0 \Rightarrow (\lambda - 4)^2 = 0$$

The two solutions are $\hat{I}»_1 = \hat{I}»_2 = 4$. Plugging 4 into the shifted matrix yields

$$\begin{bmatrix} 2 & -4 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \mathbf{0} \quad \Rightarrow \quad \mathbf{v}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

Itâ€™s clear how to get the first eigenvector, but then how do you get the second eigenvector? The answer is you donâ€™tâ€”there is only one eigenvector. MATLAB will return the following. [8]

[8] MATLAB normalized the vectors to be unit-length vectors: the vector [1 2] points in the same direction as [1 2]âˆ• $\sqrt{5}$.

```
>>Â [V,L]Â =Â eig([6Â -4;Â 1Â 2])
VÂ =
Â Â Â Â 0.8944Â Â Â -0.8944
Â Â Â Â 0.4472Â Â Â -0.4472
```

```
LÂ =
Â Â Â Â 4.0000Â Â Â 0
Â Â Â Â 0Â Â Â Â Â Â Â Â Â 4.0000
```

The columns of output matrix V are the same subspace, although one points in the opposite direction. There isnâ€™t anything "wrong" with the matrixâ€”it has a rank of 2 and a non-zero determinant. But it has a single eigenvalue that reveals only a single eigenvector. Letâ€™s see another example of a matrix with repeated eigenvalues.

$$\begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \Rightarrow \begin{vmatrix} 4-\lambda & 0 \\ 0 & 4-\lambda \end{vmatrix} \Rightarrow \lambda^2 - 8\lambda + 16 = 0 \Rightarrow (\lambda - 4)^2 = 0$$

Again, the two solutions are Î» = 4. Plugging this into the matrix yields

$$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \mathbf{0} \quad \Rightarrow \quad ? \tag{15.25}$$

Now we have an interesting situation: any vector times the zeros matrix produces the zeros vector. So all vectors are eigenvectors of this matrix. Which two vectors to select? The standard basis vectors are a good practical choice, because they are easy to work with, are orthogonal, and have unit length. Therefore, for the matrix above, V=I. To be clear: V could be I, or it could be any other 2Ã—2 full-rank matrix. This is a special case of eigendecomposition of the identity matrix. I just multiplied it by 4 so weâ€™d get the same eigenvalues as in the previous example.

Perhaps that was too extreme an example. Consider the following matrix.

$$\begin{bmatrix} 5 & -1 & 0 \\ -1 & 5 & 0 \\ 1/3 & -1/3 & 4 \end{bmatrix} \quad \Rightarrow \quad \lambda = 6, 4, 4 \tag{15.26}$$

This matrix has one distinct eigenvalue and one repeated eigenvalue. We know with certainty that the eigenvector associated with Î» = 6 will be distinct (I encourage you to confirm that a good integer-valued choice is $[3\ {-1}\ 1]^T$); what will happen when we plug 4 into the shifted matrix?

$$\begin{bmatrix} 1 & -1 & 0 \\ -1 & 1 & 0 \\ 1/3 & -1/3 & 0 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad \Rightarrow \quad \mathbf{v_1} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \ \mathbf{v_2} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \tag{15.27}$$

Again, we can ask the question whether these are the only two eigenvectors. Iâ€™m not referring to scaled versions of these vectors such as $Î±v_2$, I mean whether we could pick equally good eigenvectors with different directions from the two listed above.

Consider vector v_2 . Because the third column of the shifted matrix is all zeros, the third element of the eigenvector can be any number. I could have picked [1 1 1] or [1 1 10]. As long as the first two elements are the same, the third element can be anything. The pair of eigenvectors I picked is a convenient choice because they are orthogonal and contains 0â€™s and 1â€™s; but these werenâ€™t the only possible vectors I could have selected.

Now youâ€™ve seen several examples of the possible outcomes of repeated eigenvalues: there can be only one eigenvector or distinct eigenvectors, or an infinity of possible sets of distinct eigenvectors. Which of these possibilities is the case depends on the numbers in the matrix.

To understand why this can happen, letâ€™s revisit the proof in the beginning of this section. We need to modify the first assumption, though: We now assume that the two eigenvalues are the same, thus $\hat{I}»_1 = \hat{I}»_2$. Most of the rest of the proof continues as already written above. Below is Equation 15.24 , re-written for convenience.

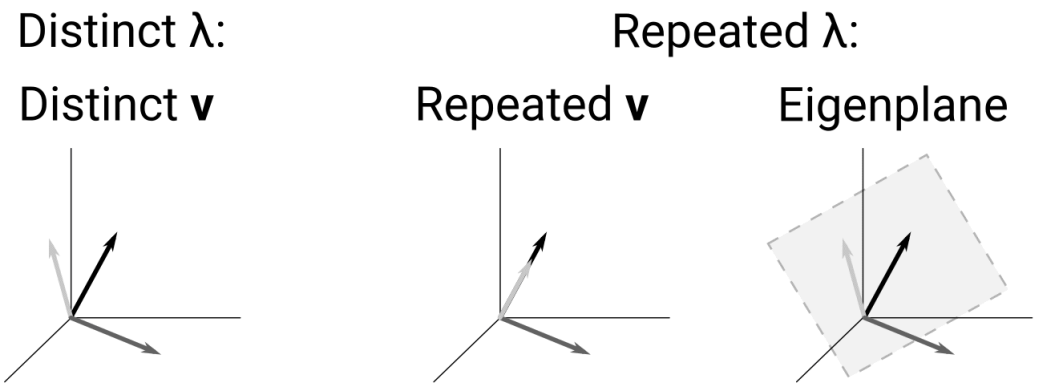
$$(\lambda_1 - \lambda_2)\beta_2\mathbf{v}_2 = \mathbf{0}$$

Again, we have the product of three terms producing the zeros vector. What do we know now? We still donâ€™t accept the zeros vector as an eigenvector, so v_2 â‰ 0. However, we also know that the first term is equal to zero. Thus, the equation really is

$$0\beta_2\mathbf{v}_2 = \mathbf{0}$$

What can we now conclude? Not much. We have the zeros vector without the trivial solution of a zeros eigenvector, so assumption 2 can be maintained (any vector set that includes the zeros vector is linearly dependent). We could now drop assumption 2 and say that the set of eigenvectors could dependent or independent, depending on the entries in the matrix.

Geometric interpretation of repeated eigenvalues Hereâ€™s the situation: Each distinct eigenvalue has its own distinct eigenvector, and that eigenvector is a basis vector for a 1D subspace (Figure 15.5).



FigureÂ 15.5: The geometric possibilities of eigenvectors with repeated eigenvalues, shown in â„ ^{3} for convenient visualization.

Repeated eigenvalues can lead to one of two situations. First, both eigenvectors can lie on the same 1D subspace. In that case, the eigenspace wonâ€™t span the entire ambient space â„ ^{M} ; it will be a smaller-dimensional subspace. The second possibility is that there are two distinct eigenvectors associated with one eigenvalue. In this case, there isnâ€™t a unique eigenvector; instead, there is a unique eigenplane and the two eigenvectors are basis vectors for that eigenplane. Any two independent vectors in the plane can be used as a basis. It is convenient to define those vectors to be orthogonal, and this is what computer programs will return.

-|*|- **Reflection** Do you really need to worry about repeated eigenvalues for applications of eigendecomposition? Repeated eigenvalues may seem like some weird quirk of abstract mathematics, but real datasets can have eigenvalues that are exactly repeated or statistically indistinguishable. In my own research on multivariate neural time series analysis, I find nearly identical eigenvalues to be infrequent, but common enough that I keep an eye out for them. -|*|-

15.7 Complex eigenvalues or eigenvectors

If 4ac > b^2 in Equation 15.6 , then you end up with the square root of a negative number, which means the eigenvalues will be complex numbers. And complex eigenvalues lead to complex eigenvectors.

Donâ€™t be afraid of complex numbers or complex solutions; they are perfectly natural and can arise even from matrices that contain all real values. The typical example of complex eigenvalues is the identity matrix with one row-swap and a minus sign:

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \qquad \text{â‡’ Î› = } \begin{bmatrix} i & 0 \\ 0 & -i \end{bmatrix}, \text{ V = } \begin{bmatrix} 1 & 1 \\ -i & i \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} \text{ â‡’ Î› = } \begin{bmatrix} i & 0 & 0 \\ 0 & -i & 0 \\ 0 & 0 & 1 \end{bmatrix}, \text{ V = } \begin{bmatrix} 0 & 0 & 1 \\ -1 & -1 & 0 \\ -i & i & 0 \end{bmatrix}$$

Complex solutions can also arise from "normal" matrices with real-valued entries. Consider the following matrix, which I generated in MATLAB from random integers (that is, I did not carefully hand-select this matrix or look it up in a secret online website for math teachers who want to torment their students). (Numbers are rounded to nearest tenth.)

$$\begin{bmatrix} -1 & 15 \\ -6 & 4 \end{bmatrix} \quad \Rightarrow \quad \Lambda = \begin{bmatrix} 1.5+9.2i & 0 \\ 0 & 1.5-9.2i \end{bmatrix}, \ \mathbf{V} = \begin{bmatrix} .85 & .85 \\ .1+.5i & .1-.5i \end{bmatrix}$$

It is no coincidence that the two solutions is a pair of complex conjugates. For a 2Ã—2 matrix, complex conjugate pair solutions are immediately obvious from Equation 15.7 (page Â§): A complex number can only come from the square-root in the numerator,

which is preceded by a Â± sign. Thus, the two solutions will have the same real part and flipped-sign imaginary part.

This generalizes to larger matrices: A real-valued matrix with complex eigenvalues has solutions that come in pairs: Î» and Î». Furthermore, their associated eigenvectors also come in conjugate pairs v and v. This can be shown by inspecting the complex conjugate of the fundamental eigenvalue equation.

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

Av = Î»v

Av = Î»v

Equation 15.28 follows from the previous line because the matrix is real-valued, thus A = A.

Complex-valued solutions in eigendecomposition can be difficult to work with in applications with real datasets, but there is nothing in principle weird or strange about them. _____

Practice problems Identify the eigenvalues from the following matrices (consult page Â§ for the quadratic formula).

a) $\begin{bmatrix} 2 & -3 \\ 1 & 2 \end{bmatrix}$

b) $\begin{bmatrix} 3 & 7 \\ -7 & 3 \end{bmatrix}$

c) $\begin{bmatrix} a & b \\ -b & a \end{bmatrix}$

Answers

a) Î» = 2 Â± i$\sqrt{3}$

b) Î» = 3 Â± i7

c) Î» = a Â± ib

_____

15.8 Eigendecomposition of a symmetric matrix

By this point in the book, you know that Iâ€™m a HUGE fan of symmetric matrices. And Iâ€™m not the only oneâ€”everyone who works with matrices has a special place in their heart for the elation that comes with symmetry across the diagonal. In this section, you are going to learn two additional properties that make symmetric matrices really great to work with.

Eigendecomposition of a symmetric matrix has two notable features: orthogonal eigenvectors (assuming distinct eigenvalues; for repeated eigenvalues the eigenvectors can be crafted to be orthogonal, as discussed earlier), and real-valued (as opposed to complex) solutions.

Orthogonal eigenvectors If the matrix is symmetric, then all of its eigenvectors are pairwise orthogonal. There is an example showing this property earlier in this chapter, and I encourage you to find the symmetric matrix and confirm that its eigenvectors are orthogonal.

But of course we need to prove this rigorously for all symmetric matrices. The goal is to show that the dot product between any pair of eigenvectors is zero. We start from two assumptions: (1) matrix A is symmetric ($A = A^T$) and (2) $\hat{I}»_1$ and $\hat{I}»_2$ are distinct eigenvalues of A (thus $\hat{I}»_1$ â‰ $\hat{I}»_2$). v_1 and v_2 are their corresponding eigenvectors. Iâ€™m going to write a series of equalities; make sure you can follow each step from left to right.

$$\lambda_1 \mathbf{v}_1^T \mathbf{v}_2 = (\mathbf{A}\mathbf{v}_1)^T \mathbf{v}_2 = \mathbf{v}_1^T \mathbf{A}^T \mathbf{v}_2 = \mathbf{v}_1^T \lambda_2 \mathbf{v}_2 = \lambda_2 \mathbf{v}_1^T \mathbf{v}_2 \tag{15.28}$$

The middle steps are actually just way-points; we care about the equality between the first and last terms. Iâ€™ll write them below, and then set the equation to zero.

$$\lambda_1 \mathbf{v}_1^T \mathbf{v}_2 = \lambda_2 \mathbf{v}_1^T \mathbf{v}_2 \tag{15.29}$$

$$\lambda_1 \mathbf{v}_1^T \mathbf{v}_2 - \lambda_2 \mathbf{v}_1^T \mathbf{v}_2 = 0 \tag{15.30}$$

Notice that both terms contain the dot product $v_1{}^T v_2$, which can be factored out, bringing us to the crux of the proof:

$$(\lambda_1 - \lambda_2)\mathbf{v}_1^T \mathbf{v}_2 = 0 \tag{15.31}$$

Equation 15.31 says that two quantities multiply to produce 0, which means that one or both of those quantities must be zero. ($\hat{I}»_1$ â^’ $\hat{I}»_2$) cannot equal zero because we began from the assumption that they are distinct. Therefore, $v_1{}^T v_2$ must be zero, which means that v_1 âŠ¥ v_2, i.e., the two eigenvectors are orthogonal. Note that this proof is valid only for symmetric matrices, when $A^T = A$. Otherwise, youâ€™ll get stuck in the middle of line 15.28 â€”it wonâ€™t be possible to set the first and last expressions equal to each other.

Orthogonal eigenvectors are a big deal. It means that the dot product between any two non-identical columns will be zero:

$$\mathbf{v}_i^T \mathbf{v}_j = \begin{cases} \|\mathbf{v}_i\|^2 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \tag{15.32}$$

When putting all of those eigenvectors as columns into a matrix V, then $V^T V$ is a diagonal matrix.

But wait, thereâ€™s more! Remember that eigenvectors are important because of their direction, not their magnitude. And remember that I wrote that itâ€™s convenient to have unit-length eigenvectors. So letâ€™s rewrite 15.32 assuming unit-norm eigenvectors:

$$\mathbf{v}_i^{\mathrm{T}}\mathbf{v}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \tag{15.33}$$

I hope this looks familiar. This is also the definition of an orthogonal matrix. And that means:

$$V^T V = I \tag{15.34}$$

$$V^T = V^{\hat{a}^{\hat{}}'1} \tag{15.35}$$

Thus, the eigenvectors of a symmetric matrix form an orthogonal matrix. This in an important property with implications for statistics, multivariate signal processing, data compression, and other applications. Youâ€™ll see several examples of this property later in this book, for example in Chapter 19.

Real solutions Now letâ€™s examine the property that real-valued symmetric matrices always have real-valued eigenvalues (and therefore also real-valued eigenvectors). There are 6 steps to this proof. Before reading the text below, try to understand the proof only from the equations.

[9] 1) $Av = \hat{I}»v$

2) $(Av)^H = (\hat{I}»v)^H$

3) $v^H A = \hat{I}»^H v^H$

4) $v^H Av = \hat{I}»^H v^H v$

5) $\hat{I}»v^H v = \hat{I}»^H v^H v$

6) $\hat{I}» = \hat{I}»^H$

Could you understand it just from the equations? Hereâ€™s my explanation, just in case you want to confirm your own understanding. Step 1 is the basic eigenvalue equation. Step 2 says to take the Hermitian of both sides of the equation, which is then implemented in Step 3. Because A is symmetric and comprises real numbers, $A^H = A^T = A$. In Step 4, both sides of the equation are right-multiplied by the eigenvector v. In Step 5, Av is turned into its equivalent $\hat{I}»v$. $v^H v$ is the magnitude squared of vector v and can simply be divided away (remember that vâ‰ 0). This brings us to the conclusion that $\hat{I}» = \hat{I}»^H$. A number is equal to its complex conjugate (a + ib = a âˆ’ ib) only when b = 0, i.e., it is a real-valued number. This concludes the proof that $\hat{I}»$ must have an imaginary part equal to 0, which means it is real-valued. If the matrix is not symmetric, we cannot proceed to Step 3.

[9] Recall from Chapter 8 that the superscript ^{H} indicates the Hermitian transpose, which means transpose and flip the sign of the imaginary parts.

Final note for this section: Some people use different letters to indicate the eigendecomposition of a symmetric matrix. In other texts or lectures you might see the following options.

$$A = PDP^{â^'1}$$
$$A = UDU^{â^'1}$$

Practice problems Each problem gives a matrix and its unscaled eigenvectors. Fill in the missing eigenvector element. Then find the eigenvalues using the trace and determinant. Finally, compute $VÎ›V^T$ to confirm that it reproduces the original matrix.

a) $\begin{bmatrix} -2 & 2 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} -2 & 1 \\ * & 2 \end{bmatrix}$

b) $\begin{bmatrix} 52 & 16 \\ 16 & 28 \end{bmatrix}, \begin{bmatrix} 2 & * \\ -4 & -2 \end{bmatrix}$

c) $\begin{bmatrix} 3 & -3 \\ -3 & 3 \end{bmatrix}, \begin{bmatrix} * & 1 \\ 1 & -1 \end{bmatrix}$

Answers

a) $â^— = 1$
$Î» = â^'3,2$

b) $â^— = â^'4$
$Î» = 20,60$

c) $â^— = 1$
$Î» = 0,6$

15.9 Eigenvalues of singular matrices

Every singular matrix has at least one zero-valued eigenvalue. And every full-rank matrix has no zero-valued eigenvalues. Before discussing why singular matrices have zero-valued eigenvalues, I want to show a few examples to illustrate that eigenvectors associated with zero-valued eigenvalues are not unusual (for example, they are not zeros, complex-valued, necessarily irrational, or have any bizarre properties; theyâ€™re just regular eigenvectors).

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix} \quad \Rightarrow \quad |\mathbf{A} - \lambda\mathbf{I}| = 0 \quad \Rightarrow \quad \begin{vmatrix} 1-\lambda & 2 \\ 1 & 2-\lambda \end{vmatrix} = 0$$

$$(1-\lambda)(2-\lambda) - 2 = 0 \quad \Rightarrow \quad \lambda^2 - 3\lambda = 0 \quad \Rightarrow$$

$$\lambda(\lambda - 3) = 0 \quad \Rightarrow \quad \lambda_1 = 0,\ \lambda_2 = 3$$

Now we can substitute in each Î» to find the two eigenvectors.

$$\lambda_1 : \begin{bmatrix} 1-0 & 2 \\ 1 & 2-0 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \Rightarrow \quad \mathbf{v} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

$$\lambda_2 : \begin{bmatrix} 1-3 & 2 \\ 1 & 2-3 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \Rightarrow \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Lest you think I carefully selected that matrix to give a zero eigenvalue, the next example shows that even in the general case of a matrix where one column is a multiple of another column (thus, a rank-1 matrix), one of the eigenvalues will be zero.

$$\mathbf{A} = \begin{bmatrix} a & \sigma a \\ b & \sigma b \end{bmatrix} \quad \Rightarrow \quad |\mathbf{A} - \lambda\mathbf{I}| = 0 \quad \Rightarrow \quad \begin{vmatrix} a-\lambda & \sigma a \\ b & \sigma b-\lambda \end{vmatrix} = 0$$

$$(a-\lambda)(\sigma b - \lambda) - \sigma ab = 0 \quad \Rightarrow \quad \lambda^2 - (a+\sigma b)\lambda = 0 \quad \Rightarrow$$

$$\lambda(\lambda - (a+\sigma b)) = 0 \quad \Rightarrow \quad \lambda_1 = 0,\ \lambda_2 = a + \sigma b$$

You can see from this example that the off-diagonal product (â^'(Ïƒ aÃ—b)) will cancel with the "left" terms from the diagonal product (aÃ—Ïƒ b); it doesnâ€™t matter what values you assign to a, b, and Ïƒ. (The same effect happens if you construct one row to be a multiple of the other, which you should try on your own.) The canceling of the constant term means all terms on the left-hand side of the characteristic polynomial contain Î», which means 0 will always be a solution.

At this point, you might be jumping to the conclusion that the rank of a matrix corresponds to the number of non-zero eigenvalues. Although this is a good thought, itâ€™s incorrect (it is, however, true for singular values, which you will learn in the next chapter). Let me make that more clear: The number of non-zero eigenvalues equals the rank for some matrices, but this is not generally true for all matrices. Youâ€™ve already seen an example of this in the section about conditions for diagonalizability. As an additional demonstration, create matrix A above using a=-4, b=2, Ïƒ=2. You will find that that matrix has both eigenvalues equal to zero, and yet is a rank-1 matrix. (This is another example of a nilpotent matrix. You can confirm that the suggested matrix has AA = 0.)

Letâ€™s see how this would work with a 3Ã—3 rank-2 matrix (I encourage you to work through this problem on your own before inspecting the equations below).

$$\mathbf{A} = \begin{bmatrix} a & d & \sigma a \\ b & e & \sigma b \\ c & f & \sigma c \end{bmatrix} \quad \Rightarrow \quad \begin{vmatrix} a-\lambda & d & \sigma a \\ b & e-\lambda & \sigma b \\ c & f & \sigma c-\lambda \end{vmatrix} = 0$$

$$(a-\lambda)(e-\lambda)(\sigma c-\lambda) + d\sigma be + \sigma abf - \sigma ac(e-\lambda) - db(\sigma c-\lambda) - \sigma bf(a-\lambda) = 0$$

$$-\lambda^3 + (a+e+\sigma c)\lambda^2 - (ae + a\sigma c + e\sigma c - \sigma ac - db - \sigma bf)\lambda = 0$$

There is a bit of algebra in that final step, but you can see that Î» appears in all terms, which means Î» = 0 is one of the three solutions.

I hope you find those examples compelling. There are several explanations for why singular matrices have at least one zero-valued eigenvalue. One is that the determinant of a matrix equals the product of the eigenvalues, and the determinant of a singular matrix is 0, so at least one eigenvalue must be zero.

But I think itâ€™s more insightful to reconsider Equation 15.4 , rewritten below for convenience.

$$(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0} \qquad (15.4)$$

We can interpret this equation in two ways. First letâ€™s assume that Î» = 0. Then weâ€™re not actually shifting the matrix and the equation can be rewritten as Av = 0. Because we do not consider the zeros vector to be an eigenvector, matrix A already has a non-trivial vector in its null space, hence it is singular.

Another way to interpret Equation 15.4 is to work the other way around: start from the assumption that A is singular, hence there is a non-trivial solution to Av = 0. The way to reconcile that statement with Equation 15.4 is to assume that at least one Î» = 0.

Practice problems Compute the missing eigenvalues of these matrices (hint: remember that the sum of eigenvalues equals the trace of the matrix).

a) $\begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$,Î» = 1,?

b) $\begin{bmatrix} 3 & 4 \\ 6 & 8 \end{bmatrix}$,Î» =?,?

c) $\begin{bmatrix} 2 & -2 & -2 \\ 3 & -3 & -2 \\ 2 & -2 & -2 \end{bmatrix}$,Î» = â^'2,?,?

Answers Printed below are all eigenvalues.

a) Î» = 1,0
b) Î» = 0,11
c) Î» = â^'2,â^'1,0

15.10 Eigenlayers of a matrix

This section will tie together eigendecomposition with the "layer perspective" of matrix multiplication. It will also set you up to understand the "spectral theory of matrices" and applications such as data compression and PCA. Weâ€™re going to discover how to reconstruct a matrix one "layer" at a time, where each layer comes from an eigenvector.

Consider computing the outer product of one eigenvector with itself. That will produce an MÃ—M rank-1 matrix. The norm of this matrix will also be 1, because it is formed from a unit-norm vector. An M Ã—M matrix has M eigenvectors, and thus, M outer product matrices can be formed from the set of eigenvectors.

What would happen if we sum together all of those outer product matrices? Well, nothing terribly important. Itâ€™s a valid operation, but that sum would not equal the original matrix A. Why not? Itâ€™s because eigenvectors have no intrinsic length; they need the eigenvalues to scale them.

Therefore, weâ€™ll multiply each eigenvector outer product matrix by its corresponding eigenvalue. And now we have an interesting situation, because this sum will exactly reproduce the original matrix. In other words, we can reconstruct the matrix one "eigenlayer" at a time.

$$\mathbf{A} = \sum_{i=1}^{M} \mathbf{v}_i \lambda_i \mathbf{v}_i^{\mathrm{T}} \tag{15.36}$$

Expanding out the summation sign leads to the insight that we are re-expressing diagonalization:

$$\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^{\mathrm{T}} \tag{15.37}$$

But actually, this is not exactly Equation 15.21 (page Â§). Previously we right-multiplied by $V^{â^'1}$ but here weâ€™re right-multiplying by V^{T} . Can you guess what that means? It means that reconstructing a matrix using Equation 15.36 is valid only for symmetric matrices, because $V^{â^'1} = V^{T}$.

But donâ€™t worry, reconstructing a matrix via eigenlayers is still valid for non-symmetric matrices. We just need a different formulation. In fact, now the vector on the right is the ith row of $V^{â^'1}$. This comes from the definition of the outer product, as the multiplication between a column vector and a row vector. Some additional notation should minimize confusion.

$$W = V^{-T} \tag{15.38}$$

$$^{10}A = â^' _{i=1}{}^{M} v_i Î»_i w_i{}^{T} \tag{15.39}$$

Now we have the outer product between the eigenvector and the corresponding row of the inverse of the eigenvector matrix transposed, which here is printed as the ith column of matrix W. I hope it is clear why Equation 15.36 is a simplification of Equation 15.39 : $V^{-T} = V$ for an orthogonal matrix.

[10] What happens here for $\hat{I}»_i = 0$?

It is important to appreciate that Equation 15.36 [11] is valid only when the eigenvectors are unit-normalized. The eigenvectors need to be unit-normalized so that they provide only direction with no magnitude, allowing the magnitude to be specified by the eigenvalue. That equation could be generalized to non-unit vectors by dividing by the magnitudes of the vectors.

[11] Eigenvector normalization is also relevant for the SVD.

On the other hand, Equation 15.39 does not require unit-normalized eigenvectors. Can you think about why thatâ€™s the case? Itâ€™s because Equation 15.39 includes the matrix inverse. Thus, $VV^{â^{'}1} = I$ regardless of the magnitude of the individual eigenvectors, whereas $VV^T = I$ only when each eigenvector is unit-normalized.

Letâ€™s see an example with a 2Ã—2 matrix.

$$A = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix} \begin{cases} \lambda_1 = -1, & \mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \\ \lambda_2 = 5, & \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \end{cases}$$

$$A_1 = v_1 \hat{I}»_1 v_1{}^T = \begin{bmatrix} -.5 & .5 \\ .5 & -.5 \end{bmatrix}$$

$$A_2 = v_2 \hat{I}»_2 v_2{}^T = \begin{bmatrix} 2.5 & 2.5 \\ 2.5 & 2.5 \end{bmatrix}$$

$$A_1 + A_2 = \begin{bmatrix} -.5 & .5 \\ .5 & -.5 \end{bmatrix} + \begin{bmatrix} 2.5 & 2.5 \\ 2.5 & 2.5 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix} = A$$

It is easy to confirm that the rank of A is 2 and the ranks of A_1 and A_2 are both 1.

Practice problems The matrices below are eigenvectors and eigenvalues. Reconstruct the matrix from which these were extracted by summing over eigenlayers. Itâ€™s safer to apply Equation 15.39 if you donâ€™t know for sure that the matrix is symmetric.

a) $\begin{bmatrix} 2 & 3 \\ -4 & 2 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$

b) $\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$

c) $\begin{bmatrix} -2 & 2 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$

Answers

a) $\begin{bmatrix} 2.5 & .75 \\ 1 & 1.5 \end{bmatrix}$

b) $\begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$

c) $\begin{bmatrix} 1.5 & 1 & 0 \\ .25 & 1.5 & 0 \\ .5 & -1 & 3 \end{bmatrix}$

-|*|- **Reflection** Who cares about eigenlayers? It may seem circuitous to deconstruct a matrix only to reconstruct it again. But consider this: do you need to sum up all of the layers? What if you would sum only the layers with the largest k < r eigenvalues? That will actually be a low-rank approximation of the original matrix. Or maybe this is a data matrix and you identity certain eigenvectors that reflect noise; you can then reconstruct the data without the "noise layers." More on this in the next few chapters! -|*|-

15.11 Matrix powers and inverse

One of the reasons that eigendecomposition has many applications is that diagonal matrices are really easy to compute with. Youâ€™ve already seen examples of this with matrix multiplication and the inverse. In this section Iâ€™m going to show you two applications of eigendecomposition that follow from the special properties of diagonal matrices.

Matrix powers Compute $A^2 = AA$ for the following two matrices:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \quad , \quad \mathbf{A} = \begin{bmatrix} 1 & 0 \\ 2 & 3 \end{bmatrix}$$

For simple matrices like this, a few minutes of pen-and-paper work will easily reveal the answer. What if the matrix were larger, and what if you had to compute $A^5 = AAAAA$?

Raising matrices to powers can be computationally intensive. But watch what happens when we raise the diagonalized version of a matrix to a higher power.

$$\mathbf{A}^2 = (\mathbf{V}\mathbf{D}\mathbf{V}^{-1})^2 = \mathbf{V}\mathbf{D}\mathbf{V}^{-1}\mathbf{V}\mathbf{D}\mathbf{V}^{-1} = \mathbf{V}\mathbf{D}^2\mathbf{V}^{-1} \tag{15.40}$$

The matrix of eigenvectors cancels with its inverse in the middle of that expression ($V^{âˆ’1}$ V = I). And you will remember from the chapter on matrix multiplication that

multiplying two diagonal matrices simply involves multiplying their diagonal elements and ignoring all the off-diagonal zeros. Letâ€™s see how this looks for A^3 .

$$\mathbf{A}^3 = (\mathbf{V}\mathbf{D}\mathbf{V}^{-1})^3 = \mathbf{V}\mathbf{D}\mathbf{V}^{-1}\mathbf{V}\mathbf{D}\mathbf{V}^{-1}\mathbf{V}\mathbf{D}\mathbf{V}^{-1} = \mathbf{V}\mathbf{D}^3\mathbf{V}^{-1} \quad (15.41)$$

You see where this is going: The general form for matrix powers is:

$$\mathbf{A}^n = (\mathbf{V}\mathbf{D}\mathbf{V}^{-1})^n = \mathbf{V}\mathbf{D}^n\mathbf{V}^{-1} \quad (15.42)$$

Therefore, once youâ€™ve completed the eigendecomposition, taking matrices to higher powers requires very few FLOPs (floating point operations, the count of basic computer arithmetic operations). Of course, if you will square a matrix only once, then it might be more hassle than advantage to diagonalize it. But Equation 15.42 becomes very efficient if you need to raise the matrix to powers multiple times.

While weâ€™re on the subject of matrix powers and eigendecomposition, we can take a small tangent to show that squaring a matrix squares its eigenvalues without changing the eigenvectors. Hereâ€™s the proof (assume v is an eigenvector of A and Î» is the associated eigenvalue):

$$\mathbf{A}^2\mathbf{v} = \mathbf{A}\mathbf{A}\mathbf{v} = \mathbf{A}\lambda\mathbf{v} = \lambda(\mathbf{A}\mathbf{v}) = \lambda^2\mathbf{v} \quad (15.43)$$

This generalizes to any power. Extending this proof to A^n [12] involves additional replacements of Av with Î»v. _____

[12] In case you were curious: $A^0 = I$

Eqn. box title=Matrix powers and eigenvalues

For eigenvalue/vector Î»v of matrix A,

$$A^n v = \lambda^n v \quad (15.44)$$

_____

This proof also shows why nilpotent matrices have zero-valued eigenvalues: If there is some k for which $A^k = 0$, then $0 = A^k v = \lambda^k v$, which means that Î» = 0 if vâ‰ 0._____

_____

Practice problems Compute A^3 for the following matrices. The eigenvalues are provided but youâ€™ll have to compute the eigenvectors.

a) $\begin{bmatrix} 1 & 0 \\ 6 & -1 \end{bmatrix}$, Î» = 1,â^’1

b) $\begin{bmatrix} 4 & -2 \\ -1 & 5 \end{bmatrix}$, Î» = 3,6

c) $\begin{bmatrix} 4 & 0 & 1 \\ -2 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}$, Î» = 1,2,3

Answers

a) $\begin{bmatrix} 1 & 0 \\ 6 & -1 \end{bmatrix}$

b) $\begin{bmatrix} 90 & -126 \\ -63 & 153 \end{bmatrix}$

c) $\begin{bmatrix} 46 & 0 & 19 \\ -38 & 1 & -12 \\ -38 & 0 & -11 \end{bmatrix}$

Matrix inverse The other application in this section follows the same logic: Diagonalize a matrix, apply some operation to the diagonal elements of Î›, then optionally reassemble the three matrices into one.

Remember from Chapter 12 that the inverse of a diagonal matrix is the diagonal elements individually inverted. Thatâ€™s the key insight to inverse-via-eigendecomposition.

$$\mathbf{A}^{-1} = (\mathbf{V}\mathbf{D}\mathbf{V}^{-1})^{-1} = (\mathbf{V}^{-1})^{-1}\mathbf{D}^{-1}\mathbf{V}^{-1} = \mathbf{V}\mathbf{D}^{-1}\mathbf{V}^{-1} \quad (15.45)$$

Of course, this procedure is valid only for matrices with all non-zero diagonal elements, which unsurprisingly excludes all singular matrices.

You might wonder whether this is really a short-cut, considering that V still needs to be inverted. There are two computational advantages of Equation 15.45 . One advantage is obviously from inverting a symmetric matrix (with an orthogonal eigenvectors matrix), where $V^{âˆ’1} = V^T$. A second advantage is that because the eigenvectors are normalized, V has a low condition number and is therefore more numerically stable. Thus, the V of a non-symmetric matrix might be easier to invert than A. (Condition number is a measure of the "spread" of a matrix that characterizes the stability of a matrix to minor perturbations or noise; youâ€™ll learn more about this quantity in the next chapter. The point is that even if A is theoretically invertible, the inverse of V may have a more accurate inverse.)

Equation 15.45 also has a theoretical advantage, because it helps build intuition for the algorithm to compute the pseudoinverse via the singular value decomposition.

Practice problems Each exercise provides a matrix, its eigenvectors, and eigenvalues. Compute $A^{âˆ’1}$ by $VD^{âˆ’1}V^{âˆ’1}$. Then confirm that $AA^{âˆ’1} = I$.

a) $\begin{bmatrix} 1 & 2 \\ 3 & -4 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix}$,Î» = 2,âˆ’5

b) $\begin{bmatrix} 3 & -2 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$, Î» = 2,1

Answers Matrices are $A^{â^'1}$

a) $\begin{bmatrix} .4 & .2 \\ .3 & -.1 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 1 \\ -.5 & 1.5 \end{bmatrix}$

15.12 Generalized eigendecomposition

One quick glance will reveal that the following two equations are equivalent.

$$Av = Î»v$$

$$Av = Î»Iv$$

What if we replace I with another (suitably sized) matrix? _____

Eqn. box title=Generalized eigenvalue equation

$$Av = Î»Bv \quad (15.46)$$

$$AV = BVÎ› \quad (15.47)$$

Generalized eigendecomposition is also called simultaneous diagonalization of two matrices and leads to several equations that are not immediately easy to interpret, including:

$$V^{â^'1} B^{â^'1} AV = Î› \quad (15.48)$$

$$A = BVÎ›V^{â^'1} \quad (15.49)$$

$$B = V^{â^'1} Î›^{â^'1} AV \quad (15.50)$$

Perhaps a better way to interpret generalized eigendecomposition is to think about "regular" eigendecomposition on a matrix product involving an inverse._____

Eqn. box title=Interpretation of generalized eigendecomposition

$$(B^{â^'1} A)v = Î»v \quad (15.51)$$

$$Cv = Î»v, C = B^{â^'1} A$$

This interpretation is valid only when B is invertible. In practice, even when B is invertible, inverting large or high-conditioned matrices can lead to numerical inaccuracies and therefore should be avoided. Computer programs will perform the

eigendecomposition without actually inverting matrices. Nonetheless, Equation 15.51 helps build intuition.

In section 6.7 I showed how the product of two symmetric matrices is generally not symmetric. That is important because even if matrices A and B are both symmetric, $B^{\hat{}\text{'}1}$ A will generally not be symmetric. That means that the special properties of eigendecomposition for symmetric matrices (orthogonal eigenvectors, real-valued eigenvalues) do not apply to generalized eigendecomposition.

Code Generalized eigendecomposition is easy to implement. Just be mindful of the order of function inputs: Following Equation 15.51 , A must be the first input and B must be the second input. Numpy cannot perform generalized eigendecomposition (at the time of this writing), but scipyâ€™s implementation can.

Code blockÂ 15.3: Python

```
importÂ numpyÂ asÂ npÂ
fromÂ scipy.linalgÂ importÂ eigÂ
nÂ =Â 3Â
AÂ =Â np.random.randn(n,n)Â
BÂ =Â np.random.randn(n,n)Â
evals,evecsÂ =Â eig(A,B)
```

Code blockÂ 15.4: MATLAB

```
nÂ =Â 3;Â
AÂ =Â randn(n);Â
BÂ =Â randn(n);Â
[evecs,evals]Â =Â eig(A,B);
```

-|*|- **Reflection** You can also think of $B^{\hat{}\text{'}1}$ A as the matrix version of a ratio of A to B. This interpretation makes generalized eigendecomposition a computational workhorse for several multivariate data science and machine-learning applications, including linear discriminant analysis, source separation, and classifiers. -|*|-

15.13 Exercises

1.Â

Find the eigenvalues of the following matrices.

a) $\begin{bmatrix} 4 \end{bmatrix}$

b) $\begin{bmatrix} 0 & 3 \\ 5 & 0 \end{bmatrix}$

c) $\begin{bmatrix} 3 & 0 \\ 0 & 5 \end{bmatrix}$

d)

$\begin{bmatrix} 2 & 5 \\ 6 & 3 \end{bmatrix}$

e) $\begin{bmatrix} -4 & 1 \\ 1 & 3 \end{bmatrix}$

f) $\begin{bmatrix} -2 & 2 \\ -3 & 2 \end{bmatrix}$

g) $\begin{bmatrix} 2 & 5 & -1 \\ 0 & 4 & 3 \\ 0 & 0 & 1 \end{bmatrix}$

h) $\begin{bmatrix} a & 34 & \sqrt{23} \\ 0 & b & e^{i\pi^3} \\ 0 & 0 & c \end{bmatrix}$

i) $\begin{bmatrix} a & 0 & 0 \\ 34 & b & 0 \\ \sqrt{23} & e^{i\pi^3} & c \end{bmatrix}$

2.Â

Diagonalize the following matrices.

a) $\begin{bmatrix} 1 & 1 \\ -3 & 5 \end{bmatrix}$

b) $\begin{bmatrix} -1 & 0 \\ -1 & 0 \end{bmatrix}$

c) $\begin{bmatrix} 4 & 0 & 1 \\ -2 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}$

3.Â

The following pairs of matrices show a matrix and its eigenvectors. Without computing eigenvalues, determine the missing eigenvector component.

a) $\begin{bmatrix} -2 & 2 \\ 2 & 1 \end{bmatrix}, \begin{bmatrix} -2 & 1 \\ * & 2 \end{bmatrix}$

b) $\begin{bmatrix} 52 & 16 \\ 16 & 28 \end{bmatrix}, \begin{bmatrix} 2 & * \\ -4 & -2 \end{bmatrix}$

c) $\begin{bmatrix} 3 & -3 \\ -3 & 3 \end{bmatrix}, \begin{bmatrix} * & 1 \\ 1 & -1 \end{bmatrix}$

4.Â

I wrote that finding eigenvectors involves computing the null space of Aâˆ’Î»I. What would happen if you started from Î»I â^’ A? Try this using the matrix in Equation 15.15 (page Â§) to see whether the results differ. Either way, explain why this happens.

5.Â

Left-multiply both sides of the eigenvalue equation the eigenvalue equation (Av = Î»v) by v^T . Assume that âˆ¥vâˆ¥ = 1 and simplify. What can you conclude?

15.14 Answers

1.Â

Following are the missing pieces.

a) 4
b) Â±$\sqrt{15}$
c) 3, 5
d) -3, 8
e) (âˆ’1 Â±$\sqrt{53}$)âˆ•2
f) Â±$\sqrt{2}$ i
g) 2,4,1
h) a,b,c
i) a,b,c

2.Â

Matrices below are eigenvalues, eigenvectors (non-normalized).

a) $\begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 3 \end{bmatrix}$
b) $\begin{bmatrix} -1 & 0 \\ 0 & 0 \end{bmatrix}, \begin{bmatrix} \pi & 0 \\ \pi & 1 \end{bmatrix}$
c) $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 2 \end{bmatrix}, \begin{bmatrix} 0 & 1 & -1 \\ 1 & -1 & 2 \\ 0 & -1 & 2 \end{bmatrix}$

3.Â

Following are the eigenvector values.

a) âˆ— = 1
b) âˆ— = âˆ’4
c) âˆ— = 1

4.Â

You get the same results: Same eigenvalues and same eigenvectors. Itâ€™s a bit awkward in my opinion, because you have to flip the signs of all the matrix elements, but conceptually itâ€™s the same thing as writing (a âˆ’ b) = âˆ’(b âˆ’ a).

5.Â

Pre- and post-multiplying a matrix by a vector produces a scalar. In a few chapters, I will call the general expression v^T Av the quadratic form, which will reveal some interesting properties about the definiteness and eigenvalues of the matrix.

The second thing to notice is that the matrix reduces to its eigenvalue when transformed on both sides by its eigenvector. That is: v^T Av = Î». Again, this leads to some interesting discoveries about the matrix, which you will learn about in Chapter 17.

15.15 Code challenges

1.Â
In this code challenge, you will explore the relationship between Equations 15.46 and 15.51 in the section on generalized eigendecomposition. Create two matrices A and B. Then perform (1) generalized eigendecomposition of both matrices, and (2) "regular" eigendecomposition on the matrix $B^{â^{\wedge}'1}$ A; inspect whether the eigenvalues are the same. Try this for 2Ã—2 matrices, and then again for larger matrices like 10Ã—10 or 50Ã—50. Do you also need to compare the eigenvectors or is it sufficient to compare the eigenvalues?

2.Â
In this chapter, you learned that diagonalization means to transform a matrix into a diagonal matrix, with the eigenvectors providing the transformation matrix. What happens if the matrix A is already a diagonal matrix? What are its eigenvalues and eigenvectors? Use code to explore this empirically by taking the eigendecomposition of diagonal matrices. Then explain why you get those results.

3.Â
In Chapter 5 you learned about Hankel matrices (Â§), and I mentioned that they have aesthetically pleasing properties in eigendecomposition. Now is your chance to explore this. Create a 50Ã—50 Hankel matrix and take its eigendecomposition. Sort the eigenvectors according to descending eigenvalues. Then produce a figure that shows Hankel matrix and its eigenvectors matrix. Then plot the first few eigenvectors.

15.16 Code solutions

1.Â
The answer is that for random-numbers matrices, the two approaches produce basically identical eigenvalues in MATLAB, but different results in Python. These are very difficult numerical problems to solve. The two methods are similar in MATLAB because the matrices are invertible and have a low condition number. In the next chapter, youâ€™ll see that this conclusion changes for matrices with a larger condition number (youâ€™ll learn about condition number in the next chapter). It is not necessary to compare the eigenvectorsâ€”if the eigenvalues differ,

so do the eigenvectors. When comparing eigenvalues, itâ€™s important to make sure theyâ€™re sorted!

Code blockÂ 15.5: Python

```
importÂ numpyÂ asÂ npÂ
fromÂ scipy.linalgÂ importÂ eigÂ
importÂ matplotlib.pyplotÂ asÂ pltÂ
Â
avediffsÂ =Â np.zeros(100)Â
forÂ nÂ inÂ range(1,101):Â
Â Â Â Â AÂ =Â np.random.randn(n,n)Â
Â Â Â Â BÂ =Â np.random.randn(n,n)Â
Â Â Â Â l1Â =Â eig(A,B)[0]Â
Â Â Â Â l2Â =Â eig(np.linalg.inv(B)@A)[0]Â
Â
Â Â Â Â #Â importantÂ toÂ sortÂ eigvalsÂ
Â Â Â Â l1.sort()Â
Â Â Â Â l2.sort()Â
Â
Â Â Â Â avediffs[n-1]Â =Â np.mean(np.abs(l1-l2))Â
Â
plt.plot(avediffs);
```

Code blockÂ 15.6: MATLAB

```
forÂ n=1:100Â
Â Â Â Â AÂ =Â randn(n);Â
Â Â Â Â BÂ =Â randn(n);Â
Â Â Â Â l1Â =Â eig(A,B);Â
Â Â Â Â l2Â =Â eig(inv(B)*A);Â
Â
Â Â Â Â %Â importantÂ toÂ sortÂ evalsÂ
Â Â Â Â l1Â =Â sort(l1);Â
Â Â Â Â l2Â =Â sort(l2);Â
Â Â Â Â avediffs(n)Â =Â mean(abs(l1-l2));Â
endÂ
plot(avediffs,â€™s-â€™)Â
xlabel(â€™MatrixÂ sizeâ€™)Â
ylabel(â€™\Delta\lambdaâ€™)
```

2.Â

Conveniently, the eigenvalues of a diagonal matrix are simply the diagonal elements, while the eigenvectors matrix is the identity matrix. This is because (A â^’ Î»I) is made singular simply by setting Î» to each diagonal element.

In fact, the eigenvalues of any triangular matrix (including diagonal matrices) are simply the elements on the main diagonal (see exercise question 1g-i). However, V = I only for a diagonal matrix. The eigenvectors matrix of a triangular matrix is itself triangular. Thatâ€™s not shown in the code below, but I encourage you to test it!

Code blockÂ 15.7: Python

```
importÂ numpyÂ asÂ npÂ
DÂ =Â np.diag(range(1,6))Â
L,VÂ =Â np.linalg.eig(D)
```

Code blockÂ 15.8: MATLAB

```
DÂ =Â diag(1:5);Â
[V,L]Â =Â eig(D)
```

3.Â

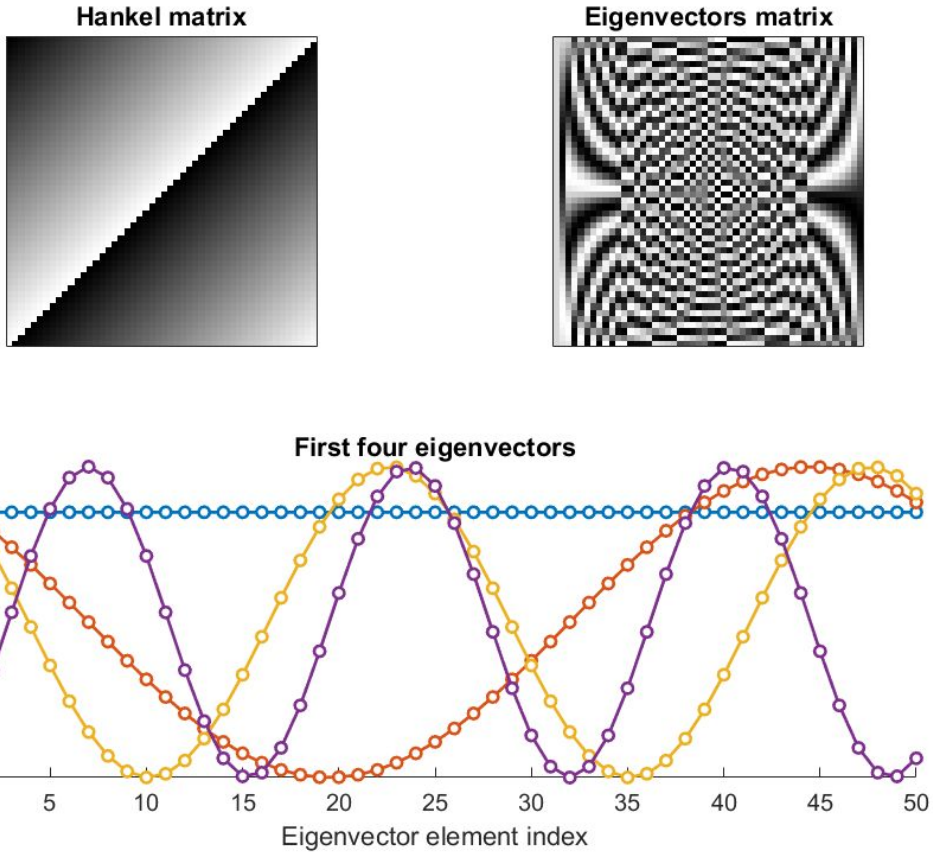
The eigenvectors matrix looks cute, like a pixelated flower from a 1970â€™s computer. Plotting the eigenvectors reveals their remarkable propertyâ€”theyâ€™re sine waves! (Figure is shown below the code.) If you are familiar with signal processing, then this might look familiar from the Fourier transform. In fact, there is a deep connection between the Hankel matrix and the Fourier transform. By the way, you can run this code on a Toeplitz matrix for comparison (spoiler alert: Not nearly as cool!).

Code blockÂ 15.9: Python

```
importÂ numpyÂ asÂ npÂ
importÂ matplotlib.pyplotÂ asÂ pltÂ
fromÂ scipy.linalgÂ importÂ hankelÂ
Â
tÂ =Â np.arange(1,51)Â
lstrowÂ =Â np.append(t[-1],np.arange(1,t[-1]))Â
HÂ =Â hankel(t,r=lstrow)Â
d,VÂ =Â np.linalg.eig(H)Â
VÂ =Â V[:,np.argsort(d)[::-1]]Â
Â
plt.subplot(221)Â #Â theÂ matrixÂ
plt.imshow(H)Â
plt.subplot(222)Â #Â theÂ eigenvectorsÂ
plt.imshow(V)Â
plt.subplot(212)Â #Â someÂ evecsÂ
plt.plot(V[:,:4]);
```

Code blockÂ 15.10: MATLAB

```
NÂ =Â 50;Â
TÂ =Â toeplitz(1:N);Â
HÂ =Â hankel(1:N,[NÂ 1:N-1]);Â
Â
[V,D]Â =Â eig(H);Â
[~,sidx]Â =Â sort(diag(D),â€™descendâ€™);Â
VÂ =Â V(:,sidx);Â
Â
subplot(221)Â %Â theÂ matrixÂ
imagesc(H),Â axisÂ squareÂ
subplot(222)Â %Â allÂ eigenvectorsÂ
imagesc(V),Â axisÂ squareÂ
subplot(212)Â %Â aÂ fewÂ evecsÂ
plot(V(:,1:4),â€™o-â€™)
```



figureImage for code challenge 3.

ChapterÂ 16
Singular value decomposition

16.1 Singular value decomposition

Singular value decomposition, or SVD, is closely related to eigendecomposition. In fact, eigendecomposition can be seen as a special case of the SVD in which the to-be-decomposed (decomposing?) matrix is square. Alternatively, SVD can be seen as a generalization of eigendecomposition to a matrix of any size. This latter interpretation is more consistent with how it will be introduced.

Before learning about the mechanics of the SVD, I want to present the big picture and the terminology. The core idea of SVD is to provide sets of basis vectorsâ€”called singular vectorsâ€”for the four matrix subspaces: the row space, the null space, the column space, and the left-null space; and to provide scalar singular values that encode the "importance" of each singular vector. The singular vectors are similar to eigenvectors, and the singular values are similar to eigenvalues (though their exact relationships are complicated and will be described later). The full SVD is expressed using the following matrix letters.

Eqn. box title=Singular value decomposition

$$A = U\hat{I}\pounds V^{T} \tag{16.1}$$

A The MÃ—N matrix to be decomposed. It can be square or rectangular, and any rank.

U The left singular vectors matrix (MÃ—M), which provides an orthonormal basis for â„ ^{M}. This includes the column space of A and its complementary left-null space.

Î£ The singular values matrix (MÃ—N), which is diagonal and contains the singular values (the ith singular value is indicated Ïf_{i}). All singular values are non-negative (that is, positive or zero) and real-valued.

V The right singular vectors matrix (NÃ—N), which provides an orthonormal basis for â„ ^{N}. That includes the row space of A and its complementary null space. Notice that the decomposition contains V^{T} ; hence, although the right singular vectors are in the columns of V, it is usually more convenient to speak of the right singular vectors as being the rows of V^{T} .

As you can see in the descriptions above, the sizes of the right-hand-side matrices depend on the size of A. Figure 16.1 shows graphical representations of the SVD for square, tall, and wide rectangular matrices.

$$44MM\mathbf{A} = 44MM\mathbf{U}\,44MM\Sigma\,44MM\mathbf{V}^{\mathrm{T}}$$

$$53MN\mathbf{A} = 55MM\mathbf{U}\,53MN\Sigma\,33NN\mathbf{V}^{\mathrm{T}}$$

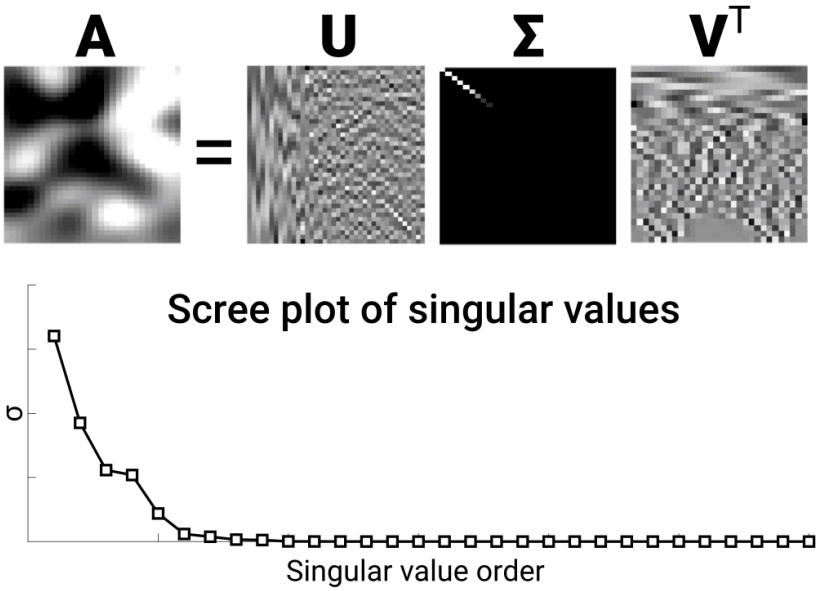
$$35MN\mathbf{A} = 33MM\mathbf{U}\,35MN\Sigma\,55NN\mathbf{V}^{\mathrm{T}}$$

FigureÂ 16.1: The sizes of SVD matrices for different sizes of the to-be-decomposed matrix.

Notice that the size of U corresponds to the number of rows in A, that the size of V corresponds to the number of columns in A, and that the size of Î£ is the same as that of A. These sizes allow for U to be an orthonormal basis for â„ ^{M} and for V to be an orthonormal basis for â„ ^{N}.

Letâ€™s see what the SVD looks like for a real matrix. Figure 16.2 shows a matrix (created by applying a 2D smoothing kernel to random noise; the grayscale intensity at each pixel in the image is mapped onto the numerical value at that element of the matrix) and its SVD.

So thatâ€™s the purpose of the SVD: Represent a matrix using two orthogonal matrices surrounding a diagonal matrix. Now letâ€™s figure out how to create those three matrices, and then the rest of the chapter will give you opportunities to develop intuition for why the SVD is so useful.



FigureÂ 16.2: Visualization of the SVD of a smoothed random numbers matrix. The lower plot shows the singular values, which are the diagonal of the ÎŁ matrix. This plot is called a "scree plot" in statistics nomenclature. The singular vectors are in the columns of U and the rows of V^T .

16.2 Computing the SVD

Given the importance of the SVD in theoretical and applied linear algebra, you might think that the SVD is really difficult to compute. Well, youâ€™re wrong (if thatâ€™s what you thought). In fact, once you know eigendecomposition, the SVD is almost trivial to compute.

Letâ€™s start by considering the eigendecomposition of matrix A of size Mâ‰ N. Actually, eigendecomposition is not defined for this non-square matrix. However, $A^T A$ is eigendecomposable. Replacing $A^T A$ with its SVD matrices gives us the following.

$$^{1}\ A^T A = (U\Sigma V^T)^T (U\Sigma V^T) \quad (16.2)$$

$$= V\Sigma^T U^T U\Sigma V^T \quad (16.3)$$

$$= V\Sigma^2 V^T \quad (16.4)$$

Weâ€™ve now discovered how to find the V and ÎŁ matrices from the SVD: V are the eigenvectors of $A^T A$ and ÎŁ are the squared eigenvalues of $A^T A$. Remember that because ÎŁ is a diagonal matrix, then Σ^2 simplifies to the diagonal elements (the eigenvalues) squared.

[1] U is orthogonal, ergo $U^T U = I$.

You can immediately see why the singular values are non-negativeâ€”any real number squared will be non-negative. This doesnâ€™t show immediately why the singular values are real-valued, though; that comes from the proof that the eigenvalues of a symmetric matrix are real-valued (page Â§).

Weâ€™re missing the U matrix, but Iâ€™m sure youâ€™ve already figured it out: Take the eigendecomposition of matrix AA^T :

$$AA^T = (U\Sigma V^T)(U\Sigma V^T)^T \quad (16.5)$$

$$= U\Sigma V^T V\Sigma^T U^T \quad (16.6)$$

$$= U\Sigma^2 U^T \quad (16.7)$$

So there you have itâ€”the way to compute the SVD of any rectangular matrix is to apply the following steps:

1. Compute the eigendecomposition of $A^T A$ to get $\hat{\Sigma}$ and V.
2. Compute the eigendecomposition of AA^T to obtain U.

It is actually not necessary to compute both steps to obtain the SVD. After applying either one of these steps, you can compute the third matrix directly using one of the following two formulas.

$$\mathbf{AV}\Sigma^{-1} = \mathbf{U}$$

$$\hat{\Sigma}^{â^{\wedge}1} U^T A = V^T$$

Quick question: How do you know that U and V are orthogonal matrices? Think of your answer and then check the footnote[2].

I explained earlier in this chapter that the $\hat{\Sigma}$ matrix is the same size as A, and yet computing $\hat{\Sigma}^T \hat{\Sigma}$ actually produces a square matrix, even if A is rectangular. Thus, in practice, the "real" $\hat{\Sigma}$ matrix is the same size as A and its diagonal elements are drawn from the diagonal elements of $\hat{\Sigma}^T \hat{\Sigma}$. Equivalently, you could cut out the "excess" rows or columns from square $\hat{\Sigma}^T \hat{\Sigma}$ to trim it down to the size of A.

[2] Because they come from the eigendecomposition of a symmetric matrix.

When computing SVD by handâ€”which I recommend doing at least a few times to solidify the conceptâ€”you should first decide whether to apply step 1 and then Equation 16.8 , or step 2 and Equation 16.8 . The best strategy depends on the size of the matrix, because you want to compute the eigendecomposition of whichever of $A^T A$ or AA^T is smaller. On the other hand, following steps 1 and 2 is easier than it looks, because you need to solve for the squared eigenvalues (the singular values on the diagonal of $\hat{\Sigma}^2$) only once.

Normalizing singular vectors Singular vectors, like eigenvectors, are important because of their direction. Thus, in some sense, normalizing each singular vector to be unit length is arbitrary. However, the goal of the SVD is to have an exact decomposition of the matrix. If the singular vectors are not properly normalized, then Aâ‰ U$\hat{\Sigma}$$V^T$.

Therefore, in practice, all singular vectors must be normalized to unit vectors. For the same reason, the signs of the singular vectors are not arbitrary. The singular values are all non-negative, so flipping signs of singular vectors may be necessary for reconstructing the matrix.

This is noticeably different from diagonalization via eigendecomposition, where the eigenvectors are sign- and magnitude-invariant. The key to understanding the difference is that the eigenvalues matrix is flanked on both sides by the eigenvectors matrix and its inverse (V$\hat{\Sigma}$›$V^{â^{\wedge}1}$); any non-unit-magnitudes in V can be absorbed into $V^{â^{\wedge}1}$. But in the SVD, U and V^T are not inverses of each other (indeed, they may even have different dimensionalities), and thus the magnitude of singular vectors is not canceled.

You can see the effects of not normalizing in the exercises below. It will take you a while to work through these two exercises by hand, but I believe itâ€™s worth the effort and will help you understand the SVD.

Practice problems Perform the following steps on each matrix below: (1) Compute the SVD by hand. Construct the singular vectors to be integer matrices for ease of visual inspection. (2) Using your non-normalized singular vectors, compute U$\hat{\Sigma}$$V^T$ to confirm that it does not equal the original matrix. (3) Normalize each singular vector (each column of U and row of V^T). (4) Re-compute U$\hat{\Sigma}$$V^T$ and confirm that it exactly equals the original matrix.

a) $\begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$

b) $\begin{bmatrix} 1 & -1 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}$

Answers Matrices are ordered as U, $\hat{\Sigma}$, and V^T . Top rows show integer vectors; bottom rows show normalized versions.

a) $\begin{bmatrix}1 & -1\\ 1 & 1\end{bmatrix}, \begin{bmatrix}\sqrt{3} & 0 & 0\\ 0 & 1 & 0\end{bmatrix}, \begin{bmatrix}1 & 2 & 1\\ 1 & 0 & -1\\ 1 & -1 & 1\end{bmatrix}$; product = $\begin{bmatrix}\sqrt{3}-1 & 2\sqrt{3} & \sqrt{3}+1\\ \sqrt{3}+1 & 2\sqrt{3} & \sqrt{3}-1\end{bmatrix}\frac{-1}{\sqrt{2}}\begin{bmatrix}1 & -1\\ 1 & 1\end{bmatrix}, \begin{bmatrix}\sqrt{3} & 0 & 0\\ 0 & 1 & 0\end{bmatrix}$,
$\begin{bmatrix}-1/\sqrt{6} & -2/\sqrt{6} & -1/\sqrt{6}\\ 1/\sqrt{2} & 0 & -1/\sqrt{2}\\ 1/\sqrt{3} & -1/\sqrt{3} & 1/\sqrt{3}\end{bmatrix}$; product=$\begin{bmatrix}1 & 1 & 0\\ 0 & 1 & 1\end{bmatrix}$

b) $\begin{bmatrix}1 & 1 & -1\\ -2 & 0 & -1\\ -1 & 1 & 1\end{bmatrix}, \begin{bmatrix}\sqrt{6} & 0\\ 0 & \sqrt{2}\\ 0 & 0\end{bmatrix}, \begin{bmatrix}0 & 1\\ 1 & 0\end{bmatrix}$; product=$\begin{bmatrix}\sqrt{2} & \sqrt{6}\\ 0 & -2\sqrt{6}\\ \sqrt{2} & -\sqrt{6}\end{bmatrix}\begin{bmatrix}1/\sqrt{6} & -1/\sqrt{2} & -1/\sqrt{3}\\ -2/\sqrt{6} & 0 & -1/\sqrt{3}\\ -1/\sqrt{6} & -1/\sqrt{2} & 1/\sqrt{3}\end{bmatrix}, \begin{bmatrix}\sqrt{6} & 0\\ 0 & \sqrt{2}\\ 0 & 0\end{bmatrix}$,
$\begin{bmatrix}0 & -1\\ -1 & 0\end{bmatrix}$; product=$\begin{bmatrix}1 & -1\\ 0 & 2\\ 1 & 1\end{bmatrix}$

16.3 Singular values and eigenvalues

The previous section seemed to imply the trivial relationship that the eigenvalues of $A^T A$ equal the squared singular values of A. That is true, but there is a more nuanced relationship between the eigenvalues and the singular values of a matrix. That relationship is organized into three cases.

Case 1: eig($A^T A$) vs. svd(A) The eigenvalues equal the squared singular values, for the reasons explained in the previous section. Letâ€™s see a quick example:

$$A = \begin{bmatrix}3 & 1 & 0\\ 1 & 1 & 0\end{bmatrix}$$

$$A^T A = \begin{bmatrix}10 & 4 & 0\\ 4 & 2 & 0\\ 0 & 0 & 0\end{bmatrix}$$

Î»($A^T A$) = 0, .3431, 11.6569

Ïƒ(A) = .5858, 3.4142

Ïƒ^2 (A) = .3431, 11.6569

Why are there three Î»s but only two Ïƒs? Itâ€™s because $A^T A$ is 3Ã—3 but Î£ has the same size as the matrix A, which is 2Ã—3; hence, the diagonal has only two elements. But the non-zero Î»â€™s equal the squared Ïƒâ€™s.

This case concerns the eigenvalues of $A^T A$, not the eigenvalues of A. In fact, there are no eigenvalues of A because it is not a square matrix. This brings us to our second case.

Case 2: eig($A^T A$) vs. svd($A^T A$) In this case, the eigenvalues and singular values are identicalâ€”without squaring the singular values. This is because eigendecomposition and SVD are the same operation for a square symmetric matrix (more on this point later).

Case 3a: eig(A) vs. svd(A) for real-valued Î» This is different from Case 2 because here we assume that A is not symmetric, which means that the eigenvalues can be real-valued or complex-valued, depending on the elements in the matrix. We start by considering the case of a matrix with all real-valued eigenvalues. Of course, the matrix does need to be square for it to have eigenvalues, so letâ€™s add another row to the example matrix above.

$$A = \begin{bmatrix}3 & 1 & 0\\ 1 & 1 & 0\\ 1 & 1 & 1\end{bmatrix}$$

Î»(A) = .5858, 1, 3.4142

Ïƒ(A) = .4938, 1.1994, 3.6804

There is no easy-to-spot relationship between the eigenvalues and the singular values. In fact, there isnâ€™t really a relationship at all.[3] Of course, there is the relationship that $W\Lambda W^{-1} = U\Sigma V^T$, in other words, the entire

eigendecomposition equals the entire SVD (because they are both equal to A). But that doesnâ€™t say anything about a necessary relationship between Î› and Î£.

[3] Here Iâ€™m using W for eigenvectors to avoid confusion with the right singular vectors V.

Case 3b: eig(A) vs. svd(A) for complex-valued Î» The lack of trivial relationship between eigenvalues and singular values is even more apparent when a matrix has complex-valued eigenvalues. You learned in the previous chapter that a real-valued matrix can have complex-valued eigenvalues. So if A has complex-valued eigenvalues, what can we say about its singular values?

In fact, the singular values of any matrixâ€”real-valued or complex-valuedâ€”are always real-valued. Guaranteed. Why? Because the SVD can be obtained from the eigendecomposition of the matrix times its transpose, and that matrix is always symmetric. (The SVD of complex matrices uses the Hermitian transpose instead of the regular transpose.)

Cases 2 and 3 may initially seem contradictory. $A^T A$ is simply a matrix, so if we set $C = A^T A$, then Iâ€™ve written that Î»(C)=Ïƒ(C) but Î»(A)â‰ Ïƒ(A). Would you like to guess the difference and then the reason for the difference before reading the next paragraph?

The difference is that C is defined as a matrix times its transpose, whereas A is not. And the reason why this matters is that the 2-norm of the [4] eigenvectors matrix of a symmetric matrix is 1, whereas the 2-norm of the eigenvectors matrix for a non-symmetric matrix is not constrained to be 1. This isnâ€™t about the individual eigenvectorsâ€”computer programs will unit-norm each individual eigenvectorâ€”this is about the eigenvectors and singular vectors matrices being orthogonal matrices, and thus having a norm of 1.

[4] The matrix induced 2-norm is the most that a matrix can stretch a vector, and is computed as the largest singular value of the matrix.

Eigendecomposition and SVD are exact decompositions, thus, all of the "energy" in the matrix A must be contained inside the three eigendecomposition or SVD matrices. For the SVD, matrices U and V [5] are orthogonal and have a matrix norm of 1, which means that all of the energy in the right-hand side of the equation must come from matrix Î£. Thatâ€™s always the case for the SVD.

[5] all $\sigma(U) = \sigma(V) = 1$

Eigendecomposition, on the other hand, has an orthogonal eigenvectors matrix only when the matrix is symmetric. When the matrix is not symmetric, then the "total energy" in the matrix can be distributed over the eigenvectors and the eigenvalues. This is the reason why there seems to be a discrepancy between cases 2 and 3, which is actually due to the symmetry of the matrix. (For a related discussion, consult section 15.10 on reconstructing a matrix from eigenlayers.)

In conclusion, there is a clear relationship between the eigenvalues of $A^T A$ and the singular values of A (or the singular values of $A^T A$), but there is no relationship between the eigenvalues of non-symmetric A and the singular values of A.

Code As you might guess, the SVD is really easy to compute in Python and in MATLAB. If you use both programs, be very mindful that Python returns V^T whereas MATLAB returns V. Python also returns the singular values in a vector instead of in a diagonal matrix. You can use the code below to confirm the answers to the practice problems a few pages ago.

Code blockÂ 16.1: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â [[1,1,0],[0,1,1]]Â
U,s,VÂ =Â np.linalg.svd(A)
```

Code blockÂ 16.2: MATLAB

```
AÂ =Â [1Â 1Â 0;Â 0Â 1Â 1];Â
[U,S,V]Â =Â svd(A);
```

16.4 SVD of a symmetric matrix

Simply stated: The left and right singular vectors of a symmetric matrix are the same. In other words,

---

Eqn. box title=SVD of a symmetric matrix

$$A = U\hat{I}\pounds U^T \text{ , if } A = A^T \quad (16.9)$$

Proving that this is the case simply involves writing out the SVD of the matrix and its transpose:

$$A = U\hat{I}\pounds V^T \quad (16.10)$$

$$A^T = (U\hat{I}\pounds V^T)^T = V\hat{I}\pounds U^T \quad (16.11)$$

Because $A = A^T$, these two these equations must be equal:

$$\mathbf{U\Sigma V}^T = \mathbf{V\Sigma U}^T \quad (16.12)$$

And this proves that U = V, which means that the left and right singular vectors are the same.

Notice that this is not necessarily the same thing as "Case 2" of the relationship between singular values and eigenvalues, because not all symmetric matrices can be expressed as some other matrix times its transpose.

16.5 SVD and the four subspaces

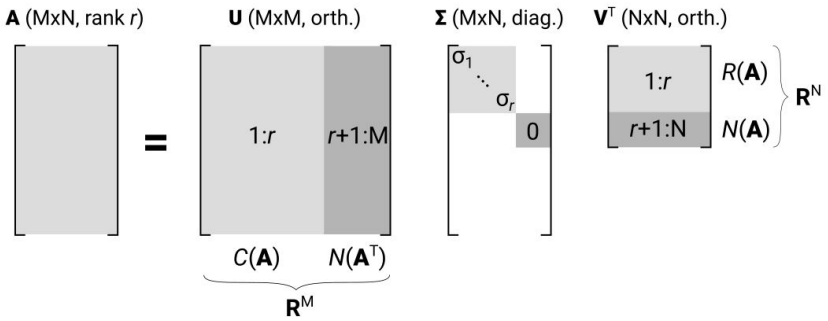
One of the remarkable features of the SVD is that it provides orthogonal basis sets for each of the four matrix subspaces. This is one of the main reasons why the SVD is such a powerful and useful decomposition.

But let me start by talking more about Î£â€”the matrix that contains the singular values on the diagonal and zeros everywhere else. Below is an example of a Î£ matrix for a 5Ã—3, rank-2 matrix.

$$\begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

By construction, $Ïf_1 > Ïf_2$. Indeed, SVD algorithms always sort the singular values descending from top-left to lower-right. Thus, zero-valued singular values will be in the lower-right of the diagonal. $Ïf_3 = 0$ because this is a rank-2 matrix. Youâ€™ll learn below that any zero-valued singular values correspond to the null space of the matrix. Therefore, the number of non-zero singular values corresponds to the dimensionality of the row and column spaces, which means that the number of non-zero singular values corresponds to the rank of the matrix. In fact, this is how programs like MATLAB and Python compute the rank of a matrix: Take its SVD and count the number of non-zero singular values.

Figure 16.3 shows the "big picture" of the SVD and the four subspaces. There is a lot going on in that figure, so letâ€™s go through each piece in turn. The overall picture is a visualization of Equation 16.1 ; as a quick review: the matrix gets decomposed into three matrices: U provides an orthogonal basis for $â„^M$ and contains the left singular vectors; Î£ is the diagonal matrix of singular values (all non-negative, real-valued); and V provides an orthogonal basis for $â„^N$ (remember that the SVD uses V^T , meaning that the rows are the singular vectors, not the columns).



FigureÂ 16.3: Visualization of how the SVD reveals basis sets for the four subspaces of a rank-r matrix. (orth. = orthogonal matrix; diag. = diagonal matrix; 1:r = columns (or rows) 1 through r.)

This figure additionally shows how the columns of U are organized into basis vectors for the column space (light gray) and left-null space (darker gray); and how the rows of V^T are organized into basis vectors for the row space (light gray) and null space (darker gray). In particular, the first r columns in U, and the first r rows in V^T , are the bases for the column and row spaces of A. The columns and rows after r get multiplied by the zero-valued singular values, and thus form bases for the null spaces. The singular vectors for the column and row spaces are sorted according to their "importance" to the matrix A, as indicated by the relative magnitude of the corresponding singular values.

You can see that the SVD reveals a lot of important information about the matrix. The points below are implicit in the visualization, and written explicitly in the interest of clarity:

- The rank of the matrix (r) is the number of non-zero singular values.
- The dimensionality of the left-null space is the number of columns in U from r + 1 to M.
- The dimensionality of the null space is the number of rows in V^T from r + 1 to N.

It is important the realize that the organization of the SVD matrix in Figure 16.3 is not a trivial result of the decomposition. Recall from the previous chapter that eigenvalues have no intrinsic sorting; likewise, when you compute the SVD by hand, the singular values are not magically revealed in descending order. Computer algorithms will sort the singular valuesâ€”and their associated singular vectorsâ€”to produce this beautiful arrangement.

Figure 16.3 also nicely captures the fact that the column space and left-null space are orthogonal: If each column in U is orthogonal to each other column, then any subset of columns is orthogonal to any other non-overlapping subset of columns. Together, all of these columns span all of â„ ^{M} , which means the rank of U is M, even if the rank of A is r < M.

The story for V^T is the same, except we deal with rows instead of columns (or, if you prefer, the columns of V) and the row space and null space of A. So, the first r rows provide an orthonormal basis for the row space of A, and the rest of the rows, which get multiplied by the zero-valued singular values, are a basis for the null space of A.

Practice problems The following triplets of matrices are UÎ£V^T that were computed from a matrix A that is not shown. From visual inspection, determine the size and rank of A, and identify the basis vectors for the four spaces of A. (Note: I re-scaled U and V to integers.)

a) $\begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 4.89 & 0 & 0 \\ 0 & 2 & 0 \end{bmatrix} \begin{bmatrix} -1 & -1 & -1 \\ 1 & 0 & -1 \\ 1 & -2 & 1 \end{bmatrix}$

b)

$$\begin{bmatrix} -1 & 0 & -3 & 0 \\ 0 & 1 & 0 & 0 \\ -3 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \pi & 0 \\ 0 & 2 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$$

Answers

a) The matrix is 2Ã—3 with rank 2. U is a basis for the column space; the left-null space is empty. The first 2 rows of V^T are a basis for the row space, and the third row is a basis for the null space.

b) The matrix is 4Ã—2 with rank 2. The first two columns of U are a basis for the column space; the latter two are a basis for the left-null space. All of V^T is a basis for the row space, and the null space is empty.

-|*|- Reflection I know, Figure 16.3 is a lot to take in of first glance. Donâ€™t expect to understand everything about the SVD just by staring at that figure. Youâ€™ll gain more familiarity and intuition about the SVD by working with it, which is the goal of the rest of this chapter! -|*|-

16.6 SVD and matrix rank

It may seems strange that the number of non-zero eigenvalues does not necessarily equal the matrix rank (demonstrated in the previous chapter), whereas above I claimed that the number of non-zero singular values is equal to the matrix rank. In this section, I will provide two explanations for this.

One key difference between eigendecomposition and SVD is that for SVD, the two singular vectors matrices span the entire ambient spaces (â„ ^{M} and â„ ^{N}), which is not necessarily the case with eigenvectors (e.g., for non-diagonalizable matrices).

Given that matrix rank is the dimensionality of the column space, it is sensible that the rank of the matrix corresponds to the number of columns in U that provide a basis for the column space (we could say the same thing about the number of rows in V^T that provide a basis for the row space). Thus, it is sufficient to demonstrate that each column in U that is in the column space of A has a non-zero singular value, and that each column in U that is in the left-null space of A has a singular value of zero. (Again, the same is true of V^T, the row space, and the null space.)

Letâ€™s start by rewriting the SVD using one pair of singular vectors and their corresponding singular value. This is analogous to the single-vector eigenvalue equation.

$$\mathbf{Av} = \mathbf{u}\sigma \tag{16.13}$$

Notice that replacing the vectors with matrices and then right-multiplying by V^T gives the SVD matrix equation that youâ€™re now familiar with.

Now let me remind you of the definition of the column space and left-null space: The column space comprises all vectors that can be expressed by some combination of the columns of A, whereas the left-null space comprises all non-trivial combinations of the columns of A that produce the zeros vector. In other words:

$$C(A) \quad : Ax = b \tag{16.14}$$

$$N(A^T) \quad : Ay = 0 \tag{16.15}$$

Now we can think about Equation 16.13 in this context: all singular vectors are non-zeros, and thus the right-hand side of Equation 16.13 must be non-zeroâ€”and thus in the column space of Aâ€”if Ïƒ is non-zero. Likewise, the only possible way for the right-hand side of Equation 16.13 to be the zeros vectorâ€”and thus in the left-null space of Aâ€”is for Ïƒ to equal zero.

Thus, any u with a corresponding non-zero Ïƒ is in the column space of the matrix, whereas any u with a corresponding zero-valued Ïƒ is in the left-null space of the matrix. The elements of v provide those weightings across the columns of A.

You can make the same argument for the row space, by starting from the equation $u^T A = Ïƒv^T$.

I hope that explanation provides intuition. There is another way to explain why the rank of A corresponds to the number of non-zero singular values. This second explanation comes from the rule about the rank of the product of matrix multiplications.

A is the product of the three SVD matrices, therefore the rank of A is constrained by the ranks of those matrices. U and V are by definition full-rank. Î£ is of size MÃ—N but could have a rank smaller than M or N. Thus, the maximum possible rank of A is the rank of Î£. However, the rank of A could not be smaller than the rank of Î£, because the ranks of A and UÎ£V^T are equal. Therefore, the rank of A must equal the smallest rank of the three SVD matrices, which is always Î£. And as a diagonal matrix, the rank of Î£ is the number of non-zero diagonal elements, which is the number of non-zero singular values.

"Effective" rank Youâ€™ve read many times in this book that computers have difficulties with really small and really large numbers. These are called rounding errors, precision errors, underflow, overflow, etc. How does a computer decide whether a singular value is small but non-zero vs. zero with rounding error?

The MATLAB source code for computing rank looks like this:

```
sÂ =Â svd(A);
rÂ =Â sum(sÂ >Â tol);
```

In other words, retrieve the singular values, and count the number of those values that are above a tolerance (variable tol). So the question is, how to set that tolerance? If itâ€™s too small, then the rank will be over-estimated; if itâ€™s too large, then the rank will be under-estimated. MATLABâ€™s solution is to set the tolerance dynamically, based on the size and elements in the matrix:

```
tolÂ =Â max(size(A))Â *Â eps(max(s));
```

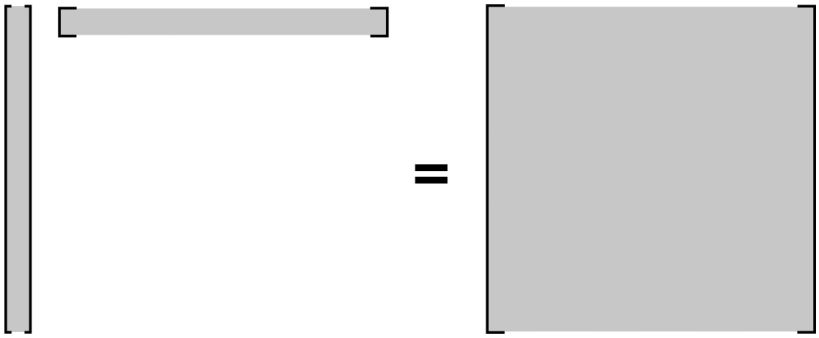
The function eps returns the distance between a number and the next-larger number that your computer is capable of representing. For example, if your computer could only represent integers, then eps=1. (And you probably need to buy a new computer...)

Without getting any deeper into the computer science of numerical processing, I wanted to give you some sense of the way that theoretical mathematical concepts are translated into code that gives acceptable results on digital computers.

16.7 SVD spectral theory

There are several "spectral theories" in mathematics, and they all involve the concept that a complicated thingâ€”such as a matrix, an operation, or a transformationâ€”can be represented by the sum of simpler things. Itâ€™s like how visible light can be decomposed into the colors of the rainbow.

There is a spectral theory of matrices, which is sometimes used as another term for eigendecomposition. Iâ€™m going to adapt this concept and bend the terminology slightly to the SVD spectral theory, which is that all matrices can be expressed as a sum of rank-1 matrices, and that the SVD provides a great decomposition to obtain these individual rank-1 matrices.



FigureÂ 16.4: Outer product, visualized.

Let me begin by reminding you of the "layer perspective" of matrix multiplication (page Â§), which involves constructing a product matrix as the sum of outer-product matrices created from the columns of a matrix on the left, and the rows of a matrix to the right. Each of those outer-product matrices has a rank of 1, because each column (or row) is a scalar multiple of one column (or one row).

With matrix multiplication via layers, the two vectors that multiply to create each "layer" of the product matrix are defined purely by their physical position in the matrix. Is that really the best way to define basis vectors to create each layer? Probably not: The position of a row or column in a matrix may not be the organizing principle of "importance"; indeed, in many matricesâ€”particularly matrices that contain dataâ€”rows or columns can be swapped or even randomized without any change in the information content of the data matrix.

The SVD provides an interesting way to construct a matrix by summing rank-1 layers that are computed using the columns of U and the rows of V^T , scaled by their corresponding singular value. The mechanics are given by re-writing the SVD formula using a summation of vectors instead of matrices.

$$\mathbf{A} = \sum_{i=1}^{r} \mathbf{u}_i \sigma_i \mathbf{v}_i^{\mathrm{T}} \tag{16.16}$$

where r is the rank of the matrix (the singular values after Ïf$_r$ are zeros, and thus can be omitted from this equation). Your delicate linear-algebraic sensibilities might be offended by going from the elegant matrix Equation 16.1 to the clumsy vector-sum Equation 16.16 . But this equation will set us up for the SVD spectral theory, and will also lead into one of the important applications of SVD, which is low-rank approximations (next section).

Letâ€™s consider only the first iteration of the summation:

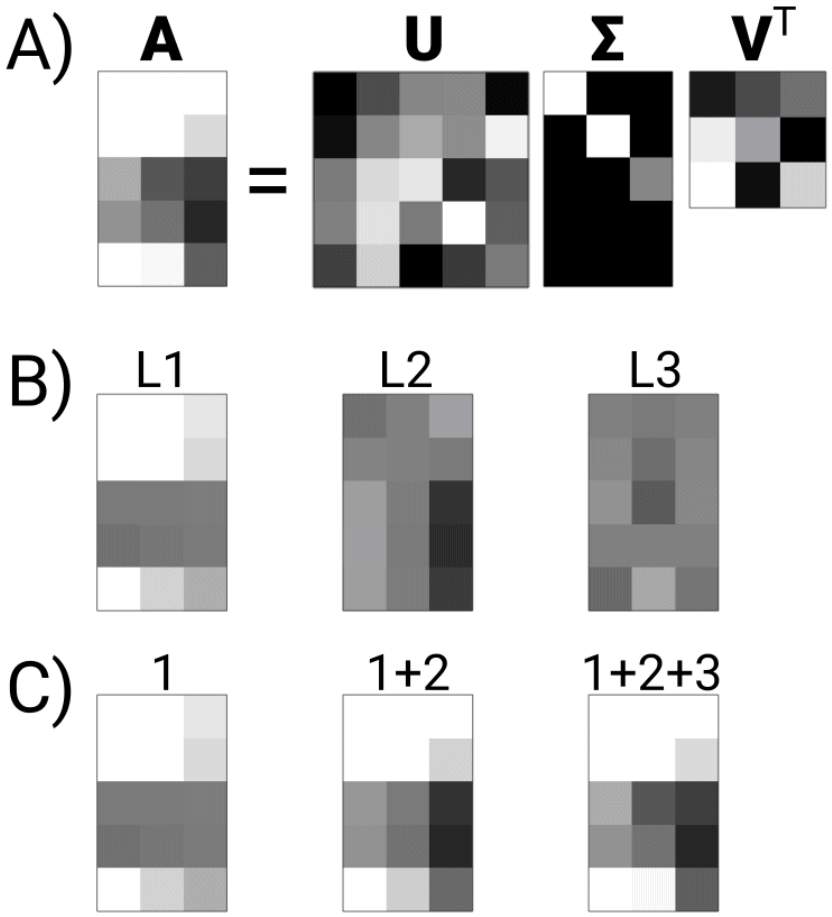
$$\mathbf{A}_1 = \mathbf{u}_1 \sigma_1 \mathbf{v}_1^{\mathrm{T}} \tag{16.17}$$

We might call matrix A_1 the "first SVD-layer of A." It is a rank-1 matrix (same size as A) formed as the outer product between the first left singular vector and the first right singular vector. Because the two vectors are unit length (â^¥u_1 â^¥ = â^¥v_1 â^¥ = 1), the 2-norm of their outer product is also 1. But is the norm of A_1 also equal to 1? No, it isnâ€™t, and thatâ€™s because the outer product gets scalar-multiplied by the corresponding singular value (notwithstanding when Ïf$_1$ = 1).

Because the singular values are sorted descending, A_1 is actually the "most meaningful" SVD-layer of matrix A ("meaningful" can be interpreted as the amount of total variance in the matrix, or as the most important feature of the matrix; more on this in the next section). A_2 will be the next-most-meaningful SVD-layer, and so on down to A_r . The SVD-layers after r have zero-valued singular values and thus contribute nothing to the final matrix.

Thus, each corresponding left and right singular vector combine to produce a layer of the matrix. This layer is like a direction in the matrix space. But that direction is simply a pointerâ€”it does not convey "importance." Instead, it is the singular value that indicates how "important" each direction is. Itâ€™s like a sign-post that points to the nearest gas station (1 km away) and to Siberia (10,000 km away). The signs (the singular vectors) have the same size; you have to look at the numbers on the signs (the singular values) to know how far each destination is.

Figure 16.5 illustrates the concept of reconstructing a matrix by successively adding SVD-layers. The three singular values of this random matrix are 3.4, 1.0, and 0.5 (rounded to the nearest tenth). In the next section, you will learn how to interpret these values and how to normalize them into a more meaningful metric.



FigureÂ 16.5: Illustration of the SVD spectral theory. Panel A shows the SVD matrices. It is visually apparent that A is a rank-3 matrix (3 non-zero singular values). Panel B shows the three SVD-layers formed from outer products of corresponding columns of U, rows of V^T, and singular values ("L1" = layer 1). Panel C shows the cumulative sum of layers. The first two maps are low-rank approximations of A while the final map ("1+2+3") is an exact reconstruction of A.

Notice that layer 1 ("L1" in panel B) captures the most prominent feature of the matrix A (the horizontal band in the middle); in the next section I will refer to this as the best rank-1 approximation of A.

Although it might not be obvious from the color scaling, each column of the L1 matrix is simply the left-most column of U scalar-multiplied by the corresponding elements of the first row of V^T, and also multiplied by $Î£_{1,1}$. Same story for matrices L2 and L3. Columns 4 and 5 of U do not contribute to reconstructing Aâ€”thatâ€™s built-in to Equation 16.16 because the rank of this matrix is r = 3â€”and itâ€™s also apparent from visualization of the multiplication: Columns 4 and 5 in U multiply rows 4 and 5 of Î£, which are all zeros. In terms of matrix spaces, columns 4 and 5 of U are in the left-null space of A.

SVD-layer 2 captures the second-most prominent feature of the matrix A, which is the vertical stripe in the lower-right. Layer 3 has a relatively small singular value (10% of the total variance in the matrix; Iâ€™ll describe later how I compute this) and therefore accounts for relatively little information in the matrix. You can see in panel C that adding SVD-layer 3â€”though technically necessary for a full reconstruction of Aâ€”changes the matrix only slightly.

16.8 SVD and low-rank approximations

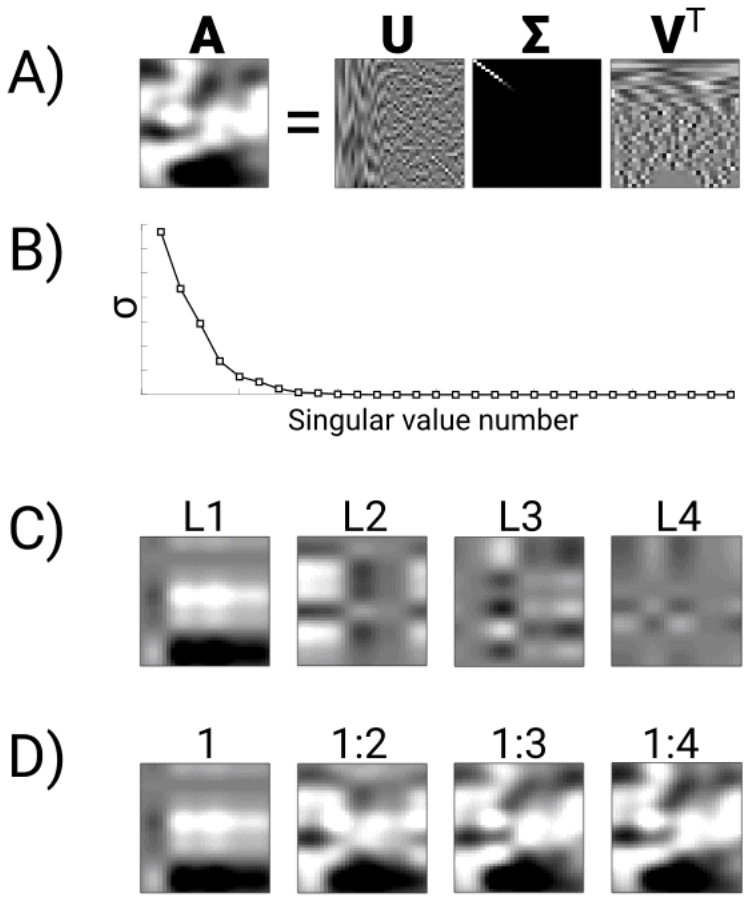
Equation 16.16 in the previous section showed how to reconstruct a matrix exactly by summing up outer products from left- and right-singular values, scalar-multiplied by the corresponding singular vaule. I also wrote that SVD-layers with smaller singular values are less important for the matrix.

This leads to the idea of low-rank approximations: Create a matrix A that is sort-of like matrix A; itâ€™s a rank-k version of the original rank-r matrix created by adapting the full vector-sum formula slightly:

Eqn. box title=Low-rank approximation

$$A = \hat{a}^{\prime}_{i=1}{}^{k}\, u_i \ddot{I} f_i\, v_i{}^{T}, \; k < r \qquad (16.18)$$

Figure 16.6 shows an example. The matrix A is a 30Ã—40 random numbers matrix that was smoothed with a 2D Gaussian kernel. Panel A shows its SVD decomposition, similar to previous figures. The important addition here is that matrix A has rank = 30, whereas the right-most matrix in panel D has a rank = 4. And yet, visually, that matrix appears extremely similar to matrix A. In other words, the rank-4 approximation of the original rank-30 matrix appears to capture all the important features.



FigureÂ 16.6: Example of using SVD for a low-rank approximation of a matrix. Panels C and D are comparable to panels B and C in Figure 16.5 . 1:4 indicates the cumulative sum of SVD-layers 1 through 4.

Why did I pick k = 4 SVD-layers to reconstruct? That is, admittedly, an arbitrary choice that was partly driven by the visual layout of the figure. There are two ways to select an appropriate value of k. One is based on visual

inspection of the "scree plot" (the plot of decreasing singular values; Figure 16.6 B); you look for the set of singular values that seem to "pop out" or are relatively large compared to the distribution of singular values. From Figure 16.6 B there appears to be anywhere between 4 and 6 relatively large values. You can already see in this example that this can be a non-trivial decision, and there might be disagreement about the "correct" number of SVD-layers to include. A related technique is called the "elbow method," and involves imagining that the scree plot is your arm bent at the elbow (the first singular value on the left is your shoulder; the last one is your hand). Look at the plot and decide which node is at the elbow; thatâ€™s the number of components the preserve. Again, itâ€™s a bit ambiguous in this example (as it often is), but Iâ€™d say the elbow is node 4 or 5.

A second method for choosing k is less arbitrary and is based on selecting a threshold, for example, all SVD-layers that contribute at least 1% of the variance of the entire matrix. This method requires converting the singular values into a more meaningful metric, which is the topic of the next section.

Why is a low-rank approximation useful? Why not simply keep A in all its glorious original-rank perfection? There are several applications of low-rank approximations; below are three examples.

1. Noise reduction: It is possible that the data features associated with large singular values represent signal, whereas data features associated with small singular values represent noise. Thus, by reconstructing the matrix using k < r layers, the matrix becomes cleaner. This technique is sometimes referred to as "projecting out noise."
2. Machine-learning classification: Many machine-learning analyses involve identifying features or patterns in data that predict some outcome variable. It may be the case that the most relevant data features are represented in the k largest singular vectors, and thus the analyses can be faster and more accurately done by training on the "most important" singular vectors instead of on the original data matrix.
3. Data compression: Letâ€™s imagine a large dataset contained in a 10,000Ã— 100,000 matrix. At floating-point precision, this matrix can take up 8 GB of hard-disk space (assuming no compression). Now letâ€™s imagine that this is a rank-100 matrix. Storing the first 100 columns of U and V, and the first 100 singular values (as a vector, not as a diagonal matrix) would take up only 0.09 GB (around 90 MB). It would then be possible to load in these vectors and scalars and recreate the full-sized A as needed.

16.9 Normalizing singular values

Imagine you have two different matrices, and find that the largest singular value of matrix A is 8 and the largest singular value of matrix B is 678. How would you interpret that difference? Are those two numbers comparable? And what does the number "8" even mean??

The answer is that those two $\ddot{I}f_{max}$ values are probably not comparable, unless you know that the matrices have numbers in exactly the same range.

Before showing you how to normalize the singular values, I want to show that singular values are scale-dependent, meaning they change with the scale of the numbers in the matrix. Below is an example; the second matrix is the same as the first but multiplied by 10. Notice that their singular values are the same except for a scaling of 10 (numbers are rounded to the nearest thousandth).

$$^{6}A = \begin{bmatrix} 8 & 4 & 10 \\ 4 & 5 & 6 \end{bmatrix}, \quad \hat{I}\pounds_A = \begin{bmatrix} 15.867 & 0 \\ 0 & 2.286 \end{bmatrix}$$

$$B = 10A = \begin{bmatrix} 80 & 40 & 100 \\ 40 & 50 & 60 \end{bmatrix}, \quad \hat{I}\pounds_B = \begin{bmatrix} 158.674 & 0 \\ 0 & 22.863 \end{bmatrix} = 10\hat{I}\pounds_A$$

So when a matrix is multiplied by a scalar, its singular values are multiplied by that same scalar. To understand why this must be the case, remember that the singular vectors are unit-normalized to create orthogonal matrices. So it is not possible for U or V to "absorb" the scalar. Instead, the only part of the SVD that can be scalar-multiplied is Î£, the singular values.

[6] The third column of Î£ was trimmed.

The point here is that it is difficult or impossible to compare "raw" singular values across matrices, because those values depend both on the numerical ranges in the matrices and on the data features embedded in the matrices.

What is the solution to this limitation? As Iâ€™m sure youâ€™ve guessed, the solution is to normalize the singular values. The normalization I will discuss here is converting to percent of total variance of the matrix.

Because the singular vectors are all unit-length and are scaled by the singular values when reconstructing the matrix from its SVD layers, the sum over all singular values can be interpreted as the total variance or "energy" in the matrix. [7] This sum over all singular values is formally called the Schatten 1-norm of the matrix. For completeness, the equation below is the full formula for the Schatten p-norm, although here we are considering only the case of p = 1.

[7] See also section 6.10 about matrix norms.

$$\|\mathbf{A}\|_p = \left(\sum_{i=1}^{r} \sigma_i^p \right)^{1/p} \tag{16.19}$$

The next step is to scale each singular value to the percent of the Schatten 1-norm:

$$\widetilde{\sigma_i} = \frac{100\sigma_i}{\|\mathbf{A}\|_1} \tag{16.20}$$

This is a useful normalization, because it allows for direct interpretation of each singular value, as well as direct comparison of singular values across different matrices. In the example matrices on page ?? , the two normalized Î£ matrices would be equal to each other.

Getting back to the problem at the outset of this section, two matrices with largest singular values of 8 and 678 are not comparable, but letâ€™s say their normalized largest singular values are 35% and 82%. In this case, 35% means that the largest SVD-layer accounts for only 35% of the total variance in the matrix. One interpretation is that this is a complicated matrix that contains a lot of information along several different directions. In contrast, a largest normalized singular value of 82% means nearly all of the variance is explained by one component, so this matrix probably contains less complex information. If this were a data matrix, it might correspond to one pattern and 18% noise.

Now letâ€™s think back to the question of how many SVD-layers to use in a low-rank approximation (that is, how to select k). When the singular values are normalized, you can pick some percent variance threshold and retain all SVD-layers that contribute at least that much variance. For example, you might keep all SVD-layers with Ï*f* > 1%, or perhaps 0.1% to retain more information. The choice of a threshold is still somewhat arbitrary, but this is at least more quantitative and reproducible than visual inspection of the scree plot.

16.10 Condition number of a matrix

The condition number of a matrix is used to evaluate the "spread" of a matrix. It is defined as the ratio of the largest to the smallest singular values, and is often indicated using the Greek letter Îº.

Eqn. box title=Condition number of a matrix

$$\text{Îº} = \frac{\sigma_{max}}{\sigma_{min}} \tag{16.21}$$

For example, the condition number of the identity matrix is 1, because all of its singular values are 1. The condition number of any singular matrix is undefined ("not a number"; NaN), because singular matrices have at least one zero-valued singular value, thus making the condition number $\frac{\sigma}{0}$ =???

The condition number of all orthogonal matrices is the same. Can you guess what that condition number is, and why? To build suspense, Iâ€™ll explain the answer at the end of this section.

A matrix is called well-conditioned if it has a low condition number and high-conditioned if it has a high condition number. However, there is no absolute threshold for when a matrix can be labeled high-conditioned. In some cases,

Îº > 10,000 is used as a threshold, but this can be application-specific. Furthermore, singular matrices can contain a lot of information and be useful in applications, but have a condition number of NaN. Likewise, near-singular matrices can have an astronomically high condition number while still being a meaningful matrix.

In data analysis and statistics, the condition number is used to indicate the stability of a matrix, in other words, the sensitivity of the matrix to small perturbations. A high condition number means that the matrix is very sensitive, which could lead to unreliable results in some analyses, for example those that require the matrix inverse.

But donâ€™t take the condition number of a matrix too seriously: Matrices can contain a lot of information or very little information regardless of their condition number. The condition number should not be used on its own to evaluate the usefulness of a matrix.

OK, the answer to the question about orthogonal matrices is that the condition number of any orthogonal matrix is 1, and thatâ€™s because all of the singular values of an orthogonal matrix are 1. This is the case because an orthogonal matrix is defined as $Q^T Q = I$, and the eigenvalues of a diagonal matrix are its diagonal elements. And you know from earlier in this chapter that the singular values of Q are the principal square roots of the singular values of $Q^T Q$.

Code You can compute the condition number on your own based on the SVD, but Python and MATLAB have built-in functions.

Code blockÂ 16.3: Python

```
importÂ numpyÂ asÂ npÂ
AÂ =Â np.random.randn(5,5)Â
sÂ =Â np.linalg.svd(A)[1]Â
condnumÂ =Â np.max(s)/np.min(s)Â
#Â compareÂ aboveÂ withÂ cond()Â
print(condnum,np.linalg.cond(A))
```

Code blockÂ 16.4: MATLAB

```
AÂ =Â randn(5,5);Â
sÂ =Â svd(A);Â
condnumÂ =Â max(s)/min(s);Â
disp([condnum,cond(A)])Â %Â comparison
```

16.11 SVD and the matrix inverse

Letâ€™s consider the inverse of a matrix and its SVD. Assume A is square full-rank.

$$A^{â^'1} = (UÎ£V^T)^{â^'1} \quad (16.22)$$

$$= VÎ£^{â^'1} U^{â^'1} \quad (16.23)$$

$$= VÎ£^{â^'1} U^T \quad (16.24)$$

Because U and V are orthogonal matrices, their inverses are trivial to compute (their transposes). Furthermore, because Î£ is a diagonal matrix, its inverse is also trivial to compute (simply invert each diagonal element; see section 12.2 on page Â§) .

Actually, $Î£^{â^'1}$ might not be trivial to compute in practice, because if some singular values are close to machine precision, then tiny rounding errors or other numerical inaccuracies can make the inverse unstable (for example, $\frac{1}{10^{-15}}$ is a very large number). This shows why the explicit inverse of an ill-conditioned matrix can be numerically unstable.

Equation 16.24 also allows us to prove that the inverse of a symmetric matrix is itself symmetric. Letâ€™s write out the SVD for a symmetric matrix and its inverse (remember that symmetric matrices have identical left and right singular vectors).

$$A^T = (VÎ£V^T)^T = VÎ£V^T \quad (16.25)$$

$$A^{â^'1} = (VÎ£V^T)^{â^'1} = VÎ£^{â^'1} V^T \quad (16.26)$$

It is immediately clear that A, A^T , and $A^{â^'1}$ have the same singular vectors; the singular values may differ, but the point is that Î£ is also symmetric, and thus $A^{â^'1}$ is symmetric as long as A is symmetric.

Expressing the inverse via the SVD may seem like an academic exercise, but this is a crucial introduction to the pseudoinverse, as you will now learn.

16.12 The MP Pseudoinverse, part 2

In section 12.8 , you read about the idea of the pseudoinverse: It is an approximation to an inverse for a singular matrix. I promised that you would find the algorithm for the Moore-Penrose pseudoinverse easily comprehensible by the time you got to this chapter. And here we are.

Hereâ€™s the algorithm: Compute the SVD of a matrix, swap and transpose the U and V matrices (just like with the full inverse), invert its non-zero singular values, then multiply to reconstitute the matrix pseudoinverse.

Eqn. box title=Pseudoinverse via SVD

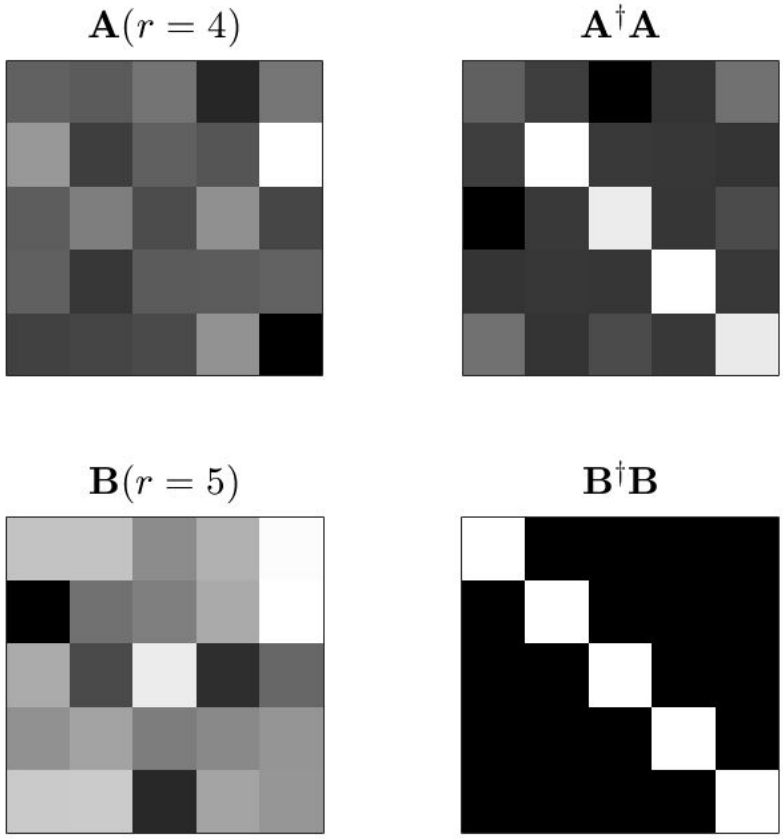
$$\begin{aligned} A^{â€} &= (UÎ£V^T)^{â€} \\ &= VÎ£^{â€} U^T \end{aligned} \tag{16.27}$$

$$Î£_{i,i}{}^{â€} = \begin{cases} \frac{1}{\sigma_i} & \text{if } \sigma_i \neq 0 \\ 0 & \text{if } \sigma_j = 0 \end{cases}$$

Notice that this procedure will work for any matrix, square or rectangular, full-rank or singular. When the matrix is square and full-rank, then the Moore-Penrose pseudoinverse will equal the true inverse, which you can see yourself by considering what happens when all of the singular values are non-zero.

Computer programs that implement the MP pseudoinverse will threshold very small-but-nonzero singular values to avoid numerical instability issues. As described earlier for computing rank, that tolerance is some multiple of the machine precision, and the idea is to treat as zero any singular values that are too small to be confidently distinguished from zero.

Figure 16.7 illustrates two examples of the MP pseudoinverse for a singular matrix (top row) and a full-rank matrix (bottom row). Notice that the pseudoinverse times the original matrix is not-quite-but-trying-to-be the identity matrix.



FigureÂ 16.7: Visualization of two matrices (left column) and those matrices times their pseudoinverses (right column). A is reduced-rank whereas B is full-rank.

One-sided inverse When the matrix is rectangular and either full column rank or full row rank, then the pseudoinverse will equal the left inverse or the right inverse. This makes computing the one-sided inverse computationally efficient, because it can be done without explicitly computing $(A^T A)^{-1}$.

To understand the relationship between the one-sided inverse and the pseudoinverse, letâ€™s work through the math of the left inverse. Recall that the left inverse of a tall matrix T is $(T^T T)^{âˆ’1} T^T$. Letâ€™s write out its SVD, simplify, and see what happens.

$$T = U\hat{I}£V^T \tag{16.28}$$

$$(T^T T)^{âˆ’1} T^T = \left((\mathbf{U}\Sigma\mathbf{V}^T)^T\mathbf{U}\Sigma\mathbf{V}^T\right)^{âˆ’1}\left(\mathbf{U}\Sigma\mathbf{V}^T\right)^T \tag{16.29}$$

$$= \left(\mathbf{V}\Sigma^T\mathbf{U}^T\mathbf{U}\Sigma\mathbf{V}^T\right)^{âˆ’1} V\hat{I}£U^T \tag{16.30}$$

$$= \left(\mathbf{V}\Sigma^2\mathbf{V}^T\right)^{âˆ’1} V\hat{I}£U^T \tag{16.31}$$

$$= V\hat{I}£^{âˆ’2} V^T V\hat{I}£U^T \tag{16.32}$$

$$= V\hat{I}£^{âˆ’1} U^T \tag{16.33}$$

I know itâ€™s a lot to work through, but the take-home message is that when you write out the SVD of a left inverse and simplify, you end up with exactly the same expression as the SVD-based inverse of the original matrix (replace $^{âˆ’1}$ with $^{â€}$ where appropriate).

Needless to say, the conclusion is the same for the right inverse, which you can work through on your own.

16.13 Code challenges

I decided against by-hand exercises here, because the SVD is only a few additional steps on top of eigendecomposition. Instead, Iâ€™ve added more code challenges to help you internalize the conceptual ideas of the SVD and implementing them on computers.

1.Â
In Chapter 13, you learned about "economy" QR decomposition, which can be useful for large tall matrices. There is a comparable "economy" version of the SVD. Your goal here is to figure out what that means. First, generate three random matrices: square, wide, and tall. Then run the full SVD to confirm that the sizes of the SVD matrices match your expectations (e.g., Figure 16.1). Finally, run the economy SVD on all three matrices and compare the sizes to the full SVD.

2.Â
Obtain the three SVD matrices from eigendecomposition, as described in section 16.2 . Then compute the SVD of that matrix using the svd() function, to confirm that your results are correct. Keep in mind the discussions of sign-indeterminacy.

3.Â
Write code to reproduce panels B and C in Figure 16.5 . Confirm that the reconstructed matrix (third matrix in panel C) is equal to the original matrix. (Note: The matrix was populated with random numbers, so donâ€™t expect your results to look exactly like those in the figure.)

4.Â
Create a random-numbers matrix with a specified condition number. For example, create a 6Ã—16 random matrix with a condition number of Îº = 42. Do this by creating random U and V matrices, an appropriate Î£ matrix, and then create $A = U\Sigma V^T$. Finally, compute the condition number of A to confirm that it matches what you specified (42).

5.Â
This and the next two challenges involve taking the SVD of a picture. A picture is represented as a matrix, with the matrix values corresponding to grayscale intensities of the pixels. We will use a picture of Einstein. You can download the file at https://upload.wikimedia.org/wikipedia/en/8/86/Einstein_tongue.jpgOf course, you can replace this with any other pictureâ€”a selfie of you, your dog, your kids, your grandmother on her wedding day... However, you may need to apply some image processing to reduce the image matrix from 3D to 2D (thus, grayscale instead of RGB) and the datatype must be double (MATLAB) or floats (Python).

After importing the image, construct a low-rank approximation using various numbers of singular values. Show the original and low-rank approximations side-by-side. Test various numbers of components and qualitatively evaluate the results. Tip: You donâ€™t need to include the top components!

6.Â
Create a scree plot of the percent-normalized singular values. Then test various thresholds for reconstructing the picture (e.g., including all components that explain at least 4% of the variance). What threshold seems reasonable?

7.Â
The final challenge for this picture-SVD is to make the assessments of the number of appropriate components more quantitative. Compute the error between the reconstruction and the original image. The error can be operationalized as the RMS (root mean square) of the difference map. That is, create a difference image as the subtraction of the original and low-rank reconstructed image, then square all matrix elements (which are pixels), average over all pixels, and take the square root of that average. Make a plot of the RMS as a function of the number of components you included. How does that function compare to the scree plot?

8.Â
What is the pseudoinverse of a column vector of constants? That is, the pseudoinverse of k1. It obviously doesnâ€™t have a full inverse, but it is clearly a full column-rank matrix. First, work out your answer on paper, then confirm it in MATLAB or Python.

9.Â
The goal here is to implement the series of equations on page Â§ and confirm that you get the same result as with the pinv() function. Start by creating a 4Ã—2 matrix of random integers between 1 and 6. Then compute its SVD (Equation 16.28). Then implement each of the next four equations in code. Finally, compute the MP pseudoinverse of the tall matrix. You will now have five versions of the pseudoinverse; make sure they are all equal.

10.Â

This challenge follows up on the first code challenge from the previous chapter (about generalized eigendecomposition implemented as two matrices vs. the product of one matrix and the otherâ€™s inverse). The goal is to repeat the exploration of differences between eig(A,B) and eig(inv(B)*A). Use only 10Ã—10 matrices, but now vary the condition number of the random matrices between 10^1 and 10^{10} . Do you come to different conclusions from the previous chapter?

11.Â

This isnâ€™t a specific code challenge, but instead a general suggestion: Take any claim or proof I made in this chapter (or any other chapter), and demonstrate that concept using numerical examples in code. Doing so (1) helps build intuition, (2) improves your skills at translating math into code, and (3) gives you opportunities to continue exploring other linear algebra principles (I canâ€™t cover everything in one book!).

16.14 Code solutions

1.Â

"Economy" SVD involves computing only the first N columns of U if A is tall, or only the first M rows of V^T if A is wide. For square A, "economy" SVD is the same as the full SVD. Below is code for a tall matrix; modifying for wide and square matrices is easy.

Code blockÂ 16.5: Python

```
importÂ numpyÂ asÂ npÂ
mÂ =Â 6Â
nÂ =Â 3Â
AÂ =Â np.random.randn(m,n)Â
Uf,sf,VfÂ =Â np.linalg.svd(A)Â
Ue,se,VeÂ =Â np.linalg.svd(A,Â
Â Â Â Â Â Â Â Â Â Â Â Â Â Â full_matrices=False)Â
print(Uf.shape,Â sf.shape,Â Vf.shape)Â #Â fullÂ
print(Ue.shape,Â se.shape,Â Ve.shape)Â #Â econ
```

Code blockÂ 16.6: MATLAB

```
mÂ =Â 6;Â
nÂ =Â 3;Â
AÂ =Â randn(m,n);Â
[Uf,Sf,Vf]Â =Â svd(A);Â %Â fÂ forÂ fullÂ
[Ue,Se,Ve]Â =Â svd(A,â€™econâ€™);Â %Â eÂ forÂ econÂ
whosÂ AÂ U*Â S*Â V*
```

2.Â

Hint: You need to sort the columns of U and V based on descending eigenvalues. You can check your results by subtracting the eigenvectors and the matching singular vectors matrices. Due to sign-indeterminacy, you will likely find a few columns of zeros and a few columns of non-zeros; comparing against âˆ’U will flip which columns are zeros and which are non-zeros. Donâ€™t forget that Python returns V^T .

Code blockÂ 16.7: Python

```
AÂ =Â np.random.randn(4,5)Â #Â matrixÂ
L2,VÂ =Â np.linalg.eig(A.T@A)Â #Â getÂ VÂ
VÂ =Â V[:,np.argsort(L2)[::-1]]#sortÂ descendÂ
L2,UÂ =Â np.linalg.eig(A@A.T)Â #Â getÂ UÂ
UÂ =Â U[:,np.argsort(L2)[::-1]]#sortÂ descendÂ
#Â createÂ SigmaÂ
SÂ =Â np.zeros(A.shape)Â
forÂ i,sÂ inÂ enumerate(np.sort(L2)[::-1]):Â
Â Â Â Â S[i,i]Â =Â np.sqrt(s)Â
U2,S2,V2Â =Â np.linalg.svd(A)Â #Â svd
```

Code blockÂ 16.8: MATLAB

```
AÂ =Â randn(4,5);Â %Â matrixÂ
[V,L2]Â =Â eig(Aâ€™*A);Â %Â getÂ VÂ
[L2,idx]Â =Â sort(diag(L2),â€™dâ€™);Â
VÂ =Â V(:,idx);Â %Â sortÂ byÂ descendingÂ LÂ
[U,L2]Â =Â eig(A*Aâ€™);Â %Â getÂ UÂ
[L2,idx]Â =Â sort(diag(L2),â€™dâ€™);Â
UÂ =Â U(:,idx);Â %Â sortÂ byÂ descendingÂ LÂ
%Â createÂ SigmaÂ
SÂ =Â zeros(size(A));Â
forÂ i=1:length(L2)Â
Â Â Â Â S(i,i)Â =Â sqrt(L2(i));Â
endÂ
[U2,S2,V2]Â =Â svd(A);Â %Â svd
```

3.Â

I changed the code slightly from the Figure to include the original matrix to the right of the reconstructed matrix. Anyway, the important part is creating the low-rank approximations in a for-loop. Be very careful with the slicing in Python!

Code blockÂ 16.9: Python

```
importÂ matplotlib.pyplotÂ asÂ pltÂ
fig,axÂ =Â plt.subplots(2,4)Â
Â
AÂ =Â np.random.randn(5,3)Â
U,s,VÂ =Â np.linalg.svd(A)Â
SÂ =Â np.diag(s)Â #Â needÂ SigmaÂ matrixÂ
Â
forÂ iÂ inÂ range(3):Â
Â
Â Â onelayerÂ =Â np.outer(U[:,i],V[i,:])*s[i]Â
Â Â ax[0,i].imshow(onelayer)Â
Â Â ax[0,i].set_title(â€™LayerÂ %gâ€™%i)Â
Â Â ax[0,i].axis(â€™offâ€™)Â
Â
Â Â lowrank=U[:,:i+1]@S[:i+1,:i+1]@V[:i+1,:]Â
Â Â ax[1,i].imshow(lowrank)Â
Â Â ax[1,i].set_title(â€™LayersÂ 0:%gâ€™%i)Â
Â Â ax[1,i].axis(â€™offâ€™)Â
Â
Â
ax[1,3].imshow(A)Â
ax[1,3].set_title(â€™Orig.Â Aâ€™)Â
ax[1,3].axis(â€™offâ€™)Â
ax[0,3].axis(â€™offâ€™);
```

Code blockÂ 16.10: MATLAB

```
AÂ =Â randn(5,3);Â
[U,S,V]Â =Â svd(A);Â
Â
forÂ i=1:3Â
Â Â Â Â subplot(2,4,i)Â
Â Â Â Â onelayerÂ =Â U(:,i)*S(i,i)*V(:,i)â€™;Â
Â Â Â Â imagesc(onelayer)Â
Â Â Â Â title(sprintf(â€™LayerÂ %gâ€™,i))Â
Â
Â Â Â Â subplot(2,4,i+4)Â
```

```
Â Â Â Â lowrankÂ =Â U(:,1:i)*S(1:i,1:i)*V(:,1:i)â€™;Â
Â Â Â Â imagesc(lowrank)Â
Â Â Â Â title(sprintf(â€™LayersÂ 1:%gâ€™,i))Â
endÂ
subplot(248)Â
imagesc(A),title(â€™OriginalÂ Aâ€™)
```

4.Â

There are two insights to this challenge. First, U and V must be orthogonal, which you can obtain via QR decomposition on random matrices. Second, you only need to specify the target singular value; the smallest can be 1 and rest can be anything in between (for simplicity, Iâ€™ve made them linearly spaced from smallest to largest).

Code blockÂ 16.11: Python

```
m=6;Â n=16Â
condnumÂ =Â 42Â
#Â createÂ UÂ andÂ VÂ fromÂ randomÂ numbersÂ
U,rÂ =Â np.linalg.qr(Â np.random.randn(m,m)Â )Â
V,rÂ =Â np.linalg.qr(Â np.random.randn(n,n)Â )Â
#Â createÂ singularÂ valuesÂ vectorÂ
sÂ =Â np.linspace(condnum,1,np.min((m,n)))Â
SÂ =Â np.zeros((m,n))Â
forÂ iÂ inÂ range(min((m,n))):Â
Â Â Â Â S[i,i]Â =Â s[i]Â
AÂ =Â U@S@V.TÂ #Â constructÂ matrixÂ
np.linalg.cond(A)Â #Â confirm!
```

Code blockÂ 16.12: MATLAB

```
m=6;Â n=16;Â
condnumÂ =Â 42;Â
%Â createÂ UÂ andÂ VÂ fromÂ randomÂ numbersÂ
[U,r]Â =Â qr(Â randn(m)Â );Â
[V,r]Â =Â qr(Â randn(n)Â );Â
%Â createÂ singularÂ valuesÂ vectorÂ
sÂ =Â linspace(condnum,1,min(m,n));Â
SÂ =Â zeros(m,n);Â
forÂ i=1:min(m,n)Â
Â Â Â Â S(i,i)Â =Â s(i);Â
endÂ
AÂ =Â U*S*Vâ€™;Â %Â constructÂ matrixÂ
cond(A)Â %Â confirm!
```

5.Â

You might have struggled a bit with transforming the image, but hopefully the SVD-related code wasnâ€™t too difficult. My code below reconstructs the image using components 1-20, but you can also try, e.g., 21-40, etc.

Code blockÂ 16.13: Python

```
importÂ numpyÂ asÂ npÂ
importÂ matplotlib.pyplotÂ asÂ pltÂ
fromÂ imageioÂ importÂ imreadÂ
picÂ =Â imread(â€™https://upload.wikimedia.org/Â
Â Â Â Â Â wikipedia/en/8/86/Einstein_tongue.jpgâ€™)Â
np.array(pic,dtype=float)Â #Â convertÂ toÂ floatÂ
Â
U,s,VÂ =Â np.linalg.svd(Â picÂ )Â
SÂ =Â np.zeros(pic.shape)Â
```

```
forÂ iÂ inÂ range(len(s)):Â
Â Â S[i,i]Â =Â s[i]Â
Â
compsÂ =Â slice(0,21)Â #Â low-rankÂ approx.Â
lowrank=U[:,comps]@S[comps,comps]@V[comps,:]Â
Â
#Â showÂ theÂ originalÂ andÂ low-rankÂ
plt.subplot(1,2,1)Â
plt.imshow(pic,cmap=â€™grayâ€™)Â
plt.title(â€™Originalâ€™)Â
plt.subplot(1,2,2)Â
plt.imshow(lowrank,cmap=â€™grayâ€™)Â
plt.title(â€™Comps.Â %g-%gâ€™Â
Â Â Â Â Â Â Â Â Â Â Â %(comps.start,comps.stop-1));
```

Code blockÂ 16.14: MATLAB

```
picÂ =Â imread(â€™https://upload.wikimedia.org/Â
Â Â Â wikipedia/en/8/86/Einstein_tongue.jpgâ€™);Â
picÂ =Â double(pic);Â %Â convertÂ toÂ doubleÂ
Â
[U,S,V]Â =Â svd(Â picÂ );Â
compsÂ =Â 1:20;Â %Â low-rankÂ approximationÂ
lowrankÂ =Â U(:,comps)Â *Â ...Â
Â Â Â Â Â Â Â Â Â S(comps,comps)*V(:,comps)â€™;Â
Â
%Â showÂ theÂ originalÂ andÂ low-rankÂ
subplot(121)Â
imagesc(pic),Â axisÂ imageÂ
title(â€™Originalâ€™)Â
subplot(122)Â
imagesc(lowrank),Â axisÂ imageÂ
title(sprintf(â€™Comps.Â %g-%gâ€™,...Â
Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â comps(1),comps(end)))Â
colormapÂ gray
```

6.Â

The low-rank calculations and plotting are basically the same as the previous exercise. The main additions here are computing percent variance explained and thresholding. Itâ€™s a good idea to check that all of the normalized singular values sum to 100.

Code blockÂ 16.15: Python

```
#Â convertÂ toÂ percentÂ explainedÂ
sÂ =Â 100*s/np.sum(s)Â
plt.plot(s,â€™s-â€™);Â plt.xlim([0,100])Â
plt.xlabel(â€™ComponentÂ numberâ€™)Â
plt.ylabel(â€™PctÂ varianceÂ explainsâ€™)Â
plt.show()Â
Â
threshÂ =Â 4Â #Â thresholdÂ inÂ percentÂ
I,J=np.ix_(s>thresh,s>thresh)Â #Â compsÂ >Â X%Â
lowrankÂ =Â np.squeeze(U[:,J]@S[I,J]@V[J,:])Â
Â
#Â showÂ theÂ originalÂ andÂ low-rankÂ
plt.subplot(1,2,1)Â
plt.imshow(pic,cmap=â€™grayâ€™)Â
plt.title(â€™Originalâ€™)Â
```

```
plt.subplot(1,2,2)Â
plt.imshow(lowrank,cmap=â€™grayâ€™)Â
plt.title(â€™%gÂ comps.Â atÂ %g%%â€™Â
Â Â Â Â Â Â Â Â Â Â Â %(len(I),thresh));
```

Code blockÂ 16.16: MATLAB

```
%Â convertÂ toÂ percentÂ explainedÂ
sÂ =Â 100*diag(S)./sum(S(:));Â
plot(s,â€™s-â€™),Â xlim([0Â 100])Â
xlabel(â€™ComponentÂ numberâ€™)Â
ylabel(â€™PctÂ varianceÂ explainsâ€™)Â
Â
threshÂ =Â 4;Â %Â thresholdÂ inÂ percentÂ
compsÂ =Â s>thresh;Â %Â compsÂ greaterÂ thanÂ X%Â
lowrankÂ =Â U(:,comps)Â *Â ...Â
Â Â Â Â Â Â Â Â S(comps,comps)*V(:,comps)â€™;Â
Â
%Â showÂ theÂ originalÂ andÂ low-rankÂ
figure,Â subplot(121)Â
imagesc(pic),Â axisÂ imageÂ
title(â€™Originalâ€™)Â
subplot(122)Â
imagesc(lowrank),Â axisÂ imageÂ
title(sprintf(â€™%gÂ compsÂ withÂ >Â %g%%â€™,...Â
Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â sum(comps),thresh))Â
colormapÂ gray
```

7.Â

The RMS error plot goes down when you include more components. Thatâ€™s sensible. The scale of the data is pixel intensity errors, with pixel values ranging from 0 to 255. However, each number in the plot is the average over the entire picture, and therefore obscures local regions of high- vs. low-errors. You can visualize the error map (variable diffimg).

Code blockÂ 16.17: Python

```
RMSÂ =Â np.zeros(len(s))Â
forÂ siÂ inÂ range(len(s)):Â
Â Â i=si+1Â #Â mindÂ theÂ indexing!Â
Â Â lowrankÂ =Â U[:,:i]@S[:i,:i]@V[:i,:]Â
Â Â diffimgÂ =Â lowrankÂ -Â picÂ
Â Â RMS[si]Â =Â np.sqrt(np.mean(Â
Â Â Â Â Â Â Â Â Â Â Â Â diffimg.flatten()**2))Â
Â
plt.plot(RMS,â€™s-â€™)Â
plt.xlabel(â€™RankÂ approximationâ€™)Â
plt.ylabel(â€™ErrorÂ (a.u.)â€™);
```

Code blockÂ 16.18: MATLAB

```
forÂ si=1:length(s)Â
Â Â lowrank=U(:,1:si)*S(1:si,1:si)*V(:,1:si)â€™;Â
Â Â diffimgÂ =Â lowrankÂ -Â pic;Â
Â Â RMS(si)Â =Â sqrt(mean(diffimg(:).^2));Â
endÂ
plot(RMS,â€™s-â€™)Â
xlabel(â€™RankÂ approximationâ€™)Â
ylabel(â€™ErrorÂ (a.u.)â€™)
```

8.Â

This code challenge illustrates that translating formulas into code is not always straightforward. I hope you enjoyed it!

Code blockÂ 16.19: Python

```
importÂ numpyÂ asÂ npÂ
XÂ =Â np.random.randint(Â
Â Â Â Â Â Â Â Â low=1,high=7,size=(4,2))Â
U,s,VÂ =Â np.linalg.svd(X)Â #Â eq.29Â
SÂ =Â np.zeros(X.shape)Â
forÂ i,ssÂ inÂ enumerate(s):Â
Â Â S[i,i]Â =Â ssÂ
longV1Â =Â np.linalg.inv(Â (U@S@V).T@U@S@VÂ )Â
Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â @Â (U@S@V).TÂ Â #Â eq.30Â
longV2Â =Â np.linalg.inv(Â V.T@S.T@U.T@U@S@VÂ )Â
Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â @Â (U@S@V).TÂ #Â eq.31Â
longV3Â =Â np.linalg.inv(V.T@S.T@S@V)Â
Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â @Â (U@S@V).TÂ #Â eq.32Â
longV4Â =Â V@np.linalg.matrix_power(S.T@S,-1)Â
Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â Â @Â V@V.T@S.T@U.TÂ #Â eq.33Â
MPpinvÂ =Â np.linalg.pinv(X)Â #Â eq.34
```

Code blockÂ 16.20: MATLAB

```
XÂ =Â randi([1Â 6],[4Â 2]);Â
[U,S,V]Â =Â svd(X);Â %Â eq.29Â
longV1Â =Â inv((U*S*Vâ€™)â€™*U*S*Vâ€™)*(U*S*Vâ€™)â€™;%30Â
longV2Â =inv(V*Sâ€™*Uâ€™*U*S*Vâ€™)*(U*S*Vâ€™)â€™;%eq.31Â
longV3Â =Â inv(V*Sâ€™*S*Vâ€™)Â *Â (U*S*Vâ€™)â€™;Â %Â eq.32Â
longV4Â =Â V*(Sâ€™*S)^(-1)*Vâ€™*V*Sâ€™*Uâ€™;Â %Â eq.33Â
MPpinvÂ =Â pinv(X);Â %Â eq.34
```

9.Â

The pseudoinverse of a column of constants is a row vector where each element is 1âˆ•kn where k is the constant and n is the dimensionality. The reason is that the vector times its pseudoinverse is actually just a dot product; summing up k n times yields nk, and thus 1âˆ•nk is the correct inverse to yield 1. (Iâ€™m not sure if this has any practical value, but I hope it helps you think about the pseudoinverse.)

Code blockÂ 16.21: Python

```
importÂ numpyÂ asÂ npÂ
kÂ =Â 5Â
nÂ =Â 13Â
aÂ =Â np.linalg.pinv(np.ones((n,1))*k)Â
aÂ -Â 1/(k*n)Â #Â checkÂ forÂ zeros
```

Code blockÂ 16.22: MATLAB

```
kÂ =Â 5;Â
nÂ =Â 13;Â
aÂ =Â pinv(ones(n,1)*k);Â
aÂ -Â 1/(k*n)Â %Â checkÂ forÂ zeros
```

10.Â

The differences between the two approaches is much more apparent. The issue is that high-conditioned matrices are more unstable and thus so are their inverses. In practical applications, B might be singular, and so an eigendecomposition on $B^{â^'1}$ A is usually not a good idea.

Code blockÂ 16.23: Python

```
importÂ numpyÂ asÂ npÂ
fromÂ scipy.linalgÂ importÂ eigÂ
importÂ matplotlib.pyplotÂ asÂ pltÂ
```

```
Â
MÂ =Â 10Â #Â matrixÂ sizeÂ
cnsÂ =Â np.linspace(10,1e10,30)Â
avediffsÂ =Â np.zeros(len(cns))Â
Â
#Â loopÂ overÂ conditionÂ numbersÂ
forÂ condiÂ inÂ range(len(cns)):Â
Â
Â Â #Â createÂ AÂ
Â Â U,rÂ =Â np.linalg.qr(Â np.random.randn(M,M)Â )Â
Â Â V,rÂ =Â np.linalg.qr(Â np.random.randn(M,M)Â )Â
Â Â SÂ =Â np.diag(np.linspace(cns[condi],1,M))Â
Â Â AÂ =Â U@S@V.TÂ #Â constructÂ matrixÂ
Â
Â Â #Â createÂ BÂ
Â Â U,rÂ =Â np.linalg.qr(Â np.random.randn(M,M)Â )Â
Â Â V,rÂ =Â np.linalg.qr(Â np.random.randn(M,M)Â )Â
Â Â SÂ =Â np.diag(np.linspace(cns[condi],1,M))Â
Â Â BÂ =Â U@S@V.TÂ #Â constructÂ matrixÂ
Â
Â Â #Â GEDsÂ andÂ sortÂ
Â Â l1Â =Â eig(A,B)[0]Â
Â Â l2Â =Â eig(np.linalg.inv(B)@A)[0]Â
Â Â l1.sort()Â
Â Â l2.sort()Â
Â
Â Â avediffs[condi]Â =Â np.mean(np.abs(l1-l2))Â
Â
plt.plot(cns,avediffs);
```

Code blockÂ 16.24: MATLAB

```
MÂ =Â 10;Â %Â matrixÂ sizeÂ
cnsÂ =Â linspace(10,1e10,30);Â
Â
%Â conditionÂ numbersÂ
forÂ condi=1:length(cns)Â
Â
Â Â Â Â %Â createÂ AÂ
Â Â Â Â [U,~]Â =Â qr(Â randn(M)Â );Â
Â Â Â Â [V,~]Â =Â qr(Â randn(M)Â );Â
Â Â Â Â SÂ =Â diag(linspace(cns(condi),1,M));Â
Â Â Â Â AÂ =Â U*S*Vâ€™;Â %Â constructÂ matrixÂ
Â
Â Â Â Â %Â createÂ BÂ
Â Â Â Â [U,~]Â =Â qr(Â randn(M)Â );Â
Â Â Â Â [V,~]Â =Â qr(Â randn(M)Â );Â
Â Â Â Â SÂ =Â diag(linspace(cns(condi),1,M));Â
Â Â Â Â BÂ =Â U*S*Vâ€™;Â %Â constructÂ matrixÂ
Â
Â Â Â Â %Â eigenvaluesÂ andÂ sortÂ
Â Â Â Â l1Â =Â eig(A,B);Â
Â Â Â Â l2Â =Â eig(inv(B)*A);Â
Â Â Â Â l1Â =Â sort(l1);Â
Â Â Â Â l2Â =Â sort(l2);Â
Â
Â Â Â Â %Â storeÂ theÂ differencesÂ
```

```
Â Â Â Â avediffs(condi)Â =Â mean(abs(l1-l2));Â
endÂ
Â
plot(cns,avediffs)Â
xlabel(â€™Cond.Â numberâ€™)Â
ylabel(â€™\Delta\lambdaâ€™)
```

ChapterÂ 17
Quadratic form and definiteness

The quadratic form is one of my favorite topics in linear algebra, because it combines math, geometry, art, data visualization, multivariate signal processing, and statistics, all into one nifty little package.

One important note before we start: The quadratic form applies only to square matrices. Thus, throughout this chapter, you can assume that all matrices are square. Some will be symmetric or non-symmetric, some will be invertible and some singular. There is some debate about whether the quadratic form should be applied to non-symmetric matrices, because many special properties of the quadratic form are valid only when the matrix is symmetric. I will relax this constraint and point out when symmetry is relevant.

17.1 Algebraic perspective

Imagine an MÃ—M matrix and an MÃ—1 column vector. You already know that a matrix times a vector will produce another M-element vector whose orientation depends on whether the vector pre- or post-multiplies the matrix. In particular, Av will produce a column vector while v^T A will produce a row vector. But what will happen if we both pre- and post-multiply by the same vector?

$$\mathbf{v}^{\mathrm{T}}\mathbf{A}\mathbf{v} = \zeta \tag{17.1}$$

Take a moment to check the sizes of the multiplications to confirm that this vector-matrix-vector product really does produce a scalar (that one is the Greek letter "zeta"). Thatâ€™s pretty neat, because in total this vector-matrix-vector expression contains 2M + M^2 elements, which can be a huge number of computations (e.g., 10,200 elements for M = 100), and it all boils down to a single number.

Equation 17.1 is called the "quadratic form of matrix A" and it represents the energy in the matrix over the coordinate space described by vector v. That "energy" definition will make more sense in the next section when you learn about the geometric perspective of the quadratic form. First I want to show a few numerical examples, using the same matrix and different vectors.

$$\mathbf{A} = \begin{bmatrix} 2 & 4 \\ 0 & 3 \end{bmatrix} \quad \mathbf{v}_1 = \begin{bmatrix} 3 \\ -1 \end{bmatrix} \quad \mathbf{v}_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\mathbf{v}_1^{\mathrm{T}}\mathbf{A}\mathbf{v}_1 = \begin{bmatrix} 3 & -1 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 3 \\ -1 \end{bmatrix} = 9 \tag{17.2}$$

$$\mathbf{v}_2^{\mathrm{T}}\mathbf{A}\mathbf{v}_2 = \begin{bmatrix} 2 & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 19 \tag{17.3}$$

Two things in particular to notice about this example: The matrix is always pre- and post-multiplied by the same vector; and the same matrix multiplied by different vectors will give different results (thatâ€™s obvious, but it becomes important later). Again, weâ€™ll return to the interpretation of this in the next section.

Now letâ€™s generalize this by using letters instead of numbers. I will use a,b,c,d for the matrix elements and x,y for the vector elements.

$$\begin{bmatrix} x & y \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + cy & bx + dy \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \quad (17.4)$$

$$= (ax + cy)x + (bx + dy)y \quad (17.5)$$

$$= ax^2 + (c + d)xy + dy^2 \quad (17.6)$$

Please take a moment to work through the multiplications by hand to confirm that you arrive at the same expression. And then take a moment to admire the beauty of what weâ€™ve done: We have converted a vector-matrix-vector expression into a polynomial.

Notice that we get three terms here: x^2 , y^2 , and their cross-product xy. The matrix elements become the coefficients on these terms, with the diagonal elements getting paired with the individual vector elements and the off-diagonal elements getting paired with the cross-term.

Now imagine that the matrix elements are fixed scalars and x and y are continuous variables, as if this were a function of two variables:

$$f(\mathbf{A}, \mathbf{v}_i) = \begin{bmatrix} x_i & y_i \end{bmatrix} \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} = ax_i^2 + (c+d)x_i y_i + dy_i^2 = \zeta_i \quad (17.7)$$

The interpretation is that for each matrix A, we can vary the vector elements x_i and y_i and obtain a scalar value for each (x_i ,y_i) pair.

Itâ€™s often fruitful in math to think about what happens to functions as their variables approach 0 and Â±âˆž. As x and y go to zero, it is trivial that Î¶ will also go to zero (anything times zeros vectors will be zero). As x and y grow towards infinity, the function can either go to plus infinity, minus infinity, or can balance out to zero, depending on the exact coefficients. For example, if a > d > 0 then Î¶ will grow to plus infinity as x tends to plus or minus infinity.

Whether this expression is always positive, always negative, or sometimes positive and sometimes negative, is an important question that defines the definiteness of the matrix. More on this later in the chapter.

[1] The squared terms are why this expression is called the quadratic form. Below is an example with a 3Ã—3 matrix.

[1] Quadratic is a general term for a 2nd degree polynomial.

$$\mathbf{v}^T \mathbf{A}\mathbf{v} \quad (17.8)$$

$$= \begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} a & f & d \\ f & b & e \\ d & e & c \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$= \begin{bmatrix} ax + fy + dz & fx + by + ez & dx + ey + cz \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad (17.9)$$

$$= ax^2 + fyx + dzx + fxy + by^2 + ezy + dxz + eyz + cz^2 \quad (17.10)$$

$$= ax^2 + by^2 + cz^2 + 2dxz + 2eyz + 2fxyz \quad (17.11)$$

Yeah, thatâ€™s quite a lot to look at. Still, you see the squared terms and the cross-terms, with coefficients defined by the elements in the matrix, and the diagonal matrix elements paired with their corresponding squared vector elements.

Iâ€™m sure youâ€™re super-curious to see how it looks for a 4Ã—4 matrix. Itâ€™s written out below. The principle is the same: diagonal matrix elements are coefficients on the squared vector elements, and the off-diagonals are coefficients on the cross-terms. Just donâ€™t expect me to be patient enough to keep this going for larger matrices...

$$\begin{bmatrix} w & x & y & z \end{bmatrix} \begin{bmatrix} a & b & c & d \\ b & e & f & g \\ c & f & h & i \\ d & g & i & j \end{bmatrix} \begin{bmatrix} w \\ x \\ y \\ z \end{bmatrix} =$$

$$aw^2 + ex^2 + hy^2 + jz^2 + 2bwx + 2cwy + 2dwz + 2fxy + 2gxz + 2iyz$$

Symmetric matrices When the matrix is symmetric, then the quadratic form is also symmetric. This is easily proven by transposing the entire expression:

$$\left(\mathbf{v}^{\mathrm{T}}\mathbf{A}\mathbf{v}\right)^{\mathrm{T}} = \mathrm{v}^{\mathrm{T}}\,\mathrm{A}^{\mathrm{T}}\,\mathrm{v}^{\mathrm{TT}} = \mathrm{v}^{\mathrm{T}}\,\mathrm{A}\mathrm{v} \quad (17.12)$$

Complex matrices The quadratic form for complex-valued matrices is nearly the same as for real-valued matrices, except that the Hermitian transpose replaces the regular transpose: [2]

[2] Recall that $\mathrm{v}^{\mathrm{T}} = \mathrm{v}^{\mathrm{H}}$ for a real-valued vector.

$$\mathbf{v}^{\mathrm{H}}\mathbf{A}\mathbf{v} = \zeta \quad (17.13)$$

If the matrix A is Hermitian (the complex version of symmetric, thus $\mathrm{A}^{\mathrm{H}} = \mathrm{A}$), then the quadratic form is real-valued. Non-Hermitian matrices will have complex-valued quadratic forms. Equations 17.12 and 17.13 are two of the reasons why some people limit the quadratic form to symmetric (and Hermitian) matrices.

Rectangular matrices Technically, the quadratic form is mathematically valid for rectangular matrices, if the first and second vectors differ.

$$\begin{bmatrix} w & x & y \end{bmatrix} \begin{bmatrix} a & b & c & d \\ b & e & f & g \\ c & f & h & i \end{bmatrix} \begin{bmatrix} w \\ x \\ y \\ z \end{bmatrix} =$$

$$aw^2 + ex^2 + hy^2 + 2bwx + 2cwy + dzw + 2fxy + gzx + izy$$

This result is slightly harder to interpret, because the left side is in â„ ^{3} whereas the right side is in â„ ^{4} (in this particular example). This situation is not further considered.

Code Not much new here, but be mindful of vector orientations: The vector on the left needs to be a row vector, regardless of its original orientation. Iâ€™ve swapped the orientations in the code just to make it a bit more confusing (which requires you to think a bit more!).

Code blockÂ 17.1: Python

```
importÂ numpyÂ asÂ npÂ
mÂ =Â 4Â
AÂ =Â np.random.randn(m,m)Â
vÂ =Â np.random.randn(1,m)Â #Â rowÂ vec.Â
v@A@v.T
```

Code blockÂ 17.2: MATLAB

```
mÂ =Â 4;Â
AÂ =Â randn(m);Â
vÂ =Â randn(1,m);Â
v*A*vâ€™
```

-|*|- **Reflection** Notice that when A = I, the quadratic form reduces to the dot product of the vector with itself, which is the magnitude-squared of the vector. Thus, putting a different matrix in between the vectors is like using this matrix to modulate, or scale, the magnitude of this vector. In fact, this is the mechanism for measuring distance in several non-Euclidean geometries. -|*|-

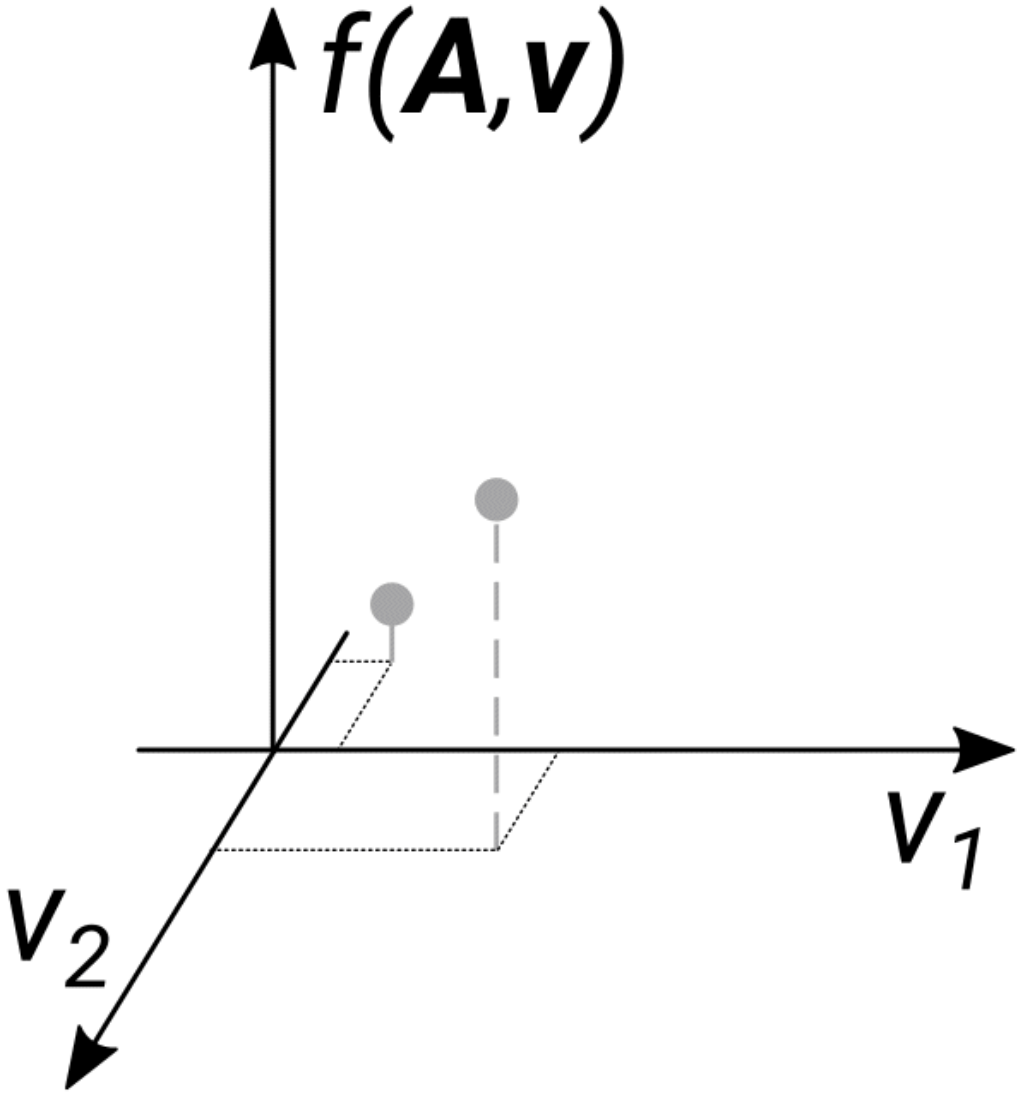
17.2 Geometric perspective

Letâ€™s return to the idea that the quadratic form is a function that takes a matrix and a vector as inputs, and produces a scalar as an output:

$$f(\mathbf{A}, \mathbf{v}) = \mathbf{v}^{\mathrm{T}}\mathbf{A}\mathbf{v} = \zeta \tag{17.14}$$

Now letâ€™s think about applying this function over and over again, for the same matrix and different elements in vector v. In fact, we can think about the vector as a coordinate space where the axes are defined by the vector elements. This can be conceptualized in

â„ ^{2} , which is illustrated in Figure 17.1 . In fact, the graph of f(A,v) for v âˆˆ â„ ^{2} is a 3D graph, because the two elements of v provide a 2D coordinate space, and the function value is mapped onto the height above (or below) that plane.

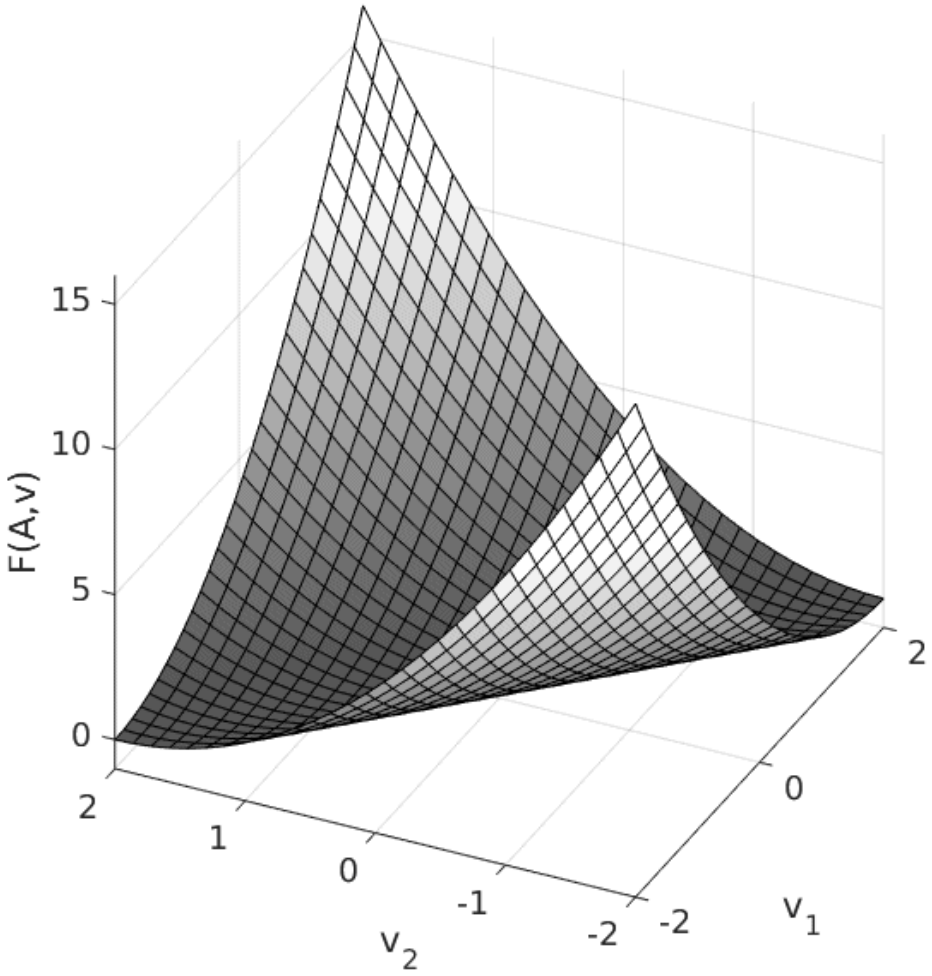


FigureÂ 17.1: A visualization of the quadratic form result of a matrix at two specific coordinates.

Thus, the 2D plane defined by the v_1 and v_2 axes is the coordinate system; each location on that plane corresponds to a unique combination of elements in the vector, that is, when setting v = [v_1 ,v_2]. The z-axis is the function result (Î¶). The vertical dashed gray lines leading to the gray dots indicate the value of Î¶ for two particular vâ€™s.

Once we have this visualization, the next step is to evaluate f(A,v) for many different possible values of v (of course, using the same A). If we keep using stick-and-button

lines like in Figure 17.1 , the plot will be impossible to interpret. So letâ€™s switch to a surface plot (17.2).



FigureÂ 17.2: A better visualization of the quadratic form surface of a matrix.

That graph is called the quadratic form surface , and itâ€™s like an energy landscape: The matrix has a different amount of "energy" at different points in the coordinate system (that is, plugging in different values into v), and this surface allows us to visualize that energy landscape.

Letâ€™s make sure this is concrete. The matrix that I used to create that surface is

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

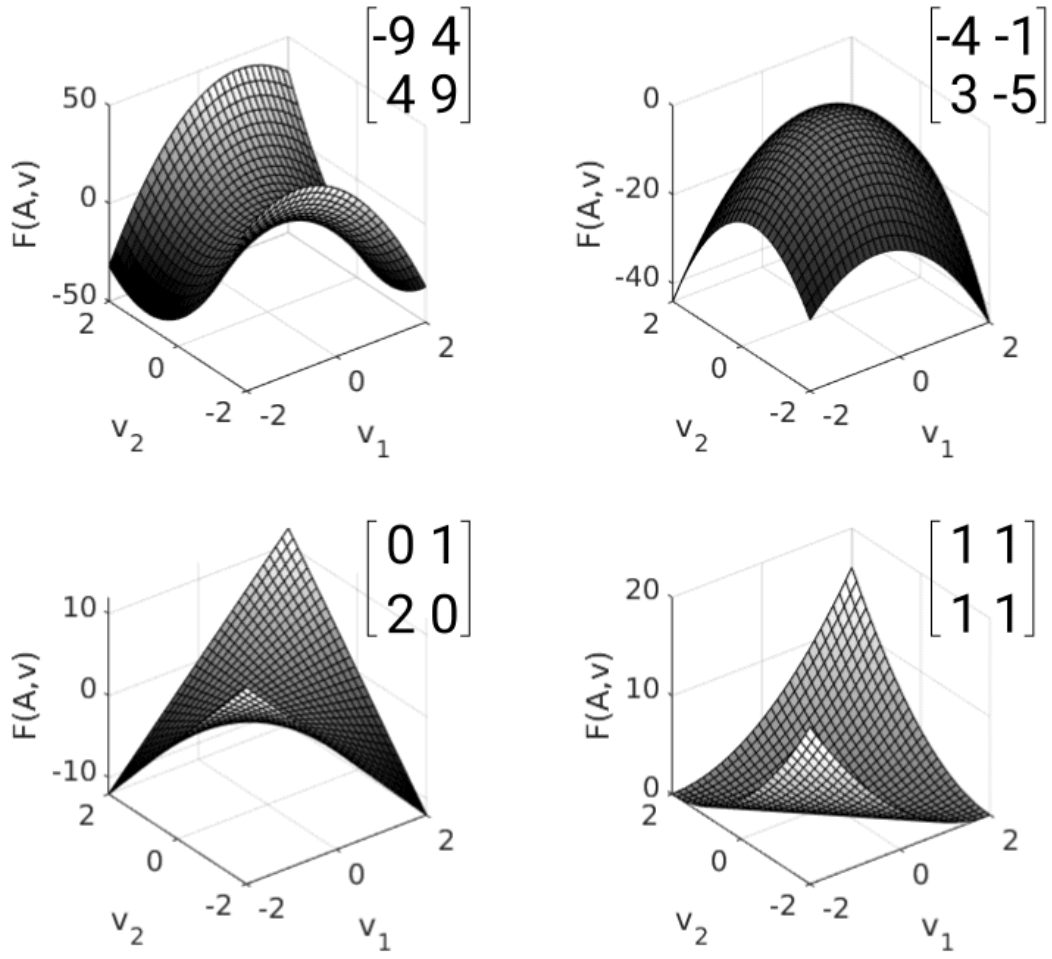
Now letâ€™s compute one specific data value, at coordinate (-2,1), which is near the lower-left corner of the plot. Letâ€™s do the math:

[3] [3]Parentheses added to indicate the operation I did first. $\left(\begin{bmatrix}-2 & 1\end{bmatrix}\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}\right)\begin{bmatrix}-2\\ 1\end{bmatrix} = \begin{bmatrix}-1 & -1\end{bmatrix}\begin{bmatrix}-2\\ 1\end{bmatrix} = 1$

The value (the height of the surface) of the function at coordinate (-2,1) is 1. That looks plausible from the graph.

The coordinates for this quadratic form surface are not bound by Â±2; the plane goes to infinity in all directions, but itâ€™s trimmed here because I cannot afford to sell this book with one infinitely large page. Fortunately, though, the characteristics of the surface you see here donâ€™t change as the axes grow; for this particular matrix, two directions of the quadratic form surface will continue to grow to infinity (away from the origin), while two other directions will continue at 0.

This surface is the result for one specific matrix; different matrices (with the same vector v) will have different surfaces. Figure 17.3 shows examples of quadratic form surfaces for four different matrices. Notice the three possibilities of the quadratic form surface: The quadratic form can bend up to positive infinity, down to negative infinity, or stay along zero, in different directions away from the origin of the [v_1 ,v_2] space.



FigureÂ 17.3: Examples of quadratic form surfaces for four different matrices. The v_1 ,v_2 axes are the same in all subplots, and the f(A,v) = Î¶ axis is adapted to each matrix.

There is more to say about the relationship between the matrix elements and the features of the quadratic form surface. In fact, the shape and sign of the quadratic form reflects the definiteness of the matrix, its eigenvalues, its invertibility, and other remarkable features. For now, simply notice that the shape and sign of the quadratic form surface depends on the elements in the matrix.

That said, one thing that all quadratic form surfaces have (for all matrices) is that they equal zero at the origin of the graph, corresponding to v = [0 0]. Thatâ€™s obvious algebraicallyâ€”the matrix is both pre- and post-multiplied by all zerosâ€”but geometrically, it means that we are interested in the shape of the matrix relative to the origin.

This visualization of the quadratic form is beautiful and insightful, but is limited to 2x2 matrices. The quadratic form surface of a 3x3 matrix, for example, is some kind of 4D hyper-sheet that is floating on top of a 3D coordinate space. It could be compressed down into 3D as a solid block where each point in that block has a color value. But we still couldnâ€™t visualize that dense block. Nonetheless, the concept is the same: the quadratic form of a matrix is zero at the origin of space, and in different directions can go up to infinity, down to minus infinity, or lie flat along zero, depending on the values in the matrix. Weâ€™re going to quantify those directions later, but there is more to explore about the quadratic form.

Code I bet youâ€™re hoping to see to code to generate those beautiful surfaces! Thatâ€™s reserved for one of the code challenges.

17.3 The normalized quadratic form

On the quadratic form surface, any direction away from the origin goes in one of three directions: down to negative infinity, up to positive infinity, or to zero. However, the surface doesnâ€™t necessarily go to infinity equally fast in all directions; some directions have steeper slopes than others. However, if a quadratic form goes up, it will eventually get to infinity [4].

[4] Philosophical side-note: What does it mean to "eventually" get to âˆžand how long does that take?

Thatâ€™s because the value of the function at each [v_1 ,v_2] coordinate is not only a function of the matrix; it is also a function of the vector v. As the vector elements grow, so does the quadratic form. That statement may feel intuitive, but weâ€™re going to take the time to prove it rigorously. This is important because the proof will help us discover how to normalize the quadratic form, which becomes the mathematical foundation of principal components analysis.

The proof involves (1) expressing the quadratic form as a vector dot product,[5] (2) applying the Cauchy-Schwarz inequality to that dot product, and then (3) applying the Cauchy-Schwarz inequality again to the matrix-vector product.

[5] The Cauchy-Schwarz inequality for the dot product was on page Â§ if you need a refresher.

To start with, think about v^T Av as the dot product between two vectors: $(v)^T$ (Av). Then apply the Cauchy-Schwarz inequality:

$$|\mathbf{v}^T\mathbf{Av}| \leq \|\mathbf{v}\|\|\mathbf{Av}\| \tag{17.15}$$

The next step is to break up âˆ¥Avâˆ¥ into the product of norms:[6]

[6] The Cauchy-Schwarz inequality for matrix-vector multiplication was on page Â§ .

$$\|\mathbf{Av}\| \leq \|\mathbf{A}\|_F\|\mathbf{v}\| \tag{17.16}$$

Then we put everything together:

$$\begin{aligned} |v^T\ Av| &\quad â‰¤ â€¾âˆ¥vâˆ¥âˆ¥Aâˆ¥_F\ âˆ¥vâˆ¥ \\ |v^T\ Av| &\quad â‰¤ â€¾âˆ¥Aâˆ¥_F\ âˆ¥vâˆ¥^2 \end{aligned} \tag{17.17}$$

Equation 17.17 is the crux of our predicament: The magnitude of the quadratic form depends on the matrix and on our coordinate system. In other words, of course the quadratic form tends to plus or minus infinity (if not zero) as it moves away from the origin of the space defined by v (and thus as the magnitude of v increases).

You can call this a feature or you can call it a bug. Either way, it impedes using the quadratic form in statistics and in machine learning. We need something like the quadratic form that reveals important directions in the matrix space independent of the magnitude of the vector that we use for the coordinate space. That motivates including some kind of normalization factor that will allow us to explore the quadratic form in a way that is independent of the vector v.

To discover the right normalization factor, letâ€™s think about the quadratic form of the identity matrix. What values of v will maximize the quadratic for v^T Iv? The mathy way of writing this question is:

$$_{\mathbf{v}}\left\{\mathbf{v}^T\mathbf{Iv}\right\} \tag{17.18}$$

This means we are looking for the argument (here v) that maximizes the expression.

Obviously, the quadratic form goes to infinity in all directions, because weâ€™re simply adding up all of the squared vector elements. So then, what normalization factor could we apply to remove this trivial growth?

Perhaps youâ€™ve already guessed it: We normalize by the squared magnitude of v. We can put this normalization factor in the denominator. Letâ€™s start with our simple 2Ã—2 example (Equation 17.6):

$$f(\mathbf{A},\mathbf{v}) = \frac{ax^2+(c+d)xy+dy^2}{x^2+y^2} \tag{17.19}$$

The more general expression looks like this:

Eqn. box title=The normalized quadratic form

$$f_{norm}(A,v) = \frac{\mathbf{v}^T\mathbf{A}\mathbf{v}}{\mathbf{v}^T\mathbf{v}} \tag{17.20}$$

Perhaps it isnâ€™t yet obvious why normalizing by the squared vector magnitude is the right thing to do. Letâ€™s revisit Equation 17.17 but now include the normalization factor.

$$\frac{|\mathbf{v}^T\mathbf{A}\mathbf{v}|}{\|\mathbf{v}\|^2} \quad \text{â‰¤} \frac{\|\mathbf{A}\|_F\|\mathbf{v}\|^2}{\|\mathbf{v}\|^2} \tag{17.21}$$

$$\frac{|\mathbf{v}^T\mathbf{A}\mathbf{v}|}{\mathbf{v}^T\mathbf{v}} \quad \text{â‰¤} \hat{A}\hat{A}_F \tag{17.22}$$

Now we see that the magnitude of the normalized quadratic form is bounded by the magnitude of the matrix, and does not depend on the vector that provides the coordinate space.

Letâ€™s apply this normalization to the quadratic form of the identity matrix.

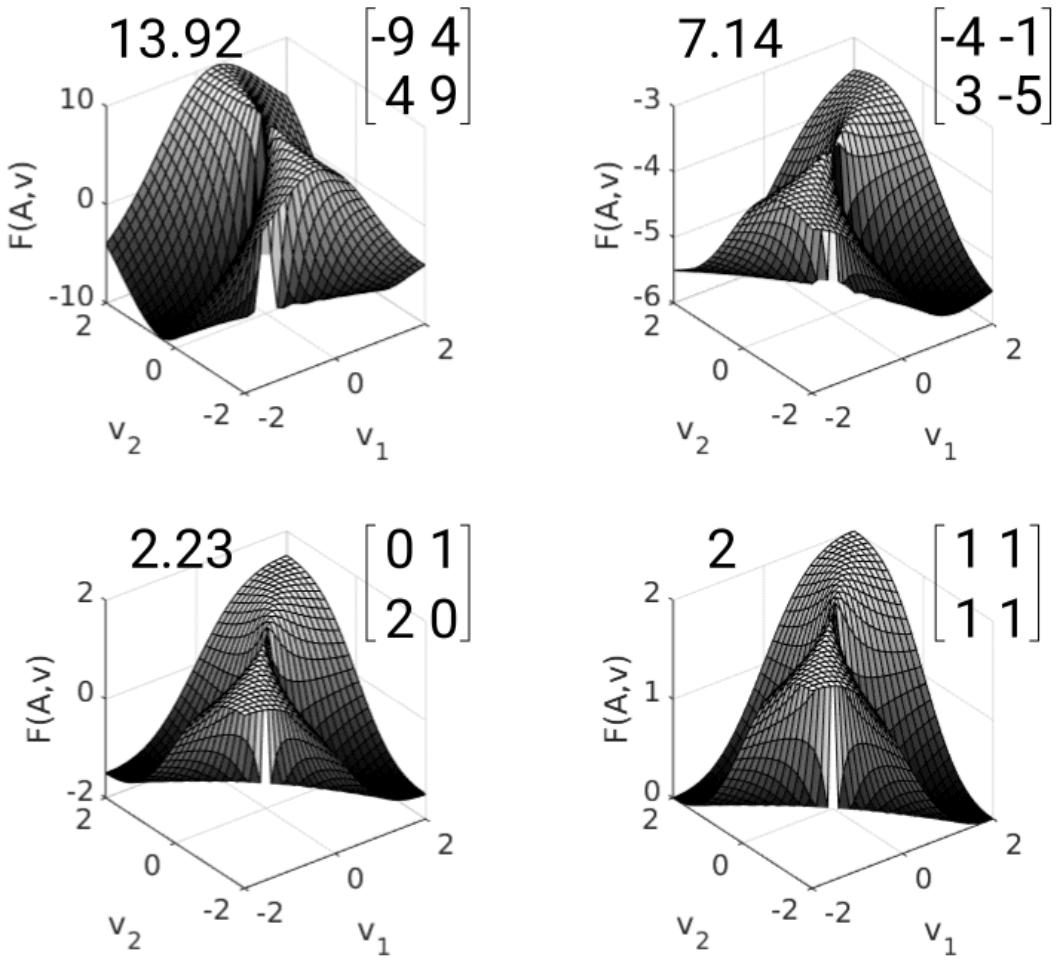
$$\frac{\mathbf{v}^T\mathbf{I}\mathbf{v}}{\|\mathbf{v}\|^2} = \mathbf{I}$$

Geometrically, this means that the normalized quadratic form surface of the identity matrix is a flat sheet on the v_1 ,v_2 plane.

Have you noticed the failure scenario yet? If you donâ€™t already know the answer, I think you can figure it out from looking again at Equation 17.20 . The failure happens when v = 0, in other words, with the zeros vector.

The normalized quadratic form surfaces look quite different from the "raw" quadratic form surfaces. Figure 17.4 shows the surfaces from the normalized quadratic forms for each of the four matrices shown in Figure 17.3 .

-|*|- **Reflection** If you are reading this book carefully and taking notes (which you should be doing!), then youâ€™ll remember that on page 13.1 I introduced the concept of "mapping over magnitude." The normalized quadratic form can be conceptualized in the same way: Itâ€™s a mapping of a matrix onto a vector coordinate space, over the magnitude of that coordinate space. -|*|-



FigureÂ 17.4: Same as Figure 17.3 but using the normalized quadratic form equation. Also notice the differences in the Î¶ axis; the number at the top-left of each graph is the Frobenius norm of that matrix.

17.4 Eigenvectors and quadratic form surfaces

Each normalized quadratic form surface in Figure 17.4 has two prominent features: a ridge and a valley. Going back to the interpretation that the quadratic form surface is a representation of the "energy" in the matrix along particular directions, the normalized surface tells us that there are certain directions in the matrix that are associated with maximal and minimal "energy" in the matrix.

(What does matrix "energy" mean? I put that term in apology quotes because it is an abstract concept that depends on what the matrix represents. In the next two chapters, the matrix will contain data covariances, and then "energy" will translate to pattern of covariance across the data features.)

Thus, normalizing out the magnitude of the vector v revealed important directions in the matrix. A 2Ã—2 matrix has two important directions, and more generally, an NÃ—N matrix has N important directions.

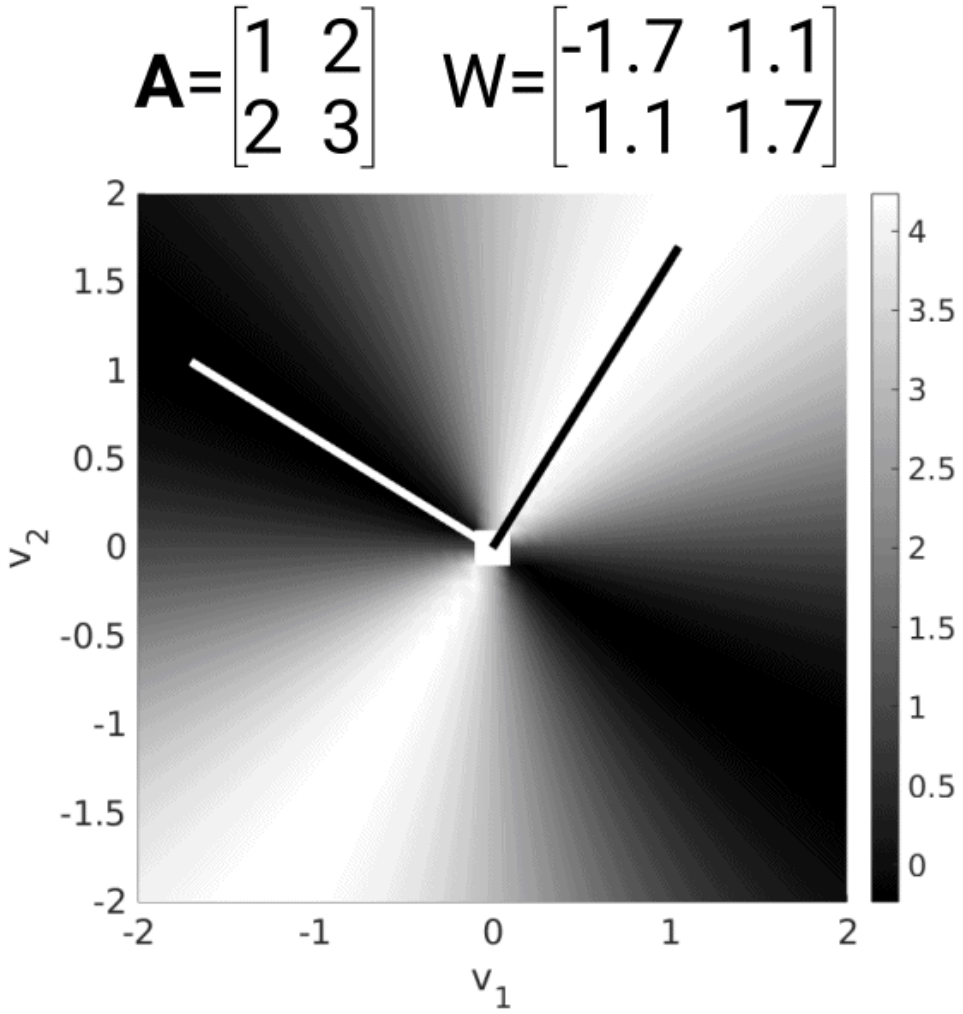
Higher-dimensional quadratic form surfaces are impossible to visualize and difficult to imagine, but the concept is this: The normalized quadratic form surface has an absolute ridge and an absolute valley, and N âˆ’ 2 saddle directions. A saddle point is a point that goes up in some directions and down in other directions (the upper-left matrix in Figure 17.3 shows an example of a saddle-point in 3D); a saddle line is a hyper-line that is straight, but the surrounding space curves up in some directions and curves down in other directions. An analogy is that [7] the absolute ridge and valley are like the global maximum and minimum, and the intermediate saddle points are like local maxima and minima.

[7] All analogies break down at some point, so you should never think too deeply about them.

For the 3D surfaces in these figures, you can identify the ridges and valleys simply by visual inspection. But thatâ€™s obviously not very precise, nor is it scalable. [8] It turns out that the eigenvectors of a symmetric matrix point along the ridges and valleys of the normalized quadratic form surface. This applies only to symmetric matrices; the eigenvectors of a non-symmetric matrix do not necessarily point along these important directions. This is another one of the special properties of symmetric matrices that make all other matrices fume with jealousy.

[8] Important information hidden inside a long paragraph. Terrible textbook writing style!

To be more precise: The eigenvector associated with the largest eigenvalue points from the origin along the ridge, and the eigenvector associated with the smallest eigenvalue points from the origin along the valley. You can see this in Figure 17.5 , which shows the quadratic form surface from the top looking down, and the eigenvectors of the matrix plotted on top.



FigureÂ 17.5: The birdâ€™s-eye-view of the quadratic form surface of a symmetric matrix, with the eigenvectors plotted on top. The matrix and its eigenvectors (W, rounded to the nearest tenth) are printed on top. Colorbar indicates the value of Î¶. Notice the small missing box at the center; this was an NaN value corresponding to v = 0.

Why is this the case? Why do the eigenvectors point along the directions of maximal and minimal "energy" in the matrix? The short answer is that the vectors that maximize the quadratic form (Equation 17.20) turn out to be the eigenvectors of the matrix. A deeper discussion of why that happens is literally the mathematical basis of principal components analysis, and so I will go through the math in that chapter.

-|*|- **Reflection** One of the most important applications of the normalized quadratic form is principal components analysis (PCA), which is a dimension-reduction and data-exploration method used in multivariate statistics, signal processing, and machine learning. The math of PCA is pretty simple: Compute a covariance matrix by multiplying the data matrix by its transpose, eigendecompose that covariance matrix, and then

multiply the data by the eigenvectors to obtain the principal components. More on this in Chapter 19! -|*|-

17.5 Definiteness, geometry, and eigenvalues

As with other properties introduced in this chapter, "definiteness" of a matrix is a property that applies only to square matrices. Some strict definitions limit definiteness to symmetric matrices, but I will relax this and say that any square matrix can have a definiteness.

Geometrically, the definiteness of a matrix refers to whether the sign of the non-normalized quadratic form surface is positive, negative, or zero in different directions away from the origin. You can see in the examples in Figure 17.3 that some matrices have positive surfaces, some negative, and some have Î¶ of both signs. (The zero point v = 0 is ignored because Î¶ = 0 is trivially the case.)

The algebraic interpretation of definiteness is whether v^T Av is positive, negative, or zero, for all possible vectors v (again, excluding v = 0).

There are five categories of definiteness. Table 17.1 provides an overview of the categories and their properties.

lightgray Category	Geometry	Eigenvalues	Invertible
Positive definite	â†‘	+	Yes
Positive semidefinite	â†‘, 0	+, 0	No
Indefinite	â†‘ and â†“	+, -	Possibly
Negative semidefinite	â†“, 0	-, 0	No
Negative definite	â†“	-	Yes

TableÂ 17.1: Properties of matrix definiteness categories. â†‘ and â†“ indicate that the quadratic form surface goes up or down relative to the origin; "0" indicates that it can also be zero outside the origin. "+" and "-" refer to the signs of the eigenvalues of the matrix.

A few observations of Table 17.1 : Definite matrices are always invertible while semidefinite matrices are never invertible (because they have at least one zero-valued eigenvalue). [9] The invertibility of indefinite matrices is unpredictable; it depends on the numbers in the matrix. Finally, matrix definiteness is exclusiveâ€”a matrix cannot have more than one definiteness label (except for the always-bizarre zeros matrix; see Reflection Box).

[9] Indefinite matrices are sometimes called indeterminate.

With this list in mind, refer back to Figure 17.3 and see if you can determine the definiteness of each matrix based on visual inspection and the associated descriptions.

Although I really enjoy looking at quadratic form surfaces, they are not the best way to determine the definiteness of a matrix. Instead, the way to determine the definiteness category of a matrix is to inspect the signs of the eigenvalues.

Question: Can you determine the definiteness of a matrix from its singular values? The answer is in the footnote.[10]

[10] The answer is No, because all singular values are non-negative.

Complex matrices For Hermitian matrices (remember that a Hermitian matrix is the complex version of a symmetric matrix: it equals its Hermitian transpose), the story is exactly the same. This is because v

HCvisarealâˆ’valuednumber,andbecausetheeigenvaluesofaHermitianmatrixarerealâˆ’valued.

-|*|- **Reflection** When it comes to definiteness, no matrix is weirder than the zeros matrix. Its quadratic form surface is flat and zero everywhere, and it actually belongs to two definiteness categories: positive semidefinite and negative semidefinite. Iâ€™m not sure if that will help you live a richer, healthier, and more meaningful life, but I thought Iâ€™d mention it just in case. -|*|-

17.6 The definiteness of $A^T A$

Here is yet another special property of the symmetric matrix $A^T A$: It is guaranteed to be positive definite or positive semidefinite [often written "positive (semi)definite"]. It cannot be indefinite or negative (semi)definite.

The proof here is fairly straightforward. Letâ€™s start by defining the quadratic form for matrix $S = A^T A$.

$$\begin{aligned} w^T Sw &= w^T (A^T A)w \\ &= (w^T A^T)(Aw) \\ &= (Aw)^T (Aw) \\ &= \hat{}¥Aw\hat{}¥^2 \end{aligned} \qquad (17.23)$$

[11]

The key take-home message here is that the quadratic form of a matrix $A^T A$ can be re-written as the squared magnitude of a matrix times a vector. Magnitudes cannot be negative. A magnitude (and, thus, the quadratic form) can be zero when the vector is zero. In this context, it means that $Aw = 0$ for at least one vector w. And because we do not consider the zeros vector here, then it means that A has a non-trivial null space, which means that it is singular.

[11] w instead of v. Try to see the concepts behind the letters.

(This is another nice example of how simply moving parentheses can provide important insights into linear algebra.)

The conclusion is that any matrix that can be expressed as $A^T A$ is positive definite if it is full-rank, and positive semidefinite if it is reduced rank.

Before going further, please take a moment to test whether the above proof works for the matrix AA^T . (Obviously it will, but itâ€™s good practice to work through it on your own.)

Important: All matrices $S = A^T A$ are symmetric, but not all symmetric matrices can be expressed as $A^T A$ (or AA^T). [12] A symmetric matrix that cannot be decomposed into another matrix times its transpose is not necessarily positive definite. Symmetry does not guarantee positive (semi)definiteness.

[12] Decomposing a symmetric matrix S into $A^T A$ is the goal of the Cholesky decomposition, which is valid only for positive (semi)definite matrices.

Here is an example:

$$\begin{bmatrix} -9 & 4 \\ 4 & 9 \end{bmatrix}, \quad \lambda s = -9.8,\ 9.8$$

17.7 Eigenvalues and matrix definiteness

A few pages ago, I wrote that the best way to determine the definiteness of a matrix is to inspect the signs of its eigenvalues. You donâ€™t even need to know the magnitudes of the eigenvalues; you only need to know whether they are positive, negative, or zero.

Positive definite case Letâ€™s start by proving that all eigenvalues of a positive definite matrix are positive. We begin by writing the eigenvalue equation and left-multiplying by the eigenvector to arrive at the quadratic form. Assume that matrix A is positive definite.

$$Av = Îv \tag{17.24}$$

$$v^T Av = v^T Îv \tag{17.25}$$

$$v^T Av = Îv^T v \tag{17.26}$$

$$v^T Av = Îâˆ¥vâˆ¥^2 \tag{17.27}$$

Equation 17.27 says that the (non-normalized) quadratic form is equal to the squared vector magnitude times the eigenvalue. Our assumption is that A is positive definite, meaning that $v^T Av$ is positive for all vectors vâ‰ 0. Thus, the left-hand side of the equation is always positive. On the right-hand side we have the product of two terms, one of which must be positive ($âˆ¥vâˆ¥^2$), and the other of which (Î») in theory could be positive or negative. But if Î» were negative, then the left-hand side must also be negative. Ergo, Î» must be positive.

But wait a minute: the quadratic form is defined for all vectors, not only for eigenvectors. You might be tempted to say that our conclusion (positive definite means all positive eigenvalues) is valid only for eigenvectors.

The key to expanding this to all vectors is to appreciate that as long as the eigenvectors matrix is full-rank and therefore spans all of â„ ^{M} , any vector can be created by some linear combination of eigenvectors. Letâ€™s see how this works for a linear combination of two eigenvectors.

$$A(v_1 + v_2) = (\hat{I}»_1 + \hat{I}»_2)(v_1 + v_2) \quad (17.28)$$

$$(v_1 + v_2)^T A(v_1 + v_2) = (v_1 + v_2)^T (\hat{I}»_1 + \hat{I}»_2)(v_1 + v_2) \quad (17.29)$$

$$= (\hat{I}»_1 + \hat{I}»_2)â^¥v_1 + v_2 â^¥^2 \quad (17.30)$$

Appreciating that $(v_1 + v_2)$ is simply a vector and $(\hat{I}»_1 + \hat{I}»_2)$ is simply a scalar, we can simplify:

$$w = v_1 + v_2 \quad (17.31)$$

$$\hat{I}^2 = (\hat{I}»_1 + \hat{I}»_2) \quad (17.32)$$

$$w^T Aw = \hat{I}^2â^¥wâ^¥^2 \quad (17.33)$$

Equation 17.33 allows us to draw the same conclusion as Equation 17.27 . In this case, the two eigenvectors were each weighted (implicitly) with 1. You can work through on your own how to modify this equation for any other linear combination of the eigenvectors; it will involve scalar-multiplying each vector v_i by some arbitrary scalar $\hat{I}±_i$. Even if you choose a negative-valued scalar, it will be squared on both sides of the equation and thus would not change the conclusion.

That conclusion is that if we assume that a matrix is positive definite, then all of its eigenvalues must be positive. You can also think about this proof the other way: If a matrix has all positive eigenvalues (meaning the right-hand side of Equation 17.33 is always positive), then it is a positive definite matrix (meaning the left-hand side of that equation is always positive).

Positive semidefinite case Letâ€™s continue our proof with positive semidefinite matrices, which Table 17.1 claims have all non-negative eigenvalues.

We donâ€™t need to write out a new set of equations. All we need to do [13] is change our assumption about the matrix: Now we assume that the quadratic form of A is non-negative, meaning that $\hat{I}¶$ can be positive and it can be zero, but it can never be negative. Below is Equation 17.27 repeated for reference.

[13] Pro life tip: Sometimes you need to reassess your assumptions about matrices, yourself, other people, life, etc.

$$\mathbf{v}^T\mathbf{A}\mathbf{v} = \lambda\|\mathbf{v}\|^2$$

Again, the right-hand-side of the equation has a term that is strictly positive and a term that could be positive, negative, or zero. The left-hand side of the equation can take values that are positive or zero, but not negative. This proves that at least one eigenvalue must be zero, and all non-zero eigenvalues must be positive. And the proof works the

other way: If all eigenvalues are zero or positive, then the quadratic form must be non-negative, which is definition of positive semidefinite.

The other categories I trust you see the pattern here. Proving the relationship between the signs of the eigenvalues and the definiteness of a matrix does not require additional math; it simply requires changing your assumption about A and re-examining the above equations. As math textbook authors love to write: The proof is left as an exercise to the reader.

17.8 Code challenges

1.Â

Write code to create and visualize the quadratic form surfaces of 2Ã—2 matrices.

2.Â

The goal of this code challenge is to explore the definiteness of random-integer matrices. Start by writing code that generates a 4Ã—4 matrix of random integers between -10 and +10, and that has all real-valued eigenvalues. You could solve this by multiplying the matrix by its transpose, but that will limit the possibilities of definiteness categories. Thus, continue creating random matrices until all 4 eigenvalues are real-valued. Next, compute the definiteness category of that matrix.

Once you have the code working, embed it inside a for-loop to generate 500 matrices. Store the definiteness category for each matrix. Finally, print out a list of the number of matrices in each definiteness category. What have you discovered about the quadratic forms of random matrices?

17.9 Code solutions

1.Â

The code below computes the normalized quadratic form surfaces. You can modify the code to create the non-normalized quadratic form surface by commenting out the division (lines 14/8 for Python/MATLAB). Mind the parentheses in the normalization step.

Code blockÂ 17.3: Python

```
importÂ numpyÂ asÂ npÂ
importÂ matplotlib.pyplotÂ asÂ pltÂ
fromÂ mpl_toolkits.mplot3dÂ importÂ Axes3DÂ
Â
AÂ =Â np.array([[-2,3],[2,8]])Â #Â matrixÂ
viÂ =Â np.linspace(-2,2,30)Â #Â vectorÂ elementsÂ
qfÂ =Â np.zeros((len(vi),len(vi)))Â
Â
#Â forÂ theÂ visualizationÂ
```

```
X,YÂ =Â np.meshgrid(vi,vi)Â
Â
forÂ iÂ inÂ range(len(vi)):Â
Â Â Â Â forÂ jÂ inÂ range(len(vi)):Â
Â Â Â Â Â Â Â Â vÂ =Â np.array([Â vi[i],vi[j]Â ])Â
Â Â Â Â Â Â Â Â qf[i,j]Â =Â v.T@A@vÂ /Â (v.T@v)Â
Â
axÂ =Â plt.axes(projection=â€™3dâ€™)Â
ax.plot_surface(X,Â Y,Â qf.T)Â
ax.set_xlabel(â€™v_1â€™),Â ax.set_ylabel(â€™v_2â€™)Â
ax.set_zlabel(â€™$\zeta$â€™)Â
plt.show()
```

Code blockÂ 17.4: MATLAB

```
AÂ =Â [1Â 2;Â 2Â 3];Â %Â matrixÂ
viÂ =Â -2:.1:2;Â %Â vectorÂ elementsÂ
quadformÂ =Â zeros(length(vi));Â
Â
forÂ i=1:length(vi)Â
Â Â Â Â forÂ j=1:length(vi)Â
Â Â Â Â Â Â Â Â vÂ =Â [vi(i)Â vi(j)]â€™;Â %Â vectorÂ
Â Â Â Â Â Â Â Â quadform(i,j)Â =Â vâ€™*A*v/(vâ€™*v);Â
Â Â Â Â endÂ
endÂ
Â
surf(vi,vi,quadformâ€™)Â
xlabel(â€™v_1â€™),Â ylabel(â€™v_2â€™)Â
zlabel(â€™$\zeta$â€™,â€™Interpreterâ€™,â€™latexâ€™)
```

2.Â

Notice that Iâ€™ve set a tolerance for "zero"-valued eigenvalues, as discussed in previous chapters for thresholds for computing rank. You will find that all or nearly all random matrices are indefinite (positive and negative eigenvalues). If you create smaller matrices (3Ã—3 or 2Ã—2), youâ€™ll find more matrices in the other categories, although indefinite will still dominate. Category number corresponds to the rows of table 17.1 .

Code blockÂ 17.5: Python

```
importÂ numpyÂ asÂ npÂ
nÂ =Â 4Â
nIterationsÂ =Â 500Â
defcatÂ =Â np.zeros(nIterations)Â
Â
forÂ iteriÂ inÂ range(nIterations):Â
```

```
Â #Â createÂ theÂ matrixÂ
Â AÂ =Â np.random.randint(-10,11,size=(n,n))Â
Â eÂ =Â np.linalg.eig(A)[0]Â
Â whileÂ ~np.all(np.isreal(e)):Â
Â Â AÂ =Â np.random.randint(-10,11,size=(n,n))Â
Â Â eÂ =Â np.linalg.eig(A)[0]Â
Â
Â #Â "zero"Â thresholdÂ (fromÂ rank)Â
Â t=n*np.spacing(np.max(np.linalg.svd(A)[1]))Â
Â #Â testÂ definitenessÂ
Â ifÂ np.all(np.sign(e)==1):Â
Â Â defcat[iteri]Â =Â 1Â #Â pos.Â defÂ
Â elifÂ np.all(np.sign(e)>-1)&sum(abs(e)<t)>0:Â
Â Â defcat[iteri]Â =Â 2Â #Â pos.Â semidefÂ
Â elifÂ np.all(np.sign(e)<1)&sum(abs(e)<t)>0:Â
Â Â defcat[iteri]Â =Â 4Â #Â neg.Â semidefÂ
Â elifÂ np.all(np.sign(e)==-1):Â
Â Â defcat[iteri]Â =Â 5Â #Â neg.Â defÂ
Â else:Â
Â Â defcat[iteri]Â =Â 3Â #Â indefiniteÂ
Â
#Â printÂ outÂ summaryÂ
forÂ iÂ inÂ range(1,6):Â
Â Â print(â€™catÂ %g:Â %gâ€™%(i,sum(defcat==i)))
```

Code blockÂ 17.6: MATLAB

```
nÂ =Â 4;Â
nIterationsÂ =Â 500;Â
defcatÂ =Â zeros(nIterations,1);Â
Â
forÂ iteri=1:nIterationsÂ
Â
Â Â %Â createÂ theÂ matrixÂ
Â Â AÂ =Â randi([-10Â 10],n);Â
Â Â evÂ =Â eig(A);Â %Â evÂ =Â EigenValuesÂ
Â Â whileÂ ~isreal(ev)Â
Â Â Â Â AÂ =Â randi([-10Â 10],n);Â
Â Â Â Â evÂ =Â eig(A);Â
Â Â endÂ
Â
Â Â %Â "zero"Â thresholdÂ (fromÂ rank)Â
Â Â tÂ =Â nÂ *Â eps(max(svd(A)));Â
Â
```

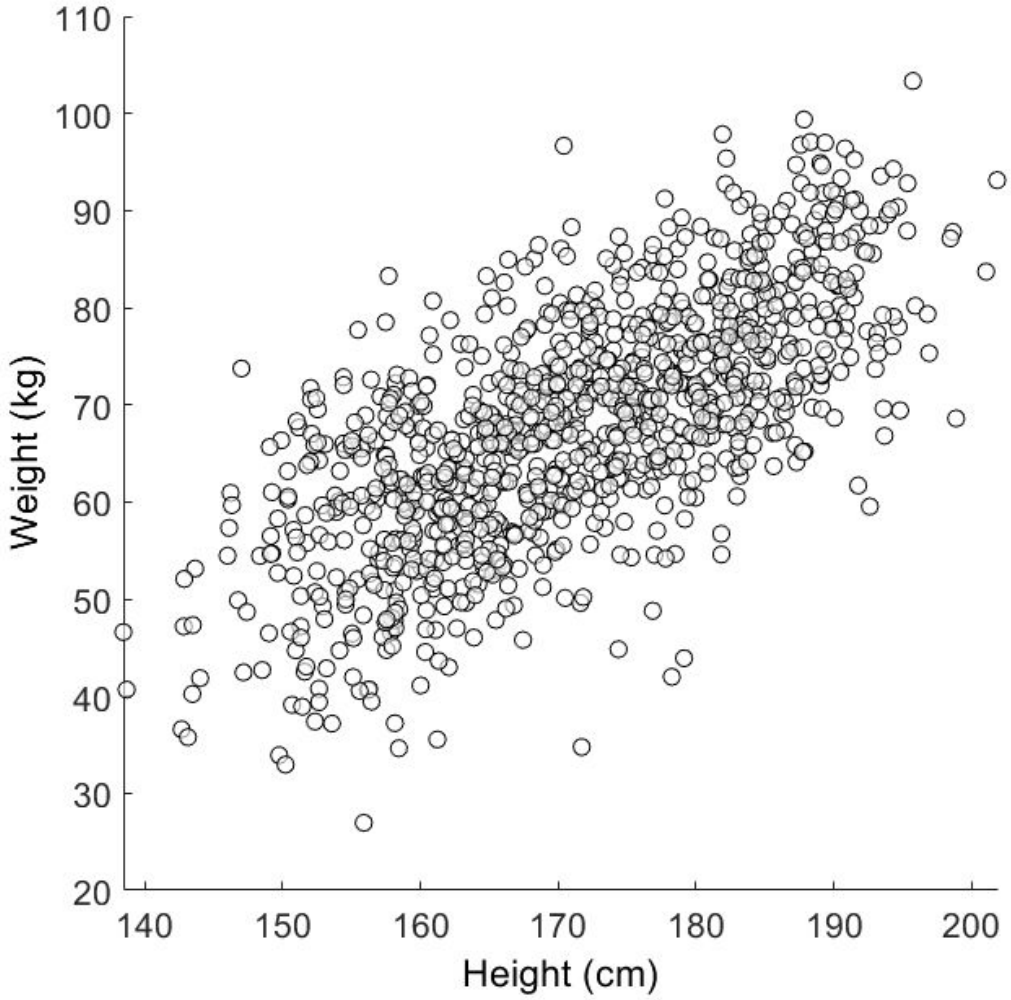
```
Â Â %Â testÂ definitenessÂ
Â Â ifÂ all(sign(ev)==1)Â
Â Â Â Â defcat(iteri)Â =Â 1;Â %Â pos.Â defÂ
Â Â elseifÂ all(sign(ev)>-1)Â &&Â sum(abs(ev)<t)>0Â
Â Â Â Â defcat(iteri)Â =Â 2;Â %Â pos.Â semidefÂ
Â Â elseifÂ all(sign(ev)<1)Â &&Â sum(abs(ev)<t)>0Â
Â Â Â Â defcat(iteri)Â =Â 4;Â %Â neg.Â semidefÂ
Â Â elseifÂ all(sign(ev)==-1)Â
Â Â Â Â defcat(iteri)Â =Â 5;Â %Â neg.Â defÂ
Â Â elseÂ
Â Â Â Â defcat(iteri)Â =Â 3;Â %Â indefiniteÂ
Â Â endÂ
endÂ
Â
%Â printÂ outÂ summaryÂ
forÂ i=1:5Â
Â Â Â Â fprintf(â€™catÂ %g:Â %g\nâ€™,i,sum(defcat==i))Â
end
```

ChapterÂ 18
Data and covariance matrices

This chapter may seem like it belongs more in a statistics textbook than a linear algebra textbook, but correlation and covariance will bring linear algebra conceptsâ€”including the dot product, matrix multiplication, transposing, symmetric matrices, and inversesâ€”to the world of statistics and data science. And covariance matrices are key to principal components analysis, which is the main focus of the next chapter.

18.1 Correlation: Motivation and interpretation

The goal of a correlation analysis is to identify commonalities between variables. Imagine, for example, the variables "height" and "weight" measured from a group of people. In general, taller people tend to weigh more than shorter people. We could visualize this relationship in a plot that shows a dot for each person, and their height on the x-axis and weight on the y-axis (Figure 18.1).

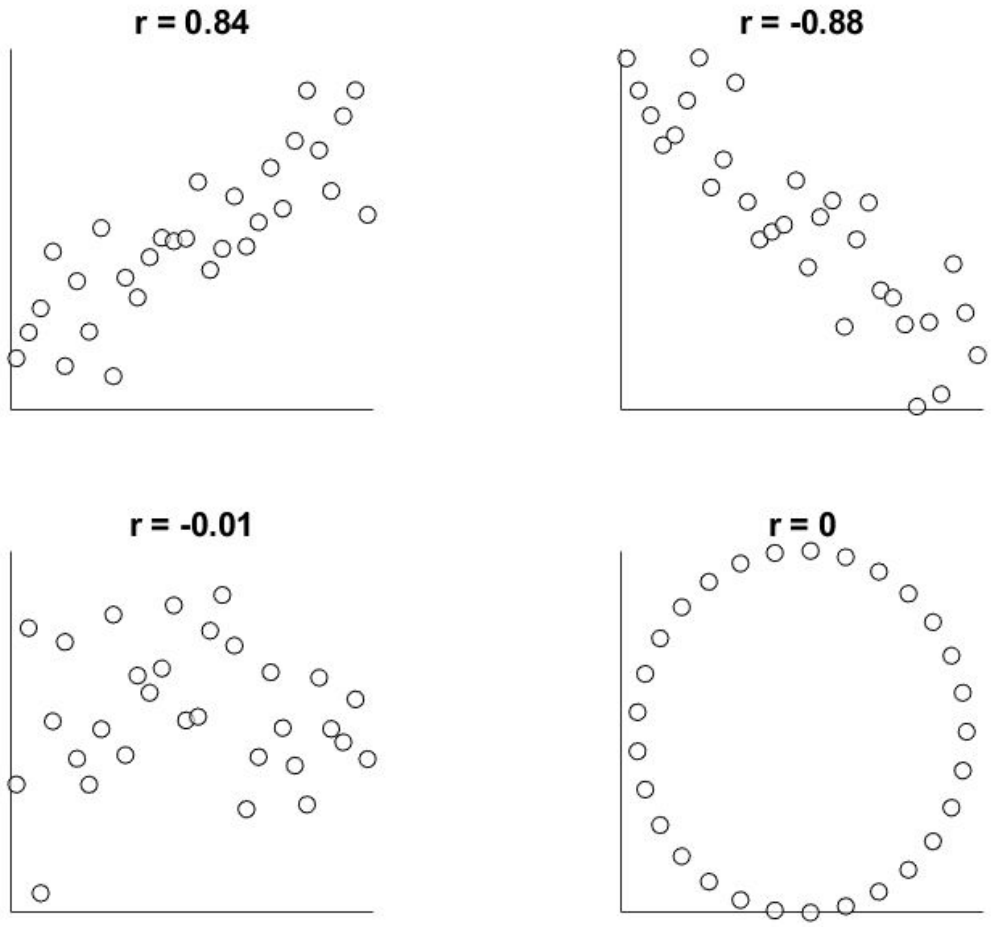


FigureÂ 18.1: A scatter plot showing the relationship between height and weight. These are made-up data.

Clearly, these two variables are related to each other; you can imagine drawing a straight line through that relationship. The correlation coefficient is the statistical analysis method that quantifies this relationship.

Correlation is a bivariate measure, meaning it can only be computed between two variables. In the next section, youâ€™ll see correlation matrices, but a correlation matrix is just a convenient way of organizing a collection of bivariate correlations.

A correlation coefficient is a number between -1 and +1. -1 means that the two variables are perfectly negatively correlated (one goes up when the other goes down), 0 means that there is no relationship between the variables, and +1 means that the two variables are perfectly positively correlated. Figure 18.2 shows examples of various relationships and their correlation coefficients.



FigureÂ 18.2: Four examples of bivariate data and their correlation coefficients ("r" value).

Limited to linearity One thing to know about the correlation is that it can detect only the linear component of an interaction between two variables. Any nonlinear interactions will not be detected by a correlation. The lower-right panel of Figure 18.2 shows an example: the value of y is clearly related to the value of x, however that relationship is nonlinear; there is no linear component of that relationship. (There are, of course, measures of nonlinear relationships, such as mutual information. Itâ€™s also possible to transform the variables so that their relationship is linearized.)

Before learning the formula to compute the correlation, itâ€™s useful to learn the formula for a covariance. Thatâ€™s because the correlation coefficient is simply a normalized version of the covariance.

A covariance is the same concept as a correlation, except it is not bound by Â±1. Instead, the covariance is in the same scale as the data. If you multiply the data by a factor of 1000, then the covariance will increase proportionally but the correlation will remain the same.

And before learning about covariance, we need to talk about variance.

18.2 Variance and standard deviation

Variance is a measure of dispersion, which is a general term to describe how different the data values are from each other.

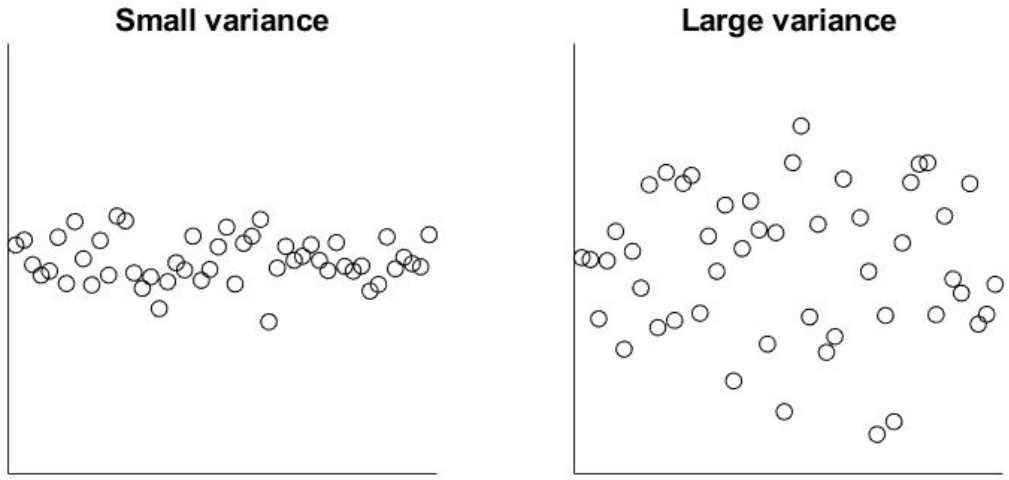
Variance is defined as the average of the squared differences between each data value and the data average. It is common to use the Greek letter Ïƒ^2 to indicate variance. In the formula below, x is a vector of data points, $\overline{x}$ is the average of all data values, and n is the total number of data points. [1]

[1] Subtracting the mean to enforce an average value of 0 is also called "mean-centering."

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(\mathbf{x}_i - \overline{\mathbf{x}})^2 \tag{18.1}$$

The scalar division by n â^' 1 is a normalization that prevents the variance from increasing simply by adding more data, analogous to dividing a sum by n to compute the average. There is a longer statistical explanation for why we scale by n â^' 1 and not n; the short answer is that mean-centering reduces the degrees of freedom by 1. As soon as we know n â^' 1 data values, we necessarily know the nth data value, because we know the dataset average.

You can imagine from the formula that when all data values are close to the mean, the variance is small; and when the data values are far away from the mean, the variance is large. Figure 18.3 shows examples of two datasets with the identical means but different variances.



FigureÂ 18.3: Examples of datasets with small and large variance.

It is obvious that we can expand the squared term in Equation 18.1 and rewrite as follows. This is relevant for understanding covariance.

$$\sigma^2 = \frac{1}{n-1}\sum_{i=1}^{n}(\mathbf{x}_i - \overline{\mathbf{x}})(\mathbf{x}_i - \overline{\mathbf{x}}) \tag{18.2}$$

Once you know the variance, you also know the standard deviation. Standard deviation is simply the square root of variance. It is implicit that standard deviation is the principal (positive) square root of variance,[2] because it doesnâ€™t make sense for a measure of dispersion to be negative, just like it doesnâ€™t make sense for a length to be negative.

[2] It is a bit confusing that σ indicates singular values in linear algebra and standard deviation in statistics. As I wrote in Chapter 14, terminological overloading is simply unavoidable in modern human civilization.

$$\sigma = \sqrt{\sigma^2} \tag{18.3}$$

18.3 Covariance

The term covariance means variance between two variables. The covariance between variables x and y is computed by modifying Equation 18.2 . _____

Eqn. box title=The covariance coefficient

$$c_{x,y} = \frac{1}{n-1} \text{â}^{\text{‘}}_{i=1}{}^{n} (x_i \text{â}^{\text{’}}\bar{x})(y_i \text{â}^{\text{’}}\bar{y}) \tag{18.4}$$

_____

_____

where n is the number of observations, and $\bar{x}$ and $\bar{y}$ are the averages over all elements in x and y. Needless to say, this equation is valid only when x and y have the same number of elements.

Equation 18.4 , though technically correct, is ugly from a linear algebra perspective. Letâ€™s see if we can make it more compact and therefore more appealing to an airplane full of linear algebraticians traveling to Hawaii for holiday (I suppose this could happen; the universe is a strange and wondrous place):

$$c_{x,y} = \frac{\mathbf{x}^{\mathrm{T}}\mathbf{y}}{n-1}, \quad \bar{\mathbf{x}} = \bar{\mathbf{y}} = 0 \tag{18.5}$$

This is looking better. It turns out that the normalization factor of n â^’ 1 is often unnecessary in applications (for example, when comparing multiple covariances that all have the same n), therefore Equation 18.5 really simplifies to $x^T y$ under the assumption that both variables are mean-centered.

This is a remarkable discovery, because it means that the covariance between two variablesâ€”one of the most fundamental building-blocks of statistics, machine-learning, and signal processingâ€”is nothing more than the dot product between two vectors that contain data.

This insight gives a huge advantage to understanding covariance and correlation for anyone who understands the algebraic and geometric properties of the dot product (that includes you!).

-|*|- **Reflection** Do data always need to be mean-centered before computing the covariance? That depends on what will be done with the covariance. For example, non-mean-centered covariance matrices produce uninterpretable PCA results, which you will see in the next chapter. Furthermore, mean-centering ensures that the diagonal of the covariance matrix contains the variances of each data feature. Thus, it is convenient to mean-center all data before computing covariances, even if it not strictly necessary for a particular application. -|*|-

18.4 Correlation coefficient

Take a moment to compare Equation 18.6 to Equation 18.4 , and identify the similiarities and differences. _____

Eqn. box title=The correlation coefficient

$$r_{x,y} = \frac{\sum_{i=1}^{n}(\mathbf{x}_i - \overline{\mathbf{x}})(\mathbf{y}_i - \overline{\mathbf{y}})}{\sqrt{\sum_{i=1}^{n}(\mathbf{x}_i - \overline{\mathbf{x}})^2}\sqrt{\sum_{i=1}^{n}(\mathbf{y}_i - \overline{\mathbf{y}})^2}} \qquad (18.6)$$

_____

_____

The numerators are the same. The denominator of Equation 18.6 has two terms, each of which is almost identical to the standard deviation, except that the 1âˆ•(nâˆ’ 1) term is missing. In fact, that normalization term is missing once in the numerator and twice in the denominator (each time inside each square root). In other words, that term cancels and thus is omitted for simplicity.

Again, the formula above is technically correct but is going to give our plane full of linear algebaticians indigestion. Let us make their flight more pleasant by rewriting Equation 18.6 , again assuming that the variables are already mean-centered.

$$r_{x,y} = \frac{\mathbf{x}^{\mathrm{T}}\mathbf{y}}{\|\mathbf{x}\|\|\mathbf{y}\|} \qquad (18.7)$$

The letter r is commonly used to indicate a correlation coefficient.

Let us meditate on this equation for a moment. This is a perfect example of the principle of mapping over magnitude: We have a dot product between vectors, normalized by the product of their magnitudes. This formula looks similar (though not identical) to the orthogonal projection of a vector onto a subspace (page Â§). And it looks even more similar to the geometric interpretation of the dot product, which Iâ€™ll discuss presently.

Equation 18.7 nicely shows how a correlation of r = 1 would be obtained. Imagine replacing y with x; the numerator and denominator would be identical. Finally, notice that if the data are already unit-normed, then the denominator is 1.

Cosine similarity I realize that Chapter 3 was a long time ago, but I hope you remember the geometric formula for the dot product, and how we derived a formula for the angle

between vectors. That was Equation 3.16 (page Â§), and I will reprint it below for convenience.

$$\cos(\theta_{ab}) = \frac{\mathbf{a}^{\mathrm{T}}\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|}$$

Remarkably, the Pearson correlation coefficient is nothing more than the cosine of the angle between two vectorsâ€”when those vectors are mean-centered. This is another example of how we arrived at the same formula from two completely different starting points (c.f. the least-squares solution).

When the data are not mean-centered, Equation 3.16 is called cosine similarity. The difference in interpretation and use of correlation vs. cosine similarity is a topic for a machine-learning course, but you can already see that the two measures are somewhere between identical and similar, depending on the average of the data values.

18.5 Covariance matrices

In applications, it is common to have larger sets of variables, perhaps dozens, hundreds, or even thousands of variables, all collected from the same individuals (or animals, planets, cells, time points, whatever is the object of study). For convenience, all of those variables can be stored in a matrix, with data features (variables) in the columns, and observations in the rows. This is just like the design matrices you learned about in the discussion of least-squares fitting (Chapter 14).

To compute all pairwise covariances, we follow the same principle as for a single variable, but translate column vectors into matrices.

$$\mathbf{C} = \mathbf{X}^{\mathrm{T}}\mathbf{X} \tag{18.8}$$

Matrix X is the data matrix, and we assume that the columns are mean-centered. (For simplicity, I omitted the factor 1âˆ•(nâˆ’ 1).) Note that there is nothing magical about transposing the first matrix; if your data are stored as features-by-observations, then the correct formula is $C = XX^T$. The covariance matrix should be features-by-features.

Assuming the columns are mean-centered, the diagonal elements of C contain the variances of each variable. That is,

$$diag(\mathbf{C}) = \begin{bmatrix}\sigma_1^2 & \sigma_2^2 & \dots & \sigma_m^2\end{bmatrix} \tag{18.9}$$

It is obvious from the math that the covariance matrix is symmetric. But it is also sensible from the perspective of a correlation: The correlation between x and y is the same as the correlation between y and x (e.g., the relationship between height and weight is the same as that between weight and height).

18.6 From correlation to covariance matrices

You learned that a correlation coefficient is obtained by scaling the covariance by the norms of the two variables.

Equation 18.10 below expands this translation from an individual correlation to a correlation matrix, and then uses the power of parentheses to shuffle the equation a bit. R is the correlation matrix and Î£ is a diagonal matrix containing standard deviations of each corresponding data feature (again, not to be confused with the Î£ in the SVD).

$$R^{3} = \hat{\Sigma}^{-1} X^T X \hat{\Sigma}^{-1} \quad (18.10)$$

$$= (\hat{\Sigma}^{-T} X^T)(X\hat{\Sigma}^{-1}) \quad (18.11)$$

$$= (X\hat{\Sigma}^{\hat{}'1})^T (X\hat{\Sigma}^{-1}) \quad (18.12)$$

You might be tempted to simplify that final expression to â^¥XÎ£$^{\hat{}'1}$ â^¥^{2} , but donâ€™t do it! XÎ£$^{\hat{}'1}$ is a matrix not a vector, so taking the norm is not the correct conclusion.

[3] Iâ€™ve also omitted the factor of 1â^•(n â^’ 1) here.

Instead, the correct conclusion is that the correlation matrix is obtained by the multiplication of the data matrix with itself (just like the covariance matrix), but first normalizing each data feature by its standard deviation. To make this more clear: set X = XÎ£$^{\hat{}'1}$, then the correlation matrix is obtained as $X^T X$.

Because the data are already mean-centered, multiplying by the inverse of the standard deviations is equivalent to z-standardizing the data. Z-standardization, also called z-scoring, involves subtracting the mean and dividing by the standard deviation. It forces the data to have a mean of zero and a standard deviation of one. This means that correlation and covariance are identical metrics when the data are z-standardized.

We can also turn this equation around to compute a covariance matrix given correlation and standard deviation matrices. (Again, the normalization factor is omitted for simplicity.)

$$\mathbf{C} = \mathbf{\Sigma R \Sigma} \quad (18.13)$$

18.7 Code challenges

1.Â

The goal here is to implement Equations 18.10 and 18.13 . Create a data matrix of random numbers with 4 features and 200 observations. Then compute the correlation and covariance matrices using the equations above. Finally, confirm that you obtain the same result as Python or MATLABâ€™s built-in covariance and correlation functions.

18.8 Code solutions

1.Â

I generated random numbers matrices, so the inter-variable correlations will be lowâ€”the expected correlations are 0, but with sampling variability and random numbers, the correlations will instead be small but non-zero. Anyway, the important thing is that the matrices match the outputs of Pythonâ€™s and MATLABâ€™s built-

in functions. Be mindful that Pythonâ€™s std() function computes a biased standard deviation by default, so you need to specify that the degrees of freedom is 1.

Code blockÂ 18.1: Python

```
importÂ numpyÂ asÂ npÂ
nÂ =Â 200Â
XÂ =Â np.random.randn(n,4)Â #Â dataÂ
XÂ =Â X-np.mean(X,axis=0)Â #Â mean-centerÂ
covMÂ =Â X.T@XÂ /Â (n-1)Â #Â covarianceÂ
stdMÂ =Â np.linalg.inv(Â np.diag(Â
Â Â Â Â Â Â Â Â Â np.std(X,axis=0,ddof=1))Â )Â #Â stdevsÂ
corMÂ =Â stdM@Â X.T@XÂ @stdMÂ /Â (n-1)Â #Â RÂ
Â
#Â compareÂ oursÂ againstÂ numpyâ€™sÂ
print(np.round(covM-np.cov(X.T),3))Â
print(np.round(corM-np.corrcoef(X.T),3))
```

Code blockÂ 18.2: MATLAB

```
nÂ =Â 200;Â
XÂ =Â randn(n,4);Â %Â dataÂ
XÂ =Â X-mean(X,1);Â %Â mean-centerÂ
covMÂ =Â Xâ€™*XÂ /Â (n-1);Â %Â covarianceÂ
stdMÂ =Â inv(Â diag(std(X))Â );Â %Â stdevsÂ
corMÂ =Â stdM*Â Xâ€™*XÂ *stdMÂ /Â (n-1);Â %Â RÂ
Â
disp(covM-cov(X))Â %Â compareÂ covariancesÂ
disp(corM-corrcoef(X))Â %Â compareÂ corrs
```

ChapterÂ 19
Principal components analysis

PCA is a multivariate data analysis method widely used in statistics, machine-learning, data mining, signal processing, image processing, compression, and lots of other nerdy topics. There are entire books and courses dedicated solely to PCA. A complete discussion of this method requires venturing into statistics. Thatâ€™s a rabbit hole too deep to fall into here. But I do want to introduce you to PCA because it nicely ties together many linear algebra concepts you learned about in this book.

Thus, in this chapter I will present the linear algebra aspects of PCA while simplifying or ignoring some of the statistical aspects.

19.1 Interpretations and applications of PCA

A multivariate dataset with N features can be conceptualized as a set of vectors in an N-dimensional space. The data are stored in a matrix with features in the columns and observations in the rows, just like the design matrices you saw in Chapter 14.[1] In the data space, each feature (thus, each column in the data matrix) is a basis vector, and all the columns together span some subspace of â„ ^{N} . But is this the best set of basis vectors? The goal of PCA is to identify a basis set such that each basis vector maximizes data covariance while being orthogonal to all other basis vectors.

[1] A "feature" is a property that you measure, such as age, salary, alcohol content, pixel intensity, number of stars in a galaxy, etc.

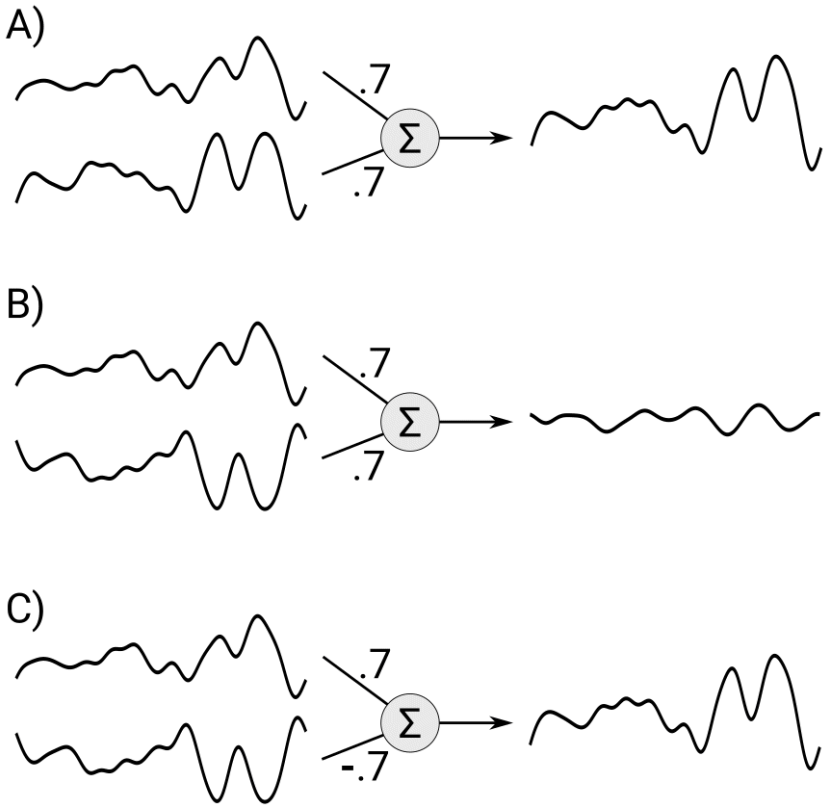
What does it mean to "maximize covariance?" It means to identify a weighted combination of data channels, such that that weighted combination has maximal variance. Figure 19.1 provides a graphical overview of three scenarios to illustrate the concept. Letâ€™s start with panel A. Two time series (at left) are clearly correlated; the PCA scalar-multiplies each data channel by .7, adds the weighted data channels together, and the result is a "component" that has higher variance than either of the individual channels.

Now letâ€™s consider panel B. The two data channels are negatively correlated. Keeping the same weighting of .7 for each channel actually reduced the variance of the result (that is, the resulting component has less variance than either individual channel). Thus, a weighting of .7 for both channels is not a good PCA solution.

Instead, PCA will negatively weight one of the channels to flip its sign (panel C). That resulting component will maximize variance.

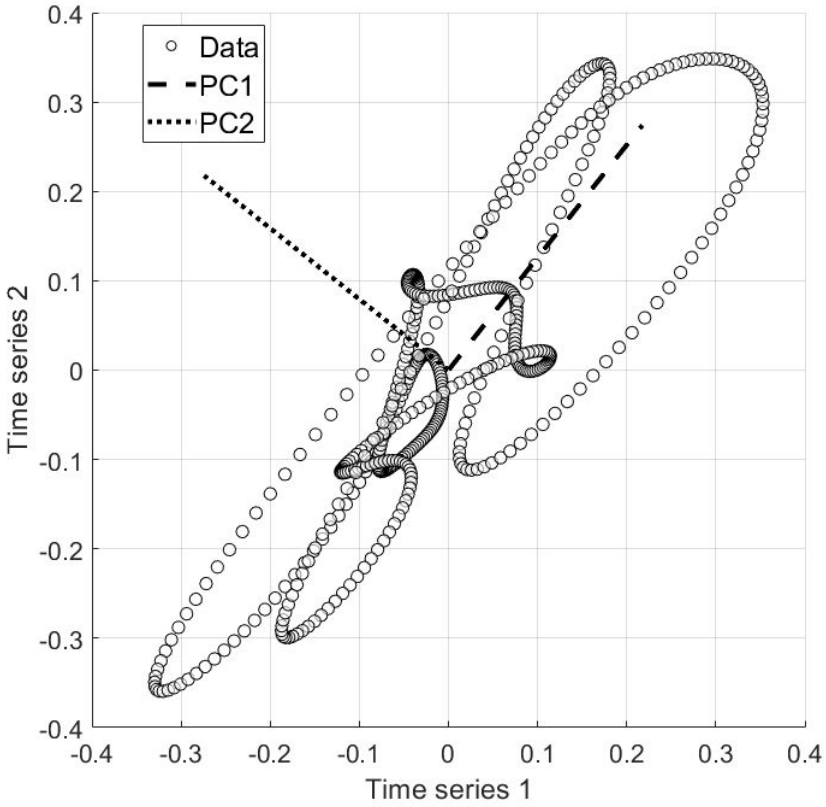
Why scalar-multiply by .7? If the goal is to maximize variance, then surely variance is maximized by using scalars of 7,000,000, or any arbitrarily large number. To avoid this triviality, PCA adds the constraint that the sum of squared weights must equal 1. In this example, $.7^2 + .7^2 = .98$. (I truncated the weights for visual simplicity, but you get the idea.)

This is the idea of PCA: You input the data, PCA finds the best set of weights, with "best" corresponding to the weights that maximize the variance of the weighted combination of data channels.



FigureÂ 19.1: Three scenarios to illustrate PCA in 2D data.

We can visualize the data shown in panel A as a scatter plot, with each axis corresponding to a channel (Figure 19.2), and each dot corresponding to a time point. The dashed line is the first principal componentâ€”the weighting of the two channels that maximizes covariance. The second principal component is the direction in the data cloud that maximizes the residual variance (that is, all the variance not explained by the first component) subject to the constraint that that component is orthogonal to the first component.



FigureÂ 19.2: The time series data in Figure 19.1 A visualized in a scatter plot, along with the principal components.

In a 2D dataset, there is only one possible second component, because there is only one possible vector in â„^{2} that is orthogonal to the first component. But for larger-dimensional datasets, there are many possible orthogonal directions.

Selecting the number of components A PCA returns M components for a M-feature dataset (because of the eigendecomposition of the MÃ—M covariance matrix). How many of those components are meaningful and should be retained?

There are quantitative and qualitative methods for determining the number of components that have large enough eigenvalues to retain for subsequent analyses. The quantitative methods involve identifying appropriate statistical thresholds, and are beyond the scope of this book.

The qualitative method involves examining the scree plot, which is a plot of the sorted eigenvalues. You learned about creating and interpreting scree plots in Chapter 16 about the SVD (e.g., Figure 16.6).

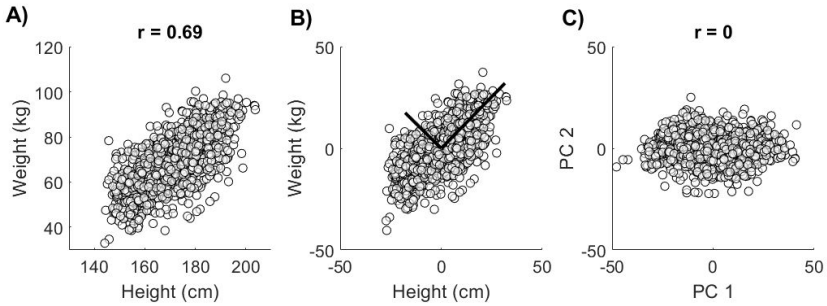
Applications A common application of PCA is data compression, where only the first k < r components are retained for subsequent analyses or data storage. The assumption is that directions in the data space associated with relatively little covariance are unimportant or noise, and therefore can be "squeezed out" of the data. The validity and appropriateness of this assumption depends on the type of data and the goals of the analysis, but thatâ€™s the idea. Notice that this application is very similar to the low-rank approximation you learned about in the SVD chapter. In fact, youâ€™ll see by the end of this chapter that PCA can be implemented via SVD.

19.2 How to perform a PCA

In this section, I will describe how a PCA is done, and then in the next section, youâ€™ll learn about the math underlying PCA. There are five steps to performing a PCA, and you will see how linear algebra is central to those steps.

1) Compute the covariance matrix of the data, as described in the previous chapter. The data matrix may be observations-by-features or features-by-observations, but the covariance matrix must be features-by-features. Each feature in the data must be mean-centered prior to computing covariance.
2) Take the eigendecomposition of that covariance matrix.
3) Sort the eigenvalues descending by magnitude, and sort the eigenvectors accordingly. There are several relevant facts that you learned in this book, including (1) the eigenvalues of a covariance matrix are all non-negative, (2) eigenvalues have no intrinsic sorting, (3) eigenvectors are paired with eigenvalues, so sorting one requires sorting the other. Eigenvalues are sometimes called latent factor scores.
4) Compute the "component scores" (called "component time series" if the data are time series). The component score is the weighted combination of all data features, where the eigenvector provides the weights. Hence, for data matrix X, component 1 is calculated as $v_1^T X$, where v_1 is the "first" eigenvector (the one with the largest associated eigenvalue).
5) Convert the eigenvalues to percent change to facilitate interpretation. This is the same procedure as converting singular values to percent variance explained.

Below is another example of a PCA on correlated data, using the height-weight data illustrated in the previous chapter. Figure 19.3 A shows the raw data, with each person represented as a dot. The Pearson correlation is r = .69.



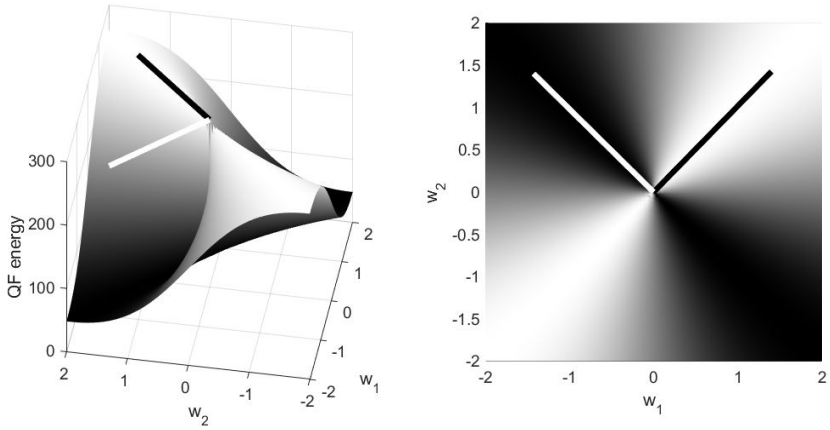
FigureÂ 19.3: Visual overview of PCA in 2D data. Note: these are simulated data. Youâ€™ll learn how to simulate these data in the code challenges.

Panel B shows the same data, mean-centered, and with the two principal components (the eigenvectors of the covariance matrix) drawn on top. Notice that the eigenvector associated with the larger eigenvalue points along the direction of the linear relationship between the two variables.

Panel C shows the same data but redrawn in PC space. Because PCs are orthogonal, the PC space is a pure rotation of the original data space. Therefore, the data projected through the PCs are decorrelated.

Because these data are in â„ ^{2} , we can visualize the normalized quadratic form surface of the data covariance matrix. Itâ€™s shown in Figure 19.4 . The two panels actually show the same data. I thought the rotated view (left panel) looked really neat but was suboptimal for visualizing the eigenvectors. The right panel is the birdâ€™s-eye-view, and more clearly shows how the eigenvectors point along the directions of maximal and minimal energy in the quadratic form surface.

As expected for a positive-definite matrix, the normalized quadratic form surface energy is above zero everywhere.



FigureÂ 19.4: Quadratic form surface of the data covariance matrix of the data shown in Figure 19.3 . The black and white lines show the two eigenvectors (scaled for visibility) corresponding to the larger (black line) and smaller (white line) eigenvalue.

19.3 The algebraic motivation for PCA

The previous section was very mechanical: I explained how to compute a PCA but didnâ€™t explain why thatâ€™s the right thing to do.

Letâ€™s get back to the problem statement: Find a direction in the data space that maximizes the data covariance. This can be translated into finding a vector v that maximizes the quadratic form of the data covariance matrix.

$$\mathbf{v}_{max} = \mathbf{v} \left\{ \mathbf{v}^{\mathrm{T}} \mathbf{C} \mathbf{v} \right\} \tag{19.1}$$

You know from Chapter 17 that this maximization problem is difficult for the non-normalized quadratic form, because the quadratic form will grow to infinity as the elements in v get larger. Fortunately, avoiding this trivial solution simply requires adding the constraint that v be a unit vector. That can be equivalently written in one of two ways:

$v_{max} = v\left\{ \mathbf{v}^{\mathrm{T}} \mathbf{C} \mathbf{v} \right\}$, s.t.âˆ¥vâˆ¥ = 1

ChapterÂ 20
Where to go from here?

20.1 The end... of the beginning!

You are reading this either because

1. You read the chapters in order, and youâ€™ve now finished the book. Congrats! Amazing! Go treat yourself to an ice cream, chocolate bar, glass of wine, pat on the back, 2-week trip to Tahiti, or whatever you like to do to reward yourself.
2. You skipped forwards because you are curious, even though you havenâ€™t finished the book. In that case, feel free to keep reading, but then get back to the important chapters!

I designed this book to contain what would be covered in 1-2 semesters of a university linear algebra course (1 or 2 depending on the particular curriculum and expectations). If you are really excited about continuing your linear algebra learning and are wondering where to go from here, then it can be challenging to know where next to turn.

I have no specific recommendations because linear algebra has such wide-ranging applications and itâ€™s not possible for me to custom-tailor such recommendations here. However, I can make two general recommendations that I hope will help you decide the right path forward for yourself.

Abstract linear algebra As you know by now, I have tried to keep this book focused on linear algebra concepts that are most relevant for applications. I like to call it "down-to-Earth" linear algebra. However, linear algebra is a huge topic in mathematics, and there are many avenues of linear algebra that are more focused on proofs, abstractions, and topics that are mathematically intriguing but less directly relevant to applications on computers.

If you are interested in following this line of study, then a quick Internet search for something like "theoretical linear algebra" or "abstract linear

algebra" will give you some places to start.

Application-specific linear algebra I intentionally avoided focusing on any particular application or any particular field, to make the book relevant for a broad audience.

But because linear algebra is so important for so many scientific and engineering disciplines, advanced studies in most STEM fields will involve linear algebra. Control engineering, computational biology, mathematical physics, multivariate signal processing, finance and mathematical economics, machine learning, deep learning... these are all disciplines that rely on linear algebra. Again, some Internet sleuthing for the keywords you are interested in should help you find a suitable book or course.

20.2 Thanks!

Thank you for choosing to learn from this book, and for trusting me with your linear algebra education. I hope you found the book useful, informative, and perhaps even a bit entertaining. Your brain is your most valuable asset, and investments in your brain always pay large dividends.

Iâ€™d be remiss if I didnâ€™t take this opportunity to invite you to check out my other educational material, which you can find linked from my website sincxpress.com.